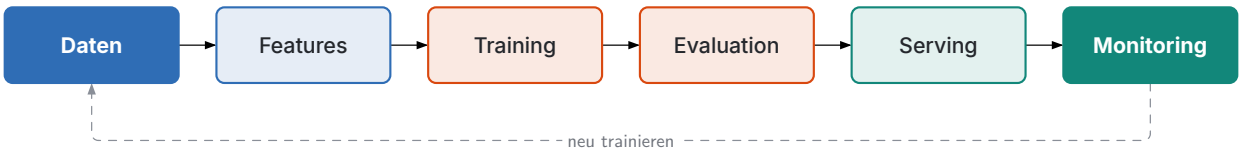
PERSÖNLICHES HANDBUCH

ML Design Patterns von Grund auf

Dreißig Patterns für Daten, Training und Betrieb, erklärt für Software Engineers ohne ML-Vorwissen, mit lauffähigen Beispielen, Fallstudien und Übungen



Für Philipp Haindl
Clean Code und Software Engineering (MDI) · WS 2027

Stand: November 2026

Inhalt

TEIL I · GRUNDLAGEN

1 Einleitung 5

1.1 Wozu dieses Handbuch . . . 5
1.2 So lesen Sie es . . . 5
1.3 Was ein ML Design Pattern ist . . . 5
1.4 Quelle und Nummerierung . . . 6
1.5 Konventionen . . . 6

2 ML-Grundlagen für Software Engineers 8

2.1 Programm und Modell . . . 8
2.2 Daten: Beispiele, Features und Label . . . 8
2.3 Ein Modell ist eine gelernte Funktion . . . 9
2.4 Loss und Gradientenabstieg . . . 10
2.5 Klassifikation: Wahrscheinlichkeiten statt Zahlen . . . 12
2.6 Entscheidungsbäume, Random Forest und Gradient Boosting . . . 13
2.7 Neuronale Netze und Embeddings . . . 15
2.8 Generalisierung: Overfitting, Splits und Datenlecks . . . 17
2.9 Metriken: Wie gut ist gut? . . . 19
2.10 Der ML-Lebenszyklus . . . 21
2.11 Begriffe für den Betrieb . . . 21
2.12 Übersetzung in die Welt der Softwareentwicklung . . . 23

3 Neuronale Netze und Repräsentationen verstehen 24

3.1 Ein Neuron: gewichtete Summe und Aktivierung . . . 24
3.2 Mehrschichtige Netze . . . 25
3.3 Softmax und Cross-Entropy: aus Zahlen werden Wahrscheinlichkeiten . . . 26
3.4 Lernen mit Backpropagation . . . 26
3.5 Ein Netz von Grund auf: die Spirale . . . 28
3.6 Mini-Batches, Epochen und Lernrate . . . 29
3.7 Regularisierung: Overfitting dämpfen . . . 31
3.8 Embeddings sind Nachschlagetabellen . . . 32
3.9 word2vec: Bedeutung aus Nachbarschaft . . . 33
3.10 Vortrainierte Modelle und Transfer Learning . . . 35
3.11 Checkpoints: was zum Zustand gehört . . . 37
3.12 Hardware und Verteilung: warum Batches, warum GPUs . . . 37
3.13 Ausblick: der Transformer . . . 39
3.14 Das Wichtigste in Kürze . . . 40
3.15 Weiterführende Literatur . . . 41

4 Landkarte der 30 Patterns 42

4.1 Die Patterns am Lebenszyklus . . . 42
4.2 Vom Symptom zum Pattern . . . 43
4.3 Abhängigkeiten zwischen Patterns . . . 44
4.4 Leicht zu verwechseln . . . 45

TEIL II · DATEN UND PROBLEM

5 Data Representation 46

5.1 Einfache Repräsentationen: aus Rohwerten werden Zahlen . . . 46
5.2 Pattern 1: Hashed Feature . . . 47
5.3 Pattern 2: Embeddings . . . 50
5.4 Pattern 3: Feature Cross . . . 53
5.5 Pattern 4: Multimodal Input . . . 55

6 Problem Representation 57

6.1 Pattern 5: Reframing . . . 57
6.2 Pattern 6: Multilabel . . . 59
6.3 Pattern 7: Ensembles . . . 61
6.4 Pattern 8: Cascade . . . 63
6.5 Pattern 9: Neutral Class . . . 66
6.6 Pattern 10: Rebalancing . . . 68

TEIL III · TRAINING UND BETRIEB

7 Model Training 72

7.1 Vorspann: Die Trainingsschleife . . . 72
7.2 Pattern 11: Useful Overfitting . . . 73
7.3 Pattern 12: Checkpoints . . . 75
7.4 Pattern 13: Transfer Learning . . . 77
7.5 Pattern 14: Distribution Strategy . . . 79
7.6 Pattern 15: Hyperparameter Tuning . . . 81

8 Resilient Serving 84

8.1 Pattern 16: Stateless Serving Function . . . 84
8.2 Pattern 17: Batch Serving . . . 86
8.3 Pattern 18: Continued Model Evaluation . . . 88
8.4 Pattern 19: Two-Phase Predictions . . . 90
8.5 Pattern 20: Keyed Predictions . . . 92

TEIL IV · REPRODUZIERBARKEIT UND VERANTWORTUNG

9 Reproducibility 95

9.1 Warum ML-Systeme schwer reproduzierbar sind . . . 95
9.2 Pattern 21: Transform . . . 96
9.3 Pattern 22: Repeatable Splitting . . . 98

9.4 Pattern 23: Bridged Schema . . . 100
9.5 Pattern 24: Windowed Inference . . . 102
9.6 Pattern 25: Workflow Pipeline . . . 104
9.7 Pattern 26: Feature Store . . . 107
9.8 Pattern 27: Model Versioning . . . 109

10 Responsible AI 112

10.1 Pattern 28: Heuristic Benchmark . . . 112
10.2 Pattern 29: Explainable Predictions . . . 114
10.3 Pattern 30: Fairness Lens . . . 117

TEIL V · ML-SYSTEME BAUEN

11 ML-Systeme in Produktion 120

11.1 Vom Notebook zum System . . . 120
11.2 Hidden Technical Debt . . . 120
11.3 MLOps und seine Reifegrade . . . 123
11.4 Experiment Tracking und Model Registry . 124
11.5 Datenvalidierung . . . 126
11.6 Testen von ML-Systemen . . . 128
11.7 Monitoring . . . 129
11.8 Rollout-Strategien . . . 131
11.9 Rollen im Team . . . 133
11.10 Die 30 Patterns im Betrieb . . . 133
11.11 Das Wichtigste in Kürze . . . 134
11.12 Weiterführende Literatur . . . 135

TEIL VI · FALLSTUDIEN

12 Fallstudie A: Kundenabwanderung bei einem Streamingdienst 136

12.1 Ausgangslage und Daten . . . 136
12.2 Schritt 1: Zuerst der Maßstab (Heuristic Benchmark) . . . 137
12.3 Schritt 2: Das naive Notebook . . . 138
12.4 Schritt 3: Der Split nach Kunden (Repeatable Splitting) . . . 138
12.5 Schritt 4: Seltene Ereignisse (Rebalancing) . . . 140
12.6 Schritt 5: Vorverarbeitung gehört ins Modell (Transform) . . . 141
12.7 Schritt 6: Warum steht dieser Kunde auf der Liste? (Explainable Predictions) . . . 143
12.8 Schritt 7: Für wen funktioniert das Modell? (Fairness Lens) . . . 145
12.9 Schritt 8: Das Modell im Betrieb (Continued Model Evaluation) . . . 146
12.10 Das fertige System . . . 148
12.11 Das Wichtigste in Kürze . . . 149

13 Fallstudie B: Vorausschauende Wartung von Industriepumpen 151

13.1 Die Aufgabe und ihre Daten . . . 151
13.2 Pattern 22: Repeatable Splitting . . . 153
13.3 Pattern 24: Windowed Inference . . . 155
13.4 Pattern 5: Reframing . . . 157
13.5 Pattern 19: Two-Phase Predictions . . . 158
13.6 Pattern 16 und 17: Einzeln oder im Paket . 160
13.7 Pattern 20: Keyed Predictions . . . 161
13.8 Pattern 12: Checkpoints . . . 162
13.9 Das Gesamtsystem . . . 164
13.10 Das Wichtigste in Kürze . . . 165
13.11 Weiterführende Literatur . . . 165

14 Fallstudie C: Produktsuche und Kaufwahrscheinlichkeit im Onlineshop 166

14.1 Ausgangslage: ein Shop, ein Score, drei Probleme . . . 166
14.2 Pattern 1: Hashed Feature . . . 167
14.3 Pattern 2: Embeddings . . . 168
14.4 Pattern 3: Feature Cross . . . 171
14.5 Pattern 4: Multimodal Input . . . 172
14.6 Pattern 8: Cascade . . . 173
14.7 Pattern 26: Feature Store . . . 174
14.8 Pattern 16 und 27: Dienst und Modellversionen . . . 176
14.9 Das Gesamtbild: Architektur und Wirkung . 178
14.10 Das Wichtigste in Kürze . . . 179

TEIL VII · UNTERRICHTEN UND ÜBEN

15 Die Patterns unterrichten 181

15.1 Der rote Faden der Lehrveranstaltung . . . 181
15.2 ML-Grundlagen in 20 Minuten . . . 181
15.3 Typische Missverständnisse je Kategorie . 183
15.4 Prüfungs- und Diskussionsfragen . . . 190
15.5 Verbindung zu den vier Praxisprojekten . 192
15.6 Den Trainer-Report im Coaching nutzen . 193

16 Übungen mit Lösungen 195

16.1 So arbeiten Sie mit diesem Kapitel . . . 195
16.2 Data Representation (Patterns 1 bis 4) . . . 195
16.3 Problem Representation (Patterns 5 bis 10) . . . 197
16.4 Model Training (Patterns 11 bis 15) . . . 198
16.5 Resilient Serving (Patterns 16 bis 20) . . . 199
16.6 Reproducibility (Patterns 21 bis 27) . . . 200
16.7 Responsible AI (Patterns 28 bis 30) . . . 202
16.8 Szenarien über mehrere Kategorien . . . 203
16.9 Lösungen . . . 204
16.10 Das Wichtigste in Kürze . . . 216
16.11 Weiterführende Literatur . . . 217

ANHANG

A Spickzettel 218
B Glossar 221
Literaturverzeichnis 226

1 Einleitung

1.1 Wozu dieses Handbuch

Wer Software baut, kennt Entwurfsmuster: Strategy, Observer, Factory. Sie benennen eine wiederkehrende Aufgabe, eine bewährte Lösung und den Preis, den man dafür zahlt. Seit einigen Jahren gibt es einen vergleichbaren Katalog für Systeme, die aus Daten lernen. Valliappa Lakshmanan, Sara Robinson und Michael Munn haben 2020 dreißig solcher Muster beschrieben [1]. Dieses Handbuch erklärt sie für Leserinnen und Leser, die Software entwerfen können, aber noch nie ein Modell trainiert haben.

Das Ziel ist zweifach. Erstens sollen Sie jedes der dreißig Patterns so weit verstehen, dass Sie in einem Projekt erkennen, wann es passt und wann nicht. Zweitens sollen Sie es Studierenden erklären können, die Data Science im Master studieren, aber nicht unbedingt eine Softwareausbildung mitbringen. Deshalb enthält jedes Pattern neben der Mechanik auch einen Unterrichtsvorschlag mit einer Verständnisfrage.

Das Buch von Lakshmanan et al. ist dicht und setzt vieles voraus: TensorFlow, Keras, BigQuery ML und ein Grundverständnis von Training und Evaluation. Dieses Handbuch schließt die Lücke an zwei Stellen: Kapitel 2 und 3 bauen das nötige Wissen in der Sprache eines Software Engineers auf, und die Pattern-Kapitel zeigen jede Idee an einem kleinen Beispiel mit Zahlen, das Sie selbst ausführen können.

1.2 So lesen Sie es

Das Handbuch ist in sieben Teile gegliedert (Abbildung 1). Teil I legt das Fundament: diese Einleitung, die ML-Grundlagen in Kapitel 2, die Funktionsweise neuronaler Netze in Kapitel 3 und in Kapitel 4 eine Landkarte, die alle dreißig Patterns an einem gemeinsamen Lebenszyklus verortet. Die Teile II bis IV behandeln die sechs Kategorien des Buchs, jeweils ein Kapitel pro Kategorie. Teil V zeigt, wie die Patterns in einem laufenden System zusammenwirken. Teil VI führt drei Fallstudien durch, in denen ein Team ein naives Notebook Schritt für Schritt verbessert. Teil VII sammelt, was beim Unterrichten hilft, und enthält Übungen mit Lösungen.

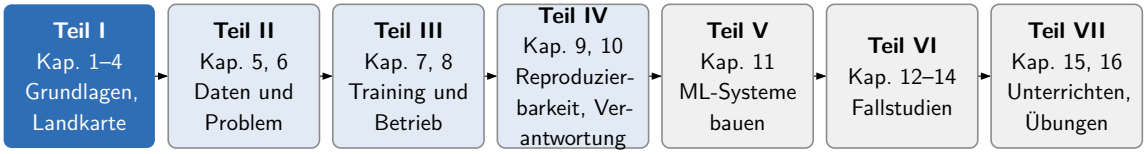


Abbildung 1: Leseweg durch das Handbuch. Die Pattern-Kapitel in den Teilen II bis IV lassen sich in beliebiger Reihenfolge lesen; Kapitel 2 und 3 kann überspringen, wer schon Modelle trainiert hat. Die Abhängigkeiten zwischen den Patterns zeigt Kapitel 4.

Wenn Sie noch nie ein Modell trainiert haben, lesen Sie Kapitel 2 von vorn nach hinten und führen die Beispiele aus; die Abschnitte bauen aufeinander auf. Wer schon Modelle trainiert hat, überfliegt nur den Lebenszyklus und die Begriffe für den Betrieb am Ende von Kapitel 2, weil die Patterns diese Vokabeln voraussetzen. Kapitel 4 lohnt sich in beiden Fällen als Übersicht, zu der man zurückkehrt, wenn man ein Symptom aus einem Projekt einem Pattern zuordnen will. Die Pattern-Kapitel sind unabhängig voneinander lesbar; Querverweise stehen im Text und gesammelt in Kapitel 4. Im Anhang finden Sie einen Spickzettel und ein Glossar.

1.3 Was ein ML Design Pattern ist

Ein Entwurfsmuster im Sinne von Gamma et al. [2] beschreibt eine Anordnung von Klassen und Objekten, die ein wiederkehrendes Entwurfsproblem löst. Das Muster lässt sich im Code sehen: Man findet die Schnittstelle, die Delegation, die Vererbungsbeziehung. Ein ML Design Pattern hat dieselbe Dreiteilung

aus Problem, Lösung und Trade-off, aber es wirkt auf einer anderen Ebene. Es betrifft die Daten, die ein Modell sieht, die Art, wie es trainiert wird, und die Umstände, unter denen es Vorhersagen liefert.

Tabelle 1: GoF-Patterns und ML Design Patterns im Vergleich.

	GoF-Pattern	ML Design Pattern
Gegenstand	Klassen und Objekte im Programm	Daten, Training und Betrieb eines Modells
Lösung	Anordnung von Code-Bausteinen, im Quelltext ablesbar	Ein Verfahren: Transformation, Trainingsschritt, Betriebsentscheidung
Prüfbarkeit	Strukturell und durch Tests, deterministisch	Statistisch: Wirkung zeigt sich in Metriken auf Daten
Problemursache	Änderungen und Erweiterungen am Programm	Eigenschaften von Daten und Lernverfahren (Ungleichgewicht, Drift)

Der zweite und dritte Punkt sind die wichtigsten. Bei Strategy sehen Sie am Code, ob das Pattern angewandt wurde. Bei Rebalancing sehen Sie im Code eine Zeile, die Beispiele gewichtet oder dupliziert; ob das Pattern *wirkt*, erfahren Sie erst, wenn Sie die Metriken des Modells auf Daten ansehen, die es nicht trainiert haben. Ein ML-Pattern ist deshalb immer an eine Messung gekoppelt, und die Pattern-Kapitel nennen jeweils, was man messen muss.

Ein weiterer Unterschied ist die Bandbreite. Manche der dreißig Patterns sind klassisches Software Engineering mit ML-Bezug: Stateless Serving Function ist ein zustandsloser Dienst, Workflow Pipeline ist Orchestrierung, Model Versioning ist Versionierung von Artefakten. Andere haben kein Gegenstück in der Softwarewelt, etwa Neutral Class oder Useful Overfitting, weil sie auf Eigenheiten des Lernens zurückgehen. Das Handbuch nennt bei jedem Pattern das bekannte SE-Prinzip, wenn es eines gibt.

Zum Status: Die dreißig Patterns sind eine Sammlung bewährter Praxis, kein Standard und nicht erschöpfend. Manche überlappen, einige haben inzwischen eigene Werkzeuge bekommen. Als gemeinsames Vokabular sind sie trotzdem nützlich; als Checkliste sollte man sie nicht abarbeiten.

1.4 Quelle und Nummerierung

Alle Patterns stammen aus *Machine Learning Design Patterns* von Lakshmanan, Robinson und Munn [1]. Das Buch ordnet sie in sechs Kategorien, denen je ein Kapitel gewidmet ist, und nummeriert die Patterns fortlaufend von 1 bis 30. Das Handbuch übernimmt diese Nummern: Pattern 21 ist im Buch wie hier Transform. Die Kapitel 1 und 8 des Buchs enthalten keine eigenen Patterns; Kapitel 1 führt in typische Herausforderungen ein, Kapitel 8 zeigt, wie Patterns zusammenspielen. Letzteres leistet in diesem Handbuch Kapitel 4.

Tabelle 2: Die sechs Kategorien des Buchs und ihre Lage im Handbuch (4 + 6 + 5 + 5 + 7 + 3 = 30 Patterns).

Kategorie im Buch	Patterns	Handbuch	Leitfrage
Data Representation	1–4	Kapitel 5	Wie werden Rohwerte zu Zahlen?
Problem Representation	5–10	Kapitel 6	Wie wird die Aufgabe lösbar formuliert?
Model Training	11–15	Kapitel 7	Wie trainiert man zuverlässig?
Resilient Serving	16–20	Kapitel 8	Wie liefert man robust aus?
Reproducibility	21–27	Kapitel 9	Wie bleibt das Ergebnis wiederholbar?
Responsible AI	28–30	Kapitel 10	Ist das Modell erklärbar und fair?

Die Namen folgen dem Inhaltsverzeichnis des Buchs. Eine Abweichung ist bekannt: Pattern 22 heißt dort *Repeatable Splitting*, im Begleit-Repository des Buchs dagegen *Repeatable Sampling*; Nummer und Inhalt sind dieselben. Zitiert wird das Buch nach Kapitel und Pattern-Nummer, nicht nach Seitenzahl, weil sich diese zwischen den Ausgaben unterscheiden. Primärquellen sind in den Pattern-Kapiteln genannt, soweit sie sich eindeutig zuordnen ließen.

1.5 Konventionen

Das Buch arbeitet überwiegend mit TensorFlow, Keras und BigQuery ML. Dieses Handbuch zeigt die Prinzipien stattdessen mit *scikit-learn*, *pandas* und *numpy*, weil sie sich ohne Cloud-Konto auf jedem

Laptop ausführen lassen. Wo sich ein Pattern nur mit einem tiefen neuronalen Netz zeigen lässt, steht das Prinzip framework-neutral im Text, mit Verweis auf die Umsetzung im Buch.

Jedes Code-Beispiel liegt als lauffähige Datei im Ordner `code/<Kapitel>/` und lässt sich mit *uv* ohne vorbereitete Umgebung starten:

Terminal

```
$ cd handbuch-ml-design-patterns/code/02
$ uv run --with scikit-learn --with pandas --with numpy python 02_lineare_regression.py
```

Gezeigte Ausgaben sind die echte Ausgabe dieser Dateien. Alle Zufallsgeneratoren haben einen festen Seed; mit denselben Bibliotheksversionen erhalten Sie dieselben Zahlen, bei anderen können Nachkommastellen abweichen. Plots und Diagramme verwenden Werte aus diesen Läufen und sind in Typst selbst gezeichnet. Fachbegriffe bleiben englisch, wenn sie in der Praxis so verwendet werden, und sind beim ersten Auftreten kurz erklärt. Unsichere Aussagen sind im Text als offen markiert, statt sie zu glätten.

2 ML-Grundlagen für Software Engineers

Dieses Kapitel baut das Wissen auf, das Sie für die dreißig Patterns brauchen. Es setzt nur voraus, dass Sie programmieren können und Schulmathematik nicht fürchten: Geraden, Ableitungen als Steigungen, Wahrscheinlichkeiten zwischen 0 und 1. Die Leitidee passt in einen Satz: *Ein Modell ist eine Funktion, deren Konstanten nicht jemand hinschreibt, sondern die aus Beispielen bestimmt werden.* Alles andere in diesem Kapitel ist die Handwerkskunst drumherum, damit man dem Ergebnis trauen kann, und die Technik, damit man es betreiben kann.

Die Beispiele sind klein, laufen in Sekunden und liegen unter `code/02/`. Sie verwenden zwei erfundene, aber realistisch gebaute Datensätze: Mietwohnungen (Fläche, Zimmer, Baujahr, Miete) und Maschinen (Temperatur, Schwingung, Ausfall). Alle Zahlen in den Plots stammen aus diesen Läufen.

2.1 Programm und Modell

Ein klassisches Programm entsteht, indem ein Mensch Regeln aufschreibt: Wenn die Fläche größer als 50 Quadratmeter ist, dann so viel Miete. Aus Regeln und Eingaben berechnet das Programm Antworten. Beim maschinellen Lernen (Machine Learning, ML) dreht sich die Richtung um. Man gibt dem Verfahren Eingaben *und* die dazugehörigen richtigen Antworten, und das Verfahren findet die Regeln selbst (Abbildung 2). Die gefundenen Regeln sind das *Modell*.

Zwei Phasen sind zu unterscheiden, und das Handbuch kehrt immer wieder zu ihnen zurück. In der *Trainingsphase* entsteht das Modell aus Beispielen; sie ist rechenintensiv, läuft selten und darf Stunden dauern. In der *Vorhersagephase* (*Inferenz*) wendet man das fertige Modell auf neue Eingaben an; sie muss schnell und zuverlässig sein, weil dort Nutzer oder andere Systeme warten. Viele Patterns lösen Probleme, die entstehen, weil beide Phasen verschiedene Anforderungen haben und doch dieselben Daten gleich verarbeiten müssen.

> **WANN ML?**
> Maschinelles Lernen lohnt sich, wenn die Regeln zu zahlreich oder zu unscharf sind, um sie aufzuschreiben, aber viele Beispiele vorliegen. Eine Mietpreisschätzung aus Fläche, Lage und Zustand ist ein solcher Fall; die Berechnung einer Mehrwertsteuer ist keiner, weil die Regel bekannt und exakt ist.

2.2 Daten: Beispiele, Features und Label

Ein *Beispiel* (englisch *example* oder *sample*) ist ein einzelner Fall, etwa eine Wohnung. Es wird durch seine *Features* beschrieben, das sind die Eingabegrößen, die dem Modell zur Verfügung stehen: Fläche, Zimmerzahl, Baujahr. Beim *überwachten Lernen* (*supervised learning*) kommt zu jedem Beispiel das *Label* hinzu, die Größe, die das Modell vorhersagen soll: die Miete. Wer ein Modell aus Beispielen trainieren will,

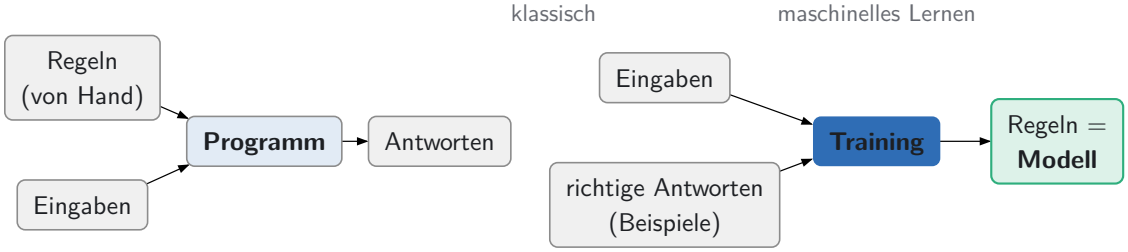


Abbildung 2: Klassisches Programm (links) und maschinelles Lernen (rechts). Beim Lernen entstehen die Regeln aus Beispielen; erst danach läuft das Modell wie jedes andere Stück Software.

braucht also eine Tabelle, in der jede Zeile ein Beispiel ist, die meisten Spalten Features sind und eine Spalte das Label.

Zwei Aufgabentypen kommen ständig vor. Ist das Label eine Zahl, spricht man von *Regression* (Miete in Euro). Ist es eine Kategorie, spricht man von *Klassifikation* (Maschine fällt aus oder nicht). Daneben gibt es das *unüberwachte Lernen* ohne Label, zum Beispiel Gruppen ähnlicher Kunden zu finden; es spielt für die Patterns eine Nebenrolle.

Die Tabelle entsteht in `01_daten.py`; `X` enthält die Features, `y` das Label.

Terminal

```
$ python 01_daten.py
   flaeche_m2  zimmer  baujahr  miete_eur
0        99.7       4     1994       1089
1        69.5       2     1956        739
2       107.3       4     1998       1163
3        92.8       3     2013       1014
4        38.5       2     2011        652

Form: (200, 4) (Zeilen = Beispiele, Spalten = Merkmale + Label)
Features X: ['flaeche_m2', 'zimmer', 'baujahr'] (200, 3)
Label y: miete_eur (200,)
```

Die Konvention `X` für die Feature-Matrix und `y` für den Label-Vektor begegnet Ihnen in fast jedem ML-Code: `X` hat eine Zeile pro Beispiel und eine Spalte pro Feature, `y` einen Eintrag pro Beispiel. Die Qualität dieser Tabelle bestimmt die Qualität des Modells oft stärker als die Wahl des Verfahrens, und viele Patterns (Hashed Feature, Embeddings, Feature Cross, Transform) betreffen den Schritt, in dem rohe Daten zu solchen Spalten werden.

2.3 Ein Modell ist eine gelernte Funktion

Das einfachste Modell, das man sich vorstellen kann, ist eine Gerade. Für die Miete einer Wohnung aus ihrer Fläche x lautet sie

$$\hat{y} = w \cdot x + b$$

Das Dach über dem y bedeutet „vorhergesagt". w und b sind die *Parameter* des Modells: die Steigung (Euro pro Quadratmeter) und der Achsenabschnitt (Grundmiete). Es sind genau die Konstanten, die ein Entwickler sonst von Hand in den Code schriebe. *Training* heißt, sie so zu bestimmen, dass die Gerade möglichst gut durch die vorhandenen Beispiele läuft. Danach ist das Modell nichts als diese Funktion mit festen Zahlen.

```python
# Ausschnitt aus 02_lineare_regression.py
X, y = df[["flaeche_m2"]], df["miete_eur"]

modell = LinearRegression().fit(X, y)   # Training: findet w und b
print(f"Steigung w = {modell.coef_[0]:.2f} Euro pro m²")
print(f"Achsenabschnitt b = {modell.intercept_:.2f} Euro")

neu = pd.DataFrame({"flaeche_m2": [75.0]})
print(f"Vorhersage für 75 m²: {modell.predict(neu)[0]:.0f} Euro")
```

Terminal

```
$ python 02_lineare_regression.py
Steigung w = 7.95 Euro pro m²
Achsenabschnitt b = 244.28 Euro
Vorhersage für 75 m²: 840 Euro

Drei Features: {'flaeche_m2': 8.21, 'zimmer': -5.18, 'baujahr': 1.68} b = -3100.2
```

Die Gerade trifft keinen Punkt exakt, und das ist in Ordnung: Das Modell soll das Muster erfassen, nicht jeden Einzelfall. Mit mehreren Features verallgemeinert sich die Formel zu $\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$, jedes Feature bekommt sein eigenes Gewicht. Im Beispiel kommen so Fläche, Zimmerzahl und Baujahr

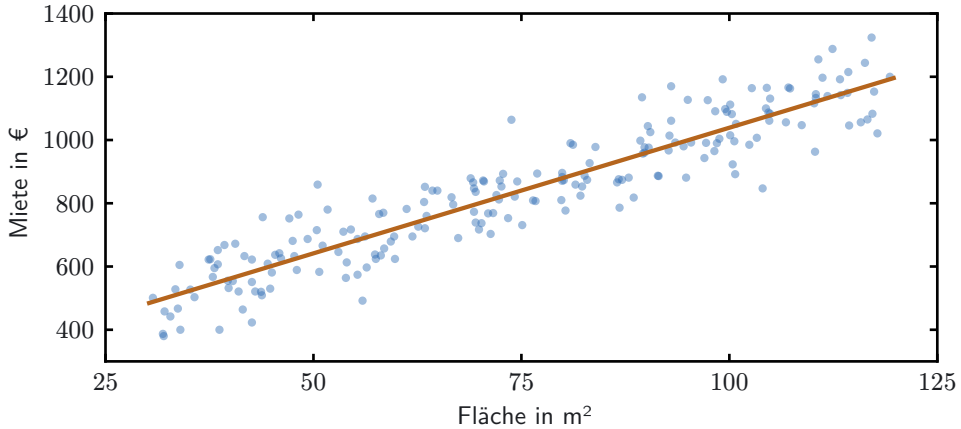


Abbildung 3: 200 Wohnungen (blaue Punkte) und die gelernte Gerade (orange). Die Parameter w und b stammen aus dem Lauf oben; die Streuung um die Gerade bleibt, weil Fläche allein die Miete nicht vollständig erklärt.

zusammen (zweite Ausgabezeile oben). Die Daten wurden mit 8,2 € pro Quadratmeter und 1,5 € pro Baujahr erzeugt, und das Training findet für Fläche und Baujahr Werte in der Nähe. Bei der Zimmerzahl liefert es dagegen ein negatives Gewicht, obwohl sie bei der Erzeugung keine eigene Rolle spielte: Sie hängt eng an der Fläche und liefert kaum Neues, sodass das Modell ihr einen kleinen, zufälligen Wert zuteilt. Gewichte sollte man deshalb nicht ungeprüft als „Wichtigkeit" eines Features lesen. Hier kennen wir die Wahrheit, weil wir die Daten selbst gebaut haben; bei echten Daten kennt man sie nicht, und genau deshalb braucht man die Methoden zur Überprüfung weiter unten.

Zwei Dinge sollten Sie mitnehmen. Erstens ist ein trainiertes Modell ein gewöhnliches Artefakt: eine Funktion und eine Handvoll Zahlen, die man speichern, versionieren und ausliefern kann. Große neuronale Netze funktionieren nach demselben Prinzip, nur mit Millionen oder Milliarden Parametern statt zweien. Zweitens bestimmt die *Form* der Funktion, was das Modell ausdrücken kann: Eine Gerade bildet keine Kurve ab, so viele Daten man ihr auch gibt. Die Wahl der Funktionsform ist eine Entwurfsentscheidung, ähnlich der Wahl einer Datenstruktur.

2.4 Loss und Gradientenabstieg

Wie findet ein Verfahren gute Werte für w und b? Es braucht zuerst ein Maß dafür, wie schlecht ein Parameterpaar ist. Dieses Maß heißt *Loss* (oder Kostenfunktion, *loss function*). Für Regression ist der verbreitetste der mittlere quadratische Fehler (*mean squared error*, MSE): Man bildet für jedes Beispiel die Differenz zwischen Vorhersage und Label, quadriert sie und mittelt:

$$\mathrm{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^{n} (w x_i + b - y_i)^2$$

Das Quadrat sorgt dafür, dass sich positive und negative Abweichungen nicht aufheben und dass große Fehler überproportional zählen. Der Loss ist eine Funktion der Parameter, nicht der Daten: Die Daten sind fest, und je nachdem, wie man w und b wählt, ergibt sich ein anderer Wert. Training ist damit ein Optimierungsproblem: Finde die Parameter mit dem kleinsten Loss.

Bei einer Geraden gibt es dafür eine geschlossene Formel. Bei den meisten anderen Modellen nicht, und dort kommt das Verfahren ins Spiel, das praktisch alles Training antreibt: der *Gradientenabstieg* (*gradient descent*). Die Idee lässt sich mit einem Parameter zeigen. Stellen Sie sich den Loss als Kurve über dem Parameter w vor, ein Tal. Man steht irgendwo an einem Hang, sieht nur die Steigung am eigenen Standort und macht einen Schritt bergab. Die Steigung ist die Ableitung des Loss nach w, der *Gradient*. Der Schritt hat die Länge Gradient mal *Lernrate*:

$$w \leftarrow w - \eta \cdot \frac{\partial\, \mathrm{MSE}}{\partial w}$$

η (eta) ist die Lernrate, eine Zahl, die der Entwickler vor dem Training festlegt. Solche Einstellungen, die nicht gelernt, sondern gewählt werden, heißen *Hyperparameter*. Man wiederholt den Schritt, bis sich der Loss kaum noch ändert.

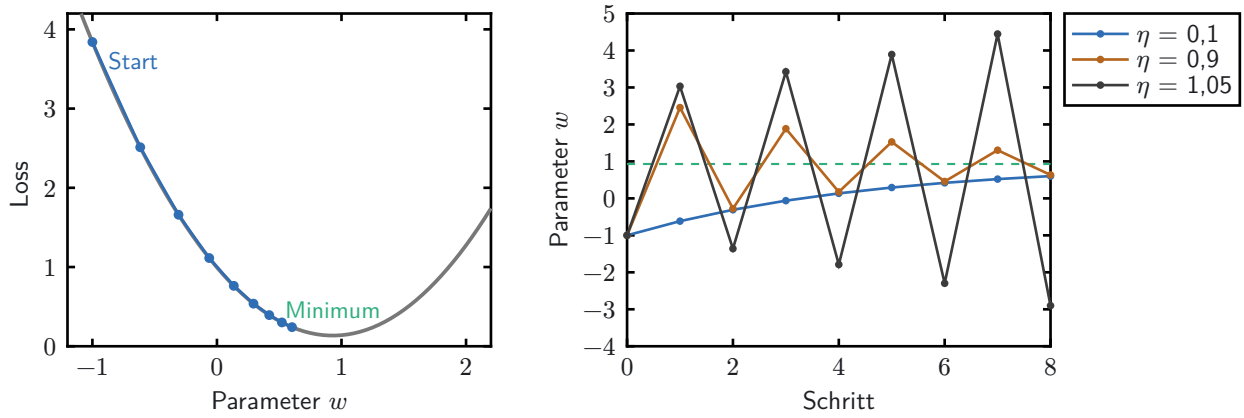


Abbildung 4: Links: Der Loss als Kurve über dem Parameter w und die ersten acht Gradientenschritte bei η = 0,1. Rechts: Der Parameter über die Schritte für drei Lernraten. Bei 0,9 pendelt w um das Minimum, bei 1,05 schaukelt es sich auf und läuft davon.

Für das Experiment sind die Daten standardisiert (Mittelwert 0, Streuung 1); dann ist der beste Achsenabschnitt 0, und es bleibt w als einzige Stellschraube. Abbildung 4 zeigt links den Weg bei Lernrate 0,1 und rechts, was bei zu großer Lernrate passiert.

Die Lernrate ist die wichtigste Stellschraube. Ist sie klein, kommt man sicher, aber langsam ans Ziel: Bei 0,1 liegt w nach acht Schritten bei 0,60, das Optimum bei 0,93. Ist sie zu groß, überspringt der Schritt das Tal, landet auf der anderen Seite höher und schaukelt sich auf, bis der Loss gegen unendlich geht. Bei 0,9 springt w noch hin und her, nähert sich aber dem Minimum; bei 1,05 wächst der Ausschlag, und der Loss steht nach acht Schritten bei 14,7 statt bei 3,8 am Start. Für dieses Beispiel lässt sich ausrechnen, dass es oberhalb einer Lernrate von etwa 1 passiert; bei echten Modellen findet man die Grenze durch Ausprobieren, was eines der Themen von Pattern 15 (Hyperparameter Tuning) ist.

Terminal

```
$ python 03_gradientenabstieg.py
Analytisches Optimum: w = 0.930, Loss = 0.135
w nach Schritt 0, 1, ..., 8 bei verschiedenen Lernraten:
Lernrate 0.1 : w =  -1.00  -0.62  -0.31  -0.06   0.14   0.29   0.42   0.52   0.60
               loss nach Schritt 8: 0.241
Lernrate 0.9 : w =  -1.00   2.46  -0.28   1.88   0.17   1.53   0.46   1.30   0.63
               loss nach Schritt 8: 0.222
Lernrate 1.05: w =  -1.00   3.03  -1.36   3.43  -1.79   3.89  -2.30   4.45  -2.90
               loss nach Schritt 8: 14.738

200 Beispiele, Batch 32 -> 7 Schritte pro Epoche
Epoche 1: w = 0.502, b = 0.003, loss = 0.317
Epoche 2: w = 0.799, b = -0.016, loss = 0.152
Epoche 3: w = 0.964, b = -0.001, loss = 0.136
Epoche 4: w = 0.939, b = 0.008, loss = 0.135
Epoche 5: w = 0.923, b = 0.041, loss = 0.137
```

Bei realen Datenmengen berechnet man den Gradienten nicht über alle Beispiele auf einmal, sondern über eine zufällige Teilmenge, den *Batch* (oder *Mini-Batch*). Das ist billiger und rauscht ein wenig, was in der Praxis eher hilft als schadet. Ein Durchlauf, in dem jedes Beispiel genau einmal in einem Batch vorkam, heißt *Epoche*. Mit 200 Beispielen und Batchgröße 32 besteht eine Epoche aus sieben Schritten (sechs volle Batches und einer mit den restlichen acht Beispielen). Der letzte Teil der Ausgabe oben zeigt den Loss über fünf Epochen: Er fällt von 0,317 auf rund 0,135 und schwankt danach leicht, weil jeder Schritt nur einen Batch sieht.

ACHTUNG
Bei einer Geraden führt der Gradientenabstieg mit passender Lernrate zum globalen Minimum, weil der Loss eine einzige Schüssel bildet. Bei neuronalen Netzen ist die Loss-Landschaft zerklüftet, und das Verfahren findet nur irgendein gutes Tal. Zwei Läufe mit verschiedenem Zufallsstart liefern deshalb verschiedene Modelle. Dass einzelne Modelle voneinander abweichen, ist ein Grund für Pattern 7 (Ensembles).

2.5 Klassifikation: Wahrscheinlichkeiten statt Zahlen

Bei der Klassifikation ist das Label eine Kategorie. Der naheliegendste Ansatz ist, eine lineare Funktion zu verwenden und das Ergebnis zu einer Wahrscheinlichkeit umzuformen. Das Modell heißt trotz seines Namens *logistische Regression* und ist ein Klassifikator. Es berechnet zuerst eine gewichtete Summe der Features, den *Logit* $z = w_1 x_1 + w_2 x_2 + b$, und schiebt ihn durch die *Sigmoid-Funktion*

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

die jede Zahl auf das Intervall zwischen 0 und 1 abbildet (Abbildung 5, links). Das Ergebnis lesen wir als Wahrscheinlichkeit, dass das Beispiel zur positiven Klasse gehört, im Beispiel: dass die Maschine ausfällt. Ein *Schwellwert* (*threshold*) macht daraus eine Entscheidung: Ab 0,5 wird die Maschine als gefährdet gemeldet. Dieser Schwellwert ist keine Eigenschaft des Modells, sondern eine Geschäftsentscheidung, die wir weiter unten bei den Metriken wieder aufgreifen.

Als Loss dient bei Klassifikation nicht der quadratische Fehler, sondern der *Log Loss* (auch *Cross-Entropy*): Er bestraft ein Modell stark, wenn es sich mit hoher Wahrscheinlichkeit für die falsche Klasse entscheidet. Das Training läuft sonst wie oben beschrieben mit Gradientenabstieg.

Der rechte Plot zeigt die Natur des Modells: Es trennt die Welt mit einer Geraden. Wo die Gerade verläuft, steht die Wahrscheinlichkeit bei 50 %; je weiter ein Punkt davon entfernt ist, desto sicherer wird das Modell. In unseren Daten tritt ein Ausfall ein, wenn die Temperatur über 80 Grad *oder* die Schwingung über 6 mm/s liegt. Diese Region hat die Form eines „L“, und eine Gerade kann sie nur grob nachbilden. Das ist die Motivation für die nächsten beiden Abschnitte.

```python
# Ausschnitt aus 04_klassifikation.py
log = LogisticRegression().fit(X_tr, y_tr)

probe = pd.DataFrame({"temperatur_c": [78.0], "schwingung_mm_s": [5.0]})
z = log.decision_function(probe)[0]   # der Logit
p = 1 / (1 + np.exp(-z))              # Sigmoid von Hand
print(f"z = {z:.3f}, sigmoid(z) = {p:.3f}")
print("predict_proba:", log.predict_proba(probe)[0].round(3))  # dasselbe Ergebnis aus sklearn
```

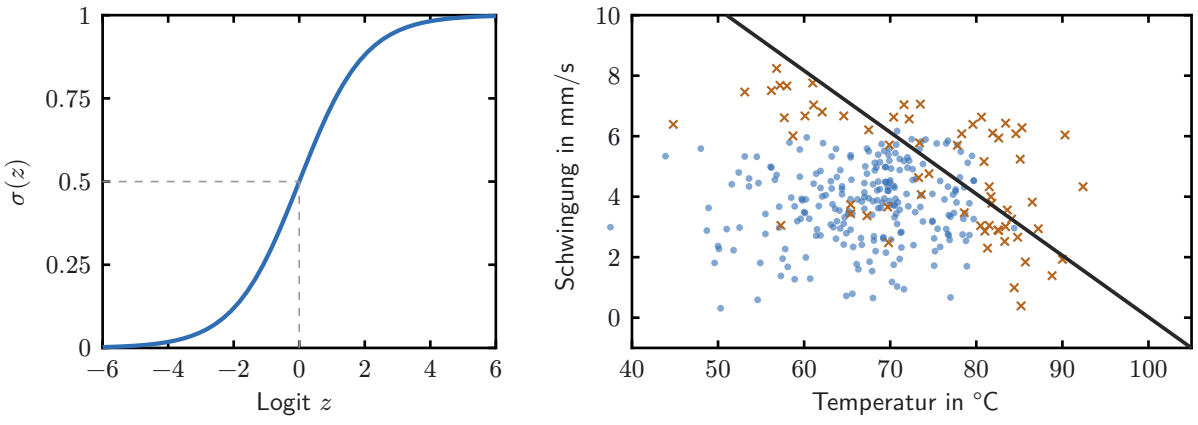


Abbildung 5: Links: Die Sigmoid-Funktion bildet jeden Logit auf eine Wahrscheinlichkeit ab; bei $z = 0$ liegt sie bei 0,5. Rechts: 300 der 1000 Maschinen (Kreise: kein Ausfall, Kreuze: Ausfall) und die gelernte Entscheidungsgrenze der logistischen Regression, an der $z = 0$ gilt.

Terminal

```
$ python 04_klassifikation.py
z = 0.161 * temperatur + 0.791 * schwingung + (-16.124)

Maschine mit 78 °C, 5 mm/s: z = 0.397, sigmoid(z) = 0.598
predict_proba: [0.402 0.598]
Schwellwert 0.5: 53 von 300 Maschinen als 'fällt aus' markiert, Genauigkeit 0.837
Schwellwert 0.3: 98 von 300 Maschinen als 'fällt aus' markiert, Genauigkeit 0.827

Drei Klassen (kein Fehler, Überhitzung, Lagerschaden)
Logits : [ 1.837  1.541 -3.378]
Softmax: [0.572 0.425 0.003] Summe = 1.0
predict_proba: [0.572 0.425 0.003]
Sigmoid je Logit: [0.863 0.824 0.033] Summe = 1.719
```

Für die Beispielmaschine mit 78 °C und 5 mm/s ergibt sich ein Logit von 0,40 und damit eine Ausfallwahrscheinlichkeit von knapp 60 %. Wie viele Maschinen das Modell meldet, hängt vom Schwellwert ab: Bei 0,5 sind es 53 von 300 (Genauigkeit 83,7 %), bei 0,3 sind es 98 (82,7 %), weil auch weniger sichere Fälle durchgehen.

Sigmoid und Softmax

Bei zwei Klassen genügt eine Wahrscheinlichkeit, denn die der Gegenklasse ergibt sich als Rest auf 1. Bei mehr als zwei Klassen braucht man eine Wahrscheinlichkeit pro Klasse, und die Summe soll 1 ergeben. Dafür dient die *Softmax-Funktion*: Das Modell berechnet pro Klasse einen Logit z_k, und

$$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

macht daraus Anteile, die sich zu 1 addieren. Die Klassen konkurrieren: Steigt die Wahrscheinlichkeit der einen, sinken die der anderen. Im Beispiel unterscheidet das Modell drei Fälle (kein Fehler, Überhitzung, Lagerschaden), und die Ausgabe oben zeigt die Logits der Beispielmaschine, die daraus berechneten Softmax-Werte und, zum Vergleich, die Werte, die man erhielte, wenn man jeden Logit einzeln durch die Sigmoid-Funktion schickte.

Der Unterschied ist für die Patterns wichtig. Mit Softmax gehört jedes Beispiel genau *einer* Klasse an. Mit einem Sigmoid pro Klasse ist jede Entscheidung unabhängig, und ein Beispiel darf zu mehreren Klassen gehören oder zu keiner. Darauf beruht Pattern 6 (Multilabel): Ein Artikel kann zugleich „Politik“ und „Wirtschaft“ sein, und die Wahrscheinlichkeiten müssen sich nicht zu 1 summieren. Ein Foto, das Hund, Katze oder Vogel zeigt, braucht dagegen Softmax.

2.6 Entscheidungsbäume, Random Forest und Gradient Boosting

Ein *Entscheidungsbaum* (*decision tree*) ist die Methode, die Softwareentwicklern am vertrautesten vorkommt, denn er besteht aus verschachtelten if-Anweisungen. Das Besondere: Die Bedingungen schreibt niemand, das Training findet sie. Es probiert für jedes Feature mögliche Schwellen aus, wählt die Frage, die die Beispiele am saubersten in Gruppen mit einheitlichem Label teilt, und wiederholt das für jede Gruppe, bis eine Tiefengrenze erreicht ist oder die Gruppen rein sind. Die Blätter am Ende tragen die Vorhersage.

Terminal

```
$ python 05_baeume.py
|--- temperatur_c <= 79.85
|   |--- schwingung_mm_s <= 6.00
|   |   |--- class: 0
|   |--- schwingung_mm_s >  6.00
|   |   |--- class: 1
|--- temperatur_c >  79.85
|   |--- temperatur_c <= 80.90
|   |   |--- class: 1
|   |--- temperatur_c >  80.90
|   |   |--- class: 1
```

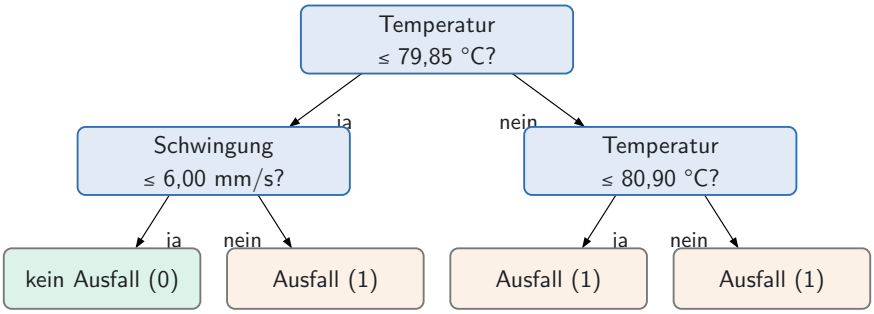


Abbildung 6: Der Entscheidungsbaum der Tiefe 2 aus der Ausgabe oben. Die Schwellen 79,85, 6,00 und 80,90 hat das Training gefunden. Die rechte Teilung ändert die Vorhersage nicht: Beide Blätter sagen „Ausfall".

Abbildung 6 zeigt denselben Baum als Diagramm. Zwei Fragen genügen für eine brauchbare Vorhersage: Ist die Schwingung hoch? Ist die Temperatur hoch? Der Baum hat die „oder"-Struktur der Daten gefunden, die eine Gerade nicht ausdrücken konnte, und man kann ihn Satz für Satz erklären. Dass die zweite Frage rechts nichts an der Vorhersage ändert, ist ein typischer Zug von Bäumen: Das Training wählt die Frage nach der Reinheit der entstehenden Gruppen, nicht danach, ob sich die Vorhersage ändert. Der große Vorzug von Bäumen ist ihre Lesbarkeit, und deshalb sind sie auch dann beliebt, wenn andere Verfahren genauer wären.

Ein einzelner Baum hat eine Schwäche: Lässt man ihn tief wachsen, merkt er sich die Trainingsbeispiele (mehr dazu im Abschnitt über Generalisierung), und kleine Änderungen an den Daten ergeben einen ganz anderen Baum. Zwei Ideen beheben das, indem sie viele Bäume kombinieren.

Random Forest trainiert viele Bäume parallel. Jeder sieht eine zufällige Stichprobe der Beispiele und bei jeder Frage nur eine zufällige Auswahl der Features, sodass die Bäume verschieden ausfallen. Bei der Vorhersage stimmen alle ab, bei Regression wird gemittelt. Die Fehler einzelner Bäume gleichen sich dabei teilweise aus [3], [4].

Gradient Boosting geht umgekehrt vor: Die Bäume entstehen nacheinander. Der erste macht eine grobe Vorhersage, der zweite lernt, den verbleibenden Fehler des ersten zu korrigieren, der dritte den Rest davon, und so weiter. (Genau genommen lernt jeder neue Baum die Richtung, in die sich der Loss am stärksten verringern lässt; beim quadratischen Fehler sind das gerade die Restfehler.) Die Vorhersage ist die Summe aller Bäume, jeweils mit einer kleinen Lernrate gewichtet [5]. Auch hier steckt der Gradientenabstieg dahinter, nur dass er nicht Parameter, sondern ganze Bäume hinzufügt.

Der zweite Teil derselben Ausgabe vergleicht alle Verfahren auf denselben 700 Trainings- und 300 Testmaschinen. Achten Sie auf den Unterschied zwischen den beiden Spalten.

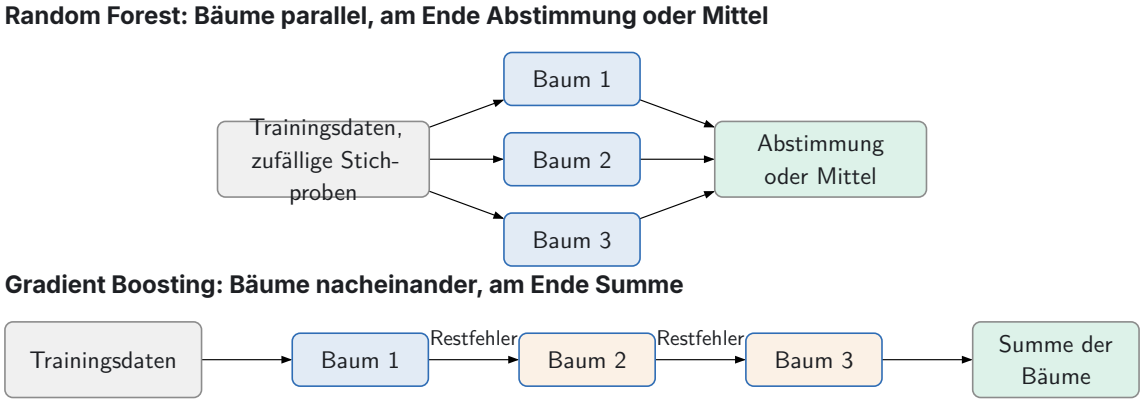


Abbildung 7: Zwei Arten, Bäume zu kombinieren. Beim Random Forest entstehen die Bäume unabhängig voneinander und stimmen ab. Beim Boosting lernt jeder Baum den Fehler, den die vorherigen übrig lassen; die Vorhersage ist die gewichtete Summe aller Bäume.

Terminal

```
Modell                            Training    Test
Logistische Regression               0.839   0.837
Baum, Tiefe 2                        0.954   0.953
Baum, unbegrenzt                     1.000   0.923
Random Forest (200 Bäume)            1.000   0.953
Gradient Boosting (100 Bäume)        0.970   0.930

Unbegrenzter Baum: Tiefe 20 , Blätter 76
```

Die logistische Regression bleibt bei rund 84 %, weil die Datenform nicht zu ihrer Geraden passt. Der flache Baum erreicht auf den Testdaten rund 95 % und ist damit so gut wie der Random Forest. Das liegt an den Daten: Wir haben 5 % der Labels absichtlich vertauscht, sodass mehr als etwa 95 % nicht zu holen sind. Der unbegrenzte Baum zeigt, was passiert, wenn man das ignoriert: Er wächst auf Tiefe 20 mit 76 Blättern, erreicht auf den Trainingsdaten 100 % und merkt sich dabei das vertauschte Rauschen. Auf den Testdaten fällt er auf etwa 92 %, also schlechter als der Baum mit zwei Fragen. Das ist Overfitting, das der nächste Abschnitt genauer betrachtet. Random Forest dagegen hält den Testwert trotz 100 % im Training, weil die vielen Bäume sich im Mittel ausgleichen.

Für Tabellendaten, also Zeilen und Spalten wie in unseren Beispielen, sind Baum-Ensembles in der Praxis oft die erste Wahl, noch vor neuronalen Netzen; ein Vergleich auf vielen Datensätzen bestätigt das für mittelgroße Tabellen (in der Größenordnung von 10 000 Beispielen) [6]. Das erklärt, warum viele Patterns in diesem Handbuch nicht an neuronale Netze gebunden sind.

2.7 Neuronale Netze und Embeddings

Ein *neuronales Netz* besteht aus vielen Einheiten, die jeweils tun, was Sie bei der logistischen Regression gesehen haben: eine gewichtete Summe der Eingaben bilden und durch eine nichtlineare Funktion schicken. Eine solche Einheit heißt *Neuron*. Man ordnet viele Neuronen in *Schichten* (*layers*) an; die Ausgaben einer Schicht sind die Eingaben der nächsten (Abbildung 8). Die Zahlen an den Verbindungen, die *Gewichte*, sind die Parameter des Modells. Die nichtlineare Funktion in jedem Neuron ist wesentlich: Ohne sie wäre eine Hintereinanderschaltung von Schichten mathematisch wieder nur eine einzige lineare Funktion.

Trainiert wird ein neuronales Netz wie jedes andere Modell: Loss berechnen, Gradient bestimmen, Parameter ein Stück bergab verschieben. Die Berechnung des Gradienten über alle Schichten hinweg heißt *Backpropagation*; sie wendet die Kettenregel der Differentialrechnung an und ist aus Sicht des Anwenders ein Implementierungsdetail der Bibliothek (Kapitel 3 rechnet sie an einem kleinen Netz von Hand nach). Das Wort *tief* in *Deep Learning* bezeichnet einfach viele Schichten. Der Gewinn liegt darin, dass die Schichten nacheinander immer abstraktere Zwischen-Features bilden, die niemand von Hand entwerfen muss: bei Bildern etwa Kanten, dann Formen, dann Objektteile. Das ist der Grund, warum neuronale Netze bei Bildern, Text und Ton so viel leisten, wo Features sich schlecht von Hand angeben lassen.

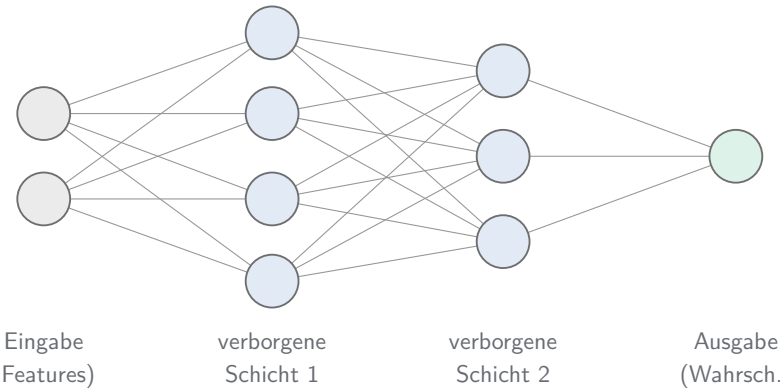


Abbildung 8: Ein kleines neuronales Netz mit 2, 4, 3 und 1 Neuronen pro Schicht. Jede Linie ist ein Gewicht; jedes Neuron außer in der Eingabe hat zusätzlich einen Bias (das b aus der Geradengleichung). Dieses Netz hat $2 \cdot 4 + 4 + 4 \cdot 3 + 3 + 3 \cdot 1 + 1 = 31$ Parameter.

```
Terminal
$ python 06_neuronale_netze.py
Genauigkeit Test: 0.937
Schicht 1: Gewichte (2, 16), Bias (16,)
Schicht 2: Gewichte (16, 8), Bias (8,)
Schicht 3: Gewichte (8, 1), Bias (1,)
Parameter insgesamt: 193
```

Das Netz erreicht auf den Testdaten rund 94 %, mit 193 Parametern, während der Baum der Tiefe 2 mit drei gelernten Schwellen auf 95 % kommt. Für kleine Tabellen ist ein neuronales Netz selten die richtige Wahl. Seine Stärke beginnt dort, wo Eingaben roh und hochdimensional sind, etwa Pixel oder Text.

Embeddings

Neuronale Netze rechnen mit Zahlen. Wie füttert man ein Netz mit einem Wort, einer Produkt-ID oder einem Nutzer? Die naive Antwort ist die *One-hot-Kodierung*: ein Vektor mit so vielen Einträgen wie Kategorien, in dem genau eine Stelle 1 ist und alle anderen 0 sind. Bei 50 000 Produkten sind das 50 000 Dimensionen, fast nur Nullen, und das Modell erfährt nicht, dass zwei Produkte einander ähnlicher sind als zwei andere.

Ein *Embedding* ersetzt diesen Vektor durch einen kurzen, dichten, etwa 8 bis 300 Einträge langen. Technisch ist es eine Tabelle E mit einer Zeile pro Kategorie; die Eingabe wählt nur eine Zeile aus. Das ist dasselbe wie One-hot-Vektor mal Matrix, nur ohne die vielen Multiplikationen mit Null. Die Einträge der Tabelle sind Parameter, die das Training mitlernt (im Beispiel unten sind sie zur Veranschaulichung zufällig gefüllt, nur der Nachschlagemechanismus ist echt), und dabei wandern Kategorien, die sich im Modell ähnlich verhalten, im Vektorraum nahe beieinander. Bekannt wurde das Prinzip durch word2vec für Wörter [7]. Wie die Tabelle beim Training Zeile für Zeile gelernt wird, zeigt Kapitel 3.

```
Terminal
Embedding-Tabelle E (eine Zeile pro Wort):
  drucker   [0.35 0.82 0.33]
  scanner   [-1.3   0.91  0.45]
  passwort  [-0.54  0.58  0.36]
  login     [0.29 0.03 0.55]
  vpn       [-0.74 -0.16 -0.48]
One-hot für 'passwort': [0. 0. 1. 0. 0.]
one_hot @ E            : [-0.54  0.58  0.36]
E[idx]                 : [-0.54  0.58  0.36]
```

Ein kleines Experiment mit einer echten, wenn auch anderen Technik zeigt das Prinzip „Nähe aus Nutzung“. Aus zwölf kurzen Support-Tickets bilden wir mit einer Matrixzerlegung (*Latent Semantic Analysis*) für jedes Wort einen dreidimensionalen Vektor und messen den Kosinus des Winkels zwischen zwei Vektoren: nahe +1 heißt gleiche Richtung, nahe 0 heißt unabhängig. Das ist nicht das Verfahren, mit dem neuronale Netze Embeddings lernen, aber es zeigt dasselbe Ergebnis: Wörter, die in denselben Zusammenhängen vorkommen (drucker und scanner, passwort und login), bekommen ähnliche Vektoren; Wörter aus verschiedenen Themen liegen nahe bei null.

One-hot
0
0
1
0
0

×

Wort	d1	d2	d3
drucker	0,35	0,82	0,33
scanner	−1,3	0,91	0,45
passwort	−0,54	0,58	0,36
login	0,29	0,03	0,55
vpn	−0,74	−0,16	−0,48

=

Vektor für „passwort“		
−0,54	0,58	0,36

Abbildung 9: Ein Embedding als Nachschlagetabelle: Der One-hot-Vektor wählt aus der Tabelle E genau eine Zeile. Die Werte stammen aus der Ausgabe oben; in einem echten Modell wären sie nicht zufällig, sondern gelernt.

```
Terminal
Ähnlichkeit(drucker, scanner) = +0.93
Ähnlichkeit(passwort, login) = +0.99
Ähnlichkeit(drucker, passwort) = +0.18
Ähnlichkeit(vpn, drucker) = -0.14
```

Die Vektoren aus dem Embedding sind keine Handarbeit, sondern ein Produkt der Daten. Die Patterns 2 (Embeddings), 4 (Multimodal Input) und 13 (Transfer Learning) bauen darauf auf.

2.8 Generalisierung: Overfitting, Splits und Datenlecks

Das Ziel eines Modells ist nicht, die Trainingsbeispiele zu reproduzieren, sondern bei Beispielen zu funktionieren, die es nie gesehen hat. Diese Fähigkeit heißt *Generalisierung*. Ein Modell kann auf zwei Arten daran scheitern.

Underfitting liegt vor, wenn das Modell zu einfach ist, um das Muster abzubilden, etwa eine Gerade für einen Verlauf mit Kurve. *Overfitting* liegt vor, wenn das Modell so flexibel ist, dass es auch das Rauschen der Trainingsdaten nachbildet: Es hat die Beispiele auswendig gelernt statt des Zusammenhangs. Aus Entwicklersicht ist das verwandt mit einem Programm, das die erwarteten Werte der Testfälle hart kodiert.

Abbildung 10 zeigt das an einem Experiment, bei dem wir die Wahrheit kennen: eine Sinuskurve mit Rauschen, 15 gleichmäßig verteilte Trainingspunkte, Polynome mit steigendem Grad. Der Grad bestimmt die Flexibilität des Modells.

Die Zahlen dahinter machen die Beobachtung messbar. Wir berechnen den Fehler (RMSE, siehe unten) einmal auf den Trainingsdaten und einmal auf 500 weiteren Punkten, die das Modell nie gesehen hat.

```
Terminal
$ python 07_overfitting.py
Grad  RMSE Training  RMSE Test
   1          0.452      0.525
   2          0.452      0.525
   3          0.312      0.315
   4          0.308      0.319
   6          0.265      0.329
   8          0.211      0.348
  10          0.160      0.451
  12          0.159      0.413
  13          0.142      1.516
  14          0.000     12.149
```

Der Trainingsfehler fällt mit steigendem Grad immer weiter und erreicht bei Grad 14 den Wert null, denn 15 Punkte lassen sich mit einem Polynom vom Grad 14 genau treffen. Der Testfehler dagegen fällt zuerst, liegt zwischen Grad 3 und 8 bei 0,32 bis 0,35 und steigt dann: leicht ab Grad 10, dramatisch bei Grad 13 und 14 (auf 12,1; rechtes Diagramm in Abbildung 10). Würde man nur den Trainingsfehler beachten,

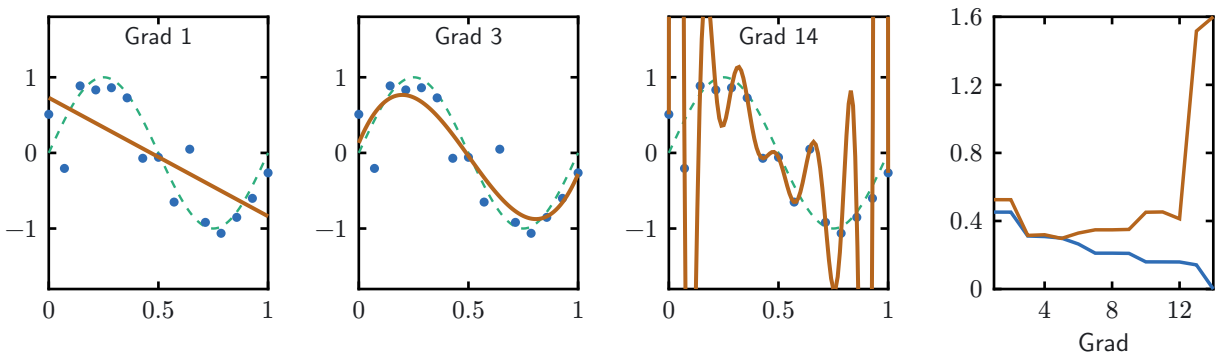


Abbildung 10: Links drei Modelle für dieselben 15 Trainingspunkte (blau); gestrichelt grün die wahre Funktion, orange das gelernte Modell (Grad 14 ist am Plotrand abgeschnitten). Rechts der RMSE auf Trainings- (blau) und Testdaten (orange) über dem Polynomgrad; der Testfehler ist bei 1,6 abgeschnitten. Grad 1 unteranpasst, Grad 3 trifft das Muster, Grad 14 läuft durch jeden Punkt und schwingt dazwischen wild.

Training (60 %)				**Validierung** (20 %)	**Test** (20 %)
Kreuzvalidierung mit $k = 5$ (die Testmenge bleibt außen vor):					
Validierung	Training	Training	Training	Training	Test
Training	Validierung	Training	Training	Training	Test
Training	Training	Validierung	Training	Training	Test
Training	Training	Training	Validierung	Training	Test
Training	Training	Training	Training	Validierung	Test

Abbildung 11: Aufteilung der Daten. Oben die feste Dreiteilung, darunter die fünf Durchläufe der Kreuzvalidierung: In jedem Durchlauf ist ein anderes Fünftel der Entwicklungsdaten die Validierungsmenge.

wählte man den schlechtesten Grad. Dass gerade Grade wie 2 und 4 kaum etwas ändern, liegt an der Sinuskurve, die zum Mittelpunkt hin punktsymmetrisch ist und deshalb fast nur ungerade Terme braucht.

Train, Validation, Test

Daraus folgt eine Grundregel: Ein Modell darf nie mit denselben Daten bewertet werden, mit denen es trainiert wurde. Man teilt die Daten deshalb in drei Teile (Abbildung 11):

- Die **Trainingsmenge** (*training set*) dient dazu, die Parameter zu lernen.
- Die **Validierungsmenge** (*validation set*) dient dazu, Entscheidungen des Entwicklers zu prüfen: welches Verfahren, welche Hyperparameter, welcher Polynomgrad. Man darf sie oft benutzen, doch jede Entscheidung, die man nach ihrem Ergebnis trifft, passt das Modell ein Stück an sie an.
- Die **Testmenge** (*test set*) wird ganz zum Schluss genau einmal benutzt, um die Qualität ehrlich zu schätzen. Sobald man nach ihrem Ergebnis noch Entscheidungen trifft, ist sie keine Testmenge mehr, sondern eine zweite Validierungsmenge.

Eine Aufteilung in 60 % Training, 20 % Validierung und 20 % Test ist verbreitet, aber keine Regel. Bei wenigen Daten ist eine feste Validierungsmenge unökonomisch und stark vom Zufall abhängig. Dann nimmt man die *Kreuzvalidierung* (*cross-validation*): Man teilt die Daten in k Teile (*folds*), trainiert k-mal auf $k-1$ Teilen, bewertet jeweils auf dem übrigen und mittelt die k Ergebnisse. Die Streuung der Einzelergebnisse zeigt gleich, wie verlässlich die Schätzung ist.

Terminal

```
$ python 08_split_und_leck.py
Training/Validierung/Test: 600 200 200
max_depth= 1: Validierung 0.895
max_depth= 2: Validierung 0.970
max_depth= 3: Validierung 0.970
max_depth= 5: Validierung 0.970
max_depth=10: Validierung 0.955

5-fache Kreuzvalidierung: [0.944 0.931 0.962 0.962 0.962]
Mittel 0.953, Streuung 0.013

Zufallsdaten, Auswahl vor dem Split  : 0.84 Genauigkeit im Mittel (Leck)
Zufallsdaten, Auswahl in der Pipeline: 0.51 Genauigkeit im Mittel (ehrlich)
```

Datenlecks

Die gefährlichste Fehlerquelle bei der Bewertung ist das *Datenleck* (*data leakage*): Information, die zum Zeitpunkt der Vorhersage nicht verfügbar wäre, gelangt in das Training oder in die Bewertung [8]. Das Ergebnis sieht großartig aus und bricht im Betrieb ein. Typische Formen sind:

- Ein Feature enthält das Label indirekt, zum Beispiel ein Feld „Rückerstattung veranlasst“ bei der Betrugserkennung, das erst nach der Betrugsprüfung gesetzt wird.
- Eine Vorverarbeitung (Skalierung, Auswahl von Features, Auffüllen fehlender Werte) wird auf allen Daten gelernt, bevor man in Training und Test teilt.
- Dasselbe Beispiel oder sehr ähnliche Beispiele (derselbe Kunde, dieselbe Maschine, Duplikate) stehen sowohl im Training als auch im Test.

- Bei Zeitreihen kommen Beobachtungen aus der Zukunft in das Training, wenn man zufällig statt zeitlich teilt.

Das Experiment am Ende der Ausgabe oben macht die zweite Form sichtbar. Wir erzeugen jeweils 100 Beispiele mit 2000 Features, die reiner Zufall sind, und zufällige Labels. Es gibt nichts zu lernen, ehrlich wären rund 50 % Genauigkeit. Dieses Experiment ist ein Klassiker der Statistik [9, Abschn. 7.10.2]. Wählt man jedoch die 20 „besten" Features auf *allen* Daten aus und führt danach erst die Kreuzvalidierung durch, sieht das Modell durch die Auswahl schon die Labels der Validierungsfolds. Im Mittel über zehn solche Zufallsdatensätze erreicht es dann 84 % Genauigkeit, obwohl nichts zu lernen ist. Liegt die Auswahl dagegen in einer Pipeline, die sie pro Fold nur auf dem Trainingsteil lernt, fällt die Genauigkeit mit 51 % auf Zufallsniveau.

ACHTUNG
Datenlecks fallen nicht durch Fehlermeldungen auf, sondern durch Zahlen, die zu gut sind. Wenn ein Modell in der Entwicklung 99 % erreicht und im Betrieb 70 %, ist ein Leck der erste Verdacht. Mehrere Patterns (etwa Transform und Repeatable Splitting) schützen davor, indem sie festlegen, *wo* und *wann* etwas gelernt wird.

2.9 Metriken: Wie gut ist gut?

Wie misst man, ob ein Klassifikator gut ist? Die erste Antwort ist die *Accuracy*, der Anteil richtiger Vorhersagen. Sie ist leicht zu erklären und bei ausgeglichenen Klassen brauchbar. Bei Ungleichgewicht trügt sie, und das ist der Normalfall: Betrug, Defekte und Ausfälle sind selten. Ein Modell, das immer „kein Betrug" sagt, hat bei 1 % Betrugsanteil 99 % Accuracy und ist völlig nutzlos.

Genauer sieht man in der *Konfusionsmatrix* (*confusion matrix*), die zählt, wie oft jede Kombination aus wahrer und vorhergesagter Klasse auftrat (Abbildung 12). Die positive Klasse ist die, nach der man sucht (Betrug). Die vier Felder heißen True Positive (TP, Betrug erkannt), False Positive (FP, falscher Alarm), False Negative (FN, Betrug übersehen) und True Negative (TN, zu Recht ignoriert).

Aus den vier Zahlen ergeben sich die gebräuchlichen Metriken:

- **Precision** $= \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$: Von allen Alarmen, wie viele waren echt? Hoch, wenn das Modell selten falschen Alarm schlägt.
- **Recall** $= \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$: Von allen echten Fällen, wie viele wurden gefunden? Hoch, wenn das Modell kaum etwas übersieht.
- **F1** ist das harmonische Mittel von Precision und Recall. Es ist nur dann hoch, wenn beide hoch sind.

Precision und Recall stehen in einem Konflikt, den der Schwellwert steuert. Senkt man ihn, schlägt das Modell häufiger an: Der Recall steigt, die Precision sinkt. Was richtig ist, entscheidet die Fachlichkeit, nicht die Statistik: Bei einer Krebsvorsorge nimmt man falschen Alarm in Kauf, um nichts zu übersehen; bei einer automatischen Kontosperrung ist ein falscher Alarm teuer.

Das folgende Experiment verwendet 20 000 Transaktionen mit rund 1 % Betrug.

Schwelle 0,5

	vorhergesagt: kein Betrug	vorhergesagt: Betrug
wahr: kein Betrug	TN **4950**	FP **0**
wahr: Betrug	FN **45**	TP **5**

Schwelle 0,1

	vorhergesagt: kein Betrug	vorhergesagt: Betrug
wahr: kein Betrug	TN **4940**	FP **10**
wahr: Betrug	FN **36**	TP **14**

Abbildung 12: Konfusionsmatrizen desselben Modells bei zwei Schwellwerten (Testdaten: 5000 Transaktionen, 50 Betrugsfälle). Grün sind richtige, orange falsche Vorhersagen. Die Zahlen stammen aus der Ausgabe unten.

Terminal

```
$ python 09_metriken.py
Betrugsanteil im Test: 0.010 (50 von 5000)

'Immer kein Betrug': Accuracy 0.9900, Recall 0.00

Schwelle 0.5: TN=4950 FP=0 FN=45 TP=5
  Accuracy 0.9910  Precision 1.00  Recall 0.10  F1 0.18

Schwelle 0.1: TN=4940 FP=10 FN=36 TP=14
  Accuracy 0.9908  Precision 0.58  Recall 0.28  F1 0.38

ROC-AUC (schwellenunabhängig): 0.688

Miete: MAE 50.3 Euro, RMSE 64.6 Euro
Mit einem Ausreißer von +1000 Euro: MAE 68.5, RMSE 149.6
```

Der Dummy, der nie Betrug meldet, erreicht 99,0 % Accuracy bei einem Recall von null. Das trainierte Modell erreicht bei Schwelle 0,5 mit 99,1 % kaum mehr, obwohl es immerhin 5 der 50 Betrugsfälle findet und dabei keinen falschen Alarm schlägt (Precision 1,00, Recall 0,10). Wer nur auf die Accuracy sieht, erkennt den Unterschied zwischen einem nutzlosen und einem etwas nützlichen Modell nicht. Bei der Schwelle 0,1 sinkt die Accuracy minimal, während der Recall von 0,10 auf 0,28 steigt; dafür fallen zehn falsche Alarme an (Precision 0,58). Ob das ein guter Tausch ist, hängt davon ab, was ein falscher Alarm und was ein übersehener Betrug kosten.

ROC-AUC

Um ein Modell unabhängig vom Schwellwert zu beurteilen, trägt man für alle möglichen Schwellwerte die Rate der gefundenen echten Fälle (Recall, hier *True-Positive-Rate*) gegen die Rate der falschen Alarme (*False-Positive-Rate*, $\frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}}$) auf. Die entstehende *ROC-Kurve* (*receiver operating characteristic*) verläuft bei einem Zufallsmodell auf der Diagonalen und bei einem perfekten Modell oben links. Die Fläche darunter, die *AUC* (*area under the curve*), fasst die Kurve in einer Zahl zusammen. Anschaulich ist sie die Wahrscheinlichkeit, dass das Modell einem zufällig gewählten positiven Beispiel eine höhere Wahrscheinlichkeit gibt als einem zufällig gewählten negativen. 0,5 entspricht Raten, 1,0 einer perfekten Trennung.

Bei sehr seltenen positiven Fällen kann die ROC-Kurve zu optimistisch wirken, weil die vielen negativen Beispiele die False-Positive-Rate klein halten. Dann ist die Precision-Recall-Kurve aussagekräftiger [10].

MAE und RMSE

Bei Regression misst man den Abstand zwischen Vorhersage und Wahrheit. Der *mittlere absolute Fehler* (MAE) mittelt die Beträge der Abweichungen und ist in der Einheit des Labels lesbar: „Im Mittel liegen wir 50 Euro daneben.“ Der *RMSE* (Wurzel des mittleren quadratischen Fehlers) quadriert vor dem Mitteln und zieht danach die Wurzel; er bestraft große Ausreißer stärker. Liegen beide weit auseinander, gibt es vereinzelt sehr grobe Fehler. Die letzten Zeilen der Ausgabe zeigen das: Ein einziger Ausreißer von

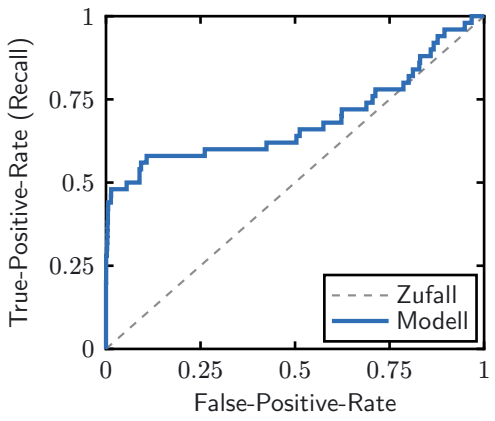


Abbildung 13: ROC-Kurve des Betrugsmodells auf den Testdaten. Je weiter die Kurve nach oben links wölbt, desto besser trennt das Modell. Die AUC steht in der Ausgabe oben (0,69): besser als Raten, aber weit von einer sauberen Trennung entfernt.

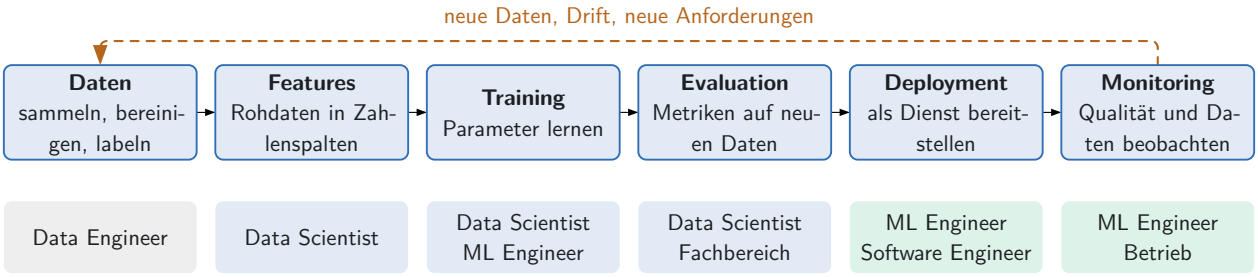


Abbildung 14: Der ML-Lebenszyklus mit typischen Rollen. Der gestrichelte Pfeil ist der Kreislauf: Beobachtungen aus dem Betrieb führen zu neuen Daten und einem neuen Training.

1000 Euro hebt den MAE von 50 auf 69 Euro, den RMSE aber von 65 auf 150 Euro, also auf mehr als das Doppelte.

2.10 Der ML-Lebenszyklus

Ein Modell zu trainieren ist ein kleiner Teil der Arbeit. Sculley et al. zeigten bereits 2015, dass der eigentliche ML-Code in einem produktiven System nur ein kleiner Kasten inmitten von Datenbeschaffung, Konfiguration, Werkzeugen, Überwachung und Serving-Infrastruktur ist [11]. Der Lebenszyklus in Abbildung 14 fasst diese Arbeit in Stufen und ordnet ihnen die Rollen zu, die sie in der Praxis meist ausfüllen. Die Rollenbezeichnungen variieren zwischen Organisationen; hier zählt, dass verschiedene Menschen mit verschiedenen Werkzeugen beteiligt sind.

Daten. Transaktionssysteme, Sensoren, Logs oder manuelle Erfassung liefern die Rohdaten, die gesammelt, bereinigt und beim überwachten Lernen mit Labels versehen werden. Die Labels sind oft die teuerste Zutat. **Features.** Aus Rohdaten entstehen die Zahlenspalten, die das Modell verarbeiten kann: Kategorien kodieren, Werte skalieren, Zeitfenster aggregieren. In dieser Stufe steckt ein großer Teil der Arbeit, und viele Patterns aus den Teilen II und IV beziehen sich darauf. **Training.** Das Modell lernt die Parameter, der Entwickler wählt die Hyperparameter; der Lauf ist rechenintensiv und kann Stunden dauern. **Evaluation.** Das Modell wird auf Daten bewertet, die es nicht gesehen hat, mit Metriken, die zur Fachlichkeit passen. Hier entscheidet sich, ob es in Betrieb gehen darf, und der Fachbereich sollte mitreden, was „gut genug" heißt. **Deployment.** Das Modell wird als Dienst oder Batch-Job bereitgestellt und ist damit Software im Betrieb mit den üblichen Anforderungen: Latenz, Verfügbarkeit, Versionierung. **Monitoring.** Im Betrieb kann die Qualität sinken, ohne dass sich eine Zeile Code ändert; die Beobachtung führt zurück zu den Daten.

Die Stufen sind weder strikt sequenziell noch einmalig: Man kehrt von der Evaluation zu den Features zurück, und im Betrieb läuft der ganze Kreis wiederholt. Kapitel 4 verteilt die dreißig Patterns auf diese Stufen.

2.11 Begriffe für den Betrieb

Vier Begriffsgruppen tauchen in den Patterns ständig auf. Sie sind schnell erklärt, aber leicht zu verwechseln.

Inferenz und Serving. *Inferenz* (*inference*) heißt, mit einem trainierten Modell Vorhersagen zu berechnen. *Serving* bezeichnet die Infrastruktur, die das anbietet. Es gibt *Online-Serving*, bei dem einzelne Anfragen innerhalb von Millisekunden beantwortet werden, und *Batch-Serving*, bei dem man viele Eingaben auf einmal verarbeitet.

Parameter und Hyperparameter. *Parameter* lernt das Training (Gewichte, Schwellen in einem Baum). *Hyperparameter* legt der Entwickler vorher fest (Lernrate, Baumtiefe, Anzahl der Schichten, Batchgröße). Hyperparameter werden an der Validierungsmenge ausgewählt, nie an der Testmenge.

Training-Serving Skew. Berechnen Training und Serving die Features unterschiedlich, sieht das Modell im Betrieb Daten, an die es nie gewöhnt wurde. Der Fehler entsteht meist durch zwei Implementierungen derselben Logik, eine im Trainingsskript und eine im Dienst. Er fällt nicht auf, weil nichts abstürzt, es werden nur die Vorhersagen schlechter.

Data Drift und Concept Drift. Im Betrieb ändert sich die Welt. Verschiebt sich die Verteilung der *Eingaben* (im Sommer sind alle Maschinen wärmer), spricht man von *Data Drift* oder Covariate Shift. Ändert sich der *Zusammenhang* zwischen Eingaben und Label (nach einer neuen Wartungsstrategie fallen Maschinen schon bei niedrigerer Temperatur aus), spricht man von *Concept Drift* [12]. Data Drift bleibt folgenlos, solange die Regel dahinter weiter gilt und das Modell in den neuen Wertebereichen zuverlässig arbeitet; Concept Drift trifft das Modell dagegen direkt, weil die Regel, die es gelernt hat, nicht mehr stimmt.

Die folgenden Experimente machen die Begriffe greifbar.

Terminal

```
$ python 10_skew_und_drift.py
Hyperparameter (vorher gewählt): max_depth = 2
Parameter (gelernt): Schwellen = [79.85  6.   80.9 ]

Offline-Genauigkeit (Celsius)      : 0.827
Serving mit Fahrenheit-Werten      : 0.280
Anteil echter Ausfälle im Test     : 0.280

Data Drift: Temperatur Training Mittel 69.3, Sommer 82.2, KS-Statistik 0.50, p = 6.1e-95
Genauigkeit im Sommer, Regel unverändert: 0.943

Concept Drift (Schwelle jetzt 75 °C): Genauigkeit 0.837 statt 0.947
```

Der erste Teil trennt die gelernten Schwellen des Baums (Parameter) von seinem Hyperparameter `max_depth`. Im zweiten Teil wird eine Pipeline aus Skalierung und logistischer Regression in Celsius trainiert, im Betrieb aber mit Fahrenheit-Werten bei unverändertem Spaltennamen gefüttert. Nichts stürzt ab, die Genauigkeit fällt jedoch von 82,7 % auf 28,0 %. Das ist exakt der Anteil echter Ausfälle: Alle Temperaturen liegen jetzt weit jenseits der Entscheidungsgrenze, und das Modell meldet für jede Maschine einen Ausfall. Im dritten Teil verschiebt der Sommer die Temperaturverteilung (Mittel 82 statt 69 °C). Der Kolmogorov-Smirnov-Test, der zwei Stichproben auf gleiche Verteilung prüft, erkennt die Verschiebung eindeutig (Statistik 0,50). Die Regel dahinter gilt aber unverändert, und ein Baum der Tiefe 3 bleibt mit 94,3 % so gut wie vorher (94,7 % auf den ursprünglichen Testdaten). Im vierten Teil ändert sich die Regel selbst: Ausfälle beginnen schon bei 75 °C. Die Eingaben sehen aus wie zuvor, die Genauigkeit desselben Baums fällt auf 83,7 %, und nur ein Vergleich mit den tatsächlichen Ausfällen zeigt das.

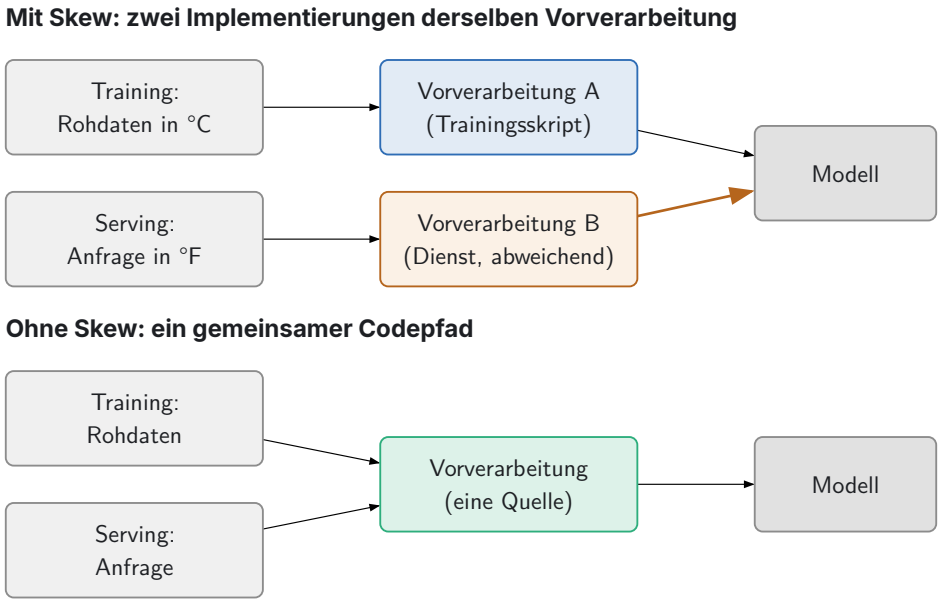


Abbildung 15: Training-Serving Skew entsteht, wenn zwei Codepfade dieselbe Vorverarbeitung unterschiedlich umsetzen (oben, orange); die Pipeline läuft fehlerfrei, liefert aber andere Zahlen. Die Abhilfe ist ein einziger Pfad, den Training und Serving gemeinsam benutzen (unten); das ist die Idee hinter den Patterns Transform und Feature Store.

2.12 Übersetzung in die Welt der Softwareentwicklung

Wer aus dem Software Engineering kommt, versteht ML am schnellsten, wenn er die neuen Begriffe an Bekanntes anbindet. Tabelle 3 tut das. Jede Analogie hinkt an einer Stelle, und gerade dort liegt oft die Lehre.

Tabelle 3: ML-Begriffe und ihre Entsprechung in der Softwareentwicklung.

ML-Begriff	Entsprechung im SE	Wo die Analogie hinkt
Trainingsdaten	Quellcode als Eingabe des Builds	Die Daten bestimmen das Verhalten; Änderungen an ihnen sind Änderungen am Programm und müssen versioniert werden.
Training	Build	Das Ergebnis hängt von Zufall und Daten ab; derselbe Build liefert nicht zwingend dasselbe Artefakt.
Modell mit Parametern	Binary mit Konstanten	Das Verhalten steht nicht im Quelltext, sondern in Zahlen; man kann es nicht lesen, nur testen.
Evaluation	Testsuite	Es gibt keine Pass/Fail-Entscheidung, sondern Metriken, die man gegen eine Schwelle interpretiert.
Overfitting	Testfälle hart kodieren	Der Effekt entsteht ohne böse Absicht, allein durch zu viel Flexibilität.
Datenleck	Ein Test, der die erwartete Antwort kennt	Es fällt nicht auf, weil nichts fehlschlägt.
Training-Serving Skew	Zwei Implementierungen derselben Logik (DRY-Verletzung)	Das System läuft, nur das Ergebnis ist schlechter.
Drift	Die Umgebung ändert sich, der Code nicht	Der Fehler zeigt sich nicht als Ausnahme, sondern als schleichend sinkende Qualität.

Mit diesem Vokabular sind Sie für die Patterns gerüstet. Kapitel 4 zeigt, wo sie am Lebenszyklus sitzen und wie sie zusammenhängen.

3 Neuronale Netze und Repräsentationen verstehen

Die Patterns in den Teilen II und III sprechen ständig von Dingen, die man als Software Engineer zunächst nur benennen, aber nicht nachbauen kann: von Embeddings, die „gelernt" werden, von Checkpoints, die den „Zustand" eines Trainings enthalten, von vortrainierten Modellen, deren Schichten man „einfriert", und von Gradienten, die auf mehreren Rechnern „gemittelt" werden. Dieses Kapitel baut die Mechanik dahinter einmal vollständig auf, mit einem kleinen Netz, das wir in numpy von Grund auf schreiben und trainieren. Das Grundlagenkapitel hat Parameter, Loss und Gradientenabstieg eingeführt und ein Netz als Black Box (`MLPClassifier`) benutzt; hier öffnen wir die Box. Danach lässt sich die Mechanik hinter Pattern 2, 12, 13 und 14 selbst nachrechnen.

LERNZIELE
Nach diesem Kapitel können Sie (1) ein Neuron und ein mehrschichtiges Netz als Folge von Matrixmultiplikationen mit Nichtlinearitäten beschreiben, (2) erklären, wie Backpropagation mit der Kettenregel aus einem Fehler für jeden Parameter einen Gradienten macht, (3) Loss, Softmax, Mini-Batch, Epoche und Lernrate in einem Trainingsprotokoll lesen und Fehlverhalten daran erkennen, (4) begründen, warum eine Embedding-Schicht eine Nachschlagetabelle ist und woher ihre Zahlen kommen, (5) erläutern, warum Features aus einer Aufgabe auf eine andere übertragbar sind und wann nicht, und (6) sagen, was in einen Checkpoint gehört und warum sich Gradienten über mehrere Worker mitteln lassen. Alle Zahlen stammen aus Skripten in `code/02b/`, die Sie selbst ausführen können.

3.1 Ein Neuron: gewichtete Summe und Aktivierung

Ein künstliches Neuron ist kleiner, als der Name vermuten lässt. Es bekommt mehrere Zahlen als Eingabe, multipliziert jede mit einem eigenen **Gewicht**, addiert die Produkte und einen weiteren Parameter, den **Bias**, und schickt das Ergebnis durch eine feste Funktion, die **Aktivierungsfunktion**. Mit Eingaben x_1, x_2 und Gewichten w_1, w_2 ist die Rechnung

$$z = w_1 x_1 + w_2 x_2 + b, \qquad a = \phi(z).$$

Die Gewichte und der Bias sind die lernbaren Parameter; x und a sind Werte, die nur für ein Beispiel gelten. Ohne ϕ wäre das Neuron eine lineare Regression, wie Sie sie aus dem Grundlagenkapitel kennen. Bereits Rosenblatts Perzeptron von 1958 folgt diesem Schema, die Aktivierung war damals eine harte Schwelle [13].

Drei Aktivierungsfunktionen begegnen einem ständig (Abbildung 17). Die **Sigmoid**-Funktion $\frac{1}{1+e^{-z}}$ drückt jeden Wert in das Intervall von 0 bis 1 und lässt sich als Wahrscheinlichkeit lesen; $\tanh$ tut dasselbe zwischen -1 und 1. Die **ReLU** (Rectified Linear Unit), $\max(0, z)$, reicht positive Werte unverändert durch und setzt negative auf null. Sie ist die Standardwahl für versteckte Schichten, weil sie billig zu berechnen ist und für positive z einen Gradienten von genau 1 hat, was das Training tiefer Netze erleichtert [14].

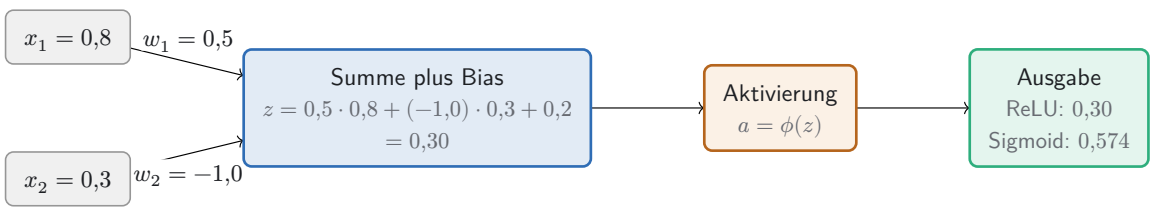


Abbildung 16: Ein Neuron mit zwei Eingaben und dem Bias $b = 0{,}2$. Die Zahlen stammen aus `01_neuron.py`.

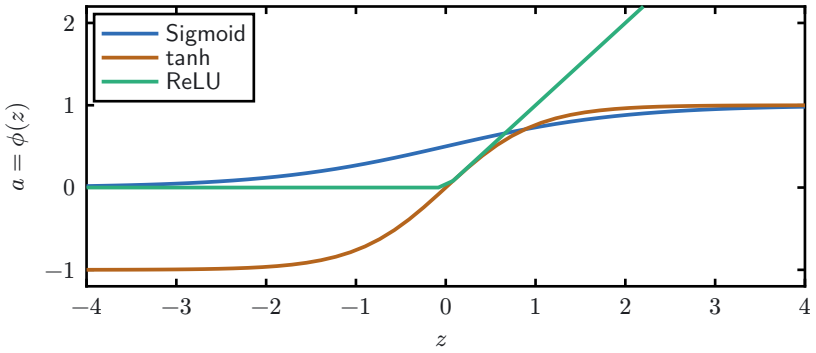


Abbildung 17: Die drei gängigen Aktivierungsfunktionen. Nur ReLU wächst unbegrenzt; Sigmoid und tanh sättigen.

Warum braucht es die Aktivierung überhaupt? Weil lineare Rechnungen sich zusammenfassen lassen. Zwei hintereinandergeschaltete lineare Schichten mit Gewichtsmatrizen A und B ergeben dieselbe Abbildung wie eine einzige Schicht mit der Matrix AB; `01_neuron.py` prüft das mit `np.allclose` nach und gibt `True` aus. Ein Netz aus beliebig vielen rein linearen Schichten bleibt also eine lineare Funktion und kann keine Kurve lernen. Erst die Nichtlinearität dazwischen macht Tiefe nützlich.

3.2 Mehrschichtige Netze

Ein **mehrschichtiges Netz** (Multilayer Perceptron, MLP) legt viele Neuronen nebeneinander zu einer Schicht und mehrere Schichten hintereinander. Die Rechnung einer Schicht für alle Neuronen und alle Beispiele auf einmal ist eine Matrixmultiplikation: Hat die Schicht m Eingänge und n Neuronen, dann ist W eine Matrix mit m Zeilen und n Spalten, und für eine Matrix X aus B Beispielen gilt

$$H = \phi(XW + b), \qquad X \in \mathbb{R}^{B\times m}, W \in \mathbb{R}^{m\times n}, H \in \mathbb{R}^{B\times n}.$$

Die Ausgabe H jeder Schicht ist die Eingabe der nächsten. Die Schichten zwischen Eingabe und Ausgabe heißen **versteckt** (hidden), weil man ihre Werte nicht vorgibt. Es hilft, sie als **Repräsentationen** zu lesen: Jede versteckte Schicht übersetzt die Eingabe in eine neue Darstellung, in der die Aufgabe der nächsten Schicht leichter ist. Das ist der Grund für den Kapiteltitel, und es wird bei Embeddings und Transfer Learning noch wichtig.

Zählen lassen sich die Parameter sofort: W_1 hat 8, W_2 16 und W_3 12 Einträge, dazu kommen 4 + 4 + 3 Biaswerte, zusammen 47. Das Netz, das wir gleich trainieren, hat Schichtgrößen 2, 32, 32, 3 und damit $2 \cdot 32 + 32 + 32 \cdot 32 + 32 + 32 \cdot 3 + 3 = 1251$ Parameter. Alles, was ein Netz „weiß", steckt in diesen Zahlen.

Wie viel zwei Schichten bringen, zeigt das Klassiker-Beispiel XOR: Die Ausgabe soll 1 sein, wenn genau eine der beiden Eingaben 1 ist. Die vier Punkte lassen sich nicht durch eine Gerade trennen, ein einzelnes Neuron scheitert daher, ein Argument, das in der Kritik von Minsky und Papert am Perzeptron eine große Rolle spielte [15]. Zwei ReLU-Neuronen mit von Hand gesetzten Gewichten genügen (Ausgabe von `01_neuron.py`):

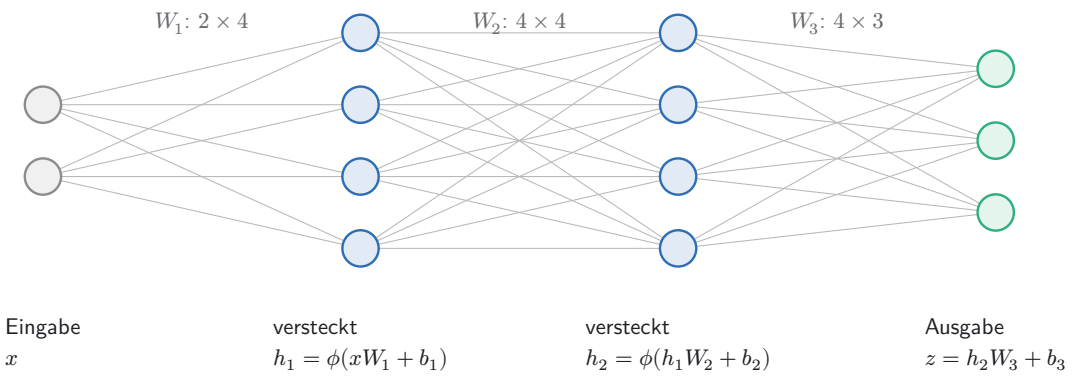


Abbildung 18: Ein kleines MLP mit zwei versteckten Schichten. Jede Linie ist ein Gewicht; die Matrizen darüber fassen sie je Schicht zusammen. Das Netz dieses Kapitels hat dieselbe Form, nur mit 32 statt 4 Neuronen je versteckter Schicht.

```
Terminal
$ uv run --with numpy python 01_neuron.py
x1 x2 | h1 h2 | Ausgabe
 0  0 |  0  0 |  0
 1  0 |  1  0 |  1
 0  1 |  1  0 |  1
 1  1 |  2  1 |  0
```

Das erste Neuron bildet $x_1 + x_2$, das zweite zieht davon 1 ab und lässt nur Überschuss durch. Die Ausgabe ist $h_1 - 2h_2$. Bei den Eingaben (1, 1) heben sich die Anteile auf. Eine versteckte Schicht hat hier eine **neue Eigenschaft** der Eingabe berechnet („beide sind an“), mit der die nächste Schicht nur noch eine lineare Entscheidung treffen muss. Der Satz von Hornik, Stinchcombe und White macht daraus eine allgemeine Aussage: Ein Netz mit einer ausreichend breiten versteckten Schicht kann jede stetige Funktion auf einem beschränkten Bereich beliebig genau annähern [16]. Bewiesen wurde das zunächst für sigmoidartige Aktivierungen, später auch für ReLU und viele andere [17]. Er sagt aber nichts darüber, wie breit „ausreichend“ ist und ob Training die passenden Gewichte findet. Das ist die eigentliche Schwierigkeit, und ihr widmet sich der Rest des Kapitels.

3.3 Softmax und Cross-Entropy: aus Zahlen werden Wahrscheinlichkeiten

Bei einer Klassifikation mit mehreren Klassen soll die Ausgabeschicht für jede Klasse eine Wahrscheinlichkeit liefern, alle zusammen ergeben 1. Die letzte Schicht rechnet zunächst ganz normal $z = hW + b$ und liefert pro Klasse einen beliebigen Zahlenwert, den **Logit**. Die **Softmax**-Funktion macht daraus Wahrscheinlichkeiten:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Die Exponentialfunktion macht jeden Wert positiv und vergrößert Unterschiede; die Division durch die Summe sorgt dafür, dass die Werte 1 ergeben. Für die Logits $(2{,}0; 1{,}0; 0{,}1)$ erhält man $e^2 = 7{,}39$, $e^1 = 2{,}72$ und $e^{0{,}1} = 1{,}11$, zusammen 11,21, und damit die Wahrscheinlichkeiten $(0{,}659; 0{,}242; 0{,}099)$. In der Implementierung zieht man vorher den größten Logit von allen ab. Das ändert das Ergebnis nicht (Zähler und Nenner kürzen sich), verhindert aber, dass e^z bei großen Logits überläuft.

Wie gut ist so eine Vorhersage? Der Loss muss klein sein, wenn die richtige Klasse eine hohe Wahrscheinlichkeit bekommt, und sehr groß, wenn das Modell sich sicher und falsch ist. Die **Cross-Entropy** leistet das:

$$L = -\ln p_y.$$

Dabei ist y die richtige Klasse. Hat sie die Wahrscheinlichkeit 0,659, ist $L = 0{,}417$; bei 0,01 sind es 4,6; bei 0,0001 schon 9,2. Wer sich sicher irrt, wird also hart bestraft. Eine Zahl dient als Bezugspunkt: Ein Modell, das bei drei gleich häufigen Klassen immer $\frac{1}{3}$ rät, hat $L = \ln 3 = 1{,}099$. Liegt der Loss zu Beginn des Trainings in dieser Größenordnung, ist das ein gutes Zeichen für eine korrekte Implementierung. Für das Training zählt eine Eigenschaft, die man leicht herleiten kann und die den Code kurz macht: Der Gradient der Cross-Entropy nach den Logits ist

$$\frac{\partial L}{\partial z_k} = p_k - [k = y].$$

Von jeder vorhergesagten Wahrscheinlichkeit zieht man also eine 1 für die richtige Klasse ab. Im Beispiel mit der richtigen Klasse 0 ist der Gradient $(-0{,}341; 0{,}242; 0{,}099)$: Der Logit der richtigen Klasse soll steigen (ein negativer Gradient bedeutet, dass der Loss mit größerem z sinkt), die der anderen sollen sinken.

3.4 Lernen mit Backpropagation

Die Kettenregel an einem Neuron

Training heißt, für jeden Parameter zu wissen, in welche Richtung und wie stark der Loss sich ändert, wenn man ihn ein wenig verschiebt. Bei einem Netz mit mehreren tausend Parametern kann man das nicht

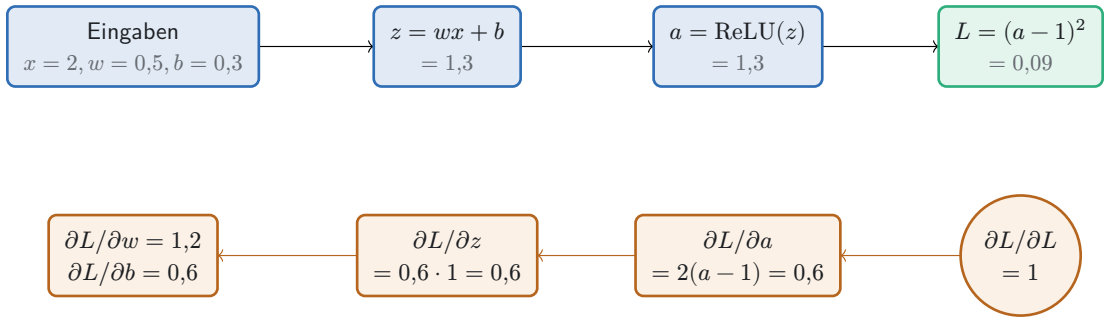


Abbildung 19: Oben der Vorwärtsdurchlauf, unten der Rückwärtsdurchlauf (orange). Jeder Rückwärtsschritt multipliziert den ankommenden Gradienten mit der lokalen Ableitung des Knotens. Zahlen aus `01_neuron.py`.

ausprobieren; man leitet es mit der Kettenregel her. Sie sagt: Hängt L von a ab und a von z und z von w, dann multipliziert man die einzelnen Änderungsraten entlang der Kette: $\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$.

Wir rechnen das an einem einzigen ReLU-Neuron durch, mit Eingabe $x = 2$, Gewicht $w = 0{,}5$, Bias $b = 0{,}3$ und dem gewünschten Wert 1. Als Loss nehmen wir hier den quadratischen Fehler $(a-1)^2$. Der Vorwärtsdurchlauf berechnet $z = 1{,}3$, $a = 1{,}3$ und $L = 0{,}09$. Dann läuft man rückwärts durch dieselbe Kette (Abbildung 19): L ändert sich mit a um $2(a-1) = 0{,}6$; die ReLU gibt bei $z > 0$ den Gradienten unverändert weiter; z hängt mit dem Faktor $x = 2$ von w ab, also ist $\partial L/\partial w = 0{,}6 \cdot 2 = 1{,}2$, und der Bias geht mit dem Faktor 1 ein: $\partial L/\partial b = 0{,}6$.

Der Gradient wird dann für einen Schritt des **Gradientenabstiegs** benutzt: Jeden Parameter verschiebt man gegen die Richtung seines Gradienten, skaliert mit der **Lernrate** η:

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}.$$

Mit $\eta = 0{,}1$ wird aus $w = 0{,}5$ der Wert 0,38 und aus $b = 0{,}3$ der Wert 0,24; die Ausgabe ist dann $0{,}38 \cdot 2 + 0{,}24 = 1{,}0$ und der Loss null. Ein einzelner Schritt trifft das Ziel selten so genau, es ist ein Beispiel mit passenden Zahlen. `01_neuron.py` bestätigt die Gradienten außerdem unabhängig: Es verschiebt w und b um $\pm 10^{-6}$ und bildet den Differenzenquotienten, was ebenfalls $1{,}2$ und $0{,}6$ ergibt.

Backpropagation für das ganze Netz

Ein Netz ist dieselbe Kette, nur länger und mit Matrizen statt einzelnen Zahlen. **Backpropagation** ist der Name für die Reihenfolge, in der man die Kettenregel auswertet; als Lernverfahren für mehrschichtige Netze bekannt gemacht haben sie Rumelhart, Hinton und Williams [18]. Man beginnt am Loss und arbeitet sich Schicht für Schicht nach vorn, und jede Schicht benutzt den Gradienten, den die Schicht hinter ihr geliefert hat. Der Gradient, der an der Schicht l ankommt, sei δ_l (eine Zeile pro Beispiel, eine Spalte pro Neuron). Für Softmax mit Cross-Entropy in der letzten Schicht gilt $\delta_L = (P - Y)/B$ mit der Matrix P der Vorhersagen und der One-Hot-Matrix Y der richtigen Antworten. Für jede Schicht rückwärts rechnet man

$$\frac{\partial L}{\partial W_l} = H_{l-1}^{\top}\delta_l, \quad \frac{\partial L}{\partial b_l} = \sum_{\text{Beispiele}} \delta_l, \quad \delta_{l-1} = \left(\delta_l W_l^{\top}\right) \odot \mathbb{1}[H_{l-1} > 0].$$

Die erste Gleichung heißt in Worten: Das Gewicht zwischen Neuron i und Neuron j ändert sich umso stärker, je aktiver i war und je größer der Fehler ist, der bei j ankommt. Die dritte reicht den Fehler durch die Gewichte nach hinten und blendet jene Neuronen aus, die bei diesem Beispiel „aus“ waren (die ReLU hat dort den Gradienten null). Der Rückwärtsdurchlauf braucht die Zwischenwerte H des Vorwärtsdurchlaufs, deshalb merkt sich dieser sie. Ein Rückwärtsdurchlauf kostet etwa doppelt so viel Rechenzeit wie ein Vorwärtsdurchlauf (je Schicht zwei Matrixprodukte statt einem), dazu Speicher für alle Zwischenwerte; das wird bei der Batch-Größe und bei der Hardware wieder wichtig.

> **TIPP**
> Mit Backpropagation muss niemand von Hand rechnen. Frameworks wie PyTorch oder TensorFlow zeichnen beim Vorwärtsdurchlauf einen Graphen der Operationen auf und leiten ihn automatisch rückwärts ab (automatic differentiation); Sie schreiben nur den Vorwärtsteil. Verstehen sollten Sie das Prinzip trotzdem: Es erklärt, warum Training so viel Speicher braucht und warum nicht differenzierbare Schritte, etwa ein hartes `if` auf einem Zwischenwert, den Gradientenfluss unterbrechen.

3.5 Ein Netz von Grund auf: die Spirale

Jetzt setzen wir alles zusammen. Die Aufgabe ist ein klassischer Test für kleine Netze: 300 Punkte in der Ebene, die auf drei ineinander verschlungenen Spiralarmen liegen und zu drei Klassen gehören. Keine Gerade trennt sie, das Netz muss eine gekrümmte Grenze lernen. Die Daten erzeugt `daten.py` mit festem Seed, das Netz (`mlp.py`, nur numpy) hat die Form 2–32–32–3 mit ReLU und Softmax. Der erste Ausschnitt zeigt den Vorwärtsdurchlauf, den Loss und die Backpropagation, also die eben besprochenen Gleichungen, in knapp 40 Zeilen:

```
def forward(self, X, dropout=0.0, rng=None):
    """Gibt Wahrscheinlichkeiten, Aktivierungen und Dropout-Masken zurück."""
    a, masken = [X], []
    for i, (W, b) in enumerate(zip(self.W, self.b)):
        z = a[-1] @ W + b
        if i == len(self.W) - 1:
            return softmax(z), a, masken
        h = np.maximum(z, 0)  # ReLU
        if dropout > 0:  # "inverted dropout": Überlebende hochskalieren
            m = (rng.random(h.shape) >= dropout) / (1 - dropout)
            h, masken = h * m, masken + [m]
        a.append(h)

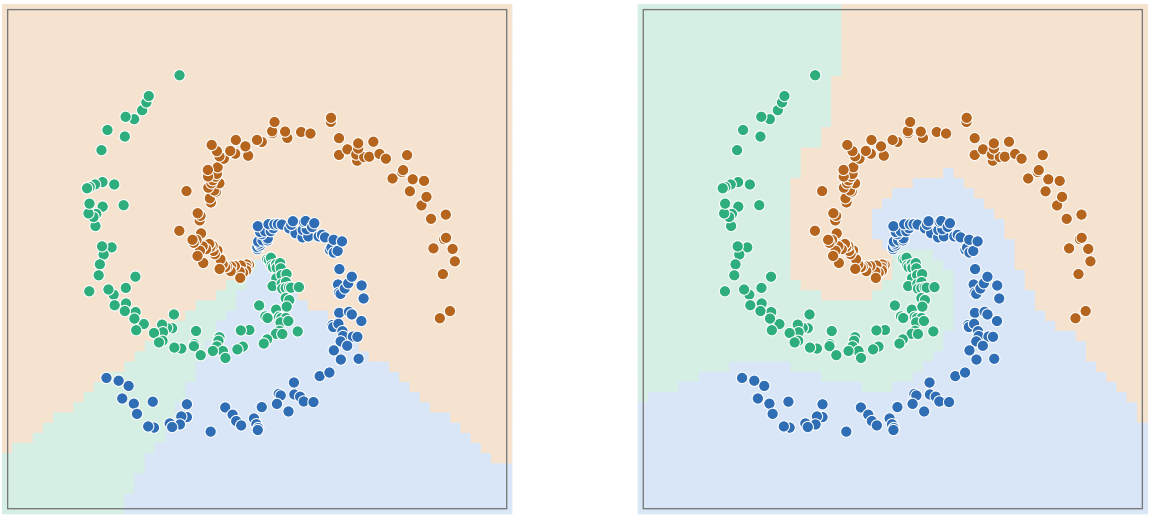
def loss(self, p, y, l2=0.0):
    """Mittlere Cross-Entropy plus L2-Strafterm l2/2 * Summe(W^2)."""
    ce = -np.log(p[np.arange(len(y)), y] + 1e-12).mean()
    return ce + 0.5 * l2 * sum((W ** 2).sum() for W in self.W)

def gradienten(self, X, y, l2=0.0, dropout=0.0, rng=None):
    """Backpropagation: Kettenregel Schicht für Schicht von hinten nach vorn."""
    p, a, masken = self.forward(X, dropout, rng)
    dz = p.copy()
    dz[np.arange(len(y)), y] -= 1  # Softmax + Cross-Entropy: p - one_hot(y)
    dz /= len(y)
    gW, gb = [None] * len(self.W), [None] * len(self.W)
    for i in reversed(range(len(self.W))):
        gW[i] = a[i].T @ dz + l2 * self.W[i]
        gb[i] = dz.sum(axis=0)
        if i > 0:
            da = dz @ self.W[i].T  # Gradient an der Ausgabe der Schicht davor
            if dropout > 0:
                da = da * masken[i - 1]
            dz = da * (a[i] > 0)  # ReLU: nur dort, wo das Neuron aktiv war
    return gW, gb, self.loss(p, y, l2)
```

Der zweite Ausschnitt ist der Kern der Trainingsschleife aus `trainiere` (der Rest sammelt Loss und Genauigkeit je Epoche). Pro Epoche mischt sie die Beispiele neu, zerlegt sie in Mini-Batches, berechnet für jeden Batch die Gradienten und macht einen Schritt:

```
for _ in range(epochen):
    reihenfolge = rng.permutation(len(X))  # neue Reihenfolge je Epoche
    for s in range(0, len(X), batch):
        i = reihenfolge[s:s + batch]
        gW, gb, _ = netz.gradienten(X[i], y[i], l2, dropout, rng)
        netz.schritt(gW, gb, lr, momentum)
```

Bevor wir trainieren, prüft `02_spirale.py` die Backpropagation: Für 15 zufällig gewählte Gewichte vergleicht es den berechneten Gradienten mit dem numerischen Differenzenquotienten. Dieser Gradient Check ist der Unit-Test unter den Trainingsverfahren und sollte bei jeder selbst geschriebenen Backpropagation laufen. Danach trainiert das Skript 100 Epochen mit Mini-Batches der Größe 32 und Lernrate 0,5:



vor dem Training: Genauigkeit 47 % nach 100 Epochen: Genauigkeit 100 %

Abbildung 20: Entscheidungsgrenze des Netzes vor und nach dem Training. Die Flächen zeigen, welche Klasse das Netz an jeder Stelle der Ebene vorhersagt, die Punkte sind die 300 Trainingsbeispiele. Vor dem Training liegen die Grenzen willkürlich, danach folgen sie den Spiralarmen.

Terminal

```
$ uv run --with numpy python 02_spirale.py
Beispiele: 300 | Parameter: 1251
Vor dem Training: Loss 0.985  (Raten wäre ln 3 = 1.099), Genauigkeit 0.473
Gradienten-Check: größte relative Abweichung über 15 Gewichte = 2.2e-08
Epoche   1: Loss 0.712  Genauigkeit 0.603
Epoche  10: Loss 0.512  Genauigkeit 0.753
Epoche  50: Loss 0.015  Genauigkeit 0.997
Epoche 100: Loss 0.003  Genauigkeit 1.000
Testdaten: Genauigkeit 0.997
```

Der Anfangsloss von 0,985 liegt knapp unter dem Raten-Wert $\ln 3 = 1{,}099$, wie erwartet. Die Abweichung des Gradient Checks von $2 \cdot 10^{-8}$ liegt im Rauschen des Differenzenquotienten; die Implementierung ist richtig. Nach einer Epoche sind es 60 % Genauigkeit, nach zehn 75 %. Der Loss verläuft zunächst unruhig (zwischen 0,24 und 0,87; die Lernrate 0,5 ist für dieses Netz schon recht groß), fällt dann in Epoche 12 auf 0,12 und bis Epoche 50 auf 0,015 (Abbildung 21). Auf 300 neu erzeugten Testpunkten erreicht das Netz 99,7 %. Die Testpunkte haben dieselben Radien und Soll-Winkel wie die Trainingspunkte, nur mit neu gezogenem Rauschen; das ist weniger streng als völlig neue Positionen, belegt aber, dass das Netz die Kurven gelernt hat und nicht einzelne Trainingspunkte auswendig.

3.6 Mini-Batches, Epochen und Lernrate

Die Trainingsschleife hat drei Stellschrauben, die jedes Trainingsprotokoll prägen. Die erste ist die **Batch-Größe**. Man könnte den Gradienten über alle 300 Beispiele bilden und pro Epoche genau einen Schritt machen, oder nach jedem einzelnen Beispiel einen Schritt. Der Mini-Batch liegt dazwischen: Der Gradient

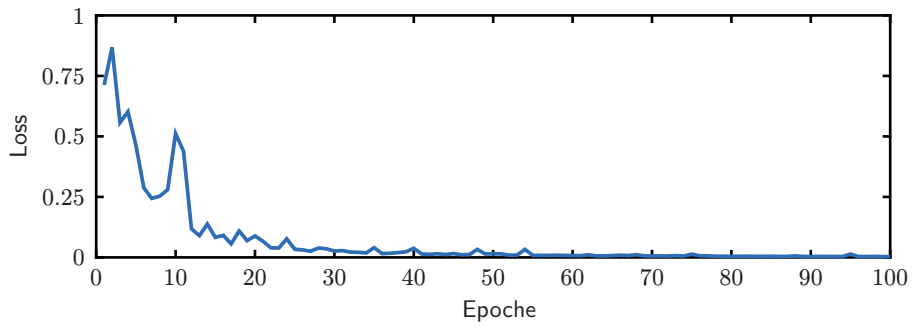


Abbildung 21: Loss auf den Trainingsdaten je Epoche (Werte aus `02_spirale.py`). Nach einer unruhigen Anfangsphase fällt er ab Epoche 12 auf unter 0,15.

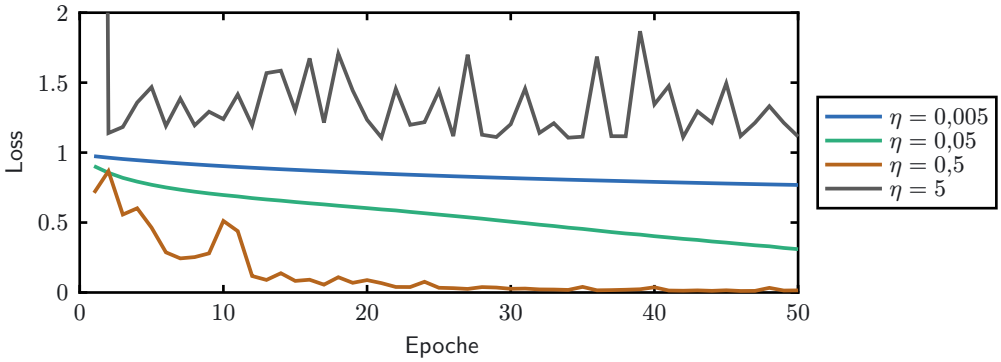


Abbildung 22: Trainings-Loss bei vier Lernraten. Die Kurve für $\eta = 5$ beginnt bei 18,4 (Epoche 1, außerhalb des Bildes) und pendelt danach zwischen 1,1 und 1,9.

über 32 zufällig gezogene Beispiele ist eine verrauschte, aber im Mittel richtige Schätzung des Gradienten über alle Daten. Man bekommt viele Schritte pro Epoche und kann die Beispiele eines Batches als eine Matrix auf einmal durchrechnen. Die zweite ist die **Epoche**, ein Durchlauf durch alle Trainingsdaten; die Zahl der Epochen bestimmt, wie lange trainiert wird. Die dritte ist die Lernrate η.

`03_lernrate.py` ändert nur eine Größe pro Lauf, alles andere (Daten, Anfangsgewichte, Reihenfolge der Beispiele) bleibt gleich. Die obere Hälfte der Ausgabe variiert die Lernrate bei Batch-Größe 32, die untere die Batch-Größe:

Terminal

```
$ uv run --with numpy python 03_lernrate.py
Lernrate | Loss nach Epoche 1 / 10 / 50 | Genauigkeit | aktive Neuronen
   0.005 |   0.974   0.903   0.769 | 0.553 | 64 von 64
    0.05 |   0.903   0.697   0.310 | 0.953 | 63 von 64
     0.5 |   0.712   0.512   0.015 | 0.997 | 61 von 64
     5.0 |  18.421   1.239   1.114 | 0.333 | 0 von 64

Batch-Größe, 50 Epochen
Batch Lernrate | Updates je Epoche | Loss nach Epoche 10 / 50
    1     0.5 |               300 | 1.153 / 1.124
    1    0.05 |               300 | 0.028 / 0.002
   32     0.5 |                10 | 0.512 / 0.015
  300     0.5 |                 1 | 0.696 / 0.314
```

Jede der vier Kurven zeigt ein typisches Bild. Bei $\eta = 0{,}005$ sinkt der Loss gleichmäßig, aber so langsam, dass er nach 50 Epochen erst bei 0,77 steht (55 % Genauigkeit): zu klein. Bei 0,05 ist das Netz nach 50 Epochen bei 95 %, aber noch nicht fertig; bei 0,5 erreicht es 99,7 %. Mit $\eta = 5$ geht der erste Schritt weit über das Ziel hinaus: Der Loss springt auf 18,4 und pendelt danach zwischen 1,1 und 1,9, ohne zu sinken. Die letzte Spalte der Ausgabe verrät den Grund. Von den 64 versteckten Neuronen ist kein einziges für irgendein Trainingsbeispiel aktiv. Der große Schritt hat die Gewichte so verschoben, dass jedes z negativ ist; die ReLU liefert dann überall null, der Gradient ebenfalls, und es gibt keinen Weg zurück. Man spricht von **toten Neuronen**. Am Ende des Laufs sagt das Netz für jede Eingabe dieselben Wahrscheinlichkeiten voraus, und der Loss liegt mit 1,11 bei etwa $\ln 3$.

Das Trainingsprotokoll liefert also Diagnosen, die man ohne weitere Werkzeuge lesen kann: Ein Loss, der springt, steigt oder um den Raten-Wert pendelt, deutet auf eine zu große Lernrate; ein Loss, der kaum sinkt, auf eine zu kleine oder auf zu wenige Epochen. Die Lernrate gilt in der Praxis als der wichtigste Hyperparameter [14], und Pattern 15 behandelt, wie man sie systematisch sucht. Moderne Optimierer wie Adam passen die Schrittweite für jeden Parameter anhand der bisherigen Gradienten an [19]. Das nimmt dem Anwender Arbeit ab, ersetzt die Lernrate aber nicht, und es hat eine Folge für Checkpoints, auf die wir noch kommen.

Nun zur Batch-Größe, der unteren Hälfte der Ausgabe. Man sollte Läufe nach der Zahl der Schritte vergleichen, nicht nach Epochen. Mit Batch 300 gibt es einen Schritt pro Epoche, nach 50 Epochen also erst 50 Schritte, und der Loss steht deshalb erst bei 0,31. Mit Batch 1 sind es 300 Schritte je Epoche, und bei Lernrate 0,05 steht der Loss schon nach zehn Epochen bei 0,03; bei Lernrate 0,5 funktioniert Batch 1

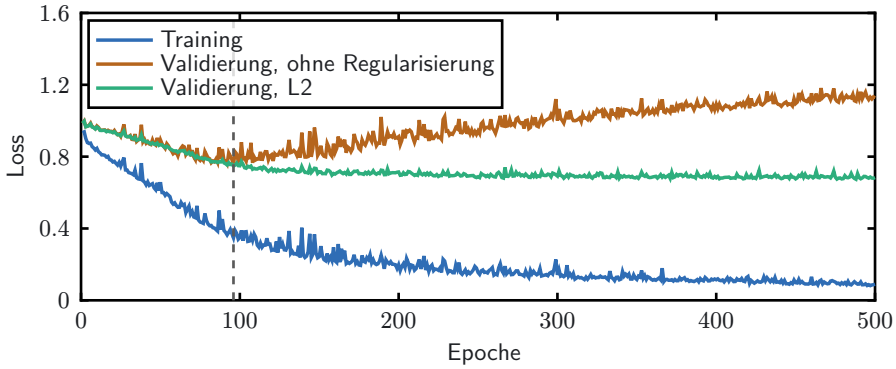


Abbildung 23: Overfitting im ersten der fünf Durchläufe. Der Trainings-Loss sinkt weiter, der Validierungs-Loss erreicht in Epoche 96 sein Minimum (gestrichelt, 0,754) und steigt dann auf 1,13. Mit L2-Strafterm (grün) bleibt er deutlich niedriger.

dagegen nicht (Loss 1,12). Der Gradient eines einzelnen Beispiels ist viel verrauschter, und derselbe Schritt ist dann offenbar zu groß. Batch-Größe und Lernrate gehören also zusammen: Größere Batches vertragen größere Lernraten. Trotzdem ist Batch 1 in der Praxis keine gute Wahl, denn 300 winzige, nacheinander ausgeführte Schritte nutzen moderne Hardware schlecht aus. Warum, klärt der Abschnitt über Hardware.

3.7 Regularisierung: Overfitting dämpfen

Overfitting kennen Sie aus dem Grundlagenkapitel: Ein Modell, das die Trainingsdaten auswendig lernt, einschließlich ihres Rauschens, und deshalb auf neuen Daten schlechter ist. Neuronale Netze sind dafür besonders anfällig, weil sie sehr viele Parameter haben. Um das zu sehen, machen wir die Aufgabe schwerer und das Netz größer: Dieselbe Spirale, aber mit stark verrauschten Punkten (Streuung 0,8), nur 60 Trainingsbeispiele und ein Netz mit 2–128–128–3 Neuronen, also über 17 000 Parameter. `04_regularisierung.py` trainiert 500 Epochen und vergleicht vier Varianten auf getrennten Validierungs- und Testdaten mit je 300 Beispielen. Alles wird mit fünf frisch gezogenen Datensätzen wiederholt.

Die Kurven in Abbildung 23 sind das Lehrbuchbild: Der Trainings-Loss fällt auf 0,09, der Validierungs-Loss dreht ab Epoche 96 nach oben. Das Netz passt sich zunehmend dem Rauschen der 60 Beispiele an. Drei Gegenmittel sind verbreitet:

- **Early Stopping** hält das Training dort an, wo der Validierungs-Loss am niedrigsten ist [20]. Praktisch läuft das Training weiter, bis sich der Validierungs-Loss einige Epochen lang nicht verbessert hat, und man behält den besten Stand; genau das macht der Checkpoint `best` in Pattern 12.
- **L2-Regularisierung** (weight decay) addiert zum Loss den Strafterm $\lambda/2 \cdot \sum w^2$. Im Gradienten steckt dann zusätzlich λw, und jeder Schritt zieht jedes Gewicht ein kleines Stück zum Nullpunkt: $w \leftarrow (1 - \eta\lambda)w - \eta g$. Kleine Gewichte ergeben glattere Funktionen, die dem Rauschen weniger folgen können.
- **Dropout** schaltet beim Training bei jedem Schritt zufällig einen Teil der Neuronen einer Schicht ab, hier 30 %, und skaliert die übrigen hoch, damit der Erwartungswert gleich bleibt. Kein Neuron kann sich darauf verlassen, dass ein bestimmtes anderes mitarbeitet, und das Netz lernt redundantere Merkmale [21]. Bei der Vorhersage ist Dropout aus. Wer Chaos Engineering kennt, erkennt die Idee wieder: Man lässt im Training absichtlich Teile ausfallen, damit das Ganze nicht von einem einzelnen abhängt.

Tabelle 4: Ergebnis der vier Varianten, Mittel und Standardabweichung über fünf Durchläufe mit neu gezogenen Daten (`04_regularisierung.py`). Der Trainings-Loss bei Early Stopping gilt für den Stopp-Zeitpunkt.

Variante	Trainings-Loss	Test-Loss	Test-Genauigkeit
ohne	0,127	1,236 ± 0,156	0,732 ± 0,022
Early Stopping	0,417	0,796 ± 0,084	0,693 ± 0,096
L2 (0.01)	0,391	0,717 ± 0,056	0,733 ± 0,015
Dropout (0.3)	0,221	0,964 ± 0,157	0,738 ± 0,016

Tabelle 4 zeigt zwei Dinge. Erstens senken alle drei Mittel den Test-Loss deutlich, am stärksten L2 (1,24 auf 0,72) und Early Stopping (auf 0,80), weniger Dropout (auf 0,96). Zweitens ändert sich die Genauigkeit kaum: Sie liegt bei 73 bis 74 %, nur beim Early Stopping mit 69 % darunter (dazu gleich mehr). Das Netz ohne Regularisierung sortiert nicht schlechter, es ist nur bei seinen Fehlern viel zu überzeugt, und die

Cross-Entropy bestraft das, die Genauigkeit nicht. Overfitting zeigt sich im Loss oft früher und deutlicher als in der Genauigkeit; das hängt mit dem Befund zusammen, dass moderne Netze zu selbstsicheren Wahrscheinlichkeiten neigen [22]. Wer Wahrscheinlichkeiten weiterverwendet, etwa für Schwellenwerte, sollte deshalb auf den Loss schauen. Der Ausreißer beim Early Stopping (Standardabweichung 0,096 in der Genauigkeit) kommt von einem der fünf Durchläufe, in dem das Validierungs-Minimum schon in Epoche 9 lag: Die Stopp-Epochen betrugen 96, 130, 9, 144 und 120. Ein einziges Minimum ist ein empfindliches Kriterium, und mit nur fünf Durchläufen und einem Rauschpegel sollte man L2 und Early Stopping nicht gegeneinander ausspielen; ihre Unterschiede liegen innerhalb einer Standardabweichung.

Regularisierung ist kein Dogma: Pattern 11 beschreibt den Fall, in dem man bewusst überanpasst, weil die Daten das gesamte Problem abdecken und es nichts zu verallgemeinern gibt. Dass große Netze trotz perfekter Anpassung an Trainingsdaten oft gut verallgemeinern, ist ein eigenes Forschungsthema [23], das wir hier offenlassen.

3.8 Embeddings sind Nachschlagetabellen

Bisher bestand die Eingabe aus Zahlen. Wörter, Produkt-IDs und Nutzerkennungen sind keine; man muss sie erst in Zahlen übersetzen. Der naheliegende Weg ist die One-Hot-Kodierung: Bei einem Vokabular aus V Wörtern bekommt jedes Wort einen Vektor der Länge V mit einer 1 an seiner eigenen Position und sonst nur Nullen. Ein Embedding ersetzt diesen langen, dünn besetzten Vektor durch einen kurzen, dichten, dessen Werte das Netz selbst lernt. Pattern 2 zeigt, wann das nützt. Hier geht es um die Mechanik, und die ist schlicht.

Eine Embedding-Schicht ist eine Gewichtsmatrix E mit einer Zeile pro Wort und d Spalten. Multipliziert man den One-Hot-Vektor eines Worts mit E, passiert Folgendes: Die Eins wählt aus der Summe über alle Zeilen genau eine aus, alle anderen werden mit null multipliziert. Das Ergebnis ist die Zeile des Worts, nichts weiter (Abbildung 24).

Wort	One-Hot	d_1	d_2	d_3
`drucker`	0	0,9	0,1	−0,3
`scanner`	0	0,8	0,2	−0,4
`passwort`	0	−0,5	0,9	0,2
`login`	**1**	−0,4	0,8	0,3
`vpn`	0	0,1	−0,7	0,9
Summe (one-hot $\times$ E)		**−0,4**	**0,8**	**0,3**

Abbildung 24: Die Rechnung one-hot $\times$ E für das Wort `login`. Jede Zeile von E wird mit der Zahl in der Spalte „One-Hot" multipliziert, anschließend addiert man alle Zeilen. Übrig bleibt die Zeile von `login`, als wäre sie nachgeschlagen worden.

Die Ausgabe von `05_embedding_lookup.py` bestätigt, dass `one_hot @ E` und `E[idx]` dasselbe Ergebnis liefern, und dass der Gradient der Tabelle nur in einer Zeile von null verschieden ist:

Terminal

```
$ uv run --with numpy python 05_embedding_lookup.py
One-Hot für 'login': [0. 0. 0. 1. 0.]
one_hot @ E        : [-0.4  0.8  0.3]
E[idx]             : [-0.4  0.8  0.3]

Gradient der Tabelle:
 [[ 0.   0.   0. ]
 [ 0.   0.   0. ]
 [ 0.   0.   0. ]
 [ 0.5 -1.   2. ]
 [ 0.   0.   0. ]]

Tabelle 50.000 x 300: 15.000.000 Gewichte = 60 MB in float32
```

Für das Training ist der zweite Teil der Ausgabe der wichtigere. Kommt von der nächsten Schicht ein Gradient $(0{,}5; -1{,}0; 2{,}0)$ an, dann erhält nur die Zeile von `login` diesen Gradienten, alle anderen Zeilen

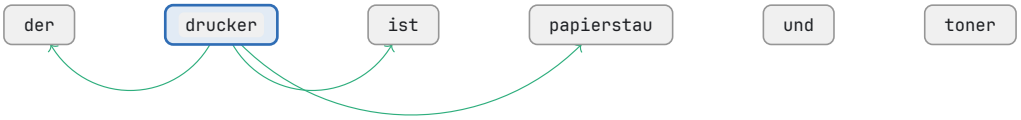
bekommen null. Pro Beispiel wird also nur ein winziger Teil der Tabelle verändert. Daraus folgen drei praktische Dinge:

- **Embedding-Schichten werden als Nachschlagen implementiert**, nicht als Matrixmultiplikation. Bei 50 000 Wörtern und 300 Dimensionen wären es 15 Millionen Multiplikationen, davon 14 999 700 mit null, gegenüber dem Kopieren von 300 Werten.
- **Selten gesehene Wörter bekommen selten Updates** und deshalb oft schlechte Vektoren. Das ist die Kehrseite des sparsamen Lernens.
- **Die Tabelle ist der größte Teil des Modells**, wenn das Vokabular groß ist: 50 000 mal 300 sind 15 Millionen Gewichte oder 60 MB in float32, bei einer Million Wörtern 1,2 GB. Pattern 1 (Hashed Feature) begrenzt V genau aus diesem Grund.

Für Software Engineers ist das eine vertraute Konstruktion: Ein Array-Zugriff `E[i]` oder ein Dictionary-Lookup, nur dass die Werte nicht fest sind, sondern Parameter, durch die ein Gradient fließen kann. Alles, was ein Embedding „weiß", steht in diesen Zahlen, und wie sie zustande kommen, ist im Grunde einfach: Sie werden zufällig initialisiert und dann wie jedes andere Gewicht per Backpropagation verändert, sodass sie der Aufgabe des Netzes dienen. Bei einer Klassifikation heißt das: Wörter, die zu ähnlichen Vorhersagen führen sollen, rücken zusammen. Die Tabelle enthält also keine eingebaute Bedeutung, sondern eine Verdichtung dessen, was die Trainingsaufgabe verlangt. Es gibt aber eine Aufgabe, die so allgemein ist, dass ihre Embeddings für viele andere Aufgaben taugen. Das ist die Vorhersage von Nachbarwörtern.

3.9 word2vec: Bedeutung aus Nachbarschaft

Die Idee hinter word2vec ist, dass Wörter, die in ähnlichen Umgebungen vorkommen, ähnliche Bedeutung haben. Mikolov und Kollegen formulierten daraus 2013 zwei einfache Netze: CBOW sagt ein Wort aus seiner Umgebung voraus, **Skip-Gram** umgekehrt die Umgebung aus einem Wort [7]. Wir betrachten Skip-Gram. Das Netz bekommt ein Wort (das Mittelwort) und soll die Wörter in einem Fenster davor und danach erraten. Dazu werden aus dem Text Trainingspaare gebildet (Abbildung 25).



Paare für das Mittelwort `drucker` (Fenster ± 2): (`drucker`, `der`), (`drucker`, `ist`), (`drucker`, `papierstau`). `toner` liegt mit Abstand 4 außerhalb.

Abbildung 25: Aus einem Satz entstehen Paare aus Mittelwort und Nachbarwort. Ein Satz mit sechs Wörtern liefert bei Fenster ± 2 insgesamt 18 Paare.

Die Architektur ist die des vorigen Abschnitts plus eine Ausgabeschicht (Abbildung 26). Das Mittelwort wird per Lookup in der Tabelle E_{ein} (Größe $V \times d$) in einen Vektor h der Länge d übersetzt. Eine zweite Matrix E_{aus} ($d \times V$) macht daraus einen Logit pro Wort des Vokabulars, und ein Softmax über alle V Wörter liefert die Wahrscheinlichkeit, dass das jeweilige Wort in der Nähe steht. Der Loss ist die Cross-Entropy gegenüber dem tatsächlichen Nachbarwort. Es gibt keine versteckte Schicht mit Aktivierung, die ganze Rechnung ist ein Lookup und eine Matrixmultiplikation. Nach dem Training verwirft man die Aufgabe, denn die Vorhersage von Nachbarwörtern ist nicht das Ziel. Behalten wird die Tabelle E_{ein}: Ihre Zeilen sind die Wortvektoren.

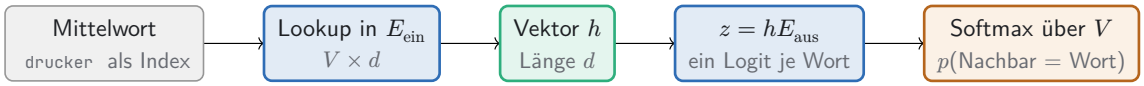


Abbildung 26: Skip-Gram als Netz. Die Vorhersageaufgabe dient nur dazu, die Tabelle E_{ein} zu trainieren; sie wird nach dem Training behalten und ist das eigentliche Ergebnis.

Der Haken ist der Softmax: Er rechnet über alle V Wörter, bei einem Vokabular von einer Million also einer Million Exponentialfunktionen pro Trainingspaar. Mikolov et al. umgehen das auf zwei Arten: mit einem hierarchischen Softmax und mit **Negative Sampling**, bei dem das Modell pro Paar nur den echten Nachbarn von einigen zufällig gezogenen Wörtern unterscheiden muss [24]. Unser Beispiel verwendet den vollen Softmax, weil das Vokabular mit 27 Wörtern winzig ist und der Mechanismus so am klarsten bleibt. Der Korpus besteht aus 600 erfundenen Helpdesk-Sätzen der Form „der <Gegenstand> ist <Meldung>

und <Meldung>“ aus drei Themen: Hardware (`drucker`, `scanner`, `papierstau`, `toner` …), Zugang (`passwort`, `login`, `gesperrt` …) und Netz (`vpn`, `router`, `timeout` …). Jeder Satz hat genau einen Gegenstand. Das Skript bildet die Paare mit einer Schleife über alle Sätze und Fensterpositionen, was zu 10 800 Trainingspaaren führt; der Kern des Trainings ist:

```
W_ein = rng.normal(0, 0.1, (V, D))  # die Wortvektoren: eine Zeile je Wort
W_aus = rng.normal(0, 0.1, (D, V))  # Gewichte der Ausgabeschicht
lr, batch = 0.5, 128
for epoche in range(30):
    rng.shuffle(paare)
    gesamt = 0.0
    for s in range(0, len(paare), batch):
        c, o = paare[s:s + batch, 0], paare[s:s + batch, 1]
        h = W_ein[c]  # Lookup wie bei einer Embedding-Schicht
        p = softmax(h @ W_aus)  # P(Wort ist Nachbar)
        gesamt += -np.log(p[np.arange(len(o)), o]).sum()
        dz = p.copy()
        dz[np.arange(len(o)), o] -= 1
        dz /= len(o)
        np.add.at(W_ein, c, -lr * (dz @ W_aus.T))  # add.at summiert Doppelte
        W_aus -= lr * (h.T @ dz)
```

Die Zeile `np.add.at(W_ein, c, …)` ist der Lookup rückwärts: Der Gradient wird nur auf die Zeilen der Mittelwörter im Batch geschrieben, und `add.at` summiert ihn richtig auf, wenn dasselbe Wort mehrmals im Batch steht. Das Ergebnis:

Terminal

```
$ uv run --with numpy python 06_skipgram.py
600 Sätze, Vokabular 27 Wörter, 10800 Trainingspaare, Vektorlänge 8
'drucker' und 'scanner' gemeinsam in einem Satz: 0 mal
Epoche  1: Loss je Paar 3.134  (Raten: ln 27 = 3.296)
Epoche  5: Loss je Paar 2.384  (Raten: ln 27 = 3.296)
Epoche 30: Loss je Paar 2.118  (Raten: ln 27 = 3.296)

Kosinus(drucker, scanner) = +0.98
Kosinus(passwort, login) = +1.00
Kosinus(vpn, router) = +0.99
Kosinus(drucker, passwort) = +0.40
Kosinus(drucker, vpn) = +0.46
Kosinus(toner, papierstau) = +0.98
Kosinus(toner, gesperrt) = +0.21
Nächste Wörter zu 'drucker': tastatur (+1.00), scanner (+0.98), monitor (+0.98)
```

Die Kosinus-Ähnlichkeit zweier Vektoren, der Kosinus des Winkels zwischen ihnen, ist 1, wenn sie in dieselbe Richtung zeigen, und 0, wenn sie senkrecht stehen. `drucker` und `scanner` erreichen 0,98, obwohl sie in keinem einzigen Satz gemeinsam vorkommen. Das Netz hat sie nicht aus gemeinsamem Auftreten ähnlich gemacht, sondern weil beide dieselben Nachbarn vorhersagen müssen (`papierstau`, `toner`, `ist` …), und das erzwingt ähnliche Vektoren. Dasselbe gilt für `passwort` und `login` (1,00) oder `vpn` und `router` (0,99). Wörter verschiedener Themen liegen weiter auseinander (0,40 und 0,46 für zwei Gegenstände, 0,21 für die Meldungen `toner` und `gesperrt`), aber nicht bei null, weil alle Sätze dieselben Funktionswörter `der`, `ist` und `und` enthalten und die Vektoren daher einen gemeinsamen Anteil haben. In einer zweidimensionalen Projektion (Abbildung 27) erkennt man die sechs Gruppen: je Thema die Gegenstände und die Meldungen.

Im Kleinen zeigt das, was Mikolov et al. an Milliarden von Wörtern beobachteten: Die Vektoren fassen Ähnlichkeit in Zahlen, und sie tragen Beziehungen, die man mit Vektorrechnung herausholen kann. Ihr bekanntestes Beispiel, das auch die Originalarbeit erwähnt, ist, dass der Vektor von „King“ minus „Man“ plus „Woman“ dem von „Queen“ am nächsten liegt [7]. Das lässt sich mit unserem winzigen Korpus nicht nachstellen, und wir behaupten es deshalb nur als berichtetes Ergebnis. Eine Warnung gehört dazu: Die Vektoren übernehmen auch Verzerrungen ihrer Trainingstexte, etwa Geschlechterstereotype bei Berufen [25]; Pattern 30 behandelt Verzerrungen allgemein. Und das Konzept ist nicht auf Wörter beschränkt. Wer Produkte, Nutzer oder Filialen als „Wörter“ und Warenkörbe oder Sitzungen als „Sätze“ behandelt,

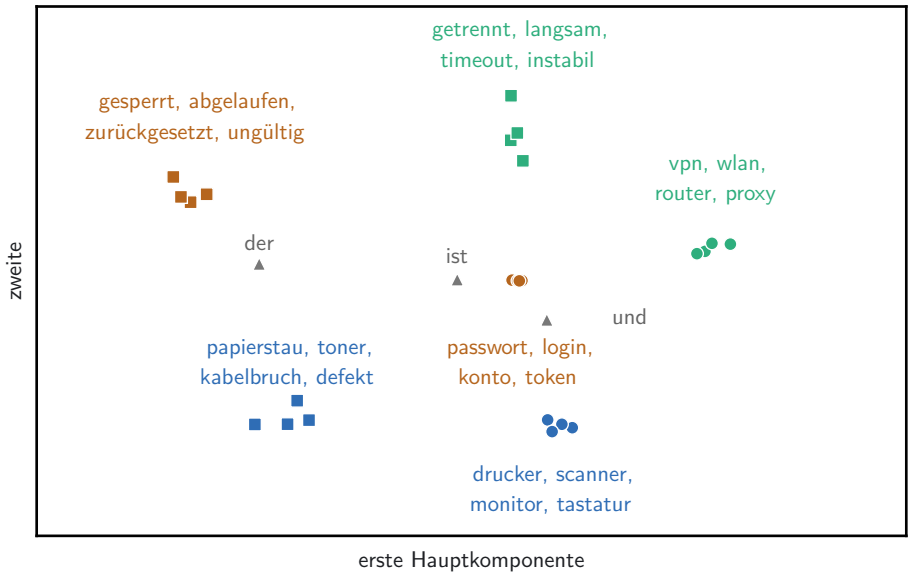


Abbildung 27: Die 27 Wortvektoren aus `06_skipgram.py`, per Hauptkomponentenanalyse auf zwei Dimensionen projiziert (59 % der Streuung bleiben erhalten). Kreise: Gegenstände, Quadrate: Meldungen, Dreiecke: Funktionswörter. Wörter desselben Themas und derselben Rolle liegen praktisch übereinander; die Funktionswörter `der`, `ist` und `und` stehen dazwischen.

bekommt Embeddings für beliebige Kategorien; Guo und Berkhahn lernen sie direkt als Eingabeschicht eines überwachten Netzes [26]. Das ist die Grundlage von Pattern 2.

3.10 Vortrainierte Modelle und Transfer Learning

Warum sollte es etwas nützen, ein Netz zuerst auf einer Aufgabe zu trainieren und dann auf einer anderen weiterzuverwenden? Weil die versteckten Schichten Repräsentationen berechnen, und manche davon sind nicht an die ursprüngliche Aufgabe gebunden. Am besten untersucht ist das bei Bildern. Ein Netz, das Fotos klassifiziert, lernt in den ersten Schichten Detektoren für Kanten und Farbübergänge, in mittleren Schichten Texturen und wiederkehrende Muster, in späteren Teile von Objekten und erst in der letzten Schicht die Klassen [27]. Kanten gibt es in jedem Foto, ob es einen Hund oder ein Auto zeigt; die Klasse „Dackel" dagegen ist nur für die ursprüngliche Aufgabe nützlich. Yosinski und Kollegen formulieren diese Beobachtung als Unterscheidung zwischen **allgemeinen** Merkmalen der unteren und **aufgabenspezifischen** Merkmalen der oberen Schichten [28].

Daraus ergibt sich das Vorgehen, das Pattern 13 beschreibt: Man nimmt ein Netz, das auf einer großen Aufgabe trainiert wurde, behält die unteren Schichten (die **Basis**), ersetzt die letzte Schicht durch eine neue für die eigene Aufgabe (den **Kopf**) und trainiert dann entweder nur den Kopf oder alles weiter, das Letztere mit einer kleinen Lernrate (**Feintuning**). Mechanisch sind das zwei Handgriffe: Die Schichten der Basis bekommen kein Update (`eingefroren`), und die Ausgabeschicht wird neu initialisiert (`neuer_kopf`):

```
netz = copy.deepcopy(vor)
netz.neuer_kopf(klassen, seed=w)
# Feintuning mit kleineren Schritten
netz.eingefroren, lr = (2, 0.1) if "eingefroren" in variante else (0, 0.02)
```

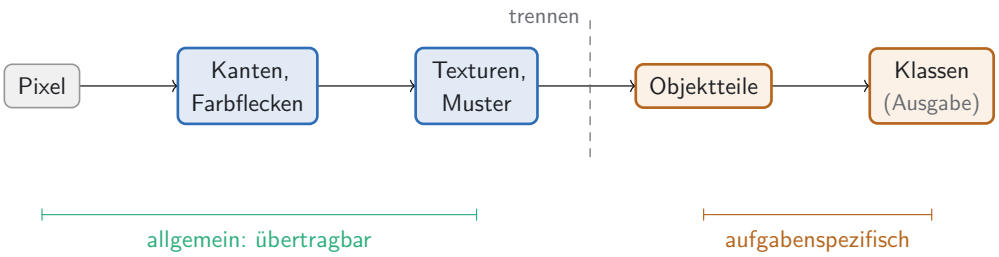


Abbildung 28: Feature-Hierarchie eines Bildnetzes. Die unteren Schichten berechnen Merkmale, die für viele Aufgaben taugen; die oberen sind auf die Ausgangsaufgabe zugeschnitten. Beim Transfer Learning behält man die Basis und ersetzt den Kopf. Der Übergang zwischen allgemein und spezifisch ist fließend; die Trennlinie ist schematisch.

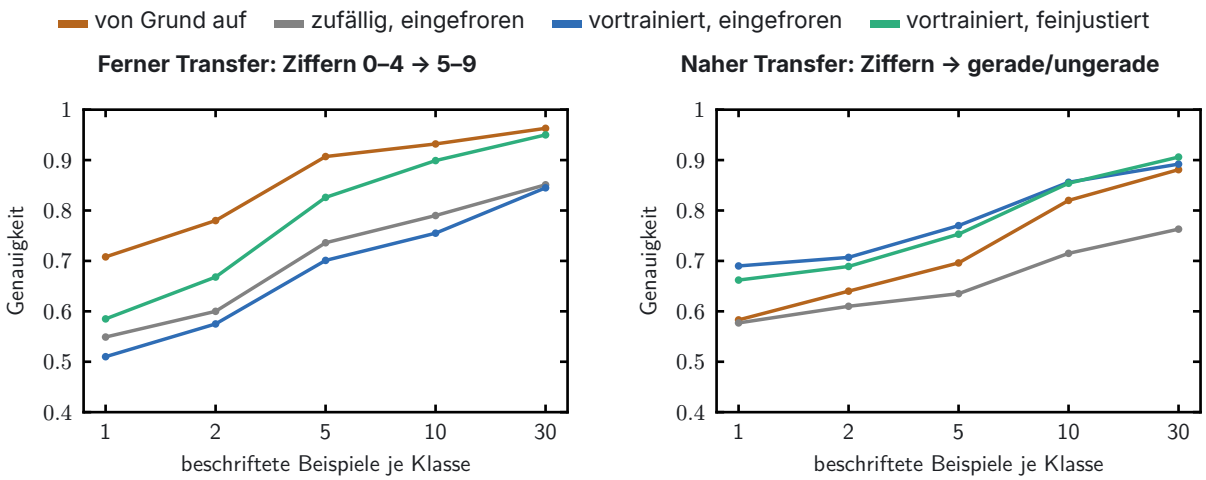


Abbildung 29: Genauigkeit auf ungesehenen Testbildern in Abhängigkeit von der Zahl beschrifteter Zielbeispiele je Klasse, Mittel über 20 Wiederholungen. Links hilft das Vortraining nicht, rechts hilft es besonders bei wenigen Beispielen.

Yosinski et al. haben das in einem ImageNet-Netz systematisch vermessen, indem sie die 1000 Klassen in zwei Hälften teilten, auf der einen Hälfte trainierten und die Schichten für die andere übertrugen. Ihre Ergebnisse lassen sich so zusammenfassen: Die Übertragbarkeit sinkt, wenn man höhere Schichten überträgt und wenn die beiden Aufgaben weiter auseinanderliegen. Ein zweiter, unerwarteter Effekt kommt hinzu: Schneidet man ein Netz mitten durch eng aufeinander abgestimmte Schichten, die sich beim Training aneinander angepasst haben, leidet die Leistung, solange die übertragenen Schichten eingefroren bleiben; Feintuning gleicht das großenteils aus. Übertragene Merkmale sind aber selbst von weit entfernten Aufgaben besser als zufällige. Und wer mit übertragenen Gewichten startet und sie weitertrainiert, erhält eine bessere Verallgemeinerung als beim Start von null, selbst nach langem Feintuning.

Kann man das im Kleinen nachvollziehen? `07_transfer.py` tut es auf den 8×8-Ziffernbildern aus scikit-learn (1797 Bilder, 64 Pixel). Ein Netz 64–64–32–Klassen wird auf einer Quellaufgabe vortrainiert; dann bekommt die Zielaufgabe nur wenige beschriftete Beispiele pro Klasse. Verglichen werden vier Varianten: von Grund auf trainiert, zufällig initialisiert mit eingefrorenen versteckten Schichten (die Kontrolle: Was leistet der Kopf auf reinen Zufalls-Merkmalen?), vortrainiert mit eingefrorener Basis und vortrainiert mit Feintuning. Wir testen zwei Fälle, und das Ergebnis des ersten ist ernüchternd.

Im ersten Fall ist die Quelle „Ziffern 0 bis 4“ und das Ziel „Ziffern 5 bis 9“: andere Klassen, gleiche Art von Bildern. Hier gewinnt das Training von Grund auf in jeder Zeile (links in Abbildung 29): Bei zehn Beispielen je Klasse erreicht es 93,2 %, die vortrainierte Basis mit Feintuning 89,9 %, mit eingefrorener Basis nur 75,5 %, und selbst die zufällige, eingefrorene Basis (79,0 %) ist besser als die vortrainierte, eingefrorene. Eine plausible, hier nicht geprüfte Erklärung: Mit rund 900 Quellbeispielen und einem Netz auf rohen Pixeln lernt es eher Prototypen der Ziffern 0 bis 4 als Strichdetektoren, die für 5 bis 9 taugen. Das ist kein Widerspruch zu Yosinski et al., aber eine Mahnung, dass Transfer nicht automatisch funktioniert: Er braucht eine Quelle, deren Merkmale wirklich allgemein sind, und das erreicht man mit viel Daten und Faltungsschichten, nicht mit 900 Mini-Bildern.

Im zweiten Fall bleibt die Eingabe dieselbe, aber die Frage ändert sich: Das Netz wird zuerst auf allen zehn Ziffern trainiert und soll dann nur noch „gerade oder ungerade“ entscheiden, mit sehr wenigen Beispielen (rechts in Abbildung 29). Das ist die Konstellation, in der ein vortrainiertes Netz in der Praxis meist eingesetzt wird: gleiche Art von Eingabe, neue Frage. Bei einem Beispiel je Klasse erreicht die eingefrorene Basis 69,0 % gegenüber 58,3 % von Grund auf und 57,7 % mit zufälliger Basis; bei zehn Beispielen sind es 85,6 % gegenüber 82,0 % und 71,5 %. Der Vorsprung schrumpft mit mehr Beispielen (bei 30 je Klasse 89,2 % gegenüber 88,1 %, und mit Feintuning 90,6 %), und ob der Abstand von etwa einem Prozentpunkt mehr ist als Zufall, lässt sich aus 20 Wiederholungen ohne Streuungsangabe nicht sagen. Beides entspricht der Erwartung: Vortraining ersetzt fehlende Daten, nicht vorhandene. Dass die zufällige Basis klar schlechter ist als die vortrainierte, belegt, dass die übertragenen Schichten tatsächlich etwas Nützliches berechnen.

ACHTUNG
Dieses Experiment ist ein Lehrmodell, kein Benchmark: ein einziger kleiner Datensatz, ein einziges Netz, 20 Wiederholungen. Es zeigt zwei Mechanismen (Einfrieren und neuer Kopf) und dass der Nutzen von der Nähe der Aufgaben abhängt. Die Größenordnungen echter Bild- und Textmodelle liegen weit darüber, und das Ergebnis des ersten Falls darf man nicht auf sie übertragen.

Embeddings und Transfer Learning sind übrigens dasselbe Prinzip in verschiedenen Größen. Lädt man die word2vec-Vektoren als Anfangswerte einer Embedding-Schicht, überträgt man die gelernte Tabelle in ein neues Netz. Große Sprachmodelle machen es mit allen Schichten: Sie werden auf riesigen Textmengen vortrainiert und anschließend für eine konkrete Aufgabe feinjustiert [29].

3.11 Checkpoints: was zum Zustand gehört

Ein Checkpoint hält den Zustand eines Trainings fest, damit man es später an derselben Stelle fortsetzen kann (Pattern 12). Jetzt, da wir das Netz kennen, lässt sich genau sagen, was zu diesem Zustand gehört: die Gewichte und Biaswerte, der Schwung des Optimierers (bei uns die Momentum-Geschwindigkeiten, bei Adam zwei weitere Werte je Parameter, sodass ein Adam-Checkpoint etwa dreimal so groß ist wie die Gewichte allein [19]), der Epochen- oder Schrittzähler und der Zustand des Zufallsgenerators, der die Reihenfolge der Beispiele bestimmt. Fehlt etwas davon, setzt man nicht dieselbe Rechnung fort, sondern eine andere. `08_checkpoint_parallel.py` trainiert 20 Epochen, sichert den Zustand in zwei Dateien, lädt ihn in ein frisches Netz und trainiert 20 weitere Epochen. Als Referenz dient ein Lauf über 40 Epochen am Stück:

```python
alles = {}
for name, liste in (("W", netz.W), ("b", netz.b), ("vW", netz.vW), ("vb", netz.vb)):
    alles.update({f"{name}{i}": a for i, a in enumerate(liste)})
np.savez(ordner / "gewichte.npz", **alles)  # Gewichte und Momentum
rest = {"epoche": 20, "rng": rng.bit_generator.state}
(ordner / "rest.json").write_text(json.dumps(rest))  # Zähler, Zufallsgenerator
```

Terminal

```
$ uv run --with numpy python 08_checkpoint_parallel.py
Checkpoint-Dateien: ['gewichte.npz', 'rest.json']

Nach insgesamt 40 Epochen: größter Unterschied der Parameter zum durchgehenden Lauf
  alles gesichert          0.00e+00
  ohne Optimizer-Zustand   5.23e-02
  ohne Zufallsgenerator    7.70e-02
```

Mit vollständigem Zustand stimmen die Parameter exakt überein (Differenz 0, Bit für Bit). Lässt man den Optimierer-Zustand weg, weichen sie um bis zu $5 \cdot 10^{-2}$ ab, lässt man den Zufallsgenerator weg, um $8 \cdot 10^{-2}$. Das sind kleine Unterschiede, die das Training trotzdem gut abschließen kann; nachgemessen haben wir das hier nicht. Für Experimente, die man vergleichen will, ist es aber ein Unterschied zwischen reproduzierbar und nicht reproduzierbar. Auch das Prinzip kennen Sie: Das ist das Memento-Muster der Gang of Four [2], ein Objekt sichert seinen inneren Zustand nach außen und lässt ihn später wiederherstellen. Wer nur ein fertiges Modell ausliefert, braucht dagegen die Gewichte samt Architektur und Vorverarbeitung; Optimierer und Zufallsgenerator sind dann entbehrlich.

3.12 Hardware und Verteilung: warum Batches, warum GPUs

Fast die gesamte Rechenarbeit eines Netzes sind Matrixmultiplikationen. Eine Schicht mit m Eingängen und n Neuronen braucht für einen Batch von B Beispielen $2Bmn$ Gleitkommaoperationen (eine Multiplikation und eine Addition je Gewicht und Beispiel). Gleichzeitig müssen die mn Gewichte aus dem Speicher geholt werden, unabhängig von B: bei float32 sind das $4mn$ Bytes. Das Verhältnis, die **arithmetische Intensität**, beträgt für große Schichten ungefähr $B/2$ Operationen pro Byte. Bei $B = 1$ ruft der Prozessor eine große Matrix ab, um sie für ein einziges Beispiel zu benutzen, und wartet überwiegend auf den Speicher; erst bei großem B wird die Rechenleistung zum Engpass. Das ist die Aussage des Roofline-Modells [30].

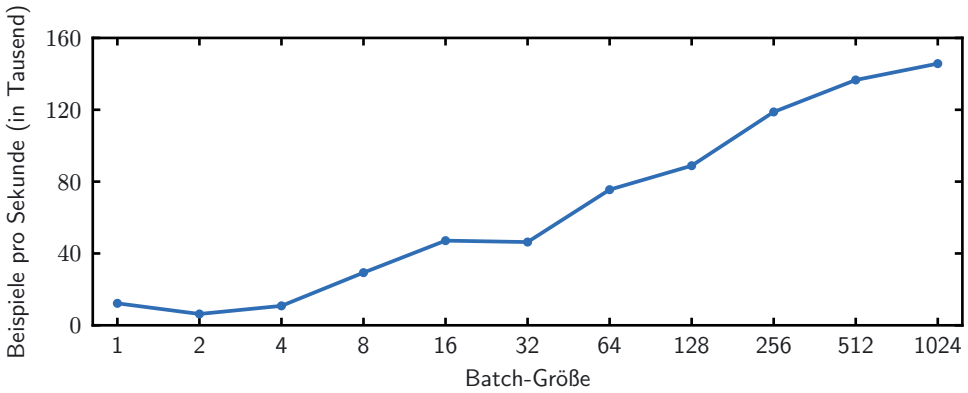


Abbildung 30: Durchsatz des Vorwärtsdurchlaufs eines 3 × (1024 × 1024)-Netzes in Abhängigkeit von der Batch-Größe, gemessen mit `09_durchsatz.py` auf einer Apple-M3-Pro-CPU. Die Werte sind maschinen- und lastabhängig und gelten als Größenordnung.

`09_durchsatz.py` misst das an einem Netz mit drei Schichten zu je 1024×1024 (12,6 MB Gewichte, 6,3 MFLOP je Beispiel) auf der CPU des Autor-Rechners (Apple M3 Pro, numpy 2.5.4). Gemessen wird der Vorwärtsdurchlauf, jeweils die schnellste von 15 Messungen:

Bei Batch 1 verarbeitet die Maschine rund 12 000 Beispiele pro Sekunde (77 GFLOP/s), bei Batch 1024 rund 146 000 (917 GFLOP/s), also etwa das Zwölffache pro Beispiel, mit derselben Hardware und denselben Gewichten. Die Kurve ist nicht ganz glatt (Batch 2 ist langsamer als Batch 1); die Ursache haben wir nicht untersucht. Wiederholt man den Lauf, weichen die Werte um einige Prozent ab (bei der Gegenprobe 11 300 und 141 000 Beispiele pro Sekunde), der Verlauf bleibt derselbe. Eine GPU treibt dieses Prinzip weiter: Sie hat sehr viele einfache Rechenwerke, die Matrixprodukte parallel ausführen, und eine hohe Speicherbandbreite. Voll ausgelastet ist sie nur, wenn jeder Aufruf genug Arbeit enthält, also mit großen Batches. Das erklärt, warum man Mini-Batches nicht nur wegen der Gradienten, sondern auch wegen der Hardware verwendet, und warum Batch 1 im Training selten sinnvoll ist. Bei der Vorhersage im Betrieb gilt dasselbe, sodass man dort Anfragen sammelt und gebündelt abarbeitet.

Aber Batches wachsen nicht beliebig. Die Zwischenwerte des Vorwärtsdurchlaufs, die Backpropagation später braucht, wachsen mit der Batch-Größe, und irgendwann ist der Speicher des Geräts voll. Dann bleibt nur, die Arbeit auf mehrere Geräte zu verteilen, die **Datenparallelität** (Pattern 14): Jeder Worker besitzt eine vollständige Kopie des Netzes und bearbeitet einen Teil des Batches (Abbildung 31). Die Gradienten der Worker werden gemittelt, und jeder führt denselben Schritt aus.

Ob das Mitteln eine Näherung ist, lässt sich nachrechnen, und die Antwort lautet: nein. Der Loss ist ein Mittelwert über die Beispiele; der Gradient eines Mittelwerts ist das Mittel der Gradienten, und das Mittel über vier Viertel ist das Mittel über das Ganze. Der zweite Teil von `08_checkpoint_parallel.py` prüft es:

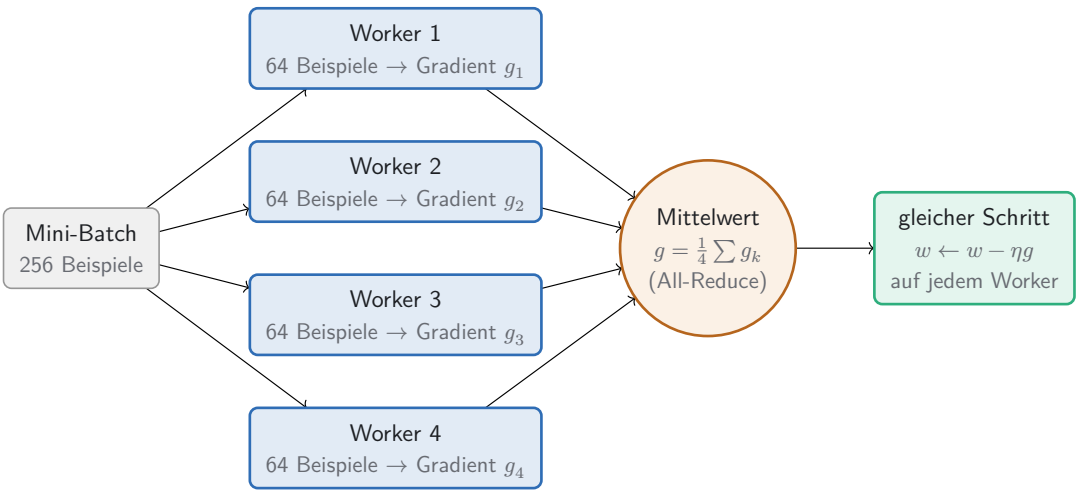


Abbildung 31: Synchrone Datenparallelität mit vier Workern. Nach dem Mitteln besitzen alle Worker dieselben Parameter, als hätte ein einziges Gerät den ganzen Batch verarbeitet.

```
shards = np.array_split(batch, 4)  # vier Teile zu je 64 Beispielen
teil = [netz.gradienten(X[s], y[s]) for s in shards]
gemittelt = [sum(len(s) * t[0][i] for s, t in zip(shards, teil)) / 256
             for i in range(3)]
```

Terminal

```
$ uv run --with numpy python 08_checkpoint_parallel.py
Gradient: ein Gerät (Batch 256) gegen Mittel aus 4 Workern (je 64): max. Abweichung 2.8e-17
ungleiche Shards (100/100/56): gewichtetes Mittel 2.8e-17, ungewichtetes Mittel 4.3e-03
Nach 100 Schritten: größter Unterschied der Parameter 2.2e-16
```

Die Abweichung von $3 \cdot 10^{-17}$ ist Rundungsrauschen, und nach 100 Schritten stimmen die Parameter eines Geräts und die der vier Worker bis auf $2 \cdot 10^{-16}$ überein. Zwei Feinheiten zeigt die mittlere Zeile: Bei ungleich großen Shards (100, 100 und 56 Beispiele) muss man nach Größe gewichten; ein einfaches Mittel der drei Gradienten weicht um $4 \cdot 10^{-3}$ ab, was im Training Verzerrungen erzeugt. Und der Mittelwert ist die einzige Stelle, an der die Worker miteinander reden müssen; wie diese Kommunikation organisiert wird, behandelt Pattern 14. Eine Folge sollten Sie kennen: Vier Worker mit je 64 Beispielen bilden einen effektiven Batch von 256. Goyal et al. empfehlen, die Lernrate bei k-facher Batch-Größe ebenfalls mit k zu multiplizieren und sie am Anfang des Trainings schrittweise zu steigern (warm-up) [31]. Man spart also nicht einfach Zeit, sondern ändert nebenbei die Trainingsdynamik.

3.13 Ausblick: der Transformer

Die meisten heutigen Sprachmodelle sind Transformer. Wer dieses Kapitel gelesen hat, kennt fast alle Bausteine; neu ist einer, die **Self-Attention** [32]. Das Problem, das sie löst: Bei Text hängt die Bedeutung eines Wortes vom Kontext ab, und der Kontext kann weit entfernt liegen. Self-Attention lässt jedes Token (Wort oder Wortteil) von allen anderen Informationen übernehmen, gewichtet danach, wie relevant sie sind. Jedes Token wird dazu mit drei gelernten Matrizen in drei Vektoren übersetzt, Q („wonach suche ich“), K („was biete ich an“) und V („was gebe ich weiter“). Die Ausgabe ist

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Wer ein Dictionary kennt, erkennt die Struktur wieder: Ein `get(key)` liefert den Wert zu genau einem passenden Schlüssel; Attention vergleicht die Anfrage mit **allen** Schlüsseln und liefert eine nach Ähnlichkeit gewichtete Mischung aller Werte. Der Softmax macht die Gewichte zu Wahrscheinlichkeiten, der Faktor $1/\sqrt{d_k}$ hält die Skalarprodukte in einem Bereich, in dem der Softmax noch brauchbare Gradienten liefert. `10_attention.py` rechnet das für drei Tokens durch (Tabelle 5).

Tabelle 5: Attention-Gewichte für drei Tokens (Zeile: fragendes Token, Spalte: betrachtetes Token) aus `10_attention.py`, mit zufälligen Embeddings und Matrizen. Jede Zeile summiert sich zu 1 (Rundungsabweichungen möglich). Im trainierten Modell würden diese Gewichte aus Daten gelernt.

	der	drucker	streikt
der	0,97	0,00	0,02
drucker	0,15	0,67	0,17
streikt	0,01	0,79	0,20

Ein Detail ist für das Verständnis wichtig: Self-Attention ist blind für die Reihenfolge. Sortiert man die Eingabe um, sortiert sich nur die Ausgabe um; `10_attention.py` prüft das mit `np.allclose` (Ergebnis `True`). Deshalb addiert man zu den Embeddings eine Information über die Position. Kosten entstehen quadratisch in der Länge: Bei n Tokens ist die Gewichtsmatrix $n \times n$, bei 1000 Tokens eine Million Einträge je Kopf und Schicht, bei 100 000 schon zehn Milliarden.

Ein Block besteht aus Self-Attention und einem gewöhnlichen zweischichtigen MLP mit ReLU, jeweils mit Residual-Verbindung, also dem Addieren der Eingabe zur Ausgabe [33], und einer Normalisierung der Werte [34]. Im ursprünglichen Modell werden mehrere Attention-Köpfe mit unterschiedlichen Matrizen parallel berechnet; die Basisversion hat 8 Köpfe, 6 Blöcke und Vektoren der Länge 512, das MLP hat 2048

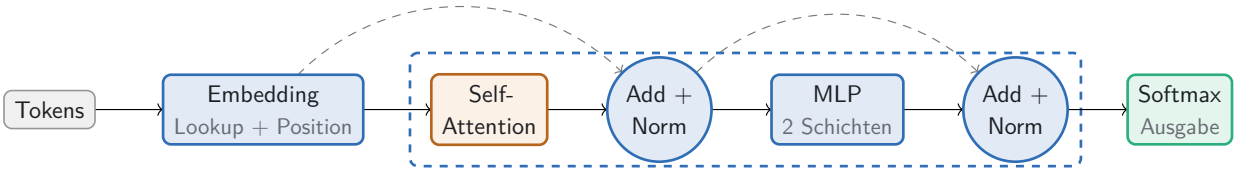


Abbildung 32: Aufbau eines Transformers (vereinfacht, nur Stapel von Blöcken). Der blau gestrichelte Rahmen wird N-mal wiederholt, die grauen Bögen sind Residual-Verbindungen. Außer Self-Attention, Residual-Verbindungen und Normalisierung kommen alle Bausteine aus diesem Kapitel.

Neuronen [32]. Dahinter steckt also keine neue Art von Maschine, sondern dieselbe Mechanik in anderer Verdrahtung: Embedding-Lookup am Eingang, Softmax und Cross-Entropy am Ausgang, Backpropagation und Adam im Training, Dropout als Regularisierung. Warum die Architektur sich durchgesetzt hat, hat einen praktischen Grund: Anders als bei rekurrenten Netzen, die Wort für Wort rechnen, lassen sich alle Positionen gleichzeitig als Matrixprodukte berechnen, und das passt zu der Hardware aus dem vorigen Abschnitt. Und Modelle wie BERT [29] sind genau das, was oben als Vortraining mit anschließendem Feintuning beschrieben wurde, nur eben mit Transformer-Blöcken. Was dieses Kapitel nicht behandelt, sind Faltungsnetze, rekurrente Netze und das Training großer Sprachmodelle; dafür sei auf die Literatur verwiesen.

3.14 Das Wichtigste in Kürze

- Ein Neuron bildet eine gewichtete Summe plus Bias und wendet eine Nichtlinearität an; ohne sie fallen mehrere Schichten zu einer einzigen linearen zusammen. Ein Netz ist eine Folge solcher Schichten, jede Schicht eine Matrixmultiplikation.
- Softmax macht Logits zu Wahrscheinlichkeiten, die Cross-Entropy bestraft sichere Fehler hart; ihr Gradient nach den Logits ist einfach $p - y$.
- Backpropagation ist die Kettenregel in der richtigen Reihenfolge: ein Vorwärtsdurchlauf, der Zwischenwerte merkt, und ein Rückwärtsdurchlauf, der Gradienten durch die Schichten reicht. Ein Gradient Check mit Differenzenquotienten prüft eine eigene Implementierung.
- Im Trainingsprotokoll lassen sich Fehler ablesen: ein springender oder um den Raten-Wert pendelnder Loss deutet auf eine zu große Lernrate (tote ReLUs), ein kaum sinkender auf eine zu kleine. Läufe vergleicht man nach der Zahl der Schritte.
- Overfitting zeigt sich im Validierungs-Loss früher als in der Genauigkeit. Early Stopping, L2 und Dropout dämpfen es; auf stark verrauschten Daten senkten sie den Test-Loss deutlich, die Genauigkeit blieb ungefähr gleich.
- Eine Embedding-Schicht ist eine Gewichtsmatrix, deren Zeilen man nachschlägt; one-hot mal Matrix ergibt genau eine Zeile. Gelernt wird sie wie jedes Gewicht, aber pro Beispiel nur in einer Zeile.
- Skip-Gram trainiert eine Embedding-Tabelle durch die Vorhersage von Nachbarwörtern; Wörter mit ähnlichen Nachbarn bekommen ähnliche Vektoren, auch wenn sie nie gemeinsam auftreten.
- Transfer Learning überträgt Schichten, die allgemeine Merkmale berechnen, und trainiert einen neuen Kopf. Es hilft, wenn Quelle und Ziel verwandt sind und das Ziel wenige Beispiele hat; ein Beispiel im Kleinen zeigt auch den Fall, in dem es nicht hilft.
- Zum Zustand eines Trainings gehören Gewichte, Optimierer-Zustand, Zähler und Zufallsgenerator. Gradienten über Worker zu mitteln ist keine Näherung, sondern gleich dem Gradienten über den ganzen Batch. Große Batches nutzen Hardware besser aus, bis der Speicher endet.

Tabelle 6: Wo die Mechanik dieses Kapitels in den Patterns wiederkehrt.

Mechanik aus diesem Kapitel	taucht wieder auf in
Embedding als Lookup, Zeilenweises Lernen	Pattern 2 (Embeddings), Pattern 4 (Multimodal Input); die Tabellengröße begrenzt Pattern 1 (Hashed Feature)
Loss-Kurven, Early Stopping, Overfitting	Pattern 11 (Useful Overfitting), Pattern 12 (Checkpoints)
Zustand des Trainings	Pattern 12 (Checkpoints), Pattern 27 (Model Versioning)
Eingefrorene Basis, neuer Kopf	Pattern 13 (Transfer Learning)
Mitteln von Gradienten, Batch-Größe, Hardware	Pattern 14 (Distribution Strategy)
Lernrate, Batch-Größe, Epochen als Stellschrauben	Pattern 15 (Hyperparameter Tuning)

3.15 Weiterführende Literatur

Als Lehrbuch für alles, was hier angerissen wurde, eignet sich Goodfellow, Bengio und Courville [14], vor allem die Kapitel zu Feedforward-Netzen, Regularisierung, Optimierung und die „Practical Methodology“. Die Originalarbeit zur Backpropagation ist kurz und lesbar [18]. Zu Embeddings lohnt der Skip-Gram-Artikel selbst [7] zusammen mit seiner Fortsetzung zu Negative Sampling [24]. Zum Transfer Learning ist die Arbeit von Yosinski et al. [28] die verständlichste Grundlage, zur Architektur des Transformers der Originalartikel [32]. Regularisierung behandeln Prechelt [20] (Early Stopping) und Srivastava et al. [21] (Dropout). Wer die Hardwareseite vertiefen will, findet im Roofline-Artikel [30] ein Modell, das für Batch-Größen ebenso gilt wie für Algorithmen im Allgemeinen.

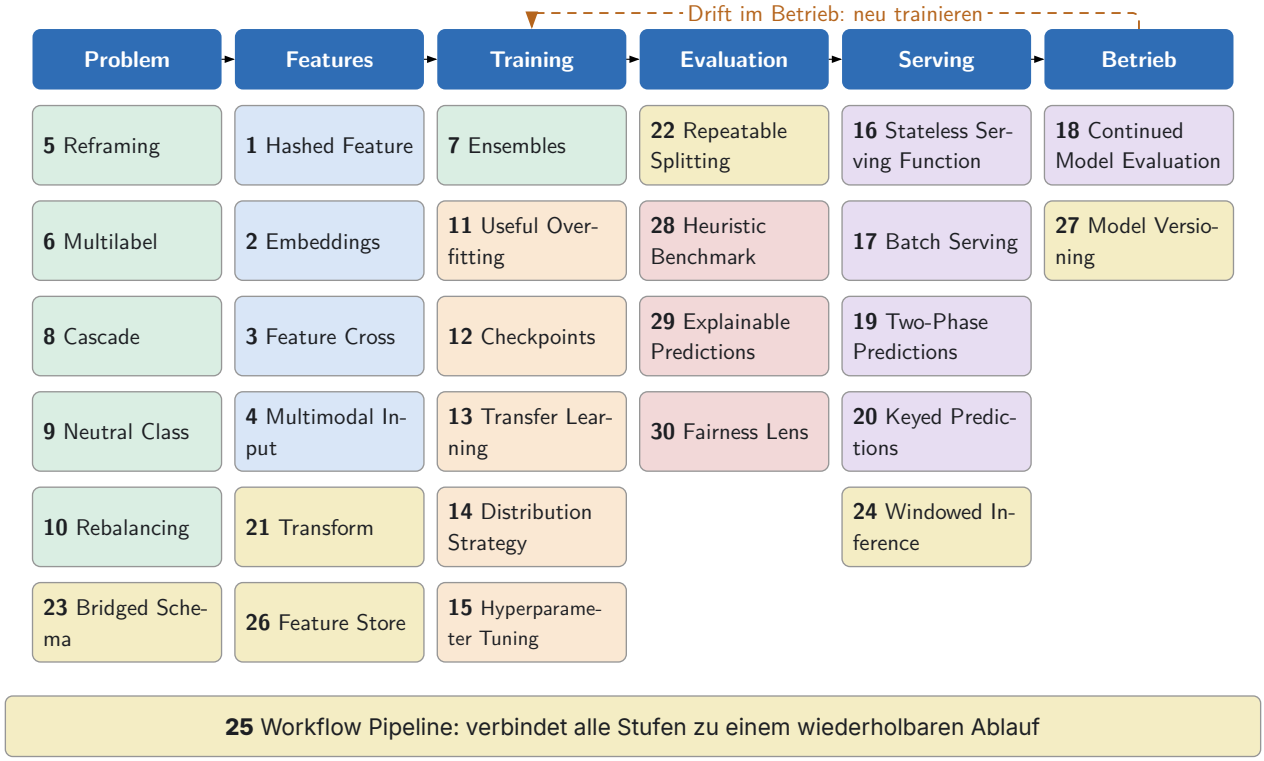


Abbildung 33: Alle 30 Patterns am ML-Lebenszyklus. Jedes Pattern steht dort, wo es hauptsächlich wirkt; manche wirken an mehreren Stellen (Hashed Feature etwa auch beim Serving). Ensembles (7) und Cascade (8) gehören im Buch zur Problem Representation.

4 Landkarte der 30 Patterns

Dieses Kapitel setzt die dreißig Patterns zueinander in Beziehung, bevor die Kapitel 5 bis 10 sie einzeln erklären. Es beantwortet drei Fragen: An welcher Stelle eines ML-Projekts wirkt welches Pattern? Welches Symptom deutet auf welches Pattern hin? Und welche Patterns hängen voneinander ab? Die Begriffe stammen aus Kapitel 2; wenn Ihnen zum Beispiel „Training-Serving Skew“ nichts sagt, lesen Sie zuerst den Abschnitt „Begriffe für den Betrieb“ dort.

4.1 Die Patterns am Lebenszyklus

Das Buch gliedert die Patterns nach Themen, nicht nach der Reihenfolge, in der man sie in einem Projekt braucht. Für die Praxis ist die zweite Ordnung oft nützlicher. Abbildung 33 legt deshalb dieselben dreißig Patterns auf den Lebenszyklus aus Kapitel 2: von der Problemformulierung über Features, Training und Evaluation bis zum Betrieb. Die Farbe verrät die Kategorie im Buch, die Nummer das Pattern.

Drei Beobachtungen lohnen sich. Erstens liegt die Mitte der Karte nicht dort, wo man sie erwartet: Das Training selbst, das viele für die Hauptsache halten, hat im Buch nur eine eigene Kategorie mit fünf Patterns. Der größere Teil der Patterns betrifft das, was vor und nach dem Training passiert, nämlich Daten, Aufgabenstellung und Betrieb. Zweitens kommt die Kategorie Reproducibility über fast alle Spalten verstreut vor, denn Reproduzierbarkeit ist keine Phase, sondern eine Eigenschaft des ganzen Ablaufs. Drittens ist der Lebenszyklus ein Kreis: Der gestrichelte Pfeil oben bedeutet, dass Beobachtungen aus

dem Betrieb ein neues Training auslösen. Genau diese Rückkopplung ist der Grund, warum Continued Model Evaluation (18) und Workflow Pipeline (25) zusammengehören.

4.2 Vom Symptom zum Pattern

In einem Projekt erscheint selten die Frage „Welches Pattern brauche ich?“. Es erscheint ein Symptom: ein Modell, das schlechter wird, eine Spalte, die den Speicher sprengt, ein Ergebnis, das niemand reproduzieren kann. Tabelle 7 ordnet typische Symptome den Patterns zu. Sie ist eine Lesehilfe und kein Entscheidungsbaum; oft passen zwei Patterns, und die Kapitel 5 bis 10 beschreiben, wie man zwischen ihnen wählt.

Tabelle 7: Symptome aus Projekten und die Patterns, die dafür infrage kommen.

Symptom	Pattern	Kap.
Daten und Features		
Eine kategoriale Spalte hat 50 000 Werte (Postleitzahl, Produkt-ID); One-hot-Kodierung sprengt den Speicher, und zur Laufzeit tauchen neue Werte auf	1 Hashed Feature, 2 Embeddings	5
Zwei Features wirken nur gemeinsam (Wochentag und Uhrzeit), ein einfaches Modell sieht die Kombination nicht	3 Feature Cross	5
Die Eingabe besteht aus Bild und Tabelle oder Text und Tabelle	4 Multimodal Input	5
Aufgabe formulieren		
Eine Zahl vorherzusagen gelingt nicht präzise, die Streuung ist groß	5 Reframing	6
Ein Beispiel gehört mehreren Klassen gleichzeitig an (Artikel mit mehreren Schlagwörtern)	6 Multilabel	6
Einzelne Modelle schwanken stark zwischen Trainingsläufen	7 Ensembles	6
Die Aufgabe zerfällt in Teilfragen, die aufeinander aufbauen, etwa ein Sonderfall vor der eigentlichen Vorhersage	8 Cascade	6
Fachleute sind sich bei Grenzfällen uneinig, Etiketten widersprechen sich	9 Neutral Class	6
Nur 1 % der Beispiele sind positiv, 99 % Accuracy bedeuten nichts	10 Rebalancing	6
Training		
Das Modell soll eine bekannte, deterministische Funktion nachbilden	11 Useful Overfitting	7
Ein Trainingslauf dauert Stunden, und ein Absturz kostet alles	12 Checkpoints	7
Für die Aufgabe gibt es zu wenige Trainingsdaten	13 Transfer Learning	7
Das Training dauert auf einem Rechner zu lange	14 Distribution Strategy	7
Hyperparameter werden nach Gefühl gewählt	15 Hyperparameter Tuning	7
Serving und Betrieb		
Das Modell hängt am Trainingscode, mehrere Clients wollen es aufrufen	16 Stateless Serving Function	8
Eine Million Vorhersagen sollen über Nacht entstehen, nicht einzeln	17 Batch Serving	8
Die Qualität sinkt im Betrieb, ohne dass es jemand bemerkt	18 Continued Model Evaluation	8
Das große Modell passt nicht auf das Gerät oder die Verbindung fällt aus	19 Two-Phase Predictions	8
Batch-Ergebnisse lassen sich den Eingaben nicht zuordnen	20 Keyed Predictions	8
Reproduzierbarkeit		
Features werden im Training anders berechnet als im Serving (Training-Serving Skew)	21 Transform, 26 Feature Store	9
Die Testmenge ist bei jedem Lauf eine andere, Ergebnisse sind nicht vergleichbar	22 Repeatable Splitting	9
Die Datenquelle hat ihr Schema geändert, alte und neue Datensätze passen nicht mehr zusammen	23 Bridged Schema	9
Ein Feature braucht ein Zeitfenster früherer Werte (Mittel der letzten Stunde)	24 Windowed Inference	9
Der Ablauf besteht aus Notebooks und manuellen Schritten	25 Workflow Pipeline	9
Mehrere Teams berechnen dieselben Features mehrfach und verschieden	26 Feature Store	9

Symptom	Pattern	Kap.
Niemand weiß, welches Modell diese Vorhersage geliefert hat, ein Rollback ist unmöglich	27 Model Versioning	9
Verantwortung		
Niemand weiß, ob das Modell besser ist als eine einfache Regel	28 Heuristic Benchmark	10
Betroffene oder Fachbereiche verlangen eine Begründung der Vorhersage	29 Explainable Predictions	10
Die Gesamtgenauigkeit stimmt, aber eine Gruppe wird schlechter behandelt	30 Fairness Lens	10

4.3 Abhängigkeiten zwischen Patterns

Die dreißig Patterns lösen jeweils ein eigenes Problem, aber sie stehen nicht allein. Manche setzen andere voraus, manche lösen dasselbe Problem auf verschiedenen Ebenen, und eines steht in Spannung zu einem anderen. Abbildung 34 zeigt die wichtigsten Beziehungen; die Tabelle danach begründet sie. Die Zuordnung ist die Lesart dieses Handbuchs, abgeleitet aus den Pattern-Beschreibungen, und keine vollständige Liste aus dem Buch. Ob das Buch jede einzelne Beziehung ausdrücklich so formuliert, ist nicht für jede Kante geprüft; das gilt als offen.

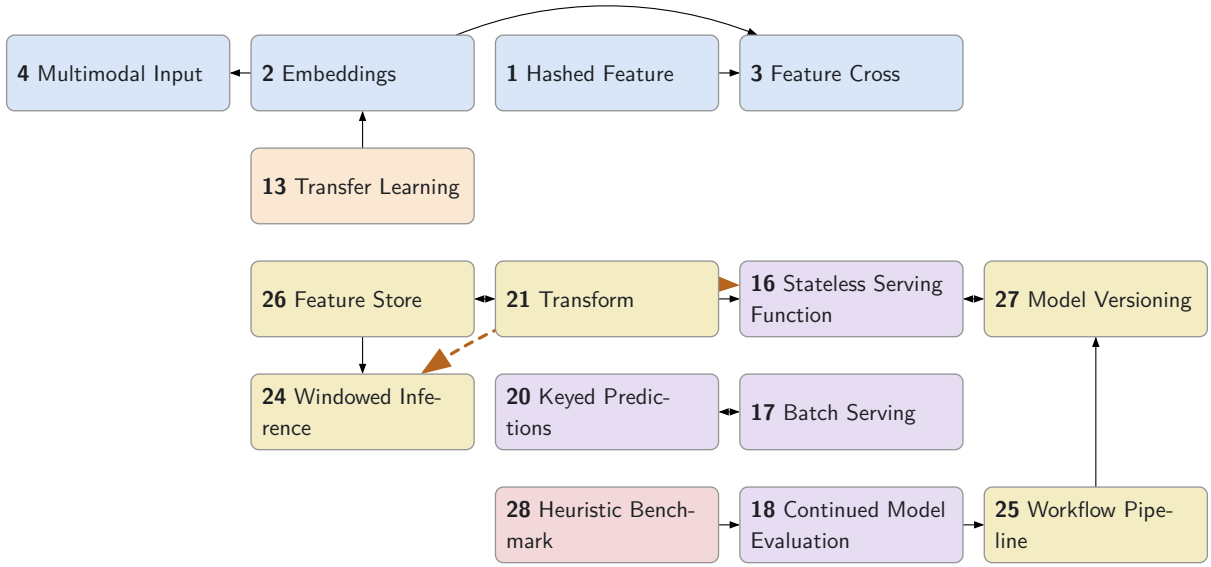


Abbildung 34: Wichtige Beziehungen zwischen Patterns. Ein Pfeil von A nach B bedeutet: A liefert B einen Baustein, eine Voraussetzung oder einen Auslöser. Doppelpfeile stehen für Patterns, die zusammen eingesetzt werden. Die orange gestrichelte Linie markiert eine Spannung.

Die folgende Tabelle erklärt die Kanten. Jede Beziehung ist so formuliert, dass Sie sie in der Lehre als Frage verwenden können: „Warum braucht Pattern A das Pattern B?“

Tabelle 8: Begründungen zu den Kanten in Abbildung 34.

Beziehung	Begründung
21 Transform ↔ 26 Feature Store	Beide sorgen dafür, dass Training und Serving dieselben Feature-Werte sehen. Transform kapselt die Rechenvorschrift im Modell-Artefakt, der Feature Store speichert berechnete Werte und teilt sie über Modelle hinweg.
21 Transform → 16 Stateless Serving Function	Die Serving-Funktion soll rohe Eingaben annehmen können. Das gelingt nur, wenn die Transformation Teil des exportierten Modells ist.
20 Keyed Predictions ↔ 17 Batch Serving	Bei Batch-Vorhersagen verlaufen Ein- und Ausgabe verteilt und in unbekannter Reihenfolge. Ein mitgeführter Schlüssel ordnet jede Vorhersage ihrem Eingabedatensatz zu.
1 Hashed Feature, 2 Embeddings → 3 Feature Cross	Ein Cross erzeugt sehr viele, meist dünn besetzte Kombinationen. Hashing kann deren Anzahl begrenzen, Embeddings können sie verdichten.
2 Embeddings → 4 Multimodal Input	Jede Eingabeart (Bild, Text, Zahl) wird in einen Vektor übersetzt; erst gleichförmige Vektoren lassen sich zu einer Eingabe zusammenfügen.

Beziehung	Begründung
13 Transfer Learning → 2 Embeddings	Vortrainierte Embeddings sind eine häufige Form, in der Wissen aus einem anderen Problem übertragen wird.
26 Feature Store → 24 Windowed Inference	Plausibel, aber vom Handbuch selbst gezogen: Feature-Werte aus einem Zeitfenster früherer Beispiele müssen irgendwo berechnet und vorgehalten werden, und ein Feature Store ist ein möglicher Ort dafür.
24 Windowed Inference ⇢ 16 Stateless Serving Function	Spannung: Eine zustandslose Funktion kennt keine früheren Beispiele, ein Zeitfenster ist aber Zustand. Der Zustand wandert deshalb aus der Funktion heraus in die vorgelagerte Pipeline.
16 Stateless Serving Function ↔ 27 Model Versioning	Versionierte Modelle laufen jeweils hinter einer eigenen zustandslosen Schnittstelle, sodass Clients Versionen parallel nutzen oder zurückschalten können.
28 Heuristic Benchmark → 18 Continued Model Evaluation	Plausibel, aber vom Handbuch selbst gezogen: Die einfache Regel kann auch im Betrieb als Messlatte mitlaufen, an der sich zeigt, ob das Modell seinen Aufwand noch verdient.
18 Continued Model Evaluation → 25 Workflow Pipeline → 27 Model Versioning	Wenn die Überwachung Qualitätsverlust zeigt, stößt sie die Pipeline an, die ein neues Modell trainiert und als neue Version veröffentlicht.

4.4 Leicht zu verwechseln

Einige Patterns klingen ähnlich, lösen aber verschiedene Probleme. Wer sie in der Lehre sauber trennt, erspart sich viele Rückfragen.

Ensembles (7), Cascade (8) und Two-Phase Predictions (19) verwenden alle mehrere Modelle. Bei einem Ensemble beantworten mehrere Modelle dieselbe Frage, und die Antworten werden zusammengeführt, um genauer und stabiler zu werden. Bei einer Cascade bestimmt das Ergebnis des ersten Modells, was das zweite tut; die Aufgabe ist in Teilfragen zerlegt. Bei Two-Phase Predictions teilt sich das System nach dem Ort des Einsatzes: ein kleines Modell läuft vor Ort, ein großes in der Cloud.

Transform (21) und Feature Store (26) verhindern beide, dass Training und Serving verschieden rechnen. Transform arbeitet pro Modell und liegt im Artefakt. Der Feature Store arbeitet pro Organisation und macht berechnete Features wiederverwendbar.

Rebalancing (10) und Neutral Class (9) betreffen beide die Klassen eines Klassifikationsproblems. Rebalancing korrigiert eine seltene Klasse, die das Training untergehen lässt. Neutral Class schafft eine zusätzliche Klasse für Fälle, bei denen die Etiketten selbst uneindeutig sind.

Stateless Serving Function (16) und Batch Serving (17) beschreiben dasselbe Modell, einmal für einzelne Anfragen in Echtzeit und einmal für große Datenmengen auf einmal. Das Modell bleibt gleich, die Art des Aufrufs ändert sich.

Explainable Predictions (29) und Fairness Lens (30) sehen auf verschiedene Ebenen. Das eine erklärt, warum das Modell für einen einzelnen Fall so entschieden hat. Das andere prüft, ob sich das Modell über Gruppen von Fällen hinweg gleich verhält.

5 Data Representation

Ein Modell rechnet. Es nimmt eine Liste von Zahlen entgegen, multipliziert sie mit gelernten Gewichten, addiert die Produkte und schiebt das Ergebnis durch einfache Funktionen. Alles, was in der Anwendung keine Zahl ist, also ein Text, eine Postleitzahl, ein Foto oder der Wochentag, muss vorher in Zahlen übersetzt werden. Diese Übersetzung heißt *Repräsentation*. Die einzelnen Zahlen, die ins Modell gehen, heißen *Features*, und die Liste aller Features eines Beispiels nennt man *Feature-Vektor*.

Dieselbe Information lässt sich auf viele Arten in Zahlen fassen, und ein Modell lernt sie unterschiedlich leicht. Die Repräsentation ist deshalb keine Randnotiz der Vorverarbeitung, sondern eine der wirksamsten Stellschrauben überhaupt. Für Software Engineers ist sie die Schnittstelle zwischen Fachdomäne und Modell, vergleichbar mit einem Datenvertrag: Wer sie ändert, ändert, was das Modell überhaupt sehen kann. Dieses Kapitel behandelt die vier Patterns aus Kapitel 2 des Buchs von V. Lakshmanan, S. Robinson, und M. Munn [1]. Davor stehen die einfachen Bausteine, auf denen sie aufbauen.

5.1 Einfache Repräsentationen: aus Rohwerten werden Zahlen

Vier Werkzeuge reichen für die meisten Spalten einer Tabelle. Wir probieren sie an 1000 künstlichen Datensätzen aus (Fläche, Jahreseinkommen, Stadtteil), die in `code/04-data-representation/01_einfache_repraesentationen.py` erzeugt werden. Alle Skripte dieses Kapitels laufen in diesem Ordner mit `uv run --with scikit-learn --with pandas python <datei>` (numpy und scipy kommen als Abhängigkeiten von scikit-learn mit); die gezeigten Ausgaben stammen aus echten Läufen mit festem Zufallssamen und liegen als Textdateien in `ausgabe/`.

Skalierung. Die Fläche einer Wohnung liegt zwischen 25 und 200, das Einkommen zwischen einigen Tausend und mehreren Hunderttausend. Beim Training mit Gradientenabstieg (das Modell verbessert seine Gewichte in vielen kleinen Schritten, siehe Kapitel 2) dominieren Features mit großen Zahlenwerten, und das Training wird langsam oder instabil. Auch Verfahren, die Abstände berechnen (etwa k-nearest-neighbors, das ein Beispiel nach seinen nächsten Nachbarn im Trainingsdatensatz einordnet), sehen ein Feature mit großem Wertebereich überproportional. Entscheidungsbäume sind dagegen unempfindlich, weil sie nur vergleichen. Zwei Standardlösungen: *Min-Max-Skalierung* bildet den Bereich auf 0 bis 1 ab, $\frac{x - x_{\min}}{x_{\max} - x_{\min}}$, und reagiert empfindlich auf Ausreißer. Der *z-Score* zieht den Mittelwert ab und teilt durch die Standardabweichung, $\frac{x - \mu}{\sigma}$; danach hat das Feature Mittelwert 0 und Standardabweichung 1. Wichtig ist, dass diese Konstanten *nur aus den Trainingsdaten* stammen und im Betrieb unverändert wiederverwendet werden. Rechnet man sie auf allen Daten aus, fließt Wissen über die Testdaten ins Training (*Data Leakage*). Dass Training und Betrieb dieselben Konstanten verwenden, ist das Anliegen von Pattern 21 (Transform, Kapitel 8).

Log-Transformation. Einkommen, Preise, Klickzahlen sind rechtsschief: viele kleine Werte, wenige sehr große. Im Beispiel liegt das Mittel bei 46 050, der Median bei 39 476, und die Schiefe (ein Maß für die Asymmetrie, 0 heißt symmetrisch) beträgt 2,05. Nach dem Logarithmus sind Mittel und Median praktisch gleich und die Schiefe ist −0,03 (Abbildung 35). Der Logarithmus macht aus Unterschieden um einen Faktor Unterschiede um einen festen Betrag: 20 000 zu 40 000 ist derselbe Schritt wie 40 000 zu 80 000. Genau so denken Menschen über Einkommen, und ein Modell mit additiven Gewichten kann das dann ebenfalls.

Bucketizing. Manchmal ist nicht der genaue Wert wichtig, sondern die Gruppe: Altersgruppen, Einkommensklassen. Man zerlegt den Wertebereich in Intervalle und ersetzt die Zahl durch die Nummer des Intervalls (oder, wie gleich beim One-Hot, durch eine Spalte je Intervall). Das ist nützlich, wenn der Zusammenhang nicht gleichmäßig ansteigt, etwa wenn das Unfallrisiko bei jungen und sehr alten Fahrern steigt und dazwischen niedrig ist. Ein lineares Modell kann einer Zahl nur *ein* Gewicht geben; mit Buckets

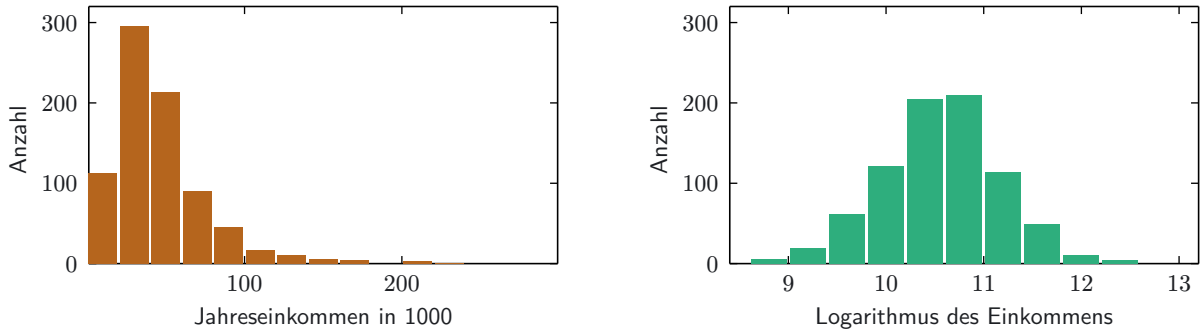


Abbildung 35: Das Einkommen der 800 Trainingsdatensätze vor (links) und nach (rechts) dem Logarithmus. Die Werte stammen aus `01_einfache_repraesentationen.py`.

bekommt jede Gruppe ihr eigenes. Der Preis ist verlorene Feinheit. Im Beispiel entstehen vier Gruppen mit gleich vielen Datensätzen (Quartile), und die Grenzen werden wieder nur aus den Trainingsdaten bestimmt.

One-Hot-Kodierung. Ein Stadtteil ist keine Zahl. Naheliegend wäre, ihn zu nummerieren (Mitte 0, Nord 1, Ost 2, Süd 3). Dann würde das Modell aber annehmen, Ost liege „doppelt so weit" von Mitte wie Nord. One-Hot vermeidet jede Reihenfolge: Es gibt eine Spalte pro Wert, und genau eine davon ist 1. Das funktioniert hervorragend bei vier Stadtteilen und wird unbrauchbar bei vier Millionen Produkt-IDs. Dort setzen die Patterns 1 und 2 an.

Terminal

```
$ uv run --with scikit-learn --with pandas python 01_einfache_repraesentationen.py
Fläche roh      : [40.0, 75.0, 120.0]
min-max (0..1)  : [0.13, 0.44, 0.83]
z-score         : [-1.77, 0.02, 2.32]

Einkommen roh: Mittel 46,050, Median 39,476, Schiefe 2.05
Einkommen log: Mittel 10.56, Median 10.58, Schiefe -0.03

Quartilgrenzen: [6447.0, 26284.0, 39476.0, 56720.0, 230625.0]
Bucket für 20.000 / 40.000 / 90.000 / 300.000: [0, 2, 3, 3]

Spalten: ['stadtteil_Mitte', 'stadtteil_Nord', 'stadtteil_Ost', 'stadtteil_Sued']
[[0 1 0 0]
 [0 0 1 0]]
```

Der Skalierer hat seine Parameter aus den Trainingsdaten gelernt (`fit`) und wendet sie danach nur noch an (`transform`).

> **BEZUG ZU SOFTWARE ENGINEERING**
> Die gelernten Konstanten (Mittelwert, Quartilgrenzen, Kategorienliste) sind Zustand, der zum Modell gehört wie ein Konfigurationsstand. Wer ein Modell ausliefert und den Skalierer vergisst, liefert eine Anwendung aus, deren Konfigurationsdatei fehlt. Das Muster, das dahinter steht, heißt in der ML-Welt *Training-Serving Skew* und wird in Kapitel 9 behandelt.

5.2 Pattern 1: Hashed Feature

Szenario. Ein Online-Shop will vorhersagen, ob ein Besucher ein Produkt kauft. Ein wichtiges Feature ist die Produkt-ID, und der Shop führt zwei Millionen verschiedene IDs, jede Woche kommen Tausende neue dazu. Mit One-Hot entstünden zwei Millionen Spalten.

Das Problem. Dahinter stecken vier verschiedene Schwierigkeiten. Erstens die Größe: Ein lineares Modell braucht pro Spalte ein Gewicht, bei zwei Millionen Spalten sind das zwei Millionen Parameter nur für dieses eine Feature, und die meisten Spalten sehen im Training kaum einen Datensatz. Zweitens das Vokabular: Das Modell muss wissen, welche ID zu welcher Spalte gehört, diese Zuordnung muss beim Training erzeugt, gespeichert und im Betrieb identisch geladen werden. Drittens der *Cold Start*: Eine ID, die im Training nie vorkam, hat keine Spalte, und das Modell weiß nichts über sie. Viertens die *Long*

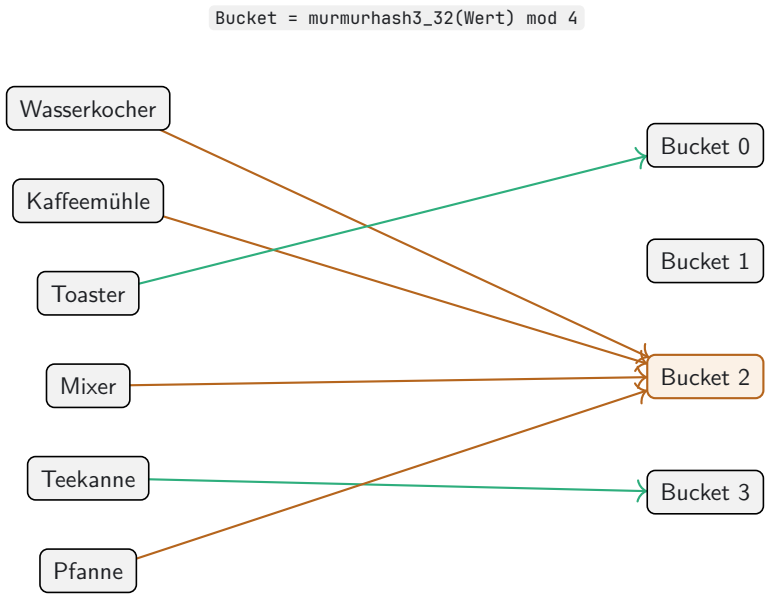


Abbildung 36: Das Prinzip an sechs Werten und vier Buckets. Die Bucket-Nummern stammen aus dem echten Hash. Orange: Mehrere Werte teilen sich einen Bucket (Kollision), Bucket 1 bleibt leer.

Tail: Die Verteilung ist stark ungleich, ein paar Produkte machen den Großteil der Daten aus, die übrigen kommen so selten vor, dass ihre Gewichte kaum gelernt werden.

Die Lösung. Das Pattern ersetzt die gelernte Zuordnung durch eine berechnete. Eine *Hashfunktion* macht aus jedem Text eine große Zahl, deterministisch: derselbe Text ergibt immer dieselbe Zahl, ähnliche Texte ergeben völlig verschiedene. Teilt man die Zahl mit Rest durch die gewünschte Spaltenzahl B, erhält man einen Index zwischen 0 und $B-1$, den *Bucket*. Das Feature besteht dann aus B Spalten (One-Hot über den Bucket) statt aus zwei Millionen. Es gibt keine Tabelle, die man speichern müsste, und jede ID, auch eine nie gesehene, landet irgendwo (Abbildung 36).

Die Zuordnung in der Abbildung stammt aus dem Skript `02_hashed_feature.py`:

Terminal

```
$ uv run --with scikit-learn --with pandas python 02_hashed_feature.py
Wasserkocher   murmurhash3_32 = 1241995430  ->  Bucket 2
Kaffeemühle    murmurhash3_32 = 2988581302  ->  Bucket 2
Toaster        murmurhash3_32 = 4077643628  ->  Bucket 0
Mixer          murmurhash3_32 = 4214966034  ->  Bucket 2
Teekanne       murmurhash3_32 = 3599985207  ->  Bucket 3
Pfanne         murmurhash3_32 = 2034520326  ->  Bucket 2
```

Kollisionen quantifiziert. Weil es weniger Buckets als Werte gibt, müssen manche Werte denselben Bucket teilen. Das ist eine *Kollision*: Das Modell kann die beiden Werte nicht mehr unterscheiden, es lernt für beide dasselbe Gewicht. Wie oft das passiert, lässt sich ausrechnen. Bei n Werten und B Buckets teilt sich ein bestimmter Wert seinen Bucket mit mindestens einem anderen mit der Wahrscheinlichkeit $1-\left(1-\frac{1}{B}\right)^{n-1}$, bei großem B etwa $1-e^{-\frac{n}{B}}$. Sind es so viele Buckets wie Werte ($B=n$), kollidieren rund 63 % aller Werte ($1-\frac{1}{e}$). Die Messung mit echten Hashes bestätigt die Rechnung:

Terminal

```
        B  belegt     leer  größter  kollidiert  Formel
    1,000   1,000       0       21      100.0%  100.0%
    5,000   4,310     690        9       86.5%   86.5%
   10,000   6,335   3,665        7       63.1%   63.2%
   50,000   9,056  40,944        5       18.2%   18.1%
  100,000   9,533  90,467        4        9.2%    9.5%
1,000,000   9,951 990,049        2        1.0%    1.0%

5000 Kategorien, 200,000 Zeilen, Basisrate 0.091
B =     16: AUC 0.567
B =     64: AUC 0.635
```

```
B =    256: AUC 0.673
B =   1024: AUC 0.714
B =   4096: AUC 0.750
B =  16384: AUC 0.766
B =  65536: AUC 0.773
One-Hot ohne Hashing (5000 Spalten): AUC 0.776

unbekannte Kategorie landet in Bucket 899
P(Klick) = 0.269  (Basisrate 0.091)

hash('Berlin') % 1000 in drei Prozessen: ['136', '617', '673']
murmurhash3_32('Berlin') % 1000 in drei Aufrufen: [359, 359, 359]
```

Die Spalte „kollidiert“ zeigt, welcher Anteil der 10 000 Produkte seinen Bucket mit mindestens einem anderen teilt; rechts daneben steht die Formel. Bei 10 000 Buckets sind es 63,1 % gemessen und 63,2 % gerechnet. Erst bei 1 000 000 Buckets, dem Hundertfachen der Werte, fällt der Anteil auf 1 %. Der Hash verteilt also gleichmäßig, und die Faustformel taugt, um die Größenordnung abzuschätzen.

Wie viele Buckets? Das ist eine Abwägung zwischen Speicher und Genauigkeit und hängt davon ab, *welche* Werte kollidieren. Das zweite Experiment simuliert 5000 Kategorien mit Zipf-artiger Häufigkeit (wenige häufige, viele seltene Werte) und einer eigenen Klickrate je Kategorie; eine logistische Regression sieht nur das gehashte Feature. Als Maß dient die AUC (Fläche unter der ROC-Kurve: 0,5 entspricht Raten, 1,0 perfekter Trennung). Mit 16 Buckets ist das Modell fast blind (0,567), mit 4096 hat es den größten Teil der Qualität zurück (0,750), und ab 16 384 Buckets, dem Dreifachen der Kategorienzahl, fehlt nur noch etwa ein AUC-Punkt zur Referenz ohne Hashing (0,766 gegen 0,776, Abbildung 37). Eine belegbare Faustregel für die Bucket-Zahl kenne ich nicht. Praktisch schätzt man die Zahl der verschiedenen Werte, rechnet die Kollisionsformel aus und behandelt die Bucket-Zahl dann wie einen Hyperparameter (Pattern 15).

Cold Start. Eine nie gesehene ID verursacht keinen Fehler, weil auch sie einen Bucket bekommt. Das ist Segen und Falle zugleich. Im Skript landet die unbekannte Kategorie in Bucket 899 und bekommt eine Klickwahrscheinlichkeit von 0,269, fast das Dreifache der Basisrate von 0,091. Das Gewicht dieses Buckets stammt von den bekannten Kategorien, die dort zufällig liegen; die Vorhersage ist also eine Art Durchschnitt zufälliger Nachbarn, sieht aber aus wie jede andere. Wer erkennen will, ob ein Wert neu ist, muss das separat prüfen. Fachlich ist das Verhalten meist besser als ein Absturz und schlechter als ein bewusst gewählter Standardwert.

FarmHash und MurmurHash. Das Buch erzeugt den Hash mit FarmHash (in BigQuery `FARM_FINGERPRINT`). scikit-learn bietet `FeatureHasher`, der die 32-Bit-Variante von MurmurHash3 verwendet (so auch `sklearn.utils.murmurhash3_32`). Das Prinzip ist gleich, die Bucket-Nummern sind es nicht; für ein Modell ist das unerheblich, solange Training und Betrieb dieselbe Funktion nutzen. Die Primärquelle ist K. Weinberger, A. Dasgupta, J. Langford, A. Smola, und J. Attenberg [35] sie leiten zusätzlich ein Vorzeichen aus dem Hash ab, damit sich Kollisionen im Mittel aufheben (`alternate_sign=True`, der Standard von `FeatureHasher`; im Experiment abgeschaltet, damit das Ergebnis mit One-Hot vergleichbar bleibt). Pythons eingebaute `hash()` ist dagegen ungeeignet: Sie ist bei Strings pro Prozess mit einem zufälligen Startwert versehen

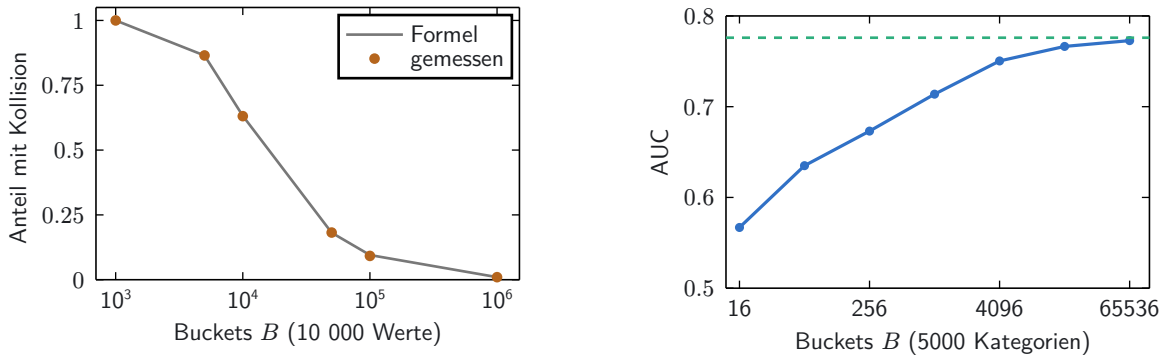


Abbildung 37: Links: Anteil der Werte mit Kollision in Abhängigkeit von der Bucket-Zahl (Punkte gemessen, Linie nach Formel). Rechts: AUC eines Klickmodells mit gehashter Kategorie, gestrichelt grün die Referenz mit echtem One-Hot. Beide Achsen sind logarithmisch.

(„gesalzen“), sodass derselbe String bei jedem Programmstart eine andere Zahl ergibt, und die letzten Zeilen der Ausgabe zeigen es, derselbe String landet in drei Prozessen in drei verschiedenen Buckets. Wer `hash()` verwendet, baut Training-Serving Skew direkt ins Feature ein.

Trade-offs und Grenzen. Ein Bucket lässt sich nicht zurückverfolgen: Das Gewicht gehört zu einer Gruppe von Werten, die man nur mit einer zusätzlichen Tabelle wieder auflösen könnte. Das Modell ist deshalb schwerer zu erklären. Kollisionen treffen die seltenen Werte härter als die häufigen, denn ein seltener Wert, der sich einen Bucket mit einem häufigen teilt, wird von diesem überstimmt. Bei Features, die Personengruppen beschreiben, kann das zur Folge haben, dass eine kleine Gruppe im Modell unsichtbar wird (Kapitel 10). Außerdem ist unklar, was passiert, wenn die Bucket-Zahl später geändert wird: Alle Zuordnungen ändern sich, das Modell muss neu trainiert werden.

Wann nicht. Bei wenigen hundert Werten ist One-Hot einfacher und exakt. Wenn jeder Wert einzeln erklärbar sein muss, etwa weil ein Regulator nachfragt, ist Hashing das falsche Werkzeug. Und wenn die Kategorie *inhaltlich* ähnliche Werte hat (Produkte derselben Marke), ist Pattern 2 besser, weil Hashing Ähnlichkeit zerstört: Zwei fast gleiche Texte landen in völlig verschiedenen Buckets.

Bezug zu Software Engineering. Der Gedanke ist so alt wie die Hash-Tabelle, nur ohne Kollisionsauflösung: Man nimmt in Kauf, dass zwei Schlüssel sich einen Platz teilen, und tauscht dafür festen Speicherbedarf und den Verzicht auf eine zentrale Zuordnungstabelle. Wie beim Sharding einer Datenbank verteilt eine Hashfunktion Last ohne Koordination, und wie dort ist die Wahl der Funktion und der Anzahl eine Entscheidung, die man später nur teuer ändern kann. Auch das Fehlen von Zustand erinnert an *Stateless Serving* (Pattern 16): Das Feature braucht beim Betrieb nichts, was nicht im Code steht.

> **SO ERKLÄRE ICH ES**
> Ein Hashed Feature ist eine Zuordnungstabelle, die man nicht speichern muss: Eine Rechenvorschrift sagt für jeden Wert, in welche von B Schubladen er kommt. Dafür landen manchmal zwei Werte in derselben Schublade, und das Modell kann sie nicht mehr unterscheiden. Lassen Sie die Studierenden ausrechnen, wie viele Schubladen sie für 10 000 Werte brauchen, damit weniger als 10 % kollidieren, und das Ergebnis mit Abbildung 37 vergleichen.
>
> **Verständnisfrage.** Warum ist `hash()` aus Python für Feature-Hashing in einem Modell, das später als Service läuft, gefährlich?
>
> **Antwort.** Weil der Wert pro Prozess zufällig gesalzen ist. Das Training liefert andere Buckets als der Service, und das Modell sieht im Betrieb Features, die es so nie gelernt hat, ohne dass ein Fehler auftritt.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Hashed Feature“; K. Weinberger, A. Dasgupta, J. Langford, A. Smola, und J. Attenberg [35].

5.3 Pattern 2: Embeddings

Szenario. Eine Suchmaschine soll aus wenigen Wörtern das Thema eines Dokuments erkennen. Das Vokabular umfasst 100 000 Wörter. Wir bauen das im Kleinen nach: 24 Wörter aus vier Themen (Sport, Finanzen, Kochen, Technik), 6000 künstliche Dokumente zu je drei Wörtern, davon 15 % der Wörter themenfremd. Die Datei ist `03_embeddings.py`.

Das Problem. Mit One-Hot ist jedes Wort ein Vektor aus lauter Nullen und einer Eins an seiner eigenen Position. Zwei verschiedene Wörter haben dann nichts gemeinsam: Der Kosinus des Winkels zwischen ihren Vektoren ist immer 0, „Aktie“ ist von „Dividende“ genauso weit entfernt wie von „Pfanne“. Die Information, dass die beiden zusammengehören, steckt nicht in der Repräsentation, und das Modell muss für jedes Wort einzeln lernen, was es bedeutet. Was es über „Aktie“ gelernt hat, hilft ihm bei einem selten gesehenen Wort wie „Dividende“ nicht. Dazu kommt wieder die Größe: Ein Vektor mit 100 000 Stellen, in dem fast alles null ist, ist ein teurer Weg, eine Zahl zwischen 0 und 99 999 zu übertragen.

Die Lösung. Ein *Embedding* ordnet jeder Kategorie einen kurzen Vektor mit reellen Zahlen zu, etwa mit 2, 8 oder 64 Stellen, in dem ähnliche Kategorien nah beieinander liegen. Die Zuordnung ist eine Tabelle W mit einer Zeile pro Kategorie und d Spalten. Der entscheidende Gedanke ist, dass man diese Tabelle nicht von Hand baut, sondern vom Netz *lernen* lässt, und zwar als ganz gewöhnliche Gewichte der ersten Schicht. Multipliziert man den One-Hot-Vektor mit der Tabelle, bleibt von der Matrixmultiplikation nur eine

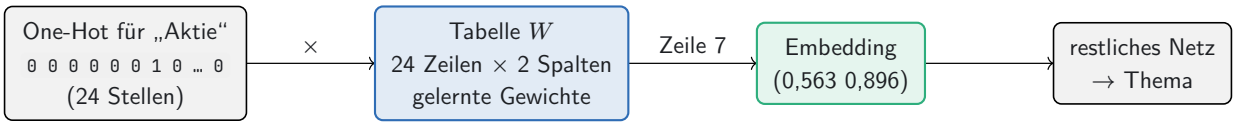


Abbildung 38: Die Embedding-Schicht ist eine Tabelle. Der One-Hot-Vektor wählt eine Zeile; die Zahlen im grünen Kasten sind die echten Werte, die das Netz in `03_embeddings.py` für „Aktie“ gelernt hat.

Zeile übrig: die des Worts (Abbildung 38). Beim Training berechnet das Netz den Fehler am Ausgang und verschiebt über Backpropagation alle beteiligten Gewichte ein Stückchen, auch die Zeilen von W. Wörter, die zu denselben Ausgaben führen sollen, werden dabei in dieselbe Richtung geschoben. Am Ende liegen sie nahe beieinander, ohne dass jemand diese Nähe vorgegeben hätte.

Im Code ist das kurz. Ein Netz mit einer versteckten Schicht aus nur zwei Neuronen bekommt die Wörter als One-Hot-Spalten (ein Dokument hat bis zu drei Einsen) und lernt, das Thema vorherzusagen. Die Gewichtsmatrix zwischen Eingabe und versteckter Schicht hat dann 24 Zeilen und 2 Spalten, und jede Zeile ist der gelernte Vektor eines Worts.

```
# Versteckte Schicht mit nur 2 Neuronen: jedes Wort bekommt einen 2-D-Vektor
net = MLPClassifier(hidden_layer_sizes=(2,), activation="tanh",
                    max_iter=2000, random_state=1).fit(X, y)
W = net.coefs_[0]       # Form (24, 2): eine Zeile pro Wort = Embedding-Tabelle
print("Form der ersten Gewichtsmatrix:", W.shape)

# One-Hot mal Matrix ist nichts anderes als das Herausgreifen einer Zeile
one_hot = np.zeros(V); one_hot[idx["Aktie"]] = 1
print("one_hot @ W == W[Zeile]:", np.allclose(one_hot @ W, W[idx["Aktie"]]))
for w in ("Aktie", "Tor", "Pfanne"):
    print(f"{w:9s}", W[idx[w]].round(2))
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 03_embeddings.py
Form der ersten Gewichtsmatrix: (24, 2)
one_hot @ W == W[Zeile]: True
Aktie     [0.56 0.9 ]
Tor       [-1.18  0.82]
Pfanne    [ 0.78 -0.7 ]

mittlerer Kosinus, gleiches Thema : +0.99
mittlerer Kosinus, anderes Thema  : -0.33
ähnlichste Wörter zu 'Aktie': ['Dividende', 'Börse', 'Zins']
unähnlichstes Wort zu 'Aktie': Compiler
Kategorien        24 -> Faustregel   2.2 Dimensionen
Kategorien     5,000 -> Faustregel   8.4 Dimensionen
Kategorien 1,000,000 -> Faustregel  31.6 Dimensionen
```

Die Ausgabe bestätigt die Behauptung: `one_hot @ W` ist identisch mit dem Herausgreifen der Zeile. Frameworks wie Keras oder PyTorch bieten deshalb eine eigene Embedding-Schicht, die gleich nachschlägt und sich die Multiplikation mit lauter Nullen spart.

Ähnlichkeit messen. Der Kosinus des Winkels zwischen zwei Vektoren, $\cos(a,b) = \frac{a \cdot b}{|a| \cdot |b|}$, ist $+1$ bei gleicher Richtung, 0 bei rechtem Winkel und -1 bei Gegenrichtung. Zwischen Wörtern desselben Themas liegt er im Mittel bei +0,99, zwischen Wörtern verschiedener Themen bei −0,33. Die ähnlichsten Wörter zu „Aktie“ sind „Dividende“, „Börse“ und „Zins“, das unähnlichste ist „Compiler“. Abbildung 39 zeigt die 24 Vektoren als Punkte in der Ebene. Das Netz hat vier Gruppen gebildet, ohne dass die Themen in den Eingabedaten als Wort vorkamen; es kannte nur die Zielklasse.

Dass es so sauber aussieht, liegt daran, dass die Daten so gebaut sind. Echte Embeddings sind unordentlicher. Das bekannteste Beispiel sind die Wortvektoren aus word2vec [7]: Ein Netz lernt, ein Wort aus seinen Nachbarwörtern vorherzusagen (oder umgekehrt), und die Embedding-Tabelle ist das Nebenprodukt. Bei Tabellendaten funktioniert derselbe Gedanke: C. Guo und F. Berkhahn [26] ersetzten kategoriale Spalten wie Filiale oder Wochentag durch gelernte *Entity Embeddings*.

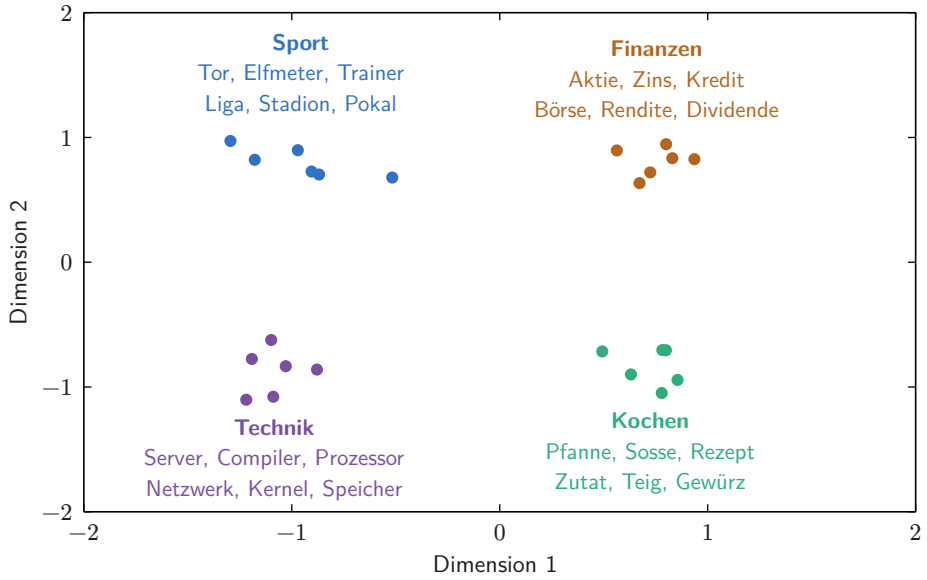


Abbildung 39: Die gelernte Embedding-Tabelle (24 Wörter, 2 Dimensionen). Wörter desselben Themas bilden eine Gruppe, obwohl das Thema nie als Eingabe vorkam. Die Punkte sind die echten Zeilen der Gewichtsmatrix; Wörter mit fast gleichen Vektoren überdecken sich; deshalb stehen die Wörter einer Gruppe als Liste daneben.

Welche Dimension? Die Dimension d ist ein Hyperparameter: zu wenige verlieren Information, zu viele kosten Speicher und fördern Überanpassung (*Overfitting*: das Modell lernt die Trainingsdaten auswendig statt des Musters). Eine bekannte Faustregel aus dem TensorFlow-Blog setzt d auf die vierte Wurzel der Kategorienzahl [36]: 2,2 für 24 Wörter, 8,4 für 5000 Kategorien, 31,6 für eine Million (das Skript rechnet es aus). Sie ist ein Startwert ohne Garantie, die beste Dimension bestimmt man über Validierungsdaten (Pattern 15). Man kann Embeddings auch vortrainiert aus einer anderen Aufgabe übernehmen, ein Fall von Transfer Learning (Pattern 13).

Trade-offs und Grenzen. Ein Embedding braucht genug Daten pro Kategorie, sonst liegt die Zeile nach dem Training noch fast dort, wo sie zufällig gestartet ist. Für eine neue Kategorie gibt es keine Zeile (Cold Start); üblich ist eine mittrainierte Zeile für „unbekannt“. Die Achsen haben keine Bedeutung für Menschen, und zweidimensionale Bilder wie Abbildung 39 sind bei realen Embeddings mit 64 oder 300 Dimensionen *Projektionen* (etwa mit PCA oder t-SNE), die Abstände verzerren. Ein Embedding enthält außerdem, was für die Trainingsaufgabe nützlich war, einschließlich der Vorurteile in den Daten: T. Bolukbasi, K.-W. Chang, J. Zou, V. Saligrama, und A. Kalai [25] zeigen, dass Wortvektoren aus Nachrichtentexten Geschlechterstereotype enthalten (Kapitel 9).

Wann nicht. Bei wenigen Kategorien mit vielen Beispielen je Kategorie reicht One-Hot, und das Modell bleibt einfacher. Mit einem sehr kleinen Datensatz kann man die Tabelle nicht sinnvoll lernen; dann sind vortrainierte Embeddings oder One-Hot die bessere Wahl.

Bezug zu Software Engineering. Ein Embedding ist ein Surrogatschlüssel mit Bedeutung. Aus einer beliebigen ID, die nur Identität trägt, wird ein Punkt in einem Raum, in dem Abstand etwas aussagt. Das macht Ähnlichkeitssuche möglich (nächste Nachbarn im Raum, wie sie Vektordatenbanken anbieten). Der Embedding-Raum ist zugleich eine Schnittstelle, und zwar eine zerbrechliche: Wird das Modell neu trainiert, entsteht ein neuer Raum, und die alten Vektoren sind mit den neuen nicht vergleichbar, so wie sich ein anderes Serialisierungsformat nicht mit dem alten lesen lässt. Embeddings gehören deshalb zusammen mit dem Modell versioniert (Pattern 27).

> **SO ERKLÄRE ICH ES**
> Stellen Sie sich eine Landkarte vor, auf der jedes Wort einen Ort bekommt. Das Netz verschiebt die Orte so lange, bis Wörter, die zu denselben Antworten führen, nah beieinander liegen. Die Nachbarschaft auf der Karte ist ein Ergebnis des Trainings, nicht der Eingabe. Technisch ist die Karte nur die erste Gewichtsmatrix des Netzes. Zeigen Sie Abbildung 39 und lassen Sie raten, welche vier Gruppen zu sehen sind.

Verständnisfrage. Warum hilft ein Embedding dem Modell bei einem selten gesehenen Wort, ein One-Hot-Vektor aber nicht?

Antwort. Weil das seltene Wort nahe bei häufigeren Wörtern liegt und das Modell dort schon gelernt hat, wie sich diese Gegend auf die Ausgabe auswirkt. Beim One-Hot hat jedes Wort eine eigene, unabhängige Spalte, aus der nichts auf andere übertragbar ist.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Embeddings"; Mikolov et al. [7]; C. Guo und F. Berkhahn [26], TensorFlow Team [36].

5.4 Pattern 3: Feature Cross

Szenario. Ein Verkehrsdienst will vorhersagen, ob eine Zufahrtsstraße ins Stadtzentrum überlastet ist. Zwei Features liegen vor: die Richtung (stadteinwärts oder stadtauswärts) und die Tageszeit (Vormittag oder Nachmittag). Die Pendlerlogik ist einfach: Morgens fahren alle hinein, nachmittags alle hinaus. Stau gibt es also, wenn der Verkehr in der Hauptrichtung läuft. Im Skript `04_feature_cross.py` entstehen daraus 20 000 Beobachtungen, bei denen in 10 % der Fälle das Ergebnis zufällig umgedreht wird.

Das Problem. Ein lineares Modell, hier eine logistische Regression, bildet eine gewichtete Summe: ein Gewicht für die Richtung, eines für die Tageszeit, dazu eine Konstante. Die Wirkung von „Vormittag" ist in dieser Summe immer dieselbe, ganz gleich, in welcher Richtung gefahren wird. In den Daten kippt sie aber mit der Richtung: Vormittags ist es stadteinwärts voll und stadtauswärts leer, nachmittags umgekehrt. Betrachtet man jedes Feature für sich, liegt die Staurate bei rund 50 %, die Features scheinen wertlos. Das ist die XOR-Funktion („entweder-oder"), und ihre Unlösbarkeit für ein einzelnes lineares Element gehört seit M. Minsky und S. Papert [15] zu den Klassikern der Disziplin.

Die Lösung. Das Pattern fügt ein neues Feature hinzu, das die Kombination beider Werte beschreibt, das *Feature Cross* (Kreuzprodukt). Aus zwei Features mit je zwei Werten wird ein Feature mit vier: „einwärts × vormittags", „einwärts × nachmittags", und so weiter. Es wird wie jede Kategorie per One-Hot kodiert, und jede der vier Zellen bekommt ein eigenes Gewicht (Abbildung 40). Das Modell bleibt linear, aber die Wechselwirkung ist jetzt ein eigenes Feature, das es gewichten kann.

```
def lauf(name, spalten, df_tr, df_te):
    enc = ColumnTransformer([("oh", OneHotEncoder(), spalten)])
    m = make_pipeline(enc, LogisticRegression()).fit(df_tr[spalten], df_tr.stau)
    print(f"{name:42s} Genauigkeit {m.score(df_te[spalten], df_te.stau):.3f}")
    return m.score(df_te[spalten], df_te.stau)

a_ohne = lauf("lineares Modell, ohne Cross", ["richtung", "tageszeit"], train, test)

# Feature Cross: ein neues kategoriales Feature aus der Kombination
for d in (train, test):
    d["richtung_x_tageszeit"] = d.richtung + "_x_" + d.tageszeit
mit_cross = ["richtung", "tageszeit", "richtung_x_tageszeit"]
a_cross = lauf("lineares Modell, mit Cross", mit_cross, train, test)
```

	Vormittag	Nachmittag
stadteinwärts	**90 %** Staurate	**10 %** Staurate
stadtauswärts	**10 %** Staurate	**90 %** Staurate



Abbildung 40: Links: Staurate je Kombination (echte Werte aus den Daten); jede Zeile und jede Spalte allein mittelt zu rund 50 %. Rechts: Genauigkeit auf Testdaten. Mit 10 % Rauschen sind 90 % das Maximum.

Terminal

```
$ uv run --with scikit-learn --with pandas python 04_feature_cross.py
tageszeit       nachmittag  vormittag
richtung
stadtauswaerts         0.9        0.1
stadteinwaerts         0.1        0.9
lineares Modell, ohne Cross                Genauigkeit 0.504
lineares Modell, mit Cross                 Genauigkeit 0.901
Netz mit versteckter Schicht, ohne Cross   Genauigkeit 0.901
Entscheidungsbaum (Tiefe 2), ohne Cross    Genauigkeit 0.901

Anzahl der Cross-Zellen:
  Wochentag x Stunde                   168
  ... x Stadtteil (40)                 6720
  ... x Stadtteil x Fahrzeugtyp (12)   80640
```

Die Genauigkeit springt von 50,4 % auf 90,1 %, und mehr ist wegen des Rauschens nicht möglich. Die Zeile „Netz mit versteckter Schicht“ zeigt eine wichtige Einschränkung des Pattern: Ein Netz mit einer versteckten Schicht lernt die Wechselwirkung auch ohne Cross (90,1 %), und ein Entscheidungsbaum der Tiefe 2 ebenfalls (90,1 %). Eine versteckte Schicht kann aus den Eingaben neue Kombinationen bilden. Feature Crosses sind daher vor allem für Modelle wichtig, die selbst nur einfach gewichten, und für Fälle mit sehr vielen, dünn besetzten Kategorien, wo ein tiefes Netz die Kombinationen aus zu wenigen Beispielen lernen müsste. H.-T. Cheng *u. a.* [37] nutzen deshalb in ihrem Empfehlungssystem bei Google Play ein breites lineares Modell mit gekreuzten Features, das sich Kombinationen „merkt“, neben einem tiefen Netz, das verallgemeinert.

Zahlenfeatures. Kreuzen kann man nur Kategorien. Eine Zahl wie der Breitengrad wird zuerst in Buckets zerlegt, dann kreuzt man die Buckets von Breite und Länge zu einem Gitter aus Kacheln mit je einem Gewicht.

Trade-offs und Grenzen. Der Preis ist kombinatorisch. Aus Wochentag (7) und Stunde (24) werden 168 Zellen, mit 40 Stadtteilen 6720, mit zusätzlich 12 Fahrzeugtypen 80 640. Je mehr Zellen, desto weniger Beispiele je Zelle, desto unsicherer die Gewichte, und das Modell wächst. Man behilft sich, indem man nur fachlich sinnvolle Paare kreuzt, seltene Zellen verwirft oder das Kreuzprodukt hasht (Pattern 1) beziehungsweise in ein Embedding überführt (Pattern 2). Welche Paare man kreuzt, bestimmt Domänenwissen. Die Einzelgewichte lassen sich nach dem Kreuzen nicht mehr isoliert lesen: Der Beitrag von „Vormittag“ hängt davon ab, welche Zelle gerade aktiv ist.

Wann nicht. Bei Bäumen und Ensembles ist das Pattern meist überflüssig, weil sie durch ihre Verzweigungen Kombinationen selbst bilden, und bei zu dünnen Zellen schadet die Kreuzung mehr, als sie nützt.

Bezug zu Software Engineering. Ein Feature Cross ist der Zustandsraum zweier Zustandsvariablen: Zwei boolesche Variablen ergeben vier Zustände, und die Wechselwirkung liegt in der Zelle, nicht in den Variablen. Dieselbe Explosion kennt man aus dem Testen von Konfigurationen, bei dem man deshalb nicht alle Kombinationen prüft. Und wer schon einmal einen zusammengesetzten Schlüssel (Composite Key) in einer Datenbank gebaut hat, kennt den Gedanken: Die Identität steckt im Tupel, nicht in den Spalten.

> **SO ERKLÄRE ICH ES**
> Zeigen Sie die Vierfeldertafel aus Abbildung 40 und fragen Sie, in welchem Feature allein der Stau erkennbar ist. Die Antwort ist: in keinem. Ein lineares Modell addiert Einflüsse, aber hier steckt die Information im Zusammenspiel, und das Cross macht aus dem Zusammenspiel eine eigene Spalte. Danach die Gegenfrage: Wann bräuchten wir das Cross nicht?
>
> **Verständnisfrage.** Ein Entscheidungsbaum erreicht ohne Cross dieselbe Genauigkeit wie das lineare Modell mit Cross. Warum?
>
> **Antwort.** Weil ein Baum zuerst nach der Richtung und dann innerhalb jedes Zweigs nach der Tageszeit teilt. Damit bildet er genau die Kombinationen, die das Cross von Hand vorgibt.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Feature Cross“; M. Minsky und S. Papert [15], H.-T. Cheng *u. a.* [37].

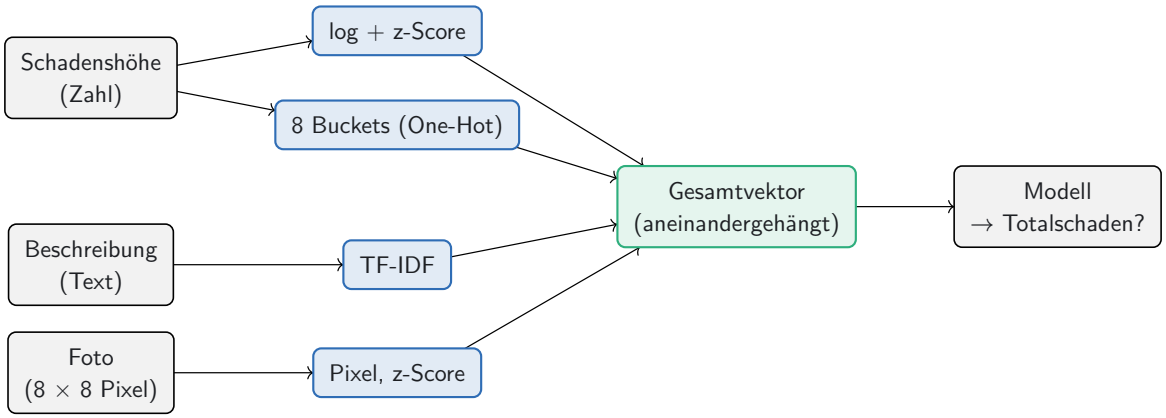


Abbildung 41: Multimodal Input: Jede Quelle bekommt ihre eigene Übersetzung in Zahlen, auch dieselbe Quelle darf mehrfach vorkommen. Danach wird alles zu einem Vektor aneinandergehängt.

5.5 Pattern 4: Multimodal Input

Szenario. Eine Versicherung will aus jeder Schadensmeldung vorhersagen, ob ein Totalschaden vorliegt. Eine Meldung besteht aus der geschätzten Schadenshöhe (Zahl), einer Freitextbeschreibung (Text) und einem Foto (Bild). Wir bauen die Daten im Skript `05_multimodal.py` künstlich: 4000 Meldungen, in jeder der drei Quellen steckt ein schwaches Signal. Das Foto ist hier ein 8 × 8 Pixel großes verrauschtes Graustufenbild und steht stellvertretend für echte Bilder.

Das Problem. Ein Modell erwartet *einen* Vektor aus Zahlen. Die drei Quellen sind aber verschieden: Eine Zahl, eine variable Folge von Wörtern, ein Raster aus Pixeln. Der naheliegende Weg ist, sich auf eine Quelle zu beschränken, meist die, die am leichtesten zu verarbeiten ist. Das verschenkt Information, die in den anderen Quellen steckt. Das zweite Problem ist weniger offensichtlich: Selbst bei *einer* Quelle ist nicht klar, welche Repräsentation die richtige ist. Man kann die Schadenshöhe roh oder logarithmiert eingeben, als Zahl oder in Buckets, einen Text als Wortzählung oder als Embedding.

Die Lösung. Jede Quelle wird in einen eigenen Teilvektor übersetzt, und die Teilvektoren werden hintereinandergehängt (*Concatenate*) zu einem Gesamtvektor, mit dem das Modell arbeitet (Abbildung 41). Die Übersetzung wählt man je Quelle: bei Zahlen Skalierung und Logarithmus, bei Text zum Beispiel TF-IDF (eine gewichtete Wortzählung, die häufige, aussagearme Wörter herunterwichtet), bei Bildern die Pixelwerte oder die Ausgabe eines bildverarbeitenden Netzes. Die zweite Variante des Pattern ist, dieselben Daten mehrfach zu repräsentieren und alle Repräsentationen anzuhängen. Das Modell darf dann wählen, was ihm nützt. In Abbildung 41 ist die Schadenshöhe deshalb zweimal vertreten.

In scikit-learn erledigt das ein `ColumnTransformer`: Er wendet auf jede Spalte oder Spaltengruppe eine eigene Übersetzung an und hängt die Ergebnisse zusammen. Das ganze Gebilde (Übersetzungen plus Modell) ist eine `Pipeline` und wird als eine Einheit trainiert und gespeichert.

```
def modell(teile):
    return make_pipeline(ColumnTransformer(teile), LogisticRegression(max_iter=1000))

tab = ("tabelle", log_zahl, ["schaden"])
buck = ("buckets", bucket, ["schaden"])
txt = ("text", TfidfVectorizer(), "text")
bild = ("bild", StandardScaler(), px)
varianten = {
    "nur Tabelle (Schadenshöhe)": [tab],
    "nur Text (Bag of Words)": [txt],
    "nur Bild (64 Pixel)": [bild],
    "Tabelle + Text": [tab, txt],
    "Tabelle + Text + Bild": [tab, txt, bild],
    "dito, Höhe zusätzlich in Buckets": [tab, buck, txt, bild],
}
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 05_multimodal.py
nur Tabelle (Schadenshöhe)                  Genauigkeit 0.657
nur Text (Bag of Words)                     Genauigkeit 0.683
nur Bild (64 Pixel)                         Genauigkeit 0.627
Tabelle + Text                              Genauigkeit 0.713
Tabelle + Text + Bild                       Genauigkeit 0.733
dito, Höhe zusätzlich in Buckets            Genauigkeit 0.733
```

Jede Quelle allein liefert nur mäßige Ergebnisse (Tabelle 65,7 %, Text 68,3 %, Bild 62,7 %), Tabelle und Text zusammen kommen auf 71,3 %, alle drei auf 73,3 %. Der Zuwachs entsteht, weil die drei Signale unabhängig voneinander Auskunft geben. Bei nur 1200 Testmeldungen sind Unterschiede von ein bis zwei Prozentpunkten allerdings nicht belastbar. Hängt man die Schadenshöhe zusätzlich in Buckets an, ändert sich nichts (73,3 %). Dass die Mehrfachrepräsentation hier wenig bringt, ist ein ehrliches Ergebnis: Sie lohnt sich, wenn der Zusammenhang nicht gleichmäßig verläuft, und das war in unserer Simulation nicht der Fall.

Trade-offs und Grenzen. Der Gesamtvektor ist so lang wie die Summe seiner Teile, und die Teile sind selten gleich lang: Ein Bild mit 10 000 Pixeln liefert 10 000 Spalten neben einer einzelnen Zahl, und bei Abstandsverfahren oder starker Regularisierung bestimmen die vielen Spalten das Ergebnis. Üblich ist, jede Quelle zuerst auf eine feste, vergleichbare Länge zu verdichten, zum Beispiel mit einem Embedding je Quelle. Zweitens sind die Quellen im Betrieb nicht immer alle da: Fehlt das Foto, braucht das Modell eine Vorgabe für diesen Fall, und die muss schon im Training vorkommen, sonst ist es auf die fehlende Quelle nicht vorbereitet. Drittens steigt der Aufwand für die Übersetzungen, und jede muss im Betrieb exakt so laufen wie im Training (Pattern 21). Eine Alternative zum Zusammenhängen vor dem Modell ist, für jede Quelle ein eigenes Modell zu trainieren und die Vorhersagen später zusammenzuführen. Das ist dann ein Ensemble (Pattern 7).

Wann nicht. Wenn eine Quelle fast die gesamte Information trägt, kosten weitere Übersetzungen mehr Wartung, als sie bringen, und wenn nicht sicher ist, dass alle Quellen im Betrieb vorliegen, ist ein Modell mit wenigen robusten Eingaben oft besser.

Bezug zu Software Engineering. Das Pattern ist ein *Adapter* je Datenquelle: Jede Schnittstelle mit eigenem Format wird auf eine gemeinsame abgebildet, hier einen Zahlenvektor. Das entspricht einem Anti-Corruption-Layer, der verhindert, dass die Eigenheiten einer Quelle ins Modell durchschlagen. Die Pipeline, die alle Adapter zusammen mit dem Modell als eine Einheit speichert, ist das, was in der Softwaretechnik ein einzelnes deploybares Artefakt wäre.

> **SO ERKLÄRE ICH ES**
> Ein Modell sieht immer nur eine Liste von Zahlen. Ob diese Zahlen aus einer Tabelle, aus Text oder aus einem Foto stammen, weiß es nicht und braucht es nicht zu wissen. Die Aufgabe des Pattern ist, jede Quelle in Zahlen zu übersetzen und alle Übersetzungen nebeneinanderzulegen. Zeigen Sie die Tabelle der Ergebnisse und lassen Sie die Studierenden erklären, warum die Kombination besser ist als jede Quelle allein.
>
> **Verständnisfrage.** Ein Bild liefert 10 000 Pixelwerte, die Schadenshöhe eine einzige Zahl. Was kann beim Zusammenhängen passieren, und was tut man dagegen?
>
> **Antwort.** Die 10 000 Pixelspalten dominieren den Vektor, und die eine Zahl geht unter. Man verdichtet jede Quelle zuerst auf eine vergleichbare Länge, etwa mit einem Embedding, oder skaliert beide Teile passend.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Multimodal Input“. Für das Beispiel gibt es keine Primärquelle, es ist für das Handbuch konstruiert.

Tabelle 9: Die sechs Patterns dieses Kapitels im Überblick.

Pattern	Situation	Kernidee
5 Reframing	Der Zielwert ist zweigipflig oder hat Spielräume, ein Punktwert führt in die Irre.	Aus Regression Klassifikation über Wertebereiche machen, das Modell liefert eine Verteilung.
6 Multilabel	Ein Beispiel gehört zu mehreren Klassen zugleich.	Pro Klasse ein eigener Ja/Nein-Ausgang (Sigmoid) statt eines gemeinsamen (Softmax).
7 Ensembles	Ein einzelnes Modell ist zu ungenau oder zu schwankend.	Mehrere Modelle trainieren, Vorhersagen kombinieren.
8 Cascade	Ein Modell vermischt Fallgruppen, die sich gegensätzlich verhalten; die Aufgabe zerfällt in Teilfragen.	Modelle hintereinander schalten: das erste entscheidet, welcher Spezialist weiterarbeitet.
9 Neutral Class	Menschen sind sich beim Labeln uneinig, die Grenze ist unscharf.	Eine dritte Klasse für „weder noch“ einführen.
10 Rebalancing	Eine Klasse ist extrem selten (unter 1 %).	Daten, Gewichte oder Schwelle anpassen und die richtigen Metriken verwenden.

6 Problem Representation

Bevor man ein Modell trainiert, muss man festlegen, *welche Frage* es beantworten soll. Aus einem fachlichen Problem lassen sich mehrere ML-Aufgaben machen, und die Wahl entscheidet, was das Modell ausgibt, welche Fehlerfunktion es minimiert und welche Fehler es überhaupt machen kann. Die sechs Patterns aus Kapitel 3 des Buchs [1] handeln davon, ein Problem so zu formulieren, dass ein Modell es lösen kann.

Zwei Metriken braucht man in diesem Kapitel ständig, deshalb vorab. Wenn ein Modell zu jedem Beispiel „positiv“ oder „negativ“ sagt, ist die *Precision* der Anteil der als positiv gemeldeten Beispiele, die tatsächlich positiv sind („wie viele meiner Alarme waren berechtigt?“), und der *Recall* der Anteil der tatsächlich positiven Beispiele, die das Modell gefunden hat („wie viele echte Fälle habe ich erwischt?“). Beide ziehen in entgegengesetzte Richtungen: Wer mehr Fälle melden will, muss mehr Fehlalarme in Kauf nehmen. Der *F1-Wert* fasst beide als harmonisches Mittel zusammen:

$$F_1 = \frac{2PR}{P + R}.$$

6.1 Pattern 5: Reframing

Szenario. Ein Lieferdienst will seinen Kunden die Lieferzeit anzeigen. Bekannt ist bei der Bestellung nur die Entfernung. Was der Dienst nicht weiß: Ob die Küche gerade überlastet ist. Im Skript `01_reframing.py` entstehen 20 000 künstliche Lieferungen. Normalerweise dauert es 15 Minuten plus 3 Minuten je Kilometer, bei überlasteter Küche kommen 40 Minuten dazu, und das passiert in der Hälfte der Fälle.

Das Problem. Das naheliegende Modell ist eine Regression: eine Zahl vorhersagen, die Lieferzeit in Minuten. Eine Regression minimiert in der Regel den mittleren quadratischen Fehler, und der ist am kleinsten, wenn die Vorhersage dem Mittelwert der möglichen Zielwerte entspricht. Bei 5 Kilometern liegen die tatsächlichen Zeiten entweder um 30 oder um 70 Minuten. Der Mittelwert ist 50 Minuten, und die Auswertung zeigt: kein einziger der Fälle zwischen 4,5 und 5,5 Kilometern liegt innerhalb von fünf Minuten um diese Vorhersage (0,0 %). Die Zahl ist im Mittel richtig und in jedem Einzelfall falsch. Der Kunde bekommt eine Angabe, die nie eintritt, und erfährt nichts von dem Risiko.

Die Lösung. Das Pattern formuliert das Problem um. Statt eine Zahl vorherzusagen, zerlegt man den Wertebereich in Klassen (*Bins*), hier 10-Minuten-Abschnitte, und trainiert einen Klassifikator, der für jede Klasse eine Wahrscheinlichkeit liefert. Das Ergebnis ist keine Zahl mehr, sondern eine Verteilung. Der Klassifikator hat keine Annahme über die Form der Verteilung nötig und kann deshalb auch zwei Gipfel darstellen.

```
# 3) Reframing: Bins von 10 Minuten, Klassifikator liefert eine Verteilung
kanten = np.arange(20, 101, 10)
bins = np.digitize(minuten, kanten) - 1                # 0 = unter 30 usw.
bins = np.clip(bins, 0, len(kanten) - 1)
clf = RandomForestClassifier(n_estimators=100, min_samples_leaf=100,
                             random_state=0).fit(X, bins)
p = clf.predict_proba([[5.0]])[0]
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 01_reframing.py
Regression für 5 km: 50 min
Anteil echter Lieferungen (4,5-5,5 km) innerhalb ±5 min um diese Vorhersage: 0.0%
Quantile (10 %, 50 %, 90 %): {0.1: 27, 0.5: 63, 0.9: 73}

Verteilung für 5 km (Bin-Untergrenze: Wahrscheinlichkeit):
  ab  20 min: 21.4%
  ab  30 min: 20.2%
  ab  40 min:  0.0%
  ab  50 min:  0.0%
  ab  60 min: 27.4%
  ab  70 min: 31.1%
  ab  80 min:  0.0%
  ab  90 min:  0.0%
```

In Abbildung 42 stehen die Wahrscheinlichkeiten des Klassifikators neben den tatsächlichen Anteilen aus den Daten. Der Klassifikator findet die beiden Gipfel (30 und 70 Minuten) und die Lücke dazwischen. Daraus lassen sich bei Bedarf wieder Punktwerte ableiten, etwa der wahrscheinlichste Abschnitt, aber auch eine ehrliche Aussage wie „mit etwas über 40 % unter 40 Minuten und mit knapp 60 % über 60 Minuten“.

Alternative: Quantil-Regression. Eine zweite Möglichkeit, Unsicherheit auszudrücken, ist die Quantil-Regression [38]: Das Modell sagt nicht den Mittelwert, sondern zum Beispiel das 10- und das 90-Prozent-Quantil voraus (der Wert, unter dem 10 % beziehungsweise 90 % der Fälle liegen). Bei 5 km ergeben sich 27 und 73 Minuten, also ein Intervall, in dem 80 % der Lieferungen liegen. Das ist kompakter als eine Verteilung, zeigt aber nicht, dass dazwischen kaum etwas passiert. Das 50-Prozent-Quantil (Median) ist bei zwei gleich großen Gipfeln kaum bestimmt, weil jeder Wert in der Lücke zwischen ihnen die Daten

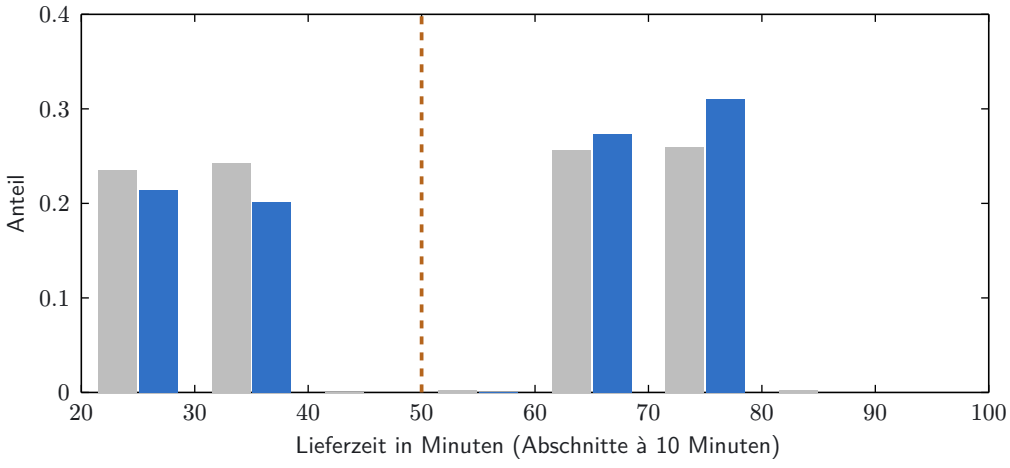


Abbildung 42: Die Verteilung der Lieferzeit bei 5 km. Die Regression sagt 50 Minuten voraus (orange), einen Wert in der Lücke zwischen den beiden Gipfeln. Der Klassifikator über 10-Minuten-Abschnitte (blau) trifft die tatsächlichen Anteile (grau) gut. Die erste Klasse fasst alles unter 30 Minuten zusammen.

ungefähr in zwei Hälften teilt, und damit wenig aussagekräftig; das geschätzte Ergebnis (63 Minuten) ist ein Zufallsprodukt der Schätzung und kein Befund.

Trade-offs und Grenzen. Die Breite der Abschnitte ist ein Kompromiss: grobe verlieren Genauigkeit innerhalb der Klasse, feine haben zu wenige Beispiele je Klasse. Der Klassifikator kennt keine Reihenfolge der Klassen und muss die Nachbarschaft aus den Daten lernen. Eine Verteilung zu schätzen verlangt mehr Daten als ein Mittelwert, die Wahrscheinlichkeiten sind nicht automatisch *kalibriert* (zuverlässig), und die Benutzeroberfläche muss die Verteilung sinnvoll darstellen.

Wann nicht. Ist die Verteilung eingipflig und annähernd symmetrisch, reicht eine Regression. Auch wenn die Anwendung wirklich nur einen Zahlenwert braucht und symmetrische Fehler hinnehmbar sind, spart man sich den Umweg. Umgekehrt kann man ein Problem, dessen Klassen eine Ordnung haben (Sterne von 1 bis 5), als Regression formulieren und so die Ordnung nutzen, die eine Klassifikation ignoriert.

Bezug zu Software Engineering. Reframing ändert den Vertrag der Schnittstelle: Statt `predict() -> float` gibt es `predict() -> Distribution`. Das ist verwandt mit ehrlichen Rückgabetypen wie `Optional` oder `Result`, die Unsicherheit in den Typ schreiben, statt Genauigkeit vorzutäuschen. Eine falsch formulierte Aufgabe behebt man nicht durch bessere Hyperparameter, so wie sich eine falsch geschnittene API nicht durch eine bessere Implementierung rettet.

> **SO ERKLÄRE ICH ES**
> Schreiben Sie an die Tafel: „Die Lieferung dauert im Schnitt 50 Minuten.“ Fragen Sie, wann das stimmt und wann nicht. Zeigen Sie dann Abbildung 42. Der Mittelwert beantwortet eine andere Frage als die, die der Kunde stellt. Die Umformulierung zwingt das Modell, die ganze Verteilung zu beschreiben und nicht nur ihre Mitte.
>
> **Verständnisfrage.** Wann ist eine Regression trotz allem die bessere Wahl?
>
> **Antwort.** Wenn die Verteilung der Zielgröße eingipflig und annähernd symmetrisch ist. Dann liegt der Mittelwert in der Nähe der tatsächlichen Werte, und eine einzelne Zahl ist eine brauchbare Antwort.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Reframing“; R. Koenker und G. Bassett [38] für die Quantil-Regression.

6.2 Pattern 6: Multilabel

Szenario. Eine Fotoplattform soll Bilder automatisch verschlagworten: Strand, Berge, Stadt, Menschen, Hund. Ein Foto kann mehrere Schlagworte tragen, „Strand“ und „Hund“ zum Beispiel. Im Skript `02_multilabel.py` erzeugt scikit-learn dafür 4000 künstliche Beispiele mit fünf Labels, im Mittel 2,25 pro Beispiel. Die Daten sind bewusst abstrakt; die Namen der Labels sind hier nur eine Gedankenstütze.

Das Problem. Das Standardwerkzeug für Klassifikation ist die *Softmax*-Funktion am Ausgang des Modells. Sie macht aus beliebigen Zahlen positive Werte, die sich zu genau 1 summieren, also eine Wahrscheinlichkeitsverteilung über Klassen, die sich gegenseitig ausschließen: Jedes Beispiel gehört genau einer Klasse an. Für ein Foto mit Strand und Hund müssten sich beide Klassen die Wahrscheinlichkeit teilen, etwa 0,5 zu 0,5, und jede zusätzliche Klasse drückt die anderen weiter nach unten, obwohl „ist ein Hund zu sehen?“ und „ist ein Strand zu sehen?“ unabhängige Fragen sind. Trainiert man ein Softmax-Modell trotzdem auf solchen Daten (hier mit einem zufällig gewählten der Labels je Beispiel) und nimmt die wahrscheinlichste Klasse als Antwort, bleibt höchstens ein Label je Beispiel richtig: Der F1-Wert über alle Labels sinkt auf 0,58 (gegen 0,81 beim Sigmoid-Netz weiter unten).

Die Lösung. Das Pattern gibt jedem Label seinen eigenen Ausgang mit einer *Sigmoid*-Funktion. Sie bildet eine beliebige Zahl auf einen Wert zwischen 0 und 1 ab und beantwortet eine einzelne Ja/Nein-Frage; jeder Ausgang ist unabhängig von den anderen, und die Summe ist beliebig (Abbildung 43). Das Ziel eines Beispiels ist kein einzelner Klassenname mehr, sondern ein Vektor aus Nullen und Einsen („Multi-Hot“: `[1, 0, 0, 1, 0]` heißt Label 0 und 3). Als Fehlerfunktion dient die binäre Kreuzentropie je Ausgang, aufsummiert (die übliche Fehlerfunktion für Ja/Nein-Ausgaben: Sie bestraft ein sicheres, aber falsches Urteil besonders stark). Die Entscheidung „Label vergeben oder nicht“ trifft ein Schwellwert, anfangs 0,5.

Softmax: eine Verteilung, Summe = 1

Label 0	1	2	3	4
0,12	0,25	0,05	0,55	0,03

Summe 1, wahr: 1 0 0 1 0

Sigmoid je Label: fünf Einzelfragen

Label 0	1	2	3	4
0,45	0,28	0,14	0,98	0,07

Summe 1,92, wahr: 1 0 0 1 0

Abbildung 43: Softmax gegen Sigmoid. Beide Seiten zeigen die echten Ausgaben für dasselbe Testbeispiel (ein Foto mit den Labels 0 und 3): links das Softmax-Modell, rechts das Netz mit Sigmoid.

```
# A) Sigmoid pro Label: bei Multilabel-Zielen nutzt MLPClassifier Sigmoid
# am Ausgang und binäre Kreuzentropie als Fehlerfunktion
net = MLPClassifier(hidden_layer_sizes=(32,), max_iter=1000,
                    random_state=0).fit(Xtr, Ytr)
P = net.predict_proba(Xte)
print("\nSigmoid-Ausgaben für 3 Beispiele (Summe muss nicht 1 sein):")
for p, y in zip(P[:3], Yte[:3]):
    print(" ", p.round(2), "Summe", p.sum().round(2), "| wahr", y)
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 02_multilabel.py
Labels pro Beispiel im Mittel: 2.25 | verschiedene Label-Kombinationen: 31

Sigmoid-Ausgaben für 3 Beispiele (Summe muss nicht 1 sein):
  [0.04 0.01 1.   0.   0.  ] Summe 1.05 | wahr [0 0 1 0 0]
  [0.45 0.28 0.14 0.98 0.07] Summe 1.91 | wahr [1 0 0 1 0]
  [0.55 0.09 0.29 0.96 0.13] Summe 2.03 | wahr [1 0 0 1 0]

Sigmoid, Schwelle 0,5       Hamming 0.175  F1 micro 0.805  macro 0.802  exakt 0.443
gewählte Schwellen: [0.3  0.28 0.45 0.3  0.38]
Sigmoid, Schwelle je Label  Hamming 0.179  F1 micro 0.818  macro 0.819  exakt 0.405
Softmax (nur Top-1)         Hamming 0.274  F1 micro 0.579  macro 0.578  exakt 0.302
  Softmax-Ausgabe, zweites Testbeispiel: [0.12 0.25 0.05 0.55 0.03] Summe 1.0
Label Powerset (31 Klassen) Hamming 0.187  F1 micro 0.783  macro 0.780  exakt 0.468
```

Die Tabelle am Ende der Ausgabe vergleicht vier Wege. Das Netz mit Sigmoid und Schwelle 0,5 erreicht einen F1 micro (alle Label-Entscheidungen zusammengezählt) von 0,805. Die *Hamming-Loss* ist der Anteil falscher Einzelentscheidungen, hier 17,5 %, und „exakt" der Anteil der Beispiele, bei denen *alle* fünf Entscheidungen stimmen (44,3 %). Der Softmax-Weg fällt deutlich ab. Der vierte Weg, das *Label Powerset*, macht aus jeder vorkommenden Kombination eine eigene Klasse (hier 31) und trainiert ein gewöhnliches Softmax-Modell. Er gewinnt bei „exakt" (46,8 %), ist aber bei F1 micro schlechter als das Sigmoid-Netz, und seine Klassenzahl kann bei K Labels bis auf 2^K wachsen, sodass seltene Kombinationen kaum Beispiele haben.

Schwellwerte. Der Schwellwert 0,5 ist eine Voreinstellung, kein Naturgesetz. Im Skript wird für jedes Label auf einer eigenen Validierungsmenge der Schwellwert gewählt, der den F1-Wert dieses Labels maximiert; es ergeben sich Werte zwischen 0,28 und 0,45. Der F1 micro steigt auf 0,818, aber „exakt" sinkt von 44,3 % auf 40,5 %. Beides ist richtig: Die Metriken messen Verschiedenes, und welche zählt, ist eine fachliche Entscheidung. Für ein Verschlagwortungssystem, bei dem ein fehlendes Schlagwort weniger schadet als ein falsches, würde man die Schwellen eher anheben.

Trade-offs und Grenzen. Unabhängige Sigmoids behandeln jedes Label für sich. Dass „Strand" und „Meer" meist zusammen auftreten, kennt das Netz nur indirekt über die gemeinsamen versteckten Schichten. Die Schwellwerte muss man wählen und pflegen. Ein fehlendes Label in den Trainingsdaten ist oft kein Nein, sondern ein „nicht geprüft", und das Modell lernt aus beidem dasselbe. Und je Label kann die Klasse extrem selten sein (Pattern 10).

Wann nicht. Schließen sich die Klassen aus (eine Ziffer ist genau eine Ziffer), ist Softmax richtig und das Modell dadurch besser eingeschränkt. Gleiches gilt, wenn man genau *eine* Antwort braucht, etwa die Zielkategorie eines Tickets.

Bezug zu Software Engineering. Softmax entspricht einem `enum`: Ein Wert aus einer festen Menge. Sigmoid entspricht einem Flags-Enum oder einer Menge von Tags: Jedes Flag ist einzeln gesetzt oder nicht. Wer von `enum` auf Flags wechselt, ändert den Typ der Schnittstelle, und das merken alle Aufrufer; so ist es auch hier.

> **SO ERKLÄRE ICH ES**
> Fragen Sie, ob ein Foto von Hund und Strand zu 100 % ein Hundefoto und zu 100 % ein Strandfoto sein kann. Mit Softmax ist das unmöglich, weil sich die Wahrscheinlichkeiten zu 1 summieren. Sigmoid stellt jedem Label eine eigene Frage, und die Antworten dürfen sich zu mehr als 1 summieren. Zeigen Sie die Beispielausgabe aus Abbildung 43 mit der Summe über 1.
>
> **Verständnisfrage.** Warum steigt der F1 micro, wenn man die Schwelle je Label wählt, aber der Anteil der exakt richtigen Beispiele sinkt?
>
> **Antwort.** Niedrigere Schwellen finden mehr der tatsächlich vorhandenen Labels und erhöhen den Recall und damit den F1 je Label. Gleichzeitig entstehen mehr zusätzliche falsche Labels, und jedes davon macht ein sonst exaktes Beispiel unexakt.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Multilabel“; G. Tsoumakas und I. Katakis [39] für einen Überblick, darin auch das Label Powerset.

6.3 Pattern 7: Ensembles

Szenario. Ein Telekommunikationsanbieter will vorhersagen, welche Kunden kündigen. Im Skript `03_ensembles.py` stehen 2000 künstliche Kundendatensätze mit 20 Features bereit, von denen 8 wirklich informativ sind, und bei 8 % der Datensätze wird das Label zufällig neu gewürfelt (`flip_y=0.08`); falsch ist davon etwa die Hälfte, also rund 4 % aller Labels, wie es bei echten Daten durch Fehler im Datensatz vorkommt. Als Maß dient die Genauigkeit (Anteil richtiger Vorhersagen) in fünffacher Kreuzvalidierung: Die Daten werden fünfmal in Trainings- und Testteil zerlegt, und man mittelt über die fünf Durchgänge. Die Angabe nach dem „±“ ist die Standardabweichung über die fünf Durchgänge.

Das Problem. Der Fehler eines Modells setzt sich (bei quadratischem Fehler; bei Klassifikation gilt es sinngemäß) aus drei Teilen zusammen. Der *Bias* ist der systematische Fehler eines zu einfachen Modells: Es kann das Muster nicht abbilden, selbst mit unendlich vielen Daten. Die *Varianz* misst, wie stark das Modell vom Zufall der Trainingsdaten abhängt: Ein anderer Datensatz derselben Art ergäbe ein anderes Modell. Der Rest ist unvermeidliches Rauschen. Ein tiefer Entscheidungsbaum hat wenig Bias und viel Varianz. Er teilt die Daten so lange, bis jeder Datensatz in einem eigenen Blatt liegt, und lernt dabei auch die falschen Labels auswendig. Das Ergebnis liegt in der Ausgabe unten bei etwa 0,77 Genauigkeit.

Die Lösung. Wenn ein einzelnes Modell instabil ist, trainiert man mehrere und kombiniert ihre Vorhersagen. Dahinter stehen drei Verfahren (Abbildung 44).

Bagging (von *bootstrap aggregating*, L. Breiman [3]) zieht aus den Trainingsdaten mehrfach Stichproben gleicher Größe *mit Zurücklegen* (*Bootstrap*), trainiert auf jeder ein eigenes Modell und lässt abstimmen oder mittelt. Weil jede Stichprobe etwas anderes enthält, machen die Modelle teilweise unterschiedliche Fehler, die sich im Durchschnitt aufheben. Bagging senkt also vor allem die Varianz. Der *Random Forest* [4] ist Bagging mit Entscheidungsbäumen, bei dem jede Verzweigung nur aus einer zufälligen Auswahl der Features wählen darf; das macht die Bäume noch unähnlicher.

Boosting trainiert die Modelle nacheinander. Jedes neue Modell konzentriert sich auf das, was die bisherigen falsch machen. AdaBoost [40] erreicht das, indem es falsch klassifizierte Datensätze höher gewichtet. *Gradient Boosting* [5] lässt das nächste Modell den verbleibenden Fehler der bisherigen Summe vorhersagen (genauer: die negative Steigung, den Gradienten, der Fehlerfunktion). Die einzelnen Modelle sind bewusst einfach (flache Bäume oder „Stümpfe“ mit einer einzigen Verzweigung), und die Summe senkt den Bias. Bei verrauschten Daten kann man mit zu vielen Runden auch das Rauschen lernen.

Stacking [41] kombiniert Modelle *verschiedener Art*. Ein zusätzliches Meta-Modell lernt, wie viel Gewicht jedem Basismodell zusteht. Wichtig: Das Meta-Modell muss mit Vorhersagen trainiert werden, die die Basismodelle für Daten gemacht haben, die sie *nicht* gesehen haben (*out-of-fold*), sonst belohnt es

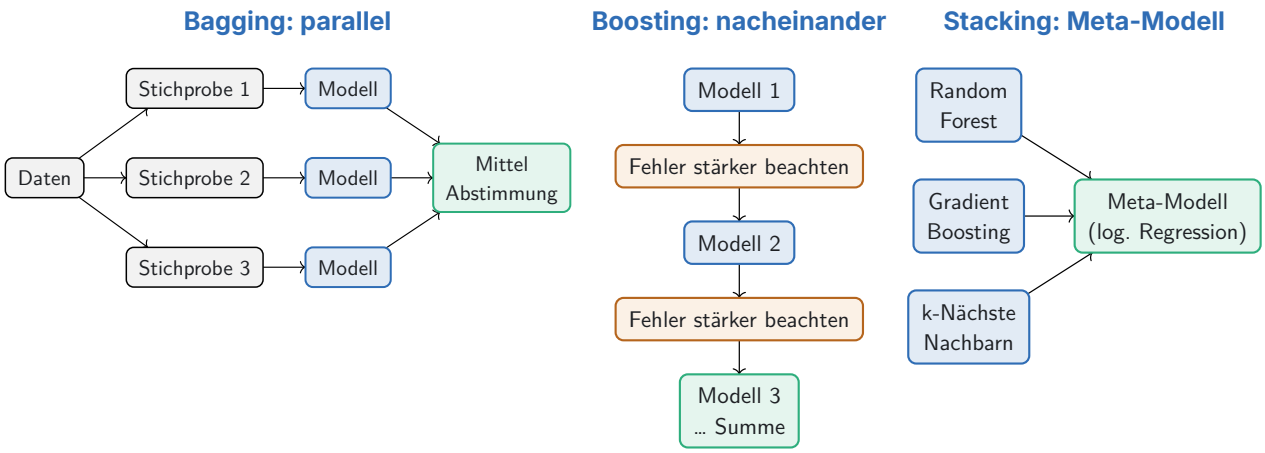


Abbildung 44: Drei Wege, Modelle zu kombinieren. Bagging trainiert parallel auf verschiedenen Stichproben, Boosting nacheinander mit wachsendem Gewicht auf den Fehlern, Stacking kombiniert verschiedene Modelltypen über ein Meta-Modell.

Modelle, die die Trainingsdaten auswendig kennen. `StackingClassifier` aus scikit-learn erledigt das über den Parameter `cv`.

```
stack = StackingClassifier(
    estimators=[("rf", RandomForestClassifier(100, random_state=0)),
                ("gb", GradientBoostingClassifier(random_state=0)),
                ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(15)))],
    final_estimator=LogisticRegression(), cv=5)
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 03_ensembles.py
ein Entscheidungsbaum              Genauigkeit 0.771 ± 0.023
Logistische Regression             Genauigkeit 0.828 ± 0.019
Bagging (100 Bäume)                Genauigkeit 0.848 ± 0.024
Random Forest (100 Bäume)          Genauigkeit 0.854 ± 0.023
AdaBoost (100 Stümpfe)             Genauigkeit 0.814 ± 0.021
Gradient Boosting (100 Bäume)      Genauigkeit 0.858 ± 0.022
Stacking (RF + GB + kNN -> LR)     Genauigkeit 0.865 ± 0.021
```

Bagging, Random Forest, Gradient Boosting und Stacking schlagen den einzelnen Baum (0,771) um acht bis neun Prozentpunkte: Bagging 0,848, Random Forest 0,854, Gradient Boosting 0,858, Stacking 0,865. Dieser Abstand ist größer als die Streuung zwischen den fünf Durchgängen (rund 0,02), die Unterschiede *innerhalb* der Ensembles sind es nicht. Welches der vier das beste ist, lässt sich aus diesem Versuch nicht ablesen. Zwei weitere Zeilen sind lehrreich. Die einfache logistische Regression (0,828) liegt vor dem tiefen Baum, auf diesen Daten ist also kein komplexes Modell nötig, um den Baum zu schlagen. Und AdaBoost mit Stümpfen (0,814) liegt unter der logistischen Regression: Ein Ensemble aus vielen schwachen Teilen ist nicht automatisch besser als ein gut gewähltes einzelnes Modell.

Trade-offs und Grenzen. Ein Ensemble ist so teuer wie die Summe seiner Teile: 100 Bäume brauchen grob das Hundertfache an Zeit und Speicher für eine Vorhersage, und das Ergebnis lässt sich schwerer erklären als ein einzelner Baum. Es hilft nur, wenn die Modelle *unterschiedliche* Fehler machen; deshalb arbeiten Bagging und Random Forest mit Zufall. Der Gewinn ist oft kleiner, als man hofft: Im Versuch bringt das Stacking gegenüber Gradient Boosting weniger als einen Prozentpunkt, weniger als die Streuung, und kostet drei Modelle plus Meta-Modell.

Wann nicht. Wenn ein einfaches Modell schon genügt, wenn die Antwortzeit knapp bemessen ist oder wenn jede Vorhersage erklärbar sein muss, ist ein Ensemble die falsche Antwort.

Bezug zu Software Engineering. Das Pattern ist Redundanz mit Mehrheitsentscheid. Man kennt es aus der Fehlertoleranz: mehrere unabhängig gebaute Versionen derselben Funktion, deren Ergebnis ein Voter zusammenführt (*N-Version Programming*, eine Technik für sicherheitskritische Systeme). Die Voraussetzung ist dort dieselbe wie hier: Die Fehler der Teile dürfen nicht dieselben sein. Der Unterschied zu Pattern

8 (Cascade) liegt in der Verschaltung: Beim Ensemble sehen alle Modelle jeden Fall und stimmen ab, bei der Cascade bestimmt das erste Modell, welches andere Modell einen Fall überhaupt sieht.

> **SO ERKLÄRE ICH ES**
> Stellen Sie eine Jury vor: Ein einzelner Juror ist launisch, jeder entscheidet etwas anders, je nachdem, welche Bewerber er zufällig zuerst gesehen hat. Fragen Sie in die Runde, ob eine Jury aus hundert launischen Jurorinnen zuverlässiger entscheidet als eine einzelne, wenn ihre Launen unabhängig sind. Bagging ist diese Jury. Boosting ist ein Juror, der aus den Fehlern des vorigen Jurors lernt, Stacking ein Vorsitzender, der gelernt hat, wem er bei welchen Fällen am meisten vertraut.
>
> **Verständnisfrage.** Warum bringt ein Ensemble aus hundert identisch trainierten Modellen nichts?
>
> **Antwort.** Weil alle dieselben Fehler machen. Das Mittel von hundert gleichen Antworten ist dieselbe Antwort. Nur wenn die Fehler der Modelle unabhängig sind (andere Stichproben, andere Features, andere Modelltypen), heben sie sich teilweise auf.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Ensembles“; L. Breiman [3] (Bagging) und [4] (Random Forests); Y. Freund und R. E. Schapire [40] (AdaBoost); J. H. Friedman [5] (Gradient Boosting); D. H. Wolpert [41] (Stacking). Implementierung: scikit-learn [42].

6.4 Pattern 8: Cascade

Szenario. Eine Versicherung prüft Schadensmeldungen auf Betrug. 75 % der Meldungen kommen von Privatkunden, 25 % von Werkstätten, die für viele Kunden abrechnen. Im Skript `04b_cascade_router.py` entstehen 40 000 künstliche Meldungen, 4 % davon Betrug. Betrug zeigt sich in beiden Gruppen an demselben Feature x_0, aber in entgegengesetzter Richtung: Bei Privatkunden ist ein hoher Wert verdächtig, bei Werkstätten ein niedriger. Die Daten sind erfunden, es geht um die Struktur. Zu welcher Gruppe eine Meldung gehört, ist im Betrieb nicht zuverlässig bekannt; drei weitere Features (r) geben nur Hinweise, und das System muss die Gruppe daraus schätzen.

Das Problem. Ein lineares Modell gibt x_0 ein einziges Gewicht und damit ein einziges Vorzeichen. Was bei Privatkunden Betrug anzeigt, würde bei Werkstätten Unverdächtige belasten, und das Modell findet einen Kompromiss, der keiner Gruppe gerecht wird. Allgemein gilt: Soll ein Modell zwei Teilpopulationen mit verschiedenen Mustern zugleich lernen, lernt es einen Durchschnitt. Es ist dieselbe Struktur wie bei Pattern 3: Der Einfluss eines Features hängt von einem anderen Feature (hier dem Segment) ab.

Die Lösung. Das Pattern zerlegt die Aufgabe in eine Kette von Modellen, bei der die Ausgabe des einen bestimmt, was das nächste tut (Abbildung 45). Ein erstes Modell, der *Router*, beantwortet die Vorfrage „Welche Art von Fall ist das?“. Danach arbeitet ein *Spezialist*, der nur Fälle dieser Art gelernt hat, und macht die eigentliche Vorhersage. Der Router ist selbst ein Klassifikator mit eigener Fehlerrate und eigenen Metriken. Im Training lernt der Router das bekannte Segment aus r, jeder Spezialist lernt auf den Fällen seines Segments.

```
# c) Cascade: Router (Segment aus r) und je ein linearer Spezialist, trainiert auf dem WAHREN
Segment
router = LogisticRegression().fit(r[tr], werkstatt[tr])
spez = {s: LogisticRegression().fit(x[tr][werkstatt[tr] == s], betrug[tr][werkstatt[tr] == s])
for s in (0, 1)}
seg_hat = router.predict(r[te])
print(f"Router-Genauigkeit auf den Testdaten: {np.mean(seg_hat == werkstatt[te]):.1%}")

def kaskade(seg):
    s = np.zeros(len(te))
    for k in (0, 1):
        m = seg == k
        s[m] = spez[k].predict_proba(x[te][m])[:, 1]
    return s

ergebnis.append(bewerte("Cascade: Router + 2 Spezialisten", kaskade(seg_hat)))
ergebnis.append(bewerte("Cascade mit wahrem Segment (Orakel)", kaskade(werkstatt[te])))
```

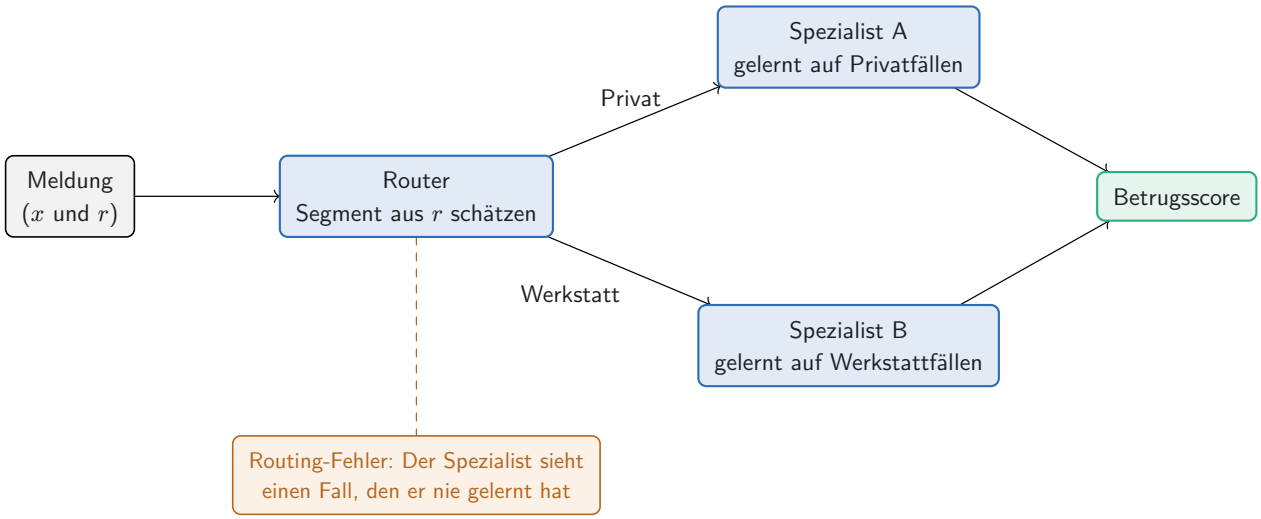


Abbildung 45: Cascade als Router mit zwei Spezialisten. Orange: Was der Router falsch einordnet, sieht der falsche Spezialist.

Terminal

```
$ uv run --with scikit-learn --with numpy python 04b_cascade_router.py
ein lineares Modell auf allen              ROC-AUC 0.641  PR-AUC 0.133
ein Baummodell auf x und r                 ROC-AUC 0.737  PR-AUC 0.150
Router-Genauigkeit auf den Testdaten: 91.7%
Cascade: Router + 2 Spezialisten           ROC-AUC 0.759  PR-AUC 0.180
Cascade mit wahrem Segment (Orakel)        ROC-AUC 0.805  PR-AUC 0.197
falsch geroutet: 8.3%; AUC nur über diese Fälle: 0.237
```

Das lineare Einzelmodell erreicht eine ROC-AUC von 0,641 (0,5 wäre Raten), die Cascade 0,759. Wüsste man das Segment sicher (Zeile „Orakel“, im Betrieb nicht verfügbar), wären es 0,805; die Differenz geht auf das Konto der Routing-Fehler. Der Router liegt in 91,7 % der Fälle richtig. Für die übrigen 8,3 % ist das Ergebnis schlimmer als Raten: Der Spezialist wendet ein Muster an, das für diese Gruppe umgekehrt gilt, und die AUC nur über diese Fälle beträgt 0,237.

Ein ehrlicher Vergleich gehört dazu. Ein Baummodell (`HistGradientBoostingClassifier`), das x und r ohne Router bekommt, erreicht 0,737, nicht weit von der Cascade entfernt. Bei rund 640 Betrugsfällen in den Testdaten ist der Abstand von zwei AUC-Punkten nicht belastbar. Wo ein flexibles Modell die Wechselwirkung selbst lernen kann, ist ein einzelnes Modell die einfachere Lösung. Die Cascade lohnt sich eher, wenn die Spezialisten bewusst einfach und erklärbar bleiben sollen oder die Gruppen verschiedene Daten, Features oder Teams haben.

Fehlerfortpflanzung und Trainingsfalle. Die Kette erbt die Fehler des Routers. Sind Router und Spezialist unabhängig zu 95 % und 90 % richtig und verdirbt jeder Routing-Fehler das Ergebnis, ist die Kette höchstens in 85,5 % der Fälle richtig. Dazu kommt eine Falle beim Training: Im Skript lernen die Spezialisten auf dem *wahren* Segment, im Betrieb bekommen sie aber alles, was der Router *sagt*. Training und Betrieb sehen dann verschiedene Daten, ein kleiner Training-Serving Skew innerhalb der Kette. Trainiert man die Spezialisten stattdessen auf den Router-Ausgaben, ist jede Änderung am Router ein Grund, sie neu zu trainieren. Man prüft deshalb die ganze Kette Ende zu Ende und protokolliert die Router-Entscheidung zu jeder Vorhersage, damit man Fehlleitungen später auswerten kann.

Verwandt: die Konfidenz-Kaskade

Derselbe Name steht auch für eine Technik mit anderem Ziel: nicht Spezialisierung, sondern Kosten. Ein billiges Modell entscheidet die klaren Fälle, nur die unsicheren gehen an ein teures Modell. Das klassische Beispiel ist die Gesichtserkennung von P. Viola und M. Jones [43]: Billige frühe Stufen verwerfen die allermeisten Bildausschnitte, nur wenige Kandidaten erreichen die teuren späteren Stufen. Hier das Prinzip an Kartenzahlungen (Abbildung 46, Skript `04_cascade.py`: 60 000 künstliche Zahlungen, 1 % Betrug; vier billige Features stehen in der Zahlung, 16 teure müssen bei anderen Diensten abgefragt werden). Die erste Stufe, eine logistische Regression auf den billigen Features, bekommt eine niedrige Schwelle: Im

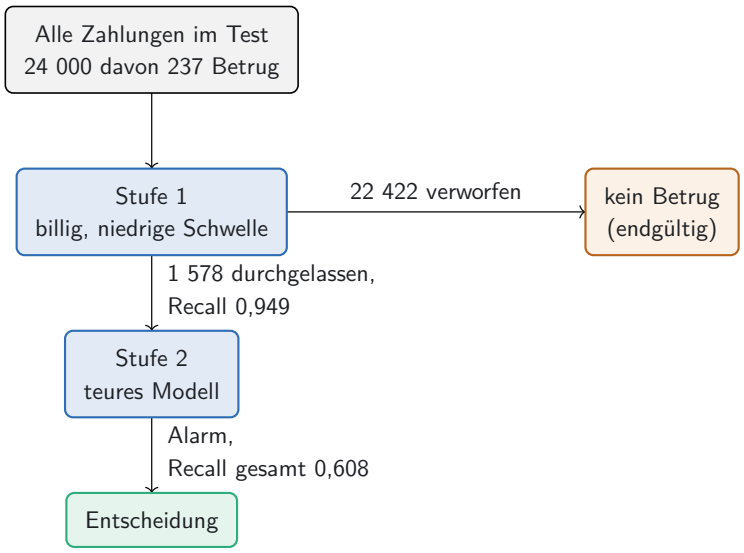


Abbildung 46: Die Konfidenz-Kaskade im Skript. Die Zahlen sind die echten Werte aus `04_cascade.py`. Was Stufe 1 verwirft, sieht Stufe 2 nie.

Training liegen fünf Prozent der Betrugsfälle unter ihr, sie lässt also fast alle echten Fälle durch und dafür viele harmlose. Auf den Testdaten sind das 6,6 % aller Zahlungen, darunter 94,9 % der Betrugsfälle.

```
# Stufe 2: stärkeres Modell auf allen Merkmalen, trainiert nur auf dem, was
# Stufe 1 durchlässt (so sieht es im Betrieb auch aus)
s2 = HistGradientBoostingClassifier(max_iter=100, random_state=0)
s2.fit(Xtr[durch_tr], ytr[durch_tr])
pred = np.zeros(len(yte), dtype=int)
pred[durch_te] = s2.predict(Xte[durch_te])
print(f"Stufe 2 auf den Durchgelassenen: "
      f"Recall {recall_score(yte[durch_te], pred[durch_te]):.3f}")
print(f"Kaskade gesamt: Recall {recall_score(yte, pred):.3f}"
      f"  Precision {precision_score(yte, pred):.3f}")
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 04_cascade.py
positive Beispiele: 0.99% (Test: 237 von 24,000)
Stufe 1: lässt 6.6% durch, Recall 0.949
Stufe 2 auf den Durchgelassenen: Recall 0.640
Kaskade gesamt: Recall 0.608  Precision 0.954
Modell auf allen: Recall 0.586  Precision 0.853
Abfragen der teuren Merkmale: Kaskade 1,578 statt 24,000
Recall 1 * Recall 2 = 0.949 * 0.640 = 0.608
```

Nur für die 1578 von 24 000 durchgelassenen Zahlungen müssen die teuren Features abgefragt werden; ein einzelnes Modell auf allen Features bräuchte sie für alle 24 000. Die zweite Stufe findet von den Durchgelassenen 64,0 %, die Kaskade insgesamt 60,8 % bei einer Precision von 95,4 %. Das einzelne Modell kommt auf 58,6 % Recall bei 85,3 % Precision. Dass die Kaskade hier nicht schlechter, sondern etwas besser abschneidet, sollte man nicht überbewerten: 237 Betrugsfälle sind wenige. Eine mögliche Erklärung ist, dass die zweite Stufe auf Daten trainiert, in denen Betrug rund 14 % ausmacht (im Test 225 von 1578) statt 1 %, also deutlich weniger unbalanciert ist (Pattern 10).

Auch hier pflanzen sich Fehler fort: Die Recalls der Stufen multiplizieren sich (94,9 % mal 64,0 % ergeben 60,8 %), und was Stufe 1 verwirft, holt keine spätere Stufe zurück. Deshalb baut man die erste Stufe auf hohen Recall. Die zweite Stufe trainiert man auf den Daten, die die erste durchlässt (`Xtr[durch_tr]`), weil sie im Betrieb nie etwas anderes sieht; ändert sich die erste Stufe, muss sie neu trainiert werden.

Trade-offs und Grenzen. Beide Varianten kosten Einfachheit: mehrere Modelle, Schwellen und Versionen, die zusammenpassen müssen und gemeinsam überwacht werden. Je Spezialist stehen weniger Beispiele zur Verfügung, was die Varianz erhöht. Pattern 19 (Two-Phase Predictions, Kapitel 8) ist

verwandt, aber der Anlass ist ein anderer: Dort läuft ein leichtes Modell auf dem Gerät und ein schweres in der Cloud, und Bandbreite und Latenz bestimmen die Aufteilung.

Wann nicht. Bevor man eine Cascade baut, probiert man die einfacheren Mittel: ein Modell mit dem Segment als zusätzlichem Feature, ein Feature Cross (Pattern 3) oder ein Baummodell, das Wechselwirkungen selbst bildet. Eine Cascade ist unnötig, wenn sich die Gruppen nicht gegensätzlich verhalten, wenn das Segment nicht zuverlässig bestimmbar ist (ein schwacher Router verdirbt alles) oder wenn je Segment zu wenige Daten vorliegen. Bei der Konfidenz-Kaskade gilt: Ein einzelnes Modell, das schnell und genau genug ist, ersetzt sie, und ohne hohen Recall bei kleiner Durchlassquote in der ersten Stufe lohnt sie nicht.

Bezug zu Software Engineering. Der Router ist ein Dispatcher: Wie beim Strategy-Pattern wird zur Laufzeit entschieden, welche Implementierung arbeitet, nur dass die Entscheidung selbst fehlbar ist. Dazu passt die Erfahrung mit Dienstketten: Verfügbarkeit und Korrektheit multiplizieren sich mit jedem Glied, und man testet die Kette Ende zu Ende statt nur ihre Teile. Die Konfidenz-Kaskade kennt man als „billig vor teuer“: ein Cache vor dem Backend, ein Bloom-Filter vor dem Datenbankzugriff, Short-Circuit-Auswertung (`a and b` ruft `b` nur auf, wenn `a` wahr ist), mit der Besonderheit, dass jedes Glied einen Fehler macht, der sich nicht mehr korrigieren lässt.

> **SO ERKLÄRE ICH ES**
> Beschreiben Sie die Telefonzentrale einer Versicherung: Die erste Mitarbeiterin fragt nur, worum es geht, und verbindet mit der Sachbearbeiterin für Privat- oder für Werkstattschäden. Das funktioniert, weil beide Sachbearbeiterinnen ihr Gebiet sehr gut kennen. Fragen Sie dann, was passiert, wenn die Zentrale falsch verbindet, und zeigen Sie Abbildung 45.
>
> **Verständnisfrage.** Warum ist ein falsch gerouteter Fall hier schlimmer als ein Fall, den ein einzelnes Modell falsch einschätzt?
>
> **Antwort.** Der Spezialist wendet ein Muster an, das für diese Gruppe umgekehrt gilt. Im Skript ist die AUC der falsch gerouteten Fälle 0,237, also schlechter als Raten, während ein einzelnes Modell zumindest einen Kompromiss anbietet.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Cascade“; P. Viola und M. Jones [43] für die Konfidenz-Kaskade. Die Daten beider Skripte sind künstlich. *Offen:* Die Beschreibung des Pattern (Router und Spezialisten, Training auf dem wahren Segment als Falle) folgt dem Glossar und den Folien dieser Lehrveranstaltung und dem Gedächtnis; die Buchstelle selbst konnte hier nicht am Text nachgeprüft werden und sollte vor der Verwendung in der Lehre nachgelesen werden.

6.5 Pattern 9: Neutral Class

Szenario. Ein Händler will Produktrezensionen automatisch als positiv oder negativ erkennen, damit der Support die negativen zuerst sieht. Für das Training haben drei Personen (die *Labeler*) jede Rezension unabhängig mit einem Label versehen, also mit der Antwort „positiv“ oder „negativ“, die das Modell später vorhersagen soll. Im Skript `05_neutral_class.py` entstehen 30 000 künstliche Rezensionen. Eine im Betrieb unbekannte „wahre Stimmung“ s bestimmt, wie die Labeler urteilen: Jede Person sagt mit der Wahrscheinlichkeit $\frac{1}{1+e^{-5s}}$ „positiv“. Bei $s = 0$ ist das ein Münzwurf, ab $|s| > 1$ sind sich praktisch alle einig. Das Modell sieht ein verrauschtes Abbild von s und fünf Spalten reines Rauschen. In 77,8 % der Fälle urteilen alle drei gleich, in den übrigen steht es 2 zu 1.

Das Problem. Aus den drei Urteilen macht man üblicherweise ein binäres Label per Mehrheit. Bei 2 zu 1 ist dieses Label aber kaum mehr als ein Zufallsergebnis: Der Text liegt auf der Grenze, und welche Seite zufällig die Mehrheit bekam, sagt über den Text nichts. Das Modell wird trotzdem darauf trainiert, diese Fälle richtig zu entscheiden, und es versucht, das Rauschen zu lernen. Im Betrieb liefert es auch für Grenzfälle eine Ja/Nein-Antwort, und nichts an der Antwort zeigt, dass es hier eine Münze war. Misst man den Fehler des binären Modells gegen das Mehrheitslabel, ergibt sich bei den einstimmigen Fällen 4,0 %, bei den strittigen 31,0 %. Die zweite Zahl sagt nicht, dass das Modell dort schlecht ist; sie misst, wie zufällig das Referenzlabel selbst ist.

Die Lösung. Das Pattern führt eine dritte Klasse ein: *neutral*. Sie steht für Beispiele, bei denen die Labeler uneinig sind, oder, wenn Bewertungen mit Sternen vorliegen, für die mittlere Stufe. Das Modell hat drei Ausgänge (positiv, neutral, negativ), und der Betrieb behandelt „neutral" anders als eine Entscheidung: Man gibt den Fall an einen Menschen, zeigt keine Empfehlung oder fällt auf einen Standard zurück. Im Skript wird das aus der Uneinigkeit direkt abgeleitet (`einig`), und es entsteht ein zweites Modell mit derselben Technik, aber drei Klassen.

```
stimmen = rng.random((n, 3)) < 1 / (1 + np.exp(-5 * s[:, None]))
summe = stimmen.sum(1)
mehrheit = (summe >= 2).astype(int)          # binäres Label per Mehrheit
einig = (summe == 0) | (summe == 3)          # alle drei gleicher Meinung
# dreiwertiges Label: 0 negativ, 1 neutral (uneinig), 2 positiv
drei = np.where(einig, np.where(summe == 3, 2, 0), 1)
```

Terminal

```
$ uv run --with scikit-learn --with pandas python 05_neutral_class.py
Labeler einstimmig: 77.8%, Uneinigkeit (2:1): 22.2%

Test: 9,000 Beispiele, davon strittig 21.4%

Binäres Modell
  Fehlerrate bei einstimmigen Fällen: 4.0%
  Fehlerrate bei strittigen Fällen  : 31.0%   (Mehrheitslabel als Referenz)
  Gesamtfehler gegen Mehrheitslabel  : 9.8%
  mittlere Sicherheit max(p,1-p) bei strittigen Fällen: 0.78

Modell mit Neutral-Klasse
  als neutral erkannt: 51.4% der strittigen, 12.8% der einstimmigen Fälle
  automatisch entschieden: 79.0% aller Fälle
  Fehlerrate unter den automatisch entschiedenen (Mehrheitslabel): 3.5%
  Fehlerrate bei einstimmigen Fällen: 7.1% (neutral-Vorhersagen mitgezählt: 13.6%)

Binäres Modell mit Konfidenzschwelle (gleicher Automatisierungsanteil)
  automatisch entschieden: 79.0%, Fehlerrate darunter: 3.4%
```

Das binäre Modell macht bei den einstimmigen Fällen 4,0 % Fehler und bei den strittigen 31,0 %. Die zweite Zahl misst gegen das Mehrheitslabel, das dort fast ein Münzwurf ist, und sagt deshalb vor allem etwas über den Zufall im Label. Auffällig ist die mittlere Sicherheit des Modells bei den strittigen Fällen: 0,78. Es tut also so, als wüsste es, was es nicht wissen kann. Das Drei-Klassen-Modell antwortet bei 51,4 % der strittigen und bei 12,8 % der einstimmigen Fälle „neutral", entscheidet 79,0 % aller Fälle automatisch und macht darunter 3,5 % Fehler.

Ein Vergleich, der das Pattern nicht schönt: Ein binäres Modell, das nur die sichersten 79,0 % der Fälle entscheidet (Konfidenzschwelle), kommt auf 3,4 % Fehler, also auf dasselbe. In diesem Versuch bringt die dritte Klasse keine bessere Genauigkeit als eine passend gewählte Schwelle. Ihr Nutzen liegt woanders: Die Schwelle muss man von Hand festlegen, die neutrale Klasse lernt aus den Daten, wo die Menschen selbst uneins sind. Das Produkt bekommt eine ausdrückliche Antwort mit eigener Bedeutung, und das Label bildet ab, was die Labeler tatsächlich gesagt haben. Ob die dritte Klasse bei echten Daten außerdem die Qualität der beiden Hauptklassen verbessert, weil das Modell nicht mehr auf Münzwurf-Labels trainiert wird, kann dieser Versuch nicht zeigen. Das bleibt hier offen.

Abbildung 47 zeigt, wo das Drei-Klassen-Modell neutral antwortet, aufgetragen über der wahren Stimmung. In den beiden Streifen um $s = 0$, in denen die Labeler am häufigsten uneins sind, antwortet das Modell in knapp zwei Dritteln der Fälle neutral (63,6 % und 66,5 %), ab $|s| > 1$ praktisch nie. Das Modell hat nur die Rezension gesehen, nicht s; die Breite der neutralen Zone ist ein Ergebnis des Trainings.

Trade-offs und Grenzen. Das Pattern setzt Information über die Uneinigkeit voraus, also mehrere Labeler je Beispiel oder eine abgestufte Bewertung, und das kostet Aufwand. Die Klasse „neutral" ist selbst unscharf, sie hängt davon ab, wer gelabelt hat, und die Labeler können gemeinsam danebenliegen. Sie ist oft klein (Pattern 10), und jemand muss entscheiden, was mit einer neutralen Vorhersage geschieht.

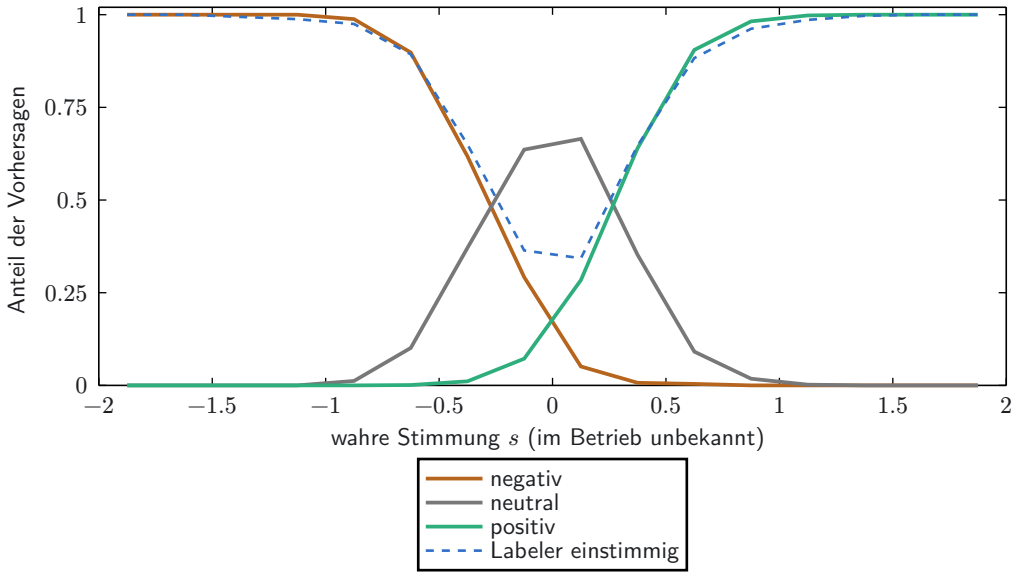


Abbildung 47: Das Drei-Klassen-Modell auf den Testdaten. Die Linien zeigen, wie oft das Modell negativ, neutral oder positiv sagt, aufgetragen über der wahren Stimmung. Gestrichelt: Anteil der Fälle, in denen alle drei Labeler einig waren.

Wann nicht. Wenn die Wahrheit objektiv feststellbar ist (der Tumor ist bösartig oder nicht, das Ergebnis liegt aus einer Biopsie vor), ist Uneinigkeit der Labeler ein Qualitätsproblem, und man sollte die Labels verbessern, nicht eine dritte Klasse einführen. Und wenn das Produkt in jedem Fall eine Entscheidung braucht und es keinen Ort für die Weiterleitung gibt, hilft eine neutrale Antwort nicht.

Bezug zu Software Engineering. Die dritte Klasse ist ein dritter Wahrheitswert. Aus der dreiwertigen Logik von SQL kennt man das (`TRUE`, `FALSE`, `UNKNOWN`), und es gilt dieselbe Faustregel: Wer ein „weiß nicht" in ein Ja/Nein zwingt, bekommt Fehler, die man nicht sieht. Die Weiterleitung neutraler Fälle an einen Menschen ist ein Fallback-Pfad, wie man ihn für kontrollierte Degradation (*Graceful Degradation*) einbaut.

> **SO ERKLÄRE ICH ES**
> Fragen Sie: Drei Kolleginnen bewerten dieselbe Rezension, zwei sagen positiv, eine sagt negativ. Ist die Rezension positiv? Lassen Sie abstimmen und erklären, dass die Antwort „positiv" hier mehr verspricht, als in den Daten steckt. Das Modell soll das nicht überspielen, sondern sagen können, dass der Fall strittig ist.
>
> **Verständnisfrage.** Worin unterscheidet sich die neutrale Klasse von einem binären Modell, das bei Wahrscheinlichkeit nahe 0,5 nicht entscheidet?
>
> **Antwort.** Das binäre Modell weiß nicht, ob die 0,5 von einem strittigen Fall oder von zu wenigen Daten kommt, und die Grenze legt man von Hand fest. Die neutrale Klasse lernt aus der Uneinigkeit der Labeler, was strittig ist, und macht sie zu einer eigenen, ausgewerteten Antwort.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Neutral Class". Die Daten sind künstlich; die Simulation der Labeler ist für das Handbuch konstruiert.

6.6 Pattern 10: Rebalancing

Szenario. Ein Zahlungsdienstleister will betrügerische Transaktionen erkennen. Im Skript `06_rebalancing.py` stehen 200 000 künstliche Transaktionen, von denen 0,5 % Betrug sind (1000 Fälle). Davon gehen 60 % ins Training (600 Betrugsfälle), je 20 % in Validierung und Test (je 200 Betrugsfälle bei 40 000 Transaktionen).

Das Problem. Ein Modell, das *nie* Betrug meldet, hat eine Genauigkeit von 99,5 % und ist wertlos. Der Fehler liegt im Verfahren. Das Training summiert den Fehler über alle Beispiele, und 99,5 % davon stammen von der Mehrheitsklasse; das einfachste Mittel, ihn zu senken, ist, überall „kein Betrug" zu sagen. Das Modell sieht außerdem sehr wenige positive Beispiele, aus denen es lernen kann, was Betrug auszeichnet, und die gewohnte Metrik, Genauigkeit, bemerkt davon nichts.

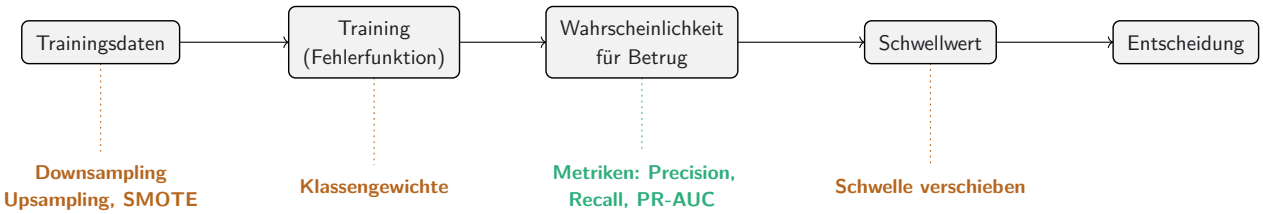


Abbildung 48: Rebalancing greift an verschiedenen Stellen an: an den Daten, an der Fehlerfunktion oder an der Entscheidung nach dem Modell. Welche Metriken man verwendet, gilt für alle Varianten.

Die Lösung. Das Pattern besteht aus zwei Teilen: den richtigen Metriken und mehreren Möglichkeiten, die Daten oder das Training anzupassen. Abbildung 48 zeigt, wo die einzelnen Maßnahmen ansetzen.

Metriken. Zuerst muss man das Richtige messen. Genauigkeit eignet sich nicht. Precision und Recall zeigen, wie viele Alarme berechtigt sind und wie viele echte Fälle gefunden werden. Die *PR-AUC* (genauer: die *average precision*) fasst die Precision-Recall-Kurve über alle Schwellen zu einer Zahl zusammen, wobei als Vergleichswert für ein Modell ohne Können die Basisrate (hier 0,5 %) gilt. Die bekanntere ROC-AUC kann bei starker Unbalance zu optimistisch wirken, weil sie die vielen Negativen als Rechengrundlage nimmt; T. Saito und M. Rehmsmeier [10] haben das ausführlich gezeigt.

Downsampling. Man wirft Beispiele der Mehrheitsklasse weg, bis das Verhältnis erträglich ist, hier 1 zu 10 (600 Positive, 6000 Negative). Das Modell sieht Betrug nun in jedem elften Beispiel. Man verliert Daten und damit Information über die Mehrheit.

Klassengewichte. Statt Daten zu verändern, gewichtet man den Fehler: Ein übersehener Betrug zählt dann etwa so viel wie 200 übersehene Normaltransaktionen (`class_weight="balanced"`). Das kostet keine Daten, man behält alles, aber die Optimierung wird empfindlicher für die wenigen positiven Beispiele.

Upsampling und SMOTE. Man vermehrt die Minderheit. Einfaches Kopieren (*Oversampling*) führt dazu, dass das Modell die wenigen Beispiele auswendig lernt. SMOTE [44] erzeugt stattdessen *synthetische* Beispiele: Es wählt ein positives Beispiel, sucht dessen nächste positive Nachbarn und legt einen neuen Punkt zufällig auf der Verbindungsstrecke zu einem davon. Im Skript kommt die Implementierung aus der Bibliothek imbalanced-learn [45].

Schwelle verschieben. Man ändert das Modell gar nicht und senkt stattdessen den Schwellwert, ab dem eine Wahrscheinlichkeit als „Betrug“ gilt, von 0,5 auf einen auf Validierungsdaten gewählten Wert. Das ist die billigste Variante und oft genauso wirksam.

```
rng = np.random.default_rng(0)
pos, neg = np.flatnonzero(ytr == 1), np.flatnonzero(ytr == 0)
weg = rng.choice(neg, size=10 * len(pos), replace=False)
down = np.concatenate([pos, weg])                    # Verhältnis 1:10
Xs, ys = SMOTE(random_state=0).fit_resample(Xtr, ytr)   # nur Trainingsdaten!

strategien = {
    "Baseline (nichts getan)": modell().fit(Xtr, ytr),
    "Klassengewichte (balanced)": modell(class_weight="balanced").fit(Xtr, ytr),
    "Downsampling 1:10": modell().fit(Xtr[down], ytr[down]),
    "SMOTE (Upsampling)": modell().fit(Xs, ys),
}
```

Für die Schwelle wird auf den Validierungsdaten der Wert gesucht, der den F1-Wert maximiert, aus den Quantilen der vorhergesagten Wahrscheinlichkeiten:

```
    # Schwelle verschieben: beste F1-Schwelle auf den Validierungsdaten suchen
    pva = m.predict_proba(Xva)[:, 1]
    kand = np.unique(np.quantile(pva, np.linspace(0.9, 0.9999, 300)))
    t = kand[np.argmax([f1_score(yva, pva >= s) for s in kand])]
```

Terminal

```
$ uv run --with scikit-learn --with imbalanced-learn python 06_rebalancing.py
Positive: 0.50%; Train 600 / 120,000, Test 200 / 40,000
```

```
Triviales Modell 'nie Betrug': Genauigkeit 0.995, Recall 0.000

Strategie                    Prec   Rec    F1   F1* PR-AUC   ROC mittl.P
Baseline (nichts getan)     0.674 0.145 0.239 0.384  0.368 0.964  0.0050
Klassengewichte (balanced)  0.037 0.900 0.071 0.366  0.314 0.963  0.1727
Downsampling 1:10           0.175 0.660 0.277 0.397  0.345 0.964  0.0459
SMOTE (Upsampling)          0.040 0.885 0.076 0.362  0.314 0.963  0.1537
F1*: F1 mit der auf den Validierungsdaten gewählten Schwelle

Konfusionsmatrix Baseline bei 0,5 (Zeilen: wahr, Spalten: Vorhersage)
[[39786    14]
 [  171    29]]
Konfusionsmatrix Klassengewichte bei 0,5
[[35112  4688]
 [   20   180]]
```

Die Tabelle am Ende der Ausgabe enthält das Wesentliche. Die ROC-AUC liegt bei allen vier Modellen bei 0,96 und unterscheidet sie nicht. Bei der Schwelle 0,5 sehen sie dagegen sehr verschieden aus. Die Baseline ist vorsichtig (Precision 0,67, Recall 0,15: nur 29 der 200 Betrugsfälle gefunden). Klassengewichte und SMOTE sind das Gegenteil: Recall 0,90 und 0,89, aber eine Precision von nur etwa 0,04. Die Konfusionsmatrix (eine Tabelle, die wahre gegen vorhergesagte Klassen zählt) der gewichteten Variante macht das konkret: 180 gefundene Betrugsfälle, aber auch 4688 Fehlalarme bei 40 000 Transaktionen. Das Downsampling liegt dazwischen (Recall 0,66, Precision 0,18) und hat bei 0,5 den besten F1-Wert (0,28).

Dieser Unterschied ist ein verschobener Arbeitspunkt und kein besseres Modell. Die PR-AUC, die die Rangfolge der Vorhersagen über alle Schwellen misst, ist bei der Baseline mit 0,368 sogar am höchsten (0,314 bis 0,345 bei den anderen). Wählt man für jedes Modell die Schwelle auf den Validierungsdaten (Spalte F1*), liegen alle vier zwischen 0,36 und 0,40. Bei nur 200 positiven Testbeispielen ist dieser Unterschied nicht belastbar. Rebalancing verschiebt also vor allem, an welcher Stelle der Precision-Recall-Kurve das Modell arbeitet. Die Schwelle erreicht dasselbe mit einer Zeile Code und ohne Neutraining. Für ein besseres Modell braucht es bessere Features oder mehr Daten, und nicht unbedingt ein ausgeglicheneres Training.

Trade-offs und Grenzen. Wer die Verteilung ändert, ändert die Bedeutung der Wahrscheinlichkeiten: Ein Modell, das mit 1 zu 10 oder mit Gewichten trainiert wurde, überschätzt die Betrugswahrscheinlichkeit systematisch (die Spalte „mittl.P“ in der Ausgabe zeigt es). Für Entscheidungen über einen Schwellwert ist das unproblematisch, wer die Zahlen als echte Wahrscheinlichkeiten braucht, muss nachkalibrieren. Dann gilt eine Regel ohne Ausnahme: Validierungs- und Testdaten bleiben unverändert. Man wendet Upsam-

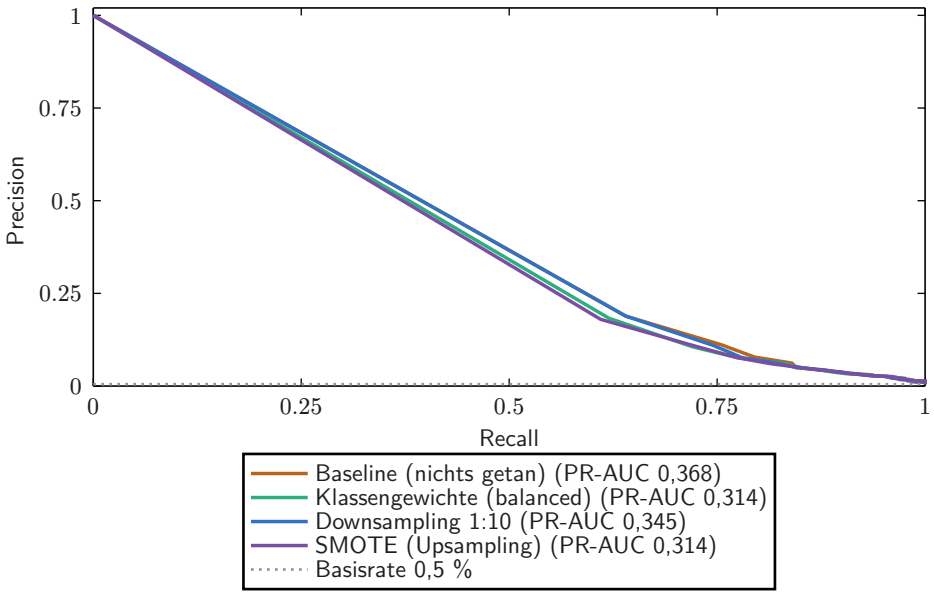


Abbildung 49: Precision-Recall-Kurven der vier trainierten Varianten auf den Testdaten. Je weiter oben rechts, desto besser; die gepunktete Linie ist die Basisrate (Raten).

pling und SMOTE nur auf Trainingsdaten an, nach der Aufteilung. Wer vor der Aufteilung synthetische Beispiele erzeugt, hat Kopien und Nachbarn im Test.

Wann nicht. Bei mäßiger Unbalance (Verhältnis 1 zu 5 oder besser) und genug positiven Beispielen ist oft keine Maßnahme nötig. Gibt es kaum Beispiele der Minderheit (einige Dutzend), hilft Rebalancing nicht; dann ist es sinnvoller, das Problem als Anomalieerkennung zu formulieren (Pattern 5): Das Modell lernt, was normal ist, und meldet Abweichungen.

Bezug zu Software Engineering. Das Pattern ähnelt dem Umgang mit seltenen Fehlerfällen im Testen. Wer nur mit Daten testet, in denen Fehler 0,5 % ausmachen, prüft vor allem den Normalfall. Für die Entwicklung legt man deshalb gezielt Fehlerfälle nach, so wie hier die Minderheit vermehrt wird. Für die Abnahme zählt die echte Verteilung, deshalb bleiben die Testdaten unverändert, und „99,5 % grün“ sagt bei so schiefen Daten nichts aus.

> **SO ERKLÄRE ICH ES**
> Beginnen Sie mit dem Modell, das nie Betrug meldet, und fragen Sie nach seiner Genauigkeit (99,5 %). Erst nach der Verblüffung kommen Precision und Recall. Anschließend die Frage, wo man ansetzen kann: an den Daten, an der Fehlerfunktion, an der Schwelle (Abbildung 48).
>
> **Verständnisfrage.** Warum darf man SMOTE nur auf den Trainingsdaten anwenden und nicht vor der Aufteilung?
>
> **Antwort.** Synthetische Beispiele liegen auf Verbindungsstrecken zwischen echten. Landet ein echtes Beispiel im Training und sein Nachbar im Test, kennt das Modell den Testfall quasi schon, und die Testergebnisse sind zu gut.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Rebalancing“; N. V. Chawla, K. W. Bowyer, L. O. Hall, und W. P. Kegelmeyer [44] (SMOTE); T. Saito und M. Rehmsmeier [10], H. He und E. A. Garcia [46]. Die Daten sind künstlich.

7 Model Training

Dieses Kapitel behandelt fünf Patterns, die alle um dieselbe Tätigkeit kreisen: das Trainieren eines Modells. Im Buch von Lakshmanan et al. [1] stehen sie in Kapitel 4. Sie beantworten Fragen, die jeder stellt, der ein Training nicht nur einmal im Notebook, sondern verlässlich und wiederholt laufen lassen muss: Wann ist Overfitting ausnahmsweise erwünscht (Pattern 11)? Wie überlebt ein langes Training einen Absturz (Pattern 12)? Wie spart man sich das Training von Grund auf (Pattern 13)? Wie verteilt man es auf mehrere Rechner (Pattern 14)? Und wie findet man die Einstellungen, mit denen das Training überhaupt gut läuft (Pattern 15)?

7.1 Vorspann: Die Trainingsschleife

Wer Software baut, kennt Schleifen, die Zustand verändern, bis eine Bedingung erfüllt ist. Training ist genau so eine Schleife. Das Modell besitzt **Parameter**, im einfachsten Fall zwei Zahlen w und b einer Geraden $\hat{y} = wx + b$, bei einem neuronalen Netz Millionen davon. Ein **Loss** misst, wie falsch die Vorhersagen für die Trainingsdaten sind, zum Beispiel als mittleren quadratischen Fehler. Der **Gradient** sagt für jeden Parameter, in welche Richtung und wie stark der Loss steigt, wenn man ihn erhöht. Man geht deshalb einen kleinen Schritt in die Gegenrichtung. Die Schrittweite heißt **Lernrate**. Wer mit Parametern und Loss noch nicht vertraut ist, findet beides in Kapitel 2 ausführlicher.

Die Daten werden in kleinen Portionen verarbeitet, den **Mini-Batches**; ein Durchlauf durch alle Trainingsdaten heißt **Epoche**. Zwischen den Schritten hält die Schleife nur die Parameter fest. Deshalb lässt sie sich gut parallelisieren, unterbrechen und fortsetzen, und darauf bauen die Patterns dieses Kapitels auf.

Das folgende Programm trainiert eine Gerade mit reinem numpy. Die Daten sind so erzeugt, dass die Wahrheit $y = 3x + 2$ plus Rauschen mit Standardabweichung 1,5 lautet; wir kennen also die richtige Antwort und können dem Training zusehen.

```
w, b = 0.0, 0.0           # die Parameter des Modells, Startwert beliebig
lr = 0.002                # Lernrate: wie weit pro Schritt korrigiert wird
batch = 20                # Mini-Batch: so viele Beispiele pro Schritt

# eine Epoche = alle Daten einmal gesehen
for epoch in range(1, 101):
    idx = rng.permutation(len(x))
    for start in range(0, len(x), batch):
        i = idx[start:start + batch]
        fehler = (w * x[i] + b) - y[i]          # Vorhersage minus Wahrheit
        # Ableitung des quadratischen Fehlers nach w
        grad_w = 2 * np.mean(fehler * x[i])
        grad_b = 2 * np.mean(fehler)
        w -= lr * grad_w                        # einen kleinen Schritt bergab
        b -= lr * grad_b
    if epoch in (1, 2, 5, 10, 25, 50, 100):
```

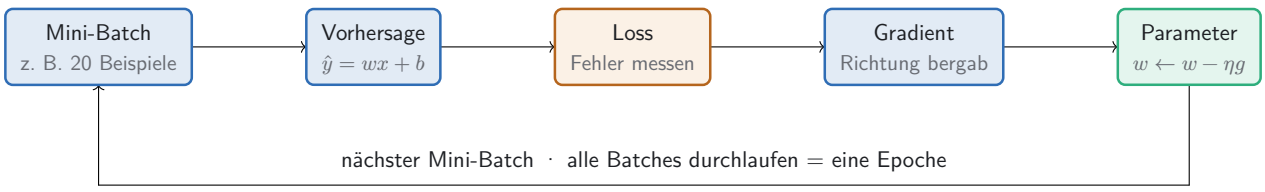


Abbildung 50: Die Trainingsschleife. Nach jedem Mini-Batch ändern sich nur die Parameter (η ist die Lernrate, g der Gradient).

```
        loss = np.mean((w * x + b - y) ** 2)
        print(f"Epoche {epoch:2d}: w={w:5.2f}  b={b:5.2f}  Loss={loss:6.2f}")
```

Terminal

```
$ uv run --with numpy python trainingsschleife.py
Epoche  1: w= 2.46  b= 0.39  Loss= 23.81
Epoche  2: w= 3.07  b= 0.49  Loss=  3.72
Epoche  5: w= 3.25  b= 0.55  Loss=  2.48
Epoche 10: w= 3.25  b= 0.59  Loss=  2.46
Epoche 25: w= 3.23  b= 0.72  Loss=  2.40
Epoche 50: w= 3.19  b= 0.89  Loss=  2.34
Epoche 100: w= 3.17  b= 1.14  Loss=  2.28
```

Der Loss fällt in den ersten Epochen steil und bleibt dann nahe bei 2,25. Das ist die Varianz des Rauschens ($1{,}5^2$); besser kann kein Modell die verrauschten Daten vorhersagen. Der Parameter b liegt nach 100 Epochen erst bei 1,14 statt bei 2, denn der Loss ist in dieser Richtung flach: Das Training ist gut genug, aber nicht „fertig“. Lernrate, Batch-Größe und Epochenzahl sind die Stellschrauben, um die sich die fünf Patterns drehen. In scikit-learn steckt die Schleife hinter `fit`, etwa bei `MLPClassifier` und `SGDClassifier`; `LinearRegression` rechnet die Lösung dagegen direkt aus.

7.2 Pattern 11: Useful Overfitting

Szenario und Problem. Ein Ingenieurbüro simuliert, wie eine Federung nach einem Stoß ausschwingt. Der Solver liefert die Auslenkung $x(t, c)$ in Abhängigkeit von Zeit t und Dämpfung c. Er ist exakt, braucht aber (angenommen) mehrere Sekunden pro Aufruf, und für die Optimierung des Entwurfs werden Millionen Aufrufe benötigt. Also trainiert man ein Modell, das die Simulation nachahmt, ein sogenanntes **Surrogatmodell**, und fragt künftig nur noch das Modell.

Die Standardregel des Machine Learning lautet: Overfitting vermeiden. **Overfitting** heißt, dass sich ein Modell an Zufälligkeiten der Trainingsdaten anpasst, die nicht verallgemeinern, vor allem an Messrauschen. Man erkennt es daran, dass der Fehler auf den Trainingsdaten klein ist und auf ungesehenen Daten groß. Regularisierung (ein Strafterm im Loss, der große Parameterwerte unattraktiv macht), Early Stopping (Abbruch, sobald sich der Fehler auf Validierungsdaten nicht mehr bessert) und kleinere Modelle bremsen die Anpassung an die Trainingsdaten. Diese Regel hat zwei Voraussetzungen: dass die Trainingsdaten die Welt nur lückenhaft und verrauscht abbilden, und dass das Modell später Eingaben sieht, die nicht im Training vorkamen. Bei einer deterministischen Simulation trifft beides nicht zu. Es gibt kein Rauschen, und der Eingaberaum lässt sich lückenlos abtasten.

Die Lösung. Das Pattern dreht die Regel unter diesen Bedingungen um: Man wählt ein Modell mit reichlich Kapazität, verzichtet auf Regularisierung und auf zurückgehaltene Testdaten (Hold-out) und trainiert, bis der Trainingsfehler praktisch null ist. Die Trainingsdaten sind der gesamte Eingaberaum, hier ein Gitter aus 61 Zeitpunkten und 9 Dämpfungswerten (549 Punkte) der Funktion $x(t, c) = e^{-ct}\cos\left(\sqrt{4 - c^2}\,t\right)$. Der Fehler auf dem Gitter ist die Qualitätsgröße, denn das Gitter sind die Eingaben, die später gefragt werden. Weil die Funktion glatt ist, prüft man zusätzlich 480 Zwischenwerte, die nie im Training vorkamen; die Simulation selbst liefert diese Kontrollpunkte. Das Skript trainiert ein kleines Netz (zwei Schichten à 32 Neuronen) ohne und mit starker Regularisierung ($\alpha = 1$):

```
for name, alpha in [("MLP ohne Regularisierung", 0.0),
                    ("MLP, alpha=1 (regularisiert)", 1.0)]:
    m = MLPRegressor(hidden_layer_sizes=(32, 32), activation="tanh",
                     solver="lbfgs",
                     alpha=alpha, max_iter=3000, random_state=0)
    m.fit(X_gitter, y_gitter)
    print(f"  {name:30.30s} {rmse(m, X_gitter, y_gitter):12.5f} "
          f"{rmse(m, X_zwischen, y_zwischen):20.5f}")
```

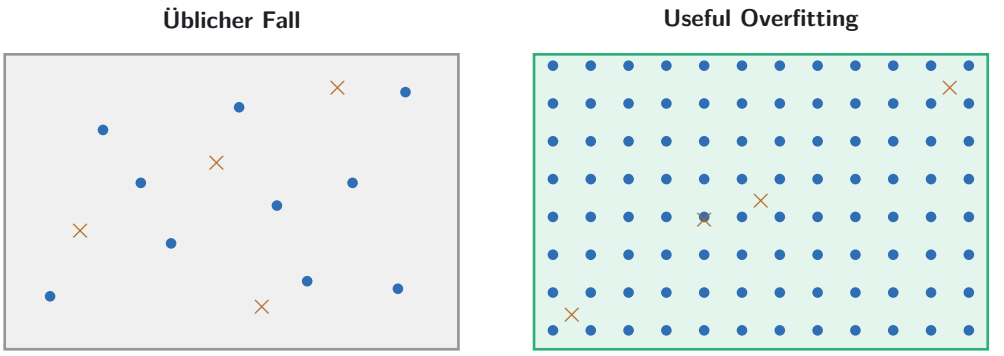


Abbildung 51: Der Eingaberaum im Vergleich. Blaue Punkte sind Trainingsdaten, Kreuze sind spätere Anfragen. Schematisch, ohne Messwerte.

Terminal

```
$ uv run --with scikit-learn --with numpy python useful_overfitting.py
Teil A: Surrogat für eine deterministische Simulation
  Trainingsraum: 549 Gitterpunkte, Zwischenwerte zum Test: 480
  Modell                          RMSE Gitter   RMSE Zwischenwerte
  MLP ohne Regularisierung            0.00642              0.00503
  MLP, alpha=1 (regularisiert)        0.10820              0.09235

Teil B: dieselbe Funktion, aber Messdaten mit Rauschen (sigma=0.3)
  Modell                     RMSE Training  Test: Messung  Test: Wahrheit
  Baum ohne Tiefenlimit              0.000          0.459           0.334
  Baum, Tiefe 5                      0.250          0.367           0.198

Teil C: Zufallslabels (zwischen X und y gibt es nichts zu lernen)
  Entscheidungsbaum            Genauigkeit Training=1.00  Test=0.47
  Random Forest (200 Bäume)    Genauigkeit Training=1.00  Test=0.52
```

In Teil A trifft das unregularisierte Netz das Gitter mit einem mittleren Fehler (RMSE) von 0,0064 und die nie gesehenen Zwischenwerte mit 0,0050. Mit starker Regularisierung liegt der Fehler bei 0,108, rund 17-mal höher: Die Maßnahme, die sonst vor Überanpassung schützt, erzwingt hier eine Glätte, die die Simulation nicht hat.

Dass diese Logik nicht allgemein gilt, zeigen die Teile B und C. In B wird dieselbe Funktion nur an 400 Zufallspunkten mit Messrauschen beobachtet. Ein Baum ohne Tiefenlimit merkt sich jede Messung (Trainingsfehler 0,000), verfehlt neue Messungen aber mit 0,459 und die wahre Funktion mit 0,334. Ein Baum der Tiefe 5 hat 0,250 Trainingsfehler, aber nur 0,198 gegenüber der Wahrheit: Der perfekt angepasste Baum hat das Rauschen mitgelernt. In C sind die Labels reine Zufallszahlen, und beide Modelle erreichen trotzdem 100 Prozent Trainingsgenauigkeit bei etwa 50 Prozent auf neuen Daten. Zhang et al. [23] haben das für tiefe neuronale Netze gezeigt: Sie lernen auch Zufallslabels vollständig auswendig. Ein Trainingsfehler von null sagt für sich also nichts über Verallgemeinerung. Bei der Simulation trägt das Wissen über die Aufgabe die Rechtfertigung (deterministisch, glatt, Eingaben abgedeckt), nicht der Trainingsfehler. Warum große Netze trotz perfekter Anpassung oft gut verallgemeinern, ist eine eigene, nicht abschließend beantwortete Forschungsfrage; ein viel diskutiertes Phänomen dazu heißt „Double Descent“ [47].

Trade-offs und Grenzen. Das Pattern gilt nur, solange der Eingaberaum abgedeckt ist. Fragt jemand außerhalb des Gitters an (etwa $t = 8$), extrapoliert das Modell ungeprüft. Ändert sich der Solver, muss neu trainiert werden, und im Betrieb sollte man Stichproben gegen die Simulation prüfen. In vielen Dimensionen scheitert das Rastern: Zehn Parameter mit je zehn Werten ergeben zehn Milliarden Punkte, und dann muss das Modell doch verallgemeinern. **Wann nicht:** bei verrauschten Labels oder Eingaben aus der offenen Welt (Nutzertexte, Fotos, Sensordaten).

Bezug zu Software Engineering. Das Modell ist ein verlustbehafteter Cache für eine teure Funktion, vergleichbar mit Memoization, nur dass es auch für nie berechnete Eingaben antwortet. Die Simulation ist das Testorakel, gegen das man das Surrogat prüft, wie bei jeder optimierten Neuimplementierung.

SO ERKLÄRE ICH ES

„Normalerweise lernt ein Modell aus Beispielen für Fälle, die es noch nie gesehen hat; dort ist Auswendiglernen gefährlich. Eine Simulation ist eine Funktion, die wir lückenlos abfragen können. Dann ist Auswendiglernen genau das, was wir wollen: eine sehr schnelle Nachschlagetabelle, die dazwischen sogar interpoliert.“ Die Studierenden sollen nicht „Overfitting ist okay“ mitnehmen, sondern „Overfitting ist bewertbar, wenn ich weiß, welche Eingaben später kommen“.

Verständnisfrage: Das Surrogat hat auf dem Gitter einen Fehler von fast null. Woran erkennen Sie, ob es auch zwischen den Gitterpunkten taugt?
Antwort: Man fragt die Simulation nach Kontrollpunkten, die nicht im Training waren, und vergleicht (hier 0,0050 auf 480 Zwischenwerten). Der Trainingsfehler allein sagt dazu nichts, wie die Zufallslabels aus Teil C zeigen.

Quellen. [1] (Kapitel 4, Pattern 11); [23]; [47].

7.3 Pattern 12: Checkpoints

Szenario und Problem. Ein Team trainiert auf einer günstigen Cloud-Instanz, die der Anbieter jederzeit entziehen darf. Das Training soll 60 Stunden dauern, nach 41 Stunden ist die Maschine weg. Ohne gespeicherten Zustand beginnt man bei null. Ein zweites Problem ist leiser: Der letzte Stand ist nicht unbedingt der beste. Der Loss auf den Trainingsdaten sinkt immer weiter, auf Validierungsdaten stagniert er irgendwann oder steigt wieder.

Was ist der „Zustand“ eines Trainings? Nicht nur die Gewichte. Adam führt pro Parameter gleitende Mittelwerte der bisherigen Gradienten und ihrer Quadrate mit (zwei zusätzliche Werte je Parameter), dazu kommen Schritt- oder Epochenzähler und, je nach Framework, der Zustand des Zufallsgenerators, der die Datenreihenfolge bestimmt. Wer nur die Gewichte sichert, wie man es für das Ausliefern tut, kann das Training grob fortsetzen, aber nicht dieselbe Rechnung wiederaufnehmen.

Die Lösung. Man schreibt in regelmäßigen Abständen den vollständigen Zustand in eine Datei. Das Skript hält zwei Dateien: `latest` wird jede Epoche überschrieben und dient der Wiederaufnahme, `best` wird nur geschrieben, wenn der Validierungs-Loss besser wird. Beendet wird das Training per **Early Stopping**: Bessert sich der Validierungs-Loss zehn Epochen lang nicht um mindestens 0,001, hört es auf, und man arbeitet mit `best` weiter.

```
modell = neues_modell()
verlauf = {"epoche": [], "train": [], "val": []}
bester_loss, bester_epoche, ohne_besserung = np.inf, 0, 0
GEDULD, MIN_DELTA, MAX_EPOCHEN = 10, 0.001, 150
for epoche in range(1, MAX_EPOCHEN + 1):
    eine_epoche(modell, X_tr, y_tr)
    lv = val_loss(modell)
    lt = log_loss(y_tr, modell.predict_proba(X_tr), labels=KLASSEN)
    verlauf["epoche"].append(epoche)
    verlauf["train"].append(round(lt, 4))
    verlauf["val"].append(round(lv, 4))
    # jederzeit wiederaufnehmbar
    joblib.dump(modell, latest)
    if lv < bester_loss - MIN_DELTA:
        bester_loss, bester_epoche, ohne_besserung = lv, epoche, 0
        joblib.dump(modell, best)                          # bester Stand bisher
    else:
        ohne_besserung += 1
    if ohne_besserung >= GEDULD:
        break
```

Terminal

```
$ uv run --with scikit-learn --with numpy python checkpoints.py
1) Early Stopping
   Epoche   1: Loss Training=1.7652  Validierung=1.7806
```

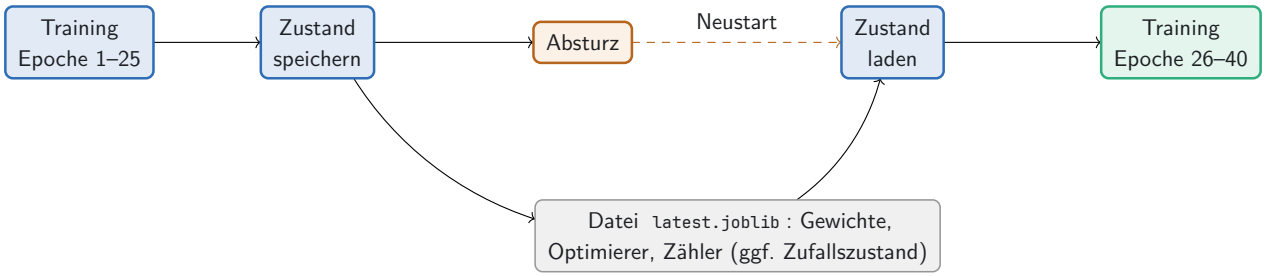


Abbildung 52: Absturz und Wiederaufnahme: Das Training setzt in Epoche 26 fort, weil der Zustand aus Epoche 25 in einer Datei liegt.

```
   Epoche   5: Loss Training=0.4153  Validierung=0.5059
   Epoche  10: Loss Training=0.1467  Validierung=0.2640
   Epoche  20: Loss Training=0.0536  Validierung=0.2034
   Epoche  30: Loss Training=0.0256  Validierung=0.1964
   Epoche  40: Loss Training=0.0143  Validierung=0.1966
   Epoche  41: Loss Training=0.0136  Validierung=0.1967
   Abbruch nach Epoche 41, bester Stand in Epoche 31 (Validierungs-Loss 0.1960)
   Genauigkeit Validierung: letzter Stand=0.943  bester Checkpoint=0.942
   Größe eines Checkpoints: 609 KiB

2) Absturz nach 25 Epochen, Neustart aus Checkpoint, 15 weitere Epochen
   gegen 40 Epochen am Stück:
   Gewichte bitgleich: True   Genauigkeit: 0.943 / 0.943

3) fit(max_iter=40) gegen zweimal fit(max_iter=20, warm_start=True)
   Gewichte bitgleich: False   Genauigkeit: 0.945 / 0.945

4) Fine-Tuning des besten Checkpoints mit 300 neuen Beispielen (10 Epochen)
   Genauigkeit Validierung: vorher=0.942  nachher=0.963
```

Das Training bricht nach Epoche 41 ab, der beste Stand war Epoche 31. Gezählt wird nur eine Verbesserung um mindestens 0,001; Epoche 33 liegt mit 0,1955 minimal tiefer als 0,1960 und zählt deshalb nicht. Der Training-Loss sinkt bis zum Abbruch weiter (von 0,0256 in Epoche 30 auf 0,0136 in Epoche 41), der Validierungs-Loss liegt seit Epoche 30 praktisch fest bei 0,196 (Abbildung 53). Hier steigt er nicht, er stagniert nur, und die Genauigkeit des besten Checkpoints (0,942) unterscheidet sich kaum vom letzten Stand (0,943): Early Stopping spart vor allem Rechenzeit. Bei stärker überanpassenden Modellen steigt der Validierungs-Loss, und dann ist `best` dem letzten Stand überlegen.

Der eigentliche Test ist die Wiederaufnahme (Teil 2 der Ausgabe): Ein Lauf bricht nach 25 Epochen ab, wird aus der Datei geladen und läuft 15 Epochen weiter, ein zweiter trainiert 40 Epochen am Stück. Die Gewichtsmatrizen sind Bit für Bit gleich, weil die Datei den Optimierer-Zustand mitführt. Ein Generatorzustand muss hier nicht gesichert werden: Mit festem `random_state` mischt scikit-learn bei jedem Aufruf von `partial_fit` neu aus dem Seed. Bei Frameworks mit fortlaufendem Zufallsgenerator gehört dessen Zustand in den Checkpoint. Teil 3 zeigt den Unterschied zu `warm_start=True`: Der Schalter lässt `fit` bei den

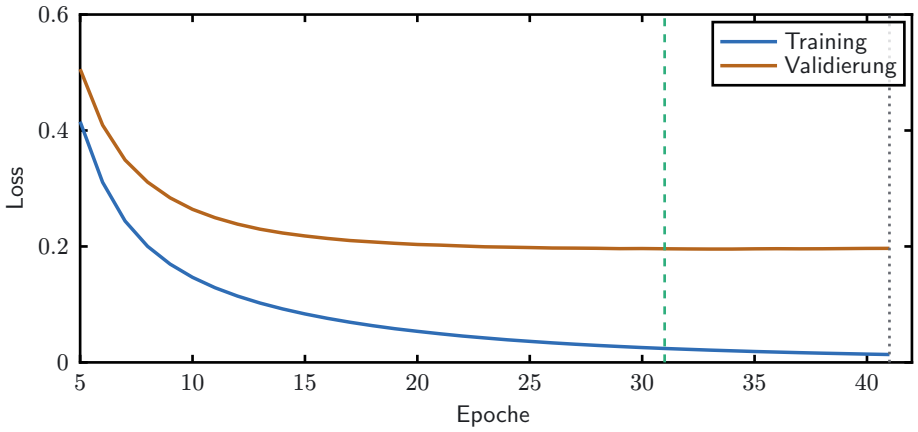


Abbildung 53: Loss je Epoche aus dem Lauf oben. Grün gestrichelt: bester Stand (Epoche 31), grau gepunktet: Abbruch (Epoche 41).

vorhandenen Gewichten weitermachen, aber der Adam-Zustand beginnt neu, und die Datenreihenfolge wird beim zweiten Aufruf wieder aus dem Seed gemischt. In der verwendeten Version (scikit-learn 1.9.1) sind die Gewichte danach nicht identisch, die Genauigkeit ist dieselbe. Für „weitertrainieren" reicht das, für „dieselbe Rechnung fortsetzen" nicht. Teil 4 lädt `best` und trainiert zehn Epochen auf 300 später eingetroffenen Beispielen weiter; die Validierungsgenauigkeit steigt von 0,942 auf 0,963. Das ist **Fine-Tuning ab Checkpoint**: Man beginnt bei einem Stand, der schon vieles kann, nicht bei null. Pattern 13 treibt den Gedanken weiter.

Trade-offs und Grenzen. Ein Checkpoint kostet Speicher und Zeit: hier 609 KiB, bei großen Netzen mit Adam ein Vielfaches der Modellgröße, weil die Optimiererwerte dazukommen. Schreibt man bei jedem Schritt, bremst das Training; schreibt man selten, verliert man bei einem Abbruch mehr. Bricht der Prozess beim Schreiben ab, bleibt eine halbe Datei zurück, deshalb schreibt man in eine temporäre Datei und benennt sie danach um. Joblib- und Pickle-Dateien speichern Verweise auf Klassen im Code, brechen unter Umständen bei Versionswechseln und führen beim Laden Code aus; man lädt sie nur aus vertrauenswürdiger Quelle. Beim Fine-Tuning nur auf neuen Daten droht, dass das Modell Altes verlernt (Catastrophic Forgetting), hier ohne Folgen, weil die Daten aus derselben Verteilung stammen. **Wann nicht:** wenn das Training in Minuten fertig ist.

Bezug zu Software Engineering. Das Muster entspricht dem Memento-Pattern der Gang of Four [2]: Der Zustand eines Objekts wird nach außen gesichert und später wiederhergestellt. Auch Savepoints in Datenbanken und wiederanlaufbare Batch-Jobs folgen dieser Idee. Early Stopping ist ein Abbruchkriterium einer Schleife, nur dass es hier auf einer Messung beruht und nicht auf einem Zähler.

> **SO ERKLÄRE ICH ES**
> „Ein Spielstand speichert nicht nur den Level, sondern Inventar, Lebenspunkte und den Zufallszustand der Welt. Genauso speichert ein Checkpoint nicht nur das Modell, sondern alles, was das Training zum Weiterlaufen braucht. Zusätzlich behalten wir den besten Spielstand, nicht nur den letzten."
>
> **Verständnisfrage:** Warum reicht es nicht, nach einem Absturz einfach die gespeicherten Gewichte zu laden und das Training neu zu starten?
> **Antwort:** Es fehlen der Optimierer-Zustand, der Fortschritt (Epochen- oder Schrittzähler) und, je nach Framework, der Zustand des Zufallsgenerators. Der Optimierer startet mit leeren Mittelwerten, die Fortsetzung ist nicht mehr dieselbe Rechnung (im Experiment mit `warm_start` : nicht bitgleich). Das Ergebnis ist meist brauchbar, aber nicht reproduzierbar.

Quellen. [1] (Kapitel 4, Pattern 12); [2].

7.4 Pattern 13: Transfer Learning

Szenario und Problem. Ein Krankenhaus besitzt 400 beschriftete Röntgenbilder und will auffällige Befunde markieren. Ein Netz mit Millionen Parametern mit 400 Bildern von Grund auf zu trainieren, endet beim Auswendiglernen. Gleichzeitig gibt es öffentlich Hunderttausende beschriftete Alltagsfotos, an denen Netze gelernt haben, Kanten, Texturen und Formen zu unterscheiden. Das Problem ist ein Missverhältnis: zu wenige Beispiele für zu viele Parameter, und jede Schicht müsste die Grundlagen des Sehens neu entdecken.

Wir bauen das im Kleinen nach. Das **Vortraining** lernt, alle zehn Ziffern zu erkennen (900 Bilder der Ziffern-Daten von scikit-learn). Die **Zielaufgabe** ist eine andere, nämlich gerade oder ungerade, mit nur 6 bis 50 beschrifteten Beispielen; diese Bilder kommen im Vortraining nicht vor. Das ist ein günstiger Fall für Transfer, denn „gerade oder ungerade" lässt sich direkt aus der Ziffer ablesen, die das Vortraining erkennen lernt. Bei Röntgenbildern und Alltagsfotos ist die Nähe der Aufgaben nicht von vornherein so groß.

Die Lösung. Tiefe Netze lernen Features in Schichten. Die unteren erfassen eher allgemeine Muster, die oberen werden zunehmend aufgabenspezifisch. Yosinski et al. [28] haben das untersucht: Übertragene untere Schichten sind nützlich, die Übertragbarkeit sinkt, je verschiedener die Aufgaben sind, und mit übertragenen Features initialisierte Netze verallgemeinern auch nach weiterem Training besser als zufällig initialisierte. Das Pattern nutzt das so:

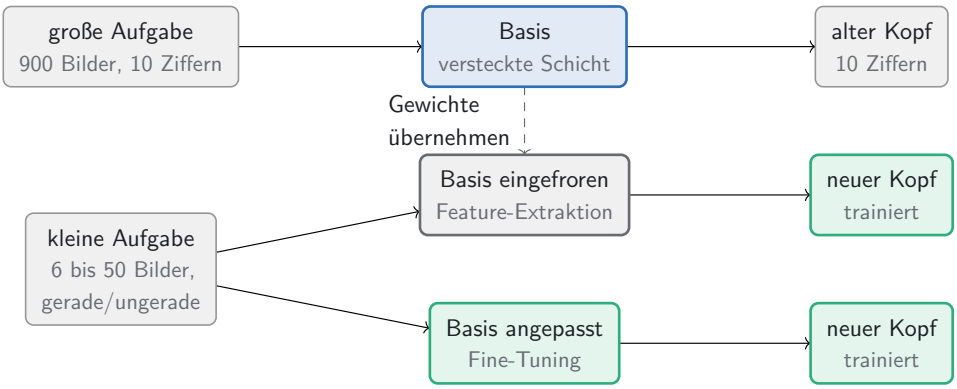


Abbildung 54: Transfer Learning: Die Basis wird übernommen, der Kopf ersetzt. Die Basis bleibt eingefroren oder wird mittrainiert.

1. Man nimmt ein Modell, das auf einer großen Aufgabe vortrainiert wurde (die **Basis**), und trennt den letzten Teil ab, den **Kopf**, der die alten Klassen ausgibt.
2. Man setzt einen neuen Kopf auf, der zu den neuen Labels passt.
3. Bei der **Feature-Extraktion** bleibt die Basis eingefroren; nur der neue Kopf wird trainiert. Beim **Fine-Tuning** wird die Basis mittrainiert, meist mit kleiner Lernrate, damit das Gelernte nicht zerstört wird.

scikit-learn kennt kein Einfrieren, deshalb bauen wir beide Varianten von Hand. Für die Feature-Extraktion rechnen wir die versteckte Schicht des vortrainierten Netzes selbst aus (`np.maximum(0, X @ W + b)` mit den Gewichten des Vortrainings) und trainieren darauf eine logistische Regression als neuen Kopf. Für das Fine-Tuning übernehmen wir die Gewichte der ersten Schicht in ein neues Netz mit neuem Kopf und trainieren alles weiter:

```
        # Fine-Tuning: erste Schicht übernehmen, neuer Kopf, alles trainieren
        ft = MLPClassifier(hidden_layer_sizes=(32,), max_iter=1, warm_start=True,
                           n_iter_no_change=1000, random_state=rep).fit(Xt, yt)
        ft.coefs_[0] = basis.coefs_[0].copy()
        ft.intercepts_[0] = basis.intercepts_[0].copy()
        ft.set_params(max_iter=200)
        ft.fit(Xt, yt)
```

Verglichen wird mit zwei Basislinien, die nichts übertragen: logistische Regression auf den Pixeln und ein Netz von Null. Jede Zeile ist der Mittelwert über 20 zufällige Stichproben:

```
Terminal
$ uv run --with scikit-learn --with numpy python transfer_learning.py
Vortraining auf 900 Bildern, Ziffernerkennung auf den übrigen 897: 0.974

Genauigkeit für gerade/ungerade, Mittel +- Std. über 20 Stichproben
Beispiele  Pixel+LogReg      von Null Feature-Extr.   Fine-Tuning
        6  0.685+-0.064  0.675+-0.058  0.734+-0.062  0.665+-0.073
       10  0.741+-0.049  0.741+-0.047  0.778+-0.040  0.730+-0.054
       20  0.798+-0.032  0.799+-0.044  0.831+-0.033  0.789+-0.050
       50  0.850+-0.020  0.870+-0.019  0.862+-0.018  0.850+-0.026
```

Die Feature-Extraktion liegt bei sehr wenigen Beispielen vorn (6 Beispiele: 0,734 gegen 0,685 für die Pixel-Regression, bei 10 und 20 gut drei bis knapp vier Prozentpunkte). Bei 50 Beispielen ist der Vorsprung weg (0,862 gegen 0,850 und 0,870 für das Netz von Null): Reichen die Daten, lernt das Modell die Features selbst. Die Effekte sind moderat, die Streuung zwischen den Stichproben (0,02 bis 0,07) ist größer als der Abstand der Mittelwerte. Das Fine-Tuning bringt in dieser Konfiguration nichts. Plausibel ist, dass das freie Training aller Gewichte mit derselben Lernrate die übernommenen Features auf 6 bis 50 Beispielen wieder überschreibt; untersucht haben wir das nicht (außerdem läuft das Fine-Tuning 200 Iterationen, das Netz von Null 500), und in Frameworks gibt man der Basis eine kleinere Lernrate als dem Kopf. Die übliche Faustregel (sie folgt nicht aus diesem kleinen Experiment): wenige Daten und ähnliche Aufgabe, dann Feature-Extraktion; viele Daten und ähnliche Aufgabe, dann Fine-Tuning; sehr verschiedene Aufgaben, dann vorsichtig sein und nur untere Schichten übernehmen.

Trade-offs und Grenzen. Die Basis legt das Eingabeformat fest: Bildgröße, Vokabular, Tokenisierung. Sie bringt außerdem ihre Herkunft mit, also die Verzerrungen ihrer Trainingsdaten und deren Lizenzbedingungen. Ist die Zielaufgabe der Quellaufgabe zu unähnlich, kann die Übertragung schaden (negativer Transfer). Große Basismodelle kosten Speicher und Latenz im Betrieb. Und vortrainierte Gewichte sind Fremdmaterial: Man sollte sie aus vertrauenswürdiger Quelle beziehen und die Version festhalten. **Wann nicht:** wenn man genug eigene Daten hat und die Domäne sehr speziell ist, oder wenn das Modell klein genug ist, um in Minuten von Grund auf zu trainieren. Das Buch zeigt das Pattern mit TensorFlow und vortrainierten Modellen (u. a. aus TF Hub).

Bezug zu Software Engineering. Transfer Learning ist Wiederverwendung statt Neuentwicklung. Die Basis ist eine Abhängigkeit, die man mit fester Version einbindet, der neue Kopf ist ein Adapter an die eigene Schnittstelle [2]. Feature-Extraktion entspricht dem Verwenden einer Bibliothek, so wie sie ist; Fine-Tuning entspricht einem Fork, den man anpasst und damit selbst pflegen muss.

> **SO ERKLÄRE ICH ES**
> „Wir bauen keinen Compiler neu, wenn wir eine Programmiersprache brauchen. Genauso müssen wir ein Netz nicht erst das Sehen lehren, wenn jemand das schon getan hat. Wir übernehmen die Schichten, die allgemeine Features bilden, und trainieren nur den Teil neu, der unsere Frage beantwortet."
>
> **Verständnisfrage:** Warum bringt die Feature-Extraktion im Experiment bei 6 Beispielen mehr als bei 50? **Antwort:** Bei wenigen Beispielen hat die Basis den größten Teil des Lernens schon erledigt, und der kleine neue Kopf kann damit auskommen. Bei mehr Beispielen kann das Modell die Features auch selbst aus den Daten lernen, der Vorsprung schrumpft (hier von etwa fünf auf rund einen Prozentpunkt).

Quellen. [1] (Kapitel 4, Pattern 13); [28].

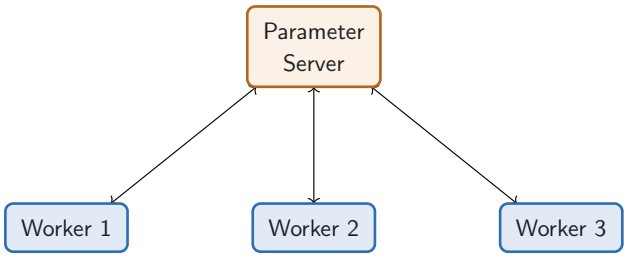
7.5 Pattern 14: Distribution Strategy

Szenario und Problem. Ein Team trainiert einen Bildklassifikator. Ein Durchlauf dauert auf einer einzelnen GPU sechs Tage (eine Annahme für dieses Beispiel), und jedes Experiment ist erst nach einer Woche bewertbar. Man möchte dieselbe Rechnung auf acht oder achtzig Geräte verteilen. Das ist weniger selbstverständlich, als es klingt, denn die Trainingsschleife ist von Natur aus sequenziell: Schritt $t+1$ braucht die Parameter, die Schritt t erzeugt hat. Man kann die Schleife nicht einfach in Stücke schneiden wie eine Schleife über unabhängige Datensätze. Dazu kommen zwei verschiedene Engpässe, die verschiedene Lösungen verlangen: Die Rechenzeit ist zu lang, oder das Modell ist zu groß für den Speicher eines einzelnen Geräts.

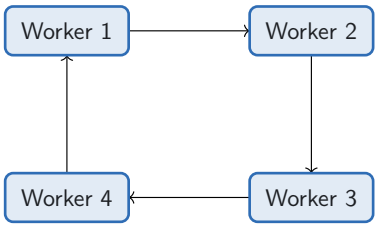
Die Lösung. Datenparallelität löst das Zeitproblem. Jeder Worker besitzt eine vollständige Modellkopie und bearbeitet einen Teil des Mini-Batches. Er berechnet den Gradienten für seinen Teil, danach werden die Gradienten gemittelt, und alle Worker führen denselben Schritt aus. Dass das Mitteln keinen Unterschied macht, lässt sich nachrechnen: Das Skript teilt 8 000 Beispiele auf vier Shards auf, und die größte Abweichung zwischen dem gemittelten Shard-Gradienten und dem Gradienten über alle Daten beträgt $1{,}8 \cdot 10^{-15}$, also Rundungsrauschen. Bei gleich großen Shards und einem Loss, der ein Mittelwert über Beispiele ist, entspricht synchrone Datenparallelität mit vier Workern à 64 Beispielen mathematisch einem Schritt mit Batch 256 auf einem Gerät, nur schneller. (Ausnahme sind Schichten, die Statistiken über den Batch bilden, etwa Batch Normalization: Dort sieht jeder Worker nur seine 64 Beispiele.) Das ist die wichtigste Einsicht dieses Patterns. Offen bleibt, wie die Gradienten zusammenkommen (Abbildung 55):

- Beim **Parameter Server** halten dedizierte Rechner die Parameter. Worker holen den aktuellen Stand, rechnen und schicken Gradienten zurück. Das System von Dean et al. [48] (DistBelief) ist so gebaut. Der Server ist die Engstelle: Sein Netzwerk muss die Daten aller Worker verkraften.
- Beim **All-Reduce** gibt es kein Zentrum. Im Ring-Verfahren zerlegt jeder Worker seinen Gradientenvektor der Länge D in N Stücke. In $N-1$ Schritten reicht jeder ein Stück an seinen Nachbarn weiter, der es aufaddiert; danach besitzt jeder Worker die Summe eines Stücks. In weiteren $N-1$ Schritten werden die fertigen Summen im Kreis verteilt. Jeder Worker sendet insgesamt $2\frac{N-1}{N} \cdot D$ Werte, also fast unabhängig von N [49].

Das Skript baut den Ring in wenigen Zeilen nach (die „Worker" sind Listeneinträge) und zählt, wie viele Werte jeder verschickt:



Parameter Server: alle Worker sprechen mit der Mitte.



Ring-All-Reduce: jeder spricht nur mit dem Nachbarn.

Abbildung 55: Zwei Wege, die Gradienten der Worker zusammenzuführen.

Terminal

```
$ uv run --with numpy python distribution_strategy.py
4) All-Reduce über einen Gradientenvektor der Länge 240.000
   Worker  Ring: je Worker  Koordinator gesamt  Summe korrekt
        4          360.000           1.440.000  True
        8          420.000           3.360.000  True
       16          450.000           7.200.000  True
```

Bei 16 Workern sendet jeder im Ring 450 000 Werte. Ein zentraler Koordinator, der (als einer der 16 Worker) 15 Gradientenvektoren einsammelt und die Summe an 15 zurückschickt, müsste allein 7,2 Millionen Werte bewegen. Das ist der Grund, warum große Trainings auf All-Reduce statt auf einen einzelnen Sammelpunkt setzen.

Die zweite Entscheidung betrifft die Synchronisation. Beim **synchronen** Training wartet jeder Schritt auf den langsamsten Worker (den „Straggler“), dafür ist das Ergebnis exakt das des großen Batches. Beim **asynchronen** Training schickt jeder Worker sein Update, sobald er fertig ist, und rechnet sofort weiter. Niemand wartet, aber die Gradienten sind „veraltet“: Sie wurden auf Parametern berechnet, die inzwischen von anderen Workern weiterverändert wurden. Das Skript simuliert das, indem ein Schritt mit dem Gradienten von vor s Schritten arbeitet:

Terminal

```
$ uv run --with numpy python distribution_strategy.py
2) Verlust nach 50 / 100 / 200 / 300 Schritten (Rauschgrenze 0.25), Lernrate 0.05
   synchron                          0.257       0.256       0.257       0.259
   asynchron, 4 Schritte alt         0.258       0.256       0.258        0.26
   asynchron, 10 Schritte alt        0.516       0.283       0.265       0.272
   asynchron, 20 Schritte alt         1.36        59.9          47    2.81e+03
```

Mit derselben Lernrate verträgt das Training Gradienten, die vier Schritte alt sind, praktisch ohne Einbuße. Bei zehn Schritten wird es langsamer, bei zwanzig kippt es und der Verlust explodiert. Die Toleranz hängt also von Lernrate und Aufgabe ab, und genau dieses Wechselspiel macht asynchrones Training heikel. Dean et al. nehmen es in ihrem asynchronen Verfahren (Downpour SGD) in Kauf, um nicht auf langsame oder ausgefallene Maschinen warten zu müssen. **Modellparallelität** ist die andere Antwort, wenn das Modell nicht in den Speicher eines Geräts passt: Man verteilt Schichten oder Teile von Schichten auf mehrere Geräte, und die Zwischenergebnisse der Schichten (Aktivierungen) wandern zwischen ihnen. Das System von Dean et al. kombinierte beides.

Zuletzt verändert Datenparallelität den effektiven Batch: Vier Worker mit je 64 Beispielen ergeben einen Batch von 256, und die Zahl der Schritte pro Epoche sinkt auf ein Viertel. Wer die Lernrate nicht anpasst, trainiert nach derselben Zahl von Epochen schlechter. Goyal et al. [31] formulieren die **Linear Scaling Rule**: Wächst der Batch um den Faktor k, wächst die Lernrate um k. Sie trainierten damit ResNet-50 (ein verbreitetes Bildklassifikationsnetz) auf ImageNet (rund 1,3 Millionen beschriftete Fotos) mit Mini-Batches bis 8 192 Beispielen in etwa einer Stunde, wobei sie zu Beginn die Lernrate langsam hochfahren („Warmup“). Unser Skript zeigt den Effekt im Kleinen:

```
Terminal
$ uv run --with numpy python distribution_strategy.py
3) Zwei Epochen Training, Verlust am Ende (Rauschgrenze 0.25)
   Batch  32, Lernrate 0.01   500 Schritte  Verlust= 0.255   Ausgangslage
   Batch 256, Lernrate 0.01    64 Schritte  Verlust= 0.738   8x Batch
   Batch 256, Lernrate 0.08    64 Schritte  Verlust= 0.256   8x Batch, 8x Rate
```

Achtfacher Batch mit unveränderter Lernrate erreicht nach zwei Epochen einen Verlust von 0,738, der achtfach vergrößerten Lernrate gelingt 0,256, fast so gut wie die Ausgangslage mit 0,255.

Trade-offs und Grenzen. Verteiltes Training skaliert nie perfekt: Die Kommunikation kostet Zeit, und bei kleinen Modellen oder kleinen Batches wird sie zum dominierenden Anteil. Mit der Zahl der Geräte wachsen Kosten und Fehlerquellen; der Ausfall eines Workers muss behandelt werden (hier helfen Checkpoints, Pattern 12). Asynchrones Training ist nicht ohne Weiteres reproduzierbar, weil die Reihenfolge der Updates vom Zufall der Laufzeiten abhängt. Und bevor man Geräte hinzufügt, sollte man prüfen, ob nicht die Eingabepipeline (Daten laden, Bilder dekodieren) das Training bremst; sie lässt sich mit zusätzlichen GPUs nicht beschleunigen. **Wann nicht:** Wenn ein Training auf einem Gerät in Minuten oder wenigen Stunden fertig ist, kostet die Verteilung mehr Komplexität, als sie Zeit spart. Und wer viele Experimente hat (Pattern 15), fährt meist besser damit, jeden Versuch auf einem eigenen Gerät laufen zu lassen, statt einen einzelnen Versuch zu verteilen.

Das Buch zeigt dieses Pattern mit den Verteilungsstrategien von TensorFlow (`tf.distribute`), die diese Bausteine hinter einer einheitlichen Schnittstelle verstecken.

Bezug zu Software Engineering. Das Muster ist Scatter-Gather in der Form von MapReduce: Teile verteilen, lokal rechnen, Ergebnisse zusammenführen. Die Wahl zwischen synchron und asynchron ist die bekannte Konsistenzfrage verteilter Systeme. Synchron ähnelt starker Konsistenz (alle sehen denselben Stand, dafür wartet man), asynchron ähnelt eventual consistency (niemand wartet, dafür arbeitet man mit veralteten Daten). Die Analogie hinkt, weil es hier um den Weg eines Optimierungsverfahrens geht und nicht um replizierte Daten. Dass 16 Geräte kein 16-faches Tempo bringen, ist Amdahls Gesetz: Der sequenzielle Anteil, hier die Kommunikation, begrenzt den Gewinn.

> **SO ERKLÄRE ICH ES**
> Vier Korrektorinnen bewerten je ein Viertel eines Klausurstapels und gleichen danach das Bewertungsschema ab. Warten alle aufeinander, ist das Schema nach jeder Runde einheitlich, aber die Langsamste bestimmt das Tempo (synchron). Trägt jede Korrektorin ihre Anpassung ins gemeinsame Schema ein, sobald sie fertig ist, geht es schneller, aber manche Anpassungen beruhen auf einer schon überholten Fassung (asynchron). Wichtig ist der Gedanke, dass mehr Rechner auch einen größeren Batch bedeuten und die Lernrate nachgezogen werden muss.
>
> **Verständnisfrage:** Warum ist synchrone Datenparallelität mit 4 Workern à 64 Beispielen mathematisch dasselbe wie ein Training mit Batch 256 auf einem Gerät?
> **Antwort:** Weil der Mittelwert der vier Teilgradienten bei gleich großen Teilen genau der Gradient über alle 256 Beispiele ist (Abweichung im Experiment: 10^{-15}). Deshalb muss man auch die Lernrate wie bei einem größeren Batch behandeln.

Quellen. [1] (Kapitel 4, Pattern 14); [48]; [31]; [49].

7.6 Pattern 15: Hyperparameter Tuning

Szenario und Problem. Wir klassifizieren handgeschriebene Ziffern (8 × 8 Pixel) mit einer Support Vector Machine (SVM), einem klassischen Verfahren, das die Klassen durch eine möglichst trennscharfe Grenze zwischen den Trainingsbeispielen unterscheidet. Die Parameter des Modells lernt das Training selbst. Daneben gibt es Einstellungen, die vor dem Training feststehen müssen: bei der SVM `C` (wie streng Fehler bestraft werden) und `gamma` (wie weit der Einfluss eines Trainingsbeispiels reicht), bei einem neuronalen Netz Lernrate, Schichtzahl oder Batch-Größe. Das sind die **Hyperparameter**. Parameter sind Zustand, den die Schleife verändert; Hyperparameter sind Konfiguration, die man ihr mitgibt.

Das Problem: Die Güte eines Modells hängt stark von dieser Konfiguration ab, und niemand kann sie vorab ausrechnen. Probiert man von Hand, ist das Vorgehen weder wiederholbar noch dokumentiert, und jeder Versuch ist ein vollständiges Training. Dazu kommt eine methodische Falle: Wer Hyperparameter anhand der Testdaten auswählt, hat die Testdaten in die Modellentwicklung einfließen lassen. Das Ergebnis ist dann zu optimistisch, und es gibt keine unabhängige Messung mehr.

Die Lösung. Man automatisiert die Suche und legt über die Trainingsschleife eine äußere Schleife: Ein Suchverfahren schlägt eine Konfiguration vor, das Modell wird trainiert und mit Kreuzvalidierung auf den Trainingsdaten bewertet (die Trainingsdaten werden in fünf Teile geteilt, jeder Teil dient einmal zur Bewertung eines Modells, das auf den übrigen vier trainiert wurde; die fünf Werte werden gemittelt), das Ergebnis geht zurück an das Verfahren. Am Ende gibt es eine beste Konfiguration, und erst jetzt kommen die Testdaten einmal zum Einsatz.

Drei Suchverfahren sind verbreitet. Bei der **Grid Search** legt man pro Hyperparameter eine Liste von Werten fest und probiert alle Kombinationen. Bei d Hyperparametern mit je k Werten sind das k^d Versuche. Bei der **Random Search** zieht man jede Konfiguration zufällig aus Verteilungen über den erlaubten Bereichen. Bei der **Bayes'schen Optimierung** baut das Verfahren aus den bisherigen Versuchen ein statistisches Ersatzmodell der Funktion „Hyperparameter → Güte" auf und wählt den nächsten Versuch dort, wo es viel Gewinn oder viel Unsicherheit erwartet [50]. Das Skript vergleicht die ersten beiden mit scikit-learn, bei gleichem Budget von 25 Versuchen:

```
C_GRID = [0.1, 1, 10, 100, 1000]
# 5 x 5 = 25 Versuche
GAMMA_GRID = [0.001, 0.01, 0.1, 1, 10]
grid = GridSearchCV(SVC(), {"C": C_GRID, "gamma": GAMMA_GRID}, cv=5)
grid.fit(X_tr, y_tr)

# gleicher Bereich
verteilung = {"C": loguniform(0.1, 1000), "gamma": loguniform(0.001, 10)}
# gleiches Budget: 25
zufall = RandomizedSearchCV(SVC(), verteilung, n_iter=25, cv=5, random_state=0)
zufall.fit(X_tr, y_tr)
```

Terminal

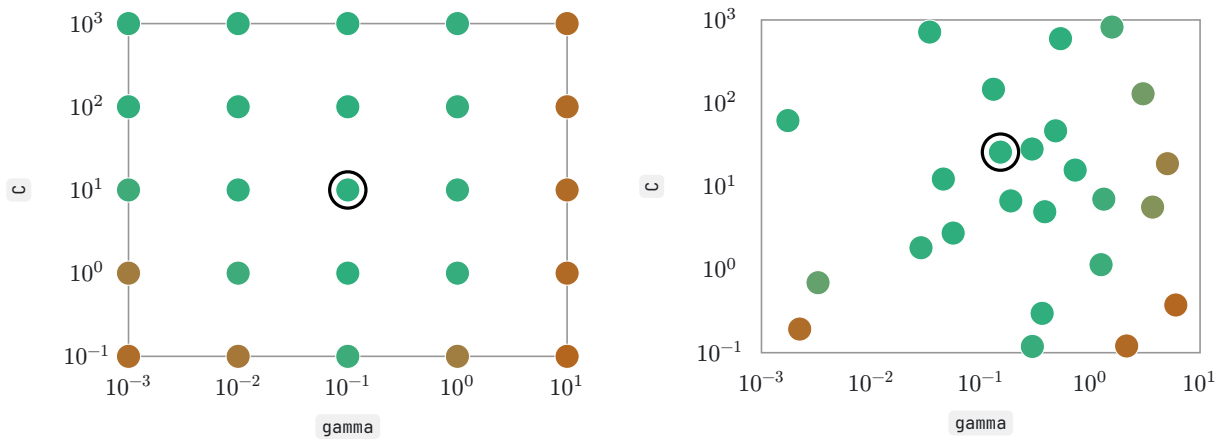
```
$ uv run --with scikit-learn --with numpy python hyperparameter_tuning.py
Gleiches Budget: 25 Versuche mit je 5-facher Kreuzvalidierung (125 Läufe)
  Verfahren             CV-Wert       C   gamma     Test
  GridSearchCV           0.9805      10     0.1   0.9917
  RandomizedSearchCV     0.9814   25.77  0.1512   0.9930
  verschiedene gamma-Werte ausprobiert: Grid=5, Random=25

Random Search mit 10 verschiedenen Seeds, bester CV-Wert:
  min=0.9814  Mittel=0.9822  max=0.9833  (Grid: 0.9805)

25 Grid-Ergebnisse: min=0.113  Median=0.965  max=0.981
  davon unter 0.5: 9 Versuche
```

Beide Verfahren landen bei etwa 98 Prozent Kreuzvalidierungsgenauigkeit und etwa 99 Prozent auf den Testdaten. Die Unterschiede sind klein: 0,9805 gegen 0,9814 im CV-Wert, und bei den 719 Testbildern entspricht der Abstand von 0,9917 zu 0,9930 einem einzigen Bild. Das ist keine Überraschung, denn die Aufgabe ist einfach und die Güte fällt in einem großen Bereich flach ab. Die Verteilung der Grid-Ergebnisse zeigt aber, wie sehr die Wahl zählt: Neun der 25 Versuche liegen unter 50 Prozent, ein Drittel des Budgets also in hoffnungslosen Ecken, und der Median liegt bei 0,965.

Bergstra und Bengio [51] erklären, warum Random Search dem Grid meist überlegen ist: Von den Hyperparametern beeinflusst oft nur eine Handvoll das Ergebnis wirklich, und man weiß vorher nicht, welche. Das Grid probiert für jeden Hyperparameter nur k verschiedene Werte, hier fünf, auch wenn es 25 Versuche macht. Die Random Search probiert in jedem Versuch einen neuen Wert für jeden Hyperparameter, hier 25 verschiedene `gamma`-Werte statt fünf (Abbildung 56). Mit zehn verschiedenen Seeds lag der beste CV-Wert der Random Search zwischen 0,9814 und 0,9833, also nie unter dem des Grids. Bei so engen Abständen sollte man das aber nicht überbewerten.



Grid Search: 25 Versuche, 5 verschiedene `gamma`

Random Search: 25 Versuche, 25 verschiedene `gamma`

Abbildung 56: Die tatsächlich probierten Konfigurationen beider Läufe (logarithmische Achsen). Grün: hoher CV-Wert, orange: niedriger. Der schwarze Ring markiert den jeweils besten Versuch.

Beide Achsen sind logarithmisch skaliert, und das ist wichtig: `C` und `gamma` wirken über mehrere Größenordnungen. Würde man `gamma` gleichmäßig zwischen 0,001 und 10 ziehen, lägen neun von zehn Versuchen über 1.

Trade-offs und Grenzen. Tuning kostet Rechenzeit: 25 Versuche mit fünffacher Kreuzvalidierung sind 125 Trainingsläufe (dazu kommt ein abschließendes Training mit der besten Konfiguration auf allen Trainingsdaten). Der beste CV-Wert ist außerdem leicht zu optimistisch, weil er das Maximum über viele verrauschte Messungen ist. Daher kommt die Regel, die Testdaten erst am Ende und nur einmal zu verwenden. Die Suchbereiche muss man selbst festlegen; liegt das Optimum außerhalb, findet es kein Verfahren. Bayes'sche Optimierung kommt meist mit weniger Versuchen aus, ist aber von Natur aus sequenziell, weil jeder Vorschlag die bisherigen Ergebnisse braucht; sie lässt sich parallelisieren, aber aufwendiger als Random Search, bei der alle Versuche unabhängig sind. scikit-learn bringt sie nicht mit; verbreitet sind Bibliotheken wie Optuna oder Keras Tuner. **Wann nicht:** Wenn ein einzelnes Training Tage dauert und das Budget fünf Versuche erlaubt, hilft kein Suchverfahren. Dann sind erfahrungsbasierte Startwerte, kleinere Probeläufe auf Teildaten oder Transfer Learning (Pattern 13) die bessere Wahl.

Bezug zu Software Engineering. Das Muster ist ein Parameter-Sweep, wie man ihn aus dem Performance-Tuning kennt: Konfiguration variieren, messen, die beste behalten. Nur dass jede Messung hier ein ganzes Training ist und die Messung rauscht, weshalb man wiederholt (Kreuzvalidierung). Die Trennung zwischen Tuning-Daten und Testdaten entspricht der Trennung zwischen dem, woran man entwickelt, und dem, wogegen man abnimmt.

> **SO ERKLÄRE ICH ES**
> „Parameter sind der Zustand, den das Programm beim Lauf verändert. Hyperparameter sind die Konfigurationsdatei, die vorher feststeht. Wir suchen die beste Konfiguration, indem wir viele ausprobieren, und messen jede an Daten, die das Modell nicht zum Lernen benutzt hat." Gut ist eine kurze Live-Demonstration: dieselbe SVM mit `gamma=10` und mit `gamma=0.1`.
>
> **Verständnisfrage:** Ein Kollege wählt `C` und `gamma` so, dass die Genauigkeit auf den Testdaten maximal ist, und berichtet diesen Wert. Was ist daran falsch?
> **Antwort:** Die Testdaten haben die Auswahl mitbestimmt, der Wert ist damit nicht mehr unabhängig und zu optimistisch. Richtig ist, auf einem Validierungsteil oder per Kreuzvalidierung zu wählen und die Testdaten genau einmal am Ende zu benutzen.

Quellen. [1] (Kapitel 4, Pattern 15); [51]; [50]; scikit-learn [42].

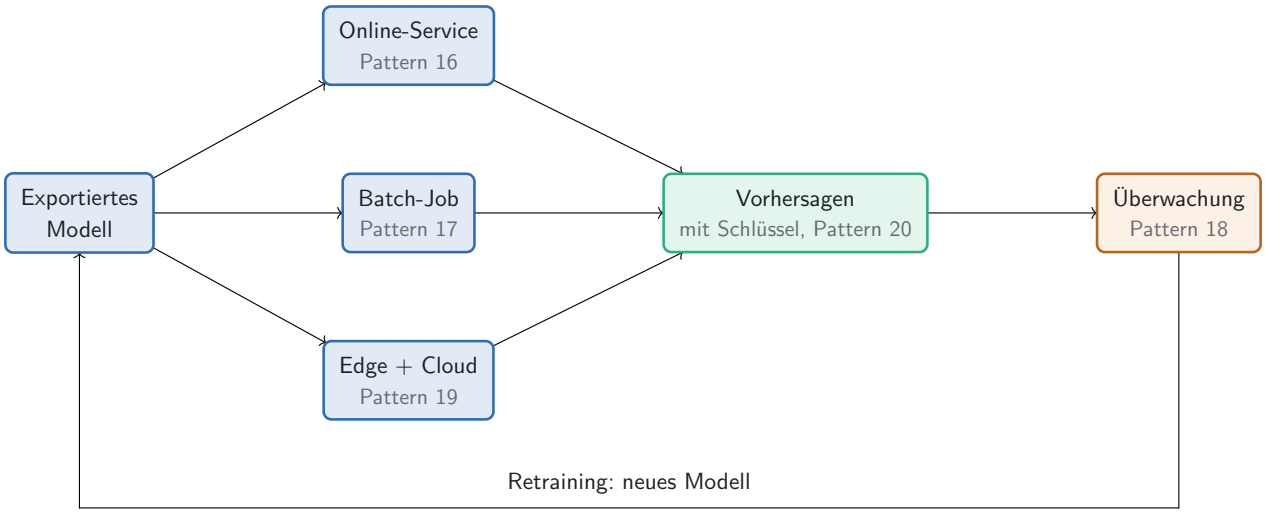


Abbildung 57: Die fünf Patterns dieses Kapitels. Ein Modell wird auf drei Wegen ausgeliefert, seine Vorhersagen tragen Schlüssel, und die Überwachung stößt bei Bedarf ein neues Training an.

TEIL III · TRAINING UND BETRIEB

8 Resilient Serving

Ein trainiertes Modell nützt erst, wenn jemand seine Vorhersagen abrufen kann. Das nennt man **Serving** (auch Inferenz): das Modell mit neuen Eingaben füttern und die Antwort zurückliefern. Dabei gibt es zwei Grundformen. Beim **Online-Serving** fragt ein Client einzelne Vorhersagen ab und erwartet die Antwort in Millisekunden, etwa eine App, die eine Lieferzeit anzeigt. Beim **Batch-Serving** bewertet ein Job viele Datensätze auf einmal, ohne dass jemand wartet, etwa über Nacht. Das Kapitel entspricht Kapitel 5 im Buch von Lakshmanan et al. [1]. „Resilient" bedeutet hier: Der Betrieb soll skalieren, Ausfälle verkraften und merken, wenn das Modell mit der Zeit schlechter wird.

8.1 Pattern 16: Stateless Serving Function

Szenario und Problem. Ein Lieferdienst möchte in seiner App die erwartete Lieferzeit anzeigen. Das Modell bekommt Entfernung, Artikelzahl, Uhrzeit und Wetter (5 000 synthetische Bestellungen mit selbst festgelegten Zusammenhängen, damit wir das Ergebnis prüfen können). Die Antwort soll in unter 100 Millisekunden kommen, zu Stoßzeiten für tausende Nutzer gleichzeitig.

Im Notebook liegt das Modell als Python-Objekt im Arbeitsspeicher, und daraus einen Dienst zu machen, scheitert auf drei Arten. Der Prozess, in dem das Objekt entstand, skaliert nicht. Die Vorverarbeitung steckt im Notebook-Code und müsste im Dienst nachgebaut werden; weicht dort das Feature „Abendspitze" (Bestellstunde 18, 19 oder 20) auch nur leicht ab, rechnet das Modell mit Eingaben, die es so nie gesehen hat (**Training-Serving Skew**). Und viele Dienste bauen unbemerkt Zustand auf, etwa Caches, Sitzungsdaten oder Zähler im Prozess; dann liefern zwei Kopien nicht mehr dasselbe, und man kann keine weiteren starten.

Die Lösung. Man exportiert Modell **und** Vorverarbeitung als ein Artefakt und stellt es hinter eine Funktion, die ihr Ergebnis nur aus den Argumenten berechnet. Das Modell wird einmal beim Start geladen und danach nur gelesen; zwischen zwei Aufrufen bleibt nichts zurück. In scikit-learn ist das Artefakt eine `Pipeline` aus Feature-Berechnung und Regressor, die sich als Datei speichern lässt:

```
    modell = Pipeline([
        ("merkmale", FunctionTransformer(abendspitze_ergaenzen)),
```

```
        ("regressor", GradientBoostingRegressor(random_state=0)),
    ]).fit(X_tr, y_tr)
```

`joblib.dump(modell, "modell.joblib")` schreibt die Datei (140 KiB). Der mittlere absolute Fehler (MAE) auf den Testdaten liegt bei 2,03 Minuten. Das ist praktisch die Untergrenze: Bei normalverteiltem Rauschen mit Standardabweichung 2,5 Minuten, wie es in die Daten eingebaut ist, hat selbst ein perfektes Modell einen MAE von rund 0,8 mal 2,5, also etwa 2,0 Minuten. Die Funktion `abendspitze_ergaenzen` steht bewusst in einer eigenen Datei: Joblib (wie Pickle) speichert eine Funktion nicht als Code, sondern als Verweis auf Modul und Namen. Beim Laden muss `merkmale.py` in derselben Version importierbar sein, sonst scheitert das Laden oder läuft mit anderem Verhalten. Modell und Hilfscode werden deshalb gemeinsam in einem Paket oder Container-Image versioniert. Die Serving-Funktion selbst ist kurz, FastAPI übernimmt die Validierung der Eingaben:

```
MODELL = joblib.load(Path(__file__).with_name("modell.joblib"))
app = FastAPI(title="Lieferzeit-Service")


class Anfrage(BaseModel):
    distanz_km: float = Field(gt=0, le=50)
    artikel: int = Field(ge=1, le=50)
    stunde: int = Field(ge=0, le=23)
    regen: int = Field(ge=0, le=1)


@app.post("/predict")
def predict(anfrage: Anfrage):
    X = pd.DataFrame([anfrage.model_dump()])
    return {"lieferzeit_min": round(float(MODELL.predict(X)[0]), 1)}
```

Der Test braucht keinen laufenden Server, denn `TestClient` ruft die Anwendung direkt auf. Er prüft außerdem die Zustandslosigkeit: Dieselben 30 Anfragen laufen einmal der Reihe nach reihum über drei unabhängig geladene Kopien des Modells und einmal gemischt über zufällig gewählte Kopien.

Terminal

```
$ uv run --with scikit-learn --with pandas --with fastapi \
    --with httpx python serving_test.py
POST /predict {'distanz_km': 6.4, 'artikel': 3, 'stunde': 19, 'regen': 1}
  -> 200 {'lieferzeit_min': 34.4}
POST /predict mit stunde=31
  -> 422 Input should be less than or equal to 23

30 Anfragen, einmal reihum, einmal gemischt auf zufällige Kopien:
identische Antworten = True
```

Gleiche Eingabe, gleiche Antwort, unabhängig von Reihenfolge und Kopie: Das ist Zustandslosigkeit im praktischen Sinn. Damit lassen sich beliebig viele Kopien hinter einen Lastverteiler stellen, bei Last weitere starten und ausgefallene ersetzen (Abbildung 58).

Trade-offs und Grenzen. Jeder Aufruf zahlt Latenz für Netz und Serialisierung, und eine neue Kopie muss erst das Modell laden (**Kaltstart**), bei großen Modellen Sekunden bis Minuten. Braucht die Vorhersage Wissen über den Nutzer, etwa seine letzten zehn Bestellungen, darf es nicht im Dienst liegen: Es muss mit der Anfrage kommen oder aus einem externen Speicher geholt werden, zum Beispiel einem Feature Store (Pattern 26), was Latenz kostet. Joblib und Pickle führen beim Laden Code aus, man lädt also nur eigene, vertrauenswürdige Dateien. Sprachneutrale Austauschformate wie ONNX vermeiden das Modul-Problem, verlangen aber, dass sich auch die Vorverarbeitung in ihren Operatoren ausdrücken lässt; das Buch exportiert TensorFlow-Modelle als SavedModel und definiert die Serving-Funktion als dessen Signatur. **Wann nicht:** für Millionen Datensätze ohne Zeitdruck (Pattern 17) oder wenn das Ergebnis ohne Netz verfügbar sein muss (Pattern 19).

Bezug zu Software Engineering. Das ist die bekannte Regel für skalierbare Dienste: zustandslose Prozesse, Zustand in externen Speicher. Fielding nennt „stateless“ als Einschränkung des REST-Architek-

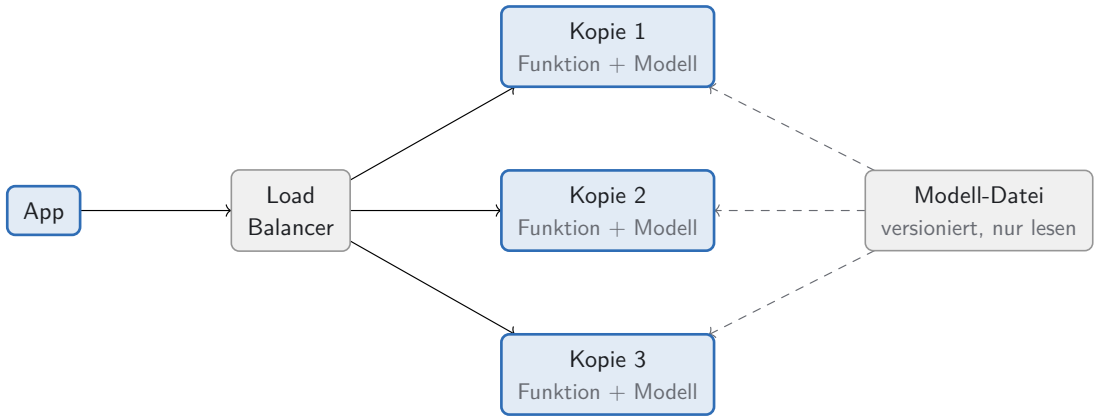


Abbildung 58: Jede Kopie lädt beim Start dieselbe Datei und hält sonst nichts. Der Lastverteiler darf Anfragen beliebig verteilen.

turstils [52], und die Twelve-Factor-App verlangt dasselbe von Prozessen [53]. Die Serving-Funktion ist eine reine Funktion: dieselben Argumente, dasselbe Ergebnis, keine Nebenwirkungen. Dass Modell und Vorverarbeitung gemeinsam ausgeliefert werden, ist Single Source of Truth, das in Pattern 21 (Transform) wiederkehrt.

> **SO ERKLÄRE ICH ES**
> „Das Modell ist eine Funktion von Features zu einer Vorhersage. Wir packen sie samt Vorverarbeitung in eine Datei, und jeder Dienstprozess lädt dieselbe Datei. Wenn ich hundert Kopien starten und Anfragen würfeln kann, ist der Dienst zustandslos."
>
> **Verständnisfrage:** Zwei identische Anfragen gehen an zwei Kopien des Dienstes und liefern verschiedene Antworten. Wo suchen Sie?
> **Antwort:** Nach versteckten Unterschieden: verschiedene Modellversionen auf den Kopien (etwa mitten in einem Rollout), Zustand, der zwischen Aufrufen im Prozess bleibt, oder Zufallselemente ohne festen Seed. Im Experiment gibt es keinen davon, deshalb sind die Antworten identisch.

Quellen. [1] (Kapitel 5, Pattern 16); [52].

8.2 Pattern 17: Batch Serving

Szenario und Problem. Der Lieferdienst will jeden Morgen für alle 200 000 offenen Bestellungen die erwartete Lieferzeit berechnen, um Fahrer einzuplanen. Niemand wartet auf eine einzelne Antwort; wichtig ist, dass bis sieben Uhr alles fertig ist. Naheliegend wäre, den Online-Dienst aus Pattern 16 200 000 Mal aufzurufen. Das funktioniert, ist aber verschwenderisch: Jeder Aufruf zahlt Fixkosten für Netz, Serialisierung, Validierung und das Aufbereiten einer einzelnen Zeile, und der Dienst müsste auf den Ansturm dimensioniert werden, obwohl er den Rest des Tages leer läuft. Das ist das N+1-Problem der Datenbankwelt: ein Roundtrip pro Zeile statt einer Abfrage für alle.

Die Lösung. Man bewertet die Daten in großen Portionen. Ein Modell bewertet viele Zeilen in einem Durchlauf, vektorisiert und ohne Python-Schleife, und der Fixaufwand pro Aufruf (Eingabe prüfen, DataFrame bauen, Pipeline durchlaufen) fällt nur einmal an. Das Skript vergleicht mit dem Modell aus Pattern 16 Einzelaufrufe und eine Batch-Vorhersage auf denselben Daten:

```
# a) Einzelaufrufe, wie ein Online-Service sie sieht
N_EINZEL = 1000
t = time.perf_counter()
for i in range(N_EINZEL):
    modell.predict(X.iloc[[i]])
einzel = N_EINZEL / (time.perf_counter() - t)

# b) alles auf einmal
t = time.perf_counter()
modell.predict(X)
alles = len(X) / (time.perf_counter() - t)
```

```
Terminal
$ uv run --with scikit-learn --with pandas --with numpy python batch_serving.py
a/b) Durchsatz in Vorhersagen pro Sekunde
  Einzelaufrufe (1000):             271
  eine Batch-Vorhersage (200.000):    602.228
  Faktor: 2220

c) 200.000 Zeilen in 4 Stücken bewertet, 1.2 s; erste Zeilen:
 bestell_id  lieferzeit_min
          0           39.8
          1           48.8
          2           45.0
```

Ein Einzelaufruf schafft hier rund 270 Vorhersagen pro Sekunde, die Batch-Vorhersage rund 600 000, also mehr als das Zweitausendfache. Die genauen Zahlen hängen von Maschine und Auslastung ab: Ein zweiter Lauf auf demselben Rechner lieferte 421 und 440 938 Vorhersagen pro Sekunde, einen Faktor von rund 1 000. Die Größenordnung ist das Ergebnis, nicht die dritte Stelle: Der Batch ist um drei Größenordnungen schneller. Der Einzelaufruf misst außerdem nur Python und scikit-learn, ohne Netz und Serialisierung. Ein echter Dienst wäre also eher langsamer.

Der dritte Teil zeigt die Betriebsform: Die Eingabedatei wird in Stücken von 50 000 Zeilen gelesen, jedes Stück bewertet und sofort an die Ausgabedatei angehängt, sodass der Speicherbedarf unabhängig von der Dateigröße bleibt. Die Spalte `bestell_id` wandert unverändert durch den Job und steht in der Ausgabe neben der Vorhersage (mehr dazu in Pattern 20). Wird die Datenmenge zu groß für eine Maschine, verteilt man dieselbe Struktur auf viele Rechner, etwa mit Spark, Apache Beam oder einer SQL-Engine, die Modelle direkt aufrufen kann; das Buch beschreibt das Pattern mit solcher verteilten Datenverarbeitung. Das Prinzip bleibt gleich: Daten in Teile zerlegen, Teile unabhängig bewerten, Ergebnisse mit Schlüssel zusammenführen.

Trade-offs und Grenzen. Batch-Vorhersagen sind so alt wie der letzte Lauf: Eine Bestellung, die um 9 Uhr eingeht, hat bis zum nächsten Morgen keine Vorhersage. Man berechnet und speichert auch Vorhersagen, die nie jemand abruft. Der Job muss mit Teilausfällen umgehen können: Läuft er nach einem Abbruch neu an, sollte er dieselben Schlüssel überschreiben und nicht doppelt anlegen (Idempotenz). Und die Features müssen im Batch-Job genauso berechnet werden wie im Online-Dienst, sonst droht wieder Training-Serving Skew; am einfachsten gelingt das, wenn beide dieselbe Pipeline-Datei laden. **Wann nicht:** wenn die Eingabe erst zur Laufzeit entsteht (Suchanfrage, Kamerabild) oder die Antwort frisch sein muss. Häufig kombiniert man beides: Das Batch-Ergebnis deckt den Normalfall ab, der Online-Dienst die Nachzügler.

Bezug zu Software Engineering. Batch- gegen Online-Verarbeitung ist ein Grundthema der Datenarchitektur [54]: Offline-Systeme optimieren Durchsatz, Online-Systeme Antwortzeit. Das Muster ist die Bulk-Schnittstelle statt vieler Einzelaufrufe, die Abhilfe gegen das N+1-Problem, und der Aufwand wird wie bei jeder Stapelverarbeitung über viele Elemente amortisiert.

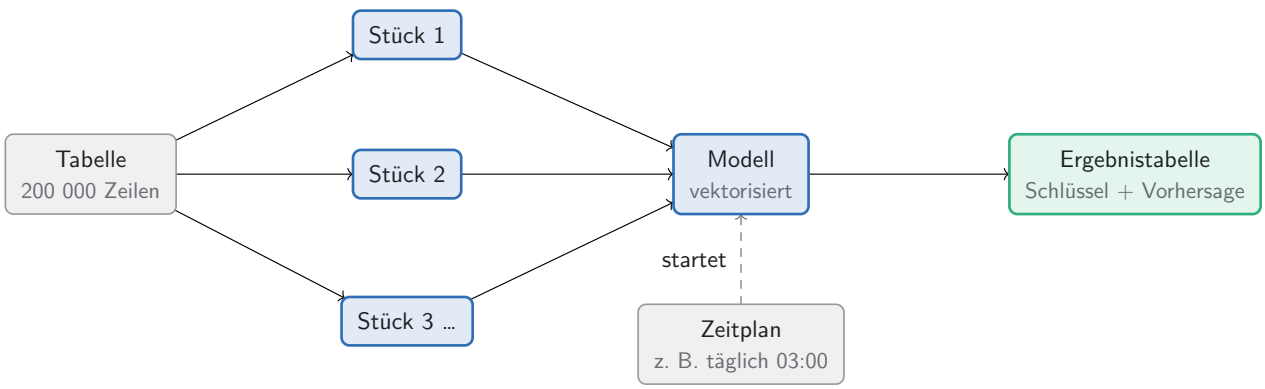


Abbildung 59: Batch Serving: Die Tabelle wird in Stücke zerlegt, jedes Stück in einem Zug bewertet, die Vorhersagen landen mit ihrem Schlüssel in einer Ergebnistabelle.

SO ERKLÄRE ICH ES
„Wer 200 000 Mal einzeln nachfragt, zahlt 200 000 Mal die Fixkosten. Das ist wie ein SELECT pro Zeile in einer Schleife. Ein Batch-Job liest alles auf einmal, rechnet vektorisiert und schreibt die Ergebnisse mit Schlüssel zurück."

Verständnisfrage: Wann würden Sie trotz des Durchsatzvorteils den Online-Dienst wählen?
Antwort: Wenn die Eingaben erst zur Laufzeit existieren (etwa eine Suchanfrage) oder die Vorhersage aktuell sein muss und ein gestriger Wert nichts nützt.

Quellen. [1] (Kapitel 5, Pattern 17); [54].

8.3 Pattern 18: Continued Model Evaluation

Szenario und Problem. Ein Onlinehändler sagt voraus, ob ein Kunde innerhalb von 30 Tagen wiederkommt. Das Modell wird mit den Daten der ersten vier Wochen trainiert, erreicht rund 86 Prozent Genauigkeit und wird ausgeliefert. Dann passiert zunächst nichts Sichtbares: keine Fehlermeldung, keine Ausnahme, kein Absturz. Trotzdem wird das Modell im Lauf der Monate schlechter, weil sich die Welt ändert, in der es eingesetzt wird: neue Produkte, eine Preisaktion, ein neuer Marketingkanal. Code verhält sich bei gleicher Eingabe morgen wie heute. Ein Modell kann auch ohne jeden Eingriff schlechter werden, weil es ein Abbild der Daten von gestern ist.

Der Fachbegriff dafür ist **Drift**. Gama et al. [12] unterscheiden, was sich ändert. Beim **Covariate Shift** ändert sich die Verteilung der Eingaben ($P(x)$), etwa weil plötzlich andere Kunden kommen, der Zusammenhang zwischen Eingabe und Ergebnis bleibt aber. Das entspricht dem, was Gama et al. **virtual drift** nennen. Beim **Concept Drift** im engeren Sinn (**real drift**) ändert sich der Zusammenhang selbst ($P(y|x)$): Dieselbe Eingabe führt nun zu einem anderen Ergebnis. Auch die zeitliche Form unterscheiden sie: plötzlich (sudden), inkrementell (viele kleine Schritte), graduell (die Daten wechseln eine Zeit lang zwischen altem und neuem Zustand hin und her) oder wiederkehrend (etwa saisonal).

Die Lösung. Das Pattern verlangt, das Modell nach dem Ausliefern weiter zu bewerten. Dazu loggt man Eingaben und Vorhersagen mit einem Schlüssel (Pattern 20), führt das tatsächliche Ergebnis nachträglich zu, sobald es bekannt ist (im Beispiel nach 30 Tagen), und berechnet regelmäßig Kennzahlen, die man mit Schwellen vergleicht. Die **Eingaben** lassen sich sofort überwachen, ohne das Ergebnis zu kennen, zum Beispiel mit dem Population Stability Index (PSI), der die Verteilung einer Eingabe mit der aus dem Training vergleicht. Die **Qualität** (Genauigkeit gegen die Wahrheit) ist die entscheidende Größe, steht aber erst mit Verzögerung zur Verfügung.

Das Skript simuliert 24 Wochen mit je 2 000 Fällen und zwei Features x_1 und x_2 (etwa Bestellhäufigkeit und Warenkorbwert). Die ersten vier Wochen dienen dem Training, in den Wochen 4 bis 7 bleibt alles gleich. In den Wochen 8 bis 13 verschiebt sich x_1 um 1,5 Standardabweichungen nach oben, der Zusammenhang bleibt (Covariate Shift). Ab Woche 14 wandert die Beziehung zwischen Features und Ergebnis über vier Wochen in eine neue (Concept Drift, inkrementell). Das alte Modell wird jede Woche bewertet. Der PSI teilt die Werte der Referenzwochen in zehn gleich stark besetzte Klassen (Dezile) und vergleicht, wie viel

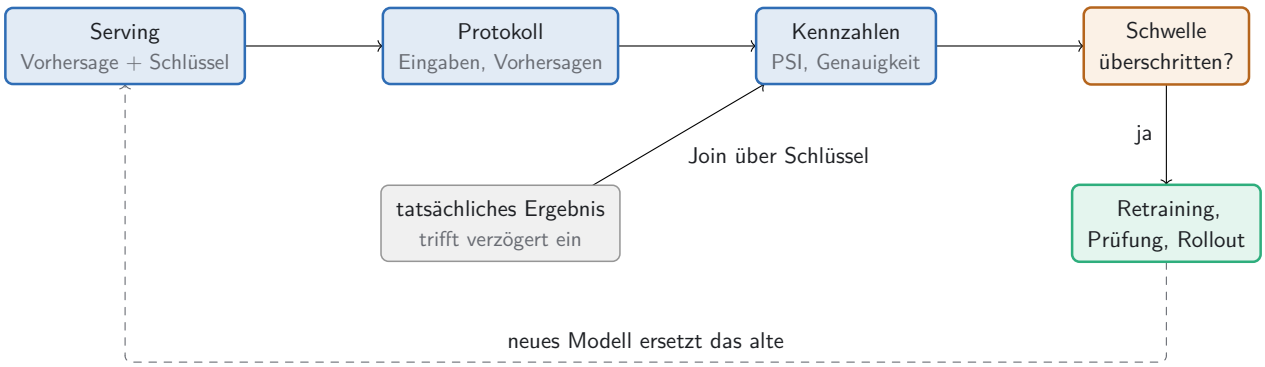


Abbildung 60: Der Überwachungskreislauf. Die Qualität wird erst gemessen, wenn das tatsächliche Ergebnis über den Schlüssel zugeordnet werden kann.

Anteil der aktuellen Woche in jede Klasse fällt. Zusätzlich prüft der Kolmogorov-Smirnov-Test (KS-Test, Kapitel 2), ob die Werte der aktuellen Woche aus derselben Verteilung stammen wie die der Referenz.

Terminal

```
$ uv run --with scikit-learn --with numpy --with scipy python drift.py
Woche Phase             Genauigkeit  PSI(x1)   KS-p(x1)  Alarm
...
    7 stabil                  0.857    0.005       0.81
    8 Covariate Shift         0.935    2.037   4.6e-321  Eingaben
...
   13 Covariate Shift         0.936    2.083   4.5e-321  Eingaben
   14 Concept Drift           0.809    0.008       0.51
   15 Concept Drift           0.706    0.018      0.089  Qualität
...
   23 Concept Drift           0.545    0.013     0.0015  Qualität

Qualitätsalarm erstmals in Woche 15.
Retraining in Woche 17 mit den Wochen 14-17:
  Woche 18: altes Modell=0.541  neues Modell=0.783
...
Zum Vergleich: Modell nur mit den Wochen 18-21 (Beziehung stabil):
  Woche 22: 0.863
  Woche 23: 0.856
```

Drei Dinge fallen auf. Erstens lösen die Wochen 8 bis 13 den Eingabe-Alarm aus: Der PSI liegt bei 2,0 bis 2,2, weit über der verbreiteten Faustregel von 0,25 (sie stammt aus der Kreditrisikomodellierung und hat keine theoretische Begründung), und der KS-Test liefert p-Werte von praktisch null. Die Genauigkeit sinkt dabei nicht, sie steigt auf rund 94 Prozent, weil die verschobenen Fälle im Mittel weiter von der Entscheidungsgrenze entfernt liegen: ein Alarm ohne Schaden. Zweitens ist es beim Concept Drift ab Woche 14 umgekehrt: Die Genauigkeit fällt von 0,86 über 0,81 und 0,71 auf rund 0,54, kaum besser als Raten bei zwei Klassen, während der PSI bei 0,01 bleibt. Die Eingaben sehen unverändert aus, nur ihre Bedeutung hat sich geändert: ein Schaden ohne Alarm. Wer sich auf einen der beiden Signalwege allein verlässt, irrt sich also in beide Richtungen. Drittens löst die Schwelle von 0,80 den Qualitätsalarm in Woche 15 aus; in Woche 14 liegt der Wert mit 0,809 knapp darüber.

Zwei Wochen nach dem Alarm wird mit den vier jüngsten Wochen neu trainiert (im Experiment steht das für Datenaufbereitung und Freigabe). Das neue Modell erreicht in den Folgewochen 0,78 statt 0,54. Es ist aber nicht wieder so gut wie am Anfang. Der Grund liegt in den Trainingsdaten: Die Wochen 14 bis 16 enthalten noch die wandernde Beziehung. Ein Modell, das nur mit den stabilen Wochen 18 bis 21 trainiert wird, erreicht in den Wochen 22 und 23 dagegen 0,863 und 0,856, so viel wie das ursprüngliche

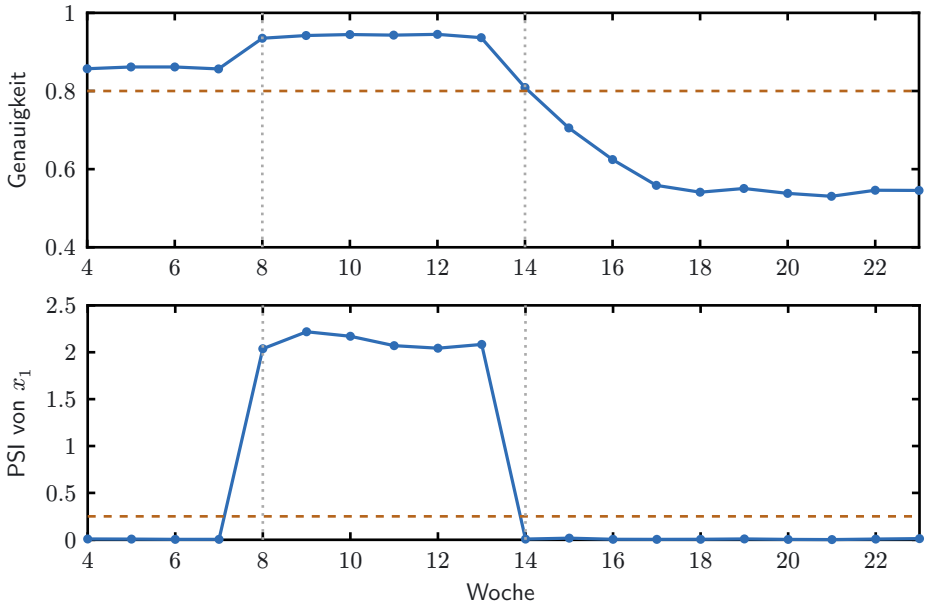


Abbildung 61: Genauigkeit (oben) und PSI der Eingabe x_1 (unten) je Woche. Orange gestrichelt: Alarmschwellen 0,80 und 0,25. Die gepunkteten Linien trennen die Phasen stabil, Covariate Shift (Wochen 8 bis 13) und Concept Drift (ab Woche 14).

Modell zu Beginn. (In Woche 17 weiß man allerdings noch nicht, dass die Beziehung ab Woche 18 stabil bleibt; der Vergleich zeigt den Effekt des Fensters, kein umsetzbares Vorgehen.) Das Datenfenster für das Retraining ist also eine eigene Entscheidung: Ein zu kurzes verschenkt Daten, ein zu langes mischt veraltete Zustände ein.

Trade-offs und Grenzen. Die Qualität lässt sich nur messen, wenn die Wahrheit bekannt ist, und die kommt oft spät oder nie: Wer einen Kredit ablehnt, erfährt nicht, ob der Kunde ihn zurückgezahlt hätte. Dann bleiben von Hand beschriftete Stichproben oder Stellvertretergrößen wie die Verteilung der Vorhersagen. Schwellen sind Abwägungen: Ein enger PSI-Schwellwert erzeugt Fehlalarme (wie in den Wochen 8 bis 13), ein weiter verpasst Veränderungen, und bei großen Stichproben schlägt jeder statistische Test schon bei winzigen Verschiebungen an. Modelle können ihre eigenen Daten verändern: Wer nur die Kunden sieht, denen das Modell Werbung zeigt, lernt aus einer Auswahl, die das Modell selbst getroffen hat. Und Retraining ist kein Selbstläufer; ein automatisch trainiertes Modell kann aus fehlerhaften Daten Falsches lernen, deshalb gehört vor den Austausch eine Prüfung gegen das alte Modell. Neben dem leistungsbasierten Auslöser aus dem Beispiel sind zeitbasierte (monatlich) und datenbasierte (genug neue Beispiele, PSI-Alarm) üblich. **Wann nicht:** Eine einmalige Auswertung, die nie in den Betrieb geht, braucht keine laufende Bewertung, alles andere schon.

Bezug zu Software Engineering. Das ist Monitoring und Alerting, angewandt auf die Qualität einer Vorhersage statt auf CPU-Last. Die Qualität des Modells ist ein Service Level Indicator, die Schwelle ein Service Level Objective. Sculley et al. [11] zählen den Umgang mit einer sich ändernden Umwelt und die laufende Überwachung zu den versteckten technischen Schulden von ML-Systemen, und der ML Test Score von Breck et al. [55] enthält eine eigene Kategorie von Überwachungstests, unter anderem zu veränderten Eingabeverteilungen und zur Alterung des Modells. Auch das Retraining gehört in die Pipeline: Ein neues Modell wird wie jeder Release geprüft, bevor es das alte ablöst.

> **SO ERKLÄRE ICH ES**
> „Code ändert sich nur, wenn wir ihn ändern. Ein Modell ist dagegen eingefroren auf dem Stand der Welt, in der es trainiert wurde. Die Welt ändert sich weiter. Deshalb messen wir im Betrieb laufend, wie gut das Modell noch ist, so wie wir Antwortzeiten und Fehlerraten messen."
>
> **Verständnisfrage:** Die Eingaben haben sich deutlich verschoben (hoher PSI), die Genauigkeit ist gleich geblieben. Trainieren Sie neu?
> **Antwort:** Nicht automatisch. Das ist ein virtual drift: Das Modell trifft weiterhin zu, obwohl die Eingaben anders aussehen (im Experiment die Wochen 8 bis 13). Man schaut sich den Alarm an und beobachtet weiter, denn eine Verschiebung kann Vorbote eines echten Drifts sein oder das Modell in Bereiche führen, die es kaum kennt.

Quellen. [1] (Kapitel 5, Pattern 18); [12]; [11]; [55].

8.4 Pattern 19: Two-Phase Predictions

Szenario und Problem. Ein Smartphone soll handgeschriebene Ziffern erkennen, zum Beispiel beim Ablesen eines Zählerstands. Ein großes Modell in der Cloud erkennt sie sehr zuverlässig, aber jede Anfrage kostet Netzverkehr, Wartezeit und Geld, funktioniert ohne Verbindung nicht und überträgt das Bild an einen Server. Ein kleines Modell auf dem Gerät antwortet sofort und ohne Netz, ist aber ungenauer, weil das Gerät wenig Speicher und Rechenleistung hat. Wir bilden das nach: Das kleine Modell (**Edge**) sieht nur eine auf 4 × 4 Pixel verkleinerte Version des Bildes und rechnet eine logistische Regression, das große (**Cloud**) sieht alle 8 × 8 Pixel und ist eine SVM.

Die Lösung. Beide Modelle arbeiten nacheinander. In der ersten Phase antwortet das kleine Modell und gibt zusätzlich an, wie sicher es sich ist, hier als höchste Klassenwahrscheinlichkeit. Liegt die Sicherheit über einem Schwellwert τ, ist die Anfrage erledigt. Andernfalls geht sie in der zweiten Phase an das große Modell. Die einfachen Fälle, und das sind die meisten, verlassen das Gerät nie.

Der Schwellwert steuert den Kompromiss. Das Skript durchläuft mehrere Werte auf den Testdaten und zeigt, welcher Anteil weitergereicht wird und wie genau die kombinierte Vorhersage ist. Die Kernfunktion ist kurz: Das kleine Modell antwortet bei ausreichender Sicherheit, sonst entscheidet das große.

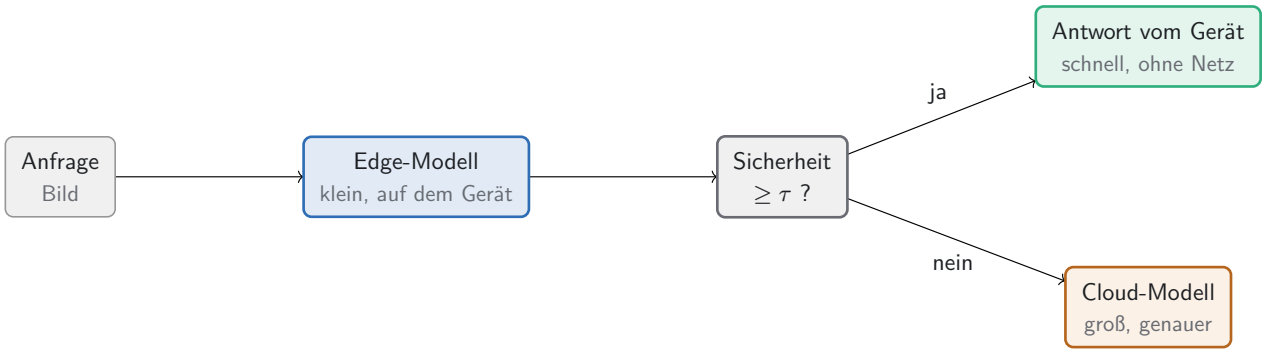


Abbildung 62: Zwei-Phasen-Vorhersage: Das Gerät entscheidet, was es selbst beantwortet und was es weiterreicht.

```python
def zweiphasig(X, tau):
    """Edge entscheidet bei Sicherheit >= tau, sonst fragt es die Cloud."""
    p = edge.predict_proba(klein(X))
    sicherheit, vorhersage = p.max(axis=1), edge.classes_[p.argmax(axis=1)]
    weiter = sicherheit < tau
    if weiter.any():
        vorhersage = vorhersage.copy()
        vorhersage[weiter] = cloud.predict(X[weiter])
    return vorhersage, weiter
```

Terminal

```
$ uv run --with scikit-learn --with numpy python two_phase.py
Edge-Modell (16 Merkmale):  Genauigkeit Test=0.869
Cloud-Modell (64 Merkmale): Genauigkeit Test=0.980

Schwellwert tau auf den Testdaten
   tau   an Cloud weitergereicht   Genauigkeit gesamt   Edge allein entschieden
  0.00                     0.0%                0.869                     0.869
  0.50                    33.1%                0.949                     0.940
  0.70                    58.0%                0.978                     0.989
  0.80                    75.6%                0.980                     1.000
  0.90                    93.3%                0.980                     1.000
  0.95                    99.8%                0.980                     1.000
  0.99                   100.0%                0.980                       nan
  1.01                   100.0%                0.980                       nan

tau=0.59 (auf Validierungsdaten gewählt)
  Genauigkeit Test: 0.969, an die Cloud weitergereicht: 44.9%
  mittlere Latenz: 59 ms gegen 120 ms nur Cloud
  (angenommen: 5 ms Edge, 120 ms Cloud inkl. Netz)

Edge-Modell auf den Testdaten: Sicherheit gegen tatsächliche Trefferquote
  Sicherheit  Anzahl  Mittel Sicherheit  Trefferquote
  0.0 bis 0.5     149              0.38          0.72
  0.5 bis 0.7     112              0.59          0.86
  0.7 bis 0.9     159              0.80          0.99
  0.9 bis 1.0      30              0.92          1.00
```

Das Edge-Modell allein erreicht 0,869, das Cloud-Modell 0,980. Bei $\tau = 0{,}5$ reicht das Gerät nur 33 Prozent der Anfragen weiter und kommt auf 0,949; bei $\tau = 0{,}7$ sind es 58 Prozent bei 0,978, praktisch die Genauigkeit der Cloud (Abbildung 63). Erhöht man τ weiter, steigt der Anteil der weitergereichten Anfragen, die Genauigkeit aber nicht mehr.

Wichtig ist, wie τ gewählt wird: nicht auf den Testdaten, sondern auf Validierungsdaten, und nach einer Vorgabe, die man begründen kann. Die Testdaten bestätigen das Ergebnis nur noch. Außerdem setzt das Verfahren voraus, dass die Sicherheit des kleinen Modells etwas bedeutet. Dafür müssen die Wahrscheinlichkeiten die Rangfolge der Verlässlichkeit richtig abbilden: Wo das Modell sicherer ist, liegt es öfter richtig. Das Skript prüft das am Ende. Das Edge-Modell ist dabei nicht kalibriert, sondern zu vorsichtig: Bei einer Sicherheit zwischen 0,7 und 0,9 (im Mittel 0,80) liegt es in 99 Prozent der Fälle richtig, nur die Rangfolge stimmt. Für die Wahl von τ genügt das, denn τ wird ohnehin an Daten abgestimmt.

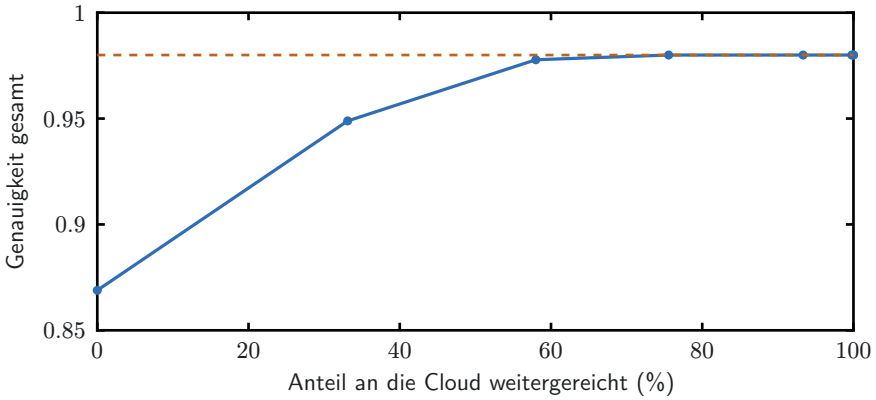


Abbildung 63: Genauigkeit der Zwei-Phasen-Vorhersage gegen den Anteil weitergereichter Anfragen (Testdaten, acht Schwellwerte). Orange gestrichelt: Genauigkeit des Cloud-Modells allein.

Moderne neuronale Netze haben dagegen oft den umgekehrten Fehler und sind zu selbstsicher; ihre Wahrscheinlichkeiten müssen vor der Schwellenwahl kalibriert werden [22], sonst entscheidet das Gerät zu viel selbst.

Mit dieser Vorgabe (kleinster Wert, der auf den Validierungsdaten höchstens einen Prozentpunkt unter der Cloud bleibt) ergibt sich $\tau = 0{,}59$. Auf den Testdaten erreicht das System damit 0,969, und 44,9 Prozent der Anfragen gehen an die Cloud. Unter der Annahme, dass eine Antwort vom Gerät 5 Millisekunden und eine Cloud-Anfrage 120 Millisekunden dauert (angenommene Werte, nicht gemessen), sinkt die mittlere Latenz von 120 auf 59 Millisekunden. Man tauscht gut einen Prozentpunkt Genauigkeit gegen den Wegfall von 55 Prozent der Cloud-Aufrufe.

Trade-offs und Grenzen. Man betreibt und versioniert zwei Modelle. Das Modell auf dem Gerät lässt sich nur mit einem App-Update austauschen, das Cloud-Modell jederzeit; die beiden können also auseinanderlaufen. Dieselbe Eingabe kann je nach Netz unterschiedliche Antworten bekommen, weil sie einmal lokal und einmal in der Cloud beantwortet wird. Ausgerechnet die unsicheren Fälle verlassen das Gerät, und das sind oft die interessanten, vielleicht auch die datenschutzrelevanten. Der Schwellwert hängt von der Verteilung der Eingaben ab und muss mit dieser mitwandern (Pattern 18). Die Latenz addiert sich bei weitergereichten Anfragen: erst das Gerät, dann das Netz. Zur Abgrenzung: Cascade (Pattern 8) zerlegt ein fachliches Problem in nacheinander geschaltete Teilprobleme, Two-Phase verteilt dieselbe Aufgabe auf zwei Orte. **Wann nicht:** Wenn die Netzlatenz keine Rolle spielt und die Cloud-Kosten gering sind, genügt das Cloud-Modell allein; und wenn das Gerät nie Netz hat, bleibt nur das kleine Modell.

Bezug zu Software Engineering. Das Muster ist ein Fast Path / Slow Path, wie man ihn von Caches kennt: Der schnelle Pfad bedient den Normalfall, der langsame den Rest. Es ist verwandt mit mehrstufigen Caches (L1/L2), und die Rückfallebene bei fehlendem Netz entspricht Graceful Degradation.

> **SO ERKLÄRE ICH ES**
> „Das ist wie der Empfang eines Hotels: Die meisten Fragen beantwortet er selbst, bei schwierigen holt er den Experten im Hinterzimmer. Wir müssen nur entscheiden, ab wann der Empfang ‚ich bin mir nicht sicher' sagt, und diesen Schwellwert wählen wir mit Daten, die nicht zum Testen gedacht sind."
>
> **Verständnisfrage:** Warum darf der Schwellwert τ nicht auf den Testdaten gewählt werden?
> **Antwort:** Dann wäre die berichtete Genauigkeit auf den Testdaten zu optimistisch, weil τ gerade für sie optimiert wurde. Man wählt auf Validierungsdaten und prüft einmal auf den Testdaten.

Quellen. [1] (Kapitel 5, Pattern 19); [22].

8.5 Pattern 20: Keyed Predictions

Szenario und Problem. Der Lieferdienst bewertet nachts nicht mehr 200 000, sondern 20 Millionen Bestellungen und verteilt die Arbeit auf viele Rechner. Jeder nimmt ein Stück der Eingabe, bewertet es und liefert seine Vorhersagen zurück. Das Problem liegt in der Reihenfolge: In einem verteilten Job kommt das Ergebnis eines Stücks an, wenn es fertig ist, nicht wenn es an der Reihe wäre. Ein langsamer Knoten, ein wiederholter Versuch oder ungleiche Stückgrößen genügen, und die Ergebnisse treffen in

anderer Reihenfolge ein, als die Eingaben abgeschickt wurden. Verlässt man sich darauf, dass die n-te Antwort zur n-ten Frage gehört, ordnet man Vorhersagen stillschweigend falschen Bestellungen zu: keine Fehlermeldung, keine Warnung, nur falsche Ergebnisse.

Das Skript simuliert das mit zwölf Stücken zu je zehn Ziffernbildern. Die Bearbeitungsdauer je Stück ist fest vorgegeben und steht für schwankende Last im Cluster (sie ist keine Messung); die Reihenfolge des Eintreffens weicht dadurch von der Eingangsreihenfolge ab, ist aber reproduzierbar.

Die Lösung. Jeder Datensatz bekommt einen eindeutigen **Schlüssel**, der unverändert durch das Modell wandert: Eingang ist (Schlüssel, Features), Ausgang ist (Schlüssel, Vorhersage). Danach spielt die Reihenfolge keine Rolle mehr, die Zuordnung geschieht per Join. Das Modell kennt den Schlüssel nicht, scikit-learn-Modelle erwarten nur Features. Der Schlüssel läuft deshalb neben dem Modell her, wird vor `predict` abgetrennt und danach wieder angefügt; im Skript steckt er als Spalte `schluessel` im DataFrame.

```python
# a) ohne Schlüssel: Ergebnisse in Ankunftsreihenfolge an die Eingabe heften
ohne = np.concatenate([p for _, _, p in eingang])
print(f"\na) positionsbasiert zugeordnet:  Genauigkeit = "
      f"{np.mean(ohne == y_neu):.3f}")

# b) mit Schlüssel: Join über die mitgereichte ID
schluessel = sum((k for _, k, _ in eingang), [])
erg = pd.DataFrame({"schluessel": schluessel, "vorhersage": ohne})
wahrheit = pd.DataFrame({"schluessel": df["schluessel"], "wahr": y_neu})
verbunden = wahrheit.merge(erg, on="schluessel")
richtig = np.mean(verbunden.wahr == verbunden.vorhersage)
print(f"b) per Schlüssel zugeordnet:     Genauigkeit = {richtig:.3f}")
print(verbunden.head(4).to_string(index=False))
```

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy \
    python keyed_predictions.py
Stücke in Reihenfolge des Eintreffens: [3, 8, 1, 10, 5, 7, 0, 4, 9, 2, 11, 6]

a) positionsbasiert zugeordnet:  Genauigkeit = 0.117
b) per Schlüssel zugeordnet:     Genauigkeit = 0.883
schluessel  wahr  vorhersage
   img-000     1           1
   img-001     7           7
   img-002     4           4
   img-003     6           6
```

Die Stücke treffen in der Reihenfolge 3, 8, 1, 10, 5, 7, 0, 4, 9, 2, 11, 6 ein. Werden die Vorhersagen in der Ankunftsreihenfolge an die Eingaben geheftet, ist die Genauigkeit 0,117, also Raten bei zehn Klassen, und das Programm meldet keinen Fehler. Mit dem Schlüssel sind es 0,883, die tatsächliche Genauigkeit des Modells auf diesen 120 Bildern.

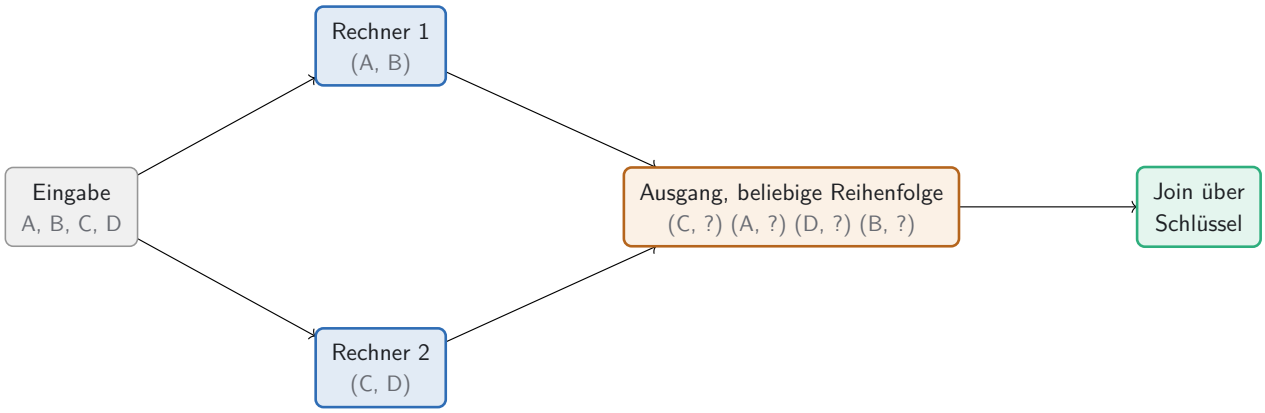


Abbildung 64: Ohne Schlüssel (Reihenfolge) wäre der Ausgang nicht zuzuordnen. Mit Schlüssel stellt der Join die Zuordnung wieder her.

Gute Schlüssel sind eindeutig, stabil und bedeutungslos nach außen: eine laufende Nummer oder eine UUID, die der Client oder der Anfang der Pipeline vergibt. Einen Schlüssel aus den Features abzuleiten (etwa einen Hash) funktioniert nur, solange keine zwei Datensätze dieselben Features haben, und genau das passiert bei kleinen Feature-Räumen ständig. Personenbezogene Angaben wie Namen oder E-Mail-Adressen gehören nicht in den Schlüssel, weil er in Logs und Zwischenspeichern auftaucht.

Der Schlüssel hilft über das Zuordnen hinaus. Mit ihm lassen sich Vorhersagen später mit dem tatsächlichen Ergebnis zusammenführen, was die laufende Bewertung braucht (Pattern 18), und einzelne Vorhersagen lassen sich im Nachhinein nachvollziehen (Pattern 29).

Trade-offs und Grenzen. Der Schlüssel kostet etwas zusätzlichen Datenverkehr. Er muss in jedem Schritt des Datenpfads erhalten bleiben, denn jede Transformation, die Zeilen zusammenfasst oder aufteilt, kann ihn verlieren. Wird ein Stück nach einem Fehler wiederholt, kommt es unter Umständen doppelt an. Der Empfänger sollte Duplikate über den Schlüssel erkennen und verwerfen. Bei Online-Diensten mit nur einer Anfrage pro Antwort (Pattern 16) ist der Schlüssel meist unnötig, weil Anfrage und Antwort über die Verbindung zusammengehören; bei Batch-Anfragen mit mehreren Zeilen ist er dagegen Pflicht. **Wann nicht:** In einem einzelnen Prozess, der eine Tabelle in einem Zug bewertet, bleibt die Reihenfolge erhalten. Man sollte sich aber nicht auf diese Eigenschaft verlassen, wenn der Job später verteilt werden könnte.

Bezug zu Software Engineering. Das Muster ist die Korrelations-ID aus der Welt der Nachrichtensysteme: Antworten auf eine Anfrage tragen deren Kennung, damit der Absender sie zuordnen kann [56]. Dieselbe Idee steckt in der Request-ID, die in verteilten Systemen durch alle Dienste gereicht und in Logs ausgegeben wird, und im Primärschlüssel einer Datenbanktabelle. Wer verteilte Verarbeitung kennt, hat die Regel parat: Eine Reihenfolge besteht in verteilten Systemen nur, wenn man sie ausdrücklich herstellt; Kleppmann behandelt das ausführlich [54].

> **SO ERKLÄRE ICH ES**
> „Wenn zwölf Leute je ein Stück eines Stapels Briefe bearbeiten und die Ergebnisse in der Reihenfolge einsammeln, in der sie fertig werden, stimmt die Reihenfolge nicht mehr. Auf jedem Brief steht deshalb eine Nummer, und auf die Nummer kommt es an, nicht auf den Platz im Stapel.“ Das Experiment aus dem Skript eignet sich gut für die Vorlesung.
>
> **Verständnisfrage:** Reicht es, die Ergebnisse nach dem Einsammeln einfach nach dem Eingang zu sortieren?
> **Antwort:** Nur, wenn man weiß, welches Ergebnis zu welcher Eingabe gehört, und dafür braucht man genau den Schlüssel. Ohne ihn ist Sortieren Raten.

Quellen. [1] (Kapitel 5, Pattern 20); [56]; [54].

9 Reproducibility

Ein Modell, das sich nicht reproduzieren lässt, lässt sich auch nicht reparieren, vergleichen oder prüfen. Dieses Kapitel behandelt die sieben Patterns, die ein ML-System in diesem Sinn zu einem kontrollierbaren Software-Artefakt machen. Alle Beispielprogramme liegen im Ordner `code/08-reproducibility/` und werden mit `uv run --with scikit-learn --with pandas --with numpy python <datei>.py` gestartet. Die gezeigten Ausgaben sind die tatsächlichen Ausgaben dieser Läufe; die Daten sind simuliert (feste Seeds), damit die Effekte reproduzierbar sind und die richtige Antwort bekannt ist.

9.1 Warum ML-Systeme schwer reproduzierbar sind

Bei klassischer Software ist die Frage „Welche Version läuft in Produktion?“ mit einem Commit-Hash beantwortet: Gleicher Code und gleiche Eingabe ergeben gleiche Ausgabe. Ein ML-System hat zwei Phasen, und das Verhalten des fertigen Modells ist das Ergebnis eines *Trainingslaufs*, in den mehr eingeht als Code. Der Commit-Hash allein reicht nicht.

Es lohnt sich, drei Quellen von Nichtreproduzierbarkeit zu trennen. Die erste ist der **Zufall**: Gewichte werden zufällig initialisiert, Trainingsdaten werden zufällig gemischt, Random Forests ziehen zufällige Stichproben, und parallele Berechnungen (insbesondere auf GPUs) summieren Gleitkommazahlen nicht immer in derselben Reihenfolge. Fixiert man den *Seed* (den Startwert des Zufallsgenerators; in scikit-learn der Parameter `random_state`), wird das meiste wiederholbar, aber nicht alles. Die zweite Quelle sind die **Daten**: Eine Tabelle, die heute 3 Millionen Zeilen hat, hat morgen 3,1 Millionen; Spalten werden umbenannt, ein Zulieferer ändert das Format (Pattern 23), und ein Split in Trainings- und Testdaten, der bei jedem Lauf neu gewürfelt wird, macht Ergebnisse unvergleichbar (Pattern 22). Die dritte ist die **Umgebung**: Bibliotheksversionen, Betriebssystem, Hardware und, besonders tückisch, der Code, der die Eingaben im Betrieb aufbereitet und nicht derselbe ist wie beim Training (Pattern 21).

D. Sculley *u. a.* [11] beschreiben, warum das mehr ist als ein Komfortproblem. ML-Systeme erzeugen auf allen Ebenen *technische Schulden*: Datenabhängigkeiten, die schwerer zu erkennen sind als Code-Abhängigkeiten, Glue Code zwischen Bibliotheken und ganze „Pipeline Jungles“ aus zusammengeklebten Vorverarbeitungsschritten. Ihr Prinzip „Changing Anything Changes Everything“ (CACE) beschreibt, dass sich in einem trainierten Modell keine Eingabe isoliert verändern lässt: Wer ein Feature ändert, verändert die Gewichte aller anderen. Reproduzierbarkeit heißt deshalb, alles festzuhalten und explizit zu machen, was in ein Modell eingeht. Auch der ML Test Score von E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55] enthält Tests für Produktionsreife, die nach genau diesen Dingen fragen; einer davon prüft, ob Features in Training und Betrieb dieselben Werte berechnen.

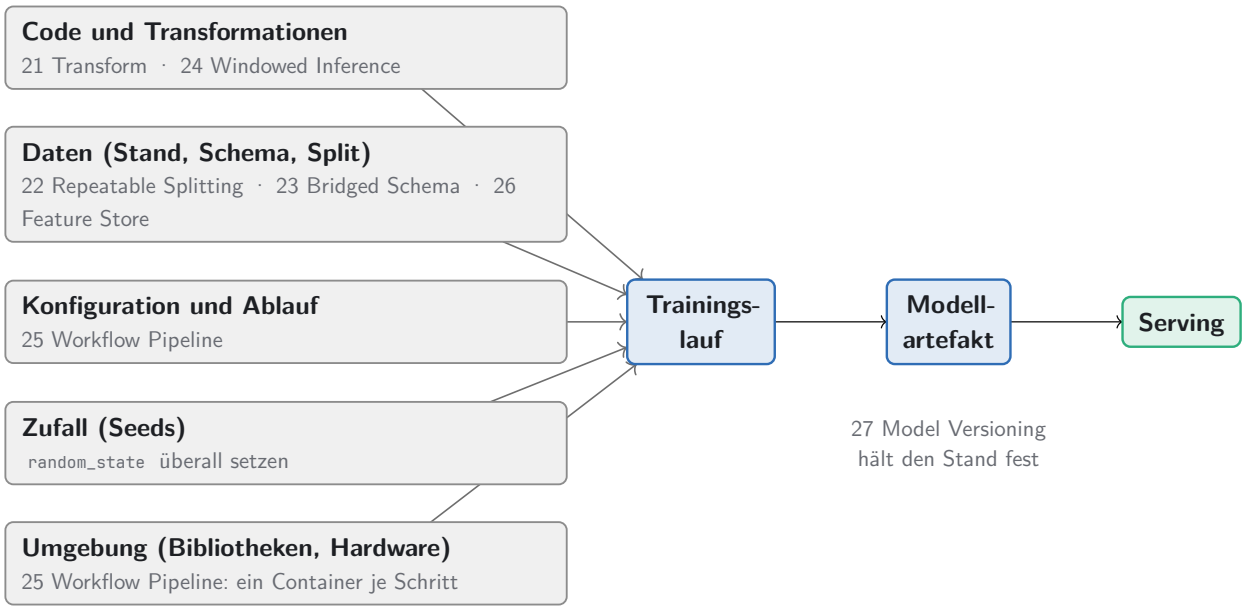


Abbildung 65: Was in ein trainiertes Modell eingeht und welches Pattern welche Quelle kontrolliert. Das Modellartefakt ist erst dann reproduzierbar, wenn alle fünf Eingänge festgehalten sind.

Die Patterns dieses Kapitels lassen sich entlang dieses Bildes lesen. *Transform*, *Windowed Inference* und *Feature Store* sorgen dafür, dass Training und Betrieb dieselben Features berechnen. *Repeatable Splitting* und *Bridged Schema* halten den Datenstand und seine Veränderung unter Kontrolle. *Workflow Pipeline* macht den gesamten Ablauf ausführbar und wiederholbar. *Model Versioning* hält fest, welches Ergebnis welches Laufs gerade im Betrieb ist.

9.2 Pattern 21: Transform

Szenario. Ein Fahrdienst sagt den Preis einer Fahrt voraus. Eingaben sind die Distanz in Kilometern und die Stadt (Wien, Graz oder Linz). Ein Data Scientist bereitet die Daten im Notebook auf: Die Distanz wird *standardisiert*, das heißt, vom Mittelwert abgezogen und durch die Standardabweichung geteilt, damit alle numerischen Features ähnliche Größenordnung haben; die Stadt wird per One-Hot-Encoding in drei Spalten mit 0/1 zerlegt. Das Modell ist gut. Zwei Wochen später baut ein anderes Team den Service und schreibt die Aufbereitung noch einmal, in seinen eigenen Worten.

Das Problem. Die Transformation hat *gelernte Parameter*: Mittelwert und Standardabweichung stammen aus den Trainingsdaten, die Liste der bekannten Städte ebenso. Das sind Teile des Modells, auch wenn sie nicht in den Gewichten stehen. Wer die Aufbereitung im Service neu implementiert, muss diese Parameter und die Reihenfolge der Spalten exakt übernehmen. Schon kleine Abweichungen führen zu *Training-Serving Skew*: Das Modell sieht im Betrieb Eingaben, die anders aufbereitet sind als im Training, und liefert ohne Fehlermeldung falsche Vorhersagen.

Das folgende Programm macht den Fehler sichtbar: Der nachgebaute Serving-Code standardisiert mit Mittelwert und Standardabweichung der gerade eingehenden Anfragen statt mit denen des Trainings, ein in Reviews leicht zu übersehender Fehler.

```
def serving_naiv(anfragen: pd.DataFrame) -> np.ndarray:
    X = pd.get_dummies(anfragen, columns=["stadt"], dtype=float)
    X = X.reindex(columns=X_train.columns, fill_value=0.0)
    d = X["distanz_km"]
    # Fehler: Statistik der Anfragen statt der Trainingsdaten
    X["distanz_km"] = (d - d.mean()) / d.std()
    return modell_naiv.predict(X)
```

Fünf Anfragen mit bekannten Preisen zeigen den Effekt:

```
Terminal
$ python 21_transform_skew.py
erwarteter Preis      : [11.1, 10.8, 14.7, 12.5, 19.2]
Serving (naiv)        : [12.85, 21.05, 27.72, 31.19, 46.3]
Serving (Pipeline)    : [11.09, 10.83, 14.68, 12.51, 19.17]
Einzelanfrage         : [11.09]
gelernter Mittelwert  : 12.085
```

Die erste Zeile ist der Preis, den das Modell im Idealfall liefern müsste (die Zufallskomponente der Daten ausgenommen), die zweite die Antwort des Service: für die letzte Fahrt 46,3 statt 19,2. Die Abweichung ist kein Zufall und kein Rauschen, sondern systematisch: Der Service standardisiert die fünf Anfragen relativ zueinander, sodass die kürzeste Fahrt immer „unterdurchschnittlich" und die längste „überdurchschnittlich" erscheint, egal wie lang sie tatsächlich sind. Bei einer einzelnen Anfrage wäre die Standardabweichung nicht definiert (pandas liefert `NaN`). Kein Syntaxtest findet das, die Zahlen sehen plausibel aus.

Die Lösung. Das Pattern trennt drei Dinge sauber voneinander: die *Eingaben* (roh, so wie der Client sie schickt), die *Features* (das, was das Modell tatsächlich verarbeitet) und die *Transformation* zwischen beiden. Die Transformation wird dann nicht neben dem Modell gepflegt, sondern ist *Teil des Modells*: Sie wird beim Training gelernt, zusammen mit den Gewichten gespeichert und im Betrieb unverändert angewendet. Es gibt dann nur noch eine Implementierung. Das Buch zeigt dies in TensorFlow (Transformationen als Schichten im Modell, TensorFlow Transform für Vorberechnungen, die TRANSFORM-Klausel in BigQuery ML) und verweist darauf, dass das Prinzip frameworkunabhängig ist [1, Kapitel 6, Pattern 21]. In scikit-learn heißt das Werkzeug `Pipeline` [42]:

```python
# --- Transform-Pattern: Transformation gehört zum Modell ---
vorverarbeitung = ColumnTransformer([
    ("zahl", StandardScaler(), ["distanz_km"]),
    ("kat", OneHotEncoder(handle_unknown="ignore"), ["stadt"]),
])
pipeline = Pipeline([("vor", vorverarbeitung), ("modell", Ridge(alpha=1.0))])
pipeline.fit(df[["distanz_km", "stadt"]], df["preis"])

puffer = io.BytesIO()
joblib.dump(pipeline, puffer)      # ein Artefakt: Transformation + Modell
puffer.seek(0)
serving_modell = joblib.load(puffer)
```

Der `ColumnTransformer` wendet auf jede Spalte ihre eigene Transformation an, die `Pipeline` hängt das Modell dahinter. `fit` lernt Mittelwert, Standardabweichung und Stadtliste aus den Trainingsdaten, `predict` wendet sie unverändert an. `joblib.dump` speichert das Ganze als ein Artefakt. Der Service lädt es und übergibt rohe Anfragen; seine eigene Aufbereitung entfällt. Die dritte Zeile der Ausgabe zeigt, dass die Preise jetzt bis auf wenige Hundertstel zu den erwarteten passen; auch eine Einzelanfrage funktioniert (vierte Zeile). Der gelernte Mittelwert der Distanz (letzte Zeile) steckt jetzt im Artefakt.

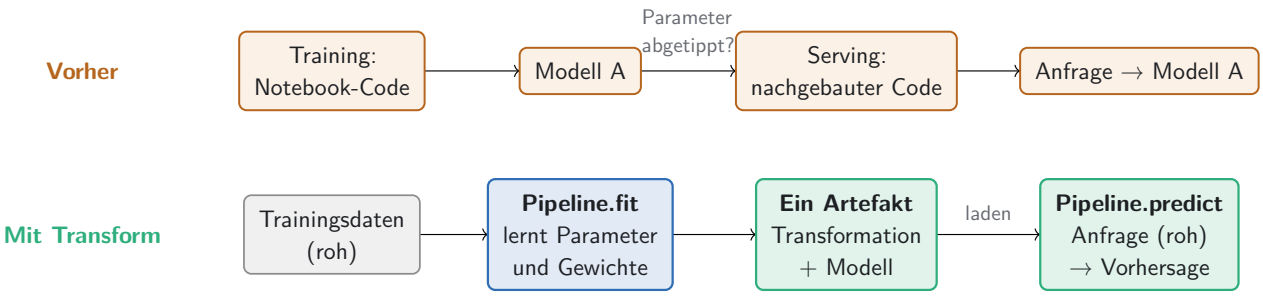


Abbildung 66: Oben der Zustand vor dem Pattern: Training und Serving haben je eigenen Aufbereitungscode, der auseinanderlaufen kann. Unten nach dem Pattern: Die Transformation wird beim Training gelernt und liegt im selben Artefakt wie das Modell.

Die Pipeline schützt auch vor einem Datenleck (vgl. Pattern 22): Ein Scaler, der vor dem Split auf *alle* Daten angepasst wird, kennt die Testdaten schon. In einer Pipeline innerhalb der Kreuzvalidierung wird er nur auf den jeweiligen Trainingsanteil angepasst.

Trade-offs und Grenzen. Die Transformation läuft bei jeder Vorhersage mit; das kostet Latenz, aufwendige Berechnungen wiederholt man besser einmal im Voraus. Das gespeicherte Artefakt ist an Bibliotheksversionen gebunden, und `joblib` / `pickle` können beim Laden beliebigen Code ausführen, dürfen also nie aus unvertrauter Quelle stammen. Das Pattern deckt zudem nur Transformationen ab, die sich aus den Eingaben *einer* Anfrage berechnen lassen; was Verlauf oder fremde Tabellen braucht („Bestellungen der letzten 30 Tage"), gehört in Windowed Inference (Pattern 24) oder einen Feature Store (Pattern 26).

Wann nicht. Bei einmaligen Analysen ohne Serving bringt das Pattern wenig. Bei Baummodellen entfällt die Standardisierung, nicht aber die Kodierung kategorialer Features.

Bezug zu Software Engineering. Das Pattern ist *Don't Repeat Yourself* und *Single Source of Truth* für Wissen: Die Aufbereitungsregel existiert an genau einer Stelle. Der Skew ist verwandt mit unterschiedlichen Test- und Produktionsumgebungen („works on my machine"); der passende Test schickt dieselben Rohdaten durch beide Wege und vergleicht.

> **SO ERKLÄRE ICH ES**
> Ich zeige zuerst die Ausgabe des fehlerhaften Service und lasse raten, was falsch ist: Das Modell ist fehlerfrei, die Zahlen sind falsch. Dann frage ich, wo der Mittelwert „wohnt": Er entsteht beim Training und gehört deshalb zum Modell. Erst danach zeige ich die Pipeline, die das erzwingt.
>
> **Verständnisfrage:** Warum reicht es nicht, Mittelwert und Standardabweichung in eine Konfigurationsdatei zu schreiben, die der Service lädt?
> **Antwort:** Es funktioniert, solange beide Dateien bei jedem Neutraining gemeinsam und konsistent aktualisiert werden. Das ist aber ein manueller Schritt zwischen zwei Artefakten, der vergessen werden kann; die Pipeline macht ihn unmöglich zu vergessen, weil es nur ein Artefakt gibt.

Quellen. [1, Kapitel 6, Pattern 21], [11], [42], [55]

9.3 Pattern 22: Repeatable Splitting

Namenshinweis: Im Inhaltsverzeichnis des Buchs heißt das Pattern „Repeatable Splitting", im Code-Repository zum Buch „Repeatable Sampling".

Szenario. Eine Klinik sammelt Messwerte von 500 Patienten, je 10 Messungen pro Person, also 5000 Zeilen. Ein Modell soll aus den Messwerten einen Befund vorhersagen. Wie jedes ML-Projekt teilt das Team die Daten in Trainings- und Testdaten: Mit dem Training lernt das Modell, mit den Testdaten, die es nie gesehen hat, misst man, wie gut es auf neuen Fällen funktioniert. Üblich ist `train_test_split(df, test_size=0.2)`.

Das Problem. Es gibt zwei getrennte Probleme, die man oft verwechselt. Das erste ist das **Datenleck** (*Data Leakage*) durch korrelierte Zeilen. Die zehn Messungen eines Patienten sind einander sehr ähnlich. Würfelt man jede Zeile einzeln, landen manche in den Trainings- und andere in den Testdaten. Das Modell kann dann Patienten *wiedererkennen* statt etwas zu verallgemeinern, und die Testgenauigkeit ist geschönt. Das zweite Problem ist die **fehlende Wiederholbarkeit**. Ein Zufallssplit mit festem Seed ist nur so lange stabil, wie die Daten unverändert bleiben; kommen neue Zeilen hinzu oder ändert sich die Sortierung, wird alles neu gemischt, und Zeilen, die gestern in den Testdaten waren, sind heute im Training. Dann sind Messungen von gestern und heute nicht mehr vergleichbar, und die Testdaten sind „verbraucht".

```
# 2) Hash-Split auf dem Gruppenschlüssel
def bucket(key: str, n: int = 10) -> int:
    digest = hashlib.sha256(key.encode("utf-8")).digest()
    return int.from_bytes(digest[:8], "big") % n

df["bucket"] = df["patient_id"].map(bucket)
```

```
tr = df[df["bucket"] < 8]
te = df[df["bucket"] >= 8]
print(f"Hash-Split         : Accuracy {bewerte(tr, te):.3f}"
      f" | Patienten in beiden Teilen: {beide(tr, te)}")
print(f"Zeilen Train/Test  : {len(tr)}/{len(te)}")

# 3) Alternative in scikit-learn
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=0)
i_tr, i_te = next(gss.split(df, groups=df["patient_id"]))
print(f"GroupShuffleSplit  : Accuracy "
      f"{bewerte(df.iloc[i_tr], df.iloc[i_te]):.3f}")
```

Terminal

```
$ python 22_repeatable_splitting.py
zeilenweiser Split : Accuracy 0.748 | Patienten in beiden Teilen: 448
Hash-Split         : Accuracy 0.572 | Patienten in beiden Teilen: 0
Zeilen Train/Test  : 4010/990
GroupShuffleSplit  : Accuracy 0.566
Bisherige 500 Patienten nach Datenzuwachs unverändert: True
hash('P0001') in zwei Prozessen gleich: False
```

Beim zeilenweisen Split stehen 448 der 500 Patienten auf beiden Seiten, und die Genauigkeit beträgt 0,748. Das ist trügerisch: Der Random Forest hat sich Patienten gemerkt. Teilt man nach Patienten (Hash-Split: kein Patient auf beiden Seiten, 4010 Zeilen im Training und 990 im Test), fällt die Genauigkeit auf 0,572, und `GroupShuffleSplit` liefert mit 0,566 praktisch dasselbe. Das ist die ehrliche Zahl: In diesem simulierten Beispiel hängt der Befund nur schwach von den Messwerten ab. Die rund 0,18 Unterschied (0,748 gegen 0,572 beziehungsweise 0,566) sind ein reines Datenleck. D. R. Roberts *u. a.* [57] zeigen für Daten mit zeitlicher, räumlicher, hierarchischer oder phylogenetischer Struktur, dass zufälliges Aufteilen den Vorhersagefehler tendenziell unterschätzt; S. Kaufman, S. Rosset, C. Perlich, und O. Stitelman [8] systematisieren Datenlecks allgemein.

Die Lösung. Das Pattern teilt nach einem **Gruppierungsschlüssel** und bestimmt die Zuordnung mit einer *Hashfunktion*. Eine Hashfunktion bildet beliebige Eingaben deterministisch auf eine Zahl ab; bei kryptografischen Hashfunktionen wie SHA-256 landen auch ähnliche Eingaben auf völlig verschiedenen Zahlen. Der Ablauf ist in drei Schritten festgelegt:

1. Den Schlüssel wählen, der korrelierte Zeilen zusammenhält: die Patienten-ID, die Kunden-ID, das Datum. Alle Zeilen mit demselben Schlüssel müssen auf dieselbe Seite fallen.
2. Den Schlüssel hashen und den Rest einer Division bilden (*Modulo*): `hash(schluessel) mod 10` ergibt einen *Bucket* von 0 bis 9.
3. Buckets den Teilen zuordnen, zum Beispiel 0–7 Training, 8 Validierung, 9 Test.

Das Buch zeigt dies mit der Funktion `FARM_FINGERPRINT` in BigQuery [1, Kapitel 6, Pattern 22]; in Python genügt `hashlib`. Die Zuordnung hängt allein vom Schlüssel ab, nicht von der Reihenfolge, der Größe des Datensatzes oder dem Zeitpunkt des Laufs. Kommen 100 neue Patienten hinzu, behalten die bisherigen 500 ihren Bucket (die Ausgabe prüft das); die Testmenge wächst, ohne dass alte Testfälle ins Training wandern.

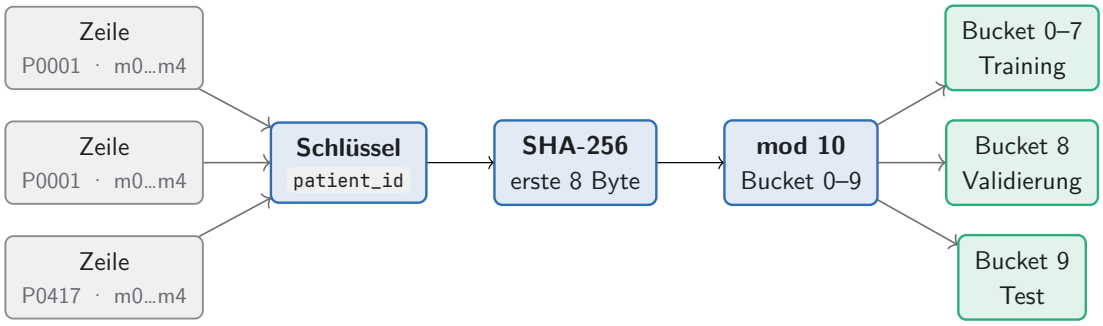


Abbildung 67: Hash-basierter Split. Beide Zeilen von Patient P0001 liefern denselben Schlüssel und damit denselben Bucket, egal in welcher Reihenfolge oder zu welchem Zeitpunkt sie eintreffen.

Ein Stolperstein steht in der letzten Zeile der Ausgabe: Pythons eingebautes `hash()` taugt dafür *nicht*. Für Strings ist es pro Prozess zufällig gesalzen (Schutz gegen Hash-Kollisionsangriffe); zwei Läufe liefern verschiedene Werte, und der Split wäre nicht wiederholbar. Man braucht eine Funktion mit stabiler Ausgabe wie die aus `hashlib`.

Alternative in scikit-learn. `GroupShuffleSplit` (und `GroupKFold` für Kreuzvalidierung) gruppieren ebenfalls und sind für ein einmaliges Experiment einfacher. Sie sind aber nur bei festem Seed *und* unverändertem Datensatz reproduzierbar. Der Hash-Split ist für Systeme gedacht, deren Daten fortlaufend wachsen und deren Testdaten über Monate vergleichbar bleiben sollen.

Trade-offs und Grenzen. Die Anteile stimmen nur ungefähr: Bei vielen Schlüsselwerten liegt der Split nahe 80/20, bei wenigen (etwa 5 verschiedene Tage) kann er stark abweichen, und ein einzelner sehr großer Kunde kann das Verhältnis verzerren. Der Hash garantiert keine ausgewogene Klassenverteilung; bei seltenen Klassen muss man das nachprüfen. Der Schlüssel darf kein Feature des Modells sein, denn sonst sieht das Modell im Training nur Schlüsselwerte, die im Test nie vorkommen. **Wann nicht:** Bei Zeitreihen und überall, wo das Modell in die Zukunft vorhersagen soll, muss der Split zeitlich sein (Training = Vergangenheit, Test = Zukunft); ein Hash-Split mischt Vergangenheit und Zukunft und überschätzt die Qualität.

Bezug zu Software Engineering. Das Verfahren ist dasselbe wie beim *Sharding* von Datenbanken: `hash(key) mod N` entscheidet über die Partition. (Konsistentes Hashing ist die Verfeinerung, die bei einem geänderten N möglichst wenige Schlüssel umzieht; hier bleibt N fest.) Der Gedanke dahinter ist *Idempotenz* und *Determinismus*: dieselbe Eingabe, dieselbe Zuordnung, unabhängig vom Zustand der Umgebung. Der Gruppierungsschlüssel entspricht der Frage, was in einem Test als „unabhängige Einheit" gilt.

> **SO ERKLÄRE ICH ES**
> Ich lasse raten, warum ein Modell mit guter Testgenauigkeit im Betrieb deutlich schlechter sein kann, und zeige dann die zwei Zahlen der Ausgabe nebeneinander. Dann der Vergleich mit einer Klausur, deren Fragen aus denselben Übungsbeispielen stammen: Sie sagt nichts darüber, ob jemand das Thema verstanden hat. Testdaten müssen *unabhängig* sein.
>
> **Verständnisfrage:** Warum löst `random_state=42` das Problem nicht?
> **Antwort:** Ein fester Seed macht den Zufall wiederholbar, aber nur für genau diese Daten in genau dieser Reihenfolge; sobald Zeilen hinzukommen, ändert sich die Zuordnung. Außerdem trennt ein Seed nichts nach Gruppen: Zeilen desselben Patienten können weiterhin auf beiden Seiten stehen.

Quellen. [1, Kapitel 6, Pattern 22], [8], [57]

9.4 Pattern 23: Bridged Schema

Szenario. Ein Fahrdienst will vorhersagen, wie viel Trinkgeld eine Fahrt bekommt. Ein wichtiges Feature ist die Zahlungsart. Über Jahre hinweg gab es in den Daten nur `cash` und `card`. Seit dem Umstieg auf ein neues Zahlungssystem unterscheidet die Erfassung bei Karten zwischen `gift_card`, `debit_card` und `credit_card`, und das Modell soll im Betrieb mit genau diesen vier Werten arbeiten. Das Beispiel orientiert sich am Zahlungsart-Beispiel des Buchs [1, Kapitel 6, Pattern 23]; Zahlen und Zielgröße (Trinkgeld) sind hier frei gewählt und simuliert.

Das Problem. Im neuen Schema gibt es erst 60 Zeilen, im alten 3000, und das Modell hat neben der Zahlungsart sechs weitere Fahrt-Features, deren Zusammenhang es aus den Daten lernen muss. Das neue Schema ist feiner, das alte ist eine *Vergröberung* davon. Drei naive Wege führen nicht zum Ziel. Nur die neuen Daten zu verwenden, verschenkt den größten Teil der Beispiele. Das neue Schema auf `cash`/`card` zurückzuführen (*Roll-up*) lässt das Modell die Unterschiede zwischen den Kartenarten nie lernen, obwohl sie im Betrieb vorkommen. Und eine fünfte Kategorie „card" zuzulassen (*Union Schema*) lehrt das Modell einen Wert, der im Betrieb nie auftritt. Das ist keine gute Lösung.

Die Lösung. Das Pattern *überbrückt* (bridges) das alte Schema ins neue, indem es die fehlende Information für die alten Zeilen *schätzt* und beide Datenstände zusammen trainiert. Es gibt zwei Methoden, die beide die Häufigkeiten der Kartenarten in den neuen Daten verwenden:

- Die **statische Methode** kodiert die alten Zeilen mit einem Vektor aus Häufigkeiten: Eine alte `card`-Zeile erhält bei `cash` eine 0 und bei den drei Kartenarten die Anteile aus den neuen Daten (etwa 0,1; 0,3; 0,6) statt der üblichen 0/1.
- Die **probabilistische Methode** zieht für jede alte `card`-Zeile zufällig eine konkrete Kartenart, mit den Anteilen aus den neuen Daten als Wahrscheinlichkeiten. Die Zeile erscheint dann mit einem echten One-Hot-Wert.

Beide setzen voraus, dass die Anteile der Kartenarten in alten und neuen Daten ungefähr gleich sind; das ist eine Annahme, keine Garantie. Die probabilistische Methode fügt dem alten Teil Rauschen hinzu (jede Zeile bekommt einen Wert, der falsch sein kann). Die statische vermeidet das, benutzt aber Eingaben, die das Modell im Betrieb nie bekommt.

```python
def bruecke_statisch(alt: pd.DataFrame, anteile: pd.Series) -> pd.DataFrame:
    """Vektor mit den Häufigkeiten der neuen Daten statt 0/1."""
    oh = onehot(alt["zahlart"])
    karte = (alt["zahlart"] == "card").to_numpy()
    for k in KARTEN:
        oh.loc[karte, k] = anteile[k]
    return oh

def bruecke_probabilistisch(alt: pd.DataFrame, anteile: pd.Series, rng):
    """Pro Zeile eine Kartenart ziehen, Wahrscheinlichkeit = Häufigkeit."""
    zahlart = alt["zahlart"].to_numpy().astype(object)
    karte = zahlart == "card"
    zahlart[karte] = rng.choice(KARTEN, karte.sum(), p=anteile[KARTEN].values)
    return onehot(zahlart)
```

Terminal

```
$ python 23_bridged_schema.py
MAE (Prozentpunkte Trinkgeld), Test im neuen Schema, 30 Wiederholungen
                              Mittel   Std
nur neue Daten                  2.65  0.14
Roll-up auf cash/card           3.76  0.06
Bridged, statisch               2.52  0.13
Bridged, probabilistisch        3.70  0.06
Referenz (Detail nie verloren)  2.40  0.04
```

Die Zahlen sind der mittlere absolute Fehler (*MAE*, in Prozentpunkten Trinkgeld) auf 2000 Testzeilen im neuen Schema, gemittelt über 30 Wiederholungen mit verschiedenen Zufallsdaten (die Streuung steht daneben). Das Ergebnis ist differenzierter, als man es erwartet. Die **statische** Methode (2,52) ist im Mittel besser als der Verzicht auf die alten Daten (2,65) und kommt der Referenz nahe, bei der das Kartendetail nie verloren ging (2,40). Der Gewinn dürfte hauptsächlich daher kommen, dass die 3000 alten Zeilen helfen, die Wirkung der sechs anderen Features zu schätzen (das ist eine Vermutung, die ich hier nicht getrennt gemessen habe); über die *Unterschiede zwischen den Kartenarten* erfahren die alten Zeilen nichts, das bleibt Aufgabe der 60 neuen. Die **probabilistische** Methode (3,70) ist hier kaum besser als der Roll-up (3,76) und deutlich schlechter als der Verzicht auf die alten Daten. Eine naheliegende Erklärung: Die zufällig gezogene Kartenart ist für die einzelne alte Zeile meist falsch, und das wirkt wie Rauschen in der Eingabe, das die gelernten Gewichte der Kartenarten aufeinander zu zieht (die Ursache habe ich nicht eigens nachgemessen). Bei anderen Modellen und Daten kann das Ergebnis anders ausfallen. Das Beispiel zeigt vor allem, dass man beide Methoden nicht blind wählen darf, sondern gegen die Alternative „alte Daten weglassen“ testen muss.

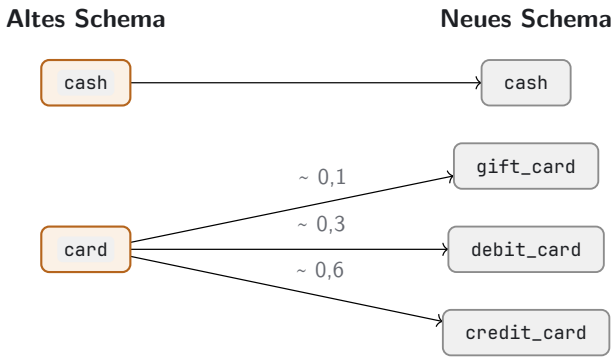


Abbildung 68: Bridged Schema. Eine alte `card`-Zeile wird auf die drei neuen Kartenarten verteilt, gewichtet mit den Häufigkeiten aus den neuen Daten (hier symbolisch 0,1/0,3/0,6). *Statisch*: ein Vektor mit diesen Anteilen. *Probabilistisch*: eine Kartenart wird gezogen.

Trade-offs und Grenzen. Beide Methoden brauchen genug neue Daten, um die Anteile zuverlässig zu schätzen, und die Annahme gleicher Verteilungen kann falsch sein, etwa wenn Kunden mit Gutscheinkarten erst durch eine Kampagne hinzugekommen sind. Je größer der Anteil geschätzter Information, desto vorsichtiger muss man bei der Interpretation sein. **Wann nicht:** Wenn die neuen Daten für sich schon reichen, oder wenn sich das Verhalten selbst (nicht nur die Erfassung) geändert hat, ist eine Brücke falsch; dann zählen nur die neuen Daten.

Bezug zu Software Engineering. Das ist *Schema Evolution* und *Rückwärtskompatibilität*, wie man sie aus Datenbankmigrationen und API-Versionierung kennt, nur dass man hier kein Feld „migrieren" kann, weil die Information nie erhoben wurde. Das Gegenstück in der Datenbank ist eine Migration mit Standardwert; die probabilistische Methode ist ein Standardwert mit Verteilung. Das Muster ähnelt dem *Adapter*: Der alte Datenstand wird an die neue Schnittstelle angepasst.

> **SO ERKLÄRE ICH ES**
> Ich beginne mit einer Datenbankanalogie: Eine Spalte wird in drei neue zerlegt, für alte Zeilen gibt es keine Werte. Standardwert, NULL oder weglassen sind die bekannten Antworten; das Pattern ist die statistische. Dann zeige ich die Vergleichstabelle der Ausgabe.
>
> **Verständnisfrage:** Warum ist es schlechter, die alte Kategorie `card` einfach als fünfte Kategorie zu behalten?
> **Antwort:** Im Betrieb kommt `card` nie vor, es gibt nur die drei feinen Kartenarten. Das Modell lernt dann ein Gewicht für einen Wert, den es nie wieder sieht, und die alten Zeilen helfen nicht beim Verstehen der drei Kartenarten.

Quellen. [1, Kapitel 6, Pattern 23]

9.5 Pattern 24: Windowed Inference

Szenario. An einer Werkzeugmaschine misst ein Sensor jede Minute die Temperatur. Ein Modell soll den nächsten Wert vorhersagen und Abweichungen melden. Es braucht dafür nicht nur den aktuellen Wert, sondern den *Mittelwert der letzten fünf Messungen* als Feature.

Das Problem. Im Training ist das einfach: Man berechnet `rolling(5).mean()` über eine Spalte in pandas. Im Betrieb kommt jeder Messwert einzeln als Anfrage. Ein Serving-Modell ist in der Regel zustandslos (Pattern 16): Es kennt nur die Anfrage, nicht die vier davor. Jemand muss also die letzten Werte festhalten, bis das Fenster voll ist, und für jeden neuen Wert das Feature bilden. Entsteht dieser Code getrennt vom Trainingscode, droht dasselbe Skew wie bei Pattern 21, und schon ein um eins verschobenes Fenster fällt kaum auf. Tückisch ist ein Fehler wie `rolling(5).mean()` ohne vorheriges `shift(1)`: Das Fenster enthält dann den aktuellen Wert, also genau den, der vorhergesagt werden soll. Im Training sieht das Modell einen Teil der Lösung mit, im Betrieb nicht.

Die Lösung. Das Pattern verlegt das Fenster aus dem Modell hinaus in eine *Stream-Verarbeitung*, die den Zustand hält. Die Verarbeitung ordnet die Ereignisse einem *Zeitfenster* zu, berechnet daraus das Feature und ruft erst dann das (zustandslose) Modell auf. Wichtig ist, dass dieselbe Fensterlogik im Training auf

historischen Daten läuft: Das Buch nutzt dafür Apache Beam, dessen Programme sowohl als Batch als auch als Stream ausgeführt werden können [1, Kapitel 6, Pattern 24]. Das zugrundeliegende Modell der Fensterung stammt aus dem Dataflow Model [58]. Es kennt drei gängige Fensterarten: *Tumbling Windows* (feste, überlappungsfreie Abschnitte, etwa jede volle Stunde; im Dataflow-Paper „fixed windows"), *Sliding Windows* (feste Länge, die sich in kleinen Schritten verschiebt; das Beispiel unten vereinfacht das zu einem Fenster über die letzten fünf Werte statt über eine Zeitspanne) und *Session Windows* (Abschnitte, die durch Pausen begrenzt werden). Für die Zuordnung zählt die *Ereigniszeit* (wann etwas passierte), nicht die Verarbeitungszeit (wann es ankam); verspätete Ereignisse werden über sogenannte Watermarks behandelt.

Das Beispiel kommt ohne Stream-Framework aus: *eine* Funktion definiert das Feature, zwei Treiber wenden sie an.

```python
def fenster_merkmal(werte: np.ndarray) -> float:
    """Die EINE Stelle, an der das Merkmal definiert ist."""
    return float(np.mean(werte))

rng = np.random.default_rng(11)
t = np.arange(1500)
temp = (60 + 5 * np.sin(t / 60) + np.cumsum(rng.normal(0, 0.4, t.size)) * 0.2
        + rng.normal(0, 1.0, t.size))
serie = pd.Series(temp, name="temperatur")

# --- Batch (Training): Fenster = die fünf Messwerte VOR dem Zeitpunkt ---
batch = serie.shift(1).rolling(FENSTER).apply(fenster_merkmal, raw=True)

# --- Stream (Serving): Zustand = Ringpuffer der letzten fünf Werte ---
class FensterZustand:
    def __init__(self, groesse: int = FENSTER) -> None:
        self.puffer: deque[float] = deque(maxlen=groesse)

    def merkmal(self) -> float | None:
        """Merkmal für den NÄCHSTEN Wert, sobald das Fenster voll ist."""
        if len(self.puffer) < self.puffer.maxlen:
            return None
        return fenster_merkmal(np.array(self.puffer))

    def aufnehmen(self, wert: float) -> None:
        self.puffer.append(wert)

zustand = FensterZustand()
stream = []
for wert in serie:
    stream.append(zustand.merkmal())   # erst Merkmal ziehen, dann Wert aufnehmen
    zustand.aufnehmen(wert)
stream = pd.Series(stream, dtype=float)
gleich = np.allclose(batch.dropna(), stream.dropna())
print("Batch und Stream identisch:", bool(gleich),
      "| gültige Werte:", int(batch.notna().sum()))
```

Terminal

```
$ python 24_windowed_inference.py
Batch und Stream identisch: True | gültige Werte: 1495
Fenster 'richtig': MAE offline 0.880 | MAE im Stream 0.880
Fenster 'leck':    MAE offline 0.727 | MAE im Stream 0.883
```

Batch- und Stream-Variante liefern auf allen 1495 gültigen Zeitpunkten dieselben Werte (`np.allclose` ist wahr). Die beiden unteren Zeilen sind die Warnung. Das Modell mit dem richtigen Fenster (`'richtig'`) hat offline wie im Stream einen mittleren absoluten Fehler von 0,880. Wurde es mit dem fehlerhaften Fenster ohne `shift` trainiert (`'leck'`), meldet die Offline-Bewertung 0,727, scheinbar 17 % besser. Im Stream, wo der aktuelle Wert nicht zur Verfügung steht, erreicht dasselbe Modell 0,883. Es ist im Betrieb also kaum schlechter als das richtige, aber die Zahl, auf die das Team seine Entscheidungen stützt, ist falsch.

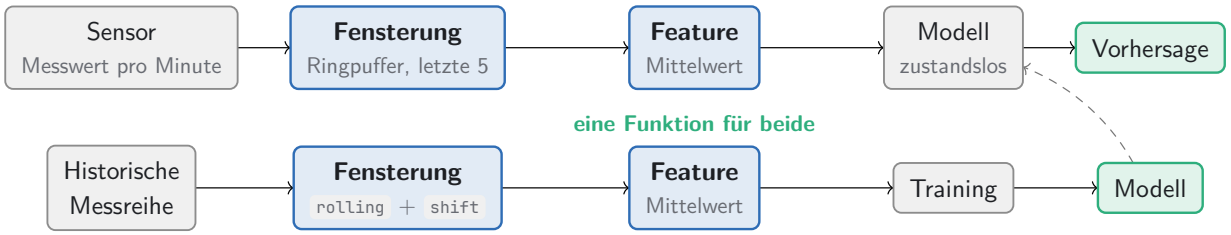


Abbildung 69: Oben der Betrieb (Stream), unten das Training (Batch): beide rufen dieselbe Feature-Funktion auf. Das Modell selbst bleibt zustandslos; der Zustand liegt im Ringpuffer der Fensterung.

Trade-offs und Grenzen. Das Fenster muss erst gefüllt sein, bevor die erste Vorhersage möglich ist (*Cold Start*). Der Zustand im Stream-Prozessor muss einen Absturz überstehen (Checkpoints), sonst fehlen nach einem Neustart Messwerte. Ereignisse kommen verspätet oder in falscher Reihenfolge an; ob man warten oder weiterrechnen soll, ist ein Kompromiss zwischen Latenz und Genauigkeit. Das größte praktische Risiko bleibt der Unterschied zwischen *Ereigniszeit* im Training und *Verarbeitungszeit* im Stream: Rechnet das Training nach Zeitstempeln der Daten und der Stream nach Ankunftszeit, stimmen die Fenster nicht überein. **Wann nicht:** Braucht das Modell kein Gedächtnis, ist das Pattern unnötiger Aufwand. Reichen Aggregate, die einmal täglich neu berechnet werden, genügt Batch Serving (Pattern 17) mit vorberechneten Features.

Bezug zu Software Engineering. Hier trennt man einen *funktionalen Kern* (die reine Funktion `fenster_merkmale`) von einer *imperativen Hülle* (dem Zustand, dem Puffer). Reine Funktionen lassen sich leicht testen, und der Test, der Batch und Stream auf denselben Daten vergleicht (`np.allclose` in der Ausgabe), ist ein klassischer Äquivalenztest. Der Ringpuffer ist eine *Queue fester Größe*, wie sie aus Rate Limitern und Buffern bekannt ist.

> **SO ERKLÄRE ICH ES**
> Ich lasse an der Tafel eine Messreihe aufschreiben und die Studierenden das Fenster-Feature für den sechsten Wert von Hand bilden. Danach frage ich, wo diese fünf Werte im Betrieb herkommen, wenn jede Anfrage nur einen Wert enthält. Daraus entsteht von selbst die Idee eines Zustandsspeichers vor dem Modell.
>
> **Verständnisfrage:** Warum ist `rolling(5).mean()` ohne `shift(1)` ein Fehler, obwohl die Zahlen plausibel aussehen?
> **Antwort:** Das Fenster schließt den aktuellen Wert ein, den das Modell vorhersagen soll. Im Training steht ein Teil der Antwort im Feature, im Betrieb nicht. Die Offline-Bewertung ist dadurch geschönt, und das Team weiß nicht, wie gut das Modell wirklich ist.

Quellen. [1, Kapitel 6, Pattern 24], [58]

9.6 Pattern 25: Workflow Pipeline

Szenario. Ein Team hat sein Churn-Modell (Vorhersage, welche Kundinnen und Kunden kündigen) in einem Notebook entwickelt: Daten laden, bereinigen, Features bauen, trainieren, bewerten, exportieren. Die Zellen müssen in einer bestimmten Reihenfolge laufen. Die Feature-Berechnung dauert drei Stunden, das Training eine. Wer das Notebook erneut ausführt, bekommt andere Zahlen, weil eine Variable aus einer gelöschten Zelle noch im Speicher liegt. Niemand weiß mehr genau, mit welchem Datenstand das Modell in Produktion gebaut wurde.

Das Problem. Ein Notebook ist ein hervorragendes Werkzeug für Exploration, aber ein schlechtes Ausführungsmodell: Zustand versteckt sich im Interpreter, Schritte sind nicht einzeln wiederholbar, Abhängigkeiten stehen nirgends. Fällt Schritt 4 aus, beginnt man bei Schritt 1. Ein Skript, das alles am Stück ausführt, bleibt ein Monolith, den man nicht stückweise wiederholen, parallelisieren oder auf andere Hardware legen kann.

Die Lösung. Das Pattern zerlegt den Ablauf in *Schritte* mit klar benannten Ein- und Ausgaben. Jeder Schritt läuft für sich, bekommt seine Eingaben als *Artefakte* (Dateien, Tabellen, Modelle) und legt seine Ausgaben als Artefakte ab. Die Abhängigkeiten zwischen den Schritten bilden einen *gerichteten azyklischen Graphen* (DAG): Pfeile zeigen, welcher Schritt auf welchen wartet, Zyklen gibt es nicht. Ein

Orchestrator führt den Graphen aus, wiederholt fehlgeschlagene Schritte, startet unabhängige Schritte parallel und merkt sich, was schon gelaufen ist. Im Buch läuft jeder Schritt in einem eigenen *Container* [1, Kapitel 6, Pattern 25]: Ein Container bündelt Code, Bibliotheken und Betriebssystemumgebung, sodass der Schritt überall gleich läuft. Damit ist auch die dritte Quelle aus der Einleitung, die Umgebung, festgeschrieben.

Das Beispielprogramm ist ein Mini-Orchestrator: ein gerichteter azyklischer Graph (DAG) aus vier Funktionen, ausgeführt in topologischer Reihenfolge (`graphlib` aus der Standardbibliothek). Das Wesentliche ist der *Fingerabdruck* jedes Schritts, ein Hash aus seinem Quelltext, seinen Parametern und den Fingerabdrücken seiner Vorgänger. Liegt ein Ergebnis mit diesem Fingerabdruck im Cache, wird der Schritt nicht neu berechnet.

```python
def fingerabdruck(name, p, vorgaenger):
    """Hash aus Quelltext, eigenen Parametern und Fingerabdrücken der Vorgänger."""
    quelle = inspect.getsource(SCHRITTE[name])
    stoff = json.dumps([quelle, p, vorgaenger], sort_keys=True)
    return hashlib.sha256(stoff.encode()).hexdigest()[:12]

def ausfuehren(params, cache: Path):
    schluessel, ergebnis = {}, {}
    for name in TopologicalSorter(ABHAENGIG).static_order():
        vor = sorted(ABHAENGIG[name])
        p = params.get(name, {})          # nur die Parameter dieses Schritts
        schluessel[name] = fingerabdruck(name, p, [schluessel[d] for d in vor])
        datei = cache / f"{name}-{schluessel[name]}.pkl"
        if datei.exists():
            ergebnis[name] = pickle.loads(datei.read_bytes())
            print(f"  {name:11s} aus Cache  ({schluessel[name]})")
        else:
            ergebnis[name] = SCHRITTE[name](p, *[ergebnis[d] for d in vor])
            datei.write_bytes(pickle.dumps(ergebnis[name]))
            print(f"  {name:11s} berechnet  ({schluessel[name]})")
    return ergebnis["evaluieren"]
```

Terminal

```
$ python 25_mini_pipeline.py
Lauf 1
  laden       berechnet  (071599f4c6bb)
  teilen      berechnet  (80fb4a4e4ae7)
  trainieren  berechnet  (bfdb54bfe93e)
  evaluieren  berechnet  (2fe7f1e49d20)
  Ergebnis: {'accuracy': 0.834}
Lauf 2 (unverändert)
  laden       aus Cache  (071599f4c6bb)
  teilen      aus Cache  (80fb4a4e4ae7)
  trainieren  aus Cache  (bfdb54bfe93e)
  evaluieren  aus Cache  (2fe7f1e49d20)
  Ergebnis: {'accuracy': 0.834}
Lauf 3 (n_estimators=300)
  laden       aus Cache  (071599f4c6bb)
  teilen      aus Cache  (80fb4a4e4ae7)
  trainieren  berechnet  (6ef78f8571bd)
  evaluieren  berechnet  (18b0f0bac33d)
  Ergebnis: {'accuracy': 0.828}
```

Im ersten Lauf werden alle vier Schritte berechnet, der Fingerabdruck steht in Klammern. Der zweite Lauf mit unveränderten Parametern kommt komplett aus dem Cache. Im dritten Lauf wird nur `n_estimators` des Trainings von 100 auf 300 geändert: `laden` und `teilen` bleiben im Cache, `trainieren` und `evaluieren` werden neu berechnet, weil sich der Fingerabdruck von `trainieren` ändert und `evaluieren` ihn als Vorgänger in den eigenen aufnimmt. Das kennt man von einem Build-System.

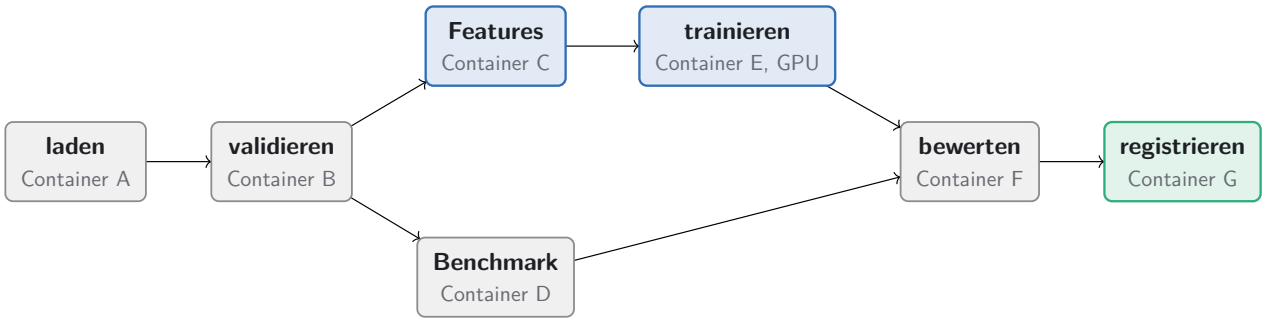


Abbildung 70: Eine Workflow Pipeline als DAG. Jeder Kasten ist ein eigener Container mit festgelegter Umgebung. Unabhängige Zweige (hier Features/Training und Benchmark) können parallel laufen, ein Fehler in `trainieren` wiederholt nur diesen Schritt.

Werkzeuge einordnen. Die Begriffe werden oft vermischt, deshalb eine Abgrenzung.

- **Apache Airflow** ist ein allgemeiner Workflow-Scheduler. DAGs werden in Python beschrieben, Tasks laufen nach Zeitplan oder auf Auslöser, auch in Containern. Airflow kennt weder Modelle noch ML-Artefakte; es ist ein Werkzeug für beliebige Datenpipelines, ETL eingeschlossen.
- **Kubeflow Pipelines** ist ein Orchestrator speziell für ML auf Kubernetes. Jeder Schritt (eine *Komponente*) ist ein Container, Pipelines werden mit einem Python-SDK definiert, Artefakte und Läufe werden mit Metadaten festgehalten. Das Buch nutzt es als Beispiel für die Umsetzung des Patterns [1, Kapitel 6, Pattern 25].
- **TFX** (TensorFlow Extended) ist keine Laufzeit, sondern eine Bibliothek standardisierter Komponenten für ML-Pipelines (unter anderem ExampleGen, StatisticsGen, SchemaGen, Transform, Trainer, Evaluator, Pusher; so heißen sie in der TFX-Dokumentation), die auf einem Orchestrator ausgeführt werden, etwa Airflow, Apache Beam oder Kubeflow Pipelines. Das Konzept der Plattform beschreiben D. Baylor *u. a.* [59].
- Leichtgewichtige Alternativen für kleinere Vorhaben sind Dagster, Prefect oder Metaflow, oder noch einfacher DVC mit einer `dvc.yaml`, die Schritte, Abhängigkeiten und Ausgaben beschreibt und das Zwischenergebnis nach Eingabe-Hash wiederverwendet.

Im Kubeflow-SDK (Version 2) sieht eine Komponente so aus; die Skizze wurde nicht ausgeführt (Ausführen braucht einen Kubernetes-Cluster) und ist unvollständig: `merkmale` wäre eine weitere Komponente nach demselben Muster.

```
from kfp import dsl

@dsl.component(base_image="python:3.12", packages_to_install=["scikit-learn"])
def trainieren(daten: dsl.Input[dsl.Dataset], modell: dsl.Output[dsl.Model],
               n_estimators: int = 100):
    ...  # liest daten.path, schreibt modell.path

@dsl.pipeline(name="churn")
def pipeline(n_estimators: int = 100):
    m = merkmale()
    trainieren(daten=m.outputs["daten"], n_estimators=n_estimators)
```

Trade-offs und Grenzen. Pipelines kosten Aufwand: Schritte brauchen Schnittstellen, Container müssen gebaut und versioniert werden, und die Fehlersuche ist anstrengender als im interaktiven Notebook, weil man nicht „einfach in die Zelle schauen“ kann. Dazu kommt Overhead für Containerstart und das Übergeben von Artefakten. **Wann nicht:** In der Explorationsphase und bei einmaligen Analysen. Der übliche Weg ist, erst im Notebook zu erkunden und die Schritte dann in eine Pipeline zu überführen, sobald der Ablauf feststeht und wiederholt werden soll.

Bezug zu Software Engineering. Das Konzept ist ein *Build-System*: Make, Bazel oder Gradle kennen DAGs, Abhängigkeiten und Caching nach Eingaben. Die Pipeline ist dazu das Äquivalent einer CI-Pipeline für Daten und Modelle. Die Container-Seite entspricht der Idee, Anwendungen als reproduzierbare Images zu bauen; die Fingerabdrücke entsprechen der Content-Addressable Storage, wie sie Git oder Docker-Layer nutzen.

SO ERKLÄRE ICH ES
Ich zeige das 3-Stunden-Notebook und frage, was passiert, wenn Zelle 5 nach einer Änderung an Zelle 2 neu laufen soll. Wer ein Build-System kennt, versteht den DAG sofort: Die Pipeline ist `make` für Daten, mit Containern statt lokaler Compiler.

Verständnisfrage: Warum identifiziert der Cache ein Ergebnis über einen Hash und nicht über den Zeitstempel der Datei?
Antwort: Ein Zeitstempel sagt nur, *wann* etwas entstand, nicht, *woraus*. Der Hash über Code, Parameter und Eingaben ändert sich genau dann, wenn sich das Ergebnis ändern könnte, und bleibt gleich, wenn jemand dieselbe Datei neu schreibt.

Quellen. [1, Kapitel 6, Pattern 25], [59], [60]

9.7 Pattern 26: Feature Store

Szenario. Zwei Teams eines Onlinehändlers trainieren Modelle, die beide das Feature „Bestellungen der letzten 30 Tage" brauchen. Das eine Team zählt gleitend 30 Tage rückwärts, das andere den Kalendermonat, ein drittes rechnet in lokaler Zeit statt UTC. Im Betrieb kann ein Service die Summe über Millionen Bestellungen nicht bei jeder Anfrage neu berechnen; man legt sie also in einer Tabelle ab, die ein Team pflegt, das nicht das Training macht.

Das Problem. Es kommen vier Probleme zusammen. Erstens *Doppelarbeit und Inkonsistenz*: Dasselbe Feature existiert in mehreren Varianten, die niemand abgleicht. Zweitens *Training-Serving Skew*: Offline wird das Feature anders berechnet als online. Drittens *Latenz*: Aggregate über große Datenmengen sind im Online-Pfad zu langsam. Viertens, am wenigsten offensichtlich, die *zeitliche Korrektheit*: Beim Bau von Trainingsdaten braucht man für jedes Beispiel den Wert, den das Feature *zum damaligen Zeitpunkt* hatte, nicht den heutigen.

Die Lösung. Ein **Feature Store** ist ein System, das Features als wiederverwendbare, benannte und versionierte Objekte verwaltet. Es besteht aus mehreren Teilen:

- einer **Registry** mit Definition, Besitzer und Beschreibung jedes Features (im Wesentlichen ein Katalog),
- einem **Offline-Store** mit der *gesamten Historie* der Feature-Werte, optimiert für große Abfragen (Data Warehouse, Parquet-Dateien), aus dem Trainingsdaten gebaut werden,
- einem **Online-Store** mit dem *jeweils letzten* Wert je Entität, optimiert für Schlüsselzugriffe in Millisekunden (Redis, DynamoDB oder Bigtable),
- einer **Materialisierung**, die regelmäßig oder kontinuierlich Werte vom Offline- in den Online-Store schreibt.

Training und Betrieb fragen denselben Katalog; die Definition des Features steht an einer Stelle. Das Buch beschreibt das Pattern entsprechend [1, Kapitel 6, Pattern 26]. Ein bekanntes Open-Source-Beispiel ist Feast [61]; Hopsworks, Tecton und mehrere Cloud-Anbieter haben eigene Produkte; das Buch nennt Feast und Hopsworks.

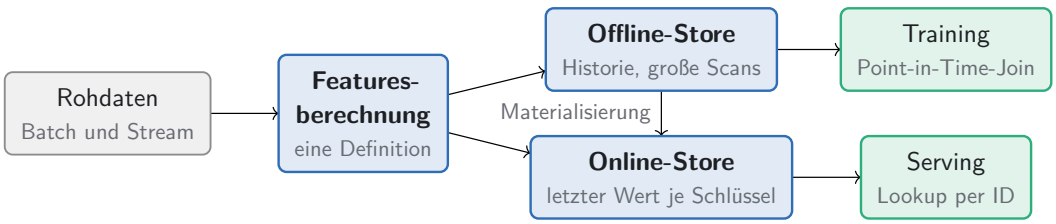


Abbildung 71: Aufbau eines Feature Stores: Eine Feature-Berechnung füllt Offline- und Online-Store; Training liest die Historie, der Betrieb liest den aktuellen Wert.

Point-in-Time-Korrektheit. Dieser Punkt wird in der Praxis häufig falsch gemacht. Angenommen, man sagt auf Basis der Bestellungen am 1. Juli voraus, wer bis Jahresende kündigt. Wer gekündigt hat, bestellt danach nichts mehr. Nimmt man als Trainings-Feature den „aktuellen" Wert der Tabelle (Dezember), steht dort für jeden Kündiger eine 0, und das Modell sieht die Antwort im Feature. Richtig ist der *As-of-Join*: Für jedes Beispiel nimmt man den letzten Wert, dessen Zeitstempel *vor oder gleich* dem Vorhersagezeitpunkt liegt.

```
# a) Naiver Join: "aktueller Stand" der Feature-Tabelle (hier Dezember)
dezember = features[features["ts"] == features["ts"].max()]
naiv = beispiele.merge(dezember[["kunde", "bestellungen_30d"]], on="kunde")

# b) Point-in-Time-Join: letzter Wert, der zum Zeitpunkt des Beispiels galt
pit = pd.merge_asof(beispiele.sort_values("ts"), features,
                    on="ts", by="kunde", direction="backward")

for name, df in [("naiver Join", naiv), ("Point-in-Time-Join", pit)]:
    auc = roc_auc_score(df["gekuendigt"], -df["bestellungen_30d"])
    print(f"{name:20s} AUC allein aus 'bestellungen_30d': {auc:.3f}")

print(pit.head(3).to_string(index=False))
```

Terminal

```
$ python 26_point_in_time.py
naiver Join          AUC allein aus 'bestellungen_30d': 0.969
Point-in-Time-Join   AUC allein aus 'bestellungen_30d': 0.576
 kunde         ts  gekuendigt  bestellungen_30d
     0 2025-07-01           0                 6
  1994 2025-07-01           1                 3
  1995 2025-07-01           0                 0
```

Die *AUC* ist hier die Wahrscheinlichkeit, dass ein Kunde mit Kündigung einen niedrigeren Feature-Wert hat als einer ohne (0,5 heißt Raten, 1 heißt perfekt). Der naive Join erreicht allein mit diesem einen Feature eine AUC von 0,969, der Point-in-Time-Join nur 0,576. Der erste Wert ist ein Zukunftsleck: Das Feature verrät nur deshalb die Kündigung, weil es nach der Kündigung gemessen wurde. Die ausgegebenen Zeilen zeigen das Ergebnis des korrekten Joins: Alle Beispiele tragen den Zeitstempel 1. Juli und den dazu passenden Feature-Wert.

Ein Feature Store macht diesen Join zur Standardoperation. In Feast sieht das so aus; die Skizze ist hier nicht ausgeführt:

```
merkmal = ["bestellverhalten:bestellungen_30d"]   # "Feature View:Feature"

# Training: Point-in-Time-Join gegen den Offline-Store
trainingsdaten = store.get_historical_features(
    entity_df=beispiele,                # Spalten: kunde, event_timestamp
    features=merkmal,
).to_df()

# Betrieb: Online-Store, nach der Materialisierung
store.materialize_incremental(end_date=datetime.now())
werte = store.get_online_features(
    features=merkmal, entity_rows=[{"kunde": 42}],
).to_dict()
```

Trade-offs und Grenzen. Ein Feature Store ist eine weitere Infrastrukturkomponente, die jemand betreiben muss, und er lohnt sich erst, wenn mehrere Modelle oder Teams Features gemeinsam nutzen. Die Aktualität des Online-Stores hängt vom Rhythmus der Materialisierung ab; ein stündlicher Lauf liefert Werte, die bis zu einer Stunde alt sind. Features, die erst aus den Daten der Anfrage entstehen (die Distanz einer Fahrt), gehören weiter in die Transformation des Modells (Pattern 21) und nicht in den Store. **Wann nicht:** Bei einem einzelnen Modell mit wenigen, billig berechenbaren Features ist eine gut getestete Funktion, die Training und Betrieb teilen, der einfachere Weg.

Bezug zu Software Engineering. Der Offline-Store ist die „Quelle der Wahrheit", der Online-Store ein auf Schlüsselzugriffe optimierter, materialisierter Cache davon, vergleichbar mit einer Read-Replica oder einer Materialized View. Die Registry entspricht einem Service-Katalog oder einer Paketverwaltung für Features; ein Feature hat einen Besitzer und eine Version wie eine Bibliothek.

SO ERKLÄRE ICH ES
Ich beginne mit zwei Teams und zwei Definitionen von „aktiver Kunde“, das kennt jeder aus Reportings. Dann zeichne ich die Zeitachse an die Tafel und markiere „heute“ und den Vorhersagezeitpunkt. Dass man beim Bau der Trainingsdaten versehentlich in die Zukunft schauen kann, überrascht viele.

Verständnisfrage: Warum braucht der Trainingsdatensatz den Wert des Features vom damaligen Zeitpunkt und nicht den aktuellen?
Antwort: Im Betrieb steht dem Modell zum Vorhersagezeitpunkt nur der damalige Stand zur Verfügung. Wer im Training spätere Werte verwendet, trainiert auf Informationen, die es im Betrieb nie geben wird (Zukunftsleck), und das Modell wirkt offline besser, als es ist.

Quellen. [1, Kapitel 6, Pattern 26], [11], [61]

9.8 Pattern 27: Model Versioning

Szenario. Ein Modell bewertet Zahlungen auf Betrug. Drei Anwendungen rufen es auf. Das Team hat ein neues, besseres Modell trainiert, das etwas anders kalibriert ist und für dieselben Eingaben teilweise andere Wahrscheinlichkeiten liefert. Man tauscht das Modell einfach aus.

Das Problem. Ein Austausch „an Ort und Stelle“ hat vier Nachteile. Clients, die sich auf das alte Verhalten verlassen (Schwellenwerte, die auf die alten Wahrscheinlichkeiten abgestimmt sind), brechen ohne Vorwarnung. Bei einer Verschlechterung gibt es keinen schnellen Rückweg. Man kann die Modelle nicht auf denselben Anfragen vergleichen. Und später ist unklar, welches Modell eine bestimmte Entscheidung getroffen hat, was bei Beschwerden und Audits wichtig ist.

Die Lösung. Das Pattern betreibt neue Modellversionen *neben* den alten, jede unter einem eigenen Endpunkt, etwa `/v1/vorhersage` und `/v2/vorhersage` [1, Kapitel 6, Pattern 27]. Clients wechseln, wenn sie bereit sind; die alte Version wird erst abgeschaltet, wenn niemand sie mehr braucht. Das setzt **Rückwärtskompatibilität** voraus: Eine neue Version soll die Eingabe der alten akzeptieren (neue Eingaben sind optional mit Standardwert) und möglichst dieselbe Ausgabestruktur liefern. Nicht vermeidbare Brüche, etwa eine andere Bedeutung des Ausgabewerts, rechtfertigen eine neue Hauptversion.

Hinter den Endpunkten steht eine **Modell-Registry**, ein Katalog der Modellartefakte mit Versionsnummer und Metadaten: Trainingsdaten-Stand, Code-Commit, Metriken, erwartete Eingabestruktur (die *Signatur*), Ersteller und Freigabestatus. Zusätzlich vergibt man *Aliase* für den Betrieb, etwa `production` und `canary`, die auf Versionen zeigen. Das Beispielprogramm baut eine winzige Registry:

```
class Registry:
    """Minimale Modell-Registry: Versionen mit Metadaten, dazu Aliase."""
    def __init__(self):
        self.versionen: dict[int, dict] = {}
        self.alias: dict[str, int] = {}

    def registrieren(self, modell, metriken: dict) -> int:
        nr = len(self.versionen) + 1
        self.versionen[nr] = {"modell": modell, "metriken": metriken}
        return nr

    def setze_alias(self, alias: str, version: int) -> None:
        self.alias[alias] = version

    def lade(self, version: int):
        return self.versionen[version]["modell"]

reg = Registry()
```

```
def vorhersage(anfrage_id: str, merkmale, canary_anteil: float = 0.0):
    """(Version, Vorhersage); Canary per Hash der Anfrage-ID, daher stabil."""
    version = reg.alias["production"]
    if "canary" in reg.alias:
```

```
        bucket = int(hashlib.sha256(anfrage_id.encode()).hexdigest(), 16) % 100
        if bucket < canary_anteil * 100:
            version = reg.alias["canary"]
    x = np.asarray(merkmale).reshape(1, -1)
    return version, int(reg.lade(version).predict(x)[0])
```

```
Terminal
$ python 27_model_versioning.py
Version 1: LogisticRegression           accuracy 0.791
Version 2: GradientBoostingClassifier   accuracy 0.843
Anfragen je Version bei 10 % Canary: {1: 911, 2: 89}
Übereinstimmung v1/v2: 0.870
production -> 2
Rollback: production -> 1
```

Version 2 (Gradient Boosting) erreicht eine Genauigkeit von 0,843 gegenüber 0,791 für Version 1 (logistische Regression). Beim Canary mit 10 % gehen 89 von 1000 Anfragen an Version 2; der Anteil weicht von 100 ab, weil die Zuordnung per Hash der Anfrage-ID erfolgt und damit nur im Mittel genau 10 % ergibt. Die beiden Versionen stimmen bei 87 % der Testanfragen überein, bei den übrigen 13 % entscheidet die Version über das Ergebnis; das sind die Fälle, die man sich im Shadow-Vergleich genauer ansieht. Der Rollback in den letzten Zeilen ist ein einziger Alias-Wechsel.

Betriebsmuster. Aus parallelen Versionen ergeben sich Einführungsstrategien, die man aus dem Deployment kennt. Beim *Shadow Mode* erhält die neue Version die Anfragen mit, ihre Antworten werden aber nur protokolliert und verglichen. Beim *Canary* bekommt ein kleiner Anteil (hier 10 %) der Anfragen die neue Version; die Zuordnung per Hash der Anfrage-ID bleibt für dieselbe ID stabil; für eine feste Zuordnung je Client hasht man die Client-ID. Beim *A/B-Test* vergleicht man beide Versionen an einer Geschäftskennzahl. Der *Rollback* ist ein Alias-Wechsel, kein Neu-Deployment. Werkzeuge: MLflow bietet eine Model Registry mit Versionen und Aliasen (ältere Versionen kannten feste Stufen wie Staging und Production, neuere frei benennbare Aliase), TFX hat mit der Komponente Pusher einen Schritt, der ein freigegebenes Modell an ein Ziel wie ein Serving-System ausliefert [59], [60].

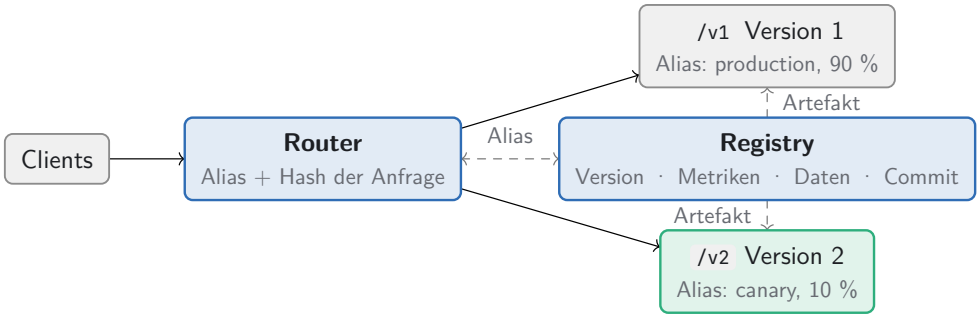


Abbildung 72: Model Versioning: mehrere Versionen laufen nebeneinander. Die Registry verwaltet die Artefakte und sagt dem Router, welche Version welchen Alias trägt; ein Rollback ändert nur diesen Alias.

Trade-offs und Grenzen. Jede laufende Version kostet Rechenleistung und Betriebsaufwand, und ohne Abkündigungsregeln sammeln sich Altversionen an. A/B-Vergleiche brauchen genug Verkehr und oft verzögerte Labels (erst Wochen später weiß man, ob die Zahlung Betrug war), weshalb sie mit Continued Model Evaluation (Pattern 18) zusammen gedacht werden müssen. Schließlich ist die Modellversion nur dann reproduzierbar, wenn auch Daten und Code hinter ihr festgehalten sind; die Registry hilft dabei nur, wenn sie diese Verweise tatsächlich speichert. **Wann nicht:** Bei internen Modellen mit genau einem Aufrufer, die in Minuten neu ausgerollt werden können, reicht ein sauberes Deployment mit Versionsmarkierung im Log.

Bezug zu Software Engineering. Das ist API-Versionierung (Semantic Versioning, Expand/Contract bei Schemaänderungen) kombiniert mit Release-Strategien wie Blue-Green und Canary Deployment. Die Registry entspricht einem Artefakt-Repository für Pakete, die Aliase den Tags in Git oder Container-Registries.

SO ERKLÄRE ICH ES

Ich frage, was ein Team tut, wenn es die Signatur einer Bibliotheksfunktion ändern will, ohne die Aufrufer zu brechen: Version parallel anbieten, Aufrufer migrieren lassen, alte Version abkündigen. Mit diesem Vergleich ist klar, dass ein Modell dasselbe Problem hat.

Verständnisfrage: Warum reicht ein Git-Tag auf dem Trainingscode nicht, um ein Modell eindeutig zu identifizieren?
Antwort: Das Modell entsteht aus Code, Daten, Parametern und Zufall. Derselbe Commit auf anderen Daten liefert ein anderes Modell. Die Registry hält das Ergebnis samt allen Verweisen fest.

Quellen. [1, Kapitel 6, Pattern 27], [59], [60]

10 Responsible AI

Ein Modell kann technisch einwandfrei sein und trotzdem untauglich: weil niemand weiß, ob es mehr leistet als eine einfache Regel, weil sich seine Entscheidungen nicht begründen lassen oder weil es für einen Teil der Betroffenen systematisch schlechter funktioniert. Die drei Patterns dieses Kapitels sind Gegenmittel dafür. Sie sind weniger Techniken als Prüfhaltungen: Ein *Heuristic Benchmark* gibt der Leistungszahl einen Maßstab, *Explainable Predictions* macht Entscheidungen nachvollziehbar, und die *Fairness Lens* prüft Leistung getrennt nach Gruppen. Alle drei lassen sich in der Entwicklung als Test- und Reviewschritt verankern.

Zur Dokumentation solcher Prüfungen hat sich die *Model Card* etabliert, ein kurzes, standardisiertes Dokument zu einem Modell, das Verwendungszweck, Trainings- und Evaluationsdaten, Kennzahlen (auch aufgeschlüsselt nach Gruppen) und bekannte Grenzen festhält [62]. Das Pendant für Datensätze sind die *Datasheets for Datasets* [63]. Die Beispiele dieses Kapitels liegen in `code/09-responsible-ai/` und werden wie in Kapitel 9 mit `uv run --with scikit-learn --with pandas --with numpy python <datei>.py` gestartet; die Daten sind simuliert, damit die richtige Antwort bekannt ist.

10.1 Pattern 28: Heuristic Benchmark

Szenario. Ein Lieferdienst sagt die Lieferdauer in Minuten voraus. Das Team trainiert ein Modell und berichtet stolz: mittlerer absoluter Fehler 2,5 Minuten. Die Geschäftsführung fragt: „Ist das gut?“ Ohne Vergleich kann das niemand sagen. 2,5 Minuten wären hervorragend, wenn eine Schätzung ohne Modell um 10 Minuten danebenläge, und enttäuschend, wenn die Disponenten mit einer Faustregel kaum mehr Fehler machen.

Das Problem. Eine Fehlerkennzahl allein hat keine Bedeutung, solange es keinen Vergleich gibt. Ohne Vergleich lassen sich drei Dinge nicht beantworten: ob das Modell den Aufwand wert ist (Training, Betrieb, Wartung), ob eine Verbesserung am Modell wirklich etwas bringt, und was man den Fachabteilungen sagen soll. Hinzu kommt eine Gefahr bei der Modellwahl: Ein komplexes Modell kann sich mit einem schlechten Bezugspunkt gut aussehen lassen.

Die Lösung. Das Pattern empfiehlt, *vor* jedem ernsthaften Modell einen einfachen **Vergleichsmaßstab** (*Heuristic Benchmark*) zu bestimmen und das Modell daran zu messen [1, Kapitel 7, Pattern 28]. Der Maßstab soll einfach, schnell zu berechnen und für Fachleute nachvollziehbar sein. Typische Kandidaten sind eine Konstante (Mittelwert oder Median der Zielgröße, bei Klassifikation die häufigste Klasse), eine Faustregel von Fachleuten („10 Minuten plus 3 Minuten pro Kilometer“), der Median je Gruppe (je Distanzband) oder ein einfaches lineares Modell. Berichtet wird in der Einheit der Fachabteilung, hier in Minuten, nicht in einer abstrakten Metrik.

```
ergebnisse = {}

# 1) Konstante: Median der Trainingsdaten
dummy = DummyRegressor(strategy="median").fit(train[merkmale], ziel)
ergebnisse["Median (Konstante)"] = mae(dummy.predict(test[merkmale]))

# 2) Faustregel der Disposition, ohne Daten festgelegt: 10 min + 3 min/km
ergebnisse["Faustregel 10 + 3/km"] = mae(10 + 3 * test["distanz_km"])

# 3) Median je Distanzband (aus Trainingsdaten gelernt, aber erklärbar)
band = lambda d: pd.cut(d, [0, 2, 4, 6, 9, 100], labels=False)
median_je_band = ziel.groupby(band(train["distanz_km"])).median()
ergebnisse["Median je Distanzband"] = mae(band(test["distanz_km"]).map(median_je_band))
```

```
# 4) Modell
modell = HistGradientBoostingRegressor(random_state=0).fit(train[merkmale], ziel)
ergebnisse["Gradient Boosting"] = mae(modell.predict(test[merkmale]))
```

```
Terminal
$ python 28_heuristic_benchmark.py
Median (Konstante)        MAE  8.14 min
Faustregel 10 + 3/km      MAE  4.25 min
Median je Distanzband     MAE  4.50 min
Gradient Boosting         MAE  2.50 min
Modell gegenüber bester Heuristik: 41.2 % weniger Fehler
```

Die Metrik ist der mittlere absolute Fehler (*MAE*) in Minuten, auf denselben Testdaten für alle vier Ansätze. Der Median als Konstante liegt bei 8,14 Minuten, die Faustregel (10 Minuten plus 3 pro Kilometer) bei 4,25, der Median je Distanzband bei 4,50. Das Gradient-Boosting-Modell erreicht 2,50 Minuten, 41 % weniger Fehler als die beste Heuristik. Damit lässt sich der Geschäftsführung etwas sagen: Die Disposition liegt mit ihrer Faustregel im Schnitt 4,3 Minuten daneben, das Modell 2,5. Ob die gewonnenen 1,8 Minuten Entwicklung und Betrieb rechtfertigen, ist jetzt eine Frage, die man diskutieren kann. Beachtenswert ist, dass die Faustregel rund 70 % des Wegs von der Konstante zum Modell zurücklegt; sie ist als Rückfallebene brauchbar, wenn das Modell ausfällt.

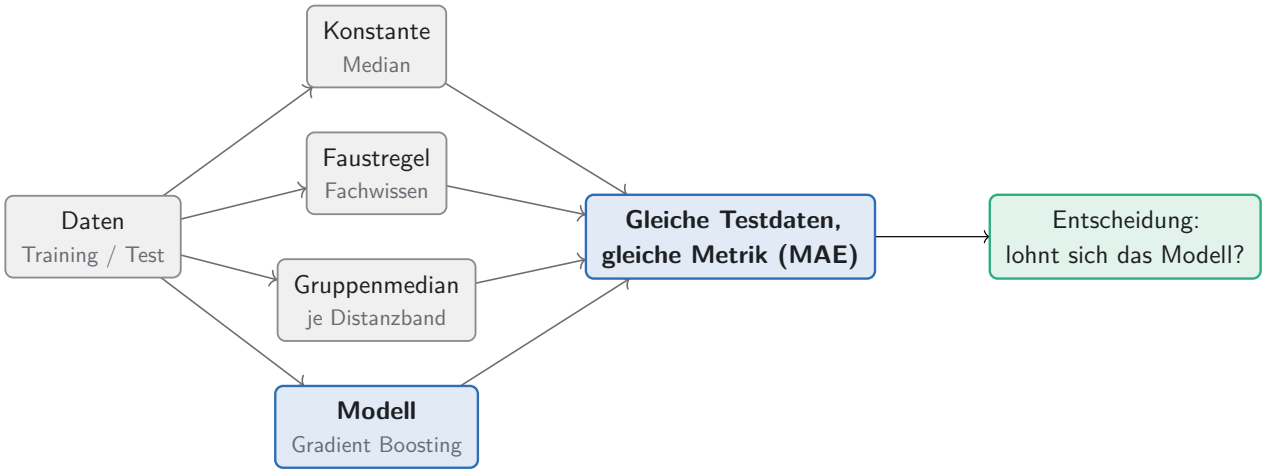


Abbildung 73: Das Modell wird nicht für sich, sondern im Vergleich mit einfachen Maßstäben bewertet, auf denselben Testdaten und mit derselben Metrik.

Trade-offs und Grenzen. Ein zu schwacher Maßstab (der Mittelwert, wo eine Faustregel existiert) ist ein Strohmann und lässt jedes Modell glänzen; ein sinnvoller Maßstab ist der *beste* einfache Ansatz, den man mit wenig Aufwand findet. Der Maßstab darf keine Informationen nutzen, die im Betrieb fehlen, und sollte wie das Modell nur aus Trainingsdaten abgeleitet werden; Konstanten der Faustregel dürfen nicht anhand der Testdaten nachjustiert werden. Der Vergleich ist auch nicht das einzige Kriterium: Ein Modell, das nur knapp besser ist, kann trotzdem sinnvoll sein, wenn seine Fehler an wichtigen Stellen kleiner sind, und ein deutlich besseres kann unbrauchbar sein, wenn es nicht erklärbar oder nicht fair ist (Pattern 29 und 30). Bei unausgewogenen Klassen ist die Genauigkeit des Mehrheitsklassen-Maßstabs bereits sehr hoch (Pattern 10).

Wann nicht: Es gibt keinen Fall, in dem man den Vergleich weglassen sollte. Sinnlos wird er nur dort, wo kein einfacher Maßstab existiert, etwa bei Bilderkennung ohne bisherigen Ablauf; dann ist der Mensch (oder ein bestehendes System) der Maßstab.

Bezug zu Software Engineering. Der Maßstab ist die *Referenzimplementierung* oder das *Test-Orakel*: Man vergleicht eine komplizierte Lösung mit einer einfachen, deren Korrektheit offensichtlich ist. Dahinter steht das Prinzip *KISS* (*keep it simple*) und die Frage nach dem Mehrwert des Aufwands, wie sie bei Performance-Regressionstests zur Baseline gestellt wird.

SO ERKLÄRE ICH ES
Ich nenne den Studierenden zuerst nur die Kennzahl des Modells und frage, ob sie gut ist. Nach dem Zögern zeige ich die Ausgabe mit den Maßstäben. Danach geht es um die Frage, welche Aussage man der Geschäftsführung machen würde, wenn das Modell nur knapp besser als die Faustregel ist.

Verständnisfrage: Warum darf man die Konstanten der Faustregel nicht an den Testdaten einstellen? **Antwort:** Dann wäre der Maßstab für genau diese Daten optimiert, und der Vergleich mit dem Modell, das nur Trainingsdaten sah, wäre unfair. Der Maßstab muss wie das Modell ohne Kenntnis der Testdaten entstehen.

Quellen. [1, Kapitel 7, Pattern 28], [42]

10.2 Pattern 29: Explainable Predictions

Szenario. Eine Bank lehnt einen Kreditantrag ab, auf Empfehlung eines Modells. Die Kundin fragt, warum. Die Sachbearbeiterin sieht nur „Wahrscheinlichkeit Bewilligung: 31 %“. Gleichzeitig will das Entwicklungsteam wissen, ob das Modell sich auf sinnvolle Dinge stützt und nicht auf ein Artefakt der Daten.

Das Problem. Die leistungsfähigsten Modelle (große Ensembles, neuronale Netze) sind für Menschen nicht lesbar. Man braucht Erklärungen aus zwei Gründen: um einzelne Entscheidungen zu begründen, gegenüber Betroffenen, Aufsicht und Fachabteilungen, und um das Modell zu *debuggen*, etwa ein Feature zu entdecken, das nur zufällig mit dem Ziel zusammenhängt. Dabei sind zwei Fragen zu unterscheiden. Eine **globale** Erklärung beschreibt, welche Features das Modell insgesamt bestimmen. Eine **lokale** Erklärung beschreibt, warum das Modell für *diesen einen* Fall so entschieden hat. Beide sind nötig, und sie beantworten verschiedene Fragen.

Die Lösung. Das Pattern nutzt **Feature Attribution**: Man verteilt die Vorhersage (oder deren Änderung gegenüber einem Referenzpunkt) auf die Eingabe-Features [1, Kapitel 7, Pattern 29]. Die zentrale Idee dahinter ist die **Baseline**, ein Referenzfall, gegen den erklärt wird. Eine lokale Erklärung beantwortet immer nur die Frage, warum die Vorhersage für diesen Fall anders ist als für die Baseline, nicht warum sie einen absoluten Wert hat. Bei tabellarischen Daten ist die Baseline oft der Durchschnittsfall der Trainingsdaten, bei Bildern ein schwarzes Bild. Die Wahl der Baseline ändert die Erklärung.

Die gängigen Verfahren unterscheiden sich in der Idee und im Anwendungsbereich:

Verfahren	Reichweite	Idee und Grenze
Permutation Importance	global	Modellunabhängig. Misst, wie stark die Leistung auf Testdaten sinkt, wenn man die Werte eines Features zufällig durchmischt. Bei korrelierten Features irreführend, siehe unten [4], [64].
LIME	lokal	Modellunabhängig. Verändert die Eingabe zufällig in der Nachbarschaft des Falls und passt dort ein einfaches lineares Modell an die Antworten des großen Modells an. Hängt von Stichprobe und Nachbarschaftsdefinition ab und kann bei Wiederholung schwanken [65].
SHAP	lokal, global	Verteilt die Abweichung der Vorhersage von der Baseline nach den Shapley-Werten der Spieltheorie auf die Features; die Beiträge summieren sich exakt zur Differenz. Die exakte Berechnung braucht alle Teilmengen von Features, daher Näherungen (KernelSHAP) oder Spezialfälle (TreeSHAP für Baummodelle) [66], [67].
Integrated Gradients	lokal	Für differenzierbare Modelle, vor allem neuronale Netze. Summiert die Gradienten entlang einer Geraden von der Baseline zur Eingabe; die Beiträge ergeben ebenfalls die Differenz zur Baseline. Braucht eine sinnvolle Baseline [68].

Zwei der Verfahren brauchen eine Anschauung. Bei den Shapley-Werten stellt man sich vor, dass die Features nacheinander in den Raum treten, vom Baseline-Fall zum tatsächlichen Antrag. Jedes verändert beim Eintreten die Vorhersage um einen Betrag, und der hängt davon ab, wer schon da ist: Eine hohe Schuldenquote wiegt anders, wenn das Einkommen bereits bekannt ist. Deshalb mittelt man über alle Reihenfolgen, und der Mittelwert ist der Beitrag des Features. Daher kommt der Rechenaufwand, denn

bei n Features sind 2^n Teilmengen zu betrachten. Integrated Gradients wählt statt aller Reihenfolgen einen einzigen Weg: Die Eingabe wird in vielen kleinen Schritten von der Baseline zum Antrag verschoben, und bei jedem Schritt notiert man, wie steil die Vorhersage in Richtung jedes Features ansteigt. Die Summe dieser Steigungen entlang des Wegs ergibt den Beitrag.

SHAP und Integrated Gradients erfüllen eine nützliche Eigenschaft: Die Beiträge sind **additiv** und summieren sich zur Differenz zwischen Vorhersage und Baseline-Vorhersage (bei Integrated Gradients „Completeness" genannt, bei Shapley-Werten „Efficiency"). Das lässt sich nachrechnen, und das Beispielprogramm tut es. Es implementiert die Shapley-Werte für fünf Features ohne Bibliothek (alle 2^5 Teilmengen) und prüft die Implementierung an einem linearen Modell, für das die Formel `coef * (x - baseline)` bekannt ist.

```python
def shapley(f, x: np.ndarray, baseline: np.ndarray) -> np.ndarray:
    """Merkmale außerhalb der Koalition werden durch die Baseline ersetzt."""
    d = len(x)
    phi = np.zeros(d)
    for i in range(d):
        rest = [j for j in range(d) if j != i]
        for k in range(d):
            gewicht = factorial(k) * factorial(d - k - 1) / factorial(d)
            for S in combinations(rest, k):
                a = baseline.copy(); a[list(S)] = x[list(S)]
                b = a.copy(); b[i] = x[i]
                phi[i] += gewicht * (f(b[None, :])[0] - f(a[None, :])[0])
    return phi
```

Terminal

```
$ python 29_explainable.py
Accuracy Test: 0.704

Permutation Importance (AUC-Abfall)
                     Mittel    Std
einkommen             0.187  0.015
schuldenquote         0.062  0.008
beschaeftigung_jahre  0.019  0.005
alter                 0.001  0.001
rauschen             -0.002  0.002

Mit Duplikat 'einkommen_netto'
                     Mittel    Std
einkommen_netto       0.060  0.008
schuldenquote         0.056  0.007
einkommen             0.043  0.008
beschaeftigung_jahre  0.016  0.004
alter                 0.000  0.001
rauschen             -0.002  0.001

Exakte Shapley-Werte für Antrag 0 (Wahrscheinlichkeit 'bewilligt'):
einkommen             -0.154
schuldenquote          0.099
beschaeftigung_jahre   0.031
alter                 -0.008
rauschen               0.001
Baseline f(b) = 0.559 | Vorhersage f(x) = 0.528 | Summe = -0.031 = f(x) - f(b): True

Shapley (Brute Force) == coef*(x-b) beim linearen Modell: True
Integrated Gradients (logistisches Modell):
einkommen             -0.186
schuldenquote          0.152
beschaeftigung_jahre   0.011
alter                  0.015
rauschen              -0.002
Summe IG = -0.0096 | f(x) - f(b) = -0.0096
```

Der Random Forest erreicht eine Genauigkeit von 0,704 (die Daten sind absichtlich verrauscht). Die *Permutation Importance* misst den Abfall der AUC auf den Testdaten, wenn ein Feature 20-mal durchmischt

wird. Die *AUC* (Fläche unter der ROC-Kurve) ist die Wahrscheinlichkeit, dass das Modell einen zufällig gewählten bewilligten Antrag höher einstuft als einen abgelehnten; 0,5 entspricht Raten, 1 ist perfekt. Das Einkommen führt mit 0,187, gefolgt von Schuldenquote (0,062) und Beschäftigungsjahren (0,019); Alter und das reine Zufalls-Feature `rauschen` liegen bei null (der leicht negative Wert ist Streuung, die Standardabweichung steht daneben).

Die zweite Tabelle zeigt die Grenze des Verfahrens. Mit einem fast identischen Feature (`einkommen_netto`, 75 % des Einkommens plus Rauschen) sinkt die Wichtigkeit beider auf 0,060 und 0,043, zusammen 0,103 statt 0,187. Das Modell nutzt die Information weiterhin, aber wird eine Spalte durchmischt, springt die andere ein.

Die Shapley-Werte erklären einen einzelnen Antrag. Die Baseline ist der Durchschnittsantragsteller der Trainingsdaten (alle Features auf ihrem Mittelwert); für ihn sagt das Modell 0,559 voraus, für Antrag 0 dagegen 0,528, also −0,031. Das Einkommen dieses Antrags senkt die Wahrscheinlichkeit gegenüber der Baseline um 0,154, die Schuldenquote hebt sie um 0,099, die Beschäftigungsjahre um 0,031. Die Beiträge summieren sich exakt zur Differenz (die Ausgabe prüft das). Dass sich hier zwei große Effekte fast aufheben, sähe man an der Gesamtvorhersage nicht. Die Probe am linearen Modell bestätigt die Implementierung: Dort ist der Shapley-Wert gleich Koeffizient mal Abstand zur Baseline, und die Brute-Force-Rechnung liefert dasselbe.

Integrated Gradients (letzte Zeilen) wurden am logistischen Modell berechnet, weil der Random Forest keine Gradienten hat. Das Verfahren integriert numerisch die Steigung der Vorhersage entlang der Geraden von der Baseline zum Antrag; die Summe der Beiträge entspricht der Differenz der Vorhersagen (−0,0096). Bei einem neuronalen Netz leitet das Framework die Gradienten automatisch ab.

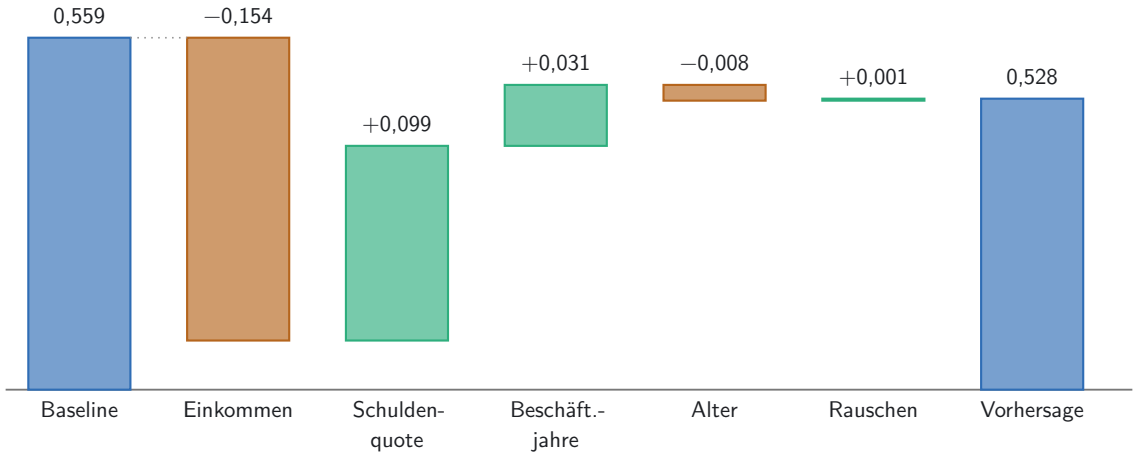


Abbildung 74: Beiträge der Features zur Vorhersage für Antrag 0 (Random Forest), als Wasserfall von der Baseline (0,559) zur Vorhersage (0,528). Die Werte stammen aus der Ausgabe oben; die senkrechte Achse beginnt bei 0,38, nicht bei 0. Die Reihenfolge der Balken ist nur Darstellung, die Shapley-Werte selbst hängen nicht von ihr ab.

Grenzen. Eine Erklärung beschreibt, wie sich das *Modell* verhält, nicht, wie die Welt funktioniert. Dass „Einkommen" den Antrag positiv beeinflusst, heißt nicht, dass eine Gehaltserhöhung die Kreditwürdigkeit kausal erhöht. Erklärungen hängen von der Baseline ab, sind bei korrelierten Features mehrdeutig (die Ausgabe zeigt, wie sich die Wichtigkeit auf zwei Spalten verteilt) und können täuschen: J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, und B. Kim [69] zeigen, dass manche populäre Bildattributionen sich kaum ändern, wenn man die Gewichte des Modells zufällig setzt. C. Rudin [70] argumentiert für Entscheidungen mit hohem Einsatz, von vornherein interpretierbare Modelle zu bauen, statt undurchsichtige Modelle nachträglich zu erklären. **Wann nicht:** Wenn ein einfaches Modell (lineare Regression, flacher Entscheidungsbaum) genügt, ist es selbst die Erklärung.

Bezug zu Software Engineering. Erklärbarkeit entspricht *Observability* und *Debuggability*: Man fragt nach dem Grund für ein Ergebnis, wie man bei einem Programm einen Stacktrace oder Log braucht. Die Baseline entspricht dem Referenzlauf, mit dem man einen fehlerhaften Lauf vergleicht (Differential Debugging). In Bereichen wie der Kreditvergabe kommen Anforderungen an die Begründbarkeit automatisierter Entscheidungen hinzu; deren genauen Umfang klärt die Rechtsabteilung.

SO ERKLÄRE ICH ES
Ich zeige den Studierenden eine Waage mit einem Startgewicht (Baseline) und fünf Gewichten, die darauf gelegt werden (Features). Die Erklärung ist die Liste der Gewichte und ihrer Wirkung, die Summe ergibt das Endgewicht. Dann frage ich, was sich ändert, wenn das Startgewicht ein anderes ist.

Verständnisfrage: Warum zeigt die Permutation Importance bei zwei fast gleichen Features für jedes nur einen kleinen Wert?
Antwort: Wird eines durchmischt, kann das Modell sich auf das andere stützen, und die Leistung sinkt kaum. Die Wichtigkeit verteilt sich auf beide, obwohl die Information für das Modell sehr wichtig ist.

Quellen. [1, Kapitel 7, Pattern 29], [4], [64], [65], [66], [67], [68], [69], [70], [71]

10.3 Pattern 30: Fairness Lens

Szenario. Dasselbe Kreditmodell, nun mit einer Gruppenzugehörigkeit A oder B, die nicht Teil der Eingaben ist. Die Gesamtgenauigkeit ist gut. Eine Auswertung nach Gruppen zeigt jedoch, dass kreditwürdige Antragsteller aus Gruppe B seltener bewilligt werden als gleich kreditwürdige aus Gruppe A.

Das Problem. Zusammenfassende Kennzahlen verbergen Unterschiede: Ein Modell kann im Durchschnitt gut und für einzelne Gruppen deutlich schlechter sein. Die Ursachen liegen selten im Algorithmus, meist in Daten und Umfeld:

- **Stichprobenverzerrung.** Eine Gruppe ist unterrepräsentiert, das Modell lernt ihre Muster schlechter. J. Buolamwini und T. Gebru [72] fanden bei drei kommerziellen Systemen zur Geschlechtsklassifikation aus Gesichtsbildern deutlich höhere Fehlerraten für Frauen mit dunkler Hautfarbe; sie verweisen unter anderem auf gängige Benchmark-Datensätze, in denen hellhäutige Personen überwiegen.
- **Label-Verzerrung.** Die Zielgröße ist selbst verzerrt oder ein schlechter Ersatz für das, was man messen will. Z. Obermeyer, B. Powers, C. Vogeli, und S. Mullainathan [73] zeigten, dass ein Algorithmus im US-Gesundheitswesen Gesundheitskosten als Ersatz für Krankheitslast verwendete und dadurch Schwarze Patientinnen und Patienten bei gleichem Bedarf benachteiligte. Im Beispiel sind es historische Entscheidungen: Kreditwürdige Antragsteller aus B wurden seltener bewilligt, und das Modell lernt das als „Wahrheit".
- **Stellvertreter-Features (Proxies).** Das Modell kann die Gruppe auch ohne Eingabe rekonstruieren, etwa über Postleitzahl oder Einkommen; sie wegzulassen („Fairness durch Nichtwissen") schützt deshalb nicht.
- **Rückkopplung.** Entscheidungen verändern die Daten, aus denen später gelernt wird.

Die Lösung. Das Pattern ist eine Prüfpraxis [1, Kapitel 7, Pattern 30]: Man wertet das Modell nicht nur insgesamt, sondern *je Gruppe* aus, zieht passende Fairness-Kennzahlen heran und greift bei Bedarf ein. Die gängigen Kennzahlen betreffen verschiedene Fragen:

- **Demographic Parity**: Die Bewilligungsrate (Anteil positiver Vorhersagen) ist in allen Gruppen gleich.
- **Equal Opportunity** [74]: Die *True Positive Rate* ist in allen Gruppen gleich, also der Anteil der tatsächlich Kreditwürdigen, die auch bewilligt werden. Wer es verdient, soll unabhängig von der Gruppe dieselbe Chance haben.
- **Equalized Odds**: Zusätzlich ist die *False Positive Rate* gleich (der Anteil nicht Kreditwürdiger, die trotzdem bewilligt werden).
- **Predictive Parity**: Eine bewilligte Person ist in allen Gruppen mit gleicher Wahrscheinlichkeit kreditwürdig (gleiche Precision). Die verwandte **Kalibrierung** verlangt dasselbe für jeden Wert des Scores: Eine Vorhersage von 70 % bedeutet in jeder Gruppe, dass etwa 70 % zurückzahlen.

Diese Kriterien sind im Allgemeinen nicht gleichzeitig erfüllbar: Unterscheiden sich die Grundraten zwischen den Gruppen, lassen sich Kalibrierung (Kleinberg et al.) beziehungsweise Predictive Parity (Chouldechova) und gleiche Fehlerraten nur im Sonderfall einer perfekten Vorhersage vereinbaren [75], [76]. Welche Kennzahl angemessen ist, ist eine Entscheidung über den Anwendungsfall, kein technisches Detail. Das Beispiel stellt die True Positive Rate in den Mittelpunkt. Das Modell hat die Gruppe nie als Eingabe; die Tabellenfunktion wertet die Vorhersagen anschließend je Gruppe aus (der Datenaufbau steht in der Datei):

```
pred = score >= 0.5
print("Ein Schwellwert 0.5 für alle (Referenz: tatsächliche Rückzahlung):")
t1 = tabelle(pred)
print(t1.round(3).to_string())
print(f"Differenz TPR (A - B): {t1.loc['A', 'TPR'] - t1.loc['B', 'TPR']:.3f}\n")

# Post-Processing nach Hardt et al.: Schwelle je Gruppe, TPR von A als Ziel
ziel = t1.loc["A", "TPR"]
schwellen = {"A": 0.5}
gut_B = (te["gruppe"] == "B").to_numpy() & te["rueckzahlung"].to_numpy()
schwellen["B"] = float(np.quantile(score[gut_B], 1 - ziel))
ist_A = (te["gruppe"] == "A").to_numpy()
pred2 = np.where(ist_A, score >= schwellen["A"], score >= schwellen["B"])
print(f"Schwellen je Gruppe: A {schwellen['A']:.2f}, B {schwellen['B']:.2f}")
t2 = tabelle(pred2)
print(t2.round(3).to_string())
print(f"Differenz TPR (A - B): {t2.loc['A', 'TPR'] - t2.loc['B', 'TPR']:.3f}")
```

Terminal

```
$ python 30_fairness_lens.py
Ein Schwellwert 0.5 für alle (Referenz: tatsächliche Rückzahlung):
        n  Bewilligungsrate    TPR    FPR  Precision
A  3378.0             0.531  0.708  0.322      0.723
B  1422.0             0.326  0.515  0.191      0.657
Differenz TPR (A - B): 0.192

Schwellen je Gruppe: A 0.50, B 0.36
        n  Bewilligungsrate    TPR    FPR  Precision
A  3378.0             0.531  0.708  0.322      0.723
B  1422.0             0.504  0.707  0.361      0.582
Differenz TPR (A - B): 0.001

Fairlearn MetricFrame:
          TPR  Selection
gruppe
A       0.708      0.531
B       0.515      0.326
TPR-Differenz (max-min): 0.192
```

Bei einem einheitlichen Schwellwert von 0,5 liegt die True Positive Rate in Gruppe A bei 0,708 und in Gruppe B bei 0,515: Von den tatsächlich Kreditwürdigen werden in A rund 71 % bewilligt, in B rund 52 %. Die Bewilligungsrate beträgt 53 % gegenüber 33 %. Die Differenz der True Positive Rates ist 0,192. Das Modell hat die Gruppe nie als Eingabe gesehen; der Unterschied entsteht über das Einkommen, das in der Simulation in Gruppe B niedriger ist, und über die verzerrten historischen Labels. Aus der Ausgabe allein lässt sich beides nicht trennen; dafür bräuchte man zusätzliche Auswertungen. Die letzten Zeilen zeigen dieselben Kennzahlen von Fairlearns `MetricFrame` zur Kontrolle.

Eingriffsmöglichkeiten. Man kann vor, während oder nach dem Training eingreifen: Daten ausgleichen oder neu gewichten (*Pre-Processing*), die Fairness-Bedingung in die Optimierung aufnehmen (*In-Processing*), oder die Entscheidungsschwelle je Gruppe anpassen (*Post-Processing*). M. Hardt, E. Price, und N. Srebro [74] zeigen das Letztere für Equal Opportunity und Equalized Odds. Das Beispiel wählt je Gruppe eine Schwelle so, dass die True Positive Rate gleich ist (Gruppe A behält 0,5, für B wird die Schwelle aus den Vorhersagen der tatsächlich kreditwürdigen B-Antragsteller bestimmt). Zur Vereinfachung geschieht das auf denselben Testdaten, mit denen anschließend ausgewertet wird; die ausgewiesene Gleichheit ist deshalb optimistisch. In der Praxis legt man die Schwelle auf getrennten Validierungsdaten fest und prüft sie dann auf den Testdaten (dieselbe Regel wie beim Heuristic Benchmark). Mit einer Schwelle von 0,36 für Gruppe B sind die True Positive Rates praktisch gleich (0,708 und 0,707). Der Eingriff hat einen Preis, den die Tabelle sichtbar macht: In Gruppe B steigt die False Positive Rate von 0,191 auf 0,361, die Precision sinkt von 0,657 auf 0,582, es werden also mehr nicht Kreditwürdige bewilligt. Gleiche Chancen für die Kreditwürdigen kosten höheres Ausfallrisiko in einer Gruppe; ob das vertretbar ist, entscheiden die

Fachabteilung und das Recht, nicht der Code. Rechtlich ist eine Behandlung nach Gruppenzugehörigkeit je nach Kontext heikel; das Handbuch gibt keine Rechtsauskunft.

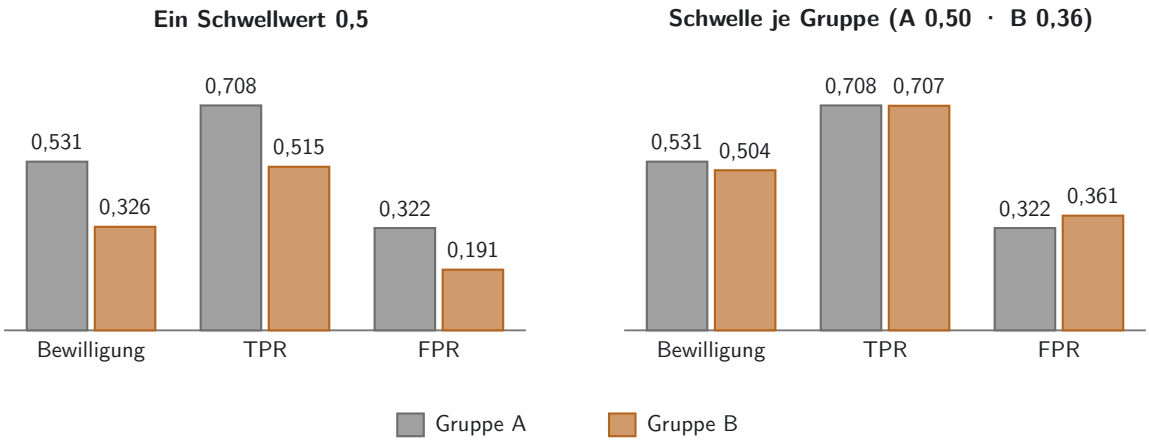


Abbildung 75: Bewilligungsrate, True Positive Rate (TPR) und False Positive Rate (FPR) je Gruppe, links mit einem einheitlichen Schwellwert, rechts mit gruppenspezifischen Schwellen. Werte aus der Ausgabe oben.

Werkzeuge. Fairlearn bietet `MetricFrame` für Kennzahlen je Gruppe und Verfahren zum Abmildern, darunter `ThresholdOptimizer` für Post-Processing [77]; das Beispiel gibt zur Kontrolle die Kennzahlen von `MetricFrame` aus (Fairlearn 0.15). Das *What-If Tool* ist eine interaktive Oberfläche, in der man Datenpunkte verändert, Modelle auf Teilgruppen vergleicht und Schwellen im Hinblick auf Fairness-Kriterien einstellt [78]. Ähnliche Werkzeuge sind TensorFlow Fairness Indicators und IBM AI Fairness 360.

Trade-offs und Grenzen. Fairness-Kennzahlen setzen voraus, dass man die Gruppenzugehörigkeit kennt, deren Erhebung selbst rechtlich und ethisch beschränkt sein kann. Das Beispiel nutzt die tatsächliche Rückzahlung als Referenz; für abgelehnte Antragsteller gibt es sie in der Praxis nie (das Problem der *selektiven Labels*). Kleine Gruppen haben unsichere Kennzahlen, überschneidende Gruppen (etwa Geschlecht und Alter) muss man auch gemeinsam betrachten, und keine Kennzahl ersetzt die Entscheidung, was in der Anwendung gerecht ist. **Wann nicht:** Bei Modellen ohne Personenbezug (Maschinenprognosen) entfällt es; wo Modelle Menschen betreffen, gehört es dazu.

Bezug zu Software Engineering. Es ist dieselbe Denkweise wie bei *Testabdeckung* und *Äquivalenzklassen*: Ein Durchschnitt kann stimmen, während eine Klasse von Eingaben versagt. Gruppenmetriken sind Tests je Äquivalenzklasse. Die Model Card ist das Gegenstück zur API-Dokumentation mit bekannten Einschränkungen.

> **SO ERKLÄRE ICH ES**
> Ich zeige zuerst nur die Gesamtgenauigkeit und lasse sie gut finden, dann die Tabelle nach Gruppen. Anschließend frage ich, wo der Fehler liegt: im Algorithmus, in den Daten oder in der Welt, aus der die Daten stammen. Die Diskussion über die Kriterien beginnt mit der Frage, wessen Fehler schwerer wiegt, ein zu Unrecht abgelehnter oder ein zu Unrecht bewilligter Antrag.
>
> **Verständnisfrage:** Warum genügt es nicht, die Gruppenzugehörigkeit als Eingabe wegzulassen?
> **Antwort:** Das Modell kann sie über Stellvertreter-Features wie Einkommen oder Wohnort rekonstruieren, und die Muster in den historischen Labels bleiben erhalten. Weglassen verhindert nur, dass man den Effekt *sieht*.

Quellen. [1, Kapitel 7, Pattern 30], [62], [72], [73], [74], [75], [76], [77], [78], [79]

11 ML-Systeme in Produktion

Ein Notebook beantwortet die Frage, ob sich aus diesen Daten ein brauchbares Modell trainieren lässt. Ein produktives ML-System beantwortet eine andere: Liefert das Modell in drei Monaten, nach zwei Änderungen an den Datenquellen, einem Teamwechsel und einem Serverausfall noch brauchbare Vorhersagen, und merken wir es, wenn nicht? Für einen Software Engineer ist das vertrautes Gelände. Man kennt Deployments, Tests, Monitoring und Rollbacks. Neu ist, dass in einem ML-System nicht nur Code, sondern auch Daten das Verhalten bestimmen, und dass Fehler leise auftreten: Das System wirft keine Exception, es liefert plausibel aussehende, falsche Zahlen.

Die Kapitel zu den Patterns haben diese einzeln behandelt. Dieses Kapitel setzt sie zusammen. Es beschreibt, was ein ML-System über das Modell hinaus enthält, wie reif der Betrieb sein kann (die Reifegrade des MLOps), welche Werkzeuge und Tests es dafür gibt und wo die Patterns in diesem Gesamtbild stehen. Wir bleiben beim Lieferdienst aus Pattern 16, damit alle Beispiele auf derselben Datengrundlage laufen. Wo dieses Kapitel ein Pattern nur erwähnt, steht die Nummer in Klammern; die Erklärung findet sich im jeweiligen Kapitel.

> **LERNZIELE**
> Nach diesem Kapitel können Sie erklären, warum der Modellcode nur einen kleinen Teil eines ML-Systems ausmacht und welche technische Schuld daraus folgt, die drei MLOps-Reifegrade nach Google Cloud sowie CI, CD und CT unterscheiden, Experimente und Modelle mit MLflow ablegen, Daten mit einem Schema prüfen, die vier Testkategorien des ML Test Score anwenden, Drift mit PSI und KS-Test erkennen und deren Grenzen benennen, Shadow, Canary und A/B-Test unterscheiden und jedes der 30 Patterns einer Station im Betrieb zuordnen.

11.1 Vom Notebook zum System

In klassischer Software legt der Code das Verhalten fest. Bei einem ML-System entsteht es aus drei Dingen zugleich: dem Trainingscode, den Trainingsdaten und der Konfiguration (Hyperparameter, Zufallsstartwert, Merkmalsauswahl). Ändert sich eines davon, ändert sich das Modell, auch wenn niemand eine Zeile Code anfasst: Es genügt, dass eine Datenquelle ihr Format leicht ändert. Man kann das Modell als Build-Artefakt verstehen, dessen Quelltext zu großen Teilen aus Daten besteht, und diese Quelle ändert sich ohne Commit.

Daraus folgen drei Eigenheiten. Erstens gibt es keine Spezifikation im üblichen Sinn: Ein Test kann nicht „die Lieferzeit ist 27 Minuten“ prüfen, nur statistische Erwartungen („der mittlere Fehler liegt unter 2,5 Minuten“). Zweitens altert ein Modell, ohne dass sich an ihm etwas ändert, weil sich die Welt ändert, aus der seine Eingaben kommen. Drittens sind die Fehler leise: Ein Modell mit falschen Eingaben rechnet trotzdem und gibt eine Zahl zurück. Das Risiko schleichender Wartungsprobleme ist entsprechend hoch, und genau das hat eine Gruppe bei Google 2015 systematisch beschrieben.

11.2 Hidden Technical Debt

Die Aussage des Papiers

Der Begriff **Technical Debt** stammt von Ward Cunningham [80]: Schnell geschriebener Code ist wie ein Kredit, er bringt sofort Tempo und kostet dauerhaft Zinsen in Form von Mehraufwand bei jeder Änderung, bis man die Schuld tilgt. D. Sculley *u. a.* [11] argumentieren, dass ML-Systeme diese Schuld besonders leicht anhäufen, weil sie alle Wartungsprobleme normaler Software haben und zusätzlich solche, die erst auf Ebene des Gesamtsystems sichtbar werden. Das Papier ist eine Sammlung von Beobachtungen aus

der Praxis bei Google, keine Studie mit Messreihen; seine Stärke ist ein präzises Vokabular für Probleme, die jeder kennt, der ein ML-System betrieben hat.

Das berühmteste Bild des Papiers zeigt den ML-Code als kleinen Kasten in der Mitte, umgeben von einer weitaus größeren Infrastruktur (Abbildung 76). Die Autoren schätzen, dass in reifen Systemen höchstens fünf Prozent des Codes eigentlicher ML-Code sind und mindestens 95 Prozent Verbindungscode (**Glue Code**). Das ist eine Schätzung aus Erfahrung, keine Messung, aber die Größenordnung deckt sich mit der Praxis: Das Modell ist oft ein `fit()`-Aufruf, der Rest ist Datenbeschaffung, Prüfung, Auslieferung und Überwachung.

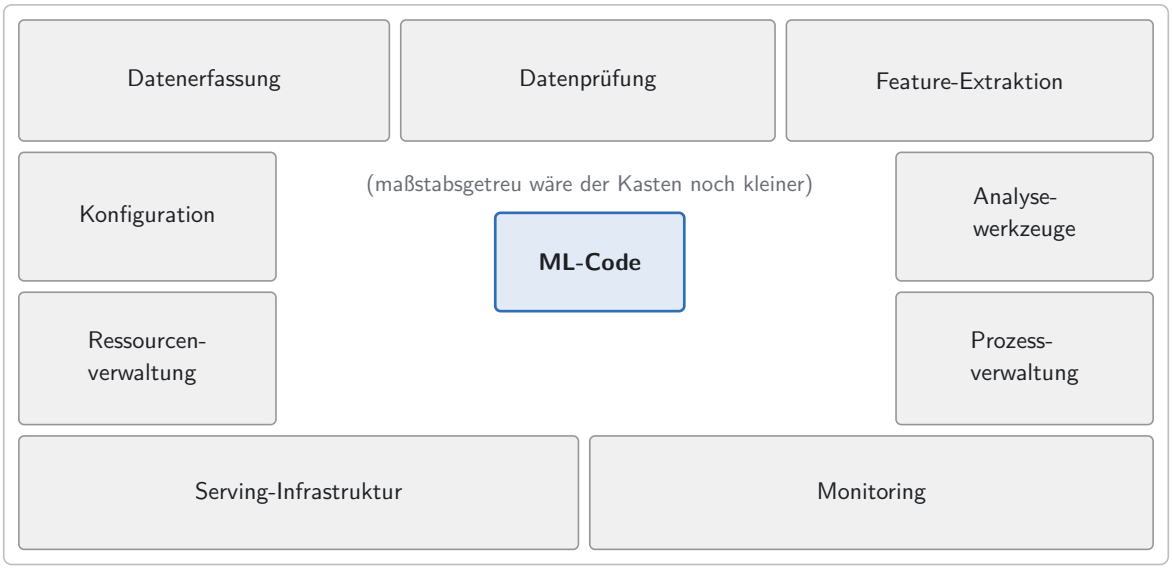


Abbildung 76: Ein ML-System besteht überwiegend aus Umgebung. Die Darstellung folgt Abbildung 1 bei [11]; die Beschriftungen sind übersetzt und frei gruppiert.

Entanglement und das CACE-Prinzip

Die erste ML-eigene Form der Schuld heißt **Entanglement**, Verflechtung. In einem klassischen System kann man eine Komponente austauschen, solange ihre Schnittstelle gleich bleibt. Ein Modell hat diese Isolation nicht, denn es lernt die Merkmale **gemeinsam**. Die Autoren fassen das im Prinzip **CACE** zusammen: Changing Anything Changes Everything. Fügt man ein Merkmal hinzu, entfernt eines oder verändert die Verteilung eines Merkmals, kann sich das Verhalten bei allen übrigen verschieben. Das gilt nicht nur für Eingaben, sondern auch für Hyperparameter, Stichprobenverfahren und Abbruchkriterien.

Das lässt sich im Kleinen zeigen. Wir ergänzen die Lieferdaten um ein Merkmal `routing_min`, die Fahrzeitschätzung eines externen Routing-Dienstes. Der Dienst kennt die tatsächliche Strecke samt Umweg, die Luftlinie `distanz_km` nicht. Beide Merkmale sind also stark korreliert, und das Modell verteilt das Gewicht auf sie:

```
train, test = daten(5000, seed=1), daten(2000, seed=2)
ohne = LinearRegression().fit(train[BASIS], train["lieferzeit_min"])
mit_cols = BASIS + ["routing_min"]
mit = LinearRegression().fit(train[mit_cols], train["lieferzeit_min"])

print("Gewichte (Minuten pro km bzw. pro Routing-Minute)")
print(f"  ohne routing_min: distanz_km = {ohne.coef_[0]:.2f}")
print(f"  mit  routing_min: distanz_km = {mit.coef_[0]:.2f}, "
      f"routing_min = {mit.coef_[3]:.2f}")
```

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy python cace_abhaengigkeiten.py
Gewichte (Minuten pro km bzw. pro Routing-Minute)
  ohne routing_min: distanz_km = 2.88
  mit  routing_min: distanz_km = 0.27, routing_min = 0.92
```

Ohne das neue Merkmal gewichtet das Modell einen Kilometer mit 2,88 Minuten, mit dem neuen nur noch mit 0,27. Das Gewicht der Distanz ist nicht „falsch“ geworden, es hat sich verschoben, weil ein anderes Merkmal einen Teil der Information übernimmt. Wer in diesem Modell die Bedeutung von `distanz_km` erklären will, muss wissen, was sonst noch im Modell steckt. Die Gegenmaßnahmen im Papier sind pragmatisch: Modelle isolieren, etwa als Ensemble getrennt trainierter Teilmodelle, und Veränderungen im Vorhersageverhalten sichtbar machen, bevor eine Änderung live geht. Beides setzt Werkzeuge voraus, die wir weiter unten besprechen.

Datenabhängigkeiten kosten mehr als Codeabhängigkeiten

Die zweite Form betrifft die Eingabedaten. Das Papier hält fest, dass Datenabhängigkeiten ähnlich teuer sind wie Codeabhängigkeiten, aber schwerer zu erkennen, weil Compiler und Linter sie nicht prüfen.

Instabile Datenabhängigkeiten entstehen, wenn ein Eingabesignal von einem System kommt, das sich ändert, typischerweise von einem anderen Modell, das sein Team verbessert, ohne uns zu informieren. Wir nehmen an, dass der Routing-Dienst nach einem Update 25 Prozent längere Fahrzeiten schätzt:

```
# Derselbe Dienst nach einem Update: Das neue Verkehrsmodell schätzt 25 % mehr.
test_update = daten(2000, seed=2, routing_faktor=1.25)


def mae(modell, merkmale, df):
    return mean_absolute_error(df["lieferzeit_min"], modell.predict(df[merkmale]))


print("\nMAE auf Testdaten (Minuten)")
varianten = [("ohne routing_min", ohne, BASIS),
             ("mit  routing_min", mit, mit_cols)]
for name, modell, cols in varianten:
    print(f"  {name}: vor dem Update {mae(modell, cols, test):.2f}, "
          f"nach dem Update {mae(modell, cols, test_update):.2f}")
```

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy python cace_abhaengigkeiten.py
MAE auf Testdaten (Minuten)
  ohne routing_min: vor dem Update 3.71, nach dem Update 3.71
  mit  routing_min: vor dem Update 2.63, nach dem Update 5.63
```

Das Modell mit dem externen Merkmal war vor dem Update das bessere (2,63 gegen 3,71 Minuten) und ist danach das deutlich schlechtere (5,63). Niemand hat das Modell geändert, kein Test schlägt an, die Eingaben sind formal gültig. Die Antwort des Software Engineerings lautet: Abhängigkeiten pinnen. Für Datensignale heißt das, eine versionierte Kopie zu verwenden (Pattern 26, Feature Store) und die Verteilung zu überwachen. **Nicht genutzte Datenabhängigkeiten** sind das Gegenstück: Merkmale, die kaum etwas zur Genauigkeit beitragen, aber Abhängigkeiten schaffen, etwa Altlasten früherer Modellversionen. Das Papier empfiehlt, durch Weglassen einzelner Merkmale regelmäßig zu prüfen, ob jedes seinen Platz verdient.

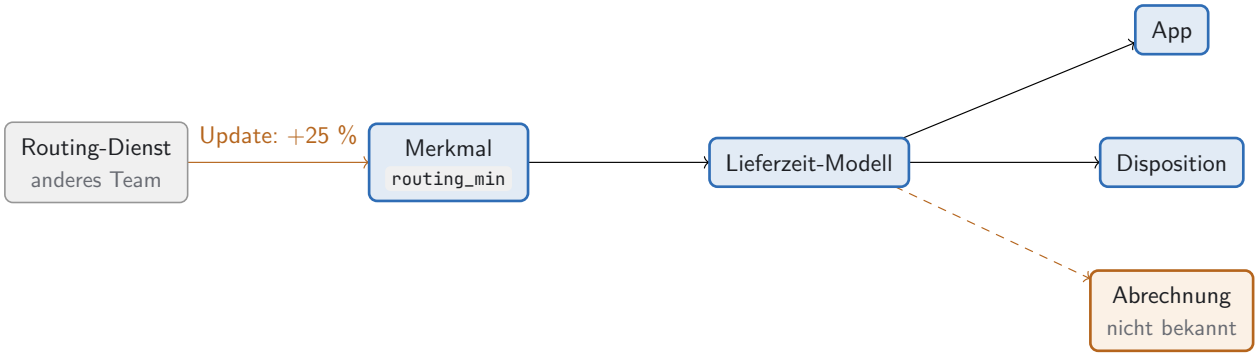


Abbildung 77: Zwei Formen von Schuld in einem Bild. Links die instabile Datenabhängigkeit: Der Dienst ändert sich, das Modell bemerkt es nicht. Rechts der undeklarierte Konsument: Die Abrechnung liest die Vorhersagen, ohne dass das Modellteam davon weiß.

Das Bild zeigt noch eine dritte Form, die **undeklarierten Konsumenten**: Vorhersagen sind leicht zugänglich, irgendwann liest sie ein System, von dem das Modellteam nichts weiß, und jede Modelländerung wird zur Änderung für Unbekannte. Es ist die undokumentierte Nutzung einer API, und die Antwort ist dieselbe: eine definierte Schnittstelle mit Authentifizierung und Protokollierung. Schlüssel in den Vorhersagen (Pattern 20) machen sichtbar, wer was abgefragt hat.

Glue Code, Pipeline Jungles und weitere Anti-Patterns

Glue Code ist der Code, der Daten zwischen allgemeinen Bibliotheken und dem eigenen System hin- und herschiebt: Spalten umbenennen, Formate umwandeln, Ergebnisse umpacken. Das Papier rät, solche Pakete hinter einer einheitlichen Schnittstelle zu kapseln; das entspricht dem bekannten Adapter-Muster [2]. **Pipeline Jungles** sind der Spezialfall in der Datenvorbereitung: Mit jeder neuen Quelle und jedem neuen Merkmal kommt ein Schritt hinzu, bis niemand mehr das Ganze versteht und Tests kaum möglich sind. Das Papier empfiehlt, Datensammlung und Merkmalsberechnung als Ganzes zu entwerfen; das Pattern 25 (Workflow Pipeline) macht aus einer losen Folge von Skripten einen benannten, versionierbaren, testbaren Ablauf. Weitere Formen sind tote experimentelle Codepfade, Konfigurationsschuld, Rückkopplungsschleifen (das Modell beeinflusst die Daten, auf denen es später trainiert wird) und fest einprogrammierte Schwellenwerte in einer Welt, in der sich Verteilungen ändern. Tabelle 10 ordnet die Formen ihren Gegenstücken zu.

Tabelle 10: Formen technischer Schuld nach [11], ihre Gegenstücke im Software Engineering und die Patterns, die sie mildern. Die Zuordnung der Patterns ist eine Einschätzung dieses Handbuchs, keine Aussage des Papiers.

Schuldform	Entspricht im klassischen Software Engineering	Patterns
Entanglement (CACE)	Starke Kopplung über gemeinsamen Zustand: eine Änderung wirkt überall	7, 18
Undeklarierte Konsumenten	Undokumentierte Nutzer einer API	20, 27
Instabile Datenabhängigkeit	Ungepinnte Version einer Bibliothek	21, 26
Glue Code	Wildwuchs aus Adaptern ohne gemeinsame Schnittstelle	16, 21
Pipeline Jungle	Gewachsene Build-Skripte ohne Struktur und Tests	25, 26
Tote Codepfade	Feature-Flags, die nie entfernt wurden	25, 27
Rückkopplung	Regelkreis, dessen Ausgang den eigenen Eingang verändert	18, 30

11.3 MLOps und seine Reifegrade

CI, CD und CT

MLOps bezeichnet die Übertragung der DevOps-Praxis auf ML-Systeme: Entwicklung und Betrieb sollen nicht zwei getrennte Welten sein, und alles, was sich automatisieren lässt, wird automatisiert. Den bekanntesten Leitfaden dazu hat Google Cloud veröffentlicht [81]. Er greift die drei Begriffe aus der Continuous-Delivery-Welt [82] auf und erweitert sie um einen vierten Gedanken, der in klassischer Software keine Entsprechung hat.

Continuous Integration (CI) heißt bei ML nicht nur, Code zu testen und zu bauen, sondern auch Daten, Schemata und Modelle. **Continuous Delivery (CD)** betrifft nicht mehr ein einzelnes Softwarepaket, sondern ein System: die Trainingspipeline, die ihrerseits ein Modell als Dienst bereitstellt. Neu ist **Continuous Training (CT)**, das automatische Neutrainieren und Bereitstellen von Modellen. Das hat kein Gegenstück in normaler Software, denn ein Webshop wird nicht dadurch schlechter, dass die Zeit vergeht. Ein Modell schon. Ausgelöst wird CT durch einen Zeitplan, neue Daten, nachlassende Qualität oder erkannten Drift.

Level 0, 1 und 2

Der Leitfaden unterscheidet drei Reifegrade. Sie sind keine Pflichtstufen, sondern beschreiben, wie viel des Weges vom Notebook zum betriebenen System automatisiert ist (Abbildung 78).

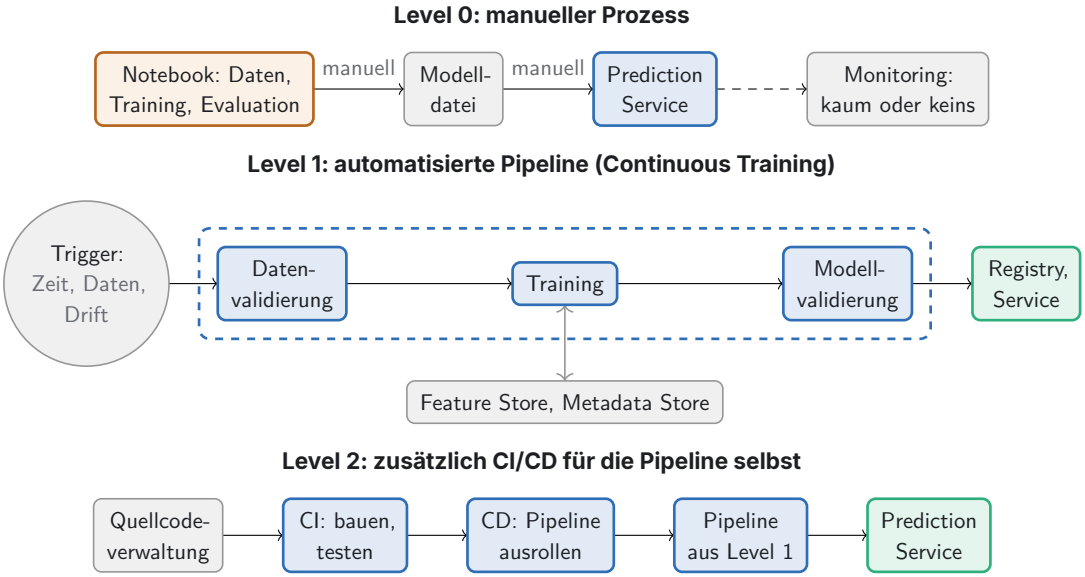


Abbildung 78: Die drei Reifegrade nach [81], vereinfacht. Orange: manuell; blau gestrichelt: automatisch wiederholte Pipeline; grün: bereitgestelltes Ergebnis. In Level 2 läuft die Pipeline aus Level 1 weiterhin und wird vom Monitoring neu angestoßen.

Level 0 ist der manuelle Prozess. Analyse, Aufbereitung, Training, Evaluation und Auslieferung laufen als einzelne, oft interaktive Schritte. Ein Data Scientist übergibt eine Modelldatei an das Engineering, das sie als Dienst bereitstellt. Releases sind selten (im Leitfaden: wenige pro Jahr), CI und CD gibt es nicht, und ausgeliefert wird nur das Modell, nicht das System. Das genügt, solange sich Modelle selten ändern. Das Risiko liegt in der Übergabe: Notebook und Dienst sind zwei Implementierungen, dazwischen entsteht Training-Serving Skew (Pattern 21 verhindert ihn), und das Monitoring fehlt häufig.

Level 1 automatisiert die ML-Pipeline für kontinuierliches Training. Der Unterschied: Ausgeliefert wird nicht mehr ein Modell, sondern die Trainingspipeline, die wiederkehrend läuft und das Modell selbst bereitstellt. Dazu kommen eine **Datenvalidierung** (ändert sich das Schema, hält die Pipeline an; ändern sich die Werte, stößt sie ein Neutrainieren an), eine **Modellvalidierung** (das neue Modell wird offline gegen das aktuelle und auf Teilmengen geprüft, online per Canary oder A/B), ein optionaler **Feature Store** gegen Skew (Pattern 26) und ein **Metadata Store**, der festhält, welche Pipelineversion mit welchen Parametern lief und wo das Vorgängermodell liegt. Ausgelöst wird auf Abruf, nach Zeitplan, bei neuen Daten, bei sinkender Qualität oder bei erkanntem Drift.

Level 2 ergänzt eine CI/CD-Strecke für die Pipeline selbst. Level 1 setzt eine stabile Pipeline voraus, Level 2 erlaubt, auch sie schnell und zuverlässig zu ändern: neue Merkmale, Architekturen, Hyperparameter. Die Auslieferung durchläuft Stufen: automatisch in der Testumgebung, halbautomatisch in der Vorproduktion, in Produktion manuell ausgelöst nach mehreren erfolgreichen Läufen. Manuell bleiben laut Leitfaden die Analyse der Daten und des Modells.

Die Reifegrade sind ein Orientierungsrahmen, kein Ziel um jeden Preis. Ein Modell, das einmal im Jahr neu trainiert wird, rechtfertigt keinen Orchestrator; eines, dessen Qualität jede Woche sinkt, schon. Die Frage lautet: Wie schnell veraltet unser Modell, und was kostet jede Woche Verzögerung? Pattern 18 liefert die Messung. In Patterns ausgedrückt kommt Level 0 mit der Auslieferung aus (16, 17, 20). Level 1 braucht zusätzlich Pattern 25 (Workflow Pipeline), 21 (Transform), 22 (Repeatable Splitting) für vergleichbare Läufe, 27 (Model Versioning) für die Registry und 18 als Auslöser. Level 2 ergänzt Freigabeprüfungen wie 28 (Heuristic Benchmark) als Mindestschwelle und 30 (Fairness Lens) als Kriterium sowie 15 (Hyperparameter Tuning) als automatisierten Schritt.

11.4 Experiment Tracking und Model Registry

Wer vierzig Trainingsläufe gemacht hat, fragt sich irgendwann, welche Parameter, Daten und welcher Code zu dem Modell mit dem MAE (mittlerer absoluter Fehler in Minuten) von 2,31 gehörten. Ein **Experiment Tracker** beantwortet das, indem er jeden Lauf mit Parametern, Kennzahlen und Artefakten festhält.

Eine **Model Registry** verwaltet die auslieferbaren Modelle als versionierte Objekte, vergleichbar mit einem Paketregister.

MLflow [60], [83] ist dafür das verbreitetste quelloffene Werkzeug. **Tracking** speichert Läufe (Runs) eines Experiments mit Parametern, Metriken und Artefakten. **Models** beschreiben ein Modell in einem einheitlichen Format mit Flavors (etwa `sklearn`), sodass Werkzeuge es laden können, ohne das Trainingsframework zu kennen. Die **Model Registry** verwaltet benannte Modelle mit Versionen, Tags und Aliasen; statt der früheren Stufen „Staging“ und „Production“ empfiehlt die Dokumentation inzwischen Aliase. Das Beispiel trainiert drei Varianten, protokolliert sie, registriert die beste und lädt sie über einen Alias, lokal mit SQLite als Datenbank. Gezeigt sind die Läufe und die Registry; die Abfrage der Läufe steht in der Datei.

```
# 1) Drei Läufe mit verschiedenen Hyperparametern, je Lauf Parameter und Modell.
modell_uris = {}
for n_baeume, tiefe in [(50, 2), (100, 3), (300, 3)]:
    with mlflow.start_run(run_name=f"gbr-{n_baeume}-{tiefe}") as lauf:
        modell = modell_trainieren(train, n_estimators=n_baeume, max_depth=tiefe)
        vorhersage = modell.predict(test[MERKMALE])
        mae = mean_absolute_error(test["lieferzeit_min"], vorhersage)
        mlflow.log_params({"n_estimators": n_baeume, "max_depth": tiefe})
        mlflow.log_metric("mae_test", mae)
        # MLflow 3 speichert mit skops und lehnt unsere eigene Funktion ab.
        # Für das eigene, vertrauenswürdige Modell: cloudpickle.
        info = mlflow.sklearn.log_model(
            modell, name="modell", serialization_format="cloudpickle",
            pip_requirements=["scikit-learn", "pandas"],  # spart die Ermittlung
        )
        modell_uris[lauf.info.run_id] = info.model_uri
```

```
# 3) Das beste Modell wird als Version registriert, Alias "champion".
bester = laeufe.iloc[0]
version = mlflow.register_model(modell_uris[bester["run_id"]], "lieferzeit")
client = MlflowClient()
client.set_registered_model_alias("lieferzeit", "champion", version.version)
print(f"\nRegistriert: lieferzeit, Version {version.version}, Alias 'champion'")

# 4) Der Serving-Code kennt nur den Alias, nie die Versionsnummer.
champion = mlflow.sklearn.load_model("models:/lieferzeit@champion")
beispiel = pd.DataFrame({"distanz_km": [3.0], "artikel": [4],
                         "stunde": [19], "regen": [1]})
minuten = champion.predict(beispiel)[0]
print(f"Vorhersage für 3 km, 4 Artikel, 19 Uhr, Regen: {minuten:.1f} min")
```

Terminal

```
$ uv run --with mlflow --with scikit-learn --with pandas --with numpy python mlflow_tracking.py
mlflow.runName n_estimators max_depth  mae_test
     gbr-100-3          100         3     2.032
     gbr-300-3          300         3     2.053
      gbr-50-2           50         2     2.258

Registriert: lieferzeit, Version 1, Alias 'champion'
Vorhersage für 3 km, 4 Artikel, 19 Uhr, Regen: 27.7 min
```

Die Tabelle ist nach dem Fehler sortiert. Das einfachste Modell (50 Bäume, Tiefe 2) ist mit 2,26 Minuten das schlechteste, das mit 100 Bäumen und Tiefe 3 mit 2,03 das beste. Dass 300 Bäume nicht besser sind (2,05), ist eine nützliche Beobachtung für sich: Mehr Aufwand zahlt sich nicht automatisch aus. Das beste Modell wurde als Version 1 von `lieferzeit` registriert, und die letzte Zeile lädt es über den Alias und liefert eine Vorhersage. Weil Parameter, Kennzahl und Modell zusammen gespeichert sind, lässt sich die Wahl später nachvollziehen.

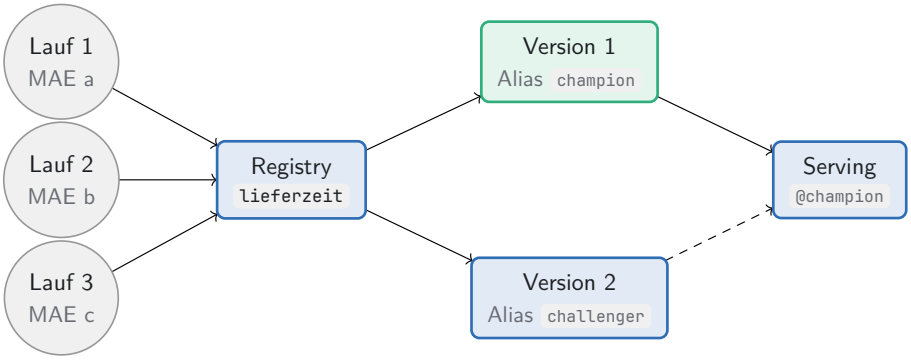


Abbildung 79: Läufe, Registry und Serving. Das Serving lädt nur den Alias. Ein neues Modell geht live, indem der Alias umgehängt wird (gestrichelt); ein Rollback ist dieselbe Operation in die andere Richtung. Versionen und Aliase sind ein Beispiel.

Der Alias entkoppelt Serving und Training, ganz wie ein Symlink bei einer Software-Auslieferung. Das ist die konkrete Form von Pattern 27 (Model Versioning): Versionen sind unveränderlich und eindeutig, die Frage „welche Version läuft gerade?“ hat genau eine Antwort, und ein Rollback verlangt kein neues Training (im ML Test Score ist das Infrastruktur-Test 7).

ACHTUNG

Das Laden von Modelldateien ist nicht harmlos. `pickle` und Verwandte (`joblib` , `cloudpickle`) können beim Laden beliebigen Code ausführen. MLflow 3 nutzt für sklearn-Modelle deshalb standardmäßig skops, das nur ausdrücklich vertraute Typen lädt, und hat unser Modell (eigene Funktion, Baumstrukturen) abgelehnt; im Beispiel wählen wir für das eigene Modell bewusst `cloudpickle` . Man lädt nie Modelldateien aus unbekannter Quelle.

11.5 Datenvalidierung

Das Modell ist nur so zuverlässig wie seine Daten, und Daten ändern sich ohne Ankündigung. Eine **Datenvalidierung** prüft, ob ein Datenbestand die Erwartungen erfüllt: Spalten, Typen, Wertebereiche, Anteile fehlender Werte, Verteilungen. Sie ist das Gegenstück zur Eingabevalidierung einer API, prüft aber meist ganze Batches und kann daher auch statistische Aussagen prüfen („im Mittel liegt die Distanz zwischen 3 und 12 km“). Geprüft wird an drei Stellen (Abbildung 80): am Eingang der Pipeline gegen Fehler der Datenquelle, vor dem Training, damit kein Modell aus fehlerhaften Daten lernt, und im Serving, bevor fehlerhafte Anfragen das Modell erreichen.

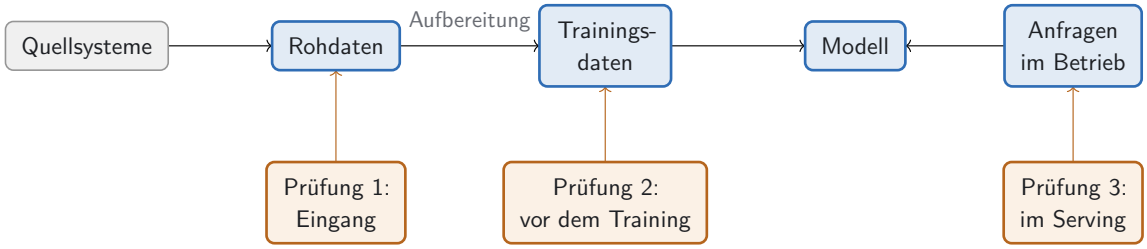


Abbildung 80: Drei Stellen für Datenvalidierung. Alle drei verwenden dasselbe Schema als gemeinsamen Vertrag.

Mit pandera [84], einer Python-Bibliothek zur Validierung von DataFrames, sieht ein solcher Vertrag für unsere Bestelldaten so aus:

```python
class Bestellungen(pa.DataFrameModel):
    """Der Vertrag zwischen Datenerfassung und Modell."""
    distanz_km: Series[float] = pa.Field(gt=0, le=30)
    artikel: Series[int] = pa.Field(ge=1, le=50)
    stunde: Series[int] = pa.Field(ge=0, le=23)
    regen: Series[int] = pa.Field(isin=[0, 1])

    class Config:
        strict = True  # unbekannte Spalten sind ein Fehler
        coerce = False  # keine stille Umwandlung

    @pa.dataframe_check
    def mittlere_distanz_plausibel(cls, df) -> bool:
        """Statistische Prüfung über den ganzen Batch, nicht über Zeilen."""
        return 3.0 <= df["distanz_km"].mean() <= 12.0
```

Das Schema steht in einer Klasse und ist damit versionierbar, reviewbar und testbar. `strict` lehnt unbekannte Spalten ab, `coerce = False` verbietet stille Typumwandlung, `lazy=True` sammelt alle Verstöße. Wir prüfen einen sauberen Batch, einen mit vier eingebauten Fehlern und einen, in dem das Quellsystem plötzlich Meilen statt Kilometer liefert:

```python
def pruefen(name, df):
    try:
        Bestellungen.validate(df, lazy=True)  # lazy: alle Verstöße sammeln
        print(f"{name}: {len(df)} Zeilen, gültig")
    except pa.errors.SchemaErrors as fehler:
        faelle = fehler.failure_cases[["column", "check", "failure_case"]]
        print(f"{name}: {len(faelle)} Verstöße")
        print(faelle.to_string(index=False))
```

Terminal

```
$ uv run --with pandera --with pandas --with numpy --with scikit-learn python daten_validierung.py
Batch A (sauber): 500 Zeilen, gültig

Batch B (kaputt): 4 Verstöße
      column                    check failure_case
Bestellungen          column_in_schema     kunde_id
  distanz_km           greater_than(0)         -1.0
      stunde less_than_or_equal_to(23)           24
       regen              isin([0, 1])            2

Batch C (Meilen statt km): 500 Zeilen, gültig
```

Batch B wird vollständig erkannt: negative Distanz, Stunde 24, unzulässige Regenkategorie und unerwartete Spalte. Batch C besteht die Prüfung, obwohl er falsch ist. Meilen sind ebenso positive Zahlen unter 30, und die mittlere Distanz fällt von 7,8 auf 4,8, bleibt also im erlaubten Bereich. Das ist die wichtigste Lehre dieses Abschnitts: Ein Schema prüft, was man vorher als Regel aufschreiben konnte. Fehler, die jede Regel einhalten und nur die Verteilung verschieben, findet erst der Vergleich mit einer Referenzverteilung, also das Monitoring.

Einordnung. TensorFlow Data Validation (TFDV), die Validierungskomponente von TFX, folgt demselben Gedanken [59], [85]: Es berechnet Statistiken, leitet daraus ein Schema ab, das ein Mensch nachbessert, prüft neue Daten dagegen und vergleicht Verteilungen zwischen Training und Serving (Skew) sowie zwischen aufeinanderfolgenden Tagen (Drift). Great Expectations [86] formuliert Erwartungen als deklarative Aussagen in Suiten, prüft Datenquellen (auch SQL-Datenbanken) gegen sie und erzeugt lesbare Berichte. pandera ist leichtgewichtig und passt, wo ohnehin pandas läuft. Wichtiger als das Werkzeug ist, dass der Vertrag existiert, versioniert wird und im Betrieb tatsächlich blockiert.

11.6 Testen von ML-Systemen

Wie testet man ein System, dessen richtiges Verhalten man nicht aufschreiben kann? E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55] haben bei Google dazu den **ML Test Score** veröffentlicht: 28 konkrete Tests und Überwachungsaufgaben in vier Kategorien zu je sieben (Features und Daten, Modellentwicklung, ML-Infrastruktur, Monitoring). Die Autoren haben sie mit den Betreibern von 36 Systemen bei Google durchgesprochen. Tabelle 11 gibt sie in eigenen Worten wieder.

Die Punktevergabe ist einfach: ein halber Punkt, wenn ein Test manuell durchgeführt und das Ergebnis dokumentiert und verteilt wird, ein ganzer, wenn ein System ihn wiederholt automatisch ausführt. Die Punkte werden je Kategorie addiert, und die Gesamtwertung ist das **Minimum** der vier Summen. Das erzwingt Ausgewogenheit: Wer die Daten perfekt, die Infrastruktur gar nicht prüft, bekommt null. Ein erfundenes Beispiel: 4,5 Punkte bei den Daten, 3,0 beim Modell, 2,0 bei der Infrastruktur und 1,5 beim Monitoring ergeben einen Score von 1,5. Nach der Interpretationstabelle der Autoren heißt ein Wert über 1 bis 2 „erster Schritt zur Produktionsreife, weitere Investitionen nötig“, und ab über 3 beginnt „starke Automatisierung, geeignet für unternehmenskritische Systeme“.

Tabelle 11: Die 28 Tests des ML Test Score in eigener Übersetzung, nach [55]. Fett: im Code dieses Kapitels umgesetzt.

Features und Daten	Modellentwicklung	ML-Infrastruktur	Monitoring
1 Erwartungen an Features stehen in einem Schema.	1 Modellspezifikation wird per Review geprüft und eingecheckt.	**1 Training ist reproduzierbar.**	1 Änderungen an Abhängigkeiten lösen Benachrichtigungen aus.
2 Jedes Feature ist nützlich.	2 Offline-Kennzahlen korrelieren mit der Wirkung online.	2 Modellspezifikation ist unit-getestet.	**2 Invarianten der Eingabedaten gelten.**
3 Kein Feature ist zu teuer.	3 Alle Hyperparameter sind getunt.	3 Pipeline ist integrationsgetestet.	3 Training und Serving laufen nicht auseinander.
4 Features erfüllen übergeordnete Vorgaben.	4 Der Einfluss veralteter Modelle ist bekannt.	4 Modellqualität wird vor dem Serving geprüft.	4 Modelle sind nicht zu alt.
5 Datenschutz in der Pipeline ist angemessen.	**5 Ein einfacheres Modell ist nicht besser.**	5 Das Modell ist debugbar.	**5 Modelle sind numerisch stabil.**
6 Neue Features lassen sich schnell ergänzen.	**6 Qualität auf wichtigen Teilmengen reicht.**	6 Modelle laufen vor dem Serving als Canary.	6 Rechenleistung hat sich nicht verschlechtert.
7 Code zur Feature-Berechnung ist getestet.	7 Inklusion (Fairness) ist getestet.	7 Rollback auf frühere Versionen ist möglich.	**7 Vorhersagequalität hat sich nicht verschlechtert.**

Vieles ist bekanntes Handwerk an neuer Stelle. **Unit-Tests** gehören an den Feature-Code, denn ein Fehler dort ist im Modell kaum zu erkennen (zum Beispiel: Gehört 20 Uhr zur Abendspitze?). **Datentests** prüfen das Schema. **Modelltests** prüfen Verhalten statt Ausgaben: Der CheckList-Ansatz von M. T. Ribeiro, T. Wu, C. Guestrin, und S. Singh [87] für Sprachmodelle, der sich übertragen lässt, unterscheidet Mindestfunktionalität, Invarianz (irrelevante Änderungen ändern die Ausgabe nicht) und Richtungserwartung (eine Änderung bewegt die Ausgabe in eine bekannte Richtung). **Infrastrukturtests** prüfen, ob Training reproduzierbar ist und das ausgelieferte Artefakt dasselbe tut wie das trainierte Modell. Die Datei `test_modell.py` setzt zehn Tests für unser Lieferzeit-Modell um; zwei im Ausschnitt, in normalem pytest-Code:

```
@pytest.mark.parametrize("name,maske", [
    ("Regen", lambda d: d["regen"] == 1),
    ("Abendspitze", lambda d: d["stunde"].between(18, 20)),
    ("weite Strecken", lambda d: d["distanz_km"] > 10),
])
def test_qualitaet_je_teilmenge(modell, daten, name, maske):
    """Die Gesamtkennzahl darf keine schlechte Teilgruppe verstecken."""
    _, test = daten
    assert mae(modell, test[maske(test)]) <= 1.5 * mae(modell, test)
```

```python
def test_extrapolation_distanz(modell):
    """Jeder Kilometer kostet Zeit, auch jenseits der Trainingsdaten."""
    beispiel = pd.DataFrame({"distanz_km": [15.0, 25.0], "artikel": 5,
                             "stunde": 12, "regen": 0})
    zeit_15, zeit_25 = modell.predict(beispiel)
    assert zeit_25 - zeit_15 > 10, (f"15 km: {zeit_15:.1f} min, "
                                    f"25 km: {zeit_25:.1f} min")
```

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy --with pytest pytest -q --
tb=line test_modell.py
......F...                                                              [100%]
...
E   AssertionError: 15 km: 43.3 min, 25 km: 43.3 min
...
FAILED test_modell.py::test_extrapolation_distanz - AssertionError: 15 km: 43...
```

Neun Tests bestehen, einer schlägt fehl, und das ist beabsichtigt. `test_extrapolation_distanz` formuliert eine fachliche Erwartung: Jeder zusätzliche Kilometer kostet Zeit, auch über die Trainingsdaten hinaus. Das Boosting-Modell hat Distanzen nur bis 15 km gesehen und liefert für 25 km exakt dieselbe Vorhersage wie für 15 km (43,3 Minuten): Baumverfahren extrapolieren nicht, sie halten jenseits des Trainingsbereichs den letzten Wert. Der Test macht eine Eigenschaft sichtbar, die man sonst erst bemerkt, wenn der Lieferdienst sein Gebiet erweitert. Im Alltag löscht man ihn nicht, sondern markiert ihn als bekanntes Problem oder ergänzt ein lineares Teilmodell für lange Strecken. Der Test auf Teilmengen stellt sicher, dass eine gute Gesamtkennzahl keine schlechte Untergruppe versteckt, und der Vergleich mit einer Faustformel ist im Kern Pattern 28 (Heuristic Benchmark) als automatisierter Wächter.

11.7 Monitoring

Drei Ebenen

Monitoring fragt auf mehreren Ebenen, ob das System noch tut, was es soll. Die erste kennt jeder Software Engineer: Antwortet der Dienst, wie schnell, mit wie vielen Fehlern? Die übrigen sind ML-spezifisch und unterscheiden sich vor allem darin, **wann** sie ein Signal liefern (Tabelle 12).

Tabelle 12: Vier Ebenen des Monitorings. Nur die letzte braucht die tatsächlichen Ergebnisse (Labels), und die liegen oft erst spät vor.

Ebene	Frage und typische Kennzahlen	Labels nötig?
System	Läuft der Dienst? Latenz, Fehlerrate, Durchsatz, Speicher	nein
Daten	Sehen die Eingaben aus wie im Training? Schemaverstöße, fehlende Werte, Verteilungsvergleich je Merkmal (PSI, KS-Test, siehe unten)	nein
Vorhersagen	Verhält sich das Modell wie gewohnt? Verteilung der Vorhersagen, PSI der Vorhersage	nein
Qualität	Stimmen die Vorhersagen? MAE, Genauigkeit, auch je Teilmenge	ja, verzögert

Der Grund für die Staffelung ist das Label-Problem: Ob die Lieferzeitvorhersage stimmte, weiß man erst nach der Lieferung, bei einem Kreditmodell nach Monaten oder nie. Die Daten- und Vorhersageebene liefern sofort Signale, wenn auch indirekte: Sie sagen nicht, dass das Modell schlechter ist, sondern dass sich etwas geändert hat, das es sein könnte. Pattern 18 (Continued Model Evaluation) führt beide Wege zusammen und zeigt in einer Simulation über 24 Wochen, dass Eingabe-Alarm und Qualitätsverlust unabhängig voneinander auftreten können. Dort stehen auch die Begriffe: Beim **Covariate Shift** ändert sich die Verteilung der Eingaben, beim **Concept Drift** der Zusammenhang mit dem Ergebnis [12], [88]. Eingabetests erkennen nur den ersten Fall.

PSI und KS-Test

Zwei Verfahren vergleichen eine Verteilung mit einer Referenz. Der **Population Stability Index** teilt den Wertebereich in Klassen (Dezile der Referenz, also zehn gleich stark besetzte Klassen; bei wenigen Werten jeder Wert) und vergleicht die Anteile p_i der Referenz mit den aktuellen q_i:

$$\mathrm{PSI} = \sum_i (q_i - p_i) \cdot \ln\left(\frac{q_i}{p_i}\right)$$

Er ist null bei gleichen Verteilungen und wächst mit der Abweichung. Üblich sind die Faustregeln unter 0,1 stabil, bis 0,25 Warnung, darüber Alarm; sie stammen aus der Kreditrisikomodellierung und haben keine statistische Begründung. Der **Kolmogorov-Smirnov-Test** (KS-Test) nimmt den größten Abstand der kumulativen Verteilungsfunktionen zweier Stichproben:

$$D = \max_x |F_{\mathrm{ref}}(x) - F_{\mathrm{akt}}(x)|$$

Er liefert zusätzlich einen p-Wert, die Wahrscheinlichkeit, einen mindestens so großen Abstand zu sehen, wenn beide Stichproben in Wahrheit aus derselben Verteilung stammen. Beide Verfahren sind auf einzelne Merkmale zugeschnitten; S. Rabanser, S. Günnemann, und Z. C. Lipton [89] vergleichen Kombinationen aus Dimensionsreduktion und Zwei-Stichproben-Tests für viele Merkmale gleichzeitig.

Wir simulieren drei Produktionswochen mit je 3 000 Bestellungen gegen die 5 000 des Trainings. Woche 1 ist sauber. In Woche 2 hat die Datenerfassung einen Zeitzonenfehler: Die Stunde wird um zwei verschoben gespeichert, alle Werte bleiben gültige Uhrzeiten. In Woche 3 hat der Lieferdienst sein Gebiet erweitert, die Distanzen reichen bis 22 statt 15 km.

```python
def psi(referenz, aktuell, klassen=10):
    """Population Stability Index. Wenige verschiedene Werte: je Wert eine Klasse.
    Sonst Dezile der Referenz."""
    referenz, aktuell = np.asarray(referenz), np.asarray(aktuell)
    if len(np.unique(referenz)) <= 25:
        werte = np.union1d(referenz, aktuell)
        p = np.array([(referenz == w).mean() for w in werte])
        q = np.array([(aktuell == w).mean() for w in werte])
    else:
        kanten = np.quantile(referenz, np.linspace(0, 1, klassen + 1))
        kanten[0], kanten[-1] = -np.inf, np.inf
        p = np.histogram(referenz, kanten)[0] / len(referenz)
        q = np.histogram(aktuell, kanten)[0] / len(aktuell)
    p, q = np.clip(p, 1e-4, None), np.clip(q, 1e-4, None)
    return float(np.sum((q - p) * np.log(q / p)))
```

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy --with scipy python monitoring.py
Referenz: 5000 Bestellungen, MAE im Training 1.94 min
...
                                PSI der Merkmale   PSI der
              distanz artikel  stunde   regen  Vorhers.   MAE
W1 sauber       0.008   0.006   0.006   0.001     0.012  2.02
W2 Zeitzone     0.005   0.005   1.937   0.000     0.003  2.99
W3 Gebiet       0.538   0.003   0.003   0.001     0.362  3.95
...
Einstufung (ok < 0,10 <= Warnung < 0,25 <= ALARM):
  W1 sauber: nichts Auffälliges
  W2 Zeitzone: stunde (ALARM, 1.94)
  W3 Gebiet: distanz_km (ALARM, 0.54), Vorhersage (ALARM, 0.36)
...
KS-Test gegen die Referenz (D = größter Abstand der Verteilungsfunktionen)
  W1 sauber    distanz: D=0.017 p=7e-01   stunde: D=0.014 p=8e-01
  W2 Zeitzone  distanz: D=0.022 p=3e-01   stunde: D=0.156 p=2e-40
  W3 Gebiet    distanz: D=0.322 p=2e-172   stunde: D=0.009 p=1e+00
```

Woche 1 liegt überall unter 0,02. In Woche 2 meldet der PSI der Stunde 1,94, einen klaren Alarm bei sonst unauffälligen Merkmalen. Das Schema hätte das nicht gefunden: 8 bis 20 sind gültige Uhrzeiten. Der Fehler wirkt sich aber aus, denn die Abendspitze fällt auf die falschen Stunden, und der MAE steigt von 2,0 auf 3,0 Minuten. Der PSI der Vorhersage bleibt dabei bei 0,003: Das Vorhersage-Monitoring sieht Verschiebungen, nicht jeden Fehler. In Woche 3 schlagen Distanz (0,54) und Vorhersage (0,36) an, der MAE steigt auf 3,95 Minuten. Jenseits von 15 km kennt das Modell keine Beispiele, wie der Test oben vorhergesagt hat.

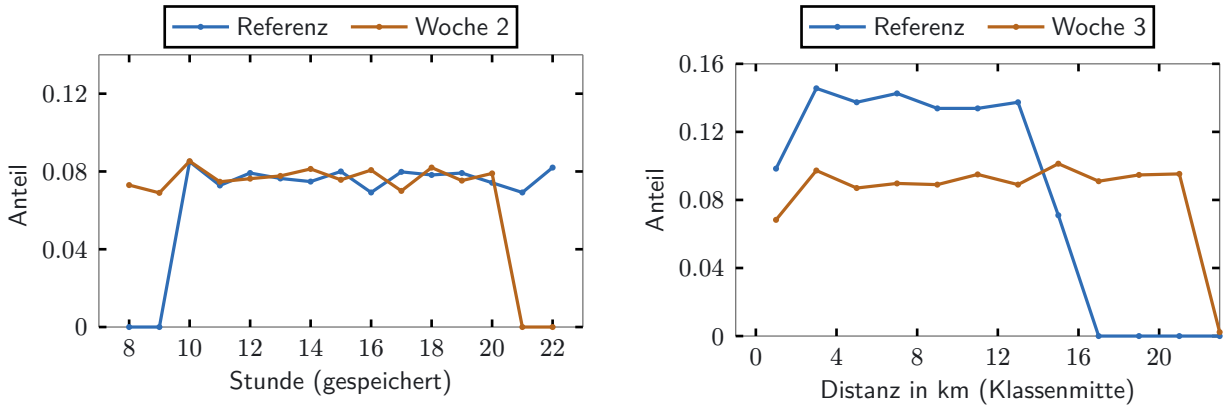


Abbildung 81: Links: Verteilung der gespeicherten Stunde in der Referenz und in Woche 2 (Zeitzonenfehler, um zwei Stunden verschoben). Rechts: Verteilung der Distanz in Klassen von 2 km in der Referenz und in Woche 3 (erweitertes Gebiet). Werte aus `monitoring.py`.

Wann ein Alarm ein Alarm ist

Die Tests liefern Zahlen, keine Entscheidungen, und drei Eigenschaften machen sie heikel. Die erste ist die **Abhängigkeit von der Stichprobengröße**: Ein Signifikanztest beantwortet, ob ein Unterschied existiert, nicht ob er relevant ist, und bei genug Daten ist jeder Unterschied „signifikant“. Wir haben die Distanz um 0,05 km verschoben, 0,3 Prozent des Wertebereichs, und den KS-Test je Stichprobengröße 20-mal wiederholt:

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy --with scipy python monitoring.py
Distanz um 0,05 km verschoben (0,3 % des Wertebereichs), KS-Test, p < 0,05:
  je     1.000 Beobachtungen: Alarm in  1 von 20 Wiederholungen
  je   100.000 Beobachtungen: Alarm in 11 von 20 Wiederholungen
  je 1.000.000 Beobachtungen: Alarm in 20 von 20 Wiederholungen
```

Bei 1 000 Beobachtungen schlägt der Test fast nie an, bei einer Million immer, obwohl die Verschiebung dieselbe und bedeutungslos klein ist. Deshalb sind Effektmaße (PSI, KS-Abstand D) mit Schwellen verlässlicher als der p-Wert allein, und man verlangt zusätzlich, dass die Abweichung über mehrere Zeitfenster anhält. Die zweite Eigenschaft ist das **Mehrfachtest-Problem**: Wer 50 Merkmale mit 5 Prozent Signifikanzniveau prüft, erwartet je Fenster etwa 2,5 Fehlalarme, auch wenn alles in Ordnung ist. Die dritte ist die **Handlungsfähigkeit**: Ein Alarm, auf den niemand reagieren kann oder muss, wird ignoriert. Jeder Alarm braucht einen Verantwortlichen und eine hinterlegte Reaktion (Datenquelle prüfen, neu trainieren, zurückrollen oder bewusst nichts tun). Die Abschnitte dieses Kapitels entsprechen den Monitoring-Aufgaben 2 (Invarianten der Eingabedaten) und 7 (Vorhersagequalität) des ML Test Score.

11.8 Rollout-Strategien

Ein offline besseres Modell ist nicht automatisch online besser: Die Offline-Kennzahl ist ein Stellvertreter für die eigentliche Wirkung (Test Model 2 im ML Test Score), und Trainings- und Produktionsdaten unterscheiden sich fast immer. Deshalb liefert man neue Modelle schrittweise aus. Drei Strategien beantworten verschiedene Fragen (Abbildung 82).

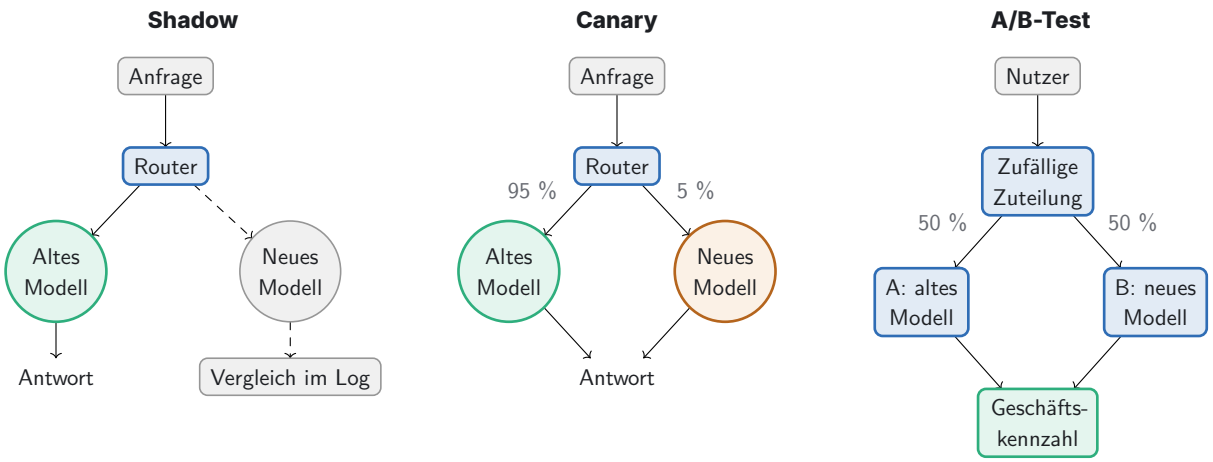


Abbildung 82: Drei Rollout-Strategien. Shadow: Das neue Modell sieht alle Anfragen, doch nur das alte antwortet. Canary: Ein kleiner Anteil der echten Antworten kommt vom neuen Modell. A/B-Test: Zufällig getrennte Gruppen werden über eine Geschäftskennzahl verglichen.

Shadow (auch Traffic Mirroring) leitet jede Anfrage zusätzlich an das neue Modell, verwirft aber dessen Antwort: reale Eingaben, echte Latenzen und Vorhersagen ohne Nutzerwirkung; sobald die Ergebnisse bekannt sind, vergleicht man beide Modelle Anfrage für Anfrage. Die Grenze: Shadow verrät nichts über die Reaktion der Nutzer und verdoppelt die Rechenlast. **Canary** gibt dem neuen Modell einen kleinen Anteil des echten Verkehrs, etwa ein Prozent, und steigert ihn stufenweise, solange Schutzgrößen (Fehlerrate, Latenz, Vorhersageverteilung) im Rahmen bleiben; sonst rollt das System zurück. Canary erkennt grobe Probleme schnell, kann aber wegen der kleinen Stichprobe kleine Verbesserungen nicht nachweisen. Das leistet der **A/B-Test**: Nutzer werden zufällig getrennten Gruppen zugeteilt, jede sieht ein Modell, verglichen wird eine Geschäftskennzahl. R. Kohavi, D. Tang, und Y. Xu [90] behandeln die Fallen ausführlich; die wichtigsten sind, nach Nutzer statt nach Anfrage zuzuteilen, die Laufzeit vorher festzulegen statt ständig nachzusehen und zu prüfen, ob die Gruppen im geplanten Verhältnis befüllt wurden.

Wie lange ein A/B-Test dauern muss, lässt sich vorab ausrechnen. Wir vergleichen das alte Modell (lineare Regression ohne Abendspitze) mit dem Boosting-Modell; Zielgröße ist der Anteil der Lieferungen, bei denen die Vorhersage höchstens drei Minuten abweicht (eine vereinfachte Kennzahl; im echten Test stünde hier eher eine Geschäftsgröße wie Beschwerden oder Stornos):

```
def n_pro_gruppe(p1, p2):
    """Beobachtungen je Gruppe, um p1 von p2 zu unterscheiden (Näherung)."""
    mittel = (p1 + p2) / 2
    streuung = (Z_ALPHA * np.sqrt(2 * mittel * (1 - mittel))
                + Z_BETA * np.sqrt(p1 * (1 - p1) + p2 * (1 - p2)))
    return streuung ** 2 / (p1 - p2) ** 2
```

```
Terminal
$ uv run --with scikit-learn --with pandas --with numpy --with scipy python rollout.py
Trefferquote (Fehler <= 3 min): alt 0.636, neu 0.760
MAE: alt 2.64 min, neu 2.04 min

A/B-Test: nötige Beobachtungen je Gruppe
  0.64 -> 0.74 (+10 Prozentpunkte)                  338
  0.64 -> 0.69 (+5 Prozentpunkte)                 1.407
  0.64 -> 0.66 (+2 Prozentpunkte)                 8.977
  0.64 -> 0.65 (+1 Prozentpunkte)                36.134
  hier: +12.4 Prozentpunkte                         215

Shadow-Modus, gepaarter Vergleich der Fehler (20.000 Anfragen)
  mittlere Verbesserung 0.59 min, t = 48.7
  nötige Anfragen (5 % / 80 % Power)                 67
```

Die Trefferquote steigt von 63,6 auf 76,0 Prozent; diese 12,4 Prozentpunkte weisen schon rund 215 Beobachtungen je Gruppe nach. Für einen Prozentpunkt wären es über 36 000: Der Bedarf wächst mit dem Kehrwert des Quadrats des Unterschieds, ein halbierter Effekt heißt ungefähr vierfachen Aufwand. Wer kleine Verbesserungen belegen will, braucht viel Verkehr oder Geduld. Der Shadow-Modus ist im Vorteil, weil beide Modelle dieselben Anfragen sehen und gepaart verglichen wird: Der mittlere Fehler sinkt um 0,59 Minuten, und schon rund 67 Anfragen genügten. Die Zahlen 215 und 67 betreffen verschiedene Zielgrößen und sind nicht gegeneinander aufzurechnen; sie zeigen aber, dass ein gepaarter Vergleich einer kontinuierlichen Größe viel weniger Daten braucht als zwei getrennte Gruppen mit einer Trefferquote.

Alle Strategien setzen einen schnellen Rollback voraus (ML Test Score, Infrastruktur-Test 7); mit der Registry ist das ein Umhängen des Alias. In der Praxis kombiniert man sie: Offline-Prüfung gegen das aktuelle Modell, Shadow, Canary mit Schutzgrößen und, wenn der Nutzen belegt werden muss, ein A/B-Test. Der Leitfaden von Google Cloud [81] nennt Canary und A/B als Online-Validierung in Level 1.

11.9 Rollen im Team

Wer die Tätigkeiten ausführt, ist eine organisatorische Frage. D. Kreuzberger, N. Kühl, und S. Hirschl [91] unterscheiden unter anderem Business Stakeholder, Solution Architect, Data Scientist, Data Engineer, Software Engineer, DevOps Engineer und ML Engineer; in kleinen Teams übernimmt eine Person mehrere Rollen. Hier genügen vier: Der **Data Scientist** formuliert das Problem, entwickelt und bewertet Modelle. Der **ML Engineer** macht aus dem Experiment ein Produkt (Pipeline, Tests, Serving, Rollout). Der **Data Engineer** verantwortet Datenquellen, Datenpipelines und Datenqualität. Die **Plattform** betreibt Orchestrierung, Registry, CI/CD und Observability.

Tabelle 13: Eine mögliche Verteilung der Verantwortung. „Führt" meint, dass die Rolle am Ende dafür geradesteht; die Zuordnung ist ein Vorschlag dieses Handbuchs und je nach Organisation anders.

Tätigkeit	Data Scientist	ML Engineer	Data Engineer	Plattform
Problem und Zielgröße festlegen	**führt**	wirkt mit	wirkt mit	
Experimente, Modellentwicklung	**führt**	wirkt mit		
Datenquellen, Datenpipelines, Schemata	wirkt mit	wirkt mit	**führt**	
Feature-Berechnung (Transform, Feature Store)	wirkt mit	**führt**	**führt**	wirkt mit
Trainingspipeline, Tests, Modellvalidierung	wirkt mit	**führt**		wirkt mit
Registry, CI/CD, Orchestrierung		wirkt mit		**führt**
Serving und Rollout		**führt**		wirkt mit
Monitoring und Alarmierung	wirkt mit	**führt**	wirkt mit	**führt**
Fairness- und Erklärbarkeitsprüfung	**führt**	wirkt mit		

Aufschlussreich sind die Nahtstellen. Zwischen Data Scientist und ML Engineer entsteht das Level-0-Problem der doppelten Implementierung; die Antwort ist eine gemeinsame, getestete Pipeline (Patterns 21 und 25). Zwischen Data Engineer und ML Engineer liegt der Feature-Vertrag (Pattern 26 oder ein gemeinsames Schema), zwischen ML Engineer und Plattform die Frage, wer nachts geweckt wird. S. Amershi *u. a.* [92] berichten aus einer Fallstudie bei Microsoft, dass ML-Teams Fachleute mit sehr unterschiedlichem Hintergrund zusammenbringen und Übergänge Reibung erzeugen.

11.10 Die 30 Patterns im Betrieb

Abbildung 83 ordnet alle 30 Patterns den Stationen eines betriebenen ML-Systems zu. Der Kreislauf läuft im Uhrzeigersinn: Daten und Features bereitstellen, trainieren, vor der Freigabe prüfen, registrieren und ausrollen, ausliefern, überwachen; das Monitoring stößt bei Bedarf einen neuen Lauf an. Oben stehen die Patterns des Modellentwurfs, unten das Pattern, das alle Schritte verbindet. Jedes Pattern erscheint genau einmal; gehört eines fachlich zu zwei Stationen, gilt die dominierende.

Die Patterns des Modellentwurfs haben Betriebsfolgen, die man leicht übersieht. Ein Hashed Feature (1) erspart ein Vokabular, das man pflegen müsste, nimmt aber Kollisionen in Kauf. Embeddings (2) sind gelernte Teile des Modells und müssen mit demselben Modellstand ausgeliefert werden. Ensemble (7)

und Cascade (8) vervielfachen die Zahl der Modelle, die versioniert, ausgerollt und überwacht werden müssen. Rebalancing (10) verändert die Klassenverhältnisse im Training, sodass die Vorhersageverteilung nicht mehr direkt mit der Realität vergleichbar ist; beim Monitoring ist das zu berücksichtigen. Umgekehrt dient die Karte als Prüfliste: Gibt es einen Feature-Vertrag? Eine Freigabeprüfung, die ein schlechteres Modell tatsächlich blockiert? Weiß der Serving-Code, welche Version er lädt? Hört jemand auf die Alarme?

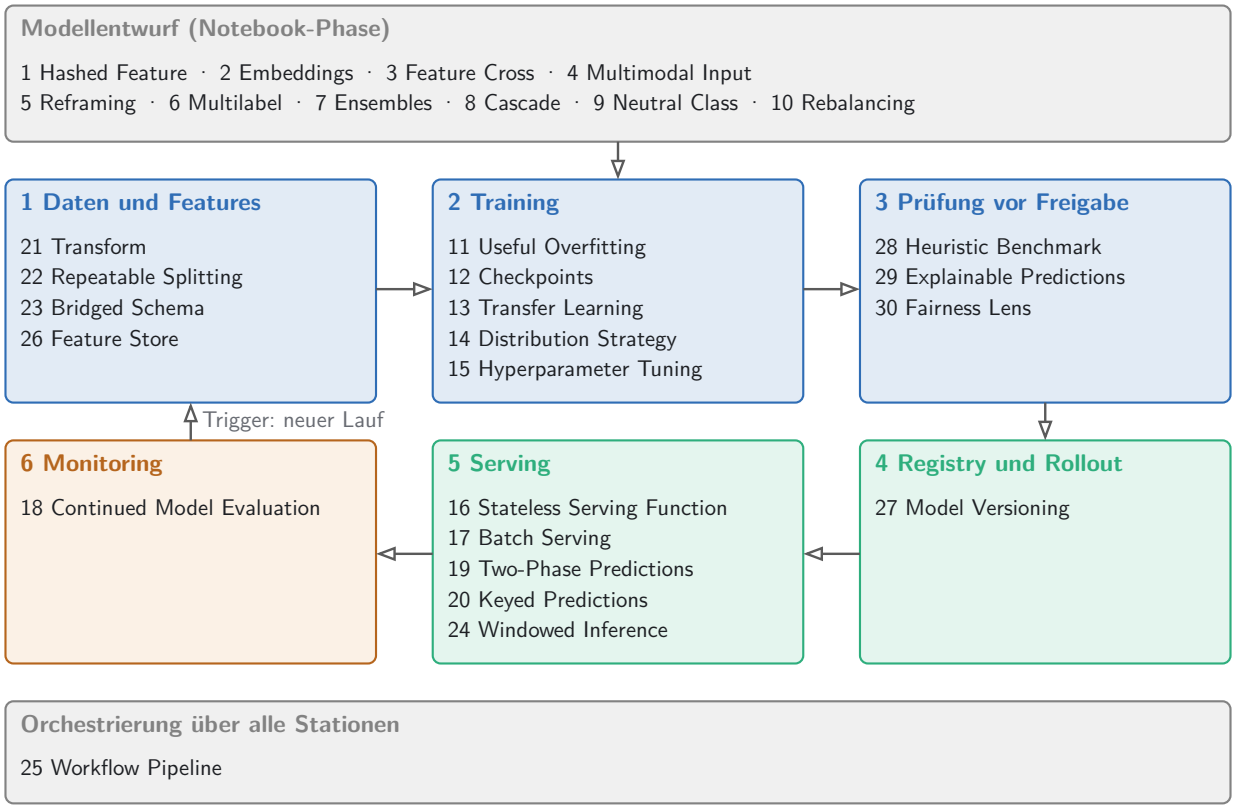


Abbildung 83: Alle 30 Patterns auf einer Landkarte des betriebenen ML-Systems. Blau: Vorbereitung und Training; grün: Auslieferung; orange: Überwachung; grau: übergreifend. Die Pfeile zeigen den Kreislauf im Uhrzeigersinn.

11.11 Das Wichtigste in Kürze

- Der ML-Code ist der kleinste Teil eines ML-Systems. Technische Schuld entsteht an den Rändern: Verflechtung der Merkmale (CACE), instabile Datenabhängigkeiten, unbekannte Konsumenten, Glue Code und gewachsene Pipelines [11]. Weil Daten das Verhalten mitbestimmen und sich ohne Commit ändern, müssen auch Daten, Modelle und Pipelines versioniert und geprüft werden.
- Die Reifegrade nach Google Cloud [81]: Level 0 liefert ein Modell manuell aus, Level 1 eine Trainingspipeline, die selbst neu trainiert (Continuous Training), Level 2 zusätzlich die Pipeline automatisiert. Welcher genügt, hängt davon ab, wie schnell das Modell veraltet.
- Experiment Tracking und Model Registry (etwa MLflow) machen Läufe vergleichbar und Modelle versioniert; ein Alias trennt Serving von Training und macht den Rollback zu einer einzigen Operation.
- Ein Schema ist der Vertrag zwischen Datenquelle und Modell. Es findet, was man als Regel formulieren konnte, nicht Fehler, die alle Regeln einhalten und nur die Verteilung verschieben.
- Der ML Test Score hat 28 Tests in vier Kategorien und zählt das Minimum der Kategorien. Modelltests prüfen Verhalten: Teilmengen, Richtungserwartungen, Vergleich mit einer Faustformel.
- PSI und KS-Test sind Frühwarnungen ohne Labels. Sie hängen von der Stichprobengröße ab, erzeugen bei vielen Merkmalen Fehlalarme und brauchen Schwellen, Verantwortliche und hinterlegte Reaktionen.
- Shadow prüft ohne Nutzerwirkung, Canary begrenzt das Risiko, der A/B-Test belegt den Nutzen. Kleine Effekte brauchen sehr viele Beobachtungen.

11.12 Weiterführende Literatur

- D. Sculley *u. a.* [11] und E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55]: die Ausgangspapiere des Kapitels, beide kurz und im Original lesenswert.
- Google Cloud [81]: der Leitfaden zu den Reifegraden, mit Architekturdiagrammen der drei Stufen.
- C. Huyen [93]: ein modernes Lehrbuch zum gesamten Thema, mit Schwerpunkt auf Daten, Deployment und Monitoring.
- A. Paleyes, R.-G. Urma, und N. D. Lawrence [94] und D. Kreuzberger, N. Kühl, und S. Hirschl [91]: eine Übersicht über Probleme aus Fallstudien zum Betrieb von ML-Systemen und eine Referenzarchitektur für MLOps.
- R. Kohavi, D. Tang, und Y. Xu [90]: das Standardwerk zu A/B-Tests mit allen Fallstricken.
- D. Sato, A. Wider, und C. Windheuser [95]: Continuous Delivery für ML, als Artikel frei lesbar.
- V. Lakshmanan, S. Robinson, und M. Munn [1]: die Patterns selbst; Kapitel 5 (Resilient Serving) und 6 (Reproducibility) des Buches sind hier am wichtigsten.

12 Fallstudie A: Kundenabwanderung bei einem Streamingdienst

In den Kapiteln zuvor stand jedes Pattern für sich: ein Szenario, ein Problem, eine Lösung. Ein Projekt verläuft anders. Man beginnt mit einem Notebook, das „funktioniert", und stößt der Reihe nach auf die Probleme, die einzelne Patterns lösen, oft erst, nachdem jemand eine falsche Zahl präsentiert hat. Dieses Kapitel erzählt ein solches Projekt: Ein Team soll vorhersagen, welche Kundinnen und Kunden eines Streamingdienstes demnächst kündigen, startet mit einem naiven Notebook und verbessert es in acht Schritten. Jeder Schritt wird mit Zahlen aus lauffähigem Code (`code/12/`) gemessen, und hinter jedem steht ein Pattern, das Sie bereits kennen. Wichtiger als jedes Detail ist die Abfolge: warum man das eine vor dem anderen tut und woran man merkt, dass es fehlt.

LERNZIELE
Nach diesem Kapitel können Sie erklären, warum ein Projekt mit einer Faustregel beginnt und nicht mit einem Modell; wie ein Split nach Zeilen eine PR-AUC von 0,26 auf 0,39 aufbläht und warum er sogar die Modellwahl verfälscht; weshalb eine hohe Genauigkeit bei seltenen Ereignissen nichts aussagt und wie man stattdessen aus Kosten eine Entscheidungsregel ableitet; wie eine Pipeline Training und Betrieb zusammenhält; was eine Erklärung und eine Auswertung nach Gruppen im Projektalltag leisten; und wie man ein Modell im Betrieb überwacht, wenn die Wahrheit erst Monate später bekannt wird.

12.1 Ausgangslage und Daten

Der Streamingdienst verkauft monatlich kündbare Abonnements in drei Tarifen (ab Monat 16 in vier). Das Retention-Team kann jeden Monat einigen Kundinnen und Kunden ein Halteangebot schicken, etwa einen Gutschein über einen Gratismonat. Es möchte wissen, *wen* es ansprechen soll: Das Modell soll jeden Kunden am Monatsende bewerten, und zwar mit einer Wahrscheinlichkeit, dass das Abonnement innerhalb der nächsten drei Monate endet. Aus den Wahrscheinlichkeiten macht eine Entscheidungsregel eine Liste.

Die Daten sind simuliert (`daten.py`, fester Seed). Das hat einen Preis und einen Nutzen. Der Preis: Echte Daten sind unordentlicher, und Effekte sind selten so sauber trennbar wie hier. Der Nutzen: Wir kennen die Mechanik hinter den Zahlen, weil wir sie gebaut haben, und können deshalb prüfen, ob eine Methode das Richtige findet. Der Dienst startet in Monat 0 und wächst; insgesamt entstehen 50 000 Kunden mit zusammen 574 865 Zeilen, im Mittel also gut elf Zeilen je Kunde. *Eine Zeile ist ein Kunde in einem Monat*, mit den Werten vom Monatsende. Das Ziel `abwanderung_3m` ist 1, wenn das Abonnement spätestens mit dem Monat m + 2 endet (Abbildung 84). Im Entwicklungsfenster trifft das auf 4,5 % der Zeilen zu: Die Daten sind stark unausgewogen.

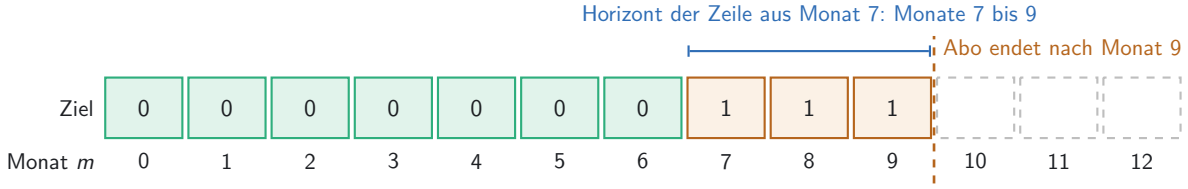


Abbildung 84: Ein Kunde, der von Monat 0 an dabei war und nach Monat 9 kündigt, liefert Zeilen für die Monate 0 bis 9. Die letzten drei tragen das Ziel 1, weil sein Abo spätestens zwei Monate nach dem jeweiligen Zeilenmonat endet. Der Kunde besteht also aus zehn Zeilen, die einander sehr ähnlich sehen.

Dieser Aufbau trägt das ganze Kapitel: Die Zeilen eines Kunden sind keine unabhängigen Beispiele. Sie teilen Alter, Land und Anschlussgeschwindigkeit, und bei Abwanderern folgen auf eine Zeile mit Ziel 1 meist zwei weitere.

Tabelle 14: Die Merkmale der Tabelle; Ziel ist `abwanderung_3m`. Die Altersgruppe in Schritt 7 wird aus `alter` abgeleitet.

Gruppe	Spalten und Bedeutung
Kunde (konstant)	`alter`, `land`, `kanal` (wie gewonnen: organisch, Werbung, Partner, Aktion), `geraet`, `anmeldetag` (Tage seit Start des Dienstes), `bandbreite_mbit` (typische Anschlussgeschwindigkeit)
Vertrag	`tarif` (basic, standard, premium, ab Monat 16 auch mobil), `preis` pro Monat, `tenure` (Monate seit Anmeldung), `aktion_restmonate` (Rest des Aktionspreises)
Nutzung	`stunden`, `sessions`, `tage_seit_login`, `stunden_trend` (Stunden dieses Monats geteilt durch den Schnitt der drei Vormonate)
Betreuung, Zahlung	`tickets_30d` (Supportanfragen), `zahlungsfehler_90d` (fehlgeschlagene Abbuchungen)

Zwei Festlegungen gelten für das ganze Kapitel. Erstens die Zeit (Abbildung 85): Die Monate 0 bis 12 stehen zur Entwicklung bereit, danach beginnt der „Betrieb“. In Monat 16 ändert sich die Welt: Der Dienst führt den Tarif *mobil* ein und erhöht die Preise von *standard* und *premium* um zwei Euro. Das Ziel einer Zeile ist erst drei Monate später bekannt, deshalb enden die Daten mit gültigem Ziel in Monat 20, obwohl der Dienst bis Monat 23 beobachtet wurde. Zweitens das Geld. Ein Halteangebot kostet 5 €, hält 30 % der angesprochenen Abwanderer, und ein gehaltener Kunde bringt 60 € Deckungsbeitrag (angenommene Zahlen, keine Messwerte). Damit lassen sich Entscheidungen in Euro bewerten: Ein Kontakt lohnt sich, wenn die Abwanderungswahrscheinlichkeit größer ist als 5 / (0,3 · 60) = 0,278, also rund 28 %.

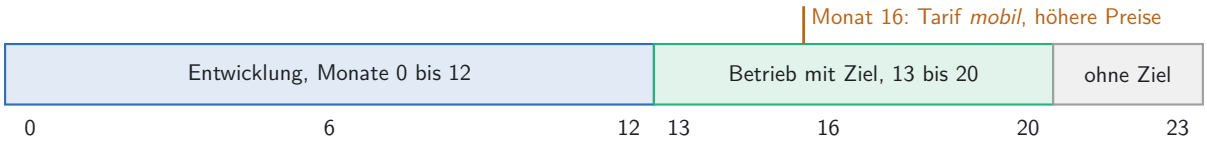


Abbildung 85: Die Zeitachse des Projekts. Bis Monat 12 wird entwickelt, ab Monat 13 läuft das Modell im Betrieb. Ab Monat 21 ist das Ziel der Zeilen noch nicht bekannt.

12.2 Schritt 1: Zuerst der Maßstab (Heuristic Benchmark)

Bevor jemand ein Modell trainiert, fragt die Teamleiterin das Retention-Team, wie es heute vorgeht. Die Antwort ist eine Faustregel: Wer seit mindestens vier Tagen nicht mehr eingeloggt war oder in diesem Monat weniger als halb so viele Stunden geschaut hat wie im Schnitt der drei Vormonate, bekommt ein Angebot. Das ist der *Heuristic Benchmark* (Pattern 28): ein einfacher Vergleichsmaßstab, den ein Modell schlagen muss, um seinen Aufwand zu rechtfertigen. Die Regel braucht kein Training, also auch keinen Split; man darf sie auf allen Entwicklungsdaten messen.

```
# Maßstab 1: "Niemand kündigt" (die häufigste Klasse)
nie = kennzahlen(y, np.zeros(len(y)))

# Maßstab 2: die Faustregel des Retention-Teams. Sie lernt nichts, braucht also
# keinen Split und darf auf allen Entwicklungsdaten gemessen werden.
regel = (dev.tage_seit_login >= 4) | (dev.stunden_trend < 0.5)
r = kennzahlen(y, regel.to_numpy().astype(float), 0.5)
r["nutzen"] = nutzen(r, len(y))
ap_login = average_precision_score(y, dev.tage_seit_login)
```

Terminal

```
$ python 01_heuristik.py
Kunden 35,041 | Zeilen 285,328 | Zeilen je Kunde 8.1
Abwanderung in den nächsten 3 Monaten: 4.5%

Niemand kündigt: Accuracy 0.955
Faustregel     : Accuracy 0.863, Precision 0.128, Recall 0.350
                 F1 0.187, markiert 12.3% der Zeilen
Nutzen mit Faustregel: -333 Euro je 1000 Kunden-Monate
PR-AUC nach Login-Tagen: 0.197 (Zufall 0.045)
```

Die Zahlen sind aufschlussreicher, als es zunächst scheint. Wer niemandem ein Angebot schickt, hat eine Genauigkeit von 0,955, und die Faustregel schneidet mit 0,863 *schlechter* ab als das Nichtstun. Genauigkeit ist bei einer Quote von 4,5 % offenbar die falsche Währung, dazu mehr in Schritt 4. Die Faustregel erwischt 35 % der späteren Abwanderer (Recall), aber nur 13 % der Angeschriebenen kündigen tatsächlich (Precision). Weil das unter den 28 % liegt, die ein Angebot kosten darf, verliert die Kampagne mit dieser Regel Geld: −333 € je 1000 Kunden-Monate. Rangiert man die Kunden nur nach der Zahl der Tage seit dem letzten Login, ergibt sich eine PR-AUC (average precision; dazu Pattern 10) von 0,197, gegenüber 0,045 bei zufälligem Raten. Das sind die Messlatten: 0,197 beim Ranking, 0 € bei der Kampagne.

12.3 Schritt 2: Das naive Notebook

Zwei Wochen später zeigt die Data Scientist ihr Notebook: ein Gradient-Boosting-Modell (viele nacheinander gelernte Entscheidungsbäume), großzügig dimensioniert, weil es „alles lernen kann“, mit `pd.get_dummies` für die Kategorien und dem Standardsplit `train_test_split` (80 % Training, 20 % Test).

```
X = pd.get_dummies(dev[ALLE0]).astype(float)       # One-Hot "von Hand" im Notebook
y = dev[ZIEL]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

modell = modell_gross().fit(X_tr, y_tr)
p = modell.predict_proba(X_te)[:, 1]
k = kennzahlen(y_te, p, 0.5)
```

Terminal

```
$ python 02_naives_notebook.py
Zeilen Training 228,262 | Test 57,066
Accuracy 0.960 | Precision 0.713 | Recall 0.195 | PR-AUC 0.385
```

Auf den ersten Blick ein Erfolg: Genauigkeit 0,960, Precision 0,71, PR-AUC 0,385, fast doppelt so viel wie beim Ranking nach Login-Tagen (0,197). Drei Dinge sollten das Team misstrauisch machen. Die Genauigkeit liegt kaum über dem Wert für „niemand kündigt“ (0,955). Das Modell findet nur 20 % der Abwanderer. Und die Testzeilen wurden zufällig aus einer Tabelle gezogen, in der jeder Kunde im Mittel 8,1 Zeilen hat. Der letzte Punkt bekommt als nächstes eine eigene Prüfung.

12.4 Schritt 3: Der Split nach Kunden (Repeatable Splitting)

Das Leck. Beim zufälligen Zeilen-Split landen die Zeilen eines Kunden in Training *und* Test. Im Skript `03_split.py` sind es 25 518 Kunden, und bei 98,6 % der Testzeilen gibt es mindestens eine Zeile desselben Kunden im Training. Das Modell muss einen Kunden nicht verstehen, es genügt, ihn wiederzuerkennen: `anmeldetag` und `bandbreite_mbit` ändern sich nie und wirken wie ein Fingerabdruck, und die benachbarte Zeile im Training verrät das Ziel (bei Abwanderern sind drei Zeilen hintereinander positiv). Ein großes Modell mit winzigen Blättern kann solche Muster auswendig lernen. Die Testzahl misst dann zum Teil Gedächtnis. Das lässt sich nachweisen: Ohne die beiden Spalten sinkt die PR-AUC des großen Modells auf dem Zeilen-Split von 0,385 auf 0,302 (Ausgabe unten). Die restliche Lücke bis 0,258 beim Kunden-Split erklärt dieser Versuch nicht. Plausibel ist, dass auch weniger auffällige, über Monate stabile Merkmale wie Alter, Land und Nutzungsniveau helfen, einen Kunden wiederzuerkennen; dazu kommt, dass beide Splits

verschiedene Testkunden verwenden. Die übliche Zufallsmischung setzt voraus, dass Zeilen unabhängig sind, und diese Voraussetzung ist hier verletzt.

Die Lösung. Das Pattern *Repeatable Splitting* (Pattern 22) trennt nach der Einheit, die unabhängig ist, hier dem Kunden, und macht die Zuordnung deterministisch. Ein Hash der Kunden-ID bestimmt den Bucket (0 bis 9). Die Buckets 0 bis 5 gehören zum Training, 6 und 7 zur Validierung, 8 und 9 zum Test, und diese Zuordnung gilt für jede Zeile jedes Kunden.

```python
def bucket(kunden_id: str, n: int = 10) -> int:
    """Stabil über Prozesse und Rechner hinweg (nicht Pythons gesalzenes hash())."""
    digest = hashlib.sha256(kunden_id.encode("utf-8")).digest()
    return int.from_bytes(digest[:8], "big") % n


def teile(df: pd.DataFrame):
    """60 % Training, 20 % Validierung, 20 % Test, nach Kunde statt nach Zeile."""
    df = mit_bucket(df)
    return df[df.bucket < 6], df[df.bucket.isin([6, 7])], df[df.bucket >= 8]
```

Der Hash ist SHA-256 und nicht Pythons eingebautes `hash()`. Die Begründung steht in der Ausgabe weiter unten: Das eingebaute `hash()` für Strings ist pro Prozess gesalzen, liefert also bei jedem Start eine andere Zuordnung (die Zahlen dieser Ausgabezeile ändern sich bei jedem Aufruf). Ein Split, der sich bei jedem Lauf ändert, ist kein Split, auf den man ein Ergebnis stützen kann.

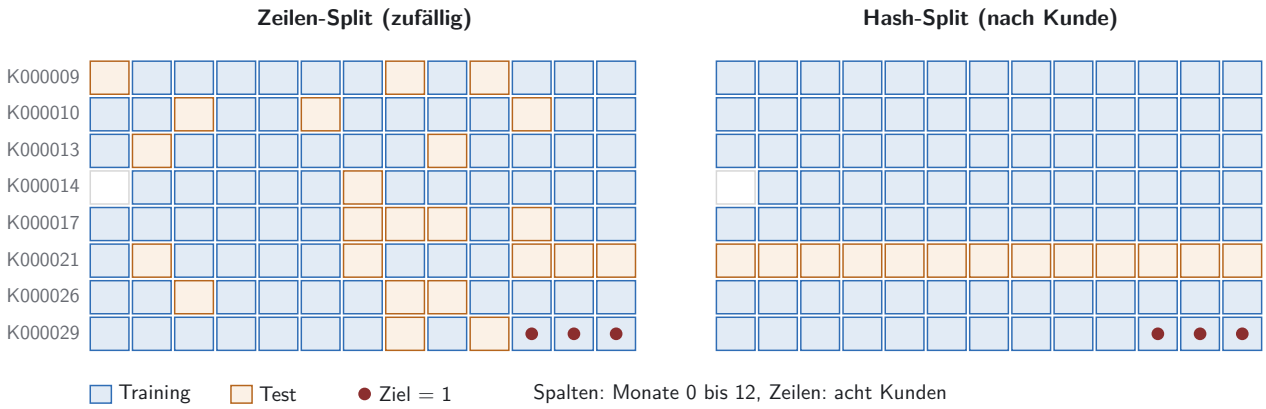


Abbildung 86: Zuordnung der Zeilen von acht Kunden unter beiden Verfahren (die ersten acht mit mindestens zehn Zeilen, echte Zuordnung aus dem Skript; beim Hash-Split sind hier Training und Validierung zusammengefasst). Links gehört fast jeder Kunde zu beiden Seiten, rechts nur zu einer. Die roten Punkte markieren Zeilen mit Ziel 1: Links liegen sie neben Trainingszeilen desselben Kunden.

```
Terminal
$ python 03_split.py
Zeilen-Split: 25,518 Kunden in Training UND Test; bei 98.6% der
              Testzeilen liegen Zeilen desselben Kunden im Training
Kunden-Split: 0 Kunden in beiden Teilen

PR-AUC (Test)       Zeilen-Split  Kunden-Split
großes Modell              0.385         0.258
kleines Modell             0.311         0.292
Großes Modell, Zeilen-Split, ohne anmeldetag und bandbreite_mbit: 0.302
Validierungskunden: großes Modell 0.236, kleines Modell 0.289

Tabelle wächst auf 574,865 Zeilen. Alte Zeilen, die die Seite wechseln:
  Zeilen-Split: 32.1%
  Hash-Split  : 0.0%
hash('K000001') % 10, drei Prozesse: ['3', '0', '9']
bucket('K000001')     , drei Prozesse: ['7', '7', '7']
```

Die Tabelle in der Ausgabe enthält drei Befunde. Der erste betrifft das Modell aus dem Notebook (das „große“): Seine PR-AUC fällt von 0,385 beim Zeilen-Split auf 0,258 beim Kunden-Split, ein Verlust von 33

%. Dieselben Daten, dasselbe Modell, nur eine andere Trennlinie. Ein Drittel der schönen Zahl der Data Scientist war Wiedererkennen statt Verallgemeinern.

Der zweite Befund ist heimtückischer. Auf dem Zeilen-Split gewinnt das große Modell (0,385 gegenüber 0,311 für ein kleineres mit flachen Bäumen und großen Blättern). Wer so auswählt, nimmt das größere. Auf den Validierungskunden dreht sich die Reihenfolge um (0,236 gegenüber 0,289), und auf den Testkunden bestätigt sie sich (0,258 gegenüber 0,292). Ein leckender Split verzerrt also nicht nur eine Zahl, sondern auch die Entscheidungen, die man auf Grundlage dieser Zahl trifft. Für den Rest des Kapitels gilt das kleinere Modell.

Der dritte Befund betrifft die Wiederholbarkeit. Kommen neue Monate hinzu, wächst die Tabelle von 285 328 auf 574 865 Zeilen. Ruft man `train_test_split` mit demselben `random_state` auf der größeren Tabelle auf, wechseln 32 % der alten Zeilen die Seite. Zeilen, mit denen ein Modell trainiert wurde, tauchen im neuen Test auf, und Vergleiche zwischen Läufen sind wertlos. Beim Hash-Split wechselt keine (0 %): Wer einmal im Test war, bleibt im Test.

> **ACHTUNG**
> Der Hash-Split schützt vor Kunden-Leck, nicht vor Zeit-Leck. Zeilen anderer Kunden aus späteren Monaten können im Training liegen, während ein Testkunde aus einem früheren Monat stammt. Für Marktveränderungen wie die in Monat 16 reicht ein Split nach Kunden deshalb nicht; dort hilft nur der zeitliche Test, den Schritt 8 durchführt.

12.5 Schritt 4: Seltene Ereignisse (Rebalancing)

Mit dem sauberen Split sieht die Lage nüchterner aus, aber noch nicht hoffnungslos: Das kleine Modell erreicht auf dem Test eine PR-AUC von 0,292. Nun fehlt noch die Entscheidung. Aus Wahrscheinlichkeiten muss eine Liste werden, und dafür gibt es Stellschrauben, die das Pattern *Rebalancing* (Pattern 10) beschreibt: Klassengewichte, Downsampling der Mehrheitsklasse und eine verschobene Schwelle. Das Skript `04_rebalancing.py` probiert sie nebeneinander aus; alle Modelle sind auf den Trainingskunden gelernt, Schwellen auf den Validierungskunden gewählt und alle Zahlen auf den Testkunden gemessen.

```
m_plain = fit(tr.index)
m_gew = fit(tr.index, class_weight="balanced")
neg = tr.index[tr[ZIEL] == 0]
pos = tr.index[tr[ZIEL] == 1]
rng = np.random.default_rng(0)
m_down = fit(pos.union(pd.Index(rng.choice(neg, size=4 * len(pos), replace=False))))
```

Terminal

```
$ python 04_rebalancing.py
Training 171,508 | Validierung 56,762 | Test 57,058

Niemand kündigt: Accuracy 0.955
Variante                          Acc  Prec   Rec    F1 PR-AUC   Euro
A  Schwelle 0.5                 0.959 0.667 0.187 0.292  0.292     89
B  Gewichte, Schwelle 0.5       0.853 0.153 0.497 0.234  0.280   -330
C  Downsampling 1:4, Schw. 0.5  0.951 0.433 0.265 0.329  0.284     77
D  F1-Schwelle 0.20             0.956 0.522 0.238 0.327  0.292     90
E  Gewichte, F1-Schwelle 0.80   0.955 0.496 0.245 0.328  0.280     87
F  Kostenregel, Schwelle 0.28   0.958 0.596 0.216 0.317  0.292     94
(Euro = Nutzen je 1000 Kunden-Monate)

Mittlere Wahrscheinlichkeit: ohne Gewichte 0.046, mit 0.394; tatsächlich 0.045
```

Die Tabelle lässt sich auf vier Beobachtungen verdichten.

Die PR-AUC bleibt, wo sie ist. Sie liegt für alle sechs Varianten zwischen 0,280 und 0,292. Gewichte und Downsampling ändern kaum, wie gut das Modell Kunden *ordnet*, und die Schwelle ändert die PR-AUC per Definition nicht; sie legt nur fest, wo die Linie „angesprochen / nicht angesprochen“ verläuft. Rebalancing ist hier eine Entscheidungsfrage, keine Lernfrage (Abbildung 87).

Die Genauigkeit zeigt nichts. Variante A erreicht 0,959, „niemand kündigt" erreicht 0,955. Der Unterschied liegt in der dritten Stelle, während der Recall zwischen 0,19 und 0,50 schwankt.

Gewichte ohne neue Schwelle sind gefährlich. Variante B (`class_weight="balanced"` und Schwelle 0,5) findet 50 % der Abwanderer, gut zweieinhalbmal so viele wie A. Dafür kündigen nur 15 % der Angesprochenen, weit unter den 28 % der Kostenrechnung; die Kampagne verliert 330 € je 1000 Kunden-Monate. Der Grund steht in der letzten Zeile der Ausgabe: Mit Gewichten liegt die mittlere vorhergesagte Wahrscheinlichkeit bei 0,394, tatsächlich sind es 0,045. Das Modell sagt nun nicht mehr „rund 5 von 100", sondern im Mittel „39 von 100". Die Zahlen sind verschoben, und die Kostenregel „ab 28 %" ist auf ihnen sinnlos. Wer gewichtet, muss die Schwelle auf Validierungsdaten neu suchen (Variante E).

Die Kostenregel genügt. Das Modell ohne Gewichte liefert Wahrscheinlichkeiten, die zur Realität passen (0,046 gegenüber 0,045). Deshalb darf man die Schwelle aus der Wirtschaftlichkeit ableiten: Variante F spricht Kunden ab 28 % an und erzielt mit 94 € den höchsten Nutzen, ohne dass dafür etwas abgestimmt werden musste (die Unterschiede zu A und D sind klein und liegen im Bereich der Stichprobenschwankung). Die auf F1 optimierte Schwelle (D) hat einen höheren F1-Wert als A, aber kaum mehr Nutzen: F1 gewichtet Precision und Recall gleich, die Kasse tut das nicht.

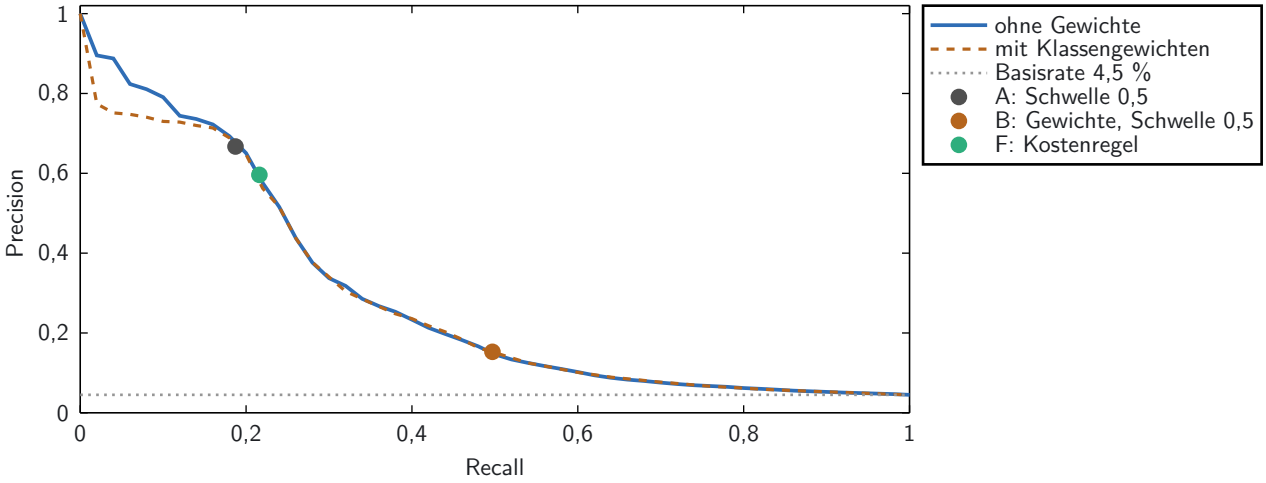


Abbildung 87: Precision-Recall-Kurven des Modells ohne und mit Klassengewichten (Testkunden). Die Kurven liegen fast aufeinander: Gewichte ändern die Rangfolge kaum. Die Punkte sind Betriebspunkte aus der Tabelle.

Für das System gilt ab hier Variante F: kein Gewicht, Schwelle aus der Kostenrechnung. Der Preis dafür ist eine Abhängigkeit, die man dokumentieren muss: Ändert sich der Wert eines Kunden oder der Preis des Angebots, ändert sich die Schwelle, ohne dass das Modell neu trainiert werden muss.

12.6 Schritt 5: Vorverarbeitung gehört ins Modell (Transform)

Bisher hat das Notebook die Daten mit `pd.get_dummies` aufbereitet, auf der ganzen Tabelle. Jetzt soll das Modell in Betrieb gehen, und das Platform-Team schreibt einen Dienst, der jeden Monat alle Kunden bewertet. Dazu muss die Aufbereitung ein zweites Mal existieren, im Service. Das Pattern *Transform* (Pattern 21) verlangt, dass sie nur einmal existiert, als Teil des Modells. In scikit-learn heißt das `Pipeline`:

```
def vorverarbeitung(mit_log=True, mit_fingerabdruck=True) -> ColumnTransformer:
    """Schiefe Spalten: log1p und standardisieren; Zahlen: standardisieren; Kategorien: One-
Hot."""
    log = "passthrough"
    if mit_log:
        log = FunctionTransformer(np.log1p, feature_names_out="one-to-one")
    zahlen = ZAHL + (ID_ARTIG if mit_fingerabdruck else [])
    return ColumnTransformer([
        ("log", Pipeline([("log", log), ("skala", StandardScaler())]), SCHIEF),
        ("zahl", StandardScaler(), zahlen),
        ("kat", OneHotEncoder(handle_unknown="ignore"), KAT),
    ])
```

Vier rechtsschiefe Spalten (`stunden`, `sessions`, `tage_seit_login`, `stunden_trend`) werden mit `log1p` gestaucht und dann standardisiert, die übrigen Zahlen nur standardisiert, die Kategorien one-hot kodiert, wobei `handle_unknown="ignore"` unbekannte Werte nicht abbrechen lässt, sondern als Nullen behandelt. Mittelwerte, Standardabweichungen und Kategorienlisten sind *gelernte* Parameter und stecken im Artefakt. Abbildung 88 zeigt, wie dasselbe Objekt Training und Betrieb bedient; Abbildung 89 zeigt, was `log1p` mit einer der Spalten macht.

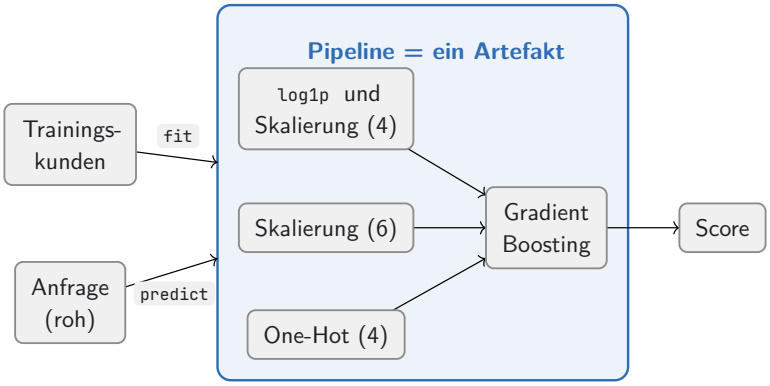


Abbildung 88: Eine Pipeline für beide Phasen. Beim Training (`fit`) lernt sie Parameter und Modell, im Betrieb (`predict`) wendet sie dieselben Parameter auf rohe Anfragen an. Die Zahlen in Klammern geben die Anzahl der Spalten an (Stand ab Schritt 6; in Schritt 5 kommen zwei Spalten zur Skalierung hinzu, die Schritt 6 entfernt).

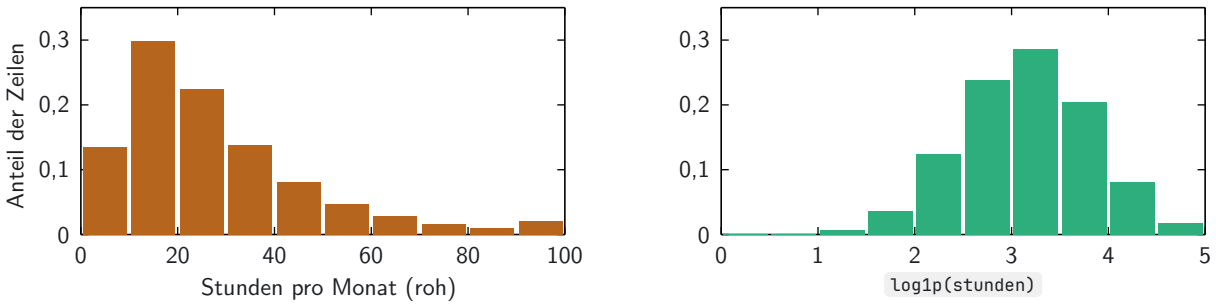


Abbildung 89: Die Stunden pro Monat vor und nach `log1p` (Testkunden; im linken Bild fasst der letzte Balken alle Werte ab 90 Stunden zusammen). Die Schiefe sinkt von 2,97 auf −0,05.

Terminal

```
$ python 05_transform.py
Logistische Regression: PR-AUC 0.252 ohne log1p, 0.247 mit log1p
Gradient Boosting     : PR-AUC 0.292 ohne log1p, 0.292 mit log1p
Schiefe von 'stunden' vor/nach log1p: 2.97 / -0.05
Pipeline vs. Einzelaufruf der Teile: größte Abweichung 0.0e+00
Service ohne log1p: PR-AUC 0.188 (Pipeline 0.292)
  mittlere Wahrscheinlichkeit 0.140 (Pipeline 0.046)
  Entscheidung ändert sich bei 21.7% der Zeilen
Unbekannter Tarif 'mobil', Pipeline : [0.023 0.03  0.037 0.032 0.031]
Unbekannter Tarif 'mobil', get_dummies: ValueError
  The feature names should match those that were passed during fit.
modell_v0.joblib gespeichert (PR-AUC Test 0.292).
```

Die Ausgabe zeigt zuerst, was die Transformation *nicht* bringt: Das Gradient-Boosting-Modell verändert sich nicht (0,292 ohne, 0,292 mit `log1p`). Bäume schneiden Wertebereiche und kennen nur die Reihenfolge der Werte, jede monotone Umformung ist ihnen gleich. Auch die logistische Regression, die mit Größenordnungen rechnet, gewinnt nichts (0,252 ohne, 0,247 mit `log1p`). Die Pipeline lohnt sich also nicht wegen der Metrik, sondern wegen des Betriebs. (Die Zeile „Pipeline vs. Einzelaufruf“ der Ausgabe ist die Gegenprobe: Ruft man Vorverarbeitung und Modell einzeln auf, kommt dasselbe heraus wie aus der Pipeline, die größte Abweichung ist null.)

Das zeigt der zweite Teil. Ein Team, das die Vorverarbeitung nachbaut, kann die richtigen Mittelwerte und Standardabweichungen übernehmen und trotzdem `log1p` vergessen. Das Modell bekommt dann Eingaben, die nicht wie im Training aussehen, und antwortet ohne Fehlermeldung: Die PR-AUC fällt

von 0,292 auf 0,188, und bei 22 % der Zeilen kippt die Entscheidung. Das ist *Training-Serving Skew* in Reinform. Die Pipeline schließt diese Fehlerklasse aus, weil es nur eine Implementierung gibt. Der letzte Teil der Ausgabe zeigt eine zweite Klasse: Taucht in Monat 16 der neue Tarif *mobil* auf, liefert die Pipeline weiter Werte, während der Notebook-Weg mit `get_dummies` mit einem `ValueError` abbricht. Der Abbruch ist laut, die Pipeline leise; Schritt 8 zeigt, dass auch „ignorieren" seinen Preis hat.

12.7 Schritt 6: Warum steht dieser Kunde auf der Liste? (Explainable Predictions)

Das Retention-Team stellt zwei Fragen. Die erste ist global: Woran erkennt das Modell Abwanderer, und passt das zu dem, was das Team über seine Kunden weiß? Die zweite ist lokal: Warum steht ausgerechnet dieser Kunde auf der Liste, und was soll das Angebot sagen? Beides beantwortet das Pattern *Explainable Predictions* (Pattern 29) mit zwei Werkzeugen (`06_erklaerung.py`).

Global: Permutation Importance. Man mischt auf den Testdaten die Werte einer Spalte zufällig durcheinander und misst, wie stark die PR-AUC sinkt. Eine Spalte, auf die sich das Modell stützt, hinterlässt eine Lücke; eine ignorierte Spalte nicht (Abbildung 90). Das Ergebnis ist plausibel: Wer lange nicht eingeloggt war (`tage_seit_login`) und weniger schaut als sonst (`stunden_trend`), ist gefährdet. Dahinter folgen die Dauer der Kundenbeziehung, der Kanal (Aktionskunden mit ihrem Preissprung nach drei Monaten), offene Supportfälle und das Alter. Dass `stunden` und `sessions` kaum zählen, heißt nicht, dass sie unwichtig sind: Sie sind mit Trend und Login korreliert, und bei korrelierten Spalten kann die Methode jede einzelne unterschätzen (siehe Pattern 29).

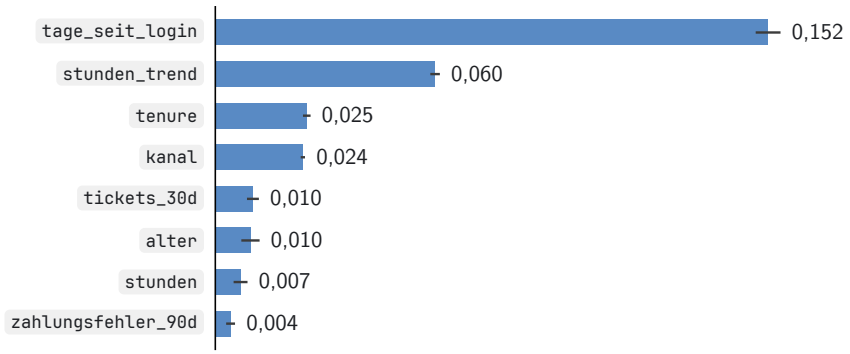


Abbildung 90: Permutation Importance des Modells (Testkunden): Abfall der PR-AUC beim Durchmischen der Spalte, die acht wichtigsten Spalten. Die dünnen Linien zeigen die Streuung über fünf Wiederholungen.

Debugging. Die Erklärung hat noch eine zweite, unspektakuläre Aufgabe. Das Modell aus Schritt 5 enthielt `anmeldetag` und `bandbreite_mbit`, die Fingerabdruck-Spalten aus Schritt 3. Ihre Wichtigkeit ist winzig (0,0044 und −0,0016), und ohne sie ändert sich die PR-AUC nicht (0,292 mit, 0,293 ohne). Eine Spalte, die für jeden Kunden konstant ist und keinen fachlichen Bezug zur Kündigung hat, kann nichts verallgemeinern, sondern nur Kunden wiedererkennen. Sie fliegt raus, bevor ein späterer Split sie wieder zum Leck macht; die weiteren Schritte verwenden das Modell ohne sie (`modell_v1.joblib`).

Lokal: Beiträge für einen Kunden. Für die einzelne Entscheidung braucht es eine Zerlegung der Vorhersage auf die Merkmale. Der exakte Weg über alle 2^{14} = 16 384 Teilmengen von 14 Merkmalen wäre je Kunde sehr aufwendig; das Skript schätzt die Shapley-Werte stattdessen durch Stichproben (Vorgehen nach E. Štrumbelj und I. Kononenko [96]). Es zieht für jede Reihenfolge eine zufällige Zeile aus 2000 Trainingszeilen als Hintergrund und ersetzt deren Merkmale in zufälliger Reihenfolge nacheinander durch die des zu erklärenden Kunden. Der Sprung der Vorhersage bei jedem Ersetzen wird dem Merkmal gutgeschrieben; über 400 Reihenfolgen ergibt sich der Mittelwert. Die Beiträge summieren sich so exakt zur Differenz zwischen der Vorhersage für den Kunden und dem Mittel der Hintergrundkunden, wie die Zeile „Summe der Beiträge" in der Ausgabe zeigt. Die Spalte `andere_ziehung` wiederholt die Schätzung mit anderen Zufallszahlen; die sieben gezeigten Werte weichen um höchstens 0,02 voneinander ab, die Rangfolge ändert sich nicht.

```
def shapley_lokal(pipe, x, hintergrund, n_reihen=400, seed=0):
    rng = np.random.default_rng(seed)
```

```
    d = len(ALLE)
    zeilen = rng.integers(len(hintergrund), size=n_reihen)
    basis = hintergrund[ALLE].to_numpy(dtype=object)[zeilen]
    reihen = np.array([rng.permutation(d) for _ in range(n_reihen)])
    xv = x[ALLE].to_numpy(dtype=object)
    # schritte[i, k]: Hintergrundkunde i, bei dem die ersten k Merkmale seiner
    # Reihenfolge durch die Werte von x ersetzt sind
    schritte = np.repeat(basis[:, None, :], d + 1, axis=1)
    for k in range(d):
        i, j = np.arange(n_reihen), reihen[:, k]
        schritte[i, k + 1:, j] = xv[j][:, None]
    tab = pd.DataFrame(schritte.reshape(-1, d), columns=ALLE).infer_objects()
    p = pipe.predict_proba(tab)[:, 1].reshape(n_reihen, d + 1)
    phi = np.zeros(d)
    for k in range(d):
        np.add.at(phi, reihen[:, k], p[:, k + 1] - p[:, k])
    return phi / n_reihen, p[:, 0].mean(), p[0, -1]
```

Terminal

```
$ python 06_erklaerung.py
Modell aus Schritt 5, Wichtigkeit von anmeldetag: +0.0044
Modell aus Schritt 5, Wichtigkeit von bandbreite_mbit: -0.0016
PR-AUC Test: mit Fingerabdruck-Spalten 0.292, ohne 0.293

Kunde K016338, Monat 10: Vorhersage 0.603, Basis 0.050
Summe der Beiträge +0.5522 = Vorhersage - Basis +0.5522
        merkmal    wert  beitrag  andere_ziehung
  stunden_trend   0.429   +0.313          +0.330
tage_seit_login       5   +0.166          +0.160
    tickets_30d       1   +0.083          +0.077
          kanal  aktion   +0.032          +0.028
          tarif premium   -0.018          -0.017
          preis   17.99   -0.014          -0.012
          alter      36   -0.013          -0.010
Rest (7 Merkmale): +0.002
```

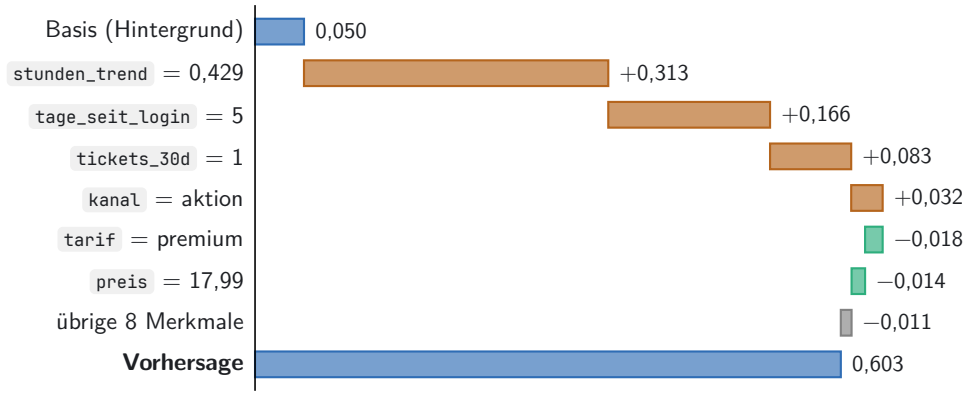


Abbildung 91: Beiträge der Merkmale zur Vorhersage für Kunde K016338 in Monat 10 (Altersgruppe 25-39, ein Abwanderer, den das Modell markiert). Orange erhöht das Risiko gegenüber dem Hintergrund, grün senkt es. Werte aus der Ausgabe oben.

Für das Retention-Team lässt sich das in einen Satz übersetzen: Dieser Kunde hat im letzten Monat nur 43 % der Stunden geschaut, die er im Schnitt der drei Vormonate geschaut hatte, war vor 5 Tagen zuletzt im Dienst und hat sich in den letzten 30 Tagen an den Support gewandt; dazu kommt, dass er über eine Aktion gewonnen wurde. Das ist ein Anlass für ein Gespräch statt eines Pauschalrabatts. Die Grenzen sind die bekannten: Die Erklärung beschreibt das *Modell*, nicht die Kündigungsgründe des Kunden; sie hängt vom Hintergrund ab, und weil Merkmale einzeln ausgetauscht werden, bildet sie auch Kombinationen, die es in Wirklichkeit nicht gibt. Als Gesprächsanlass ist sie wertvoll, als Kausalaussage nicht.

12.8 Schritt 7: Für wen funktioniert das Modell? (Fairness Lens)

Bis hierher stand pro Kennzahl nur eine Zahl in der Tabelle, ein Durchschnitt über alle. Ein Durchschnitt kann ein Loch verbergen. Die *Fairness Lens* (Pattern 30) wertet dieselben Kennzahlen getrennt nach Gruppen aus, hier nach Altersgruppe: Das Alter ist ein Merkmal des Modells und spielt in der Kundenansprache eine nachvollziehbare Rolle. Zuerst die Auswertung mit einer Schwelle für alle, dann eine Gegenmaßnahme: Gruppen, deren Recall auf den Validierungsdaten unter dem Gesamtwert liegt, bekommen eine niedrigere Schwelle, alle anderen behalten die gemeinsame.

```
ziel_recall = kennzahlen(va[ZIEL], va.p, SCHWELLE)["recall"]
schwellen = {}
for g in GRUPPEN:
    z = va[va.altersgruppe == g]
    kandidaten = np.unique(np.round(z.p, 4))[::-1]
    if kennzahlen(z[ZIEL], z.p, SCHWELLE)["recall"] >= ziel_recall:
        schwellen[g] = SCHWELLE
    else:
        schwellen[g] = next(float(s) for s in kandidaten
                            if kennzahlen(z[ZIEL], z.p, s)["recall"] >= ziel_recall)
```

Terminal

```
$ python 07_fairness.py
Eine Schwelle für alle (0.278), Testkunden
gruppe     n basisrate markiert precision recall   fpr    ap nutzen
 18-24  8346     0.056    0.019     0.720  0.248 0.006 0.352    154
 25-39 19442     0.037    0.014     0.564  0.217 0.006 0.274     73
 40-54 19856     0.047    0.022     0.607  0.284 0.009 0.337    130
   55+  9414     0.049    0.008     0.333  0.052 0.005 0.142      8
gesamt 57058     0.045    0.017     0.593  0.218 0.007 0.293     94

Ziel-Recall (Validierung): 0.222
Recall je Gruppe (Validierung): 18-24 0.17, 25-39 0.29, 40-54 0.27, 55+ 0.04
Schwellen je Gruppe: 18-24 0.181, 25-39 0.278, 40-54 0.278, 55+ 0.109
gruppe     n basisrate markiert precision recall   fpr    ap nutzen
 18-24  8346     0.056    0.033     0.514  0.303 0.017 0.352    141
 25-39 19442     0.037    0.014     0.564  0.217 0.006 0.274     73
 40-54 19856     0.047    0.022     0.607  0.284 0.009 0.337    130
   55+  9414     0.049    0.066     0.178  0.241 0.057 0.142   -118
gesamt 57058     0.045    0.028     0.418  0.262 0.017 0.293     71
```

Die erste Tabelle zeigt ein deutliches Bild. Bei den Kunden ab 55 findet das Modell nur 5 % der Abwanderer, bei den anderen Gruppen zwischen 22 % und 28 %. Die Gruppe wird nur zu 0,8 % angesprochen (die anderen zu 1,4 % bis 2,2 %), obwohl ihre tatsächliche Abwanderungsquote mit 4,9 % sogar leicht über dem Durchschnitt von 4,5 % liegt. Weil die Daten simuliert sind, kennen wir die Ursache: In `daten.py` verändern Kunden ab 55 ihre Nutzung vor einer Kündigung nur etwa ein Drittel so stark wie die übrigen. Ein Modell, das fast nur auf Nutzungsrückgänge schaut, hat bei ihnen keine Chance. In echten Daten müsste man diese Ursache erst suchen.

Welches Kriterium ist das richtige? Man fragt nach dem Schaden. Ein Halteangebot ist ein Vorteil, und übersehen zu werden ist der Nachteil, den man vermeiden will. Das spricht für *Equal Opportunity* nach M. Hardt, E. Price, und N. Srebro [74]: Von allen, die tatsächlich kündigen, soll jede Gruppe gleich viele erreichen. Die zweite Tabelle zeigt den Preis. Der Recall der Gruppe ab 55 steigt von 5 % auf 24 %, aber ihre Precision fällt von 33 % auf 18 %: Weniger als jedes fünfte Angebot erreicht einen Abwanderer. Das liegt unter der Rentabilitätsschwelle von 28 %, die Kampagne verliert in dieser Gruppe 118 € je 1000 Kunden-Monate, und der Gesamtnutzen sinkt von 94 € auf 71 €, also um 24 %. Auch die 18- bis 24-Jährigen bekommen eine niedrigere Schwelle (0,18), weil ihr Recall auf den Validierungsdaten mit 17 % unter dem Ziel von 22 % lag. Auf den Testdaten lag er mit 25 % schon vorher darüber: Das Verfahren folgt hier der Zufallsschwankung der Validierung, und die Gruppe zahlt dafür (Precision 72 % auf 51 %, Nutzen 154 € auf 141 €). Das spricht dafür, Schwellen je Gruppe nur bei deutlichen Abständen anzupassen.

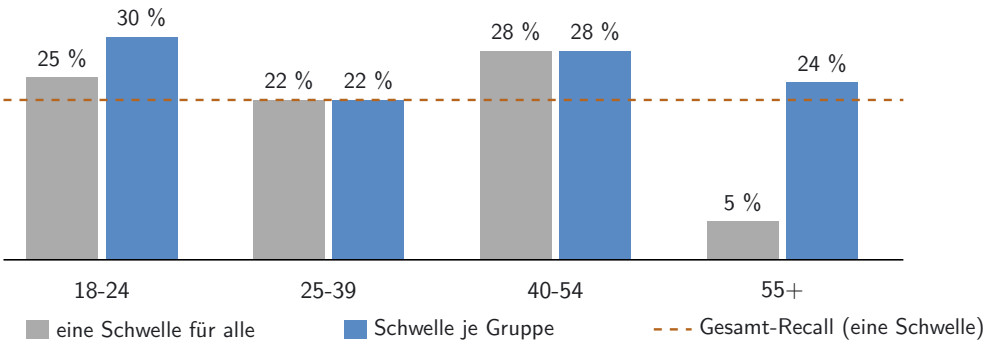


Abbildung 92: Recall je Altersgruppe auf den Testkunden, vor und nach der gruppenspezifischen Schwelle. Orange gestrichelt: Gesamt-Recall mit einer gemeinsamen Schwelle.

Ob man diese Rechnung bezahlen will, ist keine Frage für das Data-Science-Team allein. Vertretbar sind drei Antworten: die Unterschiede dokumentieren und akzeptieren, die Schwelle je Gruppe verwenden und die Kosten bewusst tragen, oder die Ursache beheben, indem man Merkmale findet, die Kündigungen dieser Gruppe ankündigen (etwa Zahlungsereignisse oder die Nutzung am Fernseher statt in der App). Die dritte ist die beste, aber nicht die schnellste. Ob die Verwendung des Alters für Entscheidungsschwellen zulässig ist, ist keine ML-Frage, sondern eine rechtliche, und bleibt hier offen. Gleichstellen lässt sich außerdem nicht alles: Bei unterschiedlichen Basisraten lassen sich gleiche Precision, gleicher Recall und gleiche Rate falscher Alarme im Allgemeinen nicht gleichzeitig erreichen, wie A. Chouldechova [76] gezeigt hat; J. Kleinberg, S. Mullainathan, und M. Raghavan [75] kommen für Risikoscores zu einem verwandten Ergebnis. In jedem Fall gehört das Ergebnis in die Dokumentation des Modells, die *Model Card* nach M. Mitchell *u. a.* [62].

12.9 Schritt 8: Das Modell im Betrieb (Continued Model Evaluation)

Das Modell geht in Monat 13 in den Betrieb. Jeden Monatsanfang bewertet ein Batch-Job alle aktiven Kunden (Pattern 17), das Retention-Team erhält die Liste. Was fehlt, ist die Antwort auf die Frage „Funktioniert es noch?“, und die ist hier schwerer als üblich, weil die Wahrheit mit Verzögerung eintrifft: Den Recall der Zeilen aus Monat *m* kennt man erst in Monat *m* + 3. Das Pattern *Continued Model Evaluation* (Pattern 18) unterscheidet deshalb zwei Wege. Eingaben und Vorhersagen lassen sich sofort überwachen, die Qualität nur gegen die später bekannten Ziele. `08_monitoring.py` wertet beides für die Monate 13 bis 20 aus, ausschließlich auf Testkunden, die das Modell nie gesehen hat. Die Verschiebung einer Verteilung misst der *Population Stability Index* (PSI) mit der verbreiteten Faustregel 0,25 als Alarmschwelle (sie stammt aus der Kreditrisikomodellierung und hat keine theoretische Begründung).

```
def psi(referenz, aktuell, kategorisch=False, bins=10):
    """Population Stability Index (Dezile der Referenz bzw. Kategorien)."""
    if kategorisch:
        kat = sorted(set(referenz) | set(aktuell))
        a = np.array([(referenz == k).mean() for k in kat])
        b = np.array([(aktuell == k).mean() for k in kat])
    else:
        q = np.linspace(0, 1, bins + 1)[1:-1]
        grenzen = np.unique(np.quantile(referenz, q))
        k = len(grenzen) + 1

        def anteile(x):
            n = np.bincount(np.searchsorted(grenzen, x), minlength=k)
            return n / n.sum()

        a, b = anteile(referenz), anteile(aktuell)
    a, b = np.maximum(a, 1e-4), np.maximum(b, 1e-4)
    return float(((b - a) * np.log(b / a)).sum())
```

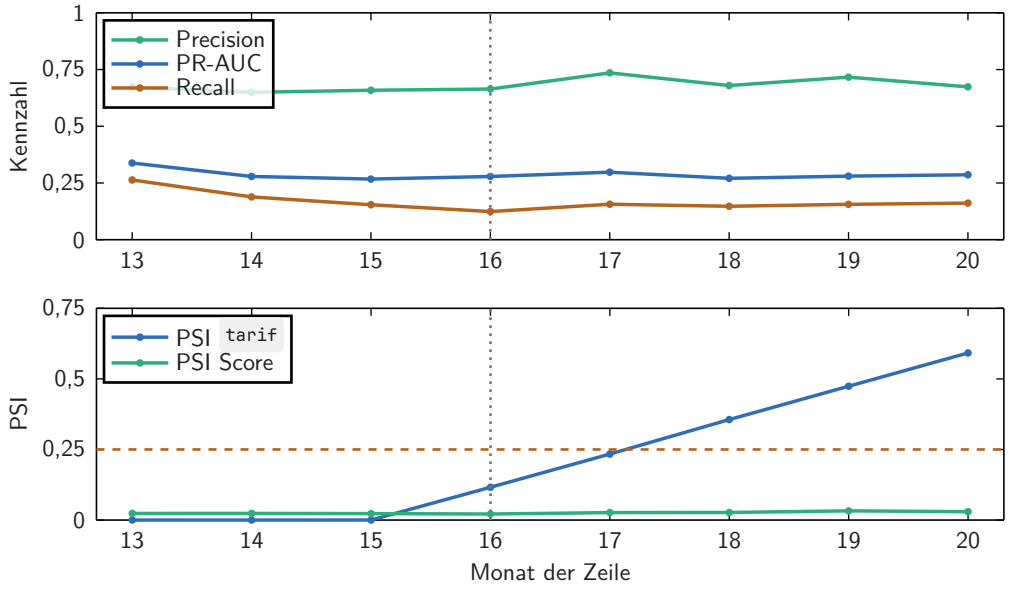


Abbildung 93: Modell v1 im Betrieb (Testkunden). Oben: Precision, PR-AUC und Recall bei Schwelle 0,28 je Zeilenmonat. Unten: PSI des Tarifs und der Modellausgabe gegenüber den Validierungskunden; orange gestrichelt die Faustregel 0,25. Der PSI des Preises (3,2 bis 3,4 ab Monat 16) liegt weit über dem Bild und ist nicht eingezeichnet. Gepunktet: Monat 16.

Terminal

```
$ python 08_monitoring.py
Modell v1 im Betrieb (Testkunden, Schwelle 0.278); Drift ab Monat 16
 monat basisrate    ap recall precision nutzen psi_tarif psi_preis psi_score
    13     0.044 0.338  0.263     0.673    121     0.000     0.001     0.023
    14     0.060 0.279  0.189     0.650    116     0.000     0.002     0.024
    15     0.077 0.267  0.154     0.659    123     0.000     0.002     0.023
    16     0.092 0.279  0.124     0.664    120     0.116     3.199     0.021
    17     0.086 0.298  0.156     0.735    151     0.234     3.290     0.026
    18     0.080 0.270  0.147     0.679    126     0.356     3.409     0.027
    19     0.076 0.280  0.156     0.717    131     0.474     3.446     0.033
    20     0.075 0.286  0.161     0.674    129     0.592     3.442     0.030

Monate 18-20: v1 gegen v2 (v2 trainiert auf Monaten 8-17)
modell  nutzen    ap  precision  recall    f1
    v1 128.618 0.278      0.689   0.155 0.253
    v2 130.011 0.300      0.569   0.182 0.276
```

Die Eingaben schlagen sofort an, die Ausgabe nie. Der PSI des Preises springt in Monat 16 auf 3,2, denn die Preiserhöhung verschiebt die Verteilung auf einen Schlag. Der PSI des Tarifs wächst schleichend (0,12 in Monat 16, 0,59 in Monat 20) und überschreitet die Schwelle von 0,25 erst in Monat 18, weil Neukunden den Tarif *mobil* wählen, den das Modell nie gesehen hat. Die Verteilung der Scores bleibt bei höchstens 0,03. Wer nur die Ausgabe überwacht, merkt nichts. Die Pipeline verarbeitet den unbekannten Tarif ohne Fehler (`handle_unknown="ignore"`), das Modell rechnet mit lauter Nullen in den Tarif-Spalten. Das ist bequem und verdeckt, dass etwas Neues im System ist.

Die Qualität sinkt, aber nicht dort, wo man zuerst schaut. Die Precision bleibt zwischen 0,65 und 0,74: Was das Modell meldet, stimmt weiter. Die PR-AUC fällt von 0,34 in Monat 13 auf 0,27 bis 0,30 in den späteren Monaten, bleibt aber weit über der Basisrate von 8 % bis 9 %: Die Rangfolge der Kunden taugt weiter. Der Recall dagegen fällt von 26 % in Monat 13 auf 12 % in Monat 16. Die Abwanderungsquote hat sich mehr als verdoppelt (4,4 % auf 9,2 %), aber die zusätzlichen Kündigungen kommen von Kunden der Tarife standard und premium, die auf die Preiserhöhung reagieren (in `daten.py` ein nicht sichtbarer Teil von ihnen, dazu ältere Kunden), und ihre Nutzung nimmt vorher nicht ab. Das Modell sieht sie nicht. Der Nutzen der Kampagne bleibt dabei bei 116 € bis 151 € je 1000 Kunden-Monate, weil er nur die Angesprochenen zählt: Die Zahl der gefundenen Abwanderer je 1000 Zeilen liegt in allen Monaten zwischen 11 und 14, während es mehr Abwanderer gibt. Das Modell findet ähnlich viele und übersieht mehr. Eine Kennzahl, die nur das Erreichte misst, kann das Verpasste nicht sehen. Dass schon die Zeilen aus den Monaten 14 und 15 mehr Abwanderer enthalten, liegt am Horizont: Ihre Ziele reichen bis in die Monate 16 und 17.

Der Alarm kommt drei Monate vor dem Beweis. Der Preis-PSI schlägt in Monat 16 an, den Recall dieses Monats kennt man erst in Monat 19. Dazwischen weiß man, dass sich etwas geändert hat, aber nicht, ob es dem Modell schadet. Die Eingabeüberwachung löst also eine Untersuchung aus, kein automatisches Retraining.

Neu trainieren hilft hier nur wenig. In Monat 20 sind die Ziele aller Zeilen bis Monat 17 bekannt. Das Modell v2 wird auf den Monaten 8 bis 17 der Trainingskunden gelernt und mit v1 auf den Monaten 18 bis 20 der Testkunden verglichen. v2 findet etwas mehr Abwanderer (Recall 15 % auf 18 %), verliert dafür Precision (69 % auf 57 %), und der Nutzen bleibt praktisch gleich (129 € gegenüber 130 €). Preisempfindlichkeit ist kein Merkmal der Tabelle, und ein Retraining kann keine Information erzeugen, die nicht in den Spalten steckt. Die Antwort auf diesen Alarm liegt eher bei den Daten und der Fachseite: ein Merkmal für die Reaktion auf Preisankündigungen oder eine gezielte Aktion rund um Preisänderungen. Auch die Fenstergröße des Retrainings (hier zehn Zeilenmonate, davon nur zwei ab dem Preissprung) ist eine eigene Entscheidung.

12.10 Das fertige System

Aus den acht Schritten ist ein System geworden, dessen Teile jeweils einem Pattern entsprechen (Abbildung 94). Der monatliche Ablauf ist ein Workflow (Pattern 25): Features berechnen, mit dem versionierten Pipeline-Artefakt (Pattern 27, 21) alle aktiven Kunden im Batch bewerten (Pattern 17), mit der Kostenschwelle (Pattern 10) eine Liste erzeugen und jedem Kunden darauf die drei wichtigsten Beiträge mitgeben (Pattern 29). Darunter läuft die Überwachung: PSI auf den Eingaben sofort, Recall und Precision je Altersgruppe (Pattern 30) mit drei Monaten Verzögerung. Der Heuristic Benchmark (Pattern 28) ist Teil des Freigabetests: Ein neues Modell sollte nur live gehen, wenn es auf den Hash-getrennten Testkunden (Pattern 22) die Faustregel und das Vorgängermodell schlägt. Die Skripte führen die Teile einzeln vor; die Verkettung zu einem Monatsjob und der Freigabetest sind Entwurf und nicht als Code ausgeführt. Einen Feature Store (Pattern 26) gibt es nicht; bei einem Monatsjob mit einer Quelle wäre er Aufwand ohne Nutzen.

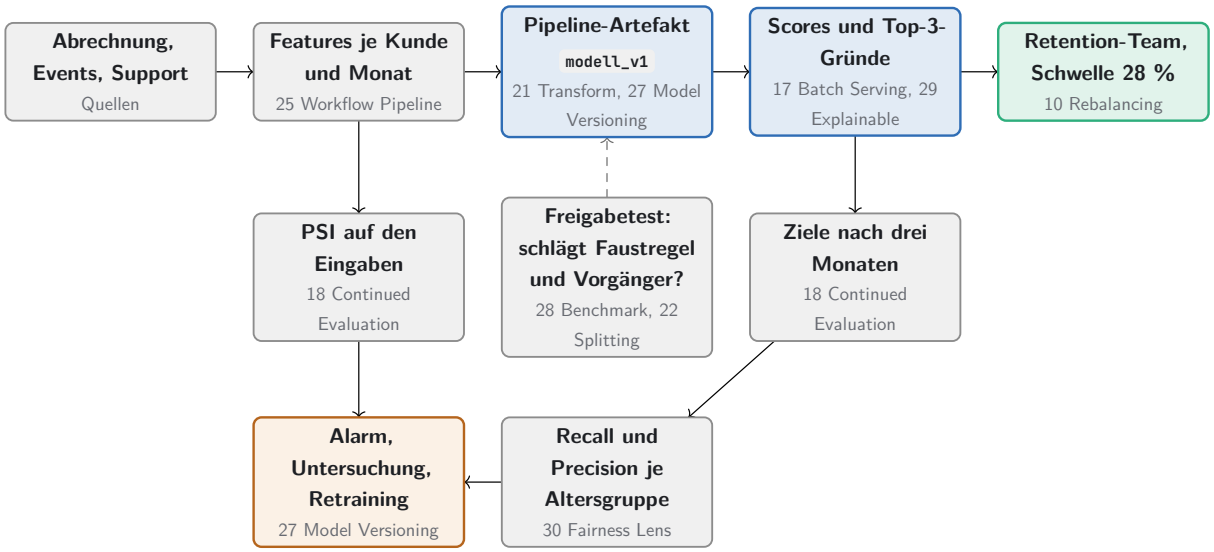


Abbildung 94: Architektur des Systems. Oben der monatliche Ablauf, darunter die Überwachung und der Freigabetest (gestrichelt). Die Zahlen nennen das jeweilige Pattern.

Zum Abschluss die Bilanz aller Schritte. Sie zeigt, dass die Schritte unterschiedliche Größen verbessern und dass die „ehrlichste“ Verbesserung einmal eine schlechtere Zahl war.

Tabelle 15: Schritt, Pattern, Metrik und Zahlen vor und nach dem Schritt (alle aus den Ausgaben der Skripte). Die Zeile zu Schritt 3 zeigt, dass der Schritt eine Zahl *verschlechtert* und das Ergebnis dadurch wahrer macht; Schritt 8 beschreibt eine Verschlechterung im Betrieb und einen schwachen Gegenzug.

Schritt	Pattern	Metrik	vorher	nachher
1	28	Ranking nach Login-Tagen (PR-AUC); Nutzen der Regel	0,045 (Zufall)	0,197 und −333 €
2	–	PR-AUC des Notebooks, Zeilen-Split	0,197	0,385

Schritt	Pattern	Metrik	vorher	nachher
3	22	PR-AUC, dasselbe Modell ehrlich gemessen	0,385	0,258
3	22	PR-AUC nach Modellwahl (Test)	0,258	0,292
4	10	Nutzen je 1000 Kunden-Monate (Recall)	89 € (0,19)	94 € (0,22)
5	21	PR-AUC, Service-Nachbau ohne `log1p`	0,188	0,292
6	29	PR-AUC ohne Fingerabdruck-Spalten	0,292	0,293
7	30	Recall ab 55 (Gesamtnutzen)	0,05 (94 €)	0,24 (71 €)
8	18	Recall v1, Monat 13 und 16	0,26	0,12
8	18	Recall v1 und v2, Monate 18 bis 20	0,15	0,18

12.11 Das Wichtigste in Kürze

- Ein Projekt beginnt mit einem Vergleichsmaßstab. Die Faustregel kostete eine Zeile Code und zeigte, dass Genauigkeit bei einer Quote von 5 % nichts aussagt und eine Kampagne mit zu geringer Precision Geld verliert.
- Zeilen desselben Kunden sind keine unabhängigen Beispiele. Ein Split nach Zeilen lässt ein großes Modell Kunden wiedererkennen, bläht die PR-AUC von 0,26 auf 0,39 auf und wählt das falsche Modell. Der Hash der Kunden-ID trennt sauber und bleibt stabil, wenn Daten dazukommen.
- Gewichte, Downsampling und Schwelle ändern vor allem die Entscheidung, nicht die Rangfolge. Mit Gewichten stimmen die Wahrscheinlichkeiten nicht mehr, und eine Schwelle aus Kosten ist nur ohne sie zulässig.
- Die Vorverarbeitung gehört ins Modell. Ein Nachbau, der `log1p` vergisst, senkt die PR-AUC von 0,29 auf 0,19, ohne eine Fehlermeldung zu erzeugen.
- Erklärungen helfen beim Debuggen und im Gespräch, sie sind keine Kausalaussagen. Gruppenauswertungen zeigen Löcher, die Durchschnitte verdecken; sie zu schließen kostet Geld, das jemand entscheiden muss.
- Im Betrieb schlagen Eingaben sofort an, Qualitätsmaße mit Verzögerung. Ein Alarm ist ein Anlass zur Untersuchung, und ob Retraining hilft, hängt davon ab, ob die fehlende Information in den Spalten steckt.

Grenzen der Fallstudie. Die Daten sind simuliert, und viele Effekte (die Fingerabdruck-Spalten, die „leisen" Kündigungen ab 55, die Preisempfindlichkeit) sind so gebaut, dass sie sichtbar werden. In echten Projekten sind sie kleiner und vermischter. Die Zahlen für Angebotskosten, Erfolgsquote und Kundenwert sind angenommen. Das Kostenmodell zählt jede Zeile als eigenen Kontakt; dass derselbe Kunde in drei aufeinanderfolgenden Monaten angesprochen würde, ist nicht modelliert. Alle Zahlen stammen aus einem Lauf mit festem Seed. Schwankungen zwischen Läufen wurden nicht gemessen, kleine Unterschiede (etwa zwischen den Varianten A, D und F) sind deshalb nicht belegt. Die Beispiele sind je Schritt mit einem Modell und einer Metrik gerechnet; weder wurden Hyperparameter systematisch gesucht noch Alternativen wie eine Überlebenszeitanalyse geprüft, die für Kündigungen oft die natürlichere Formulierung ist.

> **SO ERKLÄRE ICH ES**
> Ich lasse die Studierenden die Schritte in der Reihenfolge erleben und halte vor jedem Schritt an: „Welche Zahl würden Sie jetzt glauben?" Der stärkste Moment ist Schritt 3: Dasselbe Modell auf denselben Daten verliert durch eine andere Trennlinie ein Drittel seiner PR-AUC. Danach lasse ich sie erklären, warum das große Modell auf dem Zeilen-Split gewinnt und auf dem Kunden-Split nicht.
>
> **Verständnisfrage:** Ein Modell zeigt auf den Testdaten eine PR-AUC von 0,39, aber im Betrieb nur 0,26. Nennen Sie zwei mögliche Ursachen, die nichts mit dem Modell selbst zu tun haben.
> **Antwort:** Erstens ein Split, der Zeilen desselben Kunden auf Training und Test verteilt (Leck durch zusammengehörige Zeilen). Zweitens ein Unterschied zwischen Aufbereitung im Training und im Betrieb (Training-Serving Skew). Drittens eine Veränderung der Welt, die im Test noch nicht vorkam, oder eine andere Basisrate im Betrieb, die die PR-AUC allein schon verschiebt.

Quellen und weiterführende Literatur. Die Patterns stammen aus V. Lakshmanan, S. Robinson, und M. Munn [1] (Kapitel 3: Pattern 10; Kapitel 5: 17 und 18; Kapitel 6: 21, 22 und 25 bis 27; Kapitel 7: 28 bis 30). Leckagen (Leakage) behandeln S. Kaufman, S. Rosset, C. Perlich, und O. Stitelman [8] mit Beispielen

aus Data-Mining-Wettbewerben; die Wahl zwischen ROC- und PR-Kurve bei unausgewogenen Daten begründen T. Saito und M. Rehmsmeier [10], die Herleitung von Entscheidungsregeln aus Kosten C. Elkan [97]. Zu Shapley-Beiträgen siehe S. M. Lundberg und S.-I. Lee [66] und E. Štrumbelj und I. Kononenko [96]. Die Permutation Importance stammt von L. Breiman [4] (für Random Forests), A. Fisher, C. Rudin, und F. Dominici [64] verallgemeinern sie auf beliebige Modelle. Zur Gruppenfairness siehe M. Hardt, E. Price, und N. Srebro [74], J. Kleinberg, S. Mullainathan, und M. Raghavan [75] und A. Chouldechova [76], zum Betrieb D. Sculley *u. a.* [11] und die Prüfliste von E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55], zu den Arten von Drift J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, und A. Bouchachia [12].

13 Fallstudie B: Vorausschauende Wartung von Industriepumpen

Die erste Fallstudie arbeitet mit Tabellen: eine Zeile pro Kunde, Merkmale daneben, ein Etikett dahinter. Hier geht es um eine andere Sorte Daten, nämlich **Zeitreihen**. Ein Sensor liefert in festem Takt Messwerte, und die Reihenfolge der Werte ist Teil der Information. Das verändert, wie man Daten aufteilt, wie man Merkmale berechnet und was im Betrieb schiefgehen kann. Die meisten dieser Stolperstellen sind Patterns aus dem Buch von V. Lakshmanan, S. Robinson, und M. Munn [1], und an einer einzigen Aufgabe lässt sich zeigen, wie sie ineinandergreifen.

Die Aufgabe: Ein Chemiepark betreibt 30 Kreiselpumpen. Jede meldet jede Minute drei Werte, nämlich Schwingungsstärke (Vibration), Lagertemperatur und Förderdruck. Fällt eine Pumpe unvorbereitet aus, steht ein Teil der Anlage still. Kündigt sich der Ausfall dagegen einen Tag vorher an, lässt sich der Austausch in die nächste Schicht legen. Das Modell soll deshalb alle zehn Minuten beantworten, ob die Pumpe **in den nächsten 24 Stunden ausfällt**. Die Fachbezeichnung für dieses Vorgehen ist **Predictive Maintenance** (vorausschauende Wartung); einen Überblick über Methoden geben T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, und S. G. S. Alcalá [98].

> **LERNZIELE**
> Sie können erklären, warum Zeitreihen nach Pumpe und Zeit getrennt werden müssen, und einen Fehler in einem Fenstermerkmal finden, der in die Zukunft schaut. Sie können dieselbe Aufgabe als Regression und als Klassifikation formulieren und an den Entscheidungen messen, nicht an der Fehlerzahl. Sie können nachrechnen, wann sich ein zweistufiges Modell am Gerät lohnt, und Sie wissen, warum parallel berechnete Vorhersagen einen Schlüssel und ein langes Training Zwischenstände brauchen.

13.1 Die Aufgabe und ihre Daten

Drei Arten, Pumpen zu warten

Bei der **reaktiven** Wartung repariert man, was kaputtgeht: billig, solange nichts passiert, und teuer, wenn doch. Die **vorbeugende** Wartung tauscht Teile nach festem Plan aus, etwa alle sechs Monate, und wirft damit auch gesunde Lager weg. Die **vorausschauende** Wartung misst den Zustand und tauscht, wenn die Messwerte es verlangen. Dafür braucht man eine Vorhersage: Wie wahrscheinlich ist ein Ausfall in naher Zukunft?

Um die Qualität dieser Vorhersage nicht nur in Prozentpunkten, sondern in Geld auszudrücken, legen wir drei Kosten fest. Ein ungeplanter Ausfall kostet 18 000 Euro (Stillstand, Notfalleinsatz, Folgeschäden), ein geplanter Austausch nach einem Alarm 2 500 Euro, und ein Fehlalarm, bei dem ein Techniker vergeblich ausrückt, 1 500 Euro. Das sind keine Branchenwerte, sondern **Annahmen**, die das Beispiel rechenbar

Tabelle 16: Wegweiser durch das Kapitel. Die Nummern sind die Pattern-Nummern des Buches.

Abschnitt	Pattern	Frage
Aufteilen	22 Repeatable Splitting	Wie trennt man Training und Test sauber?
Das Fenster	24 Windowed Inference	Wie bleiben Fenstermerkmale in Training und Betrieb gleich?
Die Frage stellen	5 Reframing	Restlebensdauer schätzen oder „Ausfall in 24 h“ ankreuzen?
Zwei Stufen	19 Two-Phase Predictions	Was hält ein kleines Modell am Gerät von der Cloud fern?
Einzeln oder Paket	16, 17 Serving	Wann rechnet man online, wann im Batch?
Schlüssel	20 Keyed Predictions	Wie ordnet man Vorhersagen zu, die vertauscht ankommen?
Zwischenstände	12 Checkpoints	Wie übersteht ein langes Training den Verlust eines Rechners?

machen. In einem echten Projekt liefert sie die Instandhaltung, und die gesamte Auswertung dieses Kapitels lässt sich mit anderen Zahlen wiederholen (`auswertung.py`).

Synthetische Sensordaten

Die Daten sind erzeugt, und das aus gutem Grund. Reale Pumpendaten sind vertraulich und selten frei verfügbar. Bei simulierten Daten kennen wir außerdem die Wahrheit: Wir wissen, wann die Degradation beginnt und wie stark sie ist, und können deshalb prüfen, ob ein Verfahren das Richtige tut. Der Preis: Echte Daten sind schmutziger, mit fehlenden Werten, Sensorausfällen und unprotokollierten Eingriffen. Die Zahlen des Kapitels zeigen deshalb Größenordnungen und Wirkmechanismen, keine Prognosegüte für reale Pumpen. Öffentliche Simulationsdaten gleicher Art sind die C-MAPSS-Daten der NASA [99].

Der Generator (`pumpen.py`, fester Seed) arbeitet so. Jede Pumpe hat eigene Grundwerte für Vibration, Temperatur und Druck. Sie läuft Zyklen von sechs bis vierzehn Tagen bis zum Ausfall. Zwischen 60 und 140 Stunden vor dem Ausfall beginnt die Degradation: Vibration und Temperatur steigen, der Druck sinkt, zunächst langsam, dann beschleunigt, und in den letzten vier Stunden besonders stark. Wie stark jeder Sensor reagiert, ist pro Zyklus verschieden, sodass nicht immer derselbe Kanal verrät, was los ist. Dazu kommen ein Tagesrhythmus der Last, eine langsame Drift und harmlose Lastspitzen, die der Degradation ähneln, aber nichts bedeuten. Nach dem Ausfall folgen acht Stunden Reparatur ohne Messwerte, danach beginnt ein neuer Zyklus mit frischem Lager.

```
Terminal
$ python 01_daten.py
Zeilen: 2.345.379  Pumpen: 30  Tage: 56
Betriebszyklen: 183, davon mit Ausfall: 155
mittlere Laufzeit bis Ausfall: 9.5 Tage
Anteil Minuten mit Label 'Ausfall in 24 h': 9.517%
Merkmalszeilen (alle 10 min, Fenster 24 h): 204.589, davon Positive: 10.9%
```

Das Ergebnis sind knapp 2,35 Millionen Minutenwerte, 183 Betriebszyklen und davon 155 mit Ausfall. Die übrigen 28 sind die letzten, noch laufenden Zyklen am Datenende, bei denen wir den Ausfall nicht beobachtet haben. Abbildung 95 zeigt einen Zyklus der Pumpe P03 in den sechs Tagen vor dem Ausfall.

Was Zeitreihen anders macht

Vier Eigenschaften unterscheiden dieses Problem von einer Kundentabelle, und jede wird später zu einem Pattern.

Erstens sind die Zeilen nicht unabhängig. Zwei Messwerte im Abstand von zehn Minuten sind fast gleich, und sie haben dasselbe Etikett. Zweitens gibt es ein **Jetzt**: Zu jedem Zeitpunkt darf eine Vorhersage nur nutzen, was bis dahin gemessen wurde. Drittens liegt das Etikett in der Zukunft. Ob eine Zeile „Ausfall in

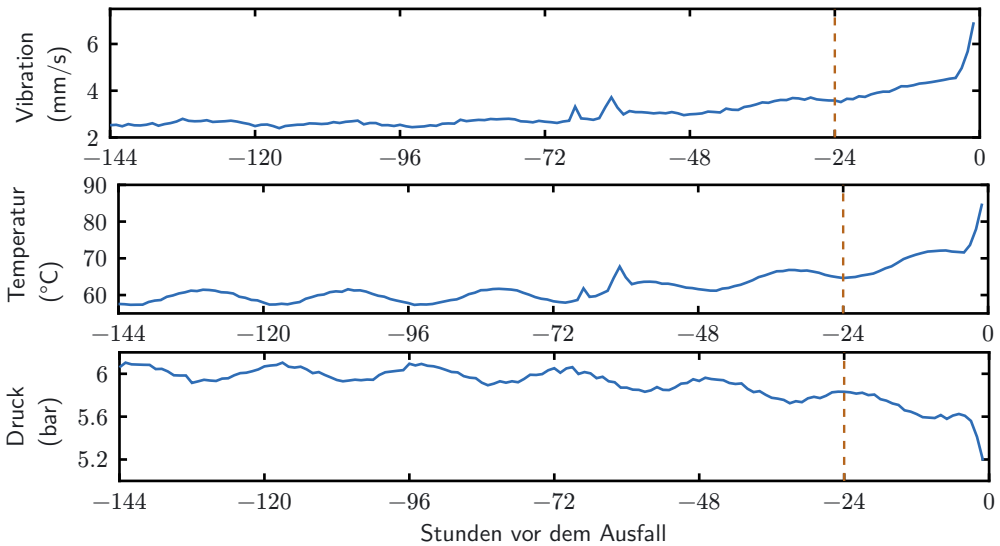


Abbildung 95: Pumpe P03, ein Zyklus bis zum Ausfall (Stundenmittel der Minutenwerte). Links vom orangen Strich liegt „gesund“ im Sinne des Etiketts, rechts davon „Ausfall in den nächsten 24 Stunden“. Der Anstieg beginnt lange vor dem Strich, bleibt aber lange klein gegen das Rauschen.

24 h“ heißt, weiß man erst 24 Stunden später, und am Datenende weiß man es gar nicht. Wir verwerfen diese Zeilen, statt sie als „gesund“ zu zählen. Viertens hat eine Pumpe Zustand: Nach der Reparatur beginnt sie von vorn, und ein Fenster über den Wechsel hinweg mischt zwei verschiedene Pumpen.

Aus den Minutenwerten entstehen alle zehn Minuten Merkmalszeilen. Jede fasst die letzten 24 Stunden einer Pumpe in zwölf Zahlen zusammen: für jeden der drei Sensoren Mittelwert und Standardabweichung, den Abstand zwischen dem Mittel der letzten zwei Stunden und dem 24-Stunden-Mittel (`delta`) sowie die Steigung pro Stunde (`anstieg`). Das sind 204 589 Zeilen, von denen etwa 11 Prozent das Etikett „Ausfall in 24 h“ tragen.

13.2 Pattern 22: Repeatable Splitting

Szenario und Problem

Wir trainieren ein Modell (Gradient Boosting aus scikit-learn, siehe J. H. Friedman [5] und F. Pedregosa *u. a.* [42]) und messen seine Güte mit der **PR-AUC**, der Fläche unter der Precision-Recall-Kurve (genauer: der Average Precision aus scikit-learn). Sie eignet sich für seltene Ereignisse besser als die Genauigkeit. Der Reflex ist `train_test_split`, also 70 Prozent der Zeilen zufällig ins Training und der Rest in den Test. Bei dieser Aufgabe ist das ein Fehler. Die Zeilen einer Pumpe liegen zehn Minuten auseinander und sehen fast gleich aus. Mischt man sie, steht fast zu jeder Testzeile ein Nachbar im Training, der denselben Wert, dieselbe Pumpe und dasselbe Etikett hat. Das Modell muss nichts verallgemeinern; es erkennt wieder, was es kennt.

Wie groß der Effekt ist, zeigt der Vergleich von vier Aufteilungen mit demselben Modell und denselben Daten.

```
def bucket(pumpe, n=10):
    """Stabil ueber Laeufe und Rechner (Pythons hash() ist es nicht)."""
    return int(hashlib.md5(pumpe.encode()).hexdigest(), 16) % n
```

```
test_pumpe = t.bucket >= 7              # 30 % der Pumpen
vor, nach = t.tag < 30, t.tag >= 46     # 3 Tage Luecke > Horizont (24 h)
alle = t.index

schemata = {
    "zufaellig je Zeile": (t.sample(frac=0.7, random_state=0).index, None),
    "nach Pumpe (Hash)": (alle[~test_pumpe], alle[test_pumpe]),
    "zeitlich": (alle[vor], alle[nach]),
    "nach Pumpe und zeitlich": (alle[~test_pumpe & vor], alle[test_pumpe & nach]),
}
```

Terminal

```
$ python 02_splitting.py
Schema                    Training    Test  Positiv  Ausf.  PR-AUC
zufaellig je Zeile          143212   61377    11.0%    155   0.992
nach Pumpe (Hash)           136877   67712    11.5%     54   0.892
zeitlich                    111122   33650    12.2%     29   0.862
nach Pumpe und zeitlich      74551   10781    12.8%     10   0.858
Test-Pumpen: P02 P03 P05 P13 P15 P17 P18 P20 P24 P25
```

Der zufällige Split meldet eine PR-AUC von 0,992, der Wert für fast perfekte Vorhersagen. Teilt man nach Pumpen, so dass ein Drittel der Pumpen dem Modell völlig unbekannt bleibt, fällt der Wert auf 0,892. Trennt man zeitlich, trainiert also auf den ersten 30 Tagen und testet ab Tag 46, sind es 0,862, und beides zusammen ergibt 0,858. Das ist der Wert, mit dem man rechnen darf, wenn das Modell im Betrieb neue Pumpen zu sehen bekommt und die Zukunft vorhersagen soll. Zwischen der ersten und der letzten Zeile liegen 0,13 PR-AUC, und der Unterschied ist kein Zufall: Er entsteht überwiegend dadurch, dass der Test dem Training zu ähnlich ist. (Die Testmengen sind verschieden groß und haben leicht unterschiedliche Anteile an Positiven, 11,0 bis 12,8 Prozent; auch das beeinflusst die PR-AUC, erklärt aber keinen Abstand von 0,13.) Zu zufälligen Splits bei Daten mit Struktur gibt es eine eigene Studie [57], zu Leckagen allgemein

[8], und für Zeitreihen untersucht [100], wann die gewöhnliche Kreuzvalidierung taugt und wann blockweise Verfahren nötig sind.

Lösung

Das Pattern verlangt zweierlei. Die Zuordnung zu Training und Test soll sich nach einem **Schlüssel** richten, der zusammengehörige Zeilen beisammen hält, und sie soll **wiederholbar** sein: Dieselbe Pumpe landet bei jedem Lauf und auf jedem Rechner in derselben Menge, auch wenn neue Pumpen oder neue Tage dazukommen. Beides leistet eine Hashfunktion auf den Schlüssel. Die Pumpen-ID wird gehasht, der Rest bei Division durch 10 ergibt einen Bucket von 0 bis 9, und die Buckets 7 bis 9 sind Testpumpen. Das Buch macht das in SQL mit einer Fingerprint-Funktion; hier steht `md5`. Pythons eingebautes `hash()` taugt nicht, weil es Zeichenketten bei jedem Programmstart anders salzt.

Für die Zeit gilt dieselbe Idee mit einem Stichtag statt eines Hashes. Zwischen den Teilen liegt eine **Lücke**. Sie ist nötig, weil das Etikett einer Zeile aus der Zukunft stammt: Eine Trainingszeile von Tag 29,8 trägt das Etikett „Ausfall in 24 h“, wenn die Pumpe an Tag 30,5 ausfällt, und weiß damit etwas über die Testperiode. Die Lücke muss mindestens so lang sein wie der Vorhersagehorizont. Wir wählen drei Tage zwischen Training, Validierung und Test (Abbildung 96).

Grenzen und Bezug zu Software Engineering

Im strengen Split stehen nur zehn Ausfälle im Test, im zeitlichen 29. Eine PR-AUC auf zehn Ereignissen schwankt stark, und man sollte sie nie ohne Hinweis auf diese Zahl berichten. Ein Zeitsplit setzt außerdem voraus, dass die Vergangenheit der Zukunft ähnelt, und wer mit demselben Test viele Läufe vergleicht, verbraucht ihn. **Wann nicht:** Bei unabhängigen Zeilen ohne Gruppen und ohne Zeitbezug genügt ein einfacher Zufallssplit mit Seed. Die Hash-Zuordnung ist **Hash-Partitionierung** wie beim Sharding von Datenbanken: Der Schlüssel bestimmt den Ort, und neue Datensätze verschieben die alten nicht. Die Lücke vor dem Test ist die Isolation, die man von Testumgebungen verlangt.

Quellen. [1] (Kapitel 6, Pattern 22); [57]; [100]; [8].

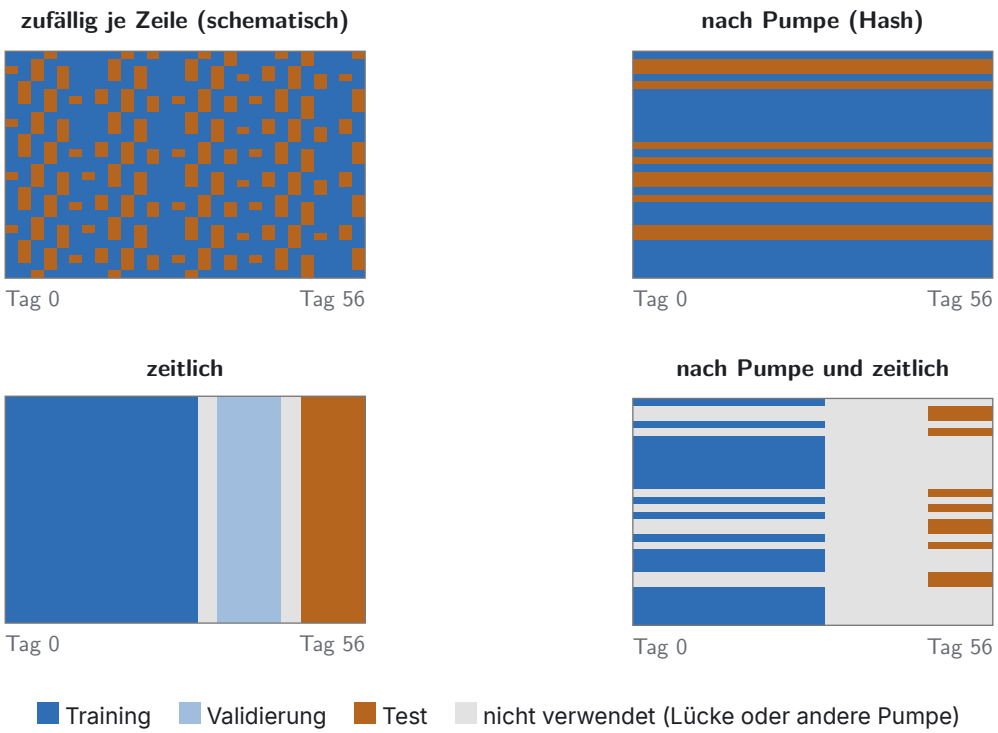


Abbildung 96: Die vier Aufteilungen als Raster aus 30 Pumpen (Zeilen) und 56 Tagen (Spalten). Im Panel „nach Pumpe“ und im Panel „nach Pumpe und zeitlich“ stehen die tatsächlich gehashten Testpumpen; das Raster des zufälligen Splits ist schematisch.

13.3 Pattern 24: Windowed Inference

Szenario und Problem

Das Modell erhält keine Rohwerte, sondern Aggregate über die letzten 24 Stunden. Im Training ist das eine Zeile pandas, im Betrieb jedoch fließen die Messwerte einzeln ein, und jemand muss die letzten 1 440 Werte pro Pumpe vorhalten. Zwei Programme, die dasselbe berechnen sollen, sind eine Einladung zum Skew. Ein klassischer Fehler lässt sich im Notebook mit einem einzigen Argument einbauen:

```
df["vib_mittel"] = df["vib"].rolling(1440, center=True).mean()   # Fenster um t herum
```

Das Argument `center=True` setzt den Zeitpunkt in die Mitte des Fensters, sodass das Fenster zur Hälfte aus der Zukunft besteht. In pandas ist das Standardverhalten das nachlaufende Fenster; `center=True` muss man aktiv anschalten, und es ist für Glättung von Kurven im Nachhinein gedacht. Eine Variante: Man berechnet einen Tageswert über `groupby(Datum)` und ordnet jeder Zeile den Mittelwert des **ganzen** Tages zu, auch der Stunden, die noch kommen. Im Training läuft das ohne Fehlermeldung. Im Betrieb fehlt die Zukunft, und das Modell sieht Merkmale, die es so nie gelernt hat.

Lösung

Das Pattern verlangt, dass die Fenster im Training und im Betrieb **dieselbe** Funktion durchlaufen und dass jedes Fenster am Vorhersagezeitpunkt endet. Im Code gibt es eine Definition der Merkmale, die ein Fenster der letzten 24 Stunden entgegennimmt und zwölf Zahlen liefert:

```
def merkmale(fenster):
    """EINE Definition der Fenstermerkmale. fenster: (n, 3, FENSTER) -> (n, 12)."""
    mittel = fenster.mean(-1)
    std = fenster.std(-1)
    delta = fenster[..., -KURZ:].mean(-1) - mittel   # letzte 2 h gegen 24 h
    t = np.arange(FENSTER) - (FENSTER - 1) / 2
    anstieg = (fenster * t).sum(-1) / (t ** 2).sum() * 60   # pro Stunde
```

Die beiden Wege unterscheiden sich nur darin, wie sie das Fenster beschaffen. Im Training schneidet `sliding_window_view` aus dem ganzen Zyklus Fenster, die bei der Zeile enden. Das Argument `versatz` verschiebt das Fensterende und erlaubt, den Fehler absichtlich einzubauen (`versatz=719` entspricht `center=True`):

```
if versatz > 0:    # Zyklusende auffuellen, wie ffill nach rolling
    x = np.vstack([x, np.repeat(x[-1:], versatz, axis=0)])
fenster = sliding_window_view(x, FENSTER, axis=0)   # (n, 3, FENSTER)
i = np.arange(n)
j = i + versatz - (FENSTER - 1)   # Fenster, das bei i + versatz endet
ok = (minute % SCHRITT == 0) & (j >= 0) & (g["fail24"].to_numpy() >= 0)
```

Im Betrieb hält die Klasse `PumpenStrom` pro Pumpe einen Ringpuffer (`deque` mit `maxlen=1440`). Sie liefert erst dann ein Merkmal, wenn der Puffer voll ist, und wird nach einer Wartung geleert, genau wie der Zyklus im Training endet:

```
def neue_messung(self, minute, werte):
    self.puffer.append(werte)
    if len(self.puffer) < FENSTER or minute % SCHRITT:
        return None
    return merkmale(np.array(self.puffer).T[None])[0]
```

Das Skript prüft die Gleichheit mit einem Äquivalenztest: Ein ganzer Zyklus läuft Minute für Minute durch den Ringpuffer, und jedes Merkmal wird mit der Batch-Variante verglichen. Danach trainiert es das Modell zweimal, auf zentrierten und auf nachlaufenden Fenstern, und wählt die Alarmschwelle jeweils so, dass die Precision auf den Validierungsdaten mindestens 0,80 beträgt.

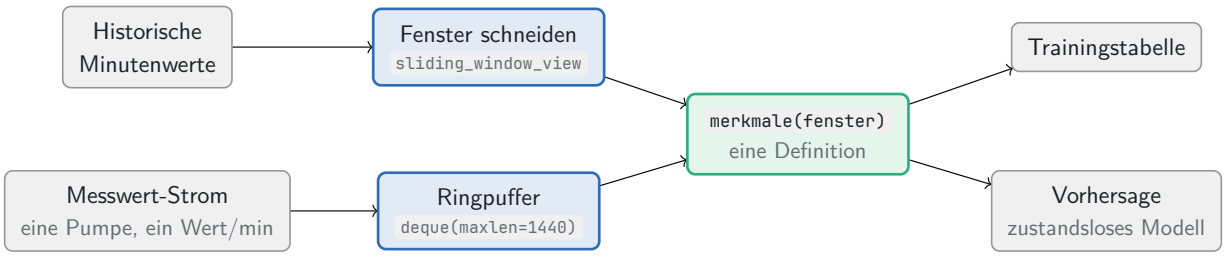


Abbildung 97: Training (oben) und Betrieb (unten) unterscheiden sich nur im Beschaffen des Fensters. Die Berechnung selbst liegt an genau einer Stelle.

```
Terminal
$ python 03_fenster.py
Vorhersagezeitpunkte verglichen: 1510
max. Abweichung Stream vs. Batch (Fenster endet bei t): 4.35e-14
max. Abweichung Stream vs. Batch (Fenster zentriert):   14.32
max. Abweichung vib_mittel zentriert vs. pandas rolling(center=True): 0.0e+00

Modell / Auswertung                          PR-AUC  Precision  Recall
zentriert trainiert, offline (Notebook)       0.924       0.81    0.84
zentriert trainiert, im Stream                0.835       0.93    0.50
nachlaufend trainiert, offline = Stream       0.862       0.83    0.69
(Schwelle: Precision >= 0,80 auf den Validierungsdaten)
Testzeilen: zentriert 35780, nachlaufend 33650
```

Der Stream stimmt mit der nachlaufenden Batch-Variante bis auf Rundungsfehler überein (Abweichung in der Größenordnung 10^{-14}), mit der zentrierten dagegen nicht. Und der Fehler ist teuer, wie die Ausgabe zeigt (Abbildung 98 veranschaulicht die beiden Fensterformen). Das Modell auf zentrierten Fenstern sieht im Notebook am besten aus: PR-AUC 0,924 und ein Recall von 0,84. Im Betrieb bleibt davon wenig. Die PR-AUC fällt auf 0,835, und der Recall sinkt auf 0,50: Nur noch die Hälfte der Zeilen, die zu einem bevorstehenden Ausfall gehören, löst Alarm aus (der Recall ist hier zeilenweise gerechnet). Das Modell, das ehrlich mit nachlaufenden Fenstern gelernt wurde, erreicht offline weniger (0,862), hält diesen Wert aber im Betrieb. Der Abstand zwischen 0,924 und 0,862 geht zum großen Teil auf das zurück, was die Zukunft im Fenster beigetragen hat. Die Testzeilen sind nicht ganz dieselben (35 780 gegen 33 650), weil man bei zentrierten Fenstern das Zyklusende auffüllen muss.

Grenzen und Bezug zu Software Engineering

Das Fenster hält Zustand, und Zustand will behandelt sein: Fällt der Stream-Prozess aus, ist der Ringpuffer weg, und die ersten 24 Stunden nach dem Neustart liefern keine Vorhersage, wenn man ihn nicht aus den Rohdaten wiederaufbaut. Verspätete Messwerte ordnet man nach Ereigniszeit ein, nicht nach Ankunftszeit; dafür gibt es in Stream-Frameworks Watermarks [58]. Wir rechnen alle zehn Minuten die vollen 1 440 Werte neu, das ist für 30 Pumpen billig, bei Hunderttausenden führte man inkrementelle Statistiken.

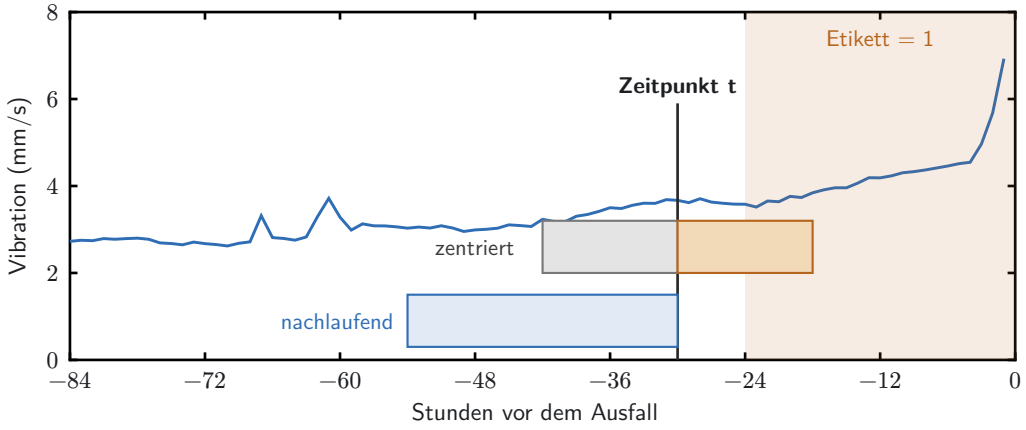


Abbildung 98: Zwei Fenster zum Zeitpunkt t, 30 Stunden vor dem Ausfall (beide 24 h breit, maßstäblich). Das nachlaufende Fenster (blau) endet bei t. Das zentrierte (grau: Vergangenheit, orange: Zukunft) reicht zur Hälfte in die Zukunft und damit in den Bereich, in dem das Etikett 1 ist, obwohl die Zeile selbst noch das Etikett 0 trägt.

Wann nicht: Genügt die aktuelle Zeile, braucht man kein Fenster. Das Pattern ist **Don't Repeat Yourself** für Berechnungen und der Kern von Pattern 21 (Transform); der Äquivalenztest ist ein Konformitätstest zwischen zwei Implementierungen, wie man ihn für Protokolle oder Parser schreibt.

Quellen. [1] (Kapitel 6, Pattern 24); [58]; [54] (Stream-Verarbeitung); [11].

13.4 Pattern 5: Reframing

Szenario und Problem

Der erste Impuls bei einer Wartungsaufgabe ist, die **Restlebensdauer** vorherzusagen (**Remaining Useful Life**, RUL): „Diese Pumpe hält noch 53 Stunden." Das ist eine Regression, also die Vorhersage einer Zahl, und sie klingt nach genau der Auskunft, die der Disponent braucht. Die Alternative ist eine Klassifikation: „Fällt die Pumpe in den nächsten 24 Stunden aus, ja oder nein?" Beide verwenden dieselben Merkmale und lassen sich aus denselben Daten ableiten; wir nehmen nur andere Etiketten. Dieses Umformulieren der Frage ist das Pattern **Reframing** [1].

Die Regression hat ein Problem, das man nicht sofort sieht. Eine Pumpe, die gesund aussieht, kann noch 2 oder noch 9 Tage laufen, und die Messwerte beider Fälle unterscheiden sich kaum, sodass kein Modell das verlässlich trennen kann. Der quadratische Fehler, mit dem die Regression trainiert wird, wird aber von genau diesen unlösbaren Fällen dominiert. Das Modell verschwendet Kapazität auf die Frage, die man nicht beantworten kann, und beantwortet die zweite, die wichtige Frage („kurz vor dem Ausfall?") nebenbei.

Lösung

Wir trainieren drei Modelle auf denselben Merkmalen: eine Regression auf die RUL in Stunden, eine Regression auf die auf 72 Stunden **gekappte** RUL (alles, was länger dauert, zählt als 72) und eine Klassifikation auf „Ausfall in 24 h". Für den Vergleich verwenden wir zwei Maßstäbe. Der erste ist die PR-AUC bezüglich des 24-Stunden-Etiketts, wobei bei den Regressionen die negative vorhergesagte RUL als Score dient. Der zweite ist eine **ereignisbezogene** Auswertung, die zählt, was der Betrieb sieht: erkannte Ausfälle und Fehlalarme. Eine Zeilenzählung ist hier irreführend, denn ein Ausfall erzeugt 144 positive Zeilen. Die Auswertung fasst Alarme zu Episoden zusammen (höchstens sechs Stunden Abstand), zählt einen Ausfall als erkannt, wenn zwischen 24 und einer Stunde vor dem Ausfall ein Alarm liegt, und wertet eine Episode als Fehlalarm, wenn sie mehr als 48 Stunden vor einem Ausfall liegt. Die Schwelle wird auf den Validierungsdaten nach den Kosten gewählt (`ereignis_bilanz` in `auswertung.py`).

Terminal

```
$ python 04_reframing.py
Zeilen Training/Validierung/Test: 111122/37006/20254; Ausfaelle im Test: 29
MAE (RUL roh) bei wahrer RUL   0 bis  24 h:   11.3 h  (4111 Zeilen)
MAE (RUL roh) bei wahrer RUL  24 bis  72 h:   24.3 h  (7239 Zeilen)
MAE (RUL roh) bei wahrer RUL  72 bis 999 h:   40.9 h  (8904 Zeilen)

Ansatz                                  PR-AUC  erkannt  Fehlalarme  Kosten/Test
Regression, Ziel = RUL in h              0.823    29/29           7       83.000
Regression, Ziel auf 72 h gekappt        0.886    29/29           7       83.000
Klassifikation, Ziel = Ausfall in 24 h   0.863    29/29           5       80.000
```

Die Regression auf die rohe RUL zeigt ihr Problem in den Fehlern: In den letzten 24 Stunden vor dem Ausfall liegt sie im Mittel um 11 Stunden daneben, bei Pumpen, die noch mehr als drei Tage laufen, um 41 Stunden. Zwischen den drei Ansätzen sind die Unterschiede in der Entscheidung klein: Alle drei erkennen alle 29 Ausfälle, die Fehlalarme liegen bei 5 bis 7, und die Kosten bei 80 000 bis 83 000 Euro in zehn Testtagen, gegenüber 522 000 Euro ohne Überwachung. Dabei zählen nur Zyklen mit beobachtetem Ausfall; Fehlalarme in Zyklen ohne Ausfall am Datenende fließen nicht ein, die Fehlalarmzahlen sind also Untergrenzen. Bei 29 Ereignissen sind ein oder zwei Fehlalarme mehr oder weniger Rauschen, und niemand sollte aus der Differenz von 3 000 Euro eine Rangfolge ableiten. In der PR-AUC steht die gekappte Regression mit 0,886 vor der Klassifikation (0,863) und der rohen Regression (0,823).

Das Ergebnis spricht nicht gegen das Reframing, sondern gegen die Annahme, die Klassifikation gewinne immer. Entscheidend war, dem Modell die unlösbare Frage zu ersparen, und das gelingt durch Kappen des Ziels ebenso wie durch den Wechsel auf die 24-Stunden-Klassifikation. Die Wahl richtet sich nach der

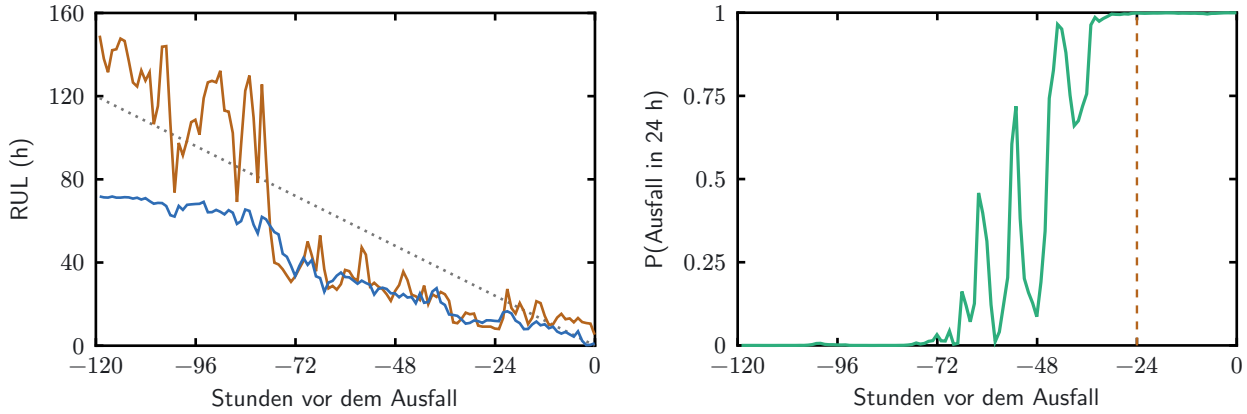


Abbildung 99: Ein Testzyklus, die letzten 120 Stunden. Links: wahre RUL (grau gepunktet), rohe Regression (orange) und auf 72 h gekappte Regression (blau). Rechts: Wahrscheinlichkeit der Klassifikation, mit der 24-Stunden-Grenze (orange gestrichelt).

Entscheidung dahinter: Die Klassifikation liefert eine Wahrscheinlichkeit, an der sich eine Alarmschwelle begründen lässt, und ist dem Techniker leichter zu erklären als eine Zahl, deren Genauigkeit mit dem Abstand zum Ausfall schwankt. Die gekappte Regression ordnet zusätzlich die letzten 72 Stunden, was die Einsatzplanung nutzen kann.

Grenzen und Bezug zu Software Engineering

Die Klassifikation wirft Information weg: Zwischen „in 25 Stunden“ und „in 200 Stunden“ unterscheidet sie nicht, und der Horizont von 24 Stunden ist eine Entscheidung, die man mit der Instandhaltung treffen muss, nicht im Code. Ändert er sich, muss neu trainiert werden. **Wann nicht:** Wenn die Zahl selbst gebraucht wird, etwa die Menge zu bestellender Ersatzteile, ist eine Regression richtig. Reframing ist eine Frage der Schnittstelle: Wie bei einer API fragt man zuerst, was der Aufrufer braucht, und gestaltet dann die Rückgabe. Eine Zahl mit schwankender Genauigkeit ist wie ein Rückgabewert ohne Vertrag.

Quellen. [1] (Kapitel 3, Pattern 5); [99] für RUL als etablierte Fragestellung; [98].

13.5 Pattern 19: Two-Phase Predictions

Szenario und Problem

Die Pumpen stehen in Hallen mit schwachem Mobilfunk. Jede Vorhersage in der Cloud kostet Datenvolumen, Rechenzeit und Strom des Gateways, und sie funktioniert nur bei Verbindung. Gleichzeitig sind etwa die Hälfte aller Zeilen unauffällig: Eine Pumpe mit konstanter Vibration und Temperatur braucht kein großes Modell, um „gesund“ zu hören. Man möchte das große Modell nur befragen, wenn das kleine unsicher ist.

Lösung

Das Pattern stellt zwei Modelle hintereinander. Ein **kleines Modell läuft am Gerät** und beantwortet einen einfachen Teil der Frage, hier: „Sieht das nach Degradation aus?“ Es ist so eingestellt, dass es **kaum etwas übersieht** (hoher Recall), auch wenn es oft zu viel meldet. Nur was es durchlässt, geht an das **große Modell in der Cloud**, das genauer rechnet.

Unser Edge-Modell ist eine logistische Regression auf drei Merkmalen (`vib_delta` , `vib_anstieg` , `temp_delta`). Es besteht aus vier Zahlen und einer Multiplikation, das läuft auch auf einem Mikrocontroller. Die Schwelle wird auf den Validierungsdaten so bestimmt, dass 98 Prozent der positiven Zeilen passieren. Das Cloud-Modell ist das Gradient Boosting aus dem vorigen Abschnitt auf allen zwölf Merkmalen.

```
cloud = HistGradientBoostingClassifier(random_state=0).fit(tr[NAMEN], tr.fail24)
sk = StandardScaler().fit(tr[EDGE])
edge = LogisticRegression(class_weight="balanced")
edge.fit(sk.transform(tr[EDGE]), tr.fail24)
# auf Rohwerte umgerechnet: Score = Merkmale @ w + b
w = edge.coef_[0] / sk.scale_
b = edge.intercept_[0] - (w * sk.mean_).sum()
```

```
def edge_score(X):          # nur eine Multiplikation: laeuft auf dem Gateway
    return X[EDGE].to_numpy() @ w + b
```

Die Ausgabe unten fasst den Lauf zusammen. Von den Testzeilen lässt Stufe 1 nur 48,1 Prozent durch, die übrigen 51,9 Prozent beantwortet das Gerät selbst mit „unauffällig“. Von den positiven Zeilen passieren 97,0 Prozent die Stufe, ein Teil geht also verloren, der in der Cloud nie ankommt. In der Ereignisrechnung hat das keine Folgen für die Erkennung: Das zweistufige System findet wie das Cloud-Modell allein alle 29 Ausfälle. Es erzeugt aber 8 statt 5 Fehlalarme, und die PR-AUC sinkt von 0,863 auf 0,852. Das Edge-Modell allein ist deutlich schlechter (PR-AUC 0,737, 20 Fehlalarme), erkennt aber ebenfalls alle Ausfälle, weil über einen ganzen Tag hinweg viele Zeilen vor dem Ausfall Alarm auslösen.

Terminal

```
$ python 05_zweiphasig.py
Edge-Schwelle auf Validierung: Recall 0.980
Anteil der Testzeilen, die an die Cloud gehen: 48.1%
Positive im Test, die Stufe 1 passieren: 97.0%
Variante                     PR-AUC  erkannt  Fehlalarme  Kosten/Test
nur Cloud                     0.863    29/29           5       80.000
nur Edge                      0.737    29/29          20      102.500
zweistufig                    0.852    29/29           8       84.500
Median je Einzelvorhersage: Edge 0.5 us, Cloud-Modell (lokal) 946 us
  alles in die Cloud: 288.000 Aufrufe/Tag = 42.048 EUR/Jahr
  zweistufig:         138.639 Aufrufe/Tag = 20.241 EUR/Jahr
  Ersparnis: 10.90 EUR je Pumpe und Jahr
Mehrkosten durch Fehler im Test: 4500 EUR bei 0.73 Pumpenjahren
  = 6139 EUR je Pumpe und Jahr
```

Was spart das? Die erste Rechnung betrifft die Aufrufe. Bei 2 000 Pumpen und einer Vorhersage alle zehn Minuten fallen 288 000 Cloud-Aufrufe pro Tag an. Mit Stufe 1 sind es 138 639 (−52 Prozent). Bei der Annahme von 0,40 Euro je 1 000 Aufrufe macht das 42 048 gegen 20 241 Euro im Jahr, also 10,90 Euro pro Pumpe und Jahr. Die zweite Rechnung betrifft die Fehler: Die drei zusätzlichen Fehlalarme kosten im Test 4 500 Euro, grob hochgerechnet auf die 0,73 Pumpenjahre Kalenderzeit des Tests sind das 6 139 Euro pro Pumpe und Jahr, also mehr als das Fünfhundertfache der Ersparnis. Drei Fehlalarme bei 29 Ausfällen sind statistisch nichts, und der Wert schwankt von Lauf zu Lauf stark. Wäre aber auch nur ein Zwanzigstel davon echt, überstiege es die gesparten Aufrufkosten noch um mehr als das Zwanzigfache.

Das ist die eigentliche Lehre des Abschnitts: Ein zweistufiges Modell rechnet sich nicht allein über die Aufrufkosten, weil die Fehlerkosten um Größenordnungen darüberliegen. Es lohnt sich dort, wo ein Aufruf teuer ist (Satellitenverbindung, Batteriebetrieb, Lizenz pro Aufruf, ein großes Modell mit Sekunden Rechenzeit je Vorhersage) und wo die Verbindung ausfallen kann. Bei der Latenz ist der Rechner belanglos: Das Edge-Modell braucht im Median 0,5 Mikrosekunden, das Cloud-Modell lokal in der Größenordnung einer Millisekunde, und gegen das Netz (Zehntel Sekunden, nicht gemessen) verschwinden beide. Bei 24 Stunden Horizont spielt Latenz ohnehin keine Rolle; bei einer Notabschaltung bei Grenzwertverletzung täte sie es, aber die ist eine Regel am Gerät und kein Modell. Dazu kommt ein Nutzen, den die Rechnung nicht zeigt: Bei Netzausfall arbeitet das Gerät allein weiter. Das kostet in unserem Test 22 500 Euro mehr als das Cloud-Modell, erkennt aber weiterhin alle 29 Ausfälle.

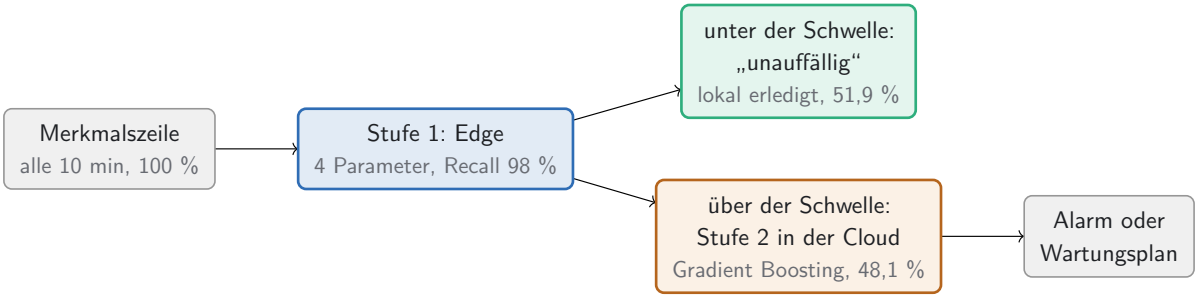


Abbildung 100: Zwei-Phasen-Vorhersage in der Pumpenflotte. Die Anteile sind die Werte des Laufs auf den Testdaten.

Grenzen und Bezug zu Software Engineering

Wer Stufe 1 falsch einstellt, bemerkt es nicht: Was sie verwirft, erreicht die Cloud nie. Man überwacht deshalb den Recall der Stufe 1 getrennt; jeder beobachtete Ausfall liefert nachträglich das Etikett für die 24 Stunden davor, auch für Zeilen, die nie in die Cloud kamen (Pattern 18, Continued Model Evaluation). Wird die Cloud neu trainiert, muss die Schwelle der Stufe 1 neu geprüft werden, und beide Modelle müssen in kompatiblen Versionen laufen (Pattern 27). Mit Pattern 8 (Cascade) ist das Pattern verwandt: Dort bestimmt das erste Modell, welches andere Modell einen Fall sieht; hier ist das erste Modell ein Vorfilter, und die Stufen trennt eine Netzgrenze mit eigenen Ausfällen. Billige Stufen vorzuschalten und teure nur für den Rest einzusetzen, ist aus der Gesichtserkennung bekannt [43]. **Wann nicht:** Wenn Aufrufe billig sind und die Verbindung stabil ist, bringt die zweite Stufe Betrieb und Risiko ohne Nutzen. Stufe 1 ist ein Vorfilter wie ein Bloom-Filter oder ein Cache vor einer teuren Operation: Falsch-Positive kosten nur den teuren Weg, Falsch-Negative sind verboten. Der Betrieb am Gerät bei Netzausfall ist ein Fallback, wie man ihn von Circuit Breakern kennt.

Quellen. [1] (Kapitel 5, Pattern 19); [43] für frühes Aussortieren durch billige Stufen.

13.6 Pattern 16 und 17: Einzeln oder im Paket

Szenario und Problem

Bisher kam jede Vorhersage einzeln: Alle zehn Minuten fragt eine Pumpe nach dem Ergebnis, ein Online-Dienst antwortet (Pattern 16, Stateless Serving Function). Das stimmt nur für einen Teil der Aufgaben. Die Disposition möchte morgens eine Liste aller Pumpen nach Ausfallrisiko, und nach einem Modellwechsel soll die gesamte Historie neu bewertet werden. Niemand wartet auf eine einzelne Antwort, und ein Online-Dienst wäre dafür verschwenderisch.

Lösung

Das Pattern 17 (Batch Serving) bewertet viele Zeilen in einem Aufruf. Dasselbe Modell, dieselbe Datei; nur die Aufrufform ändert sich. Das Skript misst beide Formen mit einem Gradient Boosting wie im vorigen Abschnitt:

Terminal

```
$ python 06_serving_modi.py
Testzeilen: 33.650
Paket mit  1.000 Zeilen:   4.7 us je Zeile (Einzelaufruf 643 us, Faktor 137)
Paket mit 33.650 Zeilen:   3.0 us je Zeile (Einzelaufruf 643 us, Faktor 212)
identische Ergebnisse: True

Flotte mit 2.000 Pumpen, Zeilen pro Tag:
  alle 10 min: 288.000
  stuendlich: 48.000
  einmal nachts: 2.000
```

Ein einzelner Aufruf kostet in der Größenordnung einer Millisekunde, auch wenn er nur eine Zeile bewertet, denn er zahlt feste Kosten für Aufruf, Prüfung und Aufbau der Rechnung. Im Paket verteilen sich diese Kosten auf alle Zeilen, und eine Zeile kostet nur noch einige Mikrosekunden. Der Faktor liegt je nach Paketgröße zwischen 100 und 300, und die Ergebnisse sind identisch. Für die Flotte heißt das: 2 000 Pumpen alle zehn Minuten sind 288 000 Zeilen am Tag, einzeln gerechnet etwa drei Minuten Rechenzeit, im Paket etwa eine Sekunde (mit den Zeiten des Laufs oben, auf einem Rechner).

Die Wahl zwischen Online und Batch richtet sich nach der **Frist der Entscheidung**, nicht nach dem Modell. Eine Notabschaltung bei Grenzwertverletzung muss in Sekunden fallen und ist eine Regel am Gerät, kein Modell. Der Alarm „Ausfall in 24 h“ hat eine Frist von zehn Minuten. Auch dann rechnet man nicht Pumpe für Pumpe: Man sammelt die Merkmalszeilen aller Pumpen eines Takts und bewertet sie als kleines Paket (**Micro-Batch**), online im Sinne der Frist und Batch im Sinne der Rechnung. Der Einsatzplan für den nächsten Tag und die Neubewertung der Historie nach einem Modellwechsel sind klassische Batch-Aufgaben.

Grenzen und Bezug zu Software Engineering

Das Paket hat eine Obergrenze durch den Arbeitsspeicher, und die erste Antwort kommt erst, wenn es fertig ist. Jede Paketform verlangt dieselbe Merkmalsberechnung wie der Betrieb (Pattern 24). **Wann nicht:** Wenn eine Entscheidung wirklich einzeln und sofort fallen muss, bleibt es beim Einzelaufruf. Die Zeiten stammen von einem ausgelasteten Entwicklungsrechner und schwanken von Lauf zu Lauf; belastbar ist die Größenordnung. Das Muster kennt man als N+1-Problem: eine Datenbankabfrage pro Zeile statt einer für alle. Das Modell bleibt gleich, nur die Schnittstelle nach außen ändert sich.

Quellen. [1] (Kapitel 5, Patterns 16 und 17).

13.7 Pattern 20: Keyed Predictions

Szenario und Problem

Die nächtliche Neubewertung läuft parallel, denn mehrere Rechner teilen sich die Arbeit. Das Skript zerlegt die Testtabelle in zwölf Pakete, die ein Pool von Workern mit unterschiedlicher Laufzeit bewertet, und sammelt die Ergebnisse in der Reihenfolge, in der sie fertig werden. Die naheliegende Fortsetzung ist, die Ergebnisse aneinanderzuhängen und der Tabelle zuzuordnen, als kämen sie in der alten Reihenfolge zurück:

```
# Ohne Schluessel: Ergebnisse aneinanderhaengen, der Tabelle zeilenweise zuordnen
ohne = np.concatenate([p for p, _ in fertig])
# Mit Schluessel: Pumpen-ID und Minute reisen mit und bestimmen die Zuordnung
mit = pd.concat([k for _, k in fertig]).merge(
    te[["pumpe", "minute", "fail24"]], on=["pumpe", "minute"], validate="one_to_one")
```

Das Ergebnis steht in der Ausgabe unten. Die Pakete kamen in der Reihenfolge 5, 6, 8, 11, 2, 4, 10, 3, 0, 7, 9, 1 zurück, und bei 59,4 Prozent der Zeilen steht danach eine Vorhersage, die zu einer anderen Zeile gehört. Von 4 173 Alarmen (Score ab 0,5) liegen 79,3 Prozent an Zeilen, die keinen verdienen. Die PR-AUC fällt von 0,863 auf 0,229; bei einem Anteil von gut 20 Prozent Positiven im Test ist das der Wert des reinen Ratens. Dabei sieht nichts kaputt aus. Der Code läuft ohne Fehler, die Ausgabe hat die richtige Länge, jeder Wert liegt zwischen 0 und 1. Nur wer die Zahlen mit einer Referenz vergleicht, merkt es. Mit Schlüssel ist das Ergebnis identisch zur sequentiellen Referenz.

```
Terminal
$ python 07_keys.py
Pakete in Reihenfolge der Fertigstellung: [5, 6, 8, 11, 2, 4, 10, 3, 0, 7, 9, 1]
Zeilen mit fremder Vorhersage (ohne Schluessel): 59.4%
Alarme ohne Schluessel: 4173, davon an einer Zeile, die keinen verdient: 79.3%
PR-AUC ohne Schluessel: 0.229
PR-AUC mit Schluessel:  0.863  (sequentielle Referenz: 0.863)
Zuordnung ueber (pumpe, minute) identisch zur Referenz: True

hoechster Score der Referenz: Zeile 15324 = P24, Minute 75640, Score 1.00
ohne Schluessel steht an dieser Stelle: 0.05
```

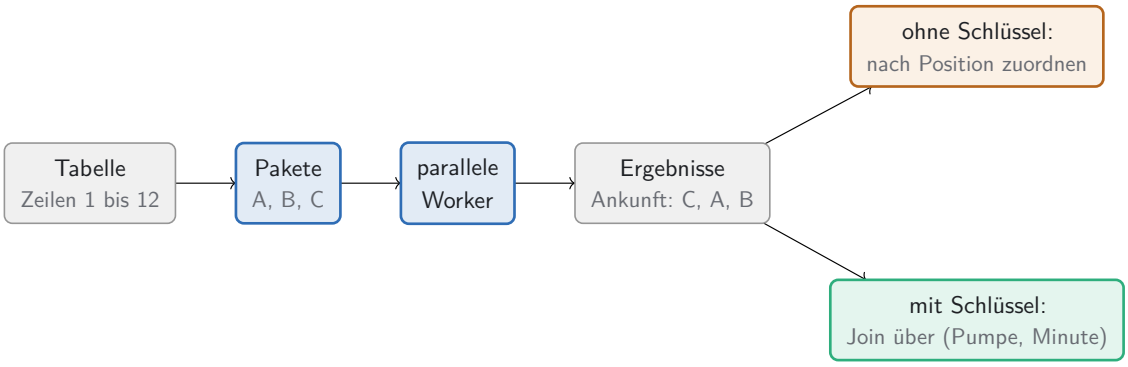


Abbildung 101: Bei paralleler Verarbeitung kommen die Ergebnisse in Ankunftsreihenfolge zurück. Nur der Schlüssel macht die Zuordnung unabhängig davon.

Lösung

Das Pattern verlangt, dass jede Anfrage einen **Schlüssel** mitführt, den der Worker unverändert an die Antwort hängt. Die Zuordnung geschieht danach über den Schlüssel, nicht über die Position. Hier besteht der Schlüssel aus Pumpen-ID und Minute, das ist eindeutig, weil eine Pumpe pro Minute nur einen Messwert hat. `merge(..., validate="one_to_one")` prüft das bei jedem Lauf nach: Taucht ein Schlüssel doppelt auf, etwa weil ein Paket nach einem Fehler wiederholt wurde, bricht der Lauf ab, statt stillschweigend zu verdoppeln. Dasselbe Prinzip löst weitere Probleme. Mit Schlüsseln kann man Wiederholungen erkennen (Idempotenz), Vorhersagen später mit dem tatsächlichen Ausgang zusammenführen (für Pattern 18) und Fehler einem Eingang zuordnen.

Grenzen und Bezug zu Software Engineering

Der Fehler ist in einem Thread-Pool zufällig verborgen: `pool.map` liefert die Ergebnisse in Eingabereihenfolge, und das Problem tritt nicht auf. Es erscheint, sobald die Reihenfolge nicht mehr garantiert ist, bei `as_completed`, bei Nachrichtenwarteschlangen, bei verteilten Frameworks mit Shuffle und bei Wiederholungen nach Fehlern. Dann ist der Test mit kleinen Daten unauffällig, und der Fehler zeigt sich erst im Betrieb. Der Schlüssel muss eindeutig sein und darf nichts enthalten, was man nicht mitschicken will; bei Personendaten ist schon die Kundennummer ein Datenschutzthema. **Wann nicht:** Bei einer sequentiellen Verarbeitung mit garantierter Reihenfolge genügt die Position. Das Pattern ist die **Korrelations-ID** aus verteilten Systemen und Nachrichtenmustern [56], ohne die auch Tracing über Dienstgrenzen nicht auskommt.

Quellen. [1] (Kapitel 5, Pattern 20); [56] (Correlation Identifier); [54] (Verarbeitung ohne garantierte Reihenfolge).

13.8 Pattern 12: Checkpoints

Szenario und Problem

Auch das Training soll nachts laufen. Die Daten kommen als 30 Tagesdateien, und wir lesen sie fünfmal durch: 150 Schritte, bei denen das Modell je eine Datei sieht. Für ein lineares Modell mit `partial_fit` (der Weg in scikit-learn, Modelle auf Daten zu trainieren, die nicht in den Speicher passen) dauert das Sekunden, bei einem großen Modell sind es Stunden. Und Stunden auf einem Rechner, den der Cluster-Manager jederzeit entziehen kann, sind riskant. Bricht der Knoten bei Schritt 97 ab, ist ohne Zwischenstand alles verloren.

Lösung

Man speichert in festen Abständen den gesamten **Zustand** des Trainings, nicht nur die Parameter, sondern alles, was für die Fortsetzung nötig ist: den Schrittzähler, die Position in den Daten, bei neuronalen Netzen außerdem den Zustand des Optimierers und den Lernratenplan sowie den Zufallszustand. Im Beispiel steckt alles Nötige im Modellobjekt und in der Schrittnummer. Entscheidend ist ein Detail: Die Datei wird zuerst unter einem Zwischennamen geschrieben und dann **atomar umbenannt**; ein Abbruch mitten im Schreiben hinterlässt höchstens eine unbrauchbare Zwischendatei, aber nie einen halb geschriebenen Zwischenstand.

```
def trainiere(modell=None, start=0, abbruch_bei=None, ckpt_alle=None,
              pfad="out/ckpt.joblib"):
    """Ein Schritt = ein Paket. Auf einem Cluster waeren das Stunden."""
    modell = modell or SGDClassifier(loss="log_loss", alpha=1e-4, random_state=0)
    for schritt in range(start, SCHRITTE):
        if schritt == abbruch_bei:
            raise Abbruch(f"Knoten entzogen vor Schritt {schritt}")
        i = pakete[schritt % len(pakete)]
        modell.partial_fit(X[i], y[i], classes=[0, 1])
        if ckpt_alle and (schritt + 1) % ckpt_alle == 0:
            stand = {"modell": modell, "naechster_schritt": schritt + 1}
            joblib.dump(stand, pfad + ".tmp")
            os.replace(pfad + ".tmp", pfad)    # atomar: nie eine halbe Datei
    return modell
```

Das Skript führt drei Läufe aus (Ausgabe unten): einen ohne Störung, einen mit Abbruch bei Schritt 97 und ohne Zwischenstand, einen mit Abbruch und Zwischenständen alle zehn Schritte.

Beide gestörten Läufe landen beim selben Modell wie der ungestörte: Alle Koeffizienten sind identisch, das prüft `np.array_equal`. Der Unterschied liegt im Aufwand: Ohne Zwischenstand sind es 97 verlorene plus 150 neue Schritte, insgesamt 247. Mit Zwischenständen setzt der Lauf bei Schritt 90 wieder an und braucht 157 Schritte, 36 Prozent weniger. Dass das Ergebnis identisch ist, ist keine Selbstverständlichkeit, sondern hängt an Determinismus: fester Seed, feste Reihenfolge der Daten, und dass der Zufallszustand nicht verloren geht (`random_state=0` im Modell erzeugt bei jedem Aufruf dieselbe Folge, deshalb muss hier nichts zusätzlich gespeichert werden). Würde man den Seed weglassen, gäbe es nach der Wiederaufnahme ein anderes Modell, nicht ein falsches, aber ein nicht reproduzierbares.

Terminal

```
$ python 08_checkpoints.py
A) ohne Unterbrechung:  150 Schritte, PR-AUC Test 0.7054
B) Knoten entzogen vor Schritt 97; kein Checkpoint, Neustart bei Schritt 0
   Aufwand: 97 + 150 = 247 Schritte, identisch zu A: True
C) Knoten entzogen vor Schritt 97
   letzter Checkpoint: vor Schritt 90, Datei 1234 Bytes
   Aufwand: 97 + 60 = 157 Schritte, identisch zu A: True
   PR-AUC Test 0.7054

 Intervall  Checkpoints  max. Verlust  mittl. Verlust
         1          150         1 Schr.         0.5 Schr.
         5           30         5 Schr.         2.5 Schr.
        10           15        10 Schr.         5.0 Schr.
        25            6        25 Schr.        12.5 Schr.
        50            3        50 Schr.        25.0 Schr.
```

Wie oft man sichern soll, ist ein Abwägen: Je kürzer das Intervall, desto weniger geht im Mittel verloren (ein halbes Intervall) und desto mehr Schreibaufwand fällt an (Tabelle am Ende des Laufs). Bei einem großen Modell mit Gigabyte großem Zwischenstand nimmt man eher Abstände von Minuten bis Stunden.

Grenzen und Bezug zu Software Engineering

Nicht jedes Modell lässt sich so trainieren. `partial_fit` gibt es für lineare Modelle, naive Bayes oder Mini-Batch-k-Means, aber nicht für das Gradient Boosting der Cloud-Stufe, das wir in Sekunden komplett neu trainieren; Checkpoints lohnen dort, wo ein Lauf Stunden dauert. Ein Zwischenstand ist an Bibliotheks- und Codeversionen gebunden (Pickle und Joblib führen beim Laden Code aus, siehe Pattern 16) und braucht Platz. Das Buch nennt zwei weitere Verwendungen, nämlich Ausgangspunkt für Feintuning und das beste Zwischenmodell als Ergebnis (Early Stopping), und als Variante **virtuelle Epochen**, die Schritte in einer festen Zahl von Beispielen statt in Durchläufen zählen. **Wann nicht:** Bei kurzem Training überwiegen Aufwand und Komplexität. Das Prinzip ist der Snapshot aus Datenbanken, das atomare Umbenennen kennt man aus Dateisystemen, und ein Job, der nach Abbruch wieder anlaufen kann, ist **idempotent**.

Quellen. [1] (Kapitel 4, Pattern 12); [42] (`partial_fit`, Out-of-Core-Lernen).

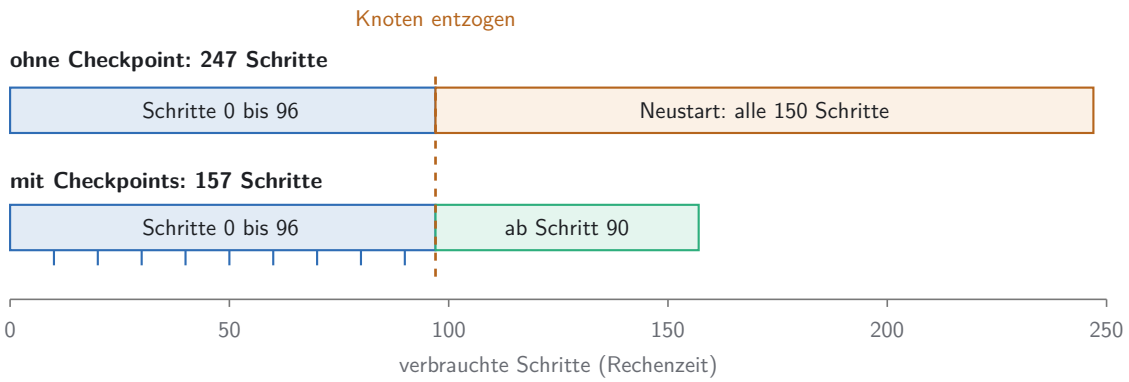


Abbildung 102: Der Abbruch bei Schritt 97 und seine Folgen. Oben muss alles wiederholt werden, unten nur das Stück seit dem letzten Zwischenstand (Schritt 90). Die kurzen Striche unter dem Balken markieren die Checkpoints.

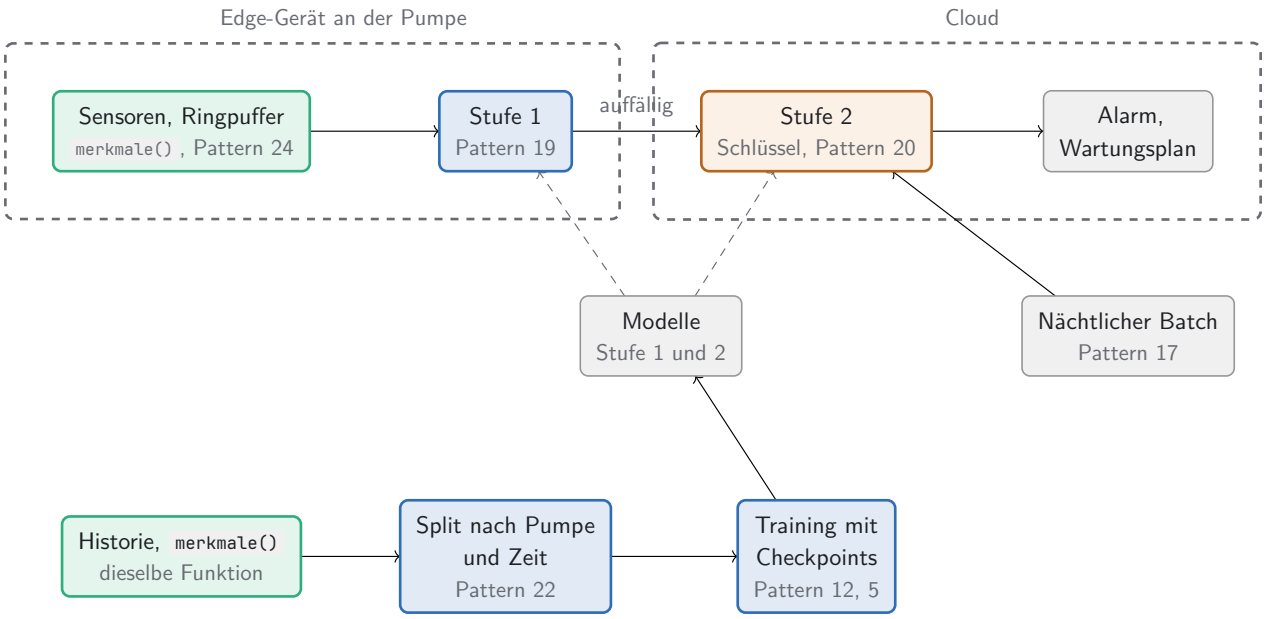


Abbildung 103: Architektur der Fallstudie. Oben der Betrieb mit Gerät und Cloud, unten der Trainingsweg. Gestrichelt: Die trainierten Modelle gehen an beide Stufen. Die grünen Knoten sind dieselbe Funktion.

13.9 Das Gesamtsystem

Die sieben Patterns ergeben eine Architektur mit zwei Orten, dem Gerät an der Pumpe und der Cloud, und einem Trainingsweg, der beide versorgt (Abbildung 103). Auf dem Gerät sitzen Ringpuffer, Merkmalsfunktion und Stufe 1. Was auffällt, geht mit Pumpen-ID und Zeitstempel in die Cloud, wo Stufe 2 bewertet und die Antwort über den Schlüssel zugeordnet wird. Dieselbe Merkmalsfunktion erzeugt im Training die Tabelle, aus der nach Pumpe und Zeit aufgeteilt wird; das Training läuft über Tagesdateien mit Zwischenständen, und die nächtliche Einsatzplanung ist ein Batch-Aufruf desselben Modells.

Die Tabelle zeigt auch, wo die Patterns nichts verbessern, sondern etwas offenlegen: Pattern 22 macht nicht das Modell besser, sondern die Messung ehrlicher, Pattern 24 und 20 verhindern Fehler, die man im Labor nicht sieht, und Pattern 5 und 19 verschieben Kompromisse, die man nur mit Kosten und Fehlerzahlen beurteilen kann. Das passt zur Beobachtung von D. Sculley *u. a.* [11], dass das Modell nur einen kleinen Teil eines produktiven ML-Systems ausmacht und der Rest aus Daten- und Betriebsinfrastruktur besteht.

Tabelle 17: Wirkung der Patterns in dieser Fallstudie. Alle Zahlen stammen aus den Läufen im Verzeichnis `code/13`; die Kosten in Euro beruhen auf den im Text genannten Annahmen.

Pattern	Ohne	Mit
22 Repeatable Splitting	PR-AUC 0,992 (Zufallssplit), geschönt	PR-AUC 0,862 (zeitlich), ehrlich; 0,858 bei neuen Pumpen und neuer Zeit
24 Windowed Inference	offline Recall 0,84, im Stream 0,50 (Fenster mit Zukunft; zeilenweise)	Recall 0,69, offline und im Stream gleich
5 Reframing	PR-AUC 0,823 (rohe RUL)	0,863 (Klassifikation), 0,886 (RUL auf 72 h gekappt)
19 Two-Phase	288 000 Cloud-Aufrufe pro Tag	138 639 (−52 %), dafür 3 Fehlalarme mehr im Test
17 Batch Serving	Einzelaufruf: Größenordnung 1 ms je Zeile	Paket: wenige µs je Zeile, Faktor 100 bis 300
20 Keyed Predictions	PR-AUC 0,229, 79 % der Alarme falsch zugeordnet	PR-AUC 0,863, Zuordnung identisch zur Referenz
12 Checkpoints	247 Schritte nach Abbruch	157 Schritte (−36 %), identisches Modell

13.10 Das Wichtigste in Kürze

- Bei Zeitreihen ist ein zufälliger Split ein Messfehler: Er meldete 0,992 PR-AUC, wo der ehrliche zeitliche Split 0,862 ergab. Man trennt nach Pumpe und nach Zeit, mit einer Lücke von mindestens der Länge des Vorhersagehorizonts.
- Ein Fenstermerkmal darf nur Vergangenheit sehen. Eine Merkmalsfunktion für Training und Betrieb und ein Äquivalenztest zwischen Batch und Stream verhindern, dass sich zwei Implementierungen auseinanderentwickeln.
- Ob Regression oder Klassifikation, entscheidet die Entscheidung dahinter. Das Ziel auf den relevanten Bereich zu beschränken, brachte in der PR-AUC mehr als die Wahl des Aufgabentyps (0,823 gegen 0,886); in den Kosten waren alle drei Ansätze nahezu gleich.
- Ein zweistufiges Modell spart Aufrufe, aber nur der Vergleich mit den Fehlerkosten zeigt, ob es sich lohnt. Die Rechnung der Fallstudie ergibt: nur bei hohen Aufrufkosten oder fehlender Verbindung.
- Wer parallel bewertet, führt einen Schlüssel mit (Pumpe und Zeitstempel). Die Position taugt nicht als Zuordnung.
- Zwischenstände mit atomarem Schreiben machen ein langes Training abbruchsicher; im Beispiel kostete ein Abbruch 157 statt 247 Schritte.

SO ERKLÄRE ICH ES

„Jedes Mal, wenn wir im Labor etwas tun, was im Betrieb nicht möglich wäre (Zeilen mischen, in die Zukunft schauen, Vorhersagen nach Position zuordnen), wird die Zahl im Labor besser und im Betrieb schlechter. Ich lasse die Studierenden raten, welchen Wert Zufallssplit und Zeitsplit ergeben, bevor ich die Zahlen zeige.“

Verständnisfrage: Ein Team meldet, sein Modell erkenne 92 Prozent der Ausfälle, im Betrieb sind es nur 55 Prozent. Wo suchen Sie zuerst?

Antwort: Nach einem Unterschied zwischen Training und Betrieb in der Datenaufbereitung, vor allem bei Fenstermerkmalen: Reicht ein Fenster in die Zukunft (`center=True` , Tagesmittel über den ganzen Tag)? Hatten Testzeilen Nachbarn im Training? Wurde die Alarmschwelle auf verfälschten Werten gewählt? Ein Äquivalenztest zwischen Batch und Stream findet den ersten Fall.

13.11 Weiterführende Literatur

Einen Überblick über Methoden der vorausschauenden Wartung geben T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, und S. G. S. Alcalá [98] die C-MAPSS-Daten [99] sind ein frei verfügbarer Vergleichsfall mit Restlebensdauer als Ziel. Zur Aufteilung von Zeitreihen und strukturierten Daten siehe C. Bergmeir und J. M. Benítez [100] und D. R. Roberts *u. a.* [57], zur Fensterung in Datenströmen T. Akidau *u. a.* [58] und das Stream-Kapitel bei M. Kleppmann [54]. Das Gradient Boosting geht auf J. H. Friedman [5] zurück, die Histogramm-Variante in scikit-learn ist von LightGBM [101] inspiriert. Zu Betriebsrisiken von ML-Systemen siehe D. Sculley *u. a.* [11] und die Checkliste von E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55].

14 Fallstudie C: Produktsuche und Kaufwahrscheinlichkeit im Onlineshop

Die Patterns der vorigen Kapitel wurden jeweils an einem eigenen, kleinen Beispiel vorgestellt. Diese Fallstudie setzt acht von ihnen in einem durchgehenden Projekt ein. Dabei zeigt sich, was ein einzelnes Beispiel verbirgt: Patterns ergänzen sich, manche leisten teilweise dasselbe, und eines von ihnen deckt ein Problem auf, das ohne es unbemerkt geblieben wäre.

> **LERNZIELE**
> Nach diesem Kapitel können Sie an einem durchgängigen Beispiel zeigen, was ein Hashed Feature gegenüber einem Wörterbuch kostet, wie ein Embedding aus Ko-Klick-Daten durch Matrixfaktorisierung entsteht, warum ein Feature Cross mehr leistet als zusätzliche Spalten, wann sich eine Cascade lohnt, woran man ein Datenleck in Popularitätsfeatures erkennt und wie ein Dienst mit zwei Modellversionen aussieht. Sie lernen außerdem, die Wirkung mehrerer Patterns ehrlich zu bewerten, wenn sie sich nicht einfach addieren.

14.1 Ausgangslage: ein Shop, ein Score, drei Probleme

Ein Onlineshop für Outdoor- und Sportartikel führt zu Beginn rund 20 000 Produkte aus zwölf Kategorien von 800 Marken, jeden Tag kommen etwa 50 neue dazu. Jedes Produkt hat eine ID, eine Kategorie, eine Marke, einen Preis und einen Titel wie „Trail Ultraleicht Laufweste Pro“. Aus den Klickdaten soll ein Modell lernen, mit welcher Wahrscheinlichkeit eine Sitzung mit einem Kauf endet. Vorhergesagt wird nach der dritten angesehenen Produktseite. Mit dem Score entscheidet der Shop, ob er eine Hilfe oder einen Gutschein einblendet und zu welcher Kategorie. Der Score steuert also andere Entscheidungen, und deshalb zählt neben seiner Trennschärfe auch, dass die Wahrscheinlichkeiten stimmen.

Die Daten sind synthetisch. Das Skript `code/14/shop.py` erzeugt 23 500 Produkte (20 000 zum Start, 50 Neuzugänge pro Tag über 70 Tage) und 150 000 Sitzungen mit mindestens drei Produktansichten, im Mittel 3,8. Jedes Produkt gehört zu einer *Welt*, einem Anlass wie Camping oder Golf. Besucher stöbern überwiegend innerhalb einer Welt; die Welt steckt in den Titelwörtern, aber in keiner Tabellenspalte. Eingebaut sind außerdem Effekte: Marken und Welten verkaufen unterschiedlich gut, in manchen Kategorien wird eher das günstige, in anderen eher das teure Modell gekauft, Titelzusätze wie „Pro“ helfen, und Produkte, die in der Vorwoche oft gekauft wurden, werden eher wieder gekauft. Die Fallstudie zeigt, wie man solche Zusammenhänge findet und im Betrieb hält, nicht, wie stark sie in einem echten Shop wären. Mit festem Zufallssamen liefert jeder Lauf dieselben Zahlen. `code/14/run_all.sh` führt alle Skripte in der Reihenfolge ihrer Nummern aus (rund dreieinhalb Minuten); das dritte legt die Embedding-Tabelle ab, die spätere Skripte lesen.

Drei Probleme. Erstens hohe Kardinalität: Produkt-ID und Marke haben tausende Werte, die meisten selten. Zweitens gemischte Eingaben: Die Tabelle sagt wenig darüber, welche Produkte zusammengehören; der Titeltext verrät es, und ein Teil steckt im Verhalten der Besucher, die ähnliche Produkte gemeinsam ansehen. Drittens der Betrieb: Es kommen laufend neue Produkte dazu, die Beliebtheit ändert sich täglich, und das Modell muss als Dienst erreichbar sein, den man austauschen kann, ohne den Shop zu stören. Abbildung 104 zeigt, wo in der Verarbeitungskette welches Pattern ansetzt.

Die Messlatte

Bevor man ein Modell verbessert, muss feststehen, woran man Verbesserung misst. Die Daten werden *nach der Zeit* geteilt: Tag 0 bis 49 zum Training (107 377 Sitzungen), Tag 50 bis 69 zum Test (42 623). Das entspricht dem Betrieb, in dem das Modell immer Sitzungen der Zukunft bewertet, und nur so treten im Test neue Produkte auf (zeitliche Splits: Pattern 22, Repeatable Splitting). Es kaufen 6,8 % der Sitzungen.

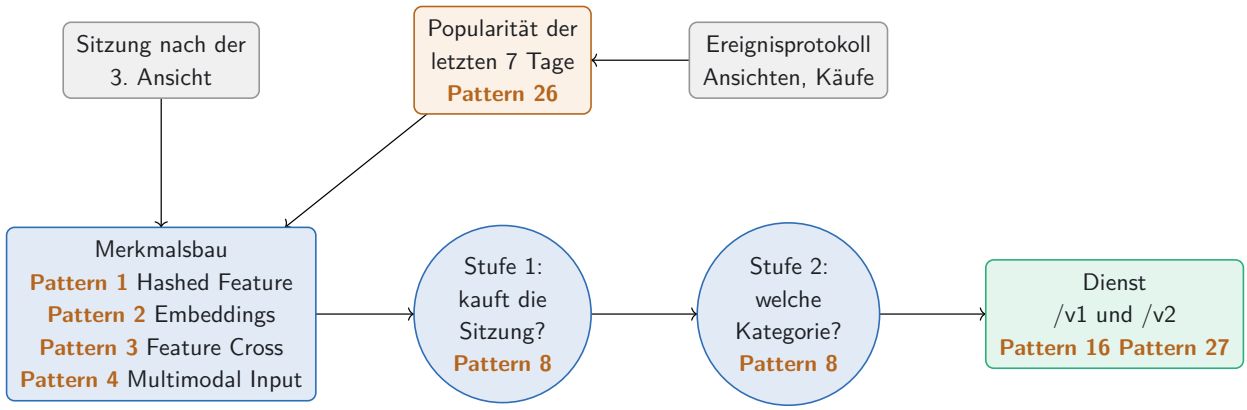


Abbildung 104: Wo die acht Patterns in der Verarbeitungskette ansetzen. Die Nummern sind die Pattern-Nummern des Buchs.

Gemessen wird dreifach: Die *AUC* (Fläche unter der ROC-Kurve) ist die Wahrscheinlichkeit, dass eine zufällige Kaufsitzung einen höheren Score erhält als eine zufällige Sitzung ohne Kauf (0,5 ist Raten, 1,0 perfekte Trennung). Die *Average Precision* (AP) fasst die Precision-Recall-Kurve zusammen und ist bei seltenen Käufen empfindlicher; Zufall erreicht nur den Kaufanteil 0,068. Der *Log-Loss* bestraft falsche Wahrscheinlichkeiten; kleiner ist besser.

Das *Ausgangsmodell* ist eine logistische Regression, also ein lineares Modell, das aus der gewichteten Summe der Features eine Wahrscheinlichkeit macht. Es bekommt Kategorie, Gerät, Uhrzeit, den logarithmierten Preis sowie Marke und Produkt-ID als One-Hot-Kodierung über ein Wörterbuch (je Wert eine Spalte, die 1 ist, wenn der Wert vorliegt, sonst 0). Gebaut werden die Features von der Klasse `Merkmale` (`code/14/merkmale.py`), in der jede Zutat der Fallstudie ein Schalter ist und die später unverändert im Dienst läuft. Das ist das Pattern Transform (21) in der einfachsten Form: eine einzige Stelle, an der Rohdaten zu Features werden. Es erreicht eine AUC von 0,667, eine AP von 0,161 und einen Log-Loss von 0,2328. Jede Verbesserung muss sich daran messen.

14.2 Pattern 1: Hashed Feature

Szenario und Problem. Die Produkt-ID ist das Paradebeispiel einer Kategorie mit hoher Kardinalität. Das Wörterbuch des Ausgangsmodells hat 19 092 Spalten für ID und Marke zusammen; es muss beim Training erzeugt, mit dem Modell gespeichert und im Dienst identisch geladen werden. Schwerer wiegt der Kaltstart: Im Test kommt bei jeder dreißigsten Sitzung ein Produkt vor, das im Training nie vorkam, und bei sieben von zehn dieser Sitzungen gehört das Produkt zu den Neuzugängen nach dem letzten Trainingstag (Abbildung 105, rechts). Das Wörterbuch bildet sie auf einen Nullvektor ab. Das Modell stürzt nicht ab, aber es weiß über genau die Produkte nichts, die der Shop gerade bewerben will.

Die Lösung. Eine Hashfunktion ersetzt das Wörterbuch durch eine Rechenvorschrift (Grundlagen und Formel stehen im Kapitel Data Representation). Hier ist sie vollständig: Der Wert wird mit MurmurHash3,

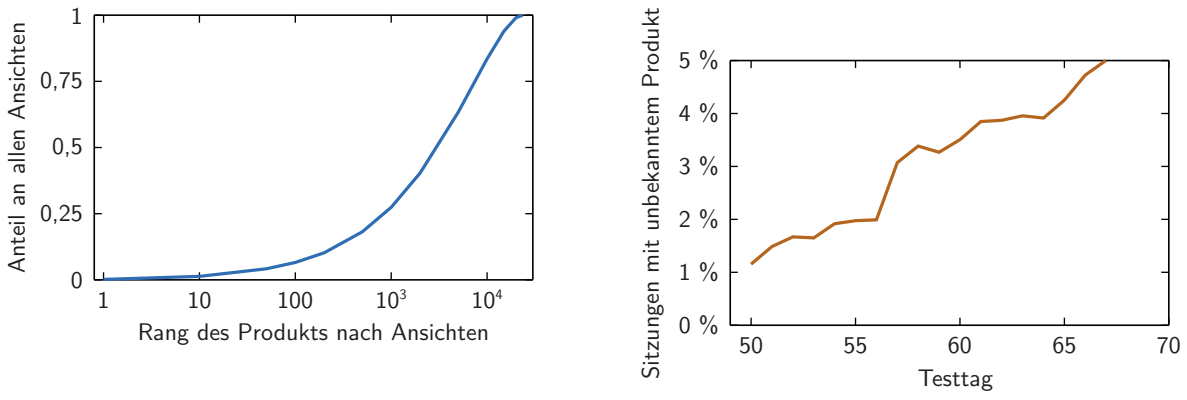


Abbildung 105: Links: Long Tail der Produktansichten. Die beliebtesten 10 % der Produkte vereinen 44 % der Ansichten, die untere Hälfte nur 12 %. Rechts: Anteil der Testsitzungen, deren aktuelles Produkt in keiner Trainingssitzung vorkam. Er wächst mit jedem Tag, weil neue Produkte dazukommen. Werte aus `01_daten.py`.

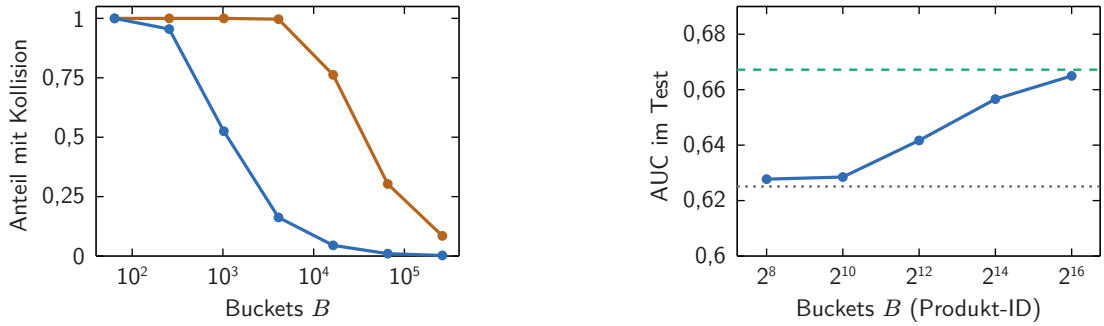


Abbildung 106: Links: gemessene Kollisionsraten für 23 500 Produkt-IDs (orange) und 800 Marken (blau). Rechts: Modellgüte, wenn die Produkt-ID mit B Buckets gehasht wird (blau), gegen Wörterbuch (grün gestrichelt) und gegen den Verzicht auf die ID (grau gepunktet). Die Marke ist in allen Läufen per Wörterbuch kodiert. Werte aus `02_hashed_feature.py`.

derselben Funktion, die scikit-learn intern für `FeatureHasher` nutzt, in eine 32-Bit-Zahl verwandelt und modulo der Bucket-Zahl gerechnet. Das Salz („produkt“, „marke“) sorgt dafür, dass gleiche Werte in verschiedenen Spalten nicht dieselben Buckets teilen.

```
def bucket(werte, b, salz):
    """Hashed Feature: Bucket je Wert per MurmurHash3, deterministisch, ohne Wörterbuch."""
    h = {w: murmurhash3_32(f"{salz}={w}", positive=True) % b
         for w in set(werte)}
    return np.array([h[w] for w in werte])
```

Die Kollisionsraten lassen sich messen und mit der Formel $1-(1-1/B)^{n-1}$ vergleichen. Bei 23 500 IDs und 16 384 Buckets teilen drei Viertel der IDs ihren Bucket mit mindestens einer anderen (76,3 % gemessen, 76,2 % gerechnet). Das klingt dramatisch, spielt aber für das Ergebnis wenig Rolle, weil die Gewichte seltener IDs ohnehin kaum gelernt werden. Entscheidend ist, was mit den häufigen Produkten passiert, und das zeigt nur das Modell.

Zwei Dinge fallen auf. Mit $B = 2^{14}$ erreicht das gehashte Feature eine AUC von 0,657 gegenüber 0,667 mit Wörterbuch; erst bei 2^{16} Buckets, also mehr Spalten als Produkte, gleicht sich der Abstand aus. Die ID trägt Information (ohne sie sinkt die AUC auf 0,625), weil 44 % aller Ansichten auf die beliebtesten 2350 Produkte entfallen. Und der Vorteil liegt hier nicht im Speicher, denn bei rund 18 000 Produkten im Training ist das Wörterbuch kaum größer als die 16 384 Buckets, sondern im Wegfall der Pflege: keine Zuordnung, die beim Laden im Dienst verloren gehen könnte, und ein neues Produkt landet in einem definierten Bucket, wo es allerdings nur das Gewicht der Produkte erbt, die zufällig denselben Bucket teilen. Für die Marke lohnt sich das Hashen bei 800 Werten nicht (AUC 0,625 mit Wörterbuch, 0,620 bei 1024 Buckets). Der Gewinn wächst mit der Kardinalität; bei zwei Millionen IDs wie im Szenario des Kapitels Data Representation sähe die Rechnung anders aus.

Trade-offs und Grenzen. Über neue Produkte sagt der Versuch wenig: Es sind 1391 Testsitzungen mit nur 106 Käufen, die AUC schwankt dort um mehrere Hundertstel, und man kann nur lesen, dass der Hash nicht schadet. Wer für neue Produkte etwas *gewinnen* will, braucht Information, die nicht an der ID hängt; das leisten die nächsten beiden Patterns. Nicht einsetzen: bei inhaltlich ähnlichen Werten (der Hash zerstört jede Nachbarschaft), bei kleiner Kardinalität und wenn man nachvollziehen muss, welcher Wert welches Gewicht trägt, denn ein Bucket ist ein Gemisch.

Bezug zu Software Engineering. Sharding nach Hash des Schlüssels: gleiche Idee, gleiche Entscheidung, die man später kaum ändern kann. Mit einem anderen B ist jedes Gewicht an einen anderen Bucket gebunden, das Modell also ein neues.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Hashed Feature“; K. Weinberger, A. Dasgupta, J. Langford, A. Smola, und J. Attenberg [35].

14.3 Pattern 2: Embeddings

Szenario und Problem. Das Ausgangsmodell sieht jede ID als etwas Eigenständiges. Es weiß nicht, dass ein Zelt und eine Isomatte, die beide in „Camping“-Sitzungen auftauchen, zusammengehören, obwohl diese Beziehung in den Klickdaten steckt: Eine Sitzung ist eine Sammlung von Produkten, die jemand

in einem Zug angesehen hat. Gesucht ist eine Darstellung, in der Produkte, die gemeinsam angesehen werden, nahe beieinanderliegen, und die ein Modell als Eingabe nehmen kann.

Die Lösung: Matrixfaktorisierung. Ein Embedding ist ein Vektor aus wenigen Zahlen (hier 32), der ein Produkt beschreibt und gelernt wird. Im Buch von Lakshmanan et al. entsteht er als versteckte Schicht eines neuronalen Netzes, das auf eine Aufgabe trainiert wird. Es geht aber auch ohne Netz, und der Weg zeigt, was ein Embedding mathematisch ist. Drei Schritte:

1. **Ko-Klick-Matrix.** Für jedes Paar von Produkten zählt man, in wie vielen Trainingssitzungen beide angesehen wurden. Das ergibt eine quadratische Matrix C mit einer Zeile und Spalte je Produkt. Im Beispiel hat sie 23 500 mal 23 500 Zellen, von denen 0,218 % belegt sind (1,2 Millionen Zellen, da jedes Paar zweimal vorkommt).
2. **Positive Pointwise Mutual Information.** Rohe Zählungen sind verzerrt, weil beliebte Produkte mit allem gemeinsam vorkommen. Man vergleicht deshalb, wie oft zwei Produkte gemeinsam vorkommen, mit dem Wert, der bei Unabhängigkeit zu erwarten wäre:

   $$\mathrm{PMI}(i,j) = \log \frac{P(i,j)}{P(i)\cdot P(j)}.$$

   Negative Werte werden auf null gesetzt (PPMI): Paare, die nie gemeinsam vorkamen, hätten ein PMI von minus unendlich, und negative Werte sind bei wenigen Beobachtungen unzuverlässig.
3. **Abgeschnittene Singulärwertzerlegung.** Die Matrix M der PPMI-Werte wird als Produkt $M \approx U\Sigma V^T$ geschrieben, wobei nur die 32 größten Singulärwerte behalten werden. Nach dem Satz von Eckart und Young ist das die beste Näherung durch eine Matrix vom Rang 32 im Sinn der kleinsten Fehlerquadrate. Die Zeilen von $U\Sigma$ sind die Embeddings: Jedes Produkt wird auf 32 Zahlen komprimiert, und Produkte mit ähnlichem Ko-Klick-Profil, also ähnlicher Zeile in M, bekommen ähnliche Vektoren.

Das ist eine Matrixfaktorisierung, dasselbe Prinzip, das Empfehlungssysteme seit Langem nutzen [102]. Der Zusammenhang zu word2vec [7] ist eng: O. Levy und Y. Goldberg [103] zeigten, dass Skip-Gram mit Negative Sampling implizit eine um $\log k$ verschobene PMI-Matrix faktorisiert, und dass eine explizite PPMI-Matrix mit SVD bei Aufgaben zur Wortähnlichkeit konkurrenzfähige Vektoren liefern kann [104]. Der Unterschied liegt im Rechenweg: word2vec lernt die Faktoren schrittweise mit Gradientenabstieg, ohne die große Matrix je aufzubauen; die SVD baut sie auf und zerlegt sie direkt. Die Implementierung ist kurz:

```
def ppmi(c):
    """Positive Pointwise Mutual Information: max(0, log P(i,j) / (P(i) P(j)))."""
    c = c.tocoo()
    gesamt = c.sum()
    zeile = np.asarray(c.sum(1)).ravel()
    pmi = np.log(c.data * gesamt / (zeile[c.row] * zeile[c.col]))
    keep = pmi > 0
    return sp.csr_matrix((pmi[keep], (c.row[keep], c.col[keep])), shape=c.shape)


def lerne_embeddings(ans, n_produkte, dim=DIM, seed=0):
    """Tabelle (n_produkte x dim); Produkte ohne Ko-Klicks bleiben Nullvektoren."""
    m = ppmi(kookkurrenz(ans, n_produkte))
    svd = TruncatedSVD(n_components=dim, random_state=seed)
    e = svd.fit_transform(m)                          # = U * Sigma
    norm = np.linalg.norm(e, axis=1, keepdims=True)
    # Länge 1, damit das Skalarprodukt der Kosinus ist; Nullvektoren bleiben 0
    e = np.divide(e, norm, out=np.zeros_like(e), where=norm > 0)
    return e, m, svd
```

`TruncatedSVD` aus scikit-learn arbeitet mit einem randomisierten Verfahren [105] und kann deshalb mit der dünnbesetzten Matrix umgehen, ohne sie dicht zu machen. Die letzten Zeilen normieren jeden Vektor auf Länge 1, damit das Skalarprodukt zweier Produkte dem Kosinus des Winkels zwischen ihnen entspricht. Produkte ohne Ko-Klicks bekommen den Nullvektor.

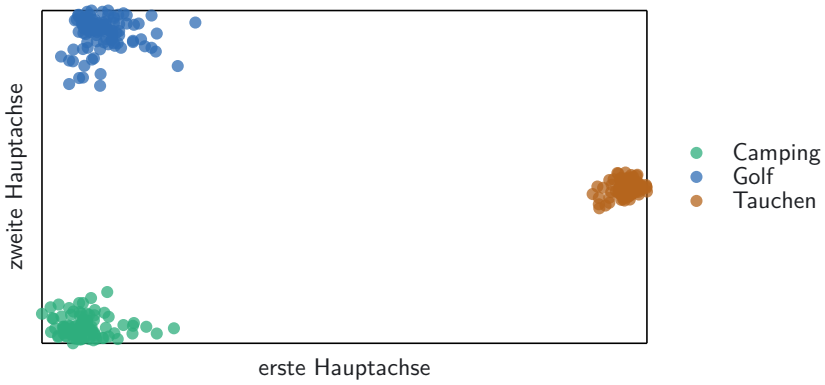


Abbildung 107: Je 100 Produkte aus drei Welten, deren 32-dimensionale Embeddings auf zwei Achsen projiziert sind (Hauptkomponenten; die beiden Achsen erfassen 80 % der Varianz). Die Farbe zeigt die Welt, die das Modell nie gesehen hat. Werte aus `03_embeddings.py`.

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy \
    python 03_embeddings.py
Ko-Klick-Matrix: 23,500 x 23,500, 1,204,686 belegte Zellen (0.218%)
PPMI-Matrix: 1,204,488 Einträge, Rang 32, Embedding-Tabelle (23500, 32)
Produkte mit Vektor: 21,595, ohne (nie gemeinsam gesehen): 1,905
Produkte mit mindestens 60 Ko-Klicks: 6,468

Biwak Lagerfeuer Tarp (Zelte, Welt Camping)
    0.96  Familienzelt Lagerfeuer Schuhe Plus    Schuhe        Camping
    0.95  Lagerfeuer Laufweste Hose Plus         Hosen         Camping
    0.95  Lagerfeuer Biwak Tasche Set            Rucksäcke     Camping
    0.95  Familienzelt Biwak Pulsmesser          Uhren         Camping
```

Zeigt die Faktorisierung, was sie soll? Für ein Zelt der Welt Camping sind die nächsten Nachbarn (größter Kosinus) eine Hose, ein Rucksack, ein Schuh und eine Uhr, alle aus Camping und alle aus einer anderen Kategorie. Das gibt die Tabelle nicht her: Nähe nach Anlass. Unter den zehn nächsten Nachbarn von 1500 geprüften Produkten stammen 100 % aus derselben Welt (Zufall: 6 %), aber nur 26 % aus derselben Kategorie. Das ist sauberer, als es in einem echten Shop wäre: Der Generator enthält genau eine Struktur, und die Faktorisierung findet sie.

Wirkung auf das Modell. `Merkmale` bildet den Mittelwert der Embeddings der drei angesehenen Produkte und hängt die 32 Zahlen als Features an. Das ist die größte Einzelverbesserung der Fallstudie. Verglichen wird mit einem Modell ohne Produkt-ID, damit sich die beiden Informationsquellen nicht vermischen; die AUC steigt von 0,625 auf 0,728, der Log-Loss fällt von 0,2401 auf 0,2247. In der Tabelle steht nirgends „Camping“, und doch erkennt das Modell am Vektor, ob die Sitzung in einer verkaufsstarken oder -schwachen Welt stattfindet. Für ein lineares Modell sind die 32 Zahlen vor allem eine Möglichkeit, *Ähnlichkeit* in einem Feature unterzubringen.

Kaltstart. Ein neues Produkt hat keine Ko-Klicks, also keinen Vektor. Eine Brücke zum nächsten Pattern hilft: Eine Ridge-Regression (lineare Regression mit einer Strafe für große Koeffizienten) lernt, aus dem TF-IDF-Vektor des Titels (siehe Pattern 4) den Embedding-Vektor der bekannten Produkte vorherzusagen, und wird auf Produkte ohne Ko-Klicks angewandt. Für 400 solche Produkte stammen dann 95 % der zehn nächsten bekannten Nachbarn aus der richtigen Welt, und die AUC auf Sitzungen mit neuem Produkt steigt von 0,684 auf 0,710. Das sind allerdings nur 1391 Sitzungen mit 106 Käufen, der Standardfehler einer einzelnen AUC liegt dort bei etwa 0,03; der Abstand ist ein Hinweis und kein Beleg. Dass die Brücke so gut gelingt, liegt auch daran, dass die Titel hier die Welt nennen. Der Preis: ein zweites Modell, das mitversioniert und mitgeliefert werden muss.

Trade-offs und Grenzen. Das Embedding ist ein Artefakt wie das Modell selbst. Ein Modell, das mit Embeddings der Version 1 trainiert wurde, darf nie Embeddings der Version 2 sehen: Die Achsen haben keine feste Bedeutung, ein neues Training dreht und mischt sie. Tabelle und Modell gehören deshalb in dasselbe versionierte Paket (siehe Abschnitt zum Dienst). Außerdem wächst die Kookkurrenzmatrix mit dem Katalog, und die Nachbarschaften geben das Verhalten der Besucher wieder, nicht Eigenschaften

der Produkte. Die Tabelle entsteht aus denselben Trainingssitzungen, auf denen das Modell lernt; weil nur Ansichten und keine Käufe eingehen, verrät sie dem Modell nichts über das Ziel, aber im Test sind die Vektoren etwas weniger zuverlässig als im Training. Nicht einsetzen: bei wenigen Werten, die ein Wörterbuch gut lernt, oder wenn es keine Verhaltensdaten gibt, aus denen Nähe entstehen kann.

Bezug zu Software Engineering. Ein Embedding verhält sich wie ein Suchindex: offline aufgebaut, online nur gelesen, und bei geändertem Aufbau inkompatibel zu allem, was mit der alten Version gebaut wurde.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Embeddings“; T. Mikolov, K. Chen, G. Corrado, und J. Dean [7], O. Levy und Y. Goldberg [103] sowie C. Guo und F. Berkhahn [26], Y. Koren, R. Bell, und C. Volinsky [102], O. Levy, Y. Goldberg, und I. Dagan [104].

14.4 Pattern 3: Feature Cross

Szenario und Problem. Der Preis eines Produkts beeinflusst die Kaufwahrscheinlichkeit, aber nicht überall gleich. Abbildung 108 zeigt die Kaufrate je Kategorie und Preisband in den Trainingsdaten. Bei Schuhen, Hosen, Zelten und Kochern sinkt sie mit steigendem Preis, bei Jacken, Rucksäcken, Helmen und Fahrradteilen steigt sie. Die Kundschaft sucht in der einen Kategorie das günstige Modell und in der anderen das Qualitätsprodukt. Das ist im Generator so angelegt, aber so etwas findet man auch in echten Daten, etwa wenn Kunden bei Verbrauchsmaterial sparen und beim Dauerläufer investieren.

Ein lineares Modell kann dieses Muster nicht abbilden, wenn Kategorie und Preis getrennte Features sind. Es lernt ein Gewicht für den Preis, das für alle Kategorien gleich ist, und ein Gewicht für jede Kategorie, das für alle Preise gleich ist. Gegenläufige Effekte heben sich im Mittel auf. Das zeigt der Versuch: Zerlegt man den Preis in acht Bänder und gibt dem Modell diese Bänder zusätzlich, ändert sich fast nichts (alle Modelle dieses Abschnitts ohne Produkt-ID; AUC 0,625 auf 0,627).

Die Lösung. Ein Feature Cross bildet aus zwei Merkmalen ein neues, das jede Kombination als eigenen Wert führt: „Zelte|Band 5“ ist eine Spalte, „Jacken|Band 5“ eine andere. Das Modell kann damit pro Zelle ein Gewicht lernen. Bei zwölf Kategorien und acht Bändern entstehen 96 Spalten; die Zahl wächst mit dem Produkt der Wertezahlen, und sobald sie unhandlich wird, kann man das Cross hashen (Pattern 1). In `Merkmale` steckt das in wenigen Zeilen:

```
        if self.kreuz:
            band = preisband(df.preis.to_numpy())
            kreuz = [f"{k}|{p}" for k, p in zip(df.kategorie, band)]
            spalten = position(kreuz, self.vok_kreuz)
            b.append(einsen(spalten, len(self.vok_kreuz)))
```

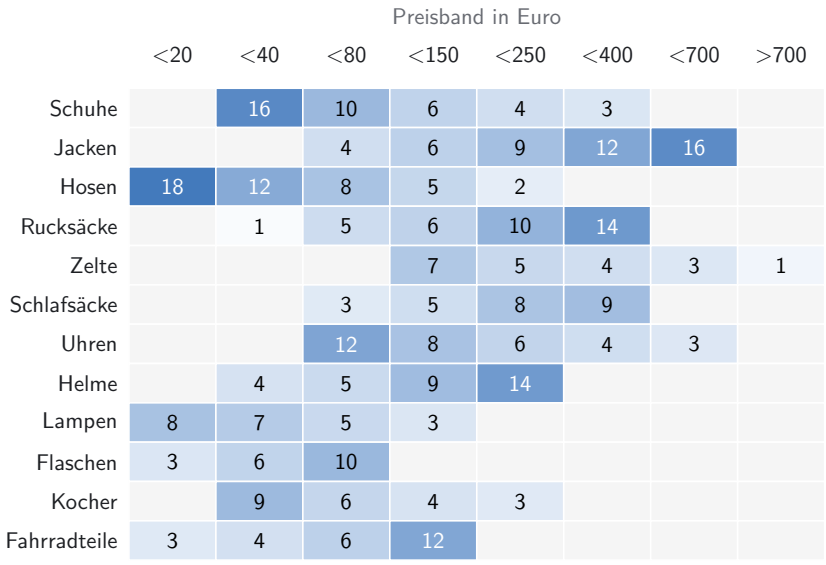


Abbildung 108: Kaufrate in Prozent je Kategorie und Preisband (Trainingstage). Leere Zellen haben weniger als 200 Sitzungen oder kommen nicht vor. Die Richtung des Preiseffekts wechselt von Zeile zu Zeile. Werte aus `04_feature_cross.py`.

Das Ergebnis: Die AUC steigt auf 0,665, der Log-Loss fällt von 0,2401 auf 0,2353. Der Gewinn kommt nicht aus mehr Information, denn Kategorie und Preis kannte das Modell schon, sondern aus einer anderen *Form*: Es darf jetzt sagen, dass sich der Preis je Kategorie unterschiedlich auswirkt. Ein Entscheidungsbaum oder ein neuronales Netz könnte solche Wechselwirkungen von selbst lernen, ein lineares Modell nicht. Das Pattern gibt einem einfachen Modell mit Domänenwissen einen Sprung, den man sonst mit einem komplexeren Modell erkaufen müsste. Gekreuzte Merkmale bilden auch den linearen „Wide"-Teil von Wide & Deep, der Kombinationen gezielt auswendig lernen soll [37].

Trade-offs und Grenzen. Jede Zelle des Cross braucht genug Beispiele. In der Abbildung 108 sind einige Zellen leer, weil Fahrradteile fast nie 700 Euro kosten; für solche Zellen lernt das Modell nichts, und das ist auch richtig so. Bei Crosses aus Merkmalen mit hoher Kardinalität (Marke mal Kategorie mal Gerät) explodiert die Spaltenzahl, und die meisten Zellen haben nur einen oder zwei Beispiele. Dann überwiegt das Rauschen. Wann man darauf verzichtet: bei Modellen, die Wechselwirkungen selbst lernen, wenn man nicht weiß, *welche* Merkmale zusammenwirken (man kreuzt nicht blind alles), und wenn die Zellen zu dünn besetzt sind.

Bezug zu Software Engineering. Ein Cross ist ein zusammengesetzter Schlüssel wie `(kategorie, preisband)` in einem Primärschlüssel oder einem `GROUP BY`: Man aggregiert über die Kombination, nicht über beide getrennt.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Feature Cross"; H.-T. Cheng *u. a.* [37].

14.5 Pattern 4: Multimodal Input

Szenario und Problem. Eine Produktzeile besteht aus verschiedenen Arten von Information: Kategorien (Kategorie, Gerät, Marke), Zahlen (Preis, Uhrzeit) und Text (der Titel). Ein Modell erwartet am Ende eine einheitliche Zahlenmatrix. Der Titel steckt voller Information, die nirgends sonst steht: Er nennt die Welt und den Zusatz „Pro" oder „Basic". Lässt man ihn weg, wirft man den Teil der Daten weg, der neue Produkte beschreibt, auch bevor es ein Verhalten dazu gibt.

Die Lösung. Jede Art von Eingabe wird in ihre passende Zahlendarstellung gebracht, die Ergebnisse werden zu einem Vektor zusammengefügt. Für Text nehmen wir *TF-IDF*: Jedes Wort des Vokabulars ist eine Spalte, der Wert ist die Häufigkeit des Wortes im Titel (term frequency) mal einem Gewicht, das seltene Wörter höher bewertet als häufige (inverse document frequency) [106], [107]. „Plus", das in vielen Titeln steht, zählt dadurch weniger als „Hüttentour". Weil Titel hier nur drei oder vier Wörter haben, ist die Häufigkeit fast immer 1, und TF-IDF ist im Wesentlichen eine gewichtete Wortliste. Die Tabellenmerkmale laufen weiter durch One-Hot und z-Score (Mittelwert abziehen, durch die Standardabweichung teilen); beide Blöcke werden spaltenweise nebeneinandergestellt.

Im Code sammelt `transform` die Blöcke in einer Liste und setzt sie am Ende mit `hstack` nebeneinander (der Ausschnitt zeigt das Ende der Methode). Die Reihenfolge der Blöcke spielt keine Rolle, muss aber im Training und im Dienst dieselbe sein, weil das Modell die Spalten nur über ihre Position kennt:

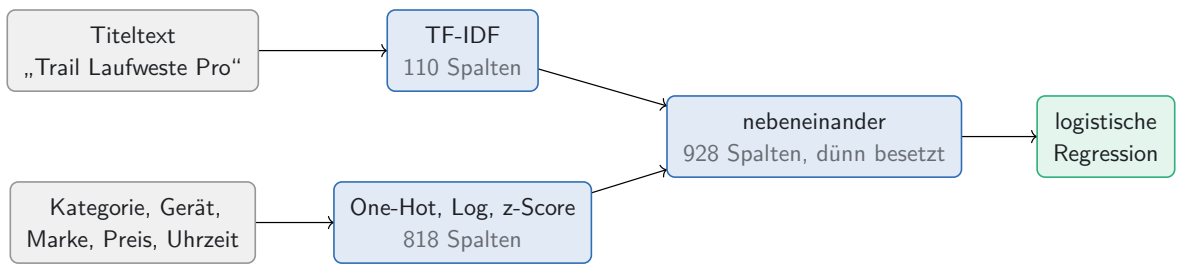


Abbildung 109: Multimodal Input: Jede Art von Eingabe bekommt ihre eigene Zahlendarstellung, danach werden die Blöcke zu einer Matrix zusammengesetzt. Die Spaltenzahlen stammen aus `05_multimodal.py`.

```
        if self.text:
            b.append(self.tfidf.transform(df.titel))
        if self.embedding is not None:
            b.append(sp.csr_matrix(self._mittel_embedding(df)))
        return sp.hstack(b, format="csr")
```

Der Text allein (0,683) schlägt die Tabelle allein (0,625), weil er die Welt verrät, und beide zusammen sind besser als jeder für sich (0,705). Die Wörter mit den größten Gewichten gehören zu Welten, die im Generator gut oder schlecht verkaufen: „Ultraleicht“, „Trail“ und „Laufweste“ (Trailrunning) erhöhen die Kaufwahrscheinlichkeit, „Bergstiefel“, „Trekking“ und „Gipfel“ (Bergwandern) senken sie. Das ist die Stärke linearer Modelle über Text: Man kann ihnen zuschauen.

Trade-offs und Grenzen. Erstens die Skalen: TF-IDF-Zeilen haben die Länge 1, die z-skalierten Tabellenwerte streuen um 0 mit Standardabweichung 1, und die Regularisierung, die große Gewichte bestraft und so vor Überanpassung schützt, behandelt beide gleich. Wie viel Gewicht ein Block bekommt, hängt dann auch von seiner Skalierung ab, nicht nur von seinem Nutzen. Zweitens die Überlappung mit Embeddings: Beide verraten die Welt, im Zusammenspiel (Tabelle am Ende des Kapitels) bringt der Text deshalb weniger, als sein Einzelwert vermuten lässt. Nicht einsetzen: wenn der Text nur Rauschen enthält oder durch andere Features abgedeckt ist, und bei langen Texten, wo ein Wortsack zu viel Struktur wegwirft; dann sind vortrainierte Sprachmodelle die bessere Wahl.

Bezug zu Software Engineering. Das Pattern ist ein Adapter: Jede Quelle hat ihr eigenes Format, am Ende steht eine gemeinsame Schnittstelle.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 2, Pattern „Multimodal Input“; K. Spärck Jones [106] und G. Salton und C. Buckley [107] zu TF-IDF.

14.6 Pattern 8: Cascade

Szenario und Problem. Der Shop will zwei Dinge wissen: ob die Sitzung kauft und wenn ja, in welcher Kategorie. Naheliegend ist ein einziges Modell mit 13 Klassen (kein Kauf plus zwölf Kategorien). Das Problem ist die Schieflage: 93 % der Sitzungen kaufen nicht, und die Frage nach der Kategorie ist nur für den kleinen Rest sinnvoll. Ein Modell, das beides gleichzeitig lernt, bekommt die Kategorien nur aus diesem kleinen Teil der Zeilen beigebracht und muss sie gegen die dominierende Klasse „kein Kauf“ abwägen.

Die Lösung. Eine Cascade zerlegt die Aufgabe in eine Folge von Modellen, die jeweils eine einfachere Frage beantworten. Stufe 1 schätzt, ob die Sitzung kauft, und sie ist ein gewöhnlicher binärer Klassifikator auf allen Trainingszeilen. Stufe 2 schätzt die Kategorie und wird nur auf den Kaufsitzungen trainiert. Im Betrieb läuft Stufe 2 nur, wenn Stufe 1 die Sitzung für aussichtsreich hält.

```
# Variante B: Cascade. Stufe 1 auf allen Zeilen, Stufe 2 nur auf den Käufern
stufe1 = modell().fit(X_tr, kauf_tr)
p_kauf_c = stufe1.predict_proba(X_te)[:, 1]
kaeufer = kauf_tr == 1
stufe2 = LogisticRegression(C=0.5, max_iter=1000).fit(X_tr[kaeufer], kat_tr[kaeufer])
kat_c = stufe2.classes_[stufe2.predict_proba(X_te).argmax(1)]
```

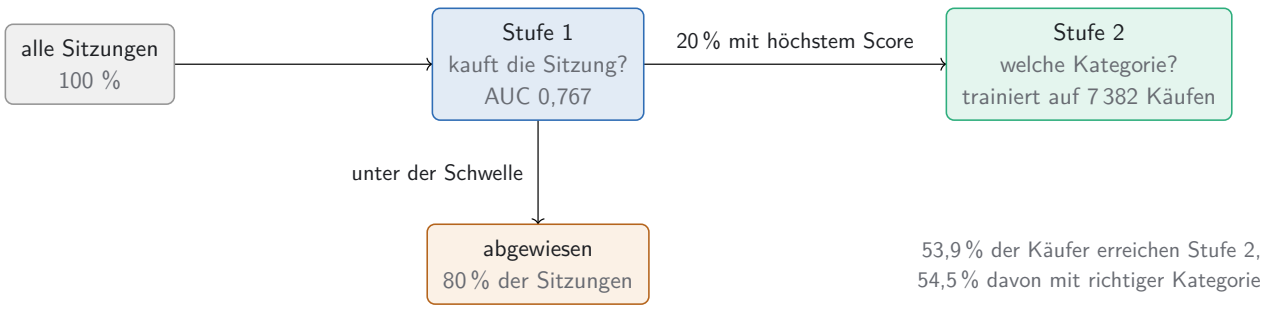


Abbildung 110: Die Cascade im Betrieb. Die Schwelle ist so gewählt, dass 20 % der Sitzungen Stufe 2 erreichen. Werte aus `06_cascade.py`.

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy \
    python 06_cascade.py
                                Kauf-AUC  Kategorie richtig
ein Modell, 13 Klassen             0.752              51.6%
Cascade (Stufe 1 + Stufe 2)        0.767              51.7%
Heuristik: Kategorie der Seite                        45.4%
immer die häufigste Kategorie                         23.9%

Betrieb: Stufe 2 nur für die Sitzungen mit dem höchsten Score
 Anteil an Stufe 2   Käufer erfasst  Kategorie richtig (erfasst)
               10%           37.3%                       57.6%
               20%           53.9%                       54.5%
               30%           66.2%                       54.1%
              100%          100.0%                       51.7%
```

Was man daraus liest. Die Cascade ist bei der Kaufvorhersage etwas besser als das Einzelmodell (AUC 0,767 gegenüber 0,752), bei der Kategorie gleichauf (51,7 % gegenüber 51,6 % bei den Käufern). Bei rund 2900 Käufen im Test liegt dieser Unterschied weit unter einer Standardabweichung; die Genauigkeit der Kategorie verbessert das Pattern also nicht. Es verbessert den *Betrieb*: Stufe 2 muss nur für ein Fünftel der Sitzungen laufen und erfasst dabei gut die Hälfte der Käufer, sie lässt sich getrennt trainieren, evaluieren und austauschen, und die Kauf-AUC verlangt kein Opfer.

Ebenfalls im Skript steht die Heuristik „Kategorie der Seite“: Wer das Produkt gerade ansieht, kauft mit 45,4 % Wahrscheinlichkeit aus dieser Kategorie. Die Modelle schlagen sie um sechs Prozentpunkte. Eine solche Heuristik gehört in jedes Projekt, bevor man Modelle baut, weil sie festlegt, was ein Modell mindestens leisten muss (Pattern 28, Heuristic Benchmark).

Trade-offs und Grenzen. Fehler pflanzen sich fort: Ein Käufer, den Stufe 1 abweist, bekommt nie eine Kategorie. Stufe 2 wird auf echten Käufern trainiert, sieht im Betrieb aber auch falsch-positive Sitzungen und hat für sie keine richtige Antwort; hier ist das harmlos, weil niemand sie nutzt. Die Schwelle zwischen den Stufen übersetzt ein Geschäftsziel (wie viele Käufer will ich erreichen?) in ein Kostenziel (wie viele Anfragen kann ich verarbeiten?). Nicht einsetzen: wenn die zweite Frage für alle Fälle sinnvoll ist oder ein einziges Modell genauso gut ist und die Pipeline einfacher bleibt.

Bezug zu Software Engineering. Pipes and Filters mit einem billigen ersten Filter: Wie ein billiger Vorfilter vor einer teuren Prüfung lässt Stufe 1 nur die Kandidaten durch, für die sich der zweite Schritt lohnt. Anders als bei einem Bloom-Filter geht dabei aber auch ein Teil der echten Treffer verloren.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 3, Pattern „Cascade“; als historisches Beispiel einer Cascade von Klassifikatoren P. Viola und M. Jones [43] zum Heuristic Benchmark Kapitel 7 des Buchs.

14.7 Pattern 26: Feature Store

Szenario und Problem. Produkte, die in der Vorwoche oft gekauft wurden, werden mit höherer Wahrscheinlichkeit wieder gekauft. Der Shop möchte dieses Signal nutzen. Drei Zahlen pro Produkt genügen: die Zahl der Ansichten und die Zahl der Käufe in den letzten sieben Tagen sowie die daraus berechnete Kaufrate. Je Sitzung werden sie über die drei angesehenen Produkte gemittelt (die Zähler als Logarithmus von 1 plus Zahl). Weil ein Produkt mit einer Ansicht und einem Kauf sonst eine Rate von 100 % hätte, wird die Rate geglättet:

$$\frac{K + m \cdot r_0}{A + m}$$

mit den Käufen K, den Ansichten A, einer groben Durchschnittsrate $r_0 = 0{,}017$ und dem Gewicht $m = 30$. Wenige Beobachtungen ziehen die Rate damit Richtung Durchschnitt, viele überstimmen ihn. Das ist ein Spezialfall der Mittelwert-Kodierung kategorialer Merkmale [108].

Solche Features haben zwei Schwierigkeiten. Sie müssen im Training und im Betrieb mit derselben Definition berechnet werden, und sie müssen zeitlich stimmen: Das Feature einer Trainingszeile darf nur

Wissen enthalten, das zum Zeitpunkt der Sitzung schon existierte. Die zweite Schwierigkeit ist heimtückischer, weil ihre Verletzung kein Fehler erzeugt, sondern ein besseres Modell vortäuscht.

Die Berechnung. Das Ereignisprotokoll des Shops wird zu einer Tabelle Tag mal Produkt aggregiert (Ansichten und Käufe je Tag), und über die Tage werden kumulierte Summen gebildet. Die Zahl im Zeitfenster ist dann die Differenz zweier Zeilen. Welche Zeilen das sind, entscheidet über Korrektheit:

```
    def fenster(self, tag, produkt, modus):
        """Ansichten und Käufe des Produkts im Fenster, das für `modus` gilt.

        pit     : Tage tag-7 bis tag-1 (Stand am Morgen des Sitzungstags)
        mit_tag : Tage tag-6 bis tag   (enthält die Sitzung selbst)
        gesamt  : alle Tage            (enthält auch die Zukunft)
        """
        tag = np.asarray(tag)
        if modus == "pit":
            von, bis = np.maximum(tag - FENSTER, 0), tag
        elif modus == "mit_tag":
            von, bis = np.maximum(tag - FENSTER + 1, 0), tag + 1
        elif modus == "gesamt":
            von, bis = np.zeros_like(tag), np.full_like(tag, self.tage)
        else:
            raise ValueError(modus)
        ans = self.cv[bis, produkt] - self.cv[von, produkt]
        kauf = self.ck[bis, produkt] - self.ck[von, produkt]
        return ans, kauf
```

Das Leck und seine Varianten. Das Skript `07_feature_store_leck.py` berechnet das Feature auf drei Arten und trainiert mit jeder ein eigenes Modell. Abbildung 111 zeigt die Fenster für eine Sitzung an Tag 55.

Die erste Variante, die Kennzahl über den gesamten Datensatz, ist die, die man mit einer einzigen `groupby` -Zeile erzeugt. Die zweite, sieben Tage bis einschließlich heute, entsteht, wenn ein nächtlicher Job die Zähler am Ende eines Tages schließt und das Training die Zähler „des Tages“ an die Sitzung dieses Tages joint. Beide enthalten den Kauf, der vorhergesagt werden soll. Nur die dritte, Tag minus sieben bis Tag minus eins, enthält ausschließlich Wissen vom Morgen des Sitzungstags.

```
Terminal
$ uv run --with scikit-learn --with pandas --with numpy \
    python 07_feature_store_leck.py
Kaufrate der drei Produkte allein als Score (AUC, Testtage):
  gesamter Datensatz                    AUC 0.831
  7 Tage bis einschließlich heute       AUC 0.885
  7 Tage bis gestern (Point in Time)    AUC 0.682

Training mit                           AUC offline  AUC online  Differenz
gesamter Datensatz                           0.823       0.714     -0.109
7 Tage bis einschließlich heute              0.878       0.744     -0.134
7 Tage bis gestern (Point in Time)           0.768       0.768     +0.000
ohne Popularitätsfeatures                    0.744       0.744     +0.000
```

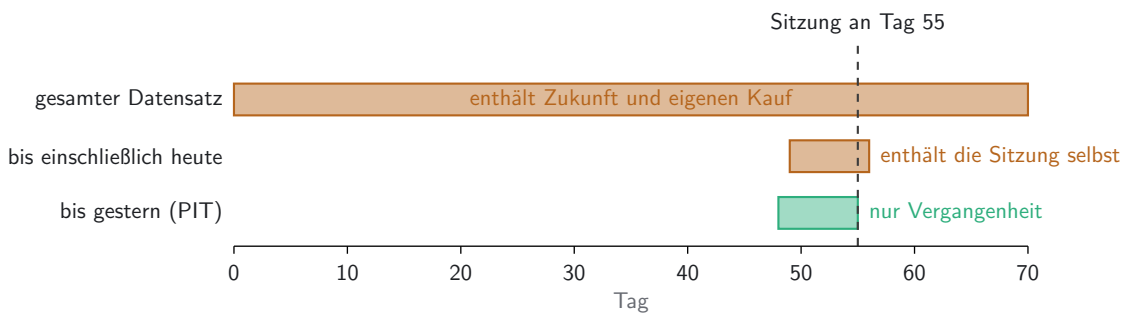


Abbildung 111: Drei Zeitfenster für das Popularitätsfeature einer Sitzung an Tag 55. Orange: das Fenster enthält Wissen, das zum Zeitpunkt der Sitzung nicht existierte. Grün: nur Tage vor der Sitzung.

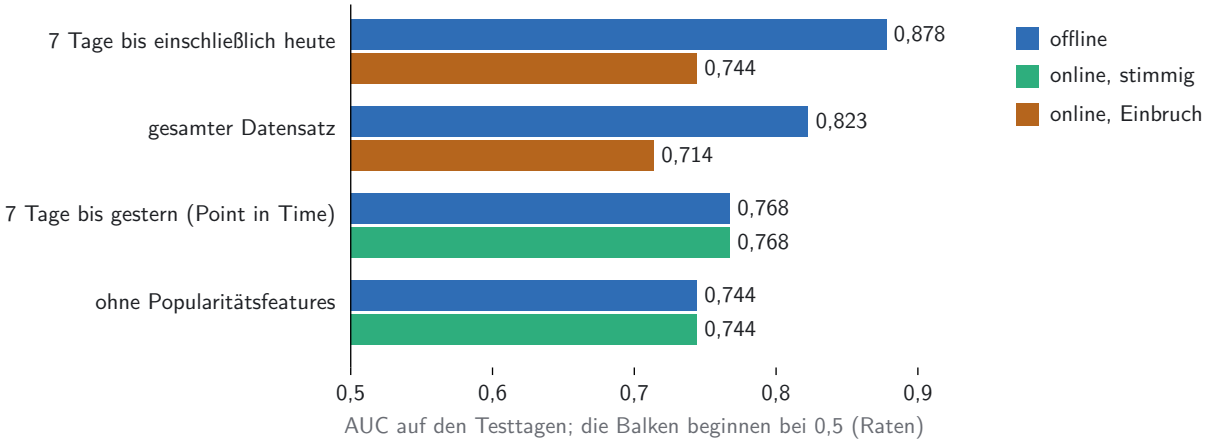


Abbildung 112: Trainiert wird jeweils mit der Variante, die links steht. Blau: Güte auf Testdaten, deren Popularitätsfeatures nach derselben Variante berechnet sind (was das Team im Notebook sieht). Grün/Orange: Güte, wenn die Features so berechnet werden, wie der Dienst sie liefert (Point in Time). Werte aus `07_feature_store_leck.py`.

Daran lassen sich drei Warnzeichen ablesen, die auch außerhalb dieses Beispiels gelten. Erstens ist ein einzelnes Popularitätsfeature *zu gut*: Allein trennt die mittlere Kaufrate der drei Produkte Käufer und Nicht-Käufer mit einer AUC von 0,885 (Fenster bis heute) beziehungsweise 0,831 (gesamter Datensatz), im korrekten Fall nur mit 0,682. Ein Zähler der Vorwoche, der besser trennt als das gesamte korrekte Modell (AUC 0,768), ist verdächtig. Zweitens fällt die Güte vom Notebook zum Betrieb: von 0,823 auf 0,714 beim gesamten Datensatz und von 0,878 auf 0,744 beim Fenster bis heute, wenn das Modell die Werte bekommt, die der Dienst tatsächlich berechnen kann. Drittens ist das Leck-Modell im Betrieb nicht besser als ein Modell ganz ohne Popularitätsfeature (0,744): Es hat gelernt, sich auf ein Signal zu verlassen, das es im Betrieb nie gibt.

Die Behebung. Das Pattern Feature Store ist die organisatorische Antwort. Ein Feature Store hat zwei Gesichter: Der *Offline-Store* liefert für das Training zu jedem Beispiel den Wert, der zu dessen Zeitstempel galt (der Point-in-Time-Join, in Feast `get_historical_features` [61]), der *Online-Store* liefert im Betrieb den aktuellen Wert mit niedriger Latenz. Beide speisen sich aus *derselben* Definition, sodass das Feature nicht auseinanderlaufen kann (Training-Serving Skew, Pattern 21). Hier ist der Feature Store auf seinen Kern reduziert: die Klasse `Protokoll` mit rund 25 Zeilen und ein Schnappschuss der Zähler zum Stichtag (Abbildung 113). Ein Produkt wie Feast ergänzt die Verwaltung vieler Features, Zeitstempel je Ereignis statt je Tag, die Aktualisierung des Online-Stores und Zugriffsrechte.

Trade-offs und Grenzen. Der Tag als kleinste Einheit ist konservativ: Er verwirft, was am Vormittag desselben Tages schon bekannt war, ist dafür aber einfach und prüfbar. Wer feiner arbeitet, muss die Latenz des Online-Stores in die Definition aufnehmen: Ein stündlich nachgezogener Zähler ist im Betrieb bis zu eine Stunde veraltet, und das Training sollte dieselbe Verzögerung abbilden. Ein Feature Store ersetzt außerdem keine Sorgfalt beim Entwurf: Auch aus korrekten Zeitfenstern kann man sich ein Leck bauen, etwa mit einem Feature, das in der Praxis nur für gekaufte Produkte befüllt wird. Nicht einsetzen: bei einem Modell mit wenigen, nicht zeitabhängigen Features; der Aufwand rechnet sich mit der Zahl der Modelle und der zeitabhängigen Features.

Bezug zu Software Engineering. Der Point-in-Time-Join ist ein „as of"-Join auf einer temporalen Tabelle, wie ihn auch SQL kennt, und das Prinzip ist das von Event Sourcing: Der Zustand zu einem Zeitpunkt ist das, was sich aus den bis dahin eingetretenen Ereignissen ergibt. Die gemeinsame Definition für Training und Betrieb ist Single Source of Truth; sie mindert die Kosten von Datenabhängigkeiten, die D. Sculley *u. a.* [11] beschreiben.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 6, Pattern „Feature Store"; zum Datenleck S. Kaufman, S. Rosset, C. Perlich, und O. Stitelman [8] Feast-Dokumentation [61].

14.8 Pattern 16 und 27: Dienst und Modellversionen

Szenario und Problem. Das Modell soll in den Shop. Sobald es eine Version 2 gibt, die mehr Features nutzt, darf die Version 1 nicht von einem Tag auf den anderen verschwinden: Andere Teams rufen sie

auf und brauchen Zeit für die Umstellung, und wenn v2 sich im Betrieb als schlechter erweist, muss man zurückgehen können. Zugleich soll der Dienst skalieren, also ohne Gedächtnis zwischen Anfragen auskommen.

Die Lösung. Der Dienst `code/14/dienst.py` ist mit FastAPI [109] gebaut. Das Pattern Stateless Serving Function verlangt, dass die Vorhersage eine Funktion ist, die nur von ihrer Eingabe abhängt. In der Praxis heißt das: Das Modell wird *einmal* beim Start des Prozesses geladen und danach nur gelesen, und kein Handler schreibt in einen Speicher, den die nächste Anfrage sieht.

```
MODELLE: dict = {}      # einmal beim Start gefüllt, danach nur gelesen
SCHNAPPSCHUSS: dict = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    ordner = Path(__file__).with_name("modelle")
    for datei in sorted(ordner.glob("v*.joblib")):
        MODELLE[datei.stem] = joblib.load(datei)
    s = np.load(ordner / "store.npz")
    SCHNAPPSCHUSS.update(ans=s["ans"], kauf=s["kauf"])
    yield
    MODELLE.clear()
```

Beim Start (`lifespan`) werden alle Versionen aus `modelle/` geladen. Jede Datei ist ein *Paket*: der angepasste `Merkmale`-Transformer mit Wörterbüchern, TF-IDF-Vokabular und Embedding-Tabelle sowie das Modell, dazu Metadaten. Das ist wichtig, weil sich Merkmalsbau, Embedding und Gewichte nicht einzeln austauschen lassen. Die Vorhersage selbst ist eine reine Funktion (`paket` schlägt die Version nach und antwortet bei einer unbekannten mit 404):

```
def paket(version: str) -> dict:
    if version not in MODELLE:
        raise HTTPException(404, f"unbekannte Version {version}; "
                                 f"vorhanden: {sorted(MODELLE)}")
    return MODELLE[version]


def vorhersage(version: str, a: Anfrage) -> float:
    """Reine Funktion von (Version, Anfrage): nichts wird gespeichert."""
    p = paket(version)
    ans, kauf = popularitaet([a.vorher_1, a.vorher_2, a.produkt_id])
    stand = {k: v[0] for k, v in aggregiere(ans, kauf).items()}
    zeile = pd.DataFrame([{**a.model_dump(), **stand}])
    x = p["merkmale"].transform(zeile)
    return float(p["klassifikator"].predict_proba(x)[0, 1])
```

Die beiden Pakete und einen Schnappschuss der Popularität zum Stichtag schreibt das Skript `trainiere_modelle.py`; im Dienst kommt die Popularität aus diesem Schnappschuss; im echten Betrieb stünde hier der Online-Store. Der Schnappschuss ändert sich während des Prozesses nicht. Ein Produkt, das der Store nicht kennt, bekommt Popularität null.

Das Pattern Model Versioning ist dann nur noch Routing: Die Version steht im Pfad, beide Pakete sind im selben Prozess geladen, und `/{version}/info` nennt Beschreibung und Güte der Version.

```
@app.post("/{version}/vorhersage", response_model=Antwort)
def vorhersage_route(version: str, anfrage: Anfrage):
    wahrscheinlichkeit = vorhersage(version, anfrage)
    return Antwort(version=version,
                   kaufwahrscheinlichkeit=round(wahrscheinlichkeit, 4))
```

Getestet wird mit dem `TestClient` von FastAPI (er braucht `httpx`), der den Dienst im selben Prozess ohne Netzwerk aufruft. Der Test prüft auch, was Zustandslosigkeit praktisch bedeutet: Dieselbe Anfrage liefert dieselbe Antwort, egal was zwischen zwei Aufrufen geschehen ist.

```
Terminal
$ uv run --with scikit-learn --with pandas --with numpy \
    --with fastapi --with httpx python test_dienst.py
v1: AUC 0.68, Tabelle, gehashte ID, Kreuz Kategorie x Preis
v2: AUC 0.768, v1 + Titeltext, Embeddings, Popularität

bekanntes Produkt    v1: 0.1420
bekanntes Produkt    v2: 0.2886
neues Produkt 99999  v1: 0.1753
neues Produkt 99999  v2: 0.4283

gleiche Anfrage nach anderen Anfragen: identische Antwort
unbekannte Version  : 404 unbekannte Version v3; vorhanden: ['v1', 'v2']
ungültige Eingabe   : 422 Input should be less than or equal to 23
```

Trade-offs und Grenzen. Die Version im Pfad ist die einfachste Art zu versionieren, lässt aber die Clients eine Entscheidung treffen, die sie meist nicht treffen wollen; Alternativen sind ein Header oder ein Gateway, das Anfragen aufteilt (Canary, A/B-Test). Zwei Versionen im selben Prozess verdoppeln den Speicher; bei großen Modellen trennt man sie in verschiedene Prozesse. Gespeichert werden die Pakete mit `joblib`, das auf Python-Pickle beruht: Man darf nur eigene, vertrauenswürdige Dateien laden, und die Pakete sind an die Bibliotheksversionen gebunden, mit denen sie erzeugt wurden. Der `TestClient` ist kein Lasttest und keine Überwachung; dazu gehören Pattern 18 (Continued Model Evaluation) und Pattern 20 (Keyed Predictions). Nicht einsetzen: wenn eine nächtliche Batch-Berechnung genügt (Pattern 17); sie braucht weder Dienst noch Versionierung.

Bezug zu Software Engineering. Beide Patterns sind bekannte Dienstentwurfsprinzipien in ML-Verkleidung. Zustandslosigkeit ist der Grund, warum man einen Dienst horizontal skalieren und jederzeit neu starten darf. Versionierte Endpunkte sind die übliche Art, eine HTTP-Schnittstelle weiterzuentwickeln, ohne Clients zu brechen (zum Architekturstil REST siehe [52]); die Regel, dass ein Paket alles enthält, was zusammengehört, ist das Prinzip eines unveränderlichen Artefakts, wie es auch bei Container-Images gilt.

Quellen. V. Lakshmanan, S. Robinson, und M. Munn [1], Kapitel 5, Pattern „Stateless Serving Function“, und Kapitel 6, Pattern „Model Versioning“; FastAPI-Dokumentation [109].

14.9 Das Gesamtbild: Architektur und Wirkung

Abbildung 113 setzt die Teile zusammen. Links steht, was täglich offline geschieht: Aus dem Ereignisprotokoll entstehen Zähler und Embeddings, `Merkmale` wird auf den Trainingsdaten angepasst, die beiden Modellstufen werden trainiert, und das Ergebnis wird als versioniertes Paket abgelegt. Rechts steht der Betrieb: Der Dienst lädt die Pakete einmal, holt die Popularität aus dem Schnappschuss des Online-Stores und beantwortet Anfragen ohne eigenes Gedächtnis. Die gestrichelte Linie dazwischen ist die Stelle, an der Training und Betrieb auseinanderlaufen können, und jedes Pattern der Fallstudie sorgt dafür, dass sie schmal bleibt.

Wirkung auf die Güte. Tabelle 18 zeigt, was jede Zutat zum Modell beiträgt, in der Reihenfolge der Patterns im Buch. Die Spalte „allein“ fügt die Zutat nur zum Ausgangsmodell hinzu, die Spalte „kumuliert“ baut auf allen vorherigen Zutaten auf. Die Cascade ist nicht Teil der Tabelle, weil sie eine andere Aufgabe beantwortet (die Kategorie), und der Dienst hat keine Auswirkung auf die Güte.

Vier Beobachtungen. Erstens addieren sich die Beiträge nicht. Der Text allein hebt die AUC um +0,045, auf dem Stand mit Embeddings und Feature Cross aber nur um +0,000, weil Text und Embeddings beide die Welt verraten; die Summe der Einzelwirkungen (+0,182) übertrifft die Gesamtwirkung (+0,101) deutlich. Wer Patterns stapelt, sollte jede Zutat daher auch in der Ablation prüfen: Was verliert das Gesamtmodell, wenn ich sie wieder herausnehme? Eine Zutat, die dann nichts kostet, ist laufender Betriebsaufwand ohne Nutzen. Zweitens kostet das Hashed Feature gegenüber dem Wörterbuch ein wenig Güte (−0,010), und das ist in Ordnung, weil sein Gewinn im Betrieb liegt. Drittens ist das Popularitätsfeature die Zutat, deren Wert sich am leichtesten falsch messen lässt: Der ehrliche Wert (+0,024) steht neben dem, den das Leck vorgetäuscht hätte (+0,134). Viertens steigt die Güte des Gesamtmodells von AUC 0,667 auf 0,768 und von AP 0,161 auf 0,248, bei einem Zufallsniveau der AP von 0,068.

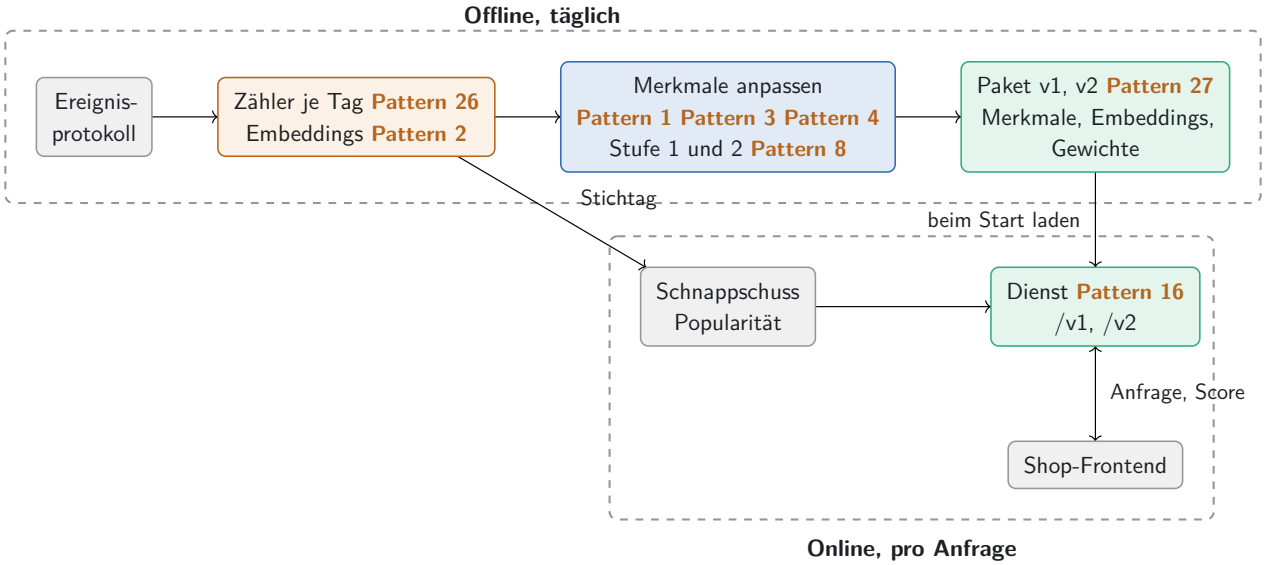


Abbildung 113: Architektur der Fallstudie. Die Nummern bezeichnen die Patterns, die an der jeweiligen Stelle wirken.

Tabelle 18: Wirkung der Zutaten auf den Testtagen, einzeln und aufbauend (Ausgabe von `08_gesamt.py`). AUC: höher ist besser; Log-Loss: niedriger ist besser.

Stufe	Spalten	AUC allein	AUC kumuliert	Δ kumuliert	AP	Log-Loss
Ausgangsmodell (Wörterbuch für ID und Marke)	19 110		0,667		0,161	0,2328
+ Hashed Feature (Produkt-ID, B = 2^{14})	17 202	0,657	0,657	−0,010	0,151	0,2347
+ Embeddings (32 Dim., Mittel der 3 Produkte)	17 234	0,734	0,730	+0,073	0,190	0,2238
+ Feature Cross (Kategorie x Preisband)	17 338	0,688	0,744	+0,014	0,201	0,2207
+ Multimodal Input (Titeltext TF-IDF)	17 448	0,712	0,744	+0,000	0,200	0,2210
+ Feature Store (Popularität, Point in Time)	17 451	0,726	0,768	+0,024	0,248	0,2133

Wirkung auf den Betrieb. Die Güte ist nur ein Teil der Wirkung. Tabelle 19 stellt zusammen, was jedes Pattern im Betrieb verändert, auch dort, wo die AUC unverändert bleibt.

Was in einem echten Shop anders wäre. Die Daten sind sauberer als echte: Die Embeddings finden eine einzige, perfekt erkennbare Struktur, die Titel sind kurz und folgen einem festen Muster, und die Effekte sind groß und konstant. In einem echten Shop wären die Nachbarschaften gröber, der Text unordentlich, Popularität und Kaufneigung verschränkter, und die Beiträge der einzelnen Patterns kleiner. Die Reihenfolge der Beobachtungen würde dennoch ähnlich ausfallen, denn sie folgt aus den Mechanismen und nicht aus den Zahlen. Und ob der Score im Shop etwas bewirkt, entscheidet am Ende kein Offline-Maß, sondern ein kontrolliertes Online-Experiment [90].

14.10 Das Wichtigste in Kürze

- Ein Hashed Feature ersetzt ein Wörterbuch durch eine Rechenvorschrift. Es kostet hier −0,010 AUC und spart die Pflege des Wörterbuchs; der Nutzen wächst mit der Kardinalität.
- Ein Embedding aus Ko-Klick-Daten ist eine Matrixfaktorisierung (PPMI-Matrix der Produktpaare, abgeschnittene SVD, Zeilen als Vektoren). Es trägt Nähe in ein Modell, die in keiner Spalte steht, und ist das wirksamste Pattern dieser Fallstudie.
- Ein Feature Cross lässt ein lineares Modell Wechselwirkungen lernen, die Einzelspalten nicht ausdrücken (Preiseffekt je Kategorie).
- Multimodal Input übersetzt jede Eingabeart passend in Zahlen und setzt die Blöcke zusammen. Text und Embedding verraten dieselbe Information und addieren sich nicht.

Tabelle 19: Wirkung der Patterns auf Güte und Betrieb, mit Werten aus den Läufen dieses Kapitels.

Pattern	Wirkung auf die Güte	Wirkung auf den Betrieb
1 Hashed Feature	AUC −0,010 gegenüber dem Wörterbuch	Kein Wörterbuch im Paket, neue IDs ohne Sonderfall; Bucket-Zahl nur mit neuem Training änderbar
2 Embeddings	AUC +0,073 kumuliert, größter Beitrag	Tabelle im versionierten Paket; Neuaufbau bei wachsendem Katalog
3 Feature Cross	AUC +0,014 kumuliert	96 Spalten; zu dünne Zellen lernen nichts
4 Multimodal Input	AUC +0,000 kumuliert, +0,045 allein	TF-IDF-Vokabular im Paket; überlappt mit Embeddings
8 Cascade	Kauf-AUC +0,015, Kategorie-Treffer gleich	Stufe 2 für 20 % der Sitzungen, erreicht 54 % der Käufer; Stufen getrennt austauschbar
26 Feature Store	AUC +0,024 kumuliert; ein Leck täuscht +0,134 vor	Eine Definition für Training und Betrieb; täglicher Schnappschuss
16, 27 Dienst und Versionen	keine	Zwei Versionen parallel; Rückfall durch Pfadwechsel; kein Zustand zwischen Anfragen

- Eine Cascade lohnt sich, wenn die zweite Frage nur für wenige Fälle gestellt werden muss. Hier bringt sie Betriebsvorteile statt Genauigkeit.
- Popularitätsfeatures müssen Point in Time berechnet werden. Ein Zähler, der zu gut trennt, und eine Lücke zwischen Offline- und Online-Güte sind die Warnzeichen eines Lecks.
- Ein Dienst, der sein Modell einmal lädt und pro Anfrage nur liest, ist zustandslos. Versionen im Pfad erlauben parallelen Betrieb und Rückfall, und das Paket einer Version enthält alles, was zusammengehört.
- Die Wirkung mehrerer Patterns misst man einzeln und aufbauend und prüft zusätzlich per Ablation, was beim Entfernen einer Zutat verloren geht. Die Summe der Einzelwirkungen überschätzt das Ganze.

SO ERKLÄRE ICH ES

Erzählen Sie die Fallstudie als Geschichte eines Teams, das ein Feature nach dem anderen hinzufügt, und lassen Sie die Studierenden vor jeder Zahl schätzen, was das Feature bringt. Die Überraschungen sind die Lernmomente: Der Text allein ist stark, im Verbund wenig; die Popularität wird erst durch das Leck „gut“. Zeigen Sie Abbildung 112 und lassen Sie erklären, warum das orange Modell schlechter ist als das ohne Feature.

Verständnisfrage. Ein Teammitglied meldet, ein neues Popularitätsfeature habe die AUC von 0,74 auf 0,88 gehoben. Was prüfen Sie zuerst?

Antwort. Wie das Feature für die Trainingszeilen berechnet wird: welches Zeitfenster, ob es Ereignisse des Zeitstempels der Zeile oder späterer Tage einschließt. Dann die Güte mit Features, so wie der Dienst sie bei einer Anfrage hätte. Ein Sprung dieser Größe durch einen Zähler der Vorwoche ist ein Warnsignal und kein Erfolg.

Weiterführende Literatur. Zu den Patterns selbst führen die Kapitel 2, 3 und 6 von V. Lakshmanan, S. Robinson, und M. Munn [1]. Zum Zusammenhang von Embeddings und Matrixfaktorisierung siehe O. Levy und Y. Goldberg [103], zum Datenleck Kaufman et al. [8], zu den Folgekosten von Datenabhängigkeiten D. Sculley *u. a.* [11] und für eine Checkliste zu ML-Systemen E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55].

15 Die Patterns unterrichten

Dieses Kapitel ist für die Lehre geschrieben und setzt voraus, dass Sie die Kapitel 5 bis 10 gelesen haben. Es sammelt, was man zusätzlich braucht, um die Patterns Studierenden zu vermitteln: einen Ablauf, der ML-Grundlagen in 20 Minuten aufbaut, die Missverständnisse, die immer wieder auftreten, Fragen mit Musterantworten, die Verbindung zu den vier Praxisprojekten und eine Anleitung, den Report des Trainers im Coaching zu nutzen. Die Demonstrationen im Kapitel sind lauffähig und liegen unter `code/10-lehre/`; die gezeigten Ausgaben stammen aus tatsächlichen Läufen.

15.1 Der rote Faden der Lehrveranstaltung

Die Studierenden begegnen den Patterns an drei Orten. Die Folien (Deck 07 und 08) erklären das Problem und die Mechanik je Pattern, das Handbuch ist Ihre Vertiefung, und in den Praxisprojekten treffen die Gruppen auf die Symptome in echtem, absichtlich fehlerhaftem Code. Der Trainer prüft, ob die Verträge der Patterns erfüllt sind, und das Coaching übersetzt rote Verträge in Fragen.

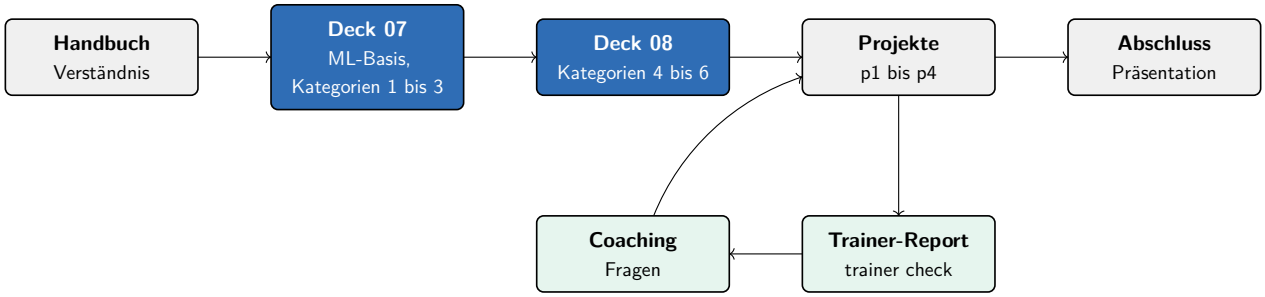


Abbildung 114: Lernorte der Lehrveranstaltung. Der Kreislauf aus Trainer-Report und Coaching läuft während der Projektarbeit mehrfach.

Eine Folge daraus bestimmt, wie Sie unterrichten: Die Gruppen sehen ein Symptom (der Service liefert andere Zahlen als das Training, ein Vertrag ist rot), bevor sie das Pattern beim Namen nennen können. Beginnen Sie deshalb in den Folien mit dem Symptom und führen Sie den Namen erst danach ein. Der Name ist die Abkürzung für eine Lösung, die man zuerst als Problem gespürt haben muss.

15.2 ML-Grundlagen in 20 Minuten

Die Patterns setzen ein kleines Fundament voraus. Wer nicht weiß, warum man Daten in Training und Test teilt, versteht weder Repeatable Splitting noch Rebalancing; wer nicht weiß, was ein Parameter ist, kann nicht erklären, warum Transform Parameter aus dem Training mitführen muss. Die 20 Minuten bauen deshalb fünf Gedanken auf, jeden mit einem Bild aus der Softwareentwicklung. Mathematik bleibt außen vor: kein Backpropagation-Beweis, keine Aktivierungsfunktionen. Die Vorkenntnisse im Hörsaal sind gemischt: Wer schon Modelle trainiert hat, kann Station 1 und 2 auf zwei Minuten kürzen, wer aus der reinen Softwareentwicklung kommt, braucht sie ganz. Eine Frage per Handzeichen zu Beginn („Wer hat schon ein Modell trainiert?“) genügt, um das zu entscheiden.

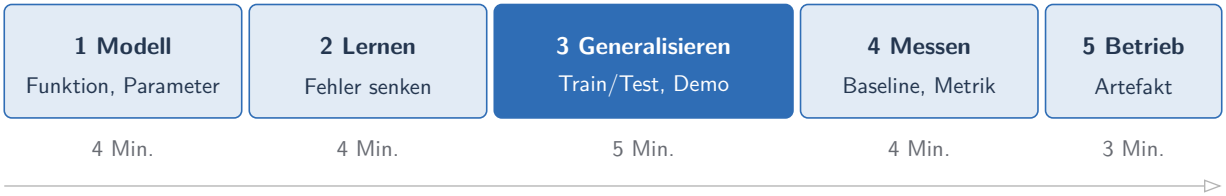


Abbildung 115: Zeitplan der 20 Minuten. Station 3 trägt die Demonstration und bekommt deshalb die meiste Zeit.

Station 1: Ein Modell ist eine Funktion mit gelernten Parametern (4 Minuten). An die Tafel kommt `miete = w · fläche + b`. Die Fläche ist ein *Feature* (Eingabe), die Miete das *Label* (der gewünschte Ausgabewert), `w` und `b` sind *Parameter*. Der Unterschied zu gewöhnlicher Software liegt darin, wer die Zahlen festlegt: Hier schreibt niemand die Regel, sondern man gibt Beispiele vor und lässt die Parameter daraus bestimmen. Parameter sind also Konfigurationswerte, die kein Mensch von Hand setzt, und ein Modell ist die Funktion samt dieser Werte.

Station 2: Lernen heißt, einen Fehler schrittweise zu verkleinern (4 Minuten). Eine *Loss Function* misst, wie falsch die Vorhersagen auf den Trainingsdaten sind. Das Training ändert die Parameter in kleinen Schritten so, dass dieser Wert sinkt, wie ein Wanderer, der im Nebel immer bergab geht. Der *Gradient* sagt ihm, wo es bergab geht, die *Learning Rate* legt die Schrittlänge fest. Eine *Epoche* ist ein Durchlauf durch alle Trainingsdaten, ein *Batch* die Teilmenge, aus der ein einzelner Schritt berechnet wird. Mehr braucht es hier nicht; die Herleitung steht in Kapitel 2.

Station 3: Gut auf bekannten Daten heißt nicht gut (5 Minuten). Das ist die wichtigste Station, und sie bekommt den Live-Lauf unten. Das Bild aus der Softwareentwicklung: Eine Funktion, die genau die vorhandenen Testfälle besteht, weil sie deren Antworten hart kodiert, ist wertlos. Ein Modell, das die Trainingsdaten auswendig lernt (*Overfitting*), tut dasselbe. Deshalb hält man Daten zurück, die das Modell nie gesehen hat (*Test Set*), und beurteilt nur anhand dieser.

Station 4: Ohne Vergleich bedeutet keine Zahl etwas (4 Minuten). „90 % richtig“ klingt gut. Gut ist es erst gegenüber einer *Baseline*, etwa der Regel „immer die häufigste Klasse“. Hier fällt in einem Satz auch die erste Warnung: Die *Accuracy* (Anteil richtiger Vorhersagen) ist bei seltenen Klassen irreführend, denn ein nutzloses Modell kann hohe Werte erreichen. *Precision* und *Recall* werden nur genannt (Treffer unter den Alarmen, gefundene unter den echten Fällen); ausführlich folgt es beim Pattern Rebalancing.

Station 5: Das Modell ist ein Artefakt, und der Betrieb ist der schwierige Teil (3 Minuten). Training erzeugt eine Datei mit Parametern. Damit sie im Betrieb dasselbe leistet wie im Test, müssen die Eingaben im Betrieb genauso aussehen wie im Training, und jemand muss Daten, Code und Version nachvollziehen können. Der eigentliche ML-Code ist in produktiven Systemen ein kleiner Teil des Ganzen [11]. Mit diesem Satz wird die Brücke zu den 30 Patterns geschlagen, die genau diese Lücken schließen.

Die Demonstration für Station 3

Alle Beispiele dieses Kapitels laufen mit uv; in den Terminalfenstern steht nur noch der Dateiname.

Terminal

```
$ uv run --with scikit-learn --with pandas --with numpy \
      python <datei>.py
```

Die gezeigten Ausgaben entstanden mit Python 3.12, scikit-learn 1.9.1, numpy 2.5.4 und pandas 3.1.1; andere Versionen können in den letzten Stellen abweichen.

Das erste Skript trainiert Entscheidungsbäume wachsender Tiefe auf zwei verrauschten, halbmondförmigen Klassen. Ein *Entscheidungsbaum* stellt dem Beispiel der Reihe nach Ja-Nein-Fragen zu seinen Features („ist x1 größer als 0,3?“) und liest am Ende die Klasse ab; die Tiefe ist die Zahl der Fragen hintereinander. Ein tieferer Baum darf mehr Fragen stellen und kann sich damit den Trainingsdaten immer genauer anpassen.

```
"""Live-Demo für "ML in 20 Minuten": Overfitting an einem Entscheidungsbaum."""
from sklearn.datasets import make_moons
from sklearn.dummy import DummyClassifier
```

```python
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0)

basis = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
test = basis.score(X_test, y_test)
print(f"Baseline (immer häufigste Klasse): Test {test:.2f}")
print("Tiefe  Train  Test")
for tiefe in [1, 2, 3, 4, 6, 8, 12, 20]:
    baum = DecisionTreeClassifier(max_depth=tiefe, random_state=0)
    baum.fit(X_train, y_train)
    train, test = baum.score(X_train, y_train), baum.score(X_test, y_test)
    print(f"{tiefe:5d}  {train:.2f}   {test:.2f}")
```

Terminal

```
$ python ml_in_20_minuten.py
Baseline (immer häufigste Klasse): Test 0.49
Tiefe  Train  Test
    1  0.80   0.80
    2  0.88   0.90
    3  0.88   0.90
    4  0.88   0.90
    6  0.90   0.88
    8  0.95   0.88
   12  1.00   0.87
   20  1.00   0.87
```

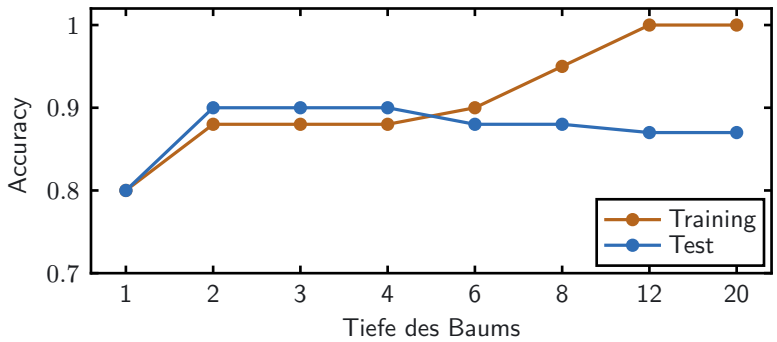


Abbildung 116: Genauigkeit auf Trainings- und Testdaten bei wachsender Baumtiefe (Werte aus dem Lauf oben).

Zu lesen ist: Die Trainingsgenauigkeit steigt bis auf 1,00, der Baum kennt jeden Trainingspunkt. Auf den Testdaten liegt das Maximum bei 0,90 (Tiefe 2 bis 4) und sinkt danach auf 0,87. Der klare Befund ist die Lücke zwischen 1,00 und 0,87, nicht der kleine Rückgang selbst: Der Testsatz hat nur 120 Beispiele, ein einziger Fall entspricht also gut 0,8 Prozentpunkten, und die Unterschiede zwischen 0,87 und 0,90 liegen im Rauschen. Sagen Sie das den Studierenden ausdrücklich. Es ist die erste Gelegenheit zu zeigen, dass auch eine Testzahl nur eine Schätzung ist. Die Baseline (immer die häufigste Klasse) liegt bei 0,49, der flachste Baum erreicht bereits 0,80.

15.3 Typische Missverständnisse je Kategorie

Missverständnisse lassen sich am besten ausräumen, indem man sie hervorlockt. Stellen Sie die Aussage als Frage („Ein Modell mit 99 % Accuracy: gut oder schlecht?“), lassen Sie abstimmen und zeigen Sie dann die Demonstration. Die folgenden Abschnitte nennen je Kategorie, was Studierende tatsächlich sagen oder annehmen, und die Richtigstellung. Wo ein kurzes Skript das Problem sichtbar macht, ist es im Anschluss gezeigt.

Data Representation (Patterns 1 bis 4)

> *„Embeddings sind dasselbe wie Hashing.“*

Beide bilden viele Werte auf wenige Zahlen ab, aber auf grundverschiedene Weise. Hashing ist eine feste Rechenvorschrift ohne Training: Welche Werte sich einen Bucket teilen, entscheidet der Zufall, und Ähnlichkeit spielt keine Rolle. Ein Embedding wird zusammen mit dem Modell gelernt und legt Werte, die sich im Modell ähnlich verhalten, nah beieinander.

> *„Mit genug Buckets verliert Hashing nichts.“*

Kollisionen werden mit mehr Buckets seltener, verschwinden aber nicht. Schon bei etwa der Wurzel aus der Bucketzahl vielen Werten ist mit der ersten Kollision zu rechnen (Geburtstagsparadox). Die Bucketzahl ist ein Kompromiss zwischen Größe und Kollisionsrate, kein Schalter für „verlustfrei“.

> *„Ein gutes Modell findet Kombinationen von allein.“*

Nur, wenn seine Form es erlaubt. Ein lineares Modell gewichtet jedes Feature für sich und kann „teuer und weit entfernt“ nicht als eigenen Fall erkennen, solange man ihm die Kombination nicht als Feature gibt (Feature Cross). Bäume und Netze können Kombinationen selbst bilden, brauchen dafür aber Kapazität und Daten.

Die Demonstration zu Hashing und Embedding nutzt 20 Begriffe aus einem Telekom-Support. Die Bucketnummern kommen aus MD5, weil Pythons eingebaute `hash()`-Funktion für Zeichenketten bei jedem Programmstart anders salzt und damit für Features ungeeignet ist. Für das Embedding genügt eine Matrix aus gemeinsamem Vorkommen, die per Singulärwertzerlegung auf vier Dimensionen verdichtet wird. Das ist kein neuronales Netz wie bei word2vec [7], zeigt aber dasselbe Prinzip: Der Vektor eines Wortes entsteht aus den Kontexten, in denen es vorkommt.

Terminal

```
$ python hashing_vs_embedding.py
20 Begriffe in 64 Buckets: 2 Buckets mit Kollision, kuendigung=49, kuendigen=10
20 Begriffe in  8 Buckets: 4 Buckets mit Kollision, kuendigung=1, kuendigen=2
Kosinus-Ähnlichkeit kuendigung / kuendigen :  0.99
Kosinus-Ähnlichkeit rechnung   / mahnung   :  1.00
Kosinus-Ähnlichkeit router     / störung   :  1.00
Kosinus-Ähnlichkeit kuendigung / router    :  0.00
Kosinus-Ähnlichkeit rechnung   / roaming   : -0.00
```

Bei 64 Buckets teilen sich zwei Buckets mehrere der 20 Begriffe, bei 8 Buckets sind es vier. Die verwandten Wörter `kuendigung` und `kuendigen` landen bei 64 Buckets in den Fächern 49 und 10; dass sie bei 8 Buckets nebeneinander liegen, ist Zufall und bedeutet nichts. Im Embedding dagegen zeigen ähnliche Wörter fast in dieselbe Richtung (Kosinus 0,99 bis 1,00), Wörter aus verschiedenen Themen stehen im rechten Winkel (0,00). Der Korpus besteht aus nur 17 kurzen Sätzen, die Werte sind deshalb auffallend glatt; auf echten Daten sind die Abstände weniger deutlich.

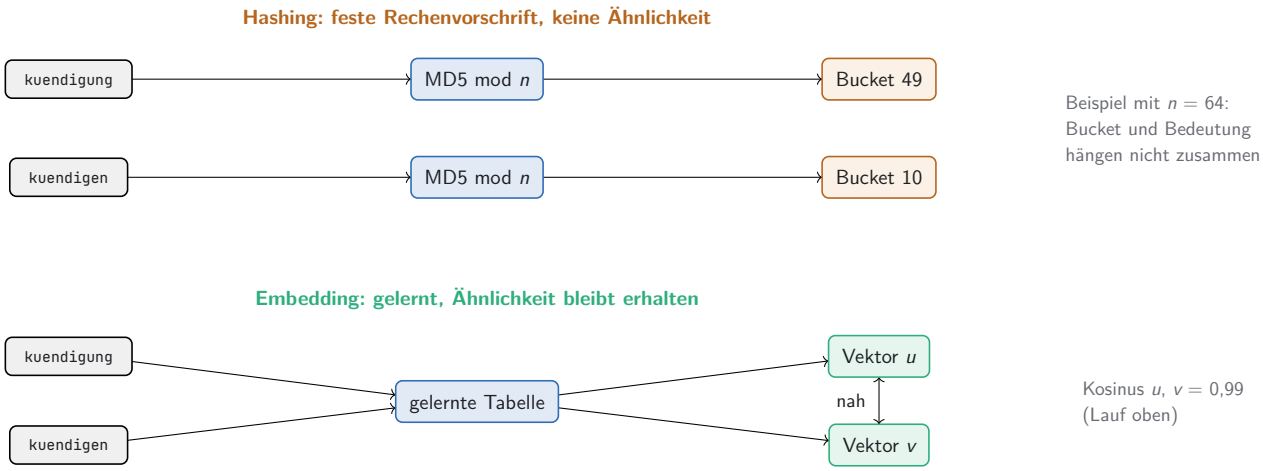


Abbildung 117: Hashing wählt den Bucket per Rechenvorschrift, ein Embedding schlägt gelernte Vektoren nach. Nur Embeddings machen ähnliche Wörter zu Nachbarn.

Für die Feature-Cross-Aussage dient ein Datensatz, in dem die Klasse vom Vorzeichen des Produkts zweier Features abhängt, also von ihrer Kombination und von keinem Feature allein.

Terminal

```
$ python feature_cross_xor.py
Logistische Regression, Rohmerkmale:      0.55
Logistische Regression, mit Feature Cross: 0.98
Entscheidungsbaum, Rohmerkmale (Tiefe 4):  1.00
```

Ohne Zusatz-Feature erreicht die logistische Regression 0,55, kaum mehr als Raten, denn die beiden Klassen sind etwa gleich häufig. Mit dem Produkt als drittem Feature sind es 0,98. Der Entscheidungsbaum schafft 1,00 ganz ohne Hilfe, weil seine Fragen aufeinander aufbauen und damit Kombinationen bilden. Dem linearen Modell fehlte also nicht Rechenkraft, sondern die passende Darstellung der Eingabe. Das ist der Kern von Feature Cross und eine gute Gelegenheit, den Unterschied zwischen „das Modell ist zu schwach“ und „die Eingabe ist falsch dargestellt“ zu üben.

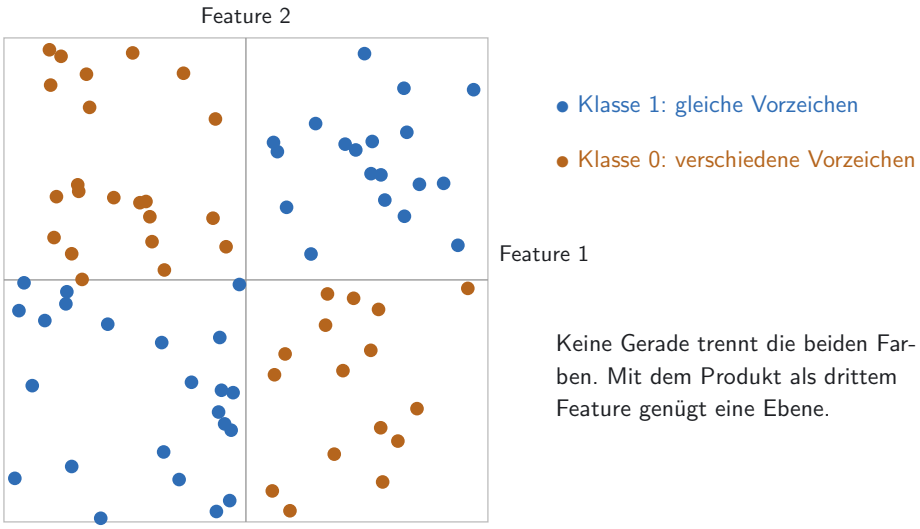


Abbildung 118: Die ersten 80 Punkte des Datensatzes. Die Klasse hängt von der Kombination der Features ab.

Problem Representation (Patterns 5 bis 10)

> *„Accuracy reicht als Gütemaß.“*

Bei seltenen Klassen nicht. Ein Modell, das immer „kein Betrug“ sagt, erreicht bei 1 % Betrug 99 % Accuracy und findet keinen einzigen Fall. Fragen Sie immer nach der Baseline und nach Precision und Recall der seltenen Klasse. Bei starkem Ungleichgewicht ist die Precision-Recall-Kurve aussagekräftiger als die ROC-Kurve [10].

„Mehr Daten lösen das Ungleichgewicht."	Mehr Daten ändern den Anteil nicht: Aus 1 % Betrug werden bei hundertfacher Datenmenge wieder 1 % Betrug, nur in absoluten Zahlen mehr. Einem flexiblen Modell können mehr Fälle der seltenen Klasse helfen; die Accuracy-Falle und der Standardschwellwert 0,5, der für seltene Klassen selten passt, bleiben. Im Lauf unten verharrt der Recall einer logistischen Regression bei hundertfacher Datenmenge bei etwa einem Drittel. Die Maßnahmen heißen Metrik, Klassengewichte, Schwellwert und gegebenenfalls Resampling (die Daten der seltenen Klasse vervielfältigen oder die der häufigen verringern).
„Ich gleiche die Klassen aus und teile danach in Training und Test."	Dann stehen Kopien derselben Fälle auf beiden Seiten, und der Test misst Wiedererkennen statt Verallgemeinerung. Resampling gehört ausschließlich auf die Trainingsdaten, nach dem Split. Das ist ein Fall von *Data Leakage*: Information, die im Betrieb nicht vorläge, fließt ins Training oder in die Bewertung [8].
„Multilabel ist Klassifikation mit den k besten Klassen."	Bei Softmax summieren sich die Wahrscheinlichkeiten zu 1; mehrere zutreffende Klassen konkurrieren dann um dieselbe Wahrscheinlichkeitsmasse. Multilabel verwendet pro Label einen eigenen Sigmoid-Ausgang (eine Funktion, die jeden Wert auf eine Zahl zwischen 0 und 1 abbildet), sodass jede Klasse unabhängig bewertet wird, und braucht keine feste Anzahl k.
„Ein Ensemble ist immer besser als sein bestes Mitglied."	Im Mittel verbessert es das Ergebnis, wenn die Mitglieder unterschiedliche Fehler machen; Bagging senkt die Varianz [3], Boosting vor allem den Bias (Gradient Boosting: [5]). Zehn identische Modelle bringen nichts, und der Betrieb wird teurer und schwerer erklärbar.
„Cascade und Ensemble sind dasselbe."	Im Ensemble beantworten mehrere Modelle parallel dieselbe Frage. In der Cascade zerlegt man die Frage, und die Ausgabe einer Stufe ist Eingabe der nächsten. Fehler früher Stufen setzen sich fort, deshalb gehört eine Cascade in eine gemeinsame Pipeline.
„Eine Neutral Class ist nur ein Schwellwert in anderer Verkleidung."	Ein Schwellwert behandelt die Unsicherheit des Modells: „Ich weiß es nicht, frag einen Menschen." Eine Neutral Class behandelt Unsicherheit in der Welt: Bei diesen Fällen sind sich auch die Fachleute uneins, und das Modell darf sich bei ihnen sehr sicher sein, dass sie uneindeutig sind.
„Reframing ist ein Trick, der Genauigkeit kostet."	Es tauscht Auflösung gegen Information: Die Antwort ist eine Verteilung über Wertebereiche statt einer Zahl, und sie darf mehrgipfelig sein. Das hilft, wo ein einzelner Mittelwert nichts aussagt oder wo Unsicherheit zählt.

Die Zahlen zu den ersten drei Aussagen liefert ein Skript: ein Datensatz mit 1 % positiver Klasse, ein Vergleich über wachsende Datenmengen und ein Resampling (hier mit 5 % positiver Klasse) vor und nach dem Split mit einem 1-Nächster-Nachbar-Modell, das Kopien besonders gnadenlos wiedererkennt.

Terminal

```
$ python accuracy_und_imbalance.py
== 1. Accuracy reicht nicht (1 % Betrug) ==
Immer 'kein Betrug'               Accuracy 0.990  Recall 0.00  Precision 0.00
Logistische Regression            Accuracy 0.992  Recall 0.19  Precision 0.86
... mit class_weight='balanced'   Accuracy 0.789  Recall 0.75  Precision 0.03

== 2. Mehr Daten: Anteil bleibt, Metrik-Falle auch ==
n=   2000  Betrugsfälle im Training:    22  Anteil 0.011  Accuracy 0.994  Recall 0.40
n=  20000  Betrugsfälle im Training:   200  Anteil 0.010  Accuracy 0.993  Recall 0.33
n= 200000  Betrugsfälle im Training:  1990  Anteil 0.010  Accuracy 0.993  Recall 0.35

== 3. Oversampling VOR dem Split täuscht (5 % Minderheit, 1-Nächster-Nachbar) ==
Duplizieren, dann splitten (falsch) Accuracy 0.979  Recall 1.00  Precision 0.94
Splitten, dann duplizieren (richtig) Accuracy 0.946  Recall 0.27  Precision 0.43
```

Drei Beobachtungen. Erstens: Die logistische Regression ohne Gewichtung erreicht 0,992 Accuracy, das ist nur 0,2 Punkte über der Baseline, aber sie findet 19 % der Betrugsfälle, die Baseline keinen. Mit `class_weight="balanced"` steigt der Recall auf 0,75, die Accuracy fällt auf 0,789, und die Precision sinkt auf 0,03: Auf einen echten Fall kommen etwa dreißig falsche Alarme. Das ist der Preis des Rebalancing, und ob er tragbar ist, hängt davon ab, was ein Alarm kostet.

Zweitens: Die Datenmenge wächst auf das Hundertfache, die Zahl der Betrugsfälle im Training von 22 auf 1990, der Anteil bleibt bei 1 %. Der Recall dieses linearen Modells bewegt sich nicht (0,40, 0,33, 0,35), und die Accuracy liegt stabil bei 0,993 bis 0,994. Für flexiblere Modelle kann mehr Material der seltenen Klasse durchaus helfen; das Skript zeigt nur, dass mehr Daten allein weder Anteil noch Metrik-Falle ändern. Drittens: Wer die Minderheit vor dem Split vervielfältigt, erhält einen Recall von 1,00 und eine Precision von 0,94. Dieselbe Rechnung mit dem Split vor dem Vervielfältigen ergibt 0,27 und 0,43. Die erste Zahl misst, ob das Modell Zwillinge wiedererkennt.

Model Training (Patterns 11 bis 15)

> *„Overfitting ist immer ein Fehler."*

Nicht, wenn man gar nicht verallgemeinern muss. Kennt man den gesamten Eingaberaum und ist die Zielfunktion deterministisch, etwa bei einer teuren numerischen Simulation, die ein Modell schneller annähern soll, ist genaues Anpassen an die Trainingsdaten das Ziel. Sobald Eingaben außerhalb des abgedeckten Raums auftreten, ist das Modell unzuverlässig.

> *„Ein Checkpoint ist die Modellversion."*

Ein Checkpoint sichert Gewichte und Zustand des Optimierers, damit man ein Training fortsetzen kann. Eine Modellversion muss zusätzlich festhalten, mit welchem Code, welchen Daten, welchen Hyperparametern und welcher Transformation das Modell entstand und wie es bewertet wurde. Ohne diese Angaben kann man ein Modell weder reproduzieren noch begründet zurückrollen.

> *„Ein vortrainiertes Modell ist immer besser als eines von Grund auf."*

Nur, wenn die Aufgabe der Vortrainingsaufgabe ähnlich genug ist. Die Übertragbarkeit der Features nimmt mit dem Abstand der Aufgaben ab [28], und man erbt Verzerrungen des Ausgangsmodells.

> *„Mit doppelt so vielen GPUs dauert das Training halb so lang."*

Die Geräte müssen ihre Gradienten abgleichen, und dieser Kommunikationsaufwand wächst. Bei synchronem, datenparallelem Training steigt außerdem die effektive Batchgröße, sodass die Lernrate angepasst werden muss, damit das Modell dieselbe Qualität erreicht [31].

> *„Hyperparameter werden beim Training gelernt, und ich darf sie am Test optimieren.“*

Hyperparameter legt man vor dem Training fest, nur die Parameter werden gelernt. Die Suche läuft über Validierungsdaten, einen zweiten zurückgehaltenen Teil, an dem man Entwicklungsentscheidungen prüft. Wer an den Testdaten optimiert, verbraucht sie, und die Abschlussbewertung wird zu optimistisch. Zufällige Suche ist bei gleichem Aufwand dem Raster oft überlegen [51].

Die Unterscheidung zwischen Checkpoint und Modellversion lässt sich in einem Bild zeigen:

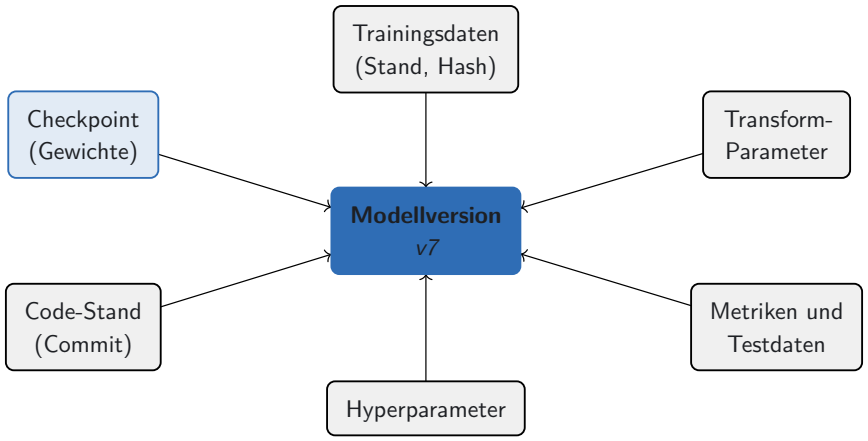


Abbildung 119: Eine Modellversion umfasst mehr als einen Checkpoint. Nur das Gesamtbild erlaubt Reproduktion und begründeten Rollback.

Terminal

```
$ python split_und_version.py
random_state=42        Zeilen, die nach 1 neuer Zeile die Seite wechseln: 245 von 1000
md5(kunde) % 10 < 8    Zeilen, die nach 1 neuer Zeile die Seite wechseln: 0 von 1000
Checkpoint nach Epoche  3: Gewichte-Hash 1f5da3eef201, Accuracy 0.850
Checkpoint nach Epoche 10: Gewichte-Hash 181d6dffc2a6, Accuracy 0.841
{
  "checkpoint": "181d6dffc2a6",
  "code": "39caa6399991",
  "daten": "161edbfd7c61",
  "hyperparameter": {
    "loss": "log_loss",
    "epochen": 10,
    "random_state": 0
  },
  "transform": "keine (Rohmerkmale)",
  "metrik": {
    "accuracy_train": 0.841
  }
}
```

Nach dem Hinzufügen einer einzigen Zeile wechseln beim Seed-Split 245 von 1000 Kunden die Seite, beim Hash-Split keiner. Das erste Ergebnis hängt an der Reihenfolge und Menge der Daten, das zweite nur an der Kunden-ID. Der Checkpoint nach Epoche 3 und der nach Epoche 10 haben unterschiedliche Gewichte (erkennbar am Hash) und sogar leicht unterschiedliche Trainings-Accuracy (0,850 gegen 0,841): Ein Checkpoint ist ein Zwischenstand, nicht notwendig der beste. Das Manifest zeigt, was zu einer Version gehört; der Code-Hash ist der SHA-256 der Skriptdatei selbst, der Datenhash der der Trainingsdaten. In einem echten Projekt stünden hier der Commit, die Datenversion und der Transform-Parameter.

Resilient Serving (Patterns 16 bis 20)

„Das Modell ist der Service."	Der Service besteht aus Vorverarbeitung, Modell und Schnittstelle. Stateless Serving Function heißt, dass diese Funktion keinen Zustand zwischen Aufrufen hält und deshalb beliebig vervielfacht werden kann. Hält ein Notebook oder ein globaler Prozesszustand etwas fest, skaliert das nicht.
„Was im Test gut war, bleibt gut."	Die Welt ändert sich. Bei *Data Drift* verschieben sich die Eingaben, bei *Concept Drift* der Zusammenhang zur Zielgröße [12]. Continued Model Evaluation vergleicht Vorhersagen laufend mit später eintreffenden Labels; wo Labels verspätet kommen, hilft zunächst die Beobachtung der Eingabeverteilung.
„Two-Phase Predictions sind eine Cascade."	Die Struktur ähnelt sich (ein kleines Modell vor einem großen), der Zweck nicht. Two-Phase verteilt die Arbeit nach Ort und Kosten: Ein kleines Modell läuft lokal, etwa auf dem Gerät, und das große in der Cloud wird nur bei Bedarf aufgerufen. Die Cascade zerlegt dagegen die fachliche Frage in Teilfragen, unabhängig davon, wo die Modelle laufen.
„Den Schlüssel braucht man nur zum Debuggen."	Bei verteilter und Batch-Verarbeitung ist die Reihenfolge der Ergebnisse nicht garantiert. Der Schlüssel macht die Zuordnung von Ergebnis und Eingabe überhaupt erst möglich und dient später als Verbindungsstelle zur Auswertung. Er darf aber nie als Feature ins Modell gelangen.

Reproducibility (Patterns 21 bis 27)

„Mit festem Seed ist der Split reproduzierbar."	Nur für exakt dieselben Daten in derselben Reihenfolge. Kommt eine einzige Zeile hinzu, wechseln viele Zeilen die Seite (siehe die Ausgabe oben). Repeatable Splitting leitet die Zuordnung aus einer stabilen Spalte ab, sodass jede Zeile immer dort bleibt, wo sie war.
„Transform bedeutet: Daten normalisieren."	Normalisieren ist ein Beispiel. Gemeint ist die gesamte Vorverarbeitung samt den im Training gelernten Parametern (Mittelwert, Vokabular, Bucketgrenzen), die als Teil des Modells exportiert und im Serving identisch angewendet wird. Es ist dasselbe Prinzip wie Single Source of Truth in der Softwareentwicklung.
„Ein Feature Store ist eine Datenbank für Features."	Er speichert Definitionen und Werte, bedient Training und Serving mit demselben Code und stellt Features zeitpunktgenau bereit (nur Werte, die zum Ereigniszeitpunkt bekannt waren). Eine gewöhnliche Datenbank leistet das nicht von selbst.

Responsible AI (Patterns 28 bis 30)

„Eine einfache Regel als Benchmark ist unter dem Niveau eines ML-Projekts."	Sie ist der Prüfstein. Schlägt das Modell „morgen so viel wie heute" nicht deutlich, lohnt sich sein Betrieb nicht. Außerdem lässt sich ein Fehlerwert nur mit einer Referenz einordnen.

„SHAP zeigt, was die Ursache der Entscheidung ist.“	SHAP [66] und LIME [65] berechnen, wie viel jedes Feature zu einer einzelnen Vorhersage beiträgt. Sie erklären damit, wie das Modell zu seiner Vorhersage kommt, nicht, wie die Welt funktioniert. Ein hoher Beitrag des Alters heißt, dass das Modell das Alter nutzt, nicht dass das Alter die Ursache ist.
„Ohne das Feature Geschlecht ist das Modell fair.“	Andere Features können es mittragen (Proxies, etwa die Postleitzahl für Herkunft). Man prüft Fairness an den Ergebnissen je Untergruppe, nicht an der Feature-Liste.
„Es gibt eine richtige Fairness-Kennzahl.“	Es gibt mehrere Kriterien, zum Beispiel gleiche Trefferquote [74] oder gleiche Kalibrierung (ein Score von 0,8 trifft in jeder Gruppe bei etwa 80 % der Fälle zu). Unterscheiden sich die Grundraten der Gruppen (der Anteil positiver Fälle), lassen sich zum Beispiel gleiche Fehlerraten und gleiche Kalibrierung nicht zugleich erfüllen, außer bei perfekter Vorhersage [76]. Welches Kriterium zählt, ist eine fachliche und ethische Entscheidung.

15.4 Prüfungs- und Diskussionsfragen

Die folgenden Fragen prüfen Verständnis, keine Wortlaute. Jede lässt sich ohne Nachschlagen beantworten, wenn man die Mechanik hinter dem Pattern begriffen hat. Ein Drittel ist als *Transfer* markiert: Hier müssen die Studierenden das Gelernte auf eine neue Lage anwenden. Die Fragen eignen sich für eine schriftliche Prüfung ebenso wie als Einzelfragen in der Abschlusspräsentation (16 der 64 Punkte), wo sie sichern, dass jede Person den eigenen Code erklären kann. Die Musterantworten nennen den Kern; eine gute Antwort darf eigene Worte und andere Beispiele verwenden.

Frage 1 Rebalancing, Heuristic Benchmark · Verständnis

Ein Betrugsmodell erreicht 99,2 % Accuracy bei einer Betrugsquote von 1 %. Der Auftraggeber ist zufrieden. Was entgegnen Sie?

> Die Baseline „nie Betrug“ erreicht bereits 99 %. Das Modell ist also nur 0,2 Punkte besser, und die Accuracy sagt nichts darüber, ob Betrugsfälle gefunden werden. Man braucht Recall und Precision der Betrugsklasse und eine Abwägung, was ein übersehener Fall und was ein falscher Alarm kosten.

Frage 2 Overfitting, Splitting · Verständnis

Ein Modell erreicht im Training 98 %, im Test 71 %. Was ist wahrscheinlich passiert, und was prüfen Sie zuerst?

> Das Modell hat sich den Trainingsdaten zu genau angepasst (Overfitting). Zu prüfen sind Komplexität des Modells, Datenmenge und Regularisierung (ein Strafterm, der große Gewichte und damit Komplexität dämpft). Außerdem sollte man klären, ob Training und Test aus derselben Verteilung stammen, etwa nach Zeit. Umgekehrt wäre ein unerwartet gutes Testergebnis der Anlass, nach Data Leakage zu suchen.

Frage 3 Feature Cross · Verständnis

Ein lineares Modell soll Fahrpreise schätzen. Der Einfluss der Entfernung ist in der Hauptverkehrszeit deutlich größer. Warum lernt das Modell das nicht von selbst, und was ändern Sie?

> Ein lineares Modell addiert gewichtete Beiträge je Feature; der Beitrag der Entfernung hängt dort nicht vom Zeitraum ab. Man ergänzt ein Feature, das Entfernung und Hauptverkehrszeit kombiniert (Feature Cross), oder wählt ein Modell, das Kombinationen von sich aus bilden kann, etwa einen Baum.

Frage 4 Transform · Transfer

Im Training wurden Temperaturen mit Mittelwert und Streuung der Trainingsdaten normalisiert. Der Service bekommt Rohwerte. Was passiert, wie bemerkt man es, und welches Prinzip der Softwaretechnik hilft?

> Das Modell sieht Zahlen aus einem anderen Wertebereich als im Training und liefert Unsinn, meist ohne Fehlermeldung (Training-Serving Skew). Man bemerkt es an verschobenen Vorhersageverteilungen oder durch einen Vertragstest, der Training und Serving vergleicht. Abhilfe: Die Transformation samt Parametern gehört zum Modellartefakt und existiert genau einmal (Single Source of Truth, DRY).

Frage 5 Repeatable Splitting · Verständnis

In einem Kundendatensatz stehen mehrere Zeilen je Kunde. Man teilt zufällig zeilenweise in Training und Test. Wo liegt das Problem?

> Zeilen desselben Kunden landen auf beiden Seiten. Das Modell erkennt dann Kunden statt Muster, und der Test fällt zu optimistisch aus. Der Split sollte nach Kunden-ID erfolgen, deterministisch aus deren Hash; bei zeitlichen Daten nach Zeit.

Frage 6 Neutral Class · Verständnis

Drei Fachleute ordnen dasselbe Support-Ticket drei verschiedenen Teams zu. Wie gehen Sie beim Labeln und Modellieren damit um?

> Die Uneinigkeit ist selbst eine Information. Statt eine Mehrheit zu erzwingen, kann man eine neutrale oder „mehrdeutig“-Klasse einführen oder Mehrfachzuordnung (Multilabel) zulassen. Ein Schwellwert wäre hier die falsche Antwort, denn er beschreibt die Unsicherheit des Modells, nicht die der Daten.

Frage 7 Keyed Predictions, Batch Serving · Verständnis

Ein Batch-Lauf über fünf Millionen Datensätze liefert die Vorhersagen in anderer Reihenfolge zurück. Warum, und was tun Sie?

> Verteilte Verarbeitung teilt die Daten auf und garantiert keine Ausgabereihenfolge. Jede Instanz bekommt einen eindeutigen Schlüssel, der unverändert in die Ausgabe wandert, sodass sich Ergebnisse zuordnen lassen. Der Schlüssel darf kein Feature des Modells sein.

Frage 8 Continued Model Evaluation · Transfer

Ein Modell wurde im Januar mit AUC 0,91 abgenommen (AUC: Gütemaß für die Rangfolge der Vorhersagen, 0,5 entspricht Raten, 1 ist perfekt). Im Juni häufen sich Beschwerden. Wie gehen Sie vor, und wie hätte man es früher gemerkt?

> Man vergleicht aktuelle Vorhersagen mit inzwischen bekannten Labels und prüft, ob sich die Eingabeverteilung geändert hat (Data Drift) oder der Zusammenhang (Concept Drift). Früher hätte laufendes Monitoring mit Schwellwerten für Metrik und Eingabeverteilung geholfen. Wo Labels verspätet kommen, ersetzt zunächst die Beobachtung der Eingaben die Metrik.

Frage 9 Model Versioning, Checkpoints · Verständnis

Was gehört zu einer Modellversion, und warum genügt der Checkpoint nicht?

> Neben den Gewichten: Code-Stand, Trainingsdaten (Stand oder Hash), Hyperparameter, Transform-Parameter, Metriken und die Testdaten, auf denen sie entstanden. Dazu kommt, dass jede Version unter eigener Kennung bereitsteht, damit Clients in Ruhe wechseln können (Pattern 27). Nur damit lässt sich ein Modell reproduzieren, mit einer anderen Version vergleichen und begründet zurückrollen.

Frage 10 Heuristic Benchmark · Verständnis

Ein Prognosemodell hat einen mittleren Fehler von 4,2 kWh. Ist das gut?

> Das lässt sich ohne Referenz nicht sagen. Man vergleicht mit einer einfachen Regel, etwa „gleiche Stunde der Vorwoche“, und mit dem, was die Entscheidung braucht. Erst die Verbesserung gegenüber der Heuristik und der fachliche Nutzen machen die Zahl aussagekräftig.

Frage 11 Explainable Predictions · Transfer

SHAP zeigt „Alter" als wichtigsten Faktor einer Kreditablehnung. Darf man sagen, das Alter habe die Ablehnung verursacht?

> Nein. SHAP zeigt, wie stark das Modell das Feature für diese Vorhersage nutzt, nicht eine Ursache in der Welt. Das Alter kann ein Proxy für anderes sein. Zudem ist zu klären, ob das Feature überhaupt verwendet werden darf.

Frage 12 Fairness Lens · Transfer

Das Feature Geschlecht wurde entfernt, trotzdem unterscheiden sich die Fehlerraten zwischen Männern und Frauen. Wie kann das sein, und was prüfen Sie?

> Andere Features können das Geschlecht mittragen (Proxies). Man berechnet Kennzahlen je Untergruppe (Slicing), wählt ein Fairness-Kriterium und macht den Zielkonflikt zwischen den Kriterien sichtbar, statt ihn zu verstecken.

15.5 Verbindung zu den vier Praxisprojekten

Jedes Projekt enthält absichtlich platzierte Mängel und fachliche ML-Probleme, die sich nur mit konkreten Patterns lösen lassen. Die Zuordnung der Patterns zu den Projekten stammt aus dem Konzept und ist gegen `CONTRACT.md` und `ASSIGNMENT.typ` der vier Projekte abgeglichen (Stand 11. November 2026); die folgende Abbildung zeigt, wie sie sich über die sechs Kategorien verteilen.

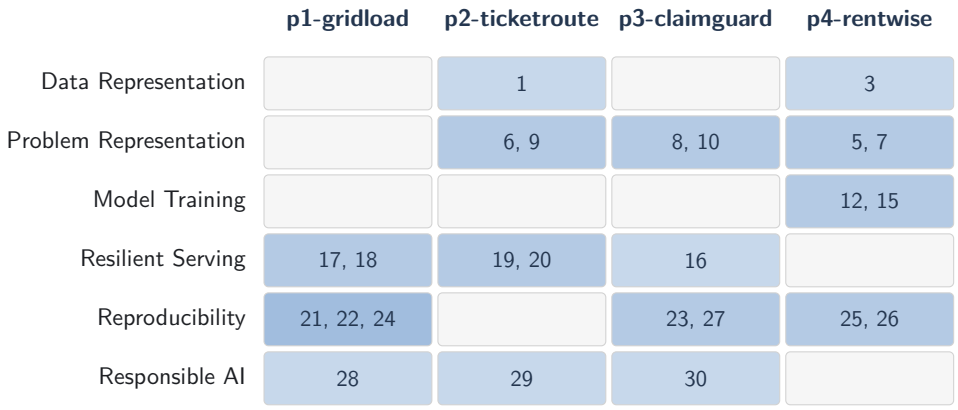


Abbildung 120: Pattern-Nummern je Projekt und Kategorie nach dem Konzept. Model Training kommt nur in p4-rentwise vor, Data Representation nur in p2 und p4.

Für die Lehre folgt daraus: Die Projekte decken 25 verschiedene der 30 Patterns ab. Pro Projekt prüft der Trainer sechs Pattern-Gruppen zu je zwei Punkten, und die Gruppen müssen mindestens vier davon umsetzen; in p4-rentwise bilden Checkpoints und Hyperparameter Tuning eine gemeinsame Gruppe, und Repeatable Splitting (Pattern 22) bringt dort bis zu zwei Bonuspunkte. Embeddings (2), Multimodal Input (4), Useful Overfitting (11), Transfer Learning (13) und Distribution Strategy (14) kommen in keinem Projekt vor. Diese fünf müssen über Folien, Demonstrationen und die Fragen oben vermittelt werden, nicht über die Projektarbeit. Die folgenden Tabellen ordnen je Projekt ein typisches Symptom dem Pattern zu, das es behebt.

> **ACHTUNG**
> Eine vollständige Liste der eingebauten Mängel mit Datei und Zeile liegt zum Stand dieser Fassung nur für p1-gridload vor (`_lehrende/p1-gridload/PLANTED-ISSUES.md`); die Symptome von p1 stammen daraus. Für p2 bis p4 sind die Symptome aus den Pattern-Beschreibungen in `ASSIGNMENT.typ` und `CONTRACT.md` abgeleitet und beschreiben, woran man das Problem typischerweise erkennt. Gleichen Sie sie mit den Mängellisten ab, sobald diese vorliegen.

p1-gridload (Stromnetz-Lastprognose, Zeitreihen-Regression)

Typisches Symptom im Projekt	Pattern
Die rollenden Fenster enden bei $t-1$, obwohl dieser Wert zum Zeitpunkt der Tagesprognose noch unbekannt ist; der Online-Pfad rechnet sie einmal und nutzt sie für	24 Windowed Inference

Typisches Symptom im Projekt	Pattern
alle 24 Stunden, und ein fehlender Messwert verschiebt alle folgenden Verzögerungen.	
Der Split ist ein zufälliger Zeilen-Split ohne Seed, Testzeilen liegen zwischen Trainingszeilen; die Auswertung teilt noch einmal anders, sodass rund 80 % ihrer Testzeilen im Training waren.	22 Repeatable Splitting
Training und Online-Pfad rechnen verschieden: Heizgradbasis 15,0 statt 15,5, Feiertage ohne bewegliche Feiertage, fehlendes Wetter als feste 10 °C.	21 Transform
Es gibt nur absolute Fehlermaße; ein Vergleichswert wie „gleiche Stunde der Vorwoche“ fehlt.	28 Heuristic Benchmark
Die Prognose läuft pro Station und Stunde mit je einem eigenen Aufruf (Minuten), und die Ausgabe trägt weder Station noch Zeitstempel.	17 Batch Serving
Nichts vergleicht neue Messwerte mit den Prognosen; die Drift ab Oktober bleibt unbemerkt, der Fehler der Prognosen ist etwa doppelt so hoch wie im Test.	18 Continued Model Evaluation

p2-ticketroute (Support-Ticket-Routing, Text)

Typisches Symptom im Projekt	Pattern
Der Wortschatz ist riesig, und neue Begriffe brechen das Vokabular.	1 Hashed Feature
Tickets betreffen mehrere Teams, das Modell erzwingt genau eines.	6 Multilabel
Teams ordnen ähnliche Tickets uneinheitlich zu; das Modell ist bei mehrdeutigen Fällen überkonfident.	9 Neutral Class
Ein teures Textmodell läuft für jedes Ticket, obwohl ein einfaches die meisten sicher löst.	19 Two-Phase Predictions
Batch-Vorhersagen lassen sich Tickets nicht eindeutig zuordnen.	20 Keyed Predictions
Niemand kann sagen, warum ein Ticket beim falschen Team gelandet ist.	29 Explainable Predictions

p3-claimguard (Versicherungs-Betrugserkennung, unbalanciert)

Typisches Symptom im Projekt	Pattern
Hohe Accuracy, kaum gefundene Betrugsfälle; oder Resampling vor dem Split.	10 Rebalancing
Verschiedene Schadenarten folgen verschiedenen Mustern; ein Modell mischt sie.	8 Cascade
Das Meldeformular wurde erweitert; alte Fälle haben das neue Feld nicht und fallen aus dem Training.	23 Bridged Schema
Die Quote auffälliger Fälle unterscheidet sich je Region oder Altersgruppe, ohne dass jemand es geprüft hat.	30 Fairness Lens
Ein neues Modell überschreibt das alte; frühere Entscheidungen sind nicht reproduzierbar.	27 Model Versioning
Der Dienst lädt das Modell bei jeder Anfrage oder hält globalen Zustand; die Vorverarbeitung steckt im Notebook.	16 Stateless Serving Function

p4-rentwise (Mietpreis-Schätzung, Regression)

Typisches Symptom im Projekt	Pattern
Die Lage wirkt je nach Wohnfläche unterschiedlich; ein lineares Modell macht systematische Fehler.	3 Feature Cross
Kundschaft will eine Preisspanne statt eines Punktwerts; die Streuung ist groß.	5 Reframing
Das Einzelmodell schwankt stark mit der Stichprobe.	7 Ensembles
Dieselben Features werden in Training und Service neu berechnet, aus zwei Quellen.	26 Feature Store
Die Schritte laufen von Hand nacheinander und lassen sich nicht einzeln wiederholen.	25 Workflow Pipeline
Ein abgebrochenes Training beginnt von vorn.	12 Checkpoints
Hyperparameter sind geraten oder wurden am Testsatz eingestellt.	15 Hyperparameter Tuning

15.6 Den Trainer-Report im Coaching nutzen

Der Trainer (`uv run trainer check`) läuft in jedem Projekt lokal, braucht laut README etwa ein bis drei Minuten und schreibt eine Zusammenfassung sowie `trainer-report.md`. Er prüft statische Analyse gegen eine Baseline, den Testlauf samt Coverage und je Pattern einen Verhaltensvertrag; mit `--only static`,

`--only tests` oder `--only patterns` (und `--pattern <name>`) lässt sich ein Teil gezielt wiederholen. Die Verträge rufen nur `contract.py` auf, mit frisch erzeugten Daten. Das Vorgehen unten hängt an den Vertragsnamen, nicht am Layout des Reports. Der Report beantwortet „ist es rot?“, das Coaching „warum, und was lernt die Gruppe daraus?“. Darum gilt: Frage statt Lösung.

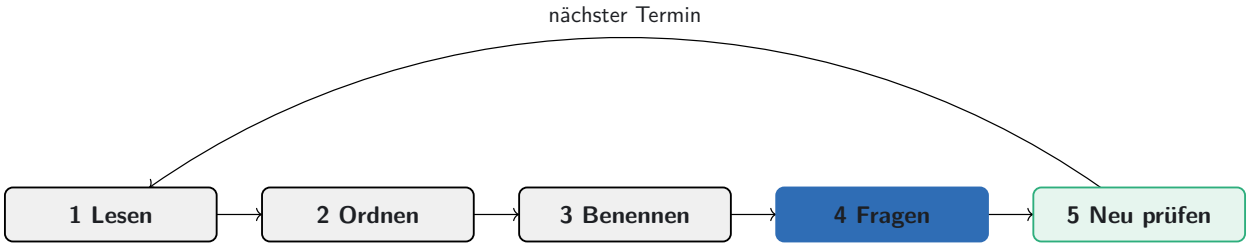


Abbildung 121: Ablauf eines Coaching-Termins von etwa 15 Minuten je Gruppe.

Report lesen und Rot ordnen. Lesen Sie den Report vor dem Termin, nicht währenddessen. Zuerst kommen Verträge, die die Messung selbst verfälschen: Datenleck beim Split und Skew zwischen Training und Serving. Solange sie rot sind, sagt jede Modellzahl wenig, und Tuning wäre vergeudet. Danach folgen Verträge zum Betrieb, zuletzt stilistische Befunde der statischen Analyse. Nehmen Sie sich pro Termin einen Befund vor; eine Gruppe, die fünf Rote auf einmal bekommt, behebt keines verstanden.

Pattern benennen, Frage stellen. Die Tabelle zeigt für vier Verträge aus dem Konzept einen Gesprächseinstieg. Er verweist auf das Symptom und nicht auf die Lösung.

Vertrag ist rot	Einstiegsfrage	Pattern
Serving liefert andere Features als Training	„Zeigt mir die Stellen im Code, an denen normalisiert wird. Wie viele sind es?“	21 Transform
Split ist nicht deterministisch oder nicht gruppiert	„Was passiert mit euren Testdaten, wenn morgen 1000 Zeilen dazukommen? Kann dieselbe Person in Training und Test stehen?“	22 Repeatable Splitting
Vorhersagen tragen keinen Schlüssel	„Wie ordnet ihr im Batch-Ergebnis eine Vorhersage dem Ticket zu, wenn die Reihenfolge nicht stimmt?“	20 Keyed Predictions
Modell schlägt Heuristik nicht	„Welche Regel hättet ihr ohne ML gebaut? Wie viel besser ist das Modell wirklich?“	28 Heuristic Benchmark

Grün prüfen. Ein grüner Vertrag zeigt, dass das Verhalten stimmt, nicht, dass die Gruppe es verstanden hat; das gilt besonders, weil LLM-Nutzung erlaubt ist. Lassen Sie eine Person die Lösung in zwei Sätzen erklären und die Alternative nennen, die man hätte wählen können. Das sind die Fragen der ADRs (Entscheidung, Alternative, Preis) und später der Einzelfragen in der Abschlusspräsentation. Wer die Entscheidung nicht begründen kann, hat sie nicht getroffen.

Coaching und Beurteilung trennen. Der Trainer liefert laut README 28 der 64 Punkte automatisch (statische Analyse 8, Testsuite 8, ML-Pattern-Verträge 12). Das Coaching bewertet nichts, es bereitet auf die Einzelfragen der Abschlusspräsentation vor (16 Punkte): Wer in den Terminen gelernt hat, eine Entscheidung zu begründen, kann es dort auch.

16 Übungen mit Lösungen

Wer ein Pattern erklären kann, kann es noch nicht unbedingt anwenden. Dieses Kapitel prüft den zweiten Schritt: Es beschreibt Symptome, wie sie in Projekten auftreten, verlangt Zahlen, wie man sie bei der Planung ausrechnen muss, und lässt kleine Experimente nachbauen, an denen eine Behauptung stimmt oder nicht stimmt. Es dient der Selbstkontrolle während des Lesens und als Fundus für Übungsblätter, Quizfragen und Prüfungsaufgaben in der Lehrveranstaltung.

LERNZIELE
Nach der Arbeit mit diesem Kapitel können Sie zu einem beschriebenen Symptom das passende Pattern nennen, begründen, warum es passt, und eine Alternative abgrenzen. Sie können die Rechnungen ausführen, die bei den Patterns immer wieder anfallen: Kollisionswahrscheinlichkeit beim Hashing, Precision und Recall aus einer Konfusionsmatrix, den Anteil der Anfragen in der zweiten Phase, die Stichprobengröße für eine Überwachung. Und Sie können ein Experiment so aufsetzen, dass es eine Aussage über ein Pattern wirklich prüft.

16.1 So arbeiten Sie mit diesem Kapitel

Die Aufgaben folgen den sechs Kategorien des Buchs von V. Lakshmanan, S. Robinson, und M. Munn [1] und tragen eine Kennung: **D** für Data Representation, **P** für Problem Representation, **T** für Model Training, **S** für Resilient Serving, **R** für Reproducibility, **V** für Verantwortung (Responsible AI) und **K** für die Szenarien über mehrere Kategorien. Die Kopfzeile nennt Art (Verständnis, Diagnose, Rechnen, Code) und Pattern. Der Schwierigkeitsgrad ist mit Sternen markiert: ★ verlangt, eine Definition anzuwenden; ★★ eine Rechnung oder Diagnose in zwei bis drei Schritten; ★★★ verbindet mehrere Konzepte, verlangt eine Herleitung oder ein eigenes Experiment. Hinter jeder Aufgabe steht ein Verweis auf die Lösung und umgekehrt. Versuchen Sie die Aufgabe zuerst ohne Blick auf die Lösung; bei Diagnoseaufgaben lohnt es sich, die Antwort aufzuschreiben, bevor man sie vergleicht.

In Rechen- und Szenarioaufgaben sind die Eingangswerte **Annahmen der Aufgabe** (Preise, Latenzen, Datenmengen), nirgends gemessen. Wo eine Zahl aus einem Lauf stammt, steht das dabei; sie kommt aus den Skripten in `code/15/` (`01_darstellung.py` für D, `02_problem.py` für P, `03_training.py` für T, `04_serving.py` für S, `05_reproduzierbarkeit.py` für R, `06_verantwortung.py` für V, `07_szenarien.py` für K). Sie laufen mit `uv run --with scikit-learn --with pandas --with numpy python <datei>`, die Ausgaben liegen in `code/15/ausgabe/`, und `08_pruefung.py` rechnet die übrigen Zahlen des Kapitels nach.

16.2 Data Representation (Patterns 1 bis 4)

D1 **Cold Start** Verständnis · Hashed Feature ★☆☆

Ein Shop führt täglich neue Produkte ein. Mit One-Hot-Kodierung der Produkt-ID wirft der Encoder bei jeder unbekannten ID einen Fehler, mit einem Hashed Feature läuft der Service weiter.
(a) Warum hilft Hashing beim Cold Start technisch?
(b) Warum löst es das Cold-Start-Problem fachlich nicht?
(c) Nennen Sie zwei weitere Dinge, die Hashing nicht leistet.

Lösung auf Seite 204

D2 Kollisionen rechnen

Rechnen und Code · Hashed Feature ★★☆

Ein Shop hat $n = 20\,000$ Produkt-IDs und hasht sie auf $B = 50\,000$ Buckets. Ein bestimmter Wert teilt seinen Bucket mit mindestens einem anderen mit der Wahrscheinlichkeit $1 - \left(1 - \frac{1}{B}\right)^{n-1}$; die erwartete Zahl belegter Buckets ist $B\left(1 - \left(1 - \frac{1}{B}\right)^{n}\right)$.

(a) Welcher Anteil der IDs kollidiert, und wie viele Buckets sind belegt?
(b) Wie groß muss B mindestens sein, damit höchstens 5 % der IDs kollidieren?
(c) Prüfen Sie beide Rechnungen mit `murmurhash3_32` aus scikit-learn.

Lösung auf Seite 205

D3 Zellen zählen

Rechnen · Feature Cross ★★☆

Ein Verkehrsdienst hat 1 000 000 Fahrten und die Features Wochentag (7 Werte), Stunde (24) und Bezirk (23).

(a) Wie viele Zellen entstehen, wenn man alle drei kreuzt, und wie viele Fahrten fallen im Mittel in eine Zelle? Wie ist es bei Wochentag × Stunde?
(b) Nennen Sie drei Wege, die Zahl der Zellen zu begrenzen, ohne auf die Wechselwirkung zu verzichten.

Lösung auf Seite 205

D4 Quadranten

Code · Feature Cross ★★★

Punkte (x, y) liegen gleichverteilt in $[-1, 1]^2$. Das Label ist 1, wenn $x \cdot y > 0$, bei 5 % der Punkte ist es vertauscht.

(a) Suchen Sie per Raster die beste Gerade auf den Rohwerten. Welche Genauigkeit erreicht sie?
(b) Zerlegen Sie x und y in je vier Buckets (Grenzen $-0{,}5$; 0; $0{,}5$) und trainieren Sie eine logistische Regression, einmal mit den Buckets einzeln, einmal mit deren Kreuzprodukt. Welche Genauigkeit ist höchstens möglich?
(c) Erreicht ein Entscheidungsbaum ohne Cross das Ergebnis des Crosses?

Lösung auf Seite 205

D5 Embedding-Tabelle

Rechnen · Embeddings ★★☆

Eine Musik-App hat 4 000 000 Titel.

(a) Welche Dimension schlägt die Faustregel „vierte Wurzel der Kategorienzahl“ vor?
(b) Wie groß ist die Embedding-Tabelle in Megabyte (float32)? Wie groß, wenn man die Titel vorher auf 500 000 Buckets hasht?
(c) Welche Titel leiden unter dem Hashing am meisten?

Lösung auf Seite 206

D6 Fehlendes Foto

Diagnose · Multimodal Input ★★☆

Ein Versicherungsmodell nutzt Schadenshöhe, Beschreibung und Foto. Im Training hat jede Meldung ein Foto. Im Betrieb fehlt es bei 30 % der Meldungen; die Pipeline setzt die Pixelwerte dann auf null. Das Modell meldet auffällig oft Totalschaden.

(a) Wie heißt das Phänomen, und warum tritt es auf?
(b) Nennen Sie zwei Gegenmaßnahmen.
(c) Das Foto liefert 10 000 standardisierte Pixelwerte, die Tabelle 5. Wie verhalten sich die Längen der beiden Teilvektoren, und was bedeutet das für abstandsbasierte Verfahren?

Lösung auf Seite 206

16.3 Problem Representation (Patterns 5 bis 10)

P1 Der Mittelwert, der nie vorkommt

Verständnis · Reframing ★☆☆

Ein Lieferdienst sagt die Lieferzeit mit einem Regressionsmodell voraus, das den quadratischen Fehler minimiert. Die Zeiten sind zweigipflig: 20 Minuten bei freier Küche, 60 bei überlasteter, jeweils in der Hälfte der Fälle, und aus den Eingaben ist nicht erkennbar, welcher Fall vorliegt.

(a) Welche Zeit sagt das Modell voraus, und warum ist das ein Problem?
(b) Wie hilft es, die Aufgabe als Klassifikation über Zeitklassen zu formulieren?
(c) Geben Sie ein Beispiel für die umgekehrte Richtung (Klassifikation als Regression).

Lösung auf Seite 206

P2 Mehrere Tags

Rechnen · Multilabel ★★☆

Ein Netz soll Artikel mit mehreren Tags versehen. Für die Tags A bis D liefert es die Logits $(2{,}2;\ 1{,}9;\ -0{,}5;\ -2{,}0)$.

(a) Berechnen Sie die Ausgaben mit Softmax und mit Sigmoid.
(b) Welche Tags vergeben Sie jeweils bei der Schwelle 0,5?
(c) Warum ist Softmax für Multilabel ungeeignet? Welche Fehlerfunktion gehört zur Sigmoid-Ausgabe?

Lösung auf Seite 206

P3 Mehrheitsentscheid

Rechnen · Ensembles ★★☆

Drei Klassifikatoren sind jeweils zu 70 % richtig und machen unabhängige Fehler.

(a) Wie genau ist die Mehrheitsentscheidung bei 3 und bei 5 Modellen?
(b) Warum bleibt der Gewinn in der Praxis hinter der Rechnung zurück?
(c) Wofür stehen Bagging, Boosting und Stacking jeweils in einem Satz?

Lösung auf Seite 206

P4 Cascade durchrechnen

Rechnen · Cascade ★★☆

Eine Bank prüft 100 000 Zahlungen am Tag, 1 % sind Betrug. Stufe 1 (billige Merkmale) lässt 8 % aller Zahlungen durch und enthält darunter 96 % der Betrugsfälle. Stufe 2 benötigt teure Abfragen (0,4 Cent je Zahlung), erkennt 90 % der durchgelassenen Betrugsfälle und markiert 1 % der durchgelassenen harmlosen Zahlungen fälschlich.

(a) Was kosten die teuren Abfragen mit und ohne Cascade?
(b) Wie hoch sind Gesamt-Recall und Precision?
(c) Welchen Recall braucht Stufe 1, damit der Gesamt-Recall mindestens 0,88 beträgt?

Lösung auf Seite 207

P5 Zwei Schwellen

Rechnen · Rebalancing ★★☆

Dasselbe Betrugsmodell wird mit zwei Schwellen auf 10 000 Zahlungen (100 Betrugsfälle) bewertet. Schwelle A: 80 Treffer, 40 Fehlalarme, 20 übersehene Fälle. Schwelle B: 90 Treffer, 150 Fehlalarme, 10 übersehene Fälle. Ein übersehener Betrug kostet 200 Euro, ein Fehlalarm 5 Euro.

(a) Berechnen Sie Genauigkeit, Precision, Recall und F1 für beide Schwellen. Wie gut ist die Regel „nie Betrug“?
(b) Berechnen Sie die Kosten. Welche Schwelle wählen Sie, und was folgt für die Rolle von F1?

Lösung auf Seite 207

P6 Uneinige Labeler

Diagnose und Rechnen · Neutral Class ★★☆

Ein Team labelt Rezensionen mit drei Personen und nimmt die Mehrheit als Label. Bei einer Rezension an der Grenze sagt jede Person unabhängig mit Wahrscheinlichkeit 0,55 „positiv“.

(a) Wie wahrscheinlich ist das Mehrheitslabel „positiv“, wie wahrscheinlich sind sich alle drei einig?
(b) Welche Folgen hat das für das Training? Welches Pattern setzen Sie ein, welche Alternative gibt es, und wann nutzt beides nichts?

Lösung auf Seite 207

P7 **Undersampling und Wahrscheinlichkeit** Rechnen und Code · Rebalancing ★★★

Von 50 000 Kunden wandern 1 000 ab (2 %).
(a) Welche Genauigkeit hat die Regel „wandert nie ab“?
(b) Welche Gewichte vergibt `class_weight="balanced"`?
(c) Man behält alle Abwanderer und nur 10 000 der Bleibenden (Faktor β). Das Modell gibt 0,5 aus. Welche Wahrscheinlichkeit gilt für die echte Kundschaft? Leiten Sie die Korrektur über die Odds her.
(d) Prüfen Sie die Korrektur an einer Simulation.

Lösung auf Seite 207

16.4 Model Training (Patterns 11 bis 15)

T1 **Overfitting als Ziel** Verständnis · Useful Overfitting ★☆☆

Ein Kollege will für die Kundenabwanderung „Useful Overfitting“ einsetzen, weil das Pattern sage, Overfitting sei gut. Was antworten Sie? Nennen Sie die Bedingungen, unter denen das Pattern gilt, und ein Beispiel, das sie erfüllt.

Lösung auf Seite 208

T2 **Checkpoint-Intervall** Rechnen und Code · Checkpoints ★★★

Ein Training braucht 60 Stunden Rechenzeit auf einer Instanz, die der Anbieter jederzeit entziehen darf (im Mittel alle 20 Stunden). Ein Checkpoint kostet 3 Minuten ($\delta = 0{,}05$ h). Bei Stunde 41 fällt die Instanz aus.
(a) Wie viel Arbeit geht verloren ohne Checkpoint, wie viel mit einem Checkpoint alle 2 Stunden?
(b) Wie viel Zeit kostet das Schreiben bei Intervallen von 2 Stunden und von 30 Minuten?
(c) Pro Intervall T fallen δ an, pro Ausfall gehen im Mittel $T/2$ Arbeit verloren. Bestimmen Sie das Intervall mit dem kleinsten Gesamtaufwand.
(d) Prüfen Sie die Näherung durch Simulation.

Lösung auf Seite 208

T3 **Falsch feinjustiert** Diagnose · Transfer Learning ★★☆

Ein Team hat 300 beschriftete Röntgenbilder. Es nimmt ein auf Alltagsfotos vortrainiertes Netz, trainiert alle Schichten mit der Lernrate 0,1 und ist schlechter als ein kleines Netz von Grund auf.
(a) Was ist die wahrscheinlichste Ursache?
(b) Welche zwei Varianten kennt das Pattern?
(c) Welche wählen Sie bei 300, welche bei 30 000 Bildern?

Lösung auf Seite 208

T4 **Acht Worker** Rechnen · Distribution Strategy ★★★

Acht Worker trainieren synchron, je 64 Beispiele pro Schritt; der Referenzlauf auf einem Gerät nutzt Batch 64 und Lernrate 0,1. Das Modell hat 25 Millionen Parameter (float32), die Rechenzeit je Schritt beträgt 0,30 s. Die Gradienten werden per Ring-All-Reduce abgeglichen; die Kommunikation überlappt nicht mit der Rechnung, ihre Zeit sei Datenvolumen durch Bandbreite.
(a) Welche effektive Batch-Größe und welche Lernrate ergibt die lineare Skalierungsregel?
(b) Wie viele Daten sendet jeder Worker je Schritt?
(c) Welche Beschleunigung ergibt sich bei 100 Gbit/s und bei 1 Gbit/s?

Lösung auf Seite 208

T5 Zufallssuche Rechnen · Hyperparameter Tuning ★★☆

(a) Wie wahrscheinlich liegt bei 60 zufällig gezogenen Konfigurationen mindestens eine im besten Zwanzigstel (5 %) des Suchraums?
(b) Wie viele Läufe braucht man für 99 %?
(c) Ein Raster hat drei Hyperparameter mit je vier Werten, aber nur einer beeinflusst das Ergebnis. Wie viele verschiedene Werte dieses einen Parameters probiert das Raster, wie viele eine Zufallssuche mit gleicher Laufzahl?

Lösung auf Seite 209

T6 Der Sieger auf den Testdaten Code · Hyperparameter Tuning ★★★

Erzeugen Sie reines Rauschen: 20 Merkmale, zufällige Labels, 100 Trainings-, 100 Test- und 5 000 frische Beispiele. Ziehen Sie 100 zufällige Konfigurationen eines Entscheidungsbaums.
(a) Wählen Sie die Konfiguration mit der besten Genauigkeit auf den 100 Testbeispielen. Was melden Sie, was erreicht sie auf den frischen Beispielen?
(b) Welchen Wert erwartet man als Maximum von 100 Zufallsergebnissen auf 100 Beispielen?
(c) Wie wählt man richtig?

Lösung auf Seite 209

T7 Wann hört das Training auf? Rechnen · Checkpoints ★★☆

Early Stopping mit Geduld 3 und Mindestverbesserung 0,001. Der Validierungs-Loss je Epoche (1 bis 14): 0,800; 0,620; 0,510; 0,450; 0,420; 0,405; 0,398; 0,399; 0,401; 0,396; 0,397; 0,402; 0,410; 0,425. In welcher Epoche stoppt das Training, und welcher Stand wird behalten? Wann wird `best` geschrieben?

Lösung auf Seite 209

16.5 Resilient Serving (Patterns 16 bis 20)

S1 Zähler im Arbeitsspeicher Diagnose · Stateless Serving Function ★★☆

Ein Prognoseservice läuft in vier Instanzen hinter einem Load Balancer. Für das Merkmal „Anfragen dieses Kunden in den letzten 10 Minuten“ zählt jede Instanz im Arbeitsspeicher mit. Seit der Skalierung auf vier Instanzen sind die Vorhersagen schlechter und hängen von der Instanz ab.
(a) Welches Pattern wird verletzt, und was geht schief?
(b) Wohin gehört der Zustand? Welche zwei Patterns kommen in Frage?

Lösung auf Seite 209

S2 Online oder Batch Rechnen · Batch Serving ★☆☆

200 000 Bestellungen sollen über Nacht bewertet werden. Annahme: Jeder Aufruf kostet 4 ms Fixaufwand (Netz, Serialisierung) plus 0,01 ms Rechenzeit je Zeile.
(a) Wie lange dauern 200 000 Einzelaufrufe, wie lange vier Batches zu 50 000 Zeilen?
(b) Wann bleibt man trotzdem bei Online-Vorhersagen?

Lösung auf Seite 209

S3 Anteil der zweiten Phase Rechnen · Two-Phase Predictions ★★☆

Ein Gerät klassifiziert Bilder zuerst mit einem kleinen Modell. Bei 82 % der Bilder liegt dessen Sicherheit über der Schwelle; dort ist es zu 99 % richtig. Die übrigen 18 % gehen an ein Cloud-Modell, das auf ihnen zu 97 % richtig ist (das kleine Modell wäre dort nur zu 71 % richtig). Es gibt 1 000 000 Anfragen am Tag, ein Bild hat 200 kB, eine Cloud-Anfrage kostet 0,0004 Euro, das Gerät antwortet in 20 ms, die Cloud-Phase kommt mit 250 ms hinzu.
(a) Wie hoch ist die Gesamtgenauigkeit?
(b) Wie viele Daten und Kosten fallen an, wie hoch ist die mittlere Latenz?
(c) Welche Annahme über die „Sicherheit“ des kleinen Modells ist entscheidend?

Lösung auf Seite 209

S4 **Schwelle wählen** Code · Two-Phase Predictions ★★★

Trainieren Sie auf den Ziffern von scikit-learn ein kleines Modell (logistische Regression auf 4×4-Mittelwerten) und ein großes (SVM auf 8×8). Bestimmen Sie auf einem Validierungsteil die Schwelle τ, bei der die Gesamtgenauigkeit höchstens einen Prozentpunkt unter der des großen Modells liegt und der Cloud-Anteil minimal ist. Berichten Sie das Ergebnis einmal auf den Testdaten.

Lösung auf Seite 210

S5 **Falsche Reihenfolge** Code · Keyed Predictions ★★☆

Ein Job bewertet vier Stücke zu je drei Bestellungen parallel. Die Antworten kommen in der Reihenfolge 2, 0, 3, 1 an und werden in dieser Reihenfolge an die Bestellungen geheftet.

(a) Bauen Sie das nach. Wie groß ist der Fehler, wie groß mit einem Join über die Bestell-ID?
(b) Was passiert, wenn ein Stück wegen einer Wiederholung doppelt ankommt?
(c) Welche Eigenschaften soll ein Schlüssel haben?

Lösung auf Seite 210

S6 **Verteilung verschoben** Rechnen · Continued Model Evaluation ★★☆

Ein Merkmal (Warenkorbwert) fällt in fünf Klassen. Anteile im Training: 20, 30, 25, 15, 10 %. Anteile im Betrieb: 10, 25, 25, 20, 20 %.

(a) Berechnen Sie den Population Stability Index $\mathrm{PSI} = \sum_i (a_i - e_i)\ln(a_i/e_i)$.
(b) Wie ordnen Sie den Wert ein?
(c) Was kann diese Kennzahl nicht erkennen?

Lösung auf Seite 210

S7 **Wie viele Labels braucht der Alarm?** Rechnen · Continued Model Evaluation ★★★

Ein Modell hatte 90 % Genauigkeit. Ein Rückgang auf 87 % soll mit 5 % Irrtumswahrscheinlichkeit und 80 % Power erkannt werden. Für den Test einer Anteilsrate gilt $n = \left(\frac{z_{\alpha/2}\sqrt{p_0(1-p_0)} + z_{\beta}\sqrt{p_1(1-p_1)}}{p_0 - p_1}\right)^2$ mit $z_{\alpha/2} = 1{,}96$ und $z_{\beta} = 0{,}84$.

(a) Wie viele beschriftete Fälle braucht man?
(b) Es kommen 100 beschriftete Fälle am Tag, die Labels treffen 30 Tage nach der Vorhersage ein. Wann erscheint der erste belastbare Alarm?
(c) Wie kommt man früher zu einem Signal?

Lösung auf Seite 210

16.6 Reproducibility (Patterns 21 bis 27)

R1 **Zwei Skalierer** Diagnose und Rechnen · Transform ★★☆

Das Training standardisiert das Alter mit Mittelwert 40 und Standardabweichung 10 aus den Trainingsdaten. Der Service rechnet `(alter - alter.mean()) / alter.std()` auf der eingehenden Anfrage (ein NumPy-Array, `std` ohne Freiheitsgradkorrektur). Eine Anfrage enthält drei Kunden mit 62, 58 und 66 Jahren.

(a) Welche z-Werte rechnen Training und Service?
(b) Was passiert bei einer Anfrage mit einem Kunden?
(c) Wie heißt der Fehler, wie behebt ihn das Pattern? Wie heißt der verwandte Fehler, wenn man den Skalierer vor dem Split auf allen Daten anpasst?

Lösung auf Seite 211

R2 Split über den Hash

Rechnen und Code · Repeatable Splitting ★★☆

10 000 Zeilen gehören zu 1 000 Patienten. Implementieren Sie `ist_training(patient_id)` so, dass 80 % der Patienten (nicht der Zeilen) im Training landen.

(a) Zeigen Sie: Anteil, keine Patienten auf beiden Seiten, Stabilität nach 200 neuen Patienten. Vergleichen Sie mit einem zufälligen Split.
(b) Wie stark streut der Trainingsanteil bei 1 000 Patienten?
(c) Warum taugt Pythons `hash()` nicht?

Lösung auf Seite 211

R3 Zwei Kartenarten

Rechnen · Bridged Schema ★★☆

Die alten Zeilen haben „Karte“, die neuen unterscheiden Kreditkarte (70 %) und Debitkarte (30 %).

(a) Man weist den alten Zeilen die Kartenart zufällig nach 70 zu 30 zu. Welcher Anteil der zugewiesenen Werte stimmt, wenn die wahren Anteile früher gleich waren?
(b) Welcher Anteil stimmt, wenn man allen alten Zeilen „Kreditkarte“ zuweist? Was wäre eine dritte, „ehrliche“ Möglichkeit?
(c) Was gilt, wenn die Anteile früher 50 zu 50 waren?

Lösung auf Seite 211

R4 Fenster

Code · Windowed Inference ★★★

Das Merkmal ist der Mittelwert der letzten drei Messwerte (einschließlich des aktuellen). Die Messwerte sind $10, 12, 11, 15, 20, 18, 17$.

(a) Berechnen Sie es als Batch (pandas `rolling`) und als Stream (feste Warteschlange). Sind die Ergebnisse gleich?
(b) Was liefert ein zentriertes Fenster, und warum ist es im Training falsch?
(c) Wie behandelt man die ersten beiden Werte?

Lösung auf Seite 211

R5 Welcher Wert zu welchem Zeitpunkt?

Code · Feature Store ★★★

Der Feature Store hält „Bestellungen in 30 Tagen“ mit Gültigkeitsbeginn: Kunde 1: 2 (ab 1.3.), 5 (ab 10.3.), 1 (ab 20.3.); Kunde 2: 7 (ab 5.3.), 3 (ab 25.3.). Beschriftete Ereignisse: Kunde 1 am 12.3. und 25.3., Kunde 2 am 3.3. und 15.3. Welchen Wert soll jedes Ereignis im Trainingsdatensatz bekommen? Was liefert der Join „neuester Wert je Kunde“? Setzen Sie den zeitpunktrichtigen Join mit pandas um.

Lösung auf Seite 212

R6 Was läuft neu?

Rechnen · Workflow Pipeline ★★☆

Eine Pipeline hat sieben Schritte: `ingest`, `clean`, `featurize`, `split`, `train`, `evaluate`, `register`. `clean` braucht `ingest`; `featurize` und `split` brauchen `clean`; `train` braucht `featurize` und `split`; `evaluate` braucht `train` und `split`; `register` braucht `train` und `evaluate`.

(a) Geben Sie eine topologische Ordnung an. Welche Schritte können parallel laufen?
(b) Welche Schritte laufen neu, wenn sich `train`, `split` oder `clean` ändert?
(c) Warum darf der Graph keinen Zyklus enthalten?

Lösung auf Seite 212

R7 Neue Pflichtfeld-Version

Diagnose · Model Versioning ★★☆

Modell v2 verlangt ein neues Pflichtfeld `kundentyp`; drei Clients senden es nicht. Außerdem liefert v2 neu trainierte Embeddings, die ein nachgelagertes Modell als Eingabe nutzt.

(a) Was bricht?
(b) Wie löst Pattern 27 das?
(c) Warum sind Embeddings verschiedener Modellversionen nicht austauschbar?

Lösung auf Seite 212

16.7 Responsible AI (Patterns 28 bis 30)

V1 Der Maßstab Code · Heuristic Benchmark ★★☆

Erzeugen Sie mit `lieferdaten()` 6 000 Lieferungen (Entfernung, Artikel, Regen; Rauschen mit Standardabweichung 2,5 Minuten).

(a) Bestimmen Sie den mittleren absoluten Fehler für den Mittelwert, die Faustregel „10 Minuten plus 3 Minuten je Kilometer“, eine lineare Regression und Gradient Boosting.

(b) Welcher Wert ist der Maßstab, den ein Modell schlagen muss? Lohnt sich Gradient Boosting?

Lösung auf Seite 212

V2 Beiträge verteilen Rechnen · Explainable Predictions ★★☆

Das Modell sagt die Lieferzeit voraus: $8 + 2{,}5 \cdot \mathrm{km} + 0{,}6 \cdot \mathrm{Artikel} + 5 \cdot \mathrm{Regen}$ (Regen ist 0 oder 1). Der Fall: 9 km, 5 Artikel, Regen.

(a) Verteilen Sie die Differenz zur Vorhersage der Baseline (Mittelwerte: 4 km, 3 Artikel, Regenanteil 0,3) als Beitrag $w_i(x_i - b_i)$ auf die Merkmale. Was ergibt die Summe?

(b) Wie ändert sich die Verteilung mit der Baseline „trockener Tag“ (Regen 0)?

Lösung auf Seite 213

V3 Die Postleitzahl als Grund Verständnis · Explainable Predictions ★★☆

Bei einem abgelehnten Kreditantrag zeigt die Erklärung den größten negativen Beitrag für die Postleitzahl. Ein Kollege rät dem Kunden, umzuziehen.

(a) Was ist an der Schlussfolgerung falsch?

(b) Nennen Sie zwei Gründe, warum Attributionen keine Ursachen sind.

(c) Was sollte der Kunde stattdessen erfahren, und welches Pattern prüft die Postleitzahl?

Lösung auf Seite 213

V4 Zwei Gruppen im Vergleich Rechnen · Fairness Lens ★★☆

Ein Kreditmodell liefert je Gruppe diese Konfusionsmatrizen (tatsächlich kreditwürdig / vom Modell angenommen):

Gruppe	Treffer	Fehlalarme	übersehen	richtig abgelehnt	Anzahl
A	320	60	80	540	1 000
B	60	40	40	360	500

(a) Berechnen Sie Genauigkeit (gesamt und je Gruppe), Auswahlrate, True-Positive-Rate, False-Positive-Rate und Precision je Gruppe.

(b) Welche Fairness-Kriterien sind verletzt, welche erfüllt?

Lösung auf Seite 213

V5 Nicht alles zugleich Rechnen · Fairness Lens ★★★

Zwei Gruppen haben die Basisraten 0,4 (A) und 0,2 (B) für das positive Ergebnis. Das Modell erreicht in beiden Gruppen dieselbe True-Positive-Rate 0,8 und False-Positive-Rate 0,1.

(a) Berechnen Sie die Precision (positiver prädiktiver Wert) und die Auswahlrate je Gruppe.

(b) Welche False-Positive-Rate müsste B haben, damit die Precision von B gleich der von A wird (bei TPR 0,8)? Was folgt?

Lösung auf Seite 214

V6 Entfernt und doch da Code · Fairness Lens ★★★

Historische Einstellungen sind verzerrt: Bei gleichem Können wurde Gruppe 1 seltener eingestellt. Das Merkmal „Gruppe“ wird nicht ins Modell gegeben, ein Merkmal („Proxy“, z. B. die Postleitzahl) korreliert aber mit ihr. Simulieren Sie das mit 40 000 Personen, trainieren Sie eine logistische Regression auf Können und Proxy beziehungsweise nur auf Können und wählen Sie jeweils die besten 30 %. Wie unterscheiden sich die Auswahlraten der Gruppen?

Lösung auf Seite 214

16.8 Szenarien über mehrere Kategorien

In echten Projekten tritt selten ein einzelnes Pattern auf. Die drei Szenarien beschreiben je ein System mit mehreren Schwächen (K1 berührt fünf der sechs Kategorien, K2 vier, K3 drei), und zu jedem gehören fünf Fragen. Die Lösung zeigt eine Zielarchitektur, in der jedes Pattern dort steht, wo es wirkt. Alle Zahlen sind Annahmen der Aufgabe.

K1 Betrugserkennung bei einem Zahlungsdienstleister

Ein Dienstleister verarbeitet 40 Millionen Transaktionen am Tag, 0,1 % sind Betrug (40 000 Fälle). Ein Gradient-Boosting-Modell nutzt 120 Merkmale, darunter 18 mit Abfragen bei Drittanbietern (zusammen 0,05 Cent je Transaktion). Das Team meldet 99,9 % Genauigkeit, die Fraud-Abteilung findet aber nur 8 % der Betrugsfälle. Ein Nachtjob bewertet verdächtige Konten auf mehreren Workern erneut; die Ergebnislisten passen nicht zu den Konten. Rückbuchungen bestätigen Betrug erst nach bis zu 60 Tagen, und die Betrüger wechseln ihre Muster alle paar Wochen. Kunden aus Gruppe G werden zu 0,6 %, aus Gruppe H zu 0,1 % fälschlich gesperrt; das Herkunftsland geht als Hashed Feature mit 64 Buckets ein.

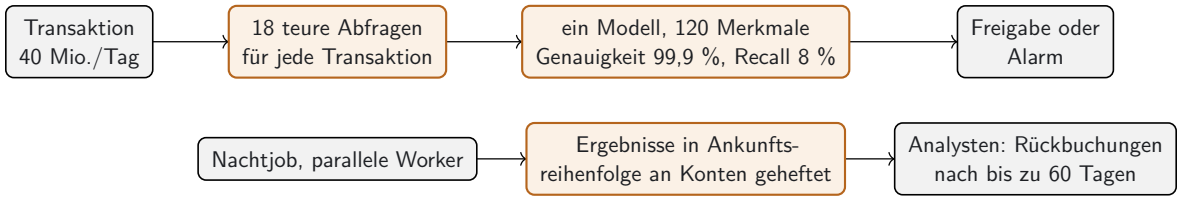


Abbildung 122: Ist-Zustand von K1. Orange: Stellen, an denen die Beschreibung ein Problem zeigt.

(a) Welche Kennzahlen ersetzen die Genauigkeit, welche Maßnahmen des Rebalancing eignen sich, und was bleibt dabei unangetastet?
(b) Entwerfen Sie eine Cascade. Welchen Recall braucht Stufe 1 für einen Gesamt-Recall von mindestens 0,85, wenn Stufe 2 90 % erkennt? Was kosten die teuren Abfragen, wenn 5 % der Transaktionen Stufe 2 erreichen?
(c) Welches Pattern behebt die falsch zugeordneten Ergebnislisten?
(d) Wie überwachen Sie das Modell bei bis zu 60 Tagen Labelverzug?
(e) Was prüfen Sie wegen der Sperrquoten, und welchen Zusammenhang mit dem Hashed Feature?

Lösung auf Seite 215

K2 Röntgen-Triage im Krankenhaus

Ein Krankenhaus will Thorax-Röntgenbilder nach Dringlichkeit sortieren: 11 840 Aufnahmen von 3 200 Patienten (3,7 je Patient), jede von drei Radiologen unabhängig beurteilt, bei 22 % uneinig. Das Team nimmt ein auf Alltagsfotos vortrainiertes Netz, teilt die Aufnahmen zufällig 80 zu 20 und erreicht 97 % Genauigkeit; im Pilot an einem zweiten Haus sind es 81 %. Heatmaps zeigen hohe Beiträge in der Bildecke, in der die einsendende Station eingeblendet ist. Während des Piloten wird das Modell zweimal ausgetauscht, ohne dass sich Auswertungen danach einem Stand zuordnen lassen. Dringende Fälle in der Altersgruppe über 80 werden deutlich seltener erkannt (Annahme).

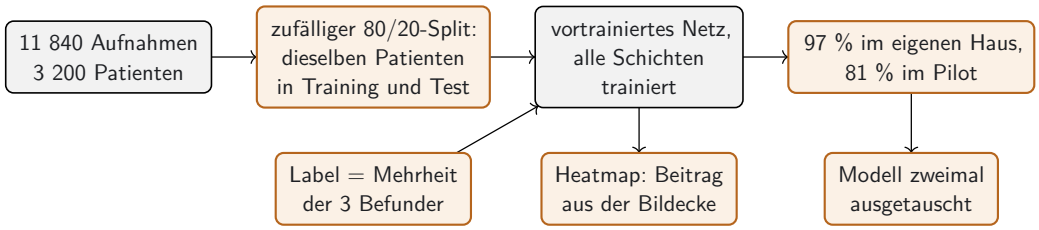


Abbildung 123: Ist-Zustand von K2.

(a) Warum 97 % im eigenen Haus, aber 81 % im Pilot? Wie wahrscheinlich steht ein Patient mit vier Aufnahmen bei einer zufälligen 80/20-Aufteilung auf beiden Seiten? Wie teilen Sie richtig, und wie viele Patienten und Aufnahmen hat der Test dann ungefähr?
(b) Welche Variante des Transfer Learning wählen Sie bei rund 9 500 Trainingsaufnahmen und warum? Wie viele Aufnahmen betrifft die Uneinigkeit der Befunder, und was tun Sie damit?
(c) Was zeigt die Heatmap, wie heißt das Phänomen, und wie gehen Sie vor?

(d) Welchen Maßstab setzen Sie vor das Netz, und wie sorgen Sie dafür, dass jede Auswertung einem Modellstand zuzuordnen ist?
(e) Was prüfen Sie wegen der Altersgruppe über 80, und worauf achten Sie bei der Aussagekraft?

Lösung auf Seite 215

K3 Verbrauchsprognose bei einem Energieversorger

Ein Versorger prognostiziert für 1,5 Millionen Smart Meter den Verbrauch des nächsten Tages. Jeder Zähler meldet 96 Viertelstundenwerte am Tag (144 Millionen Werte). Merkmale sind der Mittelwert der letzten 24 Stunden und der Wert zur gleichen Zeit vor einer Woche. Im Training berechnet ein Data Scientist den Mittelwert mit `rolling(96, center=True)`, im Service zählt jede Instanz die eingehenden Werte selbst mit. Die Nachtprognose soll den Online-Service 144 Millionen Mal aufrufen (angenommen 4 ms je Aufruf). Fünf Modellvarianten wurden probiert, auf welchem Datenstand, ist unklar. Bei einer Hitzewelle steigt der Fehler, obwohl technisch nichts ausfällt.

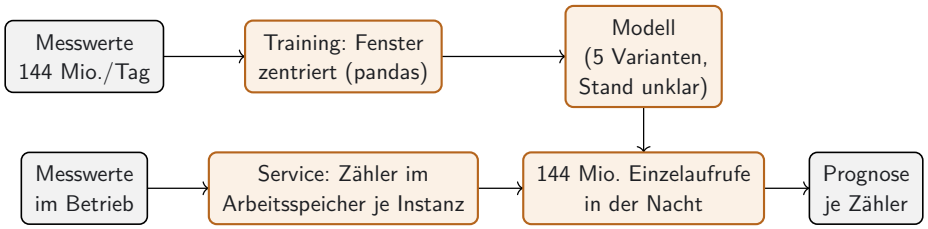


Abbildung 124: Ist-Zustand von K3. Training und Betrieb berechnen das Merkmal an verschiedenen Orten auf verschiedene Weise.

(a) Welche zwei Fehler stecken im Merkmal „Mittelwert der letzten 24 Stunden“, und welche Patterns beheben sie?
(b) Wie viel Zustand hält eine Stream-Verarbeitung, wenn sie für jeden Zähler alle 96 Werte (float64) vorhält? Wie viel genügt bei festen, nicht überlappenden Fenstern?
(c) Wie lange würden 144 Millionen Einzelaufrufe dauern, und wie sieht die Alternative aus?
(d) Welche zwei Patterns beantworten die Frage, welcher Datenstand zu welchem Modell gehört?
(e) Wie erkennen Sie die Auswirkung der Hitzewelle früh, und woran messen Sie, ob das Modell überhaupt noch etwas leistet?

Lösung auf Seite 216

16.9 Lösungen

Die Lösungen sind nach derselben Reihenfolge geordnet wie die Aufgaben. Wo eine Rechnung dabei ist, steht die Formel mit eingesetzten Zahlen; die Ergebnisse stimmen mit den Ausgaben der Skripte überein.

Data Representation

D1 Aufgabe auf Seite 195

(a) Das Pattern ersetzt eine gelernte Zuordnung (ID zu Spalte) durch eine berechnete. Jede Zeichenkette hat einen Bucket, auch eine nie gesehene, und es gibt keine Tabelle, die fehlen oder veralten könnte. **(b)** Die Vorhersage für die neue ID stützt sich auf das Gewicht ihres Buckets, und das stammt von den bekannten Werten, die zufällig dort liegen. Über das Produkt selbst weiß das Modell nichts, die Antwort sieht aber aus wie jede andere. Brauchbar wird sie erst, wenn Information über das Produkt in die Eingabe geht (Kategorie, Preisklasse, ein Embedding aus der Beschreibung) oder ein Kennzeichen „neu“ das Modell auf den Fall vorbereitet. **(c)** Hashing zerstört Ähnlichkeit: zwei fast gleiche Produktnamen landen in unverwandten Buckets, was Embeddings (Pattern 2) leisten. Und es verschleiert, welcher Wert hinter einem Gewicht steckt; ein Bucket gehört zu einer ganzen Gruppe von Werten, was die Erklärbarkeit einschränkt.

D2 Aufgabe auf Seite 196

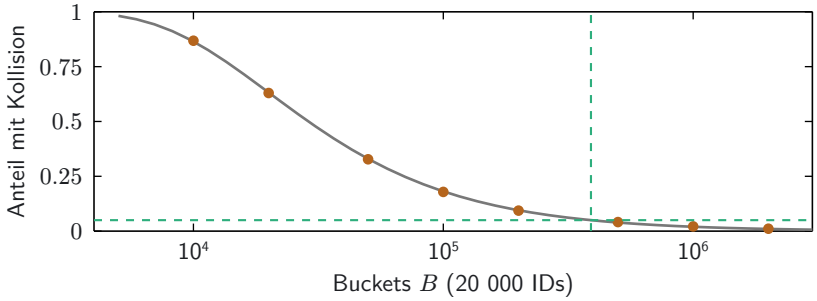


Abbildung 125: Anteil der IDs mit Kollision (Linie: Formel, Punkte: gemessen); grün gestrichelt: 5-%-Grenze bei $B = 389\ 896$.

(a) Eingesetzt: $1-(1-1/50\ 000)^{19\ 999} = 0{,}3297$, rund ein Drittel der IDs teilt sich einen Bucket. Belegte Buckets: $50\ 000\ (1-(1-1/50\ 000)^{20\ 000}) \approx 16\ 484$. Die 20 000 IDs verteilen sich also auf nur 16 484 Buckets, obwohl es 2,5-mal so viele Buckets wie IDs gibt. **(b)** Aus $1-(1-1/B)^{n-1} \le 0{,}05$ folgt $B \ge 1/\left(1-0{,}95^{1/(n-1)}\right) = 389\ 896$; die Näherung $B \ge n/(-\ln 0{,}95) = 389\ 915$ liegt daneben kaum. Man braucht das 19,5-Fache der Wertezahl. **(c)** Die Messung mit echten Hashes bestätigt die Formel (Abbildung 125). Die Konsequenz: Ein nahezu kollisionsfreies Hashing kostet fast so viel Platz wie One-Hot. Das Pattern lohnt, wenn Kollisionen hinnehmbar sind, etwa weil die meisten Werte selten vorkommen. Bei nur 2 000 Werten (Postleitzahlen) wäre es dagegen unnötig: 2 000 One-Hot-Spalten sind klein, ein Hash auf 500 Buckets ließe fast alle Werte kollidieren (98 %).

D3 Aufgabe auf Seite 196

(a) $7 \cdot 24 \cdot 23 = 3\ 864$ Zellen, im Mittel $1\ 000\ 000\ /\ 3\ 864 = 258{,}8$ Fahrten je Zelle. Wochentag × Stunde ergibt 168 Zellen und 5 952 Fahrten. Der Mittelwert täuscht: Nachts in einem Randbezirk sind es nur wenige, und die Gewichte solcher Zellen sind unsicher. **(b)** Nur fachlich sinnvolle Paare kreuzen (Stunde × Bezirk: 552 Zellen, 1 812 Fahrten); den Wochentag gröber fassen (Werktag/Wochenende × Stunde × Bezirk: $2 \cdot 24 \cdot 23 = 1\ 104$ Zellen, 906 Fahrten); seltene Zellen zusammenfassen oder verwerfen; das Kreuzprodukt hashen (Pattern 1) oder in ein Embedding überführen (Pattern 2).

D4 Aufgabe auf Seite 196

```
bx, by = np.digitize(x, kanten), np.digitize(y, kanten)  # je 4 Buckets: 0 bis 3
zelle = bx * 4 + by  # das Feature Cross: 16 Zellen
```

Terminal

```
$ python 01_darstellung.py
Logistische Regression, Buckets einzeln (8 Spalten): 0.554
Logistische Regression, Cross der Buckets (16 Spalten): 0.951
Entscheidungsbaum, Buckets einzeln, Tiefe 4:           0.951
Obergrenze durch das Rauschen:                         0.950
Beste Gerade auf den Rohwerten (Rastersuche): Training 0.654, Test 0.654
Ohne Cross, 20 Wiederholungen: Mittel 0.506, Minimum 0.384, Maximum 0.670
```

(a) Die beste Gerade auf den Rohwerten erreicht 0,654. Ohne Rauschen liegt die Grenze bei zwei Dritteln (eine eigene Rastersuche auf 400 000 Punkten ergibt 0,667; sie steht nicht im Skript), mit 5 % Rauschen bei $0{,}05 + 0{,}9 \cdot 2/3 = 0{,}65$: Eine Gerade trennt nicht alle vier Quadranten, sie schneidet nur eine Ecke ab. **(b)** Mit den Buckets einzeln findet das Modell nichts, denn in jedem x-Bucket und in jedem y-Bucket ist die Labelrate 50 %; der Zusammenhang steckt nur in der Kombination. Der Einzellauf (0,554) täuscht: Über 20 Datensätze gemittelt sind es 0,506, die Spanne reicht von 0,384 bis 0,670, je nachdem, welches zufällige Restmuster das Training gefunden hat. Mit dem Cross sind es 0,951, und die Obergrenze liegt wegen des Rauschens bei 0,95. **(c)** Der Baum erreicht 0,951 ohne Cross: Er teilt zuerst nach x und dann nach y und bildet die Kombination durch seine Verzweigung selbst. Deshalb ist das Cross vor allem für Modelle wichtig, die nur gewichtete Summen bilden.

D5 Aufgabe auf Seite 196
(a) $\sqrt[4]{4\ 000\ 000} = 44{,}7$, also $d = 45$. **(b)** $4\ 000\ 000 \cdot 45 = 180$ Millionen Parameter, mal 4 Byte sind 720 MB. Nach dem Hashing auf 500 000 Buckets sind es $500\ 000 \cdot 45 = 22{,}5$ Millionen Parameter, 90 MB, ein Achtel. Zum Vergleich hätte ein lineares Modell mit One-Hot nur 4 Millionen Gewichte (16 MB); das Embedding gibt jedem Titel 45-mal mehr Parameter, die gelernt werden wollen. **(c)** Seltene Titel, die sich einen Bucket mit einem beliebten teilen: Ihr Vektor wird von den vielen Beispielen des beliebten Titels bestimmt. Häufige Titel spüren die Kollision kaum.

D6 Aufgabe auf Seite 196
(a) Training-Serving Skew: Der Nullvektor als „Foto fehlt“ kam im Training nie vor. Nach der Standardisierung bedeutet null außerdem „durchschnittliches Bild“, nicht „kein Bild“, und das Modell wertet ihn wie jede andere Eingabe. Es verhält sich auf diesen Eingaben unvorhersehbar. **(b)** Während des Trainings die Quelle zufällig weglassen (in 30 % der Beispiele ohne Foto), damit das Modell den Fall kennt, und ein Kennzeichen „Foto vorhanden“ mitgeben; alternativ ein zweites Modell ohne Foto als Rückfall und eine Weiche davor. **(c)** $\sqrt{10\ 000} = 100$ gegen $\sqrt{5} = 2{,}24$, das Verhältnis ist 44,7. Bei abstandsbasierten Verfahren übertönt das Bild die Tabelle; man verdichtet deshalb jede Quelle zuerst auf eine vergleichbare Länge.

Problem Representation

P1 Aufgabe auf Seite 197
(a) Der quadratische Fehler wird vom Mittelwert minimiert: 40 Minuten. Diese Zeit kommt in den Daten nie vor; jede Lieferung liegt um 20 Minuten daneben. **(b)** Mit Zeitklassen (etwa bis 30, 30 bis 50, über 50 Minuten) liefert das Modell eine Wahrscheinlichkeit je Klasse, hier ungefähr 50 % für „kurz“ und 50 % für „lang“. Die App kann beides zeigen oder auf die Auslastung warten. Eine andere Möglichkeit sind Quantile statt des Mittelwerts [38]. **(c)** Sternebewertungen von eins bis fünf als Zahl vorherzusagen nutzt die Ordnung der Klassen, die eine Klassifikation ignoriert.

P2 Aufgabe auf Seite 197
(a) Softmax: $(0{,}549; 0{,}406; 0{,}037; 0{,}008)$, Summe 1. Sigmoid: $(0{,}900; 0{,}870; 0{,}378; 0{,}119)$, Summe 2,27. **(b)** Bei Softmax liegt nur Tag A über 0,5, bei Sigmoid sind es A und B. **(c)** Softmax koppelt die Ausgänge: Die Wahrscheinlichkeiten konkurrieren und summieren zu 1. Zwei Tags mit je 0,9 sind unmöglich, und bei drei zutreffenden Tags könnte keiner viel mehr als ein Drittel bekommen. Nimmt man die zwei größten, muss man vorher wissen, wie viele Tags zutreffen. Sigmoid behandelt jeden Tag als eigene Ja/Nein-Frage; dazu gehört die binäre Kreuzentropie je Tag, über die Tags gemittelt. Die Schwelle kann je Tag gewählt werden.

P3 Aufgabe auf Seite 197
(a) Drei Modelle: $3 \cdot 0{,}7^2 \cdot 0{,}3 + 0{,}7^3 = 0{,}441 + 0{,}343 = 0{,}784$. Fünf Modelle: 0,837. **(b)** Die Rechnung setzt unabhängige Fehler voraus. Modelle, die auf denselben Daten und mit ähnlichen Verfahren trainiert wurden, scheitern an denselben schwierigen Beispielen. Ein einfaches Modell der Korrelation (jedes Beispiel hat eine Schwierigkeit; gegeben diese, sind die Modelle unabhängig) macht das sichtbar (Abbildung 126): Bei einer Korrelation der Treffer von 0,14 sind es mit drei Modellen 0,754, mit 21 Modellen nur 0,832; bei 0,33 sind es 0,728 und 0,756, der Grenzwert liegt bei 0,763. Vielfalt der Modelle ist wertvoller als ihre Zahl [110]. **(c)** Bagging trainiert dasselbe Verfahren auf Bootstrap-Stichproben und mittelt, was vor allem die Varianz senkt [3]. Boosting trainiert Modelle nacheinander auf den Fehlern der Vorgänger und senkt vor allem den Bias [5]. Stacking lernt mit einem Meta-Modell, die Vorhersagen verschiedener Modelle zu kombinieren [41].

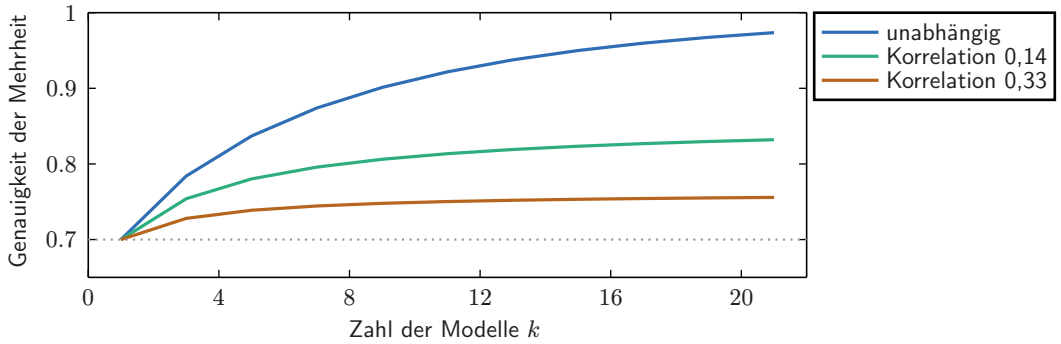


Abbildung 126: Genauigkeit der Mehrheitsentscheidung bei Einzelmodellen mit 70 % (gepunktet). Je stärker die Fehler zusammenhängen, desto früher flacht die Kurve ab. Werte aus `02_problem.py`.

P4 Aufgabe auf Seite 197

(a) Ohne Cascade: $100\,000 \cdot 0{,}4$ Cent = 400 Euro. Mit Cascade prüft Stufe 2 nur 8 000 Zahlungen: 32 Euro, also 368 Euro (92 %) weniger am Tag. **(b)** Gesamt-Recall $0{,}96 \cdot 0{,}90 = 0{,}864$. Stufe 2 sieht 960 Betrugsfälle und 7 040 harmlose Zahlungen und meldet 864 Treffer und rund 70 Fehlalarme; die Precision ist $864/(864 + 70) = 0{,}925$. **(c)** Aus $r_1 \cdot 0{,}90 \geq 0{,}88$ folgt $r_1 \geq 0{,}978$. Was Stufe 1 aussortiert, ist endgültig verloren (hier 40 Betrugsfälle), deshalb trimmt man sie auf hohen Recall und nimmt viele Fehlalarme in Kauf, die Stufe 2 wieder aussortiert.

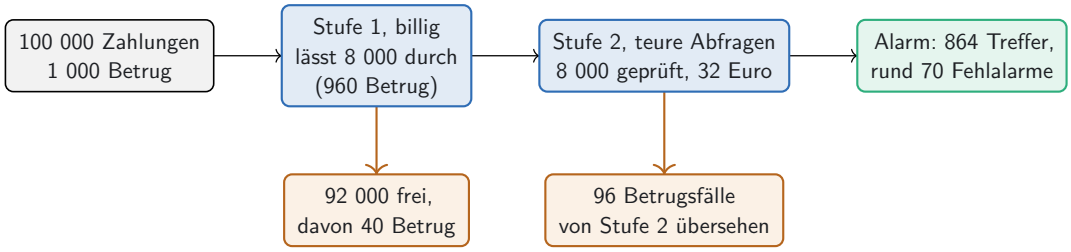


Abbildung 127: Die Cascade aus P4 mit den Zahlen der Rechnung. Orange: Betrug, der verloren geht.

P5 Aufgabe auf Seite 197

(a) Schwelle A: Genauigkeit $9\,940 \;/\; 10\,000 = 0{,}994$; Precision $80/120 = 0{,}667$; Recall $80/100 = 0{,}80$; $F_1 = 0{,}727$. Schwelle B: Genauigkeit 0,984; Precision $90/240 = 0{,}375$; Recall 0,90; $F_1 = 0{,}529$. Die Regel „nie Betrug“ hat 99,0 % Genauigkeit und ist wertlos; beide Schwellen liegen mit 99,4 % und 98,4 % um weniger als einen Prozentpunkt daneben, B sogar darunter. **(b)** Kosten A: $20 \cdot 200 + 40 \cdot 5 = 4\,200$ Euro; B: $10 \cdot 200 + 150 \cdot 5 = 2\,750$ Euro; „nie Betrug“: 20 000 Euro. B ist billiger, obwohl F1 und Genauigkeit schlechter sind. F1 gewichtet Precision und Recall gleich, der Schaden tut das nicht. Mit diesen Kosten lohnt sich ein Alarm bei kalibrierten Wahrscheinlichkeiten schon ab $p \cdot 200 > (1 - p) \cdot 5$, also ab $p > 5/205 = 2{,}4$ %. Die Schwelle ist eine Entscheidung über Kosten, kein Optimum einer Kennzahl.

P6 Aufgabe auf Seite 197

(a) Mehrheit „positiv“: $3 \cdot 0{,}55^2 \cdot 0{,}45 + 0{,}55^3 = 0{,}408 + 0{,}166 = 0{,}575$. Alle drei einig: $0{,}55^3 + 0{,}45^3 = 0{,}166 + 0{,}091 = 0{,}258$. **(b)** Das Mehrheitslabel einer solchen Rezension ist mit 57,5 % zu 42,5 % kaum mehr als ein Münzwurf. Das Modell lernt, diesen Zufall zu erklären, und liefert im Betrieb auch für Grenzfälle eine sichere Ja/Nein-Antwort. Das Pattern Neutral Class führt eine dritte Klasse ein, die aus der Uneinigkeit abgeleitet wird; der Betrieb gibt solche Fälle an einen Menschen. Alternativ kann der Anteil der Zustimmung als weiches Ziel dienen. Nichts davon hilft, wenn die Wahrheit objektiv feststellbar ist: Dann ist Uneinigkeit ein Qualitätsproblem der Labels, und man verbessert die Labels.

P7 Aufgabe auf Seite 198

(a) 98 %. **(b)** $w_c = n/(k \cdot n_c)$: positiv $50\,000\,/(2 \cdot 1\,000) = 25$, negativ $50\,000\,/(2 \cdot 49\,000) = 0{,}510$. **(c)** Mit $\beta = 10\,000 \;/\; 49\,000 = 0{,}204$ als Anteil behaltener Negativer gilt $P(\text{Auswahl} \mid \text{positiv}) = 1$ und $P(\text{Auswahl} \mid \text{negativ}) = \beta$. Damit sind die Odds in der Stichprobe $o_{\text{Stichprobe}} = o_{\text{echt}}/\beta$, also $o_{\text{echt}} = \beta \cdot o_{\text{Stichprobe}}$ [111]. Bei 0,5 ist $o_{\text{Stichprobe}} = 1$, also $o_{\text{echt}} = 0{,}204$ und $p = 0{,}204 \;/\; 1{,}204 = 0{,}170$. Bei 0,9 sind es $0{,}647$. **(d)** In der Simulation (Anteil 1,9 %) ist die mittlere vorhergesagte Wahrscheinlichkeit des vollständig trainierten Modells 0,0185, des unterabgetasteten 0,0756 und nach der Korrektur 0,0193;

die wahre Rate beträgt 0,0186. Das Ranking der Kunden ändert sich durch die Korrektur nicht, für eine Schwelle genügt also das unkorrigierte Modell. Wer mit den Zahlen rechnet (Kosten, Erwartungswerte wie in P5), muss korrigieren.

Model Training

T1 Aufgabe auf Seite 198

Das Pattern gilt nur unter besonderen Bedingungen: Der Eingaberaum ist bekannt und wird von den Trainingsdaten abgedeckt (oder die Funktion ist zwischen den Stützstellen glatt), die Labels sind exakt statt verrauscht, und es wird nur dort abgefragt, wo trainiert wurde. Ein Beispiel ist ein Surrogatmodell für eine Simulation, die man nicht millionenfach rechnen will: Die Simulation ist deterministisch, jedes Label stimmt, und Regularisierung kostet nur Genauigkeit. Bei der Kundenabwanderung sind die Labels verrauscht (viele Gründe, die nicht in den Daten stehen) und die künftigen Kunden unbekannt; ein Modell, das den Trainingsfehler auf null drückt, lernt den Zufall dieser Daten und verallgemeinert schlechter.

T2 Aufgabe auf Seite 198

(a) Ohne Checkpoint gehen 41 Stunden verloren, mit einem Checkpoint alle 2 Stunden nur 1 Stunde (letzter Stand bei Stunde 40). **(b)** Bei 2 Stunden: 30 Checkpoints, $30 \cdot 3$ min = 1,5 h (2,5 %). Bei 30 Minuten: 120 Checkpoints, 6 h (10 %). **(c)** Der Gesamtaufwand ist $C(T) = (60/T) \cdot \delta + (60/20) \cdot T/2 = 3/T + 1{,}5\ T$ Stunden. Ableiten und null setzen: $-3/T^2 + 1{,}5 = 0$, also $T = \sqrt{2} = 1{,}41$ h; allgemein $T = \sqrt{2\delta \cdot \mathrm{MTBF}}$ (Näherung erster Ordnung nach [112]; [113] gibt eine Näherung höherer Ordnung an). Der Aufwand beträgt dann 4,24 h, rund 7 % der Arbeitszeit. **(d)** Die Simulation (4 000 Läufe je Intervall) liegt nahe an der Formel und weicht erst bei Intervallen im Bereich der Ausfallzeit stark ab (15 h: 30,6 h simuliert gegen 22,7 h nach Formel), weil die Näherung kleine Intervalle voraussetzt (Abbildung 128). Ohne Checkpoints startet jeder Ausfall bei null; der erwartete Mehraufwand beträgt dann $\mathrm{MTBF}\ \left(e^{60/20} - 1\right) - 60 = 322$ Stunden.

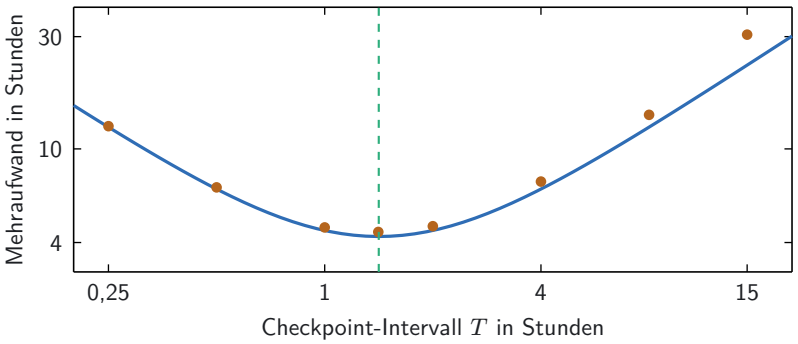


Abbildung 128: Mehraufwand für 60 Stunden Arbeit (Linie: Formel, Punkte: Simulation, grün: $T = 1{,}41$ h). Beide Achsen logarithmisch.

T3 Aufgabe auf Seite 198

(a) Eine Lernrate von 0,1 ist für vortrainierte Gewichte sehr groß und überschreibt sie in den ersten Schritten; was das Netz bei Kanten und Texturen schon konnte, geht verloren, und mit 300 Bildern lässt es sich nicht neu lernen. **(b)** Bei der Feature-Extraction friert man die vortrainierten Schichten ein und trainiert nur einen neuen Kopf. Beim Fine-Tuning trainiert man einige oder alle Schichten mit einer deutlich kleineren Lernrate als beim Vortraining weiter. **(c)** Bei 300 Bildern Feature-Extraction, höchstens der letzte Block darüber; es gibt zu wenig Daten, um viele Schichten zu verändern. Bei 30 000 Bildern lohnt Fine-Tuning mehrerer oberer Schichten, zumal Röntgenbilder sich von Alltagsfotos unterscheiden. Die Übertragbarkeit sinkt mit dem Abstand der Aufgaben, und die oberen Schichten sind aufgabenspezifisch [28].

T4 Aufgabe auf Seite 198

(a) Effektive Batch-Größe $8 \cdot 64 = 512$. Die lineare Skalierungsregel erhöht die Lernrate im selben Verhältnis: $0{,}1 \cdot 8 = 0{,}8$, in der Praxis mit einem Warm-up in den ersten Epochen [31]. **(b)** Der Gradient hat $25 \cdot 10^6 \cdot 4$ Byte = 100 MB. Ring-All-Reduce sendet je Worker $2(N-1)/N \cdot S = 1{,}75 \cdot 100$

MB = 175 MB [49]. **(c)** Bei 12,5 GB/s dauert das 14 ms. Die Effizienz ist $0{,}30\ /(0{,}30 + 0{,}014) = 0{,}955$, die Beschleunigung $8 \cdot 0{,}955 = 7{,}64$. Bei 0,125 GB/s dauert es 1,4 s, die Effizienz sinkt auf 0,176 und die Beschleunigung auf 1,41. Mehr Worker helfen nur, solange die Kommunikation gegenüber der Rechnung klein bleibt; bei langsamem Netz zählt die Bandbreite, nicht die Zahl der Geräte.

T5 Aufgabe auf Seite 199
(a) $1 - 0{,}95^{60} = 0{,}954$. **(b)** $n \geq \ln 0{,}01\ / \ln 0{,}95 = 89{,}8$, also 90 Läufe. **(c)** Das Raster hat $4^3 = 64$ Läufe, probiert aber nur 4 verschiedene Werte des wichtigen Parameters; 64 Zufallsläufe probieren 64 verschiedene Werte. Genau dieses Argument führen Bergstra und Bengio an [51].

T6 Aufgabe auf Seite 199

```
def bewerte(konfig, X, y):
    return DecisionTreeClassifier(**konfig).fit(X_tr, y_tr).score(X, y)


wahl_test = np.argmax(test_acc)  # Auswahl auf den Testdaten (falsch)
wahl_cv = np.argmax(cv_acc)      # Auswahl per Kreuzvalidierung auf den Trainingsdaten
beste_fresh = bewerte(konfigs[wahl_test], X_neu, y_neu)
cv_fresh = bewerte(konfigs[wahl_cv], X_te, y_te)
```

(a) Auf den Testdaten sieht der Sieger mit 0,63 gut aus; auf 5 000 frischen Beispielen erreicht er 0,510, also Raten. **(b)** Der Mittelwert von 100 Zufallsergebnissen auf 100 Beispielen ist 0,5, die Standardabweichung 0,05, das erwartete Maximum von 100 solchen Werten liegt bei 0,625. Eine Auswahl auf den Testdaten findet das Maximum der Streuung. **(c)** Auswahl per Kreuzvalidierung auf den Trainingsdaten, die Testdaten erst danach und einmal (0,52). Auch der Kreuzvalidierungswert des Siegers (0,64) ist optimistisch, da er das Maximum über 100 Konfigurationen ist; die ehrliche Zahl liefert nur der unberührte Test.

T7 Aufgabe auf Seite 199
`best` wird in den Epochen 1 bis 7 und in Epoche 10 geschrieben. Epoche 8 (0,399) und 9 (0,401) bessern den Bestwert 0,398 nicht um 0,001; in Epoche 10 (0,396) ist die Verbesserung 0,002, der Zähler beginnt neu. Die Epochen 11, 12 und 13 bringen keine Besserung, nach der dritten (Epoche 13) stoppt das Training. Behalten wird Epoche 10 mit 0,396. Das Skript `03_training.py` spielt den Ablauf Epoche für Epoche durch.

Resilient Serving

S1 Aufgabe auf Seite 199
(a) Verletzt ist Stateless Serving Function: Die Funktion soll ihr Ergebnis nur aus den Argumenten berechnen. Jede Instanz sieht nur den Teil der Anfragen, den der Load Balancer ihr zuteilt, und zählt systematisch zu wenig (bei vier Instanzen im Mittel ein Viertel); ein Neustart setzt den Zähler auf null. Im Training stand der volle Zähler im Merkmal, im Betrieb steht ein anderer: Skew. **(b)** Der Zustand gehört aus dem Service heraus. Entweder berechnet eine Stream-Verarbeitung das Fenster und übergibt das Ergebnis als Eingabe (Windowed Inference, Pattern 24), oder der Service holt den Wert aus einem Feature Store (Pattern 26). Der Service bleibt zustandslos und kann beliebig skaliert werden. Sticky Sessions wären ein Notbehelf, der Skalierung und Ausfallsicherheit untergräbt.

S2 Aufgabe auf Seite 199
(a) Einzelaufrufe: $200\,000 \cdot (4 + 0{,}01)$ ms = 802 s, rund 13 Minuten. Vier Batches: $4 \cdot 4$ ms $+ 200\,000 \cdot 0{,}01$ ms = 2,02 s, rund 400-mal schneller (Annahmen der Aufgabe, die Größenordnung ist typisch, der Wert nicht). **(b)** Online bleibt man, wenn die Eingabe erst im Moment der Anfrage entsteht oder die Antwort sofort gebraucht wird. Ein Batch-Ergebnis altert, wenn sich die Eingaben danach ändern.

S3 Aufgabe auf Seite 199
(a) $0{,}82 \cdot 0{,}99 + 0{,}18 \cdot 0{,}97 = 0{,}9864$. Ohne die zweite Phase wäre es $0{,}82 \cdot 0{,}99 + 0{,}18 \cdot 0{,}71 = 0{,}940$. **(b)** Der Cloud-Anteil ist 18 %. Datenvolumen: 200 GB sinken auf 36 GB am Tag; Kosten: 400 Euro auf 72 Euro. Die mittlere Latenz beträgt $20 + 0{,}18 \cdot 250 = 65$ ms (im schlechtesten Fall 270 ms). **(c)** Dass

„sicher“ auch „richtig“ bedeutet. Neuronale Netze sind oft überzuversichtlich [22]; ist die Sicherheit nicht kalibriert, bleiben falsche Antworten auf dem Gerät, weil das kleine Modell sich irrt, ohne es zu merken. Die Schwelle ist deshalb auf Validierungsdaten zu wählen, die Güte der Sicherheitsangabe zu prüfen.

S4 Aufgabe auf Seite 200

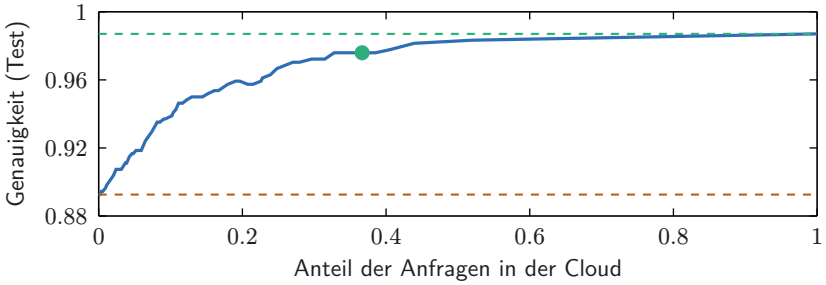


Abbildung 129: Genauigkeit gegen Cloud-Anteil auf den Testdaten. Grün gestrichelt: großes Modell allein (0,987), orange: kleines allein (0,893); Punkt: Schwelle 0,95.

```
ziel = acc_c_val - 0.01
passend = [t for t in taus if lauf(val, t)[1] >= ziel]
tau_gewaehlt = min(passend, key=lambda t: lauf(val, t)[0])
```

Das kleine Modell erreicht 0,893, das große 0,987. Die Schwelle wird auf den Validierungsdaten gewählt: $\tau = 0{,}95$ liefert dort bei 34,3 % Cloud-Anteil 0,9805. Einmal auf den Testdaten angewendet, sind es 36,7 % Cloud-Anteil und 0,9759, also 1,1 Prozentpunkte unter dem großen Modell (Abbildung 129). Das Ziel von einem Punkt wird knapp verfehlt: Die Schwelle ist auf 359 Bildern gewählt und streut. Für die Praxis heißt das, Reserve einzuplanen und die Schwelle im Betrieb zu überwachen (Pattern 18).

S5 Aufgabe auf Seite 200

```
# Reihenfolge angenommen
falsch = df.assign(vorhersage=alle["vorhersage"].values)
# Join über den Schlüssel
richtig = df.merge(alle, on="bestell_id", validate="one_to_one")
```

(a) Mit angenommener Reihenfolge ist der mittlere Fehler 13,5 (jede der zwölf Zuordnungen ist falsch, ohne Fehlermeldung), mit dem Join über die Bestell-ID null. **(b)** Ein doppelt angekommenes Stück erzeugt doppelte Schlüssel; `validate="one_to_one"` lässt den Join mit einem Fehler scheitern, statt Zeilen stillschweigend zu vervielfachen. Üblich ist, Wiederholungen vor dem Join über den Schlüssel zu entdoppeln. **(c)** Eindeutig, stabil über Wiederholungen (also nicht der Zeilenindex im Stück), vor dem Modell vergeben und nicht als Merkmal verwendet; sonst kann ein in der ID kodiertes Wissen (Reihenfolge, Zeit) ins Modell gelangen.

S6 Aufgabe auf Seite 200

(a) Die Beiträge je Klasse sind $0{,}0693$; $0{,}0091$; 0; $0{,}0144$; $0{,}0693$ (Beispiel Klasse 1: $(0{,}10 - 0{,}20)\ln(0{,}10\ /\ 0{,}20) = 0{,}0693$). Die Summe ist $\mathrm{PSI} = 0{,}162$. **(b)** Verbreitet ist die Faustregel aus dem Kreditscoring: unter 0,1 stabil, 0,1 bis 0,25 mäßige, darüber deutliche Verschiebung; eine Primärquelle dafür nenne ich nicht, die Grenzen sind Konvention. Hier liegt der Wert im mittleren Bereich: beobachten, nicht sofort handeln. **(c)** Der PSI sieht nur die Eingaben (Covariate Shift) [12]. Ob die Genauigkeit sinkt, zeigt er nicht, und ein Concept Drift, bei dem sich der Zusammenhang zwischen Eingabe und Ergebnis ändert, die Eingaben aber gleich bleiben, bleibt unsichtbar.

S7 Aufgabe auf Seite 200

(a) $\sqrt{p_0(1-p_0)} = 0{,}300$ und $\sqrt{p_1(1-p_1)} = 0{,}336$, also $n = \left(\frac{1{,}96 \cdot 0{,}300 + 0{,}84 \cdot 0{,}336}{0{,}03}\right)^2 = \left(\frac{0{,}8704}{0{,}03}\right)^2 \approx 842$. Mit den genauen Quantilen (1,960 und 0,8416) sind es 843. **(b)** Bei 100 Fällen am Tag dauert es 8,4 Tage, bis die Zahl erreicht ist; da die Labels erst nach 30 Tagen eintreffen, erscheint der Alarm frühestens nach rund 38 Tagen. Bei 100 Fällen ist das 95-%-Intervall $\pm\,5{,}9$ Punkte breit, ein Rückgang um drei

Punkte ist darin nicht zu erkennen. **(c)** Über Größen, die sofort vorliegen: die Verteilung der Eingaben (PSI, siehe S6), die Verteilung der Vorhersagen und der Anteil unsicherer Vorhersagen. Sie ersetzen die Genauigkeit nicht, geben aber früher einen Anlass nachzusehen.

Reproducibility

R1 Aufgabe auf Seite 200

(a) Das Training rechnet $(62-40)/10 = 2{,}2$; $1{,}8$; $2{,}6$. Der Service rechnet mit Mittelwert 62 und Standardabweichung 3,27 der Anfrage und erhält 0; $-1{,}22$; $1{,}22$. Das Modell hält drei Senioren für Durchschnittskunden. **(b)** Bei einem Kunden ist die Standardabweichung null, die Division scheitert oder liefert `NaN`. **(c)** Training-Serving Skew. Das Pattern Transform verlangt, dass die Konstanten (40 und 10) im Training aus den Trainingsdaten bestimmt, mit dem Modell gespeichert und im Betrieb dieselben sind, am besten als Teil einer Pipeline. Wer den Skalierer vor dem Split auf allen Daten anpasst, begeht Data Leakage: Information aus den Testdaten fließt ins Training.

R2 Aufgabe auf Seite 201

```python
def ist_training(patient_id: str, anteil: int = 8) -> bool:
    """Stabil über Prozesse und Rechner, anders als hash()."""
    h = int.from_bytes(hashlib.sha256(patient_id.encode()).digest()[:8], "big")
    return h % 10 < anteil
```

(a) Der Trainingsanteil beträgt 0,795 der Patienten und 0,798 der Zeilen; kein Patient steht auf beiden Seiten; nach 200 neuen Patienten ändert sich keine Zuordnung, weil sie nur von der ID abhängt. Bei einem zufälligen Split wechseln zwischen zwei Seeds im Mittel 32 % der Patienten die Seite (200 Paare; erwartet $2 \cdot 0{,}8 \cdot 0{,}2 = 0{,}32$). **(b)** Die Zahl der Trainingspatienten ist binomialverteilt mit Standardabweichung $\sqrt{1\,000 \cdot 0{,}8 \cdot 0{,}2} = 12{,}6$, das sind 1,3 Prozentpunkte; 0,795 ist also unauffällig. **(c)** `hash()` ist bei Zeichenketten pro Prozess zufällig gesalzen; derselbe Patient landet in einem anderen Prozess auf der anderen Seite.

R3 Aufgabe auf Seite 201

(a) Zufällig nach 70 zu 30 zugewiesen stimmt ein Wert mit der Wahrscheinlichkeit $0{,}7^2 + 0{,}3^2 = 0{,}58$, 42 % sind falsch. **(b)** Immer „Kreditkarte“ stimmt zu 70 %. Die „ehrliche“ Variante ist die statische Methode: Statt einer Kartenart bekommen die alten Zeilen die Mischung (0,7 Kredit, 0,3 Debit), die Unsicherheit steht dann in der Eingabe selbst. Das Modell sieht aber im Betrieb nie solche Mischwerte. **(c)** Waren die Anteile früher 50 zu 50, stimmt die zufällige Zuweisung nur zu $0{,}7 \cdot 0{,}5 + 0{,}3 \cdot 0{,}5 = 0{,}50$, und das Modell lernt aus den alten Zeilen ein falsches Verhältnis. Die Methode setzt gleiche Anteile in alten und neuen Daten voraus; das ist eine Annahme.

R4 Aufgabe auf Seite 201

Terminal

```
$ python 05_reproduzierbarkeit.py
   messwert  batch  stream  zentriert
0      10.0  10.00   10.00      11.00
1      12.0  11.00   11.00      11.00
2      11.0  11.00   11.00      12.67
3      15.0  12.67   12.67      15.33
4      20.0  15.33   15.33      17.67
5      18.0  17.67   17.67      18.33
6      17.0  18.33   18.33      17.50
Batch und Stream identisch: True
Zentriertes Fenster weicht ab in 6 von 7 Zeilen (nutzt die Zukunft)
```

(a) Batch und Stream liefern dieselben Werte (`np.allclose` bestätigt es). Entscheidend ist, dass die Fensterlogik nur einmal definiert ist und beide Wege sie verwenden. **(b)** Das zentrierte Fenster nimmt auch den nächsten Wert hinzu; in 6 von 7 Zeilen unterscheidet es sich vom Stream. Im Training sähe das Modell damit die Zukunft, die es im Betrieb nie hat. **(c)** Ob man mit den vorhandenen ein bis

zwei Werten mittelt (`min_periods=1`) oder die Zeilen verwirft, ist eine Entscheidung, die in Training und Betrieb gleich sein muss.

R5 Aufgabe auf Seite 201

```
richtig = pd.merge_asof(ereignisse, merkmale, left_on="zeit", right_on="gueltig_ab", by="kunde")
```

```
Terminal
$ python 05_reproduzierbarkeit.py
 kunde       zeit  zeitpunktrichtig  neuester_wert
     1 2026-03-12               5.0              1
     1 2026-03-25               1.0              1
     2 2026-03-03               NaN              3
     2 2026-03-15               7.0              3
```

Der zeitpunktrichtige Join (`merge_asof` mit `by="kunde"`) gibt jedem Ereignis den zuletzt gültigen Wert: 5, 1, kein Wert (Kunde 2 hatte am 3.3. noch keinen) und 7. Der Join „neuester Wert je Kunde" liefert 1, 1, 3 und 3 und ist bei drei von vier Ereignissen falsch, weil er Werte aus der Zukunft des Ereignisses verwendet. Das ist Leckage, die im Training gut aussieht und im Betrieb nicht reproduzierbar ist.

R6 Aufgabe auf Seite 201

(a) Eine Ordnung: `ingest`, `clean`, `featurize`, `split`, `train`, `evaluate`, `register`. `featurize` und `split` hängen nur von `clean` ab und können parallel laufen. **(b)** Ändert sich `train`: `train`, `evaluate`, `register` (3 von 7). Ändert sich `split`: `split`, `train`, `evaluate`, `register` (4 von 7), `featurize` bleibt im Cache (Abbildung 130). Ändert sich `clean`: alles außer `ingest` (6 von 7). Der Fingerabdruck eines Schritts enthält Code, Parameter und die Fingerabdrücke der Vorgänger, deshalb erbt jeder Schritt die Änderungen davor. **(c)** Ein Zyklus hätte keine Ausführungsreihenfolge: Jeder Schritt in ihm wartet auf einen anderen aus dem Zyklus.

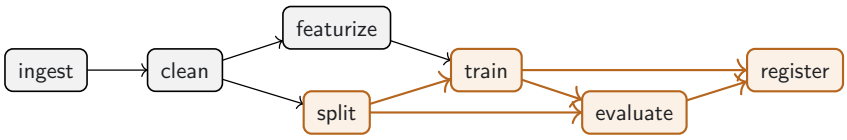


Abbildung 130: Der Graph aus R6. Orange: Schritte, die neu laufen, wenn sich `split` ändert; `ingest`, `clean` und `featurize` kommen aus dem Cache.

R7 Aufgabe auf Seite 201

(a) Ein neues Pflichtfeld ist eine inkompatible Änderung: Die drei Clients erhalten Fehler, sobald sie auf v2 umgestellt werden. Das nachgelagerte Modell erhält Vektoren aus einem anderen Raum. **(b)** v2 läuft unter eigenem Endpunkt neben v1 (`/v1/…`, `/v2/…`); die Clients wechseln, wenn sie bereit sind, v1 wird erst abgeschaltet, wenn niemand mehr zugreift. Alternativ macht man das Feld optional mit einem Standardwert, dann ist die Änderung kompatibel. In der Registry stehen Version, Eingabestruktur, Trainingsstand und Aliase wie `production` und `canary`. **(c)** Ein neu trainiertes Embedding legt die Achsen neu fest. Die Vektoren von v1 und v2 sind nicht vergleichbar, und ein Modell, das auf einem Raum trainiert wurde, versteht den anderen nicht. Das nachgelagerte Modell muss mit der Version des Embeddings gekoppelt und versioniert werden.

Responsible AI

V1 Aufgabe auf Seite 202

```
def lieferdaten(n=6000, seed=0):
    r = np.random.default_rng(seed)
    km = r.uniform(0.5, 10, n)
    artikel = r.integers(1, 9, n)
    regen = r.integers(0, 2, n)
    dauer = 12 + 2.6 * km + 0.5 * artikel + 6 * regen + r.normal(0, 2.5, n)
    return np.column_stack([km, artikel, regen]), dauer
```

```
Terminal
$ python 06_verantwortung.py
Mittelwert der Trainingsdaten    6.84
Faustregel 10 + 3 * km           5.47
Lineare Regression               2.00
Gradient Boosting                2.02
Linear gegen Faustregel: 63.5% weniger Fehler
Boosting gegen linear:   -1.4% weniger Fehler
Rauschen der Daten (Standardabweichung 2,5): erwarteter MAE im Optimum 1.99
```

(a) Die Werte stehen oben. **(b)** Der Maßstab ist die Faustregel mit 5,47 Minuten, nicht der Mittelwert mit 6,84: Er ist die stärkere und zugleich erklärbare Alternative. Die lineare Regression verringert den Fehler gegenüber der Faustregel um 63,5 %. Gradient Boosting bringt nichts (2,02 gegen 2,00 Minuten, minimal schlechter). Das liegt auch an den Daten, die das Skript linear erzeugt; bei Zusammenhängen mit Schwellen und Wechselwirkungen sähe es anders aus. Das Rauschen der Daten legt die Untergrenze des mittleren absoluten Fehlers bei $2{,}5 \cdot \sqrt{2/\pi} = 1{,}99$ fest: Die lineare Regression ist praktisch am Optimum, es gibt keine Reserve, die ein komplexeres Modell ausschöpfen könnte. Man nimmt das einfachere.

V2 Aufgabe auf Seite 202

(a) Die Vorhersage der Baseline ist $8 + 10 + 1{,}8 + 1{,}5 = 21{,}3$ Minuten, die des Falls $8 + 22{,}5 + 3 + 5 = 38{,}5$. Die Beiträge sind für die Entfernung $2{,}5 \cdot (9 - 4) = 12{,}5$, für die Artikel $0{,}6 \cdot (5 - 3) = 1{,}2$ und für den Regen $5 \cdot (1 - 0{,}3) = 3{,}5$. Die Summe 17,2 ist genau die Differenz $38{,}5 - 21{,}3$. Für ein lineares Modell sind das die exakten Shapley-Werte, das Skript prüft es durch Aufzählen aller Koalitionen. **(b)** Bei der Baseline „trockener Tag“ ist die Differenz 18,7 und der Regen trägt 5,0 statt 3,5; die anderen Beiträge bleiben. Eine Erklärung ist immer eine Aussage „gegenüber dieser Baseline“, und die Baseline gehört mit der Erklärung angegeben.

V3 Aufgabe auf Seite 202

(a) Die Erklärung beschreibt, was das Modell tut, nicht, was in der Welt wirkt. Die Postleitzahl ist vermutlich ein Stellvertreter für Merkmale, die nicht im Modell stehen (Einkommensstruktur, Wohnsituation); ein Umzug ändert deren Ursachen nicht. **(b)** Korrelierte Merkmale teilen sich Beiträge oder verdrängen einander, und das Ergebnis hängt von der Baseline und vom Verfahren ab. Dazu können Erklärungsverfahren versagen: Saliency-Verfahren für Bilder liefern teils Karten, die kaum vom Modell abhängen [69]. **(c)** Der Kunde sollte erfahren, welche seiner Angaben die Entscheidung bestimmt haben und welche davon er ändern kann. Dass die Postleitzahl den größten Beitrag hat, ist ein Anlass für die Fairness Lens (Pattern 30): Wirkt sie als Proxy für ein geschütztes Merkmal?

V4 Aufgabe auf Seite 202

(a) Gruppe A: Genauigkeit 0,86, Auswahlrate 0,38, TPR 0,80, FPR 0,10, Precision 0,842. Gruppe B: Genauigkeit 0,84, Auswahlrate 0,20, TPR 0,60, FPR 0,10, Precision 0,60. Gesamt: $(860 + 420)/\,1\,500 = 0{,}853$. **(b)** Verletzt: Gleichheit der Chancen (Equal Opportunity [74]): Von den Kreditwürdigen in B werden 60 %, in A 80 % angenommen, die Differenz beträgt 0,20. Verletzt ist auch die statistische Parität: Das Verhältnis der Auswahlraten ist $0{,}20\ /\ 0{,}38 = 0{,}526$, unter der Vier-Fünftel-Faustregel aus dem US-Arbeitsrecht (0,8). Erfüllt ist die Gleichheit der False-Positive-Rate (beide 0,10), die Genauigkeit unterscheidet sich kaum. Die Gesamtgenauigkeit verdeckt das Problem. B hat zwar die niedrigere Basisrate (20 % gegen 40 %), was die niedrigere Auswahlrate zum Teil erklärt, nicht aber die niedrigere TPR.

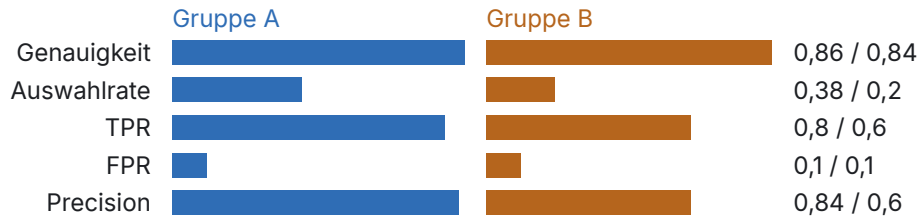


Abbildung 131: Kennzahlen der beiden Gruppen aus V4 (Balkenlänge entspricht dem Wert, Skala 0 bis 1). Die Unterschiede liegen bei Auswahlrate, TPR und Precision.

V5 Aufgabe auf Seite 202

(a) Precision Gruppe A: $(0{,}4 \cdot 0{,}8)/(0{,}4 \cdot 0{,}8 + 0{,}6 \cdot 0{,}1) = 0{,}32 \;/\; 0{,}38 = 0{,}842$. Gruppe B: $0{,}16\;/(0{,}16 + 0{,}8 \cdot 0{,}1) = 0{,}16 \;/\; 0{,}24 = 0{,}667$. Die Auswahlraten sind 0,38 und 0,24. **(b)** Soll B die Precision 0,842 haben, muss die FPR (aus der Formel der Precision nach der FPR aufgelöst) $\mathrm{TPR} \cdot \mathrm{Basisrate} \cdot (1 - \mathrm{PPV})/(\mathrm{PPV} \cdot (1 - \mathrm{Basisrate})) = 0{,}8 \cdot 0{,}2 \cdot (1 - 0{,}842)/(0{,}842 \cdot 0{,}8) = 0{,}0375$ betragen. Dann ist die FPR in den Gruppen verschieden, und die Gleichheit der Chancen fehlt. Bei verschiedenen Basisraten lassen sich gleiche Fehlerraten und gleiche Precision nicht zugleich erreichen, außer bei einer fehlerfreien Vorhersage; das zeigen Kleinberg et al. und Chouldechova [75], [76]. Welches Kriterium gilt, ist eine fachliche und rechtliche Entscheidung, keine technische.

V6 Aufgabe auf Seite 202

```
r = np.random.default_rng(5)
n = 40_000
gruppe = r.integers(0, 2, n)
koennen = r.normal(size=n)
proxy = gruppe + r.normal(0, 0.5, n)
eingestellt = (koennen - 0.9 * gruppe + r.normal(0, 0.5, n) > 0.3).astype(int)  # verzerrte
Vergangenheit
```

```
tr, te = train_test_split(np.arange(n), test_size=0.5, random_state=0)
for name, spalten in (("Können + Proxy", [koennen, proxy]), ("nur Können", [koennen])):
    M = np.column_stack(spalten)
    p = LogisticRegression().fit(M[tr], eingestellt[tr]).predict_proba(M[te])[:, 1]
    schwelle = np.quantile(p, 0.7)  # die besten 30 % werden ausgewählt
    sel = p >= schwelle
    g = gruppe[te]
    print(f"{name:<16} Auswahlrate Gruppe 0: {sel[g == 0].mean():.3f}, Gruppe 1: {sel[g ==
1].mean():.3f}")
```

Terminal

```
$ python 06_verantwortung.py
Historische Einstellquote: Gruppe 0 0.394, Gruppe 1 0.139
Korrelation Proxy und Gruppe: 0.71
Können + Proxy   Auswahlrate Gruppe 0: 0.381, Gruppe 1: 0.218
nur Können       Auswahlrate Gruppe 0: 0.302, Gruppe 1: 0.298
```

Historisch wurden Personen der Gruppe 1 mit 13,9 % deutlich seltener eingestellt als die der Gruppe 0 (39,4 %). Das Merkmal „Gruppe“ ist nicht im Modell, aber der Proxy korreliert mit 0,71 mit ihr. Mit Können und Proxy wählt das Modell 38,1 % der Gruppe 0 und 21,8 % der Gruppe 1 aus: Es hat die Verzerrung der Vergangenheit über den Proxy gelernt. Nur mit dem Können sind es 30,2 % und 29,8 %. Das Entfernen des geschützten Merkmals reicht nicht, solange Stellvertreter bleiben („Fairness through unawareness“ versagt); die Fairness Lens prüft die Auswahl je Gruppe, nicht die Merkmalsliste. Dass das reine Können-Modell gleiche Raten liefert, liegt an der Konstruktion der Daten, in denen das Können unabhängig von der Gruppe ist.

Szenarien

K1 Aufgabe auf Seite 203

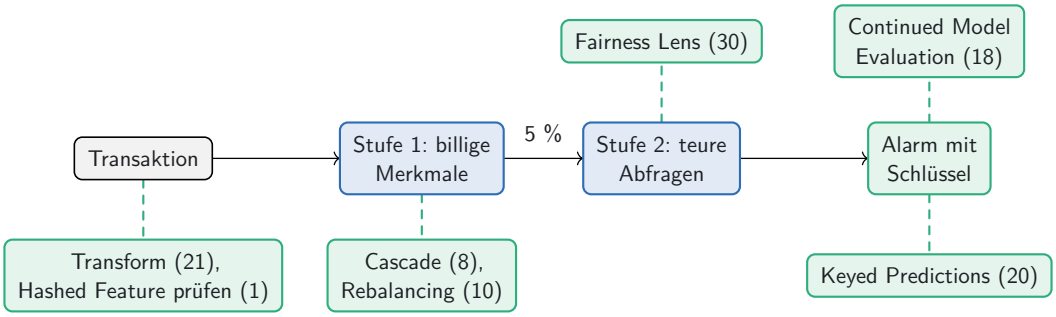


Abbildung 132: Zielarchitektur für K1. Grün: Patterns an den Stellen, an denen sie wirken.

(a) Precision und Recall (und daraus Kosten) statt Genauigkeit; die Regel „nie Betrug“ hat 99,9 %, das Modell mit 8 % Recall findet nur 3 200 von 40 000 Fällen. Geeignet sind Gewichte oder Downsampling der Nicht-Betrugsfälle im Training und eine Schwelle nach Kosten; Validierungs- und Testdaten bleiben unverändert, Wahrscheinlichkeiten werden nachkalibriert, wenn sie als Zahl verwendet werden (vgl. P7). **(b)** Stufe 1 nutzt die billigen Merkmale und lässt 5 % durch. Aus $r_1 \cdot 0{,}90 \geq 0{,}85$ folgt $r_1 \geq 0{,}944$. Die teuren Abfragen kosten dann $2\,000\,000 \cdot 0{,}05$ Cent = 1 000 Euro statt 20 000 Euro am Tag. Mit $r_1 = 0{,}95$ und einer Fehlalarmrate von 0,1 % in Stufe 2 (Annahme) sind es 34 200 Treffer und rund 1 962 Fehlalarme: Recall 0,855, Precision 0,946, 36 162 Alarme am Tag. **(c)** Keyed Predictions: Jede Transaktion trägt ihren Schlüssel durch den Job, die Zuordnung geschieht über einen Join; Reihenfolge und Wiederholungen spielen dann keine Rolle. **(d)** Continued Model Evaluation: Eingaben, Vorhersageverteilung und Alarmrate sind sofort verfügbar. Den Recall misst man auf Kohorten, deren Labels vollständig sind (Transaktionen älter als 60 Tage), mit einer Stichprobengröße wie in S7. Weil die Betrüger ihre Muster wechseln, gehört ein regelmäßiges Neutrainieren dazu (Workflow Pipeline, Pattern 25). **(e)** Fairness Lens: False-Positive-Rate je Gruppe (0,6 % gegen 0,1 %, Faktor 6). Beim Hashed Feature kollidieren seltene Herkunftsländer mit häufigen und werden von diesen überstimmt: Kleine Gruppen werden unsichtbar.

K2 Aufgabe auf Seite 203

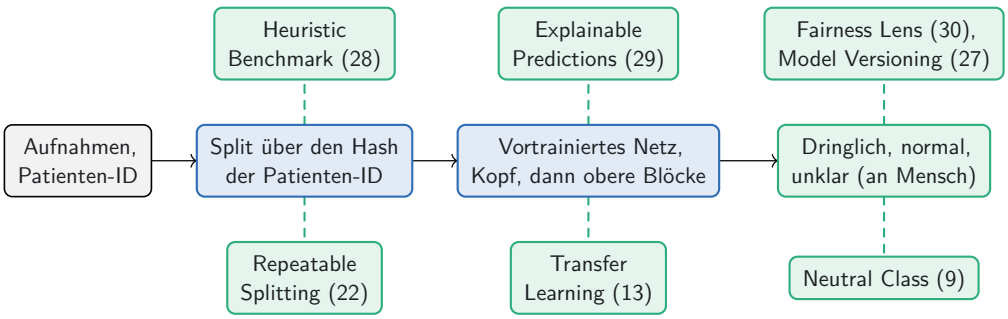


Abbildung 133: Zielarchitektur für K2.

(a) Bei einer zufälligen Aufteilung steht ein Patient mit vier Aufnahmen mit $1 - 0{,}8^4 - 0{,}2^4 = 0{,}589$ auf beiden Seiten. Das Netz erkennt Patienten wieder (Anatomie, Implantate, Geräteeinstellungen), die 97 % messen teilweise das Gedächtnis, und im Pilot fällt die Genauigkeit auf 81 %. Richtig ist der Split über einen Hash der Patienten-ID (Repeatable Splitting): Der Test hat dann rund 640 Patienten und rund 2 370 Aufnahmen, und die Zuordnung bleibt stabil, wenn Daten dazukommen. Der Pilot am zweiten Haus ist ohnehin der ehrlichste Test. **(b)** Bei rund 9 500 Aufnahmen und einem Quellbereich, der den Röntgenbildern nicht ähnelt, beginnt man mit Feature-Extraction und gibt dann obere Blöcke mit kleiner Lernrate frei (Transfer Learning). Die Uneinigkeit betrifft $0{,}22 \cdot 11\,840 = 2\,605$ Aufnahmen. Sie werden eine eigene Klasse „unklar“ (Neutral Class), die im Betrieb an einen Radiologen geht, statt eine Pseudo-Entscheidung zu erzwingen. **(c)** Die Heatmap zeigt eine Abkürzung (Shortcut Learning [114]): Die Station hängt mit der Dringlichkeit zusammen, und das Modell nutzt die Einblendung statt

der Lunge. Mit Explainable Predictions findet man das, danach schneidet man die Ecke ab oder maskiert sie und prüft am zweiten Haus. **(d)** Vor das Netz gehört ein Heuristic Benchmark, etwa eine Regel nach Alter und Aufnahmegrund; das Netz muss sie klar schlagen. Jede Vorhersage und Auswertung trägt die Modellversion; Model Versioning betreibt neue Stände neben alten, statt auszutauschen. **(e)** Fairness Lens: Recall dringender Fälle je Altersgruppe. Die Gruppe über 80 ist klein, deshalb Konfidenzintervalle angeben, sonst liest man Zufall als Befund.

K3 Aufgabe auf Seite 204

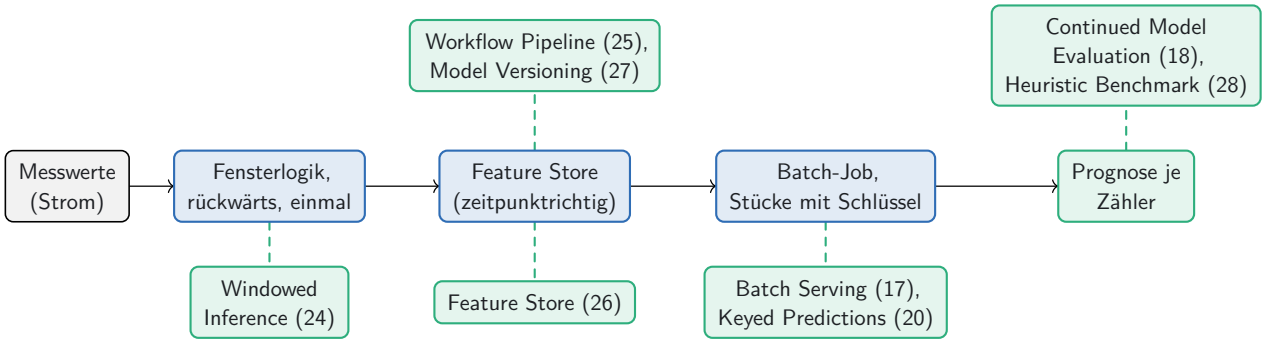


Abbildung 134: Zielarchitektur für K3.

(a) Erstens nutzt das zentrierte Fenster Werte aus der Zukunft des Prognosezeitpunkts, die es im Betrieb nicht gibt (Leckage). Zweitens rechnet jede Instanz das Merkmal an ihrem Ort mit ihrem Zustand; das verletzt Stateless Serving und erzeugt Skew. Windowed Inference definiert das Fenster einmal, rückwärts, und lässt es im Training auf historischen Daten und im Betrieb auf dem Strom laufen. Der Feature Store hält das Merkmal zeitpunktrichtig vor. **(b)** Hält man alle 96 Werte je Zähler, sind es $1\ 500\ 000 \cdot 96 \cdot 8$ Byte = 1,15 GB. Bei festen Fenstern genügen Summe und Anzahl (zwei Zahlen je Zähler), zusammen 24 MB; bei gleitenden Fenstern braucht man die Werte, um alte wieder herauszurechnen. **(c)** $144 \cdot 10^6 \cdot 4$ ms sind 160 Stunden; laufen die Aufrufe nacheinander, liegt die Prognose für den nächsten Tag erst nach knapp sieben Tagen vor. Batch Serving bewertet die Messwerte in großen Stücken, jedes Stück läuft mit Schlüssel (Zähler-ID und Zeitpunkt) durch das Modell, Keyed Predictions ordnet die Ergebnisse zu. **(d)** Eine Workflow Pipeline mit Fingerabdrücken je Schritt legt fest, welche Daten und welcher Code zu welchem Artefakt gehören; Model Versioning führt Modelle mit Datenstand und Metriken in einer Registry. **(e)** Eingaben (Temperatur, Verbrauchsverteilung) lassen sich sofort überwachen, der Fehler gegen die tatsächlichen Messwerte liegt einen Tag später vor. Als Maßstab dient ein Heuristic Benchmark wie „derselbe Wert wie vor einer Woche“: Sobald das Modell nicht mehr klar besser ist, ist es Zeit zu handeln, und die Fehlerquote beider Verfahren gemeinsam im Zeitverlauf zeigt, ob die Hitzewelle das Modell oder die Welt verändert hat.

16.10 Das Wichtigste in Kürze

Nicht jede der 40 Aufgaben fragt nach einem Fehler, aber drei Fehlerformen kehren in allen Kategorien wieder. Die erste ist eine Kennzahl, die die Entscheidung nicht abbildet: Genauigkeit bei seltenen Ereignissen (P4, P5), F1 statt Kosten, ein Sieger, der auf den Testdaten ausgewählt wurde (T6), eine Gesamtgenauigkeit, die Gruppenunterschiede verdeckt (V4). Die zweite ist Information, die im Training anders ist als im Betrieb: ein anderer Skalierer (R1), ein Fenster, das in die Zukunft schaut (R4), ein Feature-Wert zum falschen Zeitpunkt (R5), eine Reihenfolge, die sich beim Verteilen ändert (S5). Die dritte ist eine Annahme, die niemand nachgerechnet hat: Kollisionen (D2), Stufen einer Cascade (P4), die Stichprobe für einen Alarm (S7), die Wahrscheinlichkeit nach Undersampling (P7). Alle drei lassen sich vor dem Einsatz mit einer Rechnung oder einem kleinen Experiment finden, wie die Aufgaben vorführen.

Für die Lehre eignet sich als Übungsblatt für 90 Minuten die Auswahl D2, P5, S3, R2 und V4 mit K1 als Abschlussszenario; die Diagnoseaufgaben (D1, D6, S1, R7) eignen sich als Quizfragen. Die Code-Aufgaben liefern fertige Skripte, an denen sich ein Ergebnis in wenigen Minuten nachvollziehen lässt.

16.11 Weiterführende Literatur

Die Aufgaben folgen dem Buch von V. Lakshmanan, S. Robinson, und M. Munn [1]. Für die Metriken bei unbalancierten Klassen ist der Überblick von H. He und E. A. Garcia [46] hilfreich, für das Aufteilen von Daten mit Struktur die Arbeit von D. R. Roberts *u. a.* [57], für Drift die Übersicht von J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, und A. Bouchachia [12]. Für Fairness-Kriterien und ihre Grenzen ist das Lehrbuch von S. Barocas, M. Hardt, und A. Narayanan [79] die ausführlichste Quelle, für Prüfungen vor dem Betrieb der ML Test Score von E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley [55].

A Spickzettel

Diese Übersicht ist zum Nachschlagen gedacht, etwa kurz vor einer Vorlesung oder einem Coaching-Termin. Jede Zeile verdichtet ein Pattern auf das Problem, die Lösung und den Preis, den man dafür zahlt. Die Nummern entsprechen der Zählung bei [1] und den Kapiteln dieses Handbuchs. Wer ein Pattern nur anhand einer Zeile erklären will, sollte vorher das zugehörige Kapitel lesen: Die Kürze hier geht auf Kosten der Mechanik.

Nr.	Pattern	Problem	Lösung	Typischer Trade-off
Data Representation Kapitel 5				
1	**Hashed Feature**	Kategorisches Feature mit sehr vielen oder unbekannten Werten; One-Hot wird riesig, neue Werte brechen das Vokabular.	Wert mit einer deterministischen Hash-Funktion auf eine feste Zahl von Buckets abbilden.	Kollisionen verschiedener Werte; keine Rückübersetzung; Bucket-Zahl ist eine Wahl.
2	**Embeddings**	Kategorien oder Wörter als One-Hot sind riesig, dünn besetzt und kennen keine Ähnlichkeit.	Werte auf dichte, niedrigdimensionale Vektoren abbilden, die so gelernt werden, dass Ähnliches nah beieinander liegt.	Braucht Daten und Training; Dimension ist eine Wahl; Vektoren sind schwer zu deuten.
3	**Feature Cross**	Die Wirkung hängt von der Kombination zweier Features; ein Modell, das Features nur einzeln gewichtet, sieht sie nicht.	Ein zusätzliches Feature aus der Kombination bilden (Kreuzprodukt der Kategorien, ggf. gebündelt).	Dimension wächst multiplikativ; viele leere Zellen; bei Bäumen und Netzen oft unnötig.
4	**Multimodal Input**	Die Eingabe besteht aus verschiedenen Datenarten (Text, Bild, Tabelle), das Modell erwartet einen Vektor.	Jede Modalität in eine numerische Darstellung überführen und zu einer gemeinsamen Eingabe zusammenfügen.	Mehrere Vorverarbeitungspfade; unterschiedliche Skalen; was tun, wenn eine Modalität fehlt?
Problem Representation Kapitel 6				
5	**Reframing**	Eine Regression ist schlecht gestellt, etwa bei mehrgipfeliger Verteilung oder wenn Unsicherheit zählt.	Die Aufgabe umformulieren, typischerweise Regression als Klassifikation über Wertebereiche (Bins).	Auflösung geht durch die Bin-Grenzen verloren; die Ordnung der Klassen muss man sich eigens merken.
6	**Multilabel**	Ein Beispiel gehört zu mehreren Klassen zugleich; Softmax erzwingt genau eine.	Pro Label ein eigener Sigmoid-Ausgang, damit jede Wahrscheinlichkeit unabhängig ist.	Schwellwert pro Label nötig; Abhängigkeiten zwischen Labels bleiben unberücksichtigt.
7	**Ensembles**	Ein einzelnes Modell hat zu hohe Varianz oder zu hohen Bias.	Mehrere Modelle kombinieren: Bagging gegen Varianz, Boosting gegen Bias, Stacking über ein Meta-Modell.	Mehr Rechen- und Wartungsaufwand; schwerer erklärbar; ohne Verschiedenheit der Modelle kein Gewinn.
8	**Cascade**	Die Vorhersage hängt von einer vorgelagerten Fallunterscheidung; ein Modell vermischt gegensätzliche Muster.	Das Problem in eine Folge von Modellen zerlegen, deren Ausgabe das nächste Modell speist oder auswählt.	Fehler pflanzen sich fort; alle Stufen müssen gemeinsam trainiert und versioniert werden.
9	**Neutral Class**	Menschliche Labels sind bei manchen Fällen uneins; ein binäres Modell lernt Scheinunterschiede.	Eine dritte, neutrale Klasse für wirklich uneindeutige Fälle einführen.	Die Klasse muss definiert und gelabelt werden; mehr Klassen brauchen mehr Daten.

Nr.	Pattern	Problem	Lösung	Typischer Trade-off
10	**Rebalancing**	Seltene Klasse (z. B. 1 % Betrug): Das Modell sagt immer „normal“, die Accuracy ist hoch und wertlos.	Passende Metrik (Precision, Recall, PR-AUC), Klassengewichte oder Resampling, nur auf den Trainingsdaten.	Wahrscheinlichkeiten verschieben sich; Overfitting auf duplizierte Fälle; Schwellwert ist eine Fachentscheidung.
Model Training Kapitel 7				
11	**Useful Overfitting**	Manchmal braucht man keine Generalisierung: Der Eingaberaum ist bekannt, die Zielfunktion deterministisch.	Bewusst auf die Trainingsdaten überanpassen, ohne Regularisierung und Early Stopping.	Nur sicher, wenn der Eingaberaum wirklich abgedeckt bleibt; außerhalb unzuverlässig.
12	**Checkpoints**	Langes Training bricht ab, oder man möchte auf einen früheren Zustand zurück.	Zwischenstände (Gewichte, Optimizer-Zustand, Epoche) regelmäßig sichern und von dort fortsetzen.	Speicher und I/O; ein Checkpoint ist keine vollständige Modellversion (siehe Pattern 27).
13	**Transfer Learning**	Zu wenig Daten, um ein Modell von Grund auf zu trainieren.	Ein vortrainiertes Modell einer verwandten Aufgabe nutzen: Features extrahieren oder letzte Schichten nachtrainieren.	Domänenabstand; man erbt die Verzerrungen des Ausgangsmodells.
14	**Distribution Strategy**	Das Training dauert zu lang, oder Modell und Daten passen nicht auf ein Gerät.	Training auf mehrere Geräte verteilen, meist datenparallel mit synchronisierten Gradienten.	Kommunikationsaufwand; effektive Batchgröße steigt, die Lernrate muss mit; mehr Betriebskomplexität.
15	**Hyperparameter Tuning**	Hyperparameter (Lernrate, Baumtiefe …) werden nicht gelernt, falsche Werte kosten Qualität.	Systematisch suchen (Grid, Random, Bayes’sch) und auf Validierungsdaten bewerten, automatisiert.	Viele Trainingsläufe kosten Rechenzeit; Gefahr der Überanpassung an die Validierungsdaten.
Resilient Serving Kapitel 8				
16	**Stateless Serving Function**	Das Modell hängt am Trainingscode oder Notebook und lässt sich nicht skalieren.	Modell samt Vorverarbeitung als zustandslose Funktion exportieren und hinter einem Endpunkt betreiben.	Zustand und Historie müssen außerhalb liegen; Vorverarbeitung muss mit exportiert werden.
17	**Batch Serving**	Sehr viele Vorhersagen, keine Echtzeit nötig; ein Online-Endpunkt wäre überlastet oder zu teuer.	Vorhersagen als Batch-Job parallel über viele Instanzen berechnen und die Ergebnisse speichern.	Ergebnisse sind bis zum nächsten Lauf veraltet; Zuordnung zur Eingabe braucht einen Schlüssel.
18	**Continued Model Evaluation**	Das Modell altert, weil sich Daten und Zusammenhänge ändern (Drift), und niemand merkt es.	Laufend Vorhersagen mit später eintreffenden Labels vergleichen und Schwellwerte für ein Neutraining festlegen.	Labels kommen verspätet oder nie; Monitoring ist Betriebsaufwand.
19	**Two-Phase Predictions**	Ein großes Modell ist für das Gerät zu teuer, zu langsam oder offline nicht verfügbar.	Ein kleines Modell übernimmt lokal eine einfachere Teilaufgabe; das große Modell in der Cloud läuft nur, wenn es nötig ist.	Zwei Modelle pflegen; das kleine ist ungenauer; der Anteil der Cloud-Aufrufe bestimmt die Kosten.
20	**Keyed Predictions**	Bei Batch- und paralleler Verarbeitung kommen Ergebnisse nicht in Eingabereihenfolge zurück.	Jede Instanz trägt einen Schlüssel, der unverändert durch das Modell in die Ausgabe gereicht wird.	Schlüssel muss eindeutig sein und darf nicht als Feature ins Modell geraten.
Reproducibility Kapitel 9				
21	**Transform**	Vorverarbeitung ist im Training und im Serving getrennt implementiert und läuft auseinander (Training-Serving Skew).	Die Transformation als Teil des Modells festhalten, inklusive der im Training gelernten Parameter.	Modell und Transformation sind enger gekoppelt; Änderungen erfordern Neutraining.
22	**Repeatable Splitting**	Zufällige Splits ändern sich mit jeder Datenänderung; verwandte Zeilen landen auf beiden Seiten (Leckage).	Die Zuordnung aus dem Hash einer stabilen Spalte (Gruppe, Zeitpunkt) ableiten.	Spalte sorgfältig wählen; das Verhältnis stimmt nur näherungsweise.

Nr.	Pattern	Problem	Lösung	Typischer Trade-off
23	**Bridged Schema**	Eine Datenquelle ändert ihr Schema; vom neuen Feature gibt es noch zu wenige Daten.	Alte Daten auf das neue Schema überbrücken (Imputation, Zuordnung) und gemeinsam trainieren.	Überbrückte Werte sind Näherungen; die Brücke muss dokumentiert und später entfernt werden.
24	**Windowed Inference**	Die Vorhersage braucht aggregierte Historie (z. B. Mittel der letzten Stunden) aus einem Datenstrom.	Den Zustand außerhalb des Modells in Zeitfenstern aggregieren und exakt wie im Training berechnen.	Streaming-Infrastruktur; Fensterdefinition muss identisch sein; Wartezeit, bis das Fenster gefüllt ist.
25	**Workflow Pipeline**	Der Ablauf ist ein Skript-Monolith; Schritte lassen sich weder einzeln wiederholen noch nachvollziehen.	Schritte als eigenständige, containerisierte Stufen mit definierten Artefakten orchestrieren.	Aufbau- und Betriebsaufwand; für Einmal-Experimente überdimensioniert.
26	**Feature Store**	Features werden pro Team neu berechnet, im Training anders als im Serving.	Zentraler Speicher mit versionierten Feature-Definitionen und konsistentem Zugriff offline und online.	Zusätzliche Infrastruktur und Governance; lohnt erst bei mehreren Modellen oder Teams.
27	**Model Versioning**	Modelländerungen müssen vergleichbar, rückholbar und für Clients abwärtskompatibel ausgerollt werden.	Jede Modellversion mit Daten, Code und Hyperparametern festhalten und als eigene Version bereitstellen.	Mehrere Versionen parallel zu betreiben kostet; Metadaten wollen gepflegt werden.
Responsible AI Kapitel 10				
28	**Heuristic Benchmark**	Eine Kennzahl wie „Fehler 4,2“ sagt allein nichts; Beteiligte können sie nicht einordnen.	Mit einer einfachen Heuristik (Mittelwert, Regel) vergleichen; das Modell muss sie klar schlagen.	Eine gute Heuristik kostet Arbeit; ein zu schwacher Benchmark täuscht Fortschritt vor.
29	**Explainable Predictions**	Nutzer, Prüfer und Entwickler müssen verstehen, warum das Modell so entschieden hat.	Je Vorhersage Feature-Beiträge berechnen (SHAP, LIME, Integrated Gradients) oder interpretierbare Modelle wählen.	Erklärungen beschreiben das Modell, nicht die Wirklichkeit; Rechenkosten; trügerische Sicherheit.
30	**Fairness Lens**	Das Modell behandelt Untergruppen systematisch schlechter, auch ohne das sensible Merkmal (Proxies).	Daten und Metriken je Untergruppe prüfen, ein Fairness-Kriterium wählen, Gegenmaßnahmen planen.	Kriterien widersprechen einander; es gibt kein allgemein gültiges „fair“; Gruppen-Features werden zur Prüfung gebraucht.

B Glossar

Die Begriffe sind alphabetisch nach der englischen Bezeichnung geordnet, weil man sie in der Literatur, in Bibliotheken und in Vorträgen so antrifft. Die Erklärung ist deutsch und bewusst kurz; wo ein Begriff ein eigenes Pattern ist, steht die Nummer dabei, und die Mechanik findet sich im jeweiligen Kapitel.

A/B Test. Zwei Modell- oder Programmversionen laufen gleichzeitig auf aufgeteiltem Verkehr; die Kennzahlen im Echtbetrieb entscheiden, welche bleibt (Pattern 27).

Accuracy. Anteil der richtig klassifizierten Beispiele. Bei seltenen Klassen irreführend: Wer immer „kein Betrug“ sagt, erreicht bei 1 % Betrug 99 %.

Activation Function. Nichtlineare Funktion, die ein Neuron auf die gewichtete Summe anwendet, zum Beispiel Sigmoid oder ReLU. Ohne sie wäre ein Netz aus vielen Schichten nur ein einziges lineares Modell.

Adapter. Eine Komponente je Datenquelle, die ein eigenes Format auf eine gemeinsame Darstellung abbildet; im Bridged Schema (Pattern 23) übersetzt sie alte und neue Quellen in denselben Feature-Vektor.

All-Reduce. Verfahren der verteilten Gradientenmittelung ohne zentralen Rechner: Alle Geräte tauschen ihre Gradienten direkt aus und erhalten dasselbe Ergebnis (Pattern 14).

Artifact. Ergebnis eines Pipeline-Schrittes, das gespeichert und weitergereicht wird, etwa transformierte Daten, ein Modell oder ein Bericht (Pattern 25).

As-of Join. Verknüpfung, die für jedes Ereignis nur Feature-Werte nimmt, die zu dessen Zeitpunkt schon galten; setzt Point-in-Time Correctness technisch um (Pattern 26).

AUC (ROC-AUC). Fläche unter der ROC-Kurve. Entspricht der Wahrscheinlichkeit, dass ein zufälliger positiver Fall höher bewertet wird als ein zufälliger negativer; unabhängig vom Schwellwert.

Backpropagation. Verfahren, mit dem ein neuronales Netz die Gradienten für alle Gewichte berechnet: Die Kettenregel wird Schicht für Schicht rückwärts angewendet.

Bagging. Viele Modelle auf zufälligen Stichproben (mit Zurücklegen) der Trainingsdaten trainieren und ihre Vorhersagen mitteln. Senkt die Varianz [3] (Pattern 7).

Baseline. Einfache Referenzlösung wie Mittelwert, häufigste Klasse oder eine Regel. Ein Modell muss sie schlagen, sonst lohnt es sich nicht (Pattern 28).

Batch. Zwei Bedeutungen. Im Training: die Teilmenge der Daten, aus der ein Gradientenschritt berechnet wird (Mini-Batch). Im Serving: die Verarbeitung vieler Instanzen in einem Lauf (Pattern 17).

Bias. Drei Bedeutungen. Im Lernen: systematischer Fehler eines zu einfachen Modells (Underfitting). Im Neuron: der additive Term neben den Gewichten (Achsenabschnitt). Gesellschaftlich: Verzerrung zulasten von Gruppen, siehe Fairness.

Bias-Variance-Tradeoff. Einfache Modelle haben hohen Bias und geringe Varianz, flexible Modelle umgekehrt. Ensembles und Regularisierung verschieben das Gleichgewicht.

Binning. Einen kontinuierlichen Wert in Intervalle einteilen. Grundlage von Reframing (Regression als Klassifikation über Bins, Pattern 5) und von Feature Crosses auf Zahlen.

Boosting. Modelle nacheinander trainieren, wobei jedes die Fehler der bisherigen korrigiert. Senkt vor allem den Bias; Gradient Boosting ist die verbreitete Form [5].

Bucket. Eines von n Fächern, auf das ein Hash oder eine Intervalleinteilung einen Wert abbildet (Pattern 1).

Calibration. Eine Vorhersage von „0,8“ stimmt kalibriert damit überein, dass etwa 80 % solcher Fälle tatsächlich positiv sind. Resampling und Klassengewichte verschieben die Kalibrierung.

Canary Release. Eine neue Modellversion bekommt zunächst nur einen kleinen Anteil der Anfragen; erst wenn die Kennzahlen stimmen, wird der Anteil erhöht (Pattern 27).

Cascade. Folge von Modellen, bei der die Ausgabe einer Stufe die nächste speist oder auswählt (Pattern 8). Nicht dasselbe wie ein Ensemble, in dem Modelle parallel dieselbe Frage beantworten.

Checkpoint. Gesicherter Zwischenstand des Trainings: Gewichte, Zustand des Optimierers und Epochenzähler (Pattern 12). Ein Checkpoint allein ist noch keine Modellversion.

Class Imbalance. Die Klassen kommen sehr ungleich häufig vor, etwa 1 % Betrug und 99 % Normalfälle. Mehr Daten ändern das Verhältnis nicht (Pattern 10).

Classification. Aufgabe, ein Beispiel einer von endlich vielen Klassen zuzuordnen. Gegenstück zur Regression.

Cold Start. Für eine Kategorie, die im Training nie vorkam, hat das Modell keine gelernte Repräsentation, etwa für einen neuen Nutzer oder ein neues Produkt (Pattern 1, 2).

Collision. Zwei verschiedene Eingaben ergeben denselben Hashwert und landen im selben Bucket (Pattern 1).

Concatenation. Zusammenfügen mehrerer Vektoren zu einem längeren; so werden bei Multimodal Input die Darstellungen der einzelnen Modalitäten vereint (Pattern 4).

Concept Drift. Der Zusammenhang zwischen Eingaben und Zielgröße ändert sich im Lauf der Zeit; ein altes Modell wird dadurch falsch, oft ohne dass die Eingaben auffallen [12] (Pattern 18).

Confusion Matrix. Tabelle, die je Klasse zählt, wie oft richtig oder falsch vorhergesagt wurde: True Positive, False Positive, False Negative, True Negative.

Container. Verpackung eines Programms samt Abhängigkeiten, die überall gleich läuft. Pipeline-Stufen laufen meist in Containern (Pattern 25).

Cosine Similarity. Ähnlichkeit zweier Vektoren über den Winkel zwischen ihnen: 1 bei gleicher Richtung, 0 bei rechtem Winkel. Übliches Maß für die Nähe von Embeddings.

Cross-Entropy. Verlustfunktion für Klassifikation (auch Log Loss). Bestraft vor allem Vorhersagen, die sich sicher sind und falsch liegen.

Cross-Validation. Die Daten in k Teile zerlegen, k-mal auf k-1 Teilen trainieren und auf dem übrigen Teil bewerten. Liefert eine stabilere Schätzung als ein einzelner Split.

DAG. Directed Acyclic Graph, gerichteter Graph ohne Zyklen. Eine Workflow Pipeline beschreibt ihre Schritte und deren Abhängigkeiten als DAG und führt sie in topologischer Reihenfolge aus (Pattern 25).

Data Drift. Die Verteilung der Eingaben ändert sich (neue Kundengruppe, anderer Sensor), auch wenn der Zusammenhang zur Zielgröße gleich bleibt. Auch Covariate Shift genannt.

Data Leakage. Information, die zum Vorhersagezeitpunkt nicht verfügbar wäre (Testdaten, Zukunft, Zielgröße), gelangt ins Training. Das Ergebnis wirkt besser, als es im Betrieb sein wird [8].

Datasheets for Datasets. Dokumentation eines Datensatzes (Herkunft, Zusammensetzung, Eignung), das Gegenstück zur Model Card [63].

Decision Tree. Folge von Ja-Nein-Fragen an die Features; jedes Blatt liefert eine Vorhersage. Gut erklärbar, neigt ohne Begrenzung der Tiefe zu Overfitting.

Deep Learning. Lernen mit neuronalen Netzen aus vielen Schichten, die Features selbst herausbilden.

Demographic Parity. Fairness-Kriterium: Der Anteil positiver Vorhersagen ist in allen Gruppen gleich (Pattern 30).

Determinism. Dieselbe Eingabe führt immer zum selben Ergebnis, unabhängig vom Zustand der Umgebung. Ein Hash-basierter Split ist deterministisch, ein Split mit Zufallszahlengenerator nur bei festem Seed und unveränderten Daten (Pattern 22).

Dimensionality. Anzahl der Komponenten eines Vektors, hier die Zahl der Features. One-Hot-Vektoren haben so viele Dimensionen wie Kategorien, Embeddings meist nur einige zehn bis hundert.

Distribution Strategy. Aufteilung des Trainings auf mehrere Geräte, meist datenparallel mit synchronisierten Gradienten (Pattern 14).

Downsampling. Die häufige Klasse im Trainingsdatensatz verkleinern, um das Verhältnis der Klassen auszugleichen; Gegenstück zum Oversampling (Pattern 10).

Early Stopping. Das Training beenden, sobald der Fehler auf den Validierungsdaten nicht mehr sinkt. Verhindert Overfitting, ist beim Useful Overfitting aber ausdrücklich unerwünscht (Pattern 11).

Embedding. Gelernte Abbildung von Kategorien, Wörtern oder anderen Objekten auf dichte Vektoren, in denen Ähnliches nah beieinander liegt (Pattern 2). Bekanntes Beispiel: word2vec [7].

Ensemble. Mehrere Modelle, deren Vorhersagen kombiniert werden: Bagging, Boosting oder Stacking (Pattern 7).

Entity Embeddings. Embeddings für kategoriale Spalten tabellarischer Daten, etwa Filiale oder Wochentag [26].

Epoch. Ein vollständiger Durchlauf durch alle Trainingsdaten. Eine Epoche besteht aus vielen Batches.

Equal Opportunity. Fairness-Kriterium: Unter den tatsächlich Positiven ist der Anteil richtig erkannter Fälle (Recall) in allen Gruppen gleich [74].

Equalized Odds. Fairness-Kriterium: Trefferquote und Quote falscher Alarme sind in allen Gruppen gleich. Strenger als Equal Opportunity, die nur den Recall verlangt.

Event Time. Der Zeitpunkt, zu dem etwas geschah, im Gegensatz zur Processing Time, zu der es im System ankommt. Im Training und im Stream muss dieselbe Zeit gelten, sonst stimmen die Fenster nicht überein (Pattern 24).

Example. Ein einzelnes Beispiel der Trainingsdaten, in der Tabelle eine Zeile aus Features und Label (auch Sample).

Explainability. Fähigkeit, die Entscheidung eines Modells nachvollziehbar zu machen, etwa durch Feature-Beiträge je Vorhersage (Pattern 29). Erklärt das Modell, nicht die Ursachen in der Wirklichkeit.

F1-Score. Harmonisches Mittel aus Precision und Recall. Fällt stark ab, sobald eine der beiden niedrig ist.

False Positive Rate. Anteil der tatsächlich negativen Fälle, die fälschlich als positiv gemeldet werden (Quote falscher Alarme).

Feature. Eingangsgröße eines Modells, also eine Spalte der Trainingstabelle, zum Beispiel Wohnfläche oder Wochentag. Die Liste aller Features eines Beispiels heißt Feature Vector.

Feature Attribution. Verteilung einer Vorhersage auf die Eingabe-Features, also die Antwort auf die Frage, welches Feature wie viel beigetragen hat. SHAP und Integrated Gradients tun das (Pattern 29).

Feature Cross. Neues Feature aus der Kombination zweier Features, damit auch ein lineares Modell Wechselwirkungen erfassen kann (Pattern 3).

Feature Engineering. Handarbeit, aus Rohdaten Features zu bauen, die dem Modell die Aufgabe erleichtern.

Feature Store. Zentraler Speicher für versionierte Feature-Definitionen und -werte, aus dem Training und Serving konsistent lesen (Pattern 26).

Fine-Tuning. Ein vortrainiertes Modell auf eigenen Daten weitertrainieren, meist mit kleiner Lernrate (Pattern 13).

Generalization. Fähigkeit eines Modells, auf Daten gut zu sein, die es im Training nicht gesehen hat. Das eigentliche Ziel; Overfitting ist ihr Gegenteil.

Graceful Degradation. Bei Ausfall oder Unsicherheit liefert das System ein schwächeres, aber brauchbares Ergebnis statt gar keines, etwa die Weiterleitung an einen Menschen.

Gradient. Vektor der partiellen Ableitungen der Verlustfunktion nach den Parametern; zeigt in die Richtung des steilsten Anstiegs.

Gradient Descent. Parameter schrittweise entgegen dem Gradienten verschieben, um den Verlust zu verringern. Die Schrittweite bestimmt die Learning Rate.

Grid Search. Alle Kombinationen eines vorgegebenen Rasters von Hyperparametern ausprobieren. Zufällige Suche ist bei gleichem Aufwand oft wirksamer [51].

Hamming Loss. Bei Multilabel der Anteil falscher Einzelentscheidungen über alle Labels hinweg; milder als die Forderung, dass alle Labels eines Beispiels stimmen (Pattern 6).

Hash Function. Deterministische Funktion, die beliebige Eingaben auf eine Zahl fester Größe abbildet. Gleiche Eingabe, gleiches Ergebnis; verschiedene Eingaben können kollidieren (Pattern 1).

Heuristic. Handgeschriebene Faustregel ohne Lernen, etwa „morgen so viel wie heute“. Dient als Benchmark (Pattern 28).

Hidden Layer. Schicht eines Netzes zwischen Eingabe und Ausgabe. Ihre Werte sind nicht direkt beobachtbar und bilden die gelernte Repräsentation der Eingabe.

Hyperparameter. Einstellung, die vor dem Training festgelegt und nicht durch dieses gelernt wird: Lernrate, Baumtiefe, Anzahl Schichten (Pattern 15).

Idempotence. Eine Operation mehrfach auszuführen ergibt dasselbe wie einmaliges Ausführen. Ein Batch-Job, der nach einem Abbruch dieselben Schlüssel überschreibt statt neue anzulegen, ist idempotent (Pattern 17). Nicht dasselbe wie Determinismus.

Inference. Anwendung eines trainierten Modells auf neue Daten. Im Alltag gleichbedeutend mit Prediction oder Serving.

Integrated Gradients. Erklärverfahren für neuronale Netze: Die Gradienten werden entlang eines Wegs von einer Referenzeingabe zur echten Eingabe aufsummiert [68].

Key. Eindeutiger Bezeichner einer Instanz, der unverändert durch das Modell bis in die Ausgabe gereicht wird (Pattern 20).

KS-Test. Kolmogorov-Smirnov-Test, ein statistischer Test, ob zwei Stichproben aus derselben Verteilung stammen. Kann Data Drift in einem Feature anzeigen (Pattern 18).

Label. Die Zielgröße zu einem Trainingsbeispiel, also der „richtige“ Wert, den das Modell vorhersagen soll.

Label Powerset. Multilabel-Variante, die jede vorkommende Kombination von Labels als eigene Klasse behandelt und mit einem gewöhnlichen Softmax-Modell trainiert (Pattern 6).

Latency. Zeit zwischen Anfrage und Antwort. Gegenstück zum Durchsatz.

Latent Semantic Analysis. Verfahren, das Wortvektoren per Singulärwertzerlegung aus einer Matrix des gemeinsamen Vorkommens (Wort-Dokument oder Wort-Wort) gewinnt. Ein einfaches, nicht neuronales Embedding.

Learning Rate. Schrittweite beim Gradientenabstieg. Zu groß, und das Training springt über das Minimum; zu klein, und es dauert unnötig lange.

LIME. Erklärverfahren, das das Modell in der Umgebung eines Falls durch ein einfaches lineares Modell annähert [65].

Lineage. Herkunftsnachweis: Welche Daten, welcher Code und welche Parameter haben ein Artefakt wie ein Modell erzeugt (Pattern 25, 27).

Linear Model. Modell, das die Features gewichtet aufsummiert. Kann Wechselwirkungen nur darstellen, wenn man sie als Feature vorgibt.

Linear Scaling Rule. Faustregel für datenparalleles Training: Wächst die Batchgröße um den Faktor k, wächst auch die Lernrate um k [31] (Pattern 14).

Logistic Regression. Lineares Modell mit Sigmoid am Ausgang, das eine Wahrscheinlichkeit liefert. Trotz des Namens ein Klassifikator.

Logit. Roher Ausgabewert vor dem Sigmoid oder Softmax. Erst die Umrechnung macht daraus eine Wahrscheinlichkeit.

Long Tail. Sehr ungleiche Verteilung: Wenige Werte machen den Großteil der Daten aus, viele seltene Werte haben zu wenige Beispiele, um gelernt zu werden.

Loss Function. Maß dafür, wie falsch eine Vorhersage ist. Das Training minimiert ihren Mittelwert über die Trainingsdaten.

MAE. Mean Absolute Error, der mittlere Betrag der Abweichung zwischen Vorhersage und Wert. Wird erst mit einer Heuristik als Vergleich aussagekräftig (Pattern 28).

Min-Max-Skalierung. Bildet den Wertebereich eines Features auf 0 bis 1 ab; empfindlich gegenüber Ausreißern. Alternative ist der z-Score (Mittelwert 0, Standardabweichung 1).

Model. Funktion mit gelernten Parametern, die Eingaben auf Vorhersagen abbildet.

Model Card. Kurzdokument zu einem Modell: Zweck, Daten, Leistung je Untergruppe, Grenzen [62].

Model Registry. Katalog der Modellartefakte mit Versionsnummer und Metadaten wie Trainingsdaten, Commit, Metriken und erwarteter Eingabestruktur (Pattern 27).

MSE. Mean Squared Error, der mittlere quadrierte Fehler. Häufige Verlustfunktion für Regression; die Wurzel daraus ist der RMSE, der wie die Zielgröße skaliert ist.

Multilabel. Ein Beispiel kann mehreren Klassen zugleich angehören; pro Label gibt es einen eigenen Sigmoid-Ausgang (Pattern 6).

Multimodal. Eingaben unterschiedlicher Art, zum Beispiel Text, Bild und Tabelle, fließen in ein Modell (Pattern 4).

Neural Network. Modell aus Schichten gewichteter Summen und nichtlinearer Funktionen, dessen Gewichte per Gradientenabstieg gelernt werden.

Neuron. Baustein eines neuronalen Netzes: bildet die gewichtete Summe seiner Eingaben plus Bias und wendet darauf eine Aktivierungsfunktion an.

Neutral Class. Zusätzliche Klasse für Fälle, in denen auch Fachleute uneins sind (Pattern 9).

Normalization. Features auf einen vergleichbaren Wertebereich bringen, etwa Mittelwert 0 und Streuung 1 (Standardisierung) oder 0 bis 1 (Min-Max). Die Kennwerte stammen aus den Trainingsdaten und müssen im Serving identisch angewendet werden (Pattern 21).

Observability. Fähigkeit, aus Logs, Metriken und Traces nachzuvollziehen, was ein System tut. Erklärbare Vorhersagen sind das Gegenstück auf Modellebene (Pattern 29).

Offline Store / Online Store. Die zwei Seiten eines Feature Stores: Der Offline Store hält die gesamte Historie für das Training, der Online Store jeweils den letzten Wert für schnelle Abfragen im Serving (Pattern 26).

One-Hot Encoding. Eine Kategorie als Vektor mit einer 1 an ihrer Position und sonst 0. Wird bei vielen Kategorien riesig und kennt keine Ähnlichkeit.

Orchestrator. Werkzeug, das die Schritte einer Pipeline als Graph ausführt, fehlgeschlagene Schritte wiederholt und sich merkt, was schon gelaufen ist (Pattern 25).

Out-of-Fold. Vorhersagen für Daten, die das Modell beim Training nicht gesehen hat, weil sie in einer anderen Falte der Kreuzvalidierung lagen. Unverzichtbar beim Stacking, damit das Meta-Modell nicht Auswendiglernen belohnt (Pattern 7).

Overfitting. Das Modell passt sich Besonderheiten und Rauschen der Trainingsdaten an: Training gut, neue Daten schlecht.

Parameter. Vom Training gelernter Wert eines Modells, etwa ein Gewicht. Gegenstück zum Hyperparameter.

Parameter Server. Dedizierte Rechner, die in verteiltem Training die Parameter halten und Gradienten einsammeln; Alternative zu All-Reduce (Pattern 14).

Permutation Importance. Wichtigkeit eines Features, gemessen daran, wie stark die Qualität sinkt, wenn man seine Werte zufällig durchmischt (Pattern 29).

Pipeline. Verkettung von Verarbeitungsschritten von den Rohdaten bis zum Modell, im Pattern 25 als orchestrierte, wiederholbare Stufen.

Point-in-Time Correctness. Beim Zusammenstellen von Trainingsdaten nur Werte verwenden, die zum Zeitpunkt des jeweiligen Ereignisses schon bekannt waren (Pattern 26).

PR-AUC. Fläche unter der Precision-Recall-Kurve (oft als Average Precision berechnet). Bei seltener positiver Klasse aussagekräftiger als ROC-AUC [10].

Pre-/In-/Post-Processing. Drei Zeitpunkte für Fairness-Maßnahmen: Daten vor dem Training anpassen, die Bedingung im Training optimieren oder Entscheidungsschwellen nach dem Training je Gruppe verschieben (Pattern 30).

Precision. Anteil der echten Treffer unter allen als positiv vorhergesagten Fällen.

Predictive Parity. Fairness-Kriterium: Eine positive Vorhersage ist in allen Gruppen mit gleicher Wahrscheinlichkeit zutreffend (verwandt mit, aber nicht dasselbe wie Kalibrierung nach Gruppen).

Pretrained Model. Auf einer großen, verwandten Aufgabe vorab trainiertes Modell, das man wiederverwendet (Pattern 13).

Proxy Variable. Feature, das ein sensibles Merkmal indirekt mitträgt, zum Beispiel die Postleitzahl für die Herkunft. Der Grund, warum Weglassen allein nicht fair macht (Pattern 30).

PSI. Population Stability Index, eine Kennzahl, die zwei Verteilungen eines Features vergleicht: Je größer der Wert, desto stärker hat sich die Verteilung gegenüber dem Training verschoben (Pattern 18).

Random Forest. Ensemble aus vielen Entscheidungsbäumen, die auf Stichproben und zufälligen Feature-Ausschnitten trainiert werden [4].

Random Search. Hyperparameter-Suche, bei der jede Konfiguration zufällig aus vorgegebenen Bereichen gezogen wird; bei gleichem Aufwand oft besser als Grid Search [51] (Pattern 15).

Recall. Anteil der tatsächlich positiven Fälle, die das Modell findet (auch Sensitivität, True Positive Rate).

Reframing. Eine Aufgabe in einen anderen Aufgabentyp umformulieren, zum Beispiel Regression als Klassifikation (Pattern 5).

Regression. Aufgabe, eine Zahl vorherzusagen, etwa eine Miete oder einen Stromverbrauch. Gegenstück zur Klassifikation.

Regularization. Maßnahmen, die die Komplexität eines Modells begrenzen und Overfitting dämpfen, etwa Strafterme auf große Gewichte.

ReLU. Aktivierungsfunktion, die negative Werte auf 0 setzt und positive unverändert lässt.

Repeatable Splitting. Aufteilung in Training, Validierung und Test so, dass dieselbe Zeile immer in derselben Menge landet, auch wenn Daten dazukommen (Pattern 22).

Representation. Darstellung der Eingabe als Zahlen, mit der das Modell arbeitet, etwa Embeddings oder die Werte einer versteckten Schicht. Gute Repräsentationen machen die Aufgabe leichter.

ROC-Kurve. Kurve der Trefferquote gegen die Quote falscher Alarme bei variierendem Schwellwert. Bei sehr seltener positiver Klasse kann die Precision-Recall-Kurve aussagekräftiger sein [10].

Rollback. Rückkehr zu einer früheren Version, etwa eines Modells, wenn die neue sich schlechter verhält (Pattern 27).

Schema Evolution. Ein Datenschema ändert sich über die Zeit. Anders als bei Datenbanken kann man fehlende Features nicht migrieren, wenn die Information nie erhoben wurde (Pattern 23).

Seed. Startwert des Zufallsgenerators; in scikit-learn `random_state`. Macht Läufe wiederholbar, aber nicht automatisch Splits bei veränderten Daten (Pattern 22).

Selection Bias (Labels). Labels existieren nur für Fälle, die eine frühere Entscheidung durchgelassen hat, etwa nur für bewilligte Kredite. Verzerrt Bewertung und Training (Pattern 30).

Serving. Betrieb eines Modells, damit andere Systeme Vorhersagen abrufen können, online oder im Batch (Kapitel 8).

Shadow Mode. Die neue Modellversion erhält die Anfragen mit, ihre Antworten werden aber nur protokolliert und verglichen, nicht ausgeliefert (Pattern 27).

SHAP. Erklärverfahren auf Basis der Shapley-Werte aus der Spieltheorie: Jedes Feature erhält einen Beitrag zur Abweichung der Vorhersage vom Durchschnitt [66].

Sharding. Aufteilung von Daten auf Partitionen, meist per `hash(key) mod N`; dasselbe Prinzip steckt hinter Hashed Feature und Repeatable Splitting.

Sigmoid. Funktion, die jede Zahl auf das Intervall zwischen 0 und 1 drückt und deshalb als Wahrscheinlichkeit gelesen werden kann.

Single Source of Truth. Eine Information, hier eine Transformation oder Feature-Definition, existiert genau einmal und wird von allen Verbrauchern genutzt (Pattern 21, 26).

Slicing. Kennzahlen nicht nur insgesamt, sondern für Teilmengen der Daten berechnen (Region, Altersgruppe). Grundhandgriff der Fairness Lens (Pattern 30).

SMOTE. Oversampling-Verfahren, das neue Fälle der Minderheitsklasse durch Interpolation zwischen Nachbarn erzeugt statt sie zu kopieren [44].

Softmax. Funktion, die Rohwerte in Wahrscheinlichkeiten umrechnet, die sich zu 1 summieren. Passt, wenn genau eine Klasse zutrifft; für Multilabel nicht.

Stacking. Ensemble-Form, bei der ein weiteres Modell lernt, die Vorhersagen der Basismodelle zu kombinieren [41].

Stateless. Eine Funktion oder ein Dienst behält zwischen Aufrufen nichts im Speicher; jede Anfrage trägt alles mit, was sie braucht (Pattern 16).

Stream Processing. Verarbeitung von Daten laufend bei ihrem Eintreffen statt in großen Stapeln; Voraussetzung für Windowed Inference (Pattern 24).

Supervised Learning. Lernen aus Beispielen mit bekanntem Label. Fast alle Patterns dieses Handbuchs setzen es voraus.

Surrogate Model. Modell, das eine teure Berechnung oder Simulation nachahmt und sie im Betrieb ersetzt; typischer Fall des Useful Overfitting (Pattern 11).

Test Set. Daten, die erst für die abschließende Bewertung benutzt werden. Wer darauf Entscheidungen trifft, verbraucht ihre Aussagekraft.

Threshold. Schwellwert, ab dem eine Wahrscheinlichkeit als positive Vorhersage zählt. Oft nicht 0,5, sondern eine Abwägung zwischen verpassten und falschen Alarmen.

Throughput. Anzahl der Vorhersagen pro Zeiteinheit. Im Batch Serving wichtiger als die Latency (Pattern 17).

Training Set. Daten, aus denen das Modell seine Parameter lernt.

Training-Serving Skew. Die Features, die ein Modell im Betrieb sieht, weichen von denen im Training ab, meist durch zwei getrennte Implementierungen der Vorverarbeitung (Pattern 21).

Transfer Learning. Wissen eines Modells aus einer Aufgabe für eine andere nutzen; die Übertragbarkeit nimmt mit dem Abstand der Aufgaben ab [28] (Pattern 13).

Underfitting. Das Modell ist zu einfach und verfehlt schon die Trainingsdaten. Entspricht hohem Bias.

Union Schema. Ein Schema, das alle bisherigen und neuen Kategorien zulässt; kann dem Modell Werte beibringen, die im Betrieb nie vorkommen (Pattern 23).

Unsupervised Learning. Lernen ohne Labels, etwa Gruppen oder Strukturen in Daten finden. Die Patterns dieses Handbuchs setzen überwiegend Supervised Learning voraus.

Validation Set. Daten, an denen während der Entwicklung Entscheidungen fallen: Hyperparameter, Abbruchzeitpunkt, Modellauswahl.

Variance. Empfindlichkeit eines Modells gegenüber der konkreten Stichprobe der Trainingsdaten. Hohe Varianz zeigt sich als Overfitting.

Weight. Gelernter Faktor, mit dem ein Neuron eine Eingabe gewichtet; die Gewichte bilden zusammen mit den Bias-Werten die Parameter eines Netzes.

Windowing. Daten über ein Zeitfenster aggregieren, etwa den Mittelwert der letzten drei Stunden (Pattern 24).

Windows (Tumbling, Sliding, Session). Fensterarten bei Datenströmen: Tumbling sind feste, überlappungsfreie Abschnitte, Sliding verschieben sich in kleinen Schritten, Session sind durch Pausen begrenzt (Pattern 24).

Literaturverzeichnis

Die Nummern verweisen auf die Quellenangaben im Text. Das Verzeichnis enthält nur Quellen, die im Text zitiert werden.

[1] V. Lakshmanan, S. Robinson, und M. Munn, *Machine Learning Design Patterns: Solutions to Common Challenges in Data Preparation, Model Building, and MLOps*. Sebastopol, CA: O'Reilly Media, 2020.

[2] E. Gamma, R. Helm, R. Johnson, und J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[3] L. Breiman, „Bagging Predictors“, *Machine Learning*, Bd. 24, Nr. 2, S. 123–140, 1996, doi: 10.1007/BF00058655.

[4] L. Breiman, „Random Forests“, *Machine Learning*, Bd. 45, Nr. 1, S. 5–32, 2001, doi: 10.1023/A:1010933404324.

[5] J. H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine“, *The Annals of Statistics*, Bd. 29, Nr. 5, S. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[6] L. Grinsztajn, E. Oyallon, und G. Varoquaux, „Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?“, in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022.

[7] T. Mikolov, K. Chen, G. Corrado, und J. Dean, „Efficient Estimation of Word Representations in Vector Space“. 2013.

[8] S. Kaufman, S. Rosset, C. Perlich, und O. Stitelman, „Leakage in Data Mining: Formulation, Detection, and Avoidance“, *ACM Transactions on Knowledge Discovery from Data*, Bd. 6, Nr. 4, S. 15:1–15:21, 2012, doi: 10.1145/2382577.2382579.

[9] T. Hastie, R. Tibshirani, und J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2. Aufl. New York, NY: Springer, 2009.

[10] T. Saito und M. Rehmsmeier, „The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets“, *PLOS ONE*, Bd. 10, Nr. 3, S. e118432, 2015, doi: 10.1371/journal.pone.0118432.

[11] D. Sculley *u. a.*, „Hidden Technical Debt in Machine Learning Systems“, in *Advances in Neural Information Processing Systems 28*, 2015, S. 2503–2511.

[12] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, und A. Bouchachia, „A Survey on Concept Drift Adaptation“, *ACM Computing Surveys*, Bd. 46, Nr. 4, S. 44:1–44:37, 2014, doi: 10.1145/2523813.

[13] F. Rosenblatt, „The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain“, *Psychological Review*, Bd. 65, Nr. 6, S. 386–408, 1958.

[14] I. Goodfellow, Y. Bengio, und A. Courville, *Deep Learning*. MIT Press, 2016.

[15] M. Minsky und S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.

[16] K. Hornik, M. Stinchcombe, und H. White, „Multilayer Feedforward Networks Are Universal Approximators“, *Neural Networks*, Bd. 2, Nr. 5, S. 359–366, 1989.

[17] M. Leshno, V. Y. Lin, A. Pinkus, und S. Schocken, „Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function“, *Neural Networks*, Bd. 6, Nr. 6, S. 861–867, 1993.

[18] D. E. Rumelhart, G. E. Hinton, und R. J. Williams, „Learning Representations by Back-Propagating Errors“, *Nature*, Bd. 323, Nr. 6088, S. 533–536, 1986, doi: 10.1038/323533a0.

[19] D. P. Kingma und J. Ba, „Adam: A Method for Stochastic Optimization“, in *3rd International Conference on Learning Representations (ICLR)*, 2015.

[20] L. Prechelt, „Early Stopping – But When?“, *Neural Networks: Tricks of the Trade*, Bd. 1524. in Lecture Notes in Computer Science, vol. 1524. Springer, S. 55–69, 1998.

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, und R. Salakhutdinov, „Dropout: A Simple Way to Prevent Neural Networks from Overfitting“, *Journal of Machine Learning Research*, Bd. 15, S. 1929–1958, 2014.

[22] C. Guo, G. Pleiss, Y. Sun, und K. Q. Weinberger, „On Calibration of Modern Neural Networks“, in *Proceedings of the 34th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 70. 2017, S. 1321–1330.

[23] C. Zhang, S. Bengio, M. Hardt, B. Recht, und O. Vinyals, „Understanding Deep Learning Requires Rethinking Generalization“, in *International Conference on Learning Representations (ICLR)*, 2017.

[24] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, und J. Dean, „Distributed Representations of Words and Phrases and their Compositionality“, in *Advances in Neural Information Processing Systems 26*, 2013, S. 3111–3119.

[25] T. Bolukbasi, K.-W. Chang, J. Zou, V. Saligrama, und A. Kalai, „Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings“, in *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016, S. 4349–4357.

[26] C. Guo und F. Berkhahn, „Entity Embeddings of Categorical Variables“. 2016.

[27] M. D. Zeiler und R. Fergus, „Visualizing and Understanding Convolutional Networks“, in *Computer Vision – ECCV 2014*, in Lecture Notes in Computer Science, vol. 8689. Springer, 2014, S. 818–833.

[28] J. Yosinski, J. Clune, Y. Bengio, und H. Lipson, „How Transferable Are Features in Deep Neural Networks?“, in *Advances in Neural Information Processing Systems 27*, 2014, S. 3320–3328.

[29] J. Devlin, M.-W. Chang, K. Lee, und K. Toutanova, „BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding“, in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, S. 4171–4186.

[30] S. Williams, A. Waterman, und D. Patterson, „Roofline: An Insightful Visual Performance Model for Multicore Architectures“, *Communications of the ACM*, Bd. 52, Nr. 4, S. 65–76, 2009.

[31] P. Goyal *u. a.*, „Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour“. 2017.

[32] A. Vaswani *u. a.*, „Attention Is All You Need“, in *Advances in Neural Information Processing Systems 30*, 2017, S. 5998–6008.

[33] K. He, X. Zhang, S. Ren, und J. Sun, „Deep Residual Learning for Image Recognition“, in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, S. 770–778.

[34] J. L. Ba, J. R. Kiros, und G. E. Hinton, „Layer Normalization“. 2016.

[35] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, und J. Attenberg, „Feature Hashing for Large Scale Multitask Learning“, in *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009, S. 1113–1120.

[36] TensorFlow Team, „Introducing TensorFlow Feature Columns“. [Online]. Verfügbar unter: https://developers.googleblog.com/2017/11/introducing-tensorflow-feature-columns.html

[37] H.-T. Cheng *u. a.*, „Wide & Deep Learning for Recommender Systems“, in *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS)*, 2016, S. 7–10.

[38] R. Koenker und G. Bassett, „Regression Quantiles“, *Econometrica*, Bd. 46, Nr. 1, S. 33–50, 1978, doi: 10.2307/1913643.

[39] G. Tsoumakas und I. Katakis, „Multi-Label Classification: An Overview“, *International Journal of Data Warehousing and Mining*, Bd. 3, Nr. 3, S. 1–13, 2007.

[40] Y. Freund und R. E. Schapire, „A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting“, *Journal of Computer and System Sciences*, Bd. 55, Nr. 1, S. 119–139, 1997, doi: 10.1006/jcss.1997.1504.

[41] D. H. Wolpert, „Stacked Generalization“, *Neural Networks*, Bd. 5, Nr. 2, S. 241–259, 1992, doi: 10.1016/S0893-6080(05)80023-1.

[42] F. Pedregosa *u. a.*, „Scikit-learn: Machine Learning in Python“, *Journal of Machine Learning Research*, Bd. 12, S. 2825–2830, 2011.

[43] P. Viola und M. Jones, „Rapid Object Detection using a Boosted Cascade of Simple Features“, in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001, S. 511–518.

[44] N. V. Chawla, K. W. Bowyer, L. O. Hall, und W. P. Kegelmeyer, „SMOTE: Synthetic Minority Over-sampling Technique“, *Journal of Artificial Intelligence Research*, Bd. 16, S. 321–357, 2002, doi: 10.1613/jair.953.

[45] G. Lemaître, F. Nogueira, und C. K. Aridas, „Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning“, *Journal of Machine Learning Research*, Bd. 18, Nr. 17, S. 1–5, 2017.

[46] H. He und E. A. Garcia, „Learning from Imbalanced Data“, *IEEE Transactions on Knowledge and Data Engineering*, Bd. 21, Nr. 9, S. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.

[47] M. Belkin, D. Hsu, S. Ma, und S. Mandal, „Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off“, *Proceedings of the National Academy of Sciences*, Bd. 116, Nr. 32, S. 15849–15854, 2019.

[48] J. Dean *u. a.*, „Large Scale Distributed Deep Networks“, in *Advances in Neural Information Processing Systems 25*, 2012.

[49] P. Patarasuk und X. Yuan, „Bandwidth Optimal All-Reduce Algorithms for Clusters of Workstations“, *Journal of Parallel and Distributed Computing*, Bd. 69, Nr. 2, S. 117–124, 2009.

[50] J. Snoek, H. Larochelle, und R. P. Adams, „Practical Bayesian Optimization of Machine Learning Algorithms“, in *Advances in Neural Information Processing Systems 25*, 2012.

[51] J. Bergstra und Y. Bengio, „Random Search for Hyper-Parameter Optimization“, *Journal of Machine Learning Research*, Bd. 13, S. 281–305, 2012.

[52] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures“, Doctoral dissertation, 2000.

[53] A. Wiggins, „The Twelve-Factor App“. 2011.

[54] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[55] E. Breck, S. Cai, E. Nielsen, M. Salib, und D. Sculley, „The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction“, in *Proceedings of the 2017 IEEE International Conference on Big Data*, 2017.

[56] G. Hohpe und B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003.

[57] D. R. Roberts *u. a.*, „Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure“, *Ecography*, Bd. 40, Nr. 8, S. 913–929, 2017, doi: 10.1111/ecog.02881.

[58] T. Akidau *u. a.*, „The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing“, *Proceedings of the VLDB Endowment*, Bd. 8, Nr. 12, S. 1792–1803, 2015, doi: 10.14778/2824032.2824076.

[59] D. Baylor *u. a.*, „TFX: A TensorFlow-Based Production-Scale Machine Learning Platform“, in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, S. 1387–1395. doi: 10.1145/3097983.3098021.

[60] M. Zaharia *u. a.*, „Accelerating the Machine Learning Lifecycle with MLflow“, *IEEE Data Engineering Bulletin*, Bd. 41, Nr. 4, S. 39–45, 2018.

[61] Feast Contributors, „Feast: The Open Source Feature Store“. 2026.

[62] M. Mitchell *u. a.*, „Model Cards for Model Reporting“, in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, ACM, 2019, S. 220–229. doi: 10.1145/3287560.3287596.

[63] T. Gebru *u. a.*, „Datasheets for Datasets“, *Communications of the ACM*, Bd. 64, Nr. 12, S. 86–92, 2021, doi: 10.1145/3458723.

[64] A. Fisher, C. Rudin, und F. Dominici, „All Models Are Wrong, but Many Are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously“, *Journal of Machine Learning Research*, Bd. 20, Nr. 177, S. 1–81, 2019.

[65] M. T. Ribeiro, S. Singh, und C. Guestrin, „``Why Should I Trust You?'': Explaining the Predictions of Any Classifier“, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, S. 1135–1144. doi: 10.1145/2939672.2939778.

[66] S. M. Lundberg und S.-I. Lee, „A Unified Approach to Interpreting Model Predictions“, in *Advances in Neural Information Processing Systems 30*, 2017, S. 4765–4774.

[67] S. M. Lundberg *u. a.*, „From Local Explanations to Global Understanding with Explainable AI for Trees“, *Nature Machine Intelligence*, Bd. 2, Nr. 1, S. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[68] M. Sundararajan, A. Taly, und Q. Yan, „Axiomatic Attribution for Deep Networks“, in *Proceedings of the 34th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 70. 2017, S. 3319–3328.

[69] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, und B. Kim, „Sanity Checks for Saliency Maps“, in *Advances in Neural Information Processing Systems 31*, 2018, S. 9505–9515.

[70] C. Rudin, „Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead“, *Nature Machine Intelligence*, Bd. 1, Nr. 5, S. 206–215, 2019, doi: 10.1038/s42256-019-0048-x.

[71] L. S. Shapley, „A Value for n-Person Games“, *Contributions to the Theory of Games, Volume II*, Nr. 28. in Annals of Mathematics Studies. Princeton University Press, S. 307–317, 1953.

[72] J. Buolamwini und T. Gebru, „Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification“, in *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, in Proceedings of Machine Learning Research, vol. 81. 2018, S. 77–91.

[73] Z. Obermeyer, B. Powers, C. Vogeli, und S. Mullainathan, „Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations“, *Science*, Bd. 366, Nr. 6464, S. 447–453, 2019, doi: 10.1126/science.aax2342.

[74] M. Hardt, E. Price, und N. Srebro, „Equality of Opportunity in Supervised Learning“, in *Advances in Neural Information Processing Systems 29*, 2016, S. 3315–3323.

[75] J. Kleinberg, S. Mullainathan, und M. Raghavan, „Inherent Trade-Offs in the Fair Determination of Risk Scores“, in *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, in LIPIcs, vol. 67. 2017, S. 43:1–43:23. doi: 10.4230/LIPIcs.ITCS.2017.43.

[76] A. Chouldechova, „Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments“, *Big Data*, Bd. 5, Nr. 2, S. 153–163, 2017.

[77] S. Bird *u. a.*, „Fairlearn: A Toolkit for Assessing and Improving Fairness in AI“, technical report MSR-TR-2020-32, 2020.

[78] J. Wexler, M. Pushkarna, T. Bolukbasi, M. Wattenberg, F. Viégas, und J. Wilson, „The What-If Tool: Interactive Probing of Machine Learning Models“, *IEEE Transactions on Visualization and Computer Graphics*, Bd. 26, Nr. 1, S. 56–65, 2020, doi: 10.1109/TVCG.2019.2934619.

[79] S. Barocas, M. Hardt, und A. Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press, 2023.

[80] W. Cunningham, „The WyCash Portfolio Management System“, in *OOPSLA '92 Addendum to the Proceedings (Experience Report)*, 1992, S. 29–30.

[81] Google Cloud, „MLOps: Continuous delivery and automation pipelines in machine learning“. 2020.

[82] J. Humble und D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley, 2010.

[83] MLflow Contributors, „MLflow Documentation: Tracking and Model Registry“. 2026.

[84] N. Bantilan, „pandera: Statistical Data Validation of pandas Dataframes“, in *Proceedings of the 19th Python in Science Conference (SciPy 2020)*, 2020, S. 116–124.

[85] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, und M. Zinkevich, „Data Validation for Machine Learning“, in *Proceedings of Machine Learning and Systems 1 (MLSys 2019)*, 2019.

[86] Great Expectations Contributors, „Great Expectations: Data Validation and Documentation“. 2026.

[87] M. T. Ribeiro, T. Wu, C. Guestrin, und S. Singh, „Beyond Accuracy: Behavioral Testing of NLP Models with CheckList“, in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

[88] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodr\íguez, N. V. Chawla, und F. Herrera, „A unifying view on dataset shift in classification“, *Pattern Recognition*, Bd. 45, Nr. 1, S. 521–530, 2012.

[89] S. Rabanser, S. Günnemann, und Z. C. Lipton, „Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift“, in *Advances in Neural Information Processing Systems 32*, 2019.

[90] R. Kohavi, D. Tang, und Y. Xu, *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge: Cambridge University Press, 2020.

[91] D. Kreuzberger, N. Kühl, und S. Hirschl, „Machine Learning Operations (MLOps): Overview, Definition, and Architecture“, *IEEE Access*, Bd. 11, S. 31866–31879, 2023.

[92] S. Amershi *u. a.*, „Software Engineering for Machine Learning: A Case Study“, in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, S. 291–300.

[93] C. Huyen, *Designing Machine Learning Systems*. Sebastopol, CA: O'Reilly Media, 2022.

[94] A. Paleyes, R.-G. Urma, und N. D. Lawrence, „Challenges in Deploying Machine Learning: A Survey of Case Studies“, *ACM Computing Surveys*, Bd. 55, Nr. 6, 2022.

[95] D. Sato, A. Wider, und C. Windheuser, „Continuous Delivery for Machine Learning“. 2019.

[96] E. Štrumbelj und I. Kononenko, „Explaining Prediction Models and Individual Predictions with Feature Contributions“, *Knowledge and Information Systems*, Bd. 41, Nr. 3, S. 647–665, 2014.

[97] C. Elkan, „The Foundations of Cost-Sensitive Learning“, in *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI)*, 2001, S. 973–978.

[98] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, und S. G. S. Alcalá, „A systematic literature review of machine learning methods applied to predictive maintenance“, *Computers & Industrial Engineering*, Bd. 137, S. 106024, 2019.

[99] A. Saxena, K. Goebel, D. Simon, und N. Eklund, „Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation“, in *International Conference on Prognostics and Health Management (PHM)*, IEEE, 2008.

[100] C. Bergmeir und J. M. Benítez, „On the use of cross-validation for time series predictor evaluation“, *Information Sciences*, Bd. 191, S. 192–213, 2012.

[101] G. Ke *u. a.*, „LightGBM: A Highly Efficient Gradient Boosting Decision Tree“, in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[102] Y. Koren, R. Bell, und C. Volinsky, „Matrix Factorization Techniques for Recommender Systems“, *Computer*, Bd. 42, Nr. 8, S. 30–37, 2009, doi: 10.1109/MC.2009.263.

[103] O. Levy und Y. Goldberg, „Neural Word Embedding as Implicit Matrix Factorization“, in *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 2014.

[104] O. Levy, Y. Goldberg, und I. Dagan, „Improving Distributional Similarity with Lessons Learned from Word Embeddings“, *Transactions of the Association for Computational Linguistics*, Bd. 3, S. 211–225, 2015.

[105] N. Halko, P.-G. Martinsson, und J. A. Tropp, „Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions“, *SIAM Review*, Bd. 53, Nr. 2, S. 217–288, 2011, doi: 10.1137/090771806.

[106] K. Spärck Jones, „A Statistical Interpretation of Term Specificity and Its Application in Retrieval“, *Journal of Documentation*, Bd. 28, Nr. 1, S. 11–21, 1972.

[107] G. Salton und C. Buckley, „Term-Weighting Approaches in Automatic Text Retrieval“, *Information Processing & Management*, Bd. 24, Nr. 5, S. 513–523, 1988.

[108] D. Micci-Barreca, „A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems“, *ACM SIGKDD Explorations Newsletter*, Bd. 3, Nr. 1, S. 27–32, 2001, doi: 10.1145/507533.507538.

[109] S. Ramírez, „FastAPI: Dokumentation“. 2026.

[110] T. G. Dietterich, „Ensemble Methods in Machine Learning“, in *Multiple Classifier Systems (MCS 2000)*, in Lecture Notes in Computer Science, vol. 1857. Springer, 2000, S. 1–15.

[111] A. Dal Pozzolo, O. Caelen, R. A. Johnson, und G. Bontempi, „Calibrating Probability with Undersampling for Unbalanced Classification“, in *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, IEEE, 2015.

[112] J. W. Young, „A First Order Approximation to the Optimum Checkpoint Interval“, *Communications of the ACM*, Bd. 17, Nr. 9, S. 530–531, 1974.

[113] J. T. Daly, „A Higher Order Estimate of the Optimum Checkpoint Interval for Restart Dumps“, *Future Generation Computer Systems*, Bd. 22, Nr. 3, S. 303–312, 2006.

[114] R. Geirhos *u. a.*, „Shortcut Learning in Deep Neural Networks“, *Nature Machine Intelligence*, Bd. 2, Nr. 11, S. 665–673, 2020.

KI-Offenlegung — *Dieses Dokument profitierte von der Nutzung folgender generativer KI-Werkzeuge.*

Text
- Claude Opus 5.5
  – Ideenfindung, drafting, Umformulierung, Formatierung, fact-checking

Illustrationen sofern nicht anders angegeben
- GPT Image 2

Alle Inhalte wurden vom Autor überprüft, der die endgültige Verantwortung für Korrektheit und Interpretation trägt.